LLD Handbook Study Edition — Vol 2: Interview Approach and Case Studies

OOP · UML · SOLID · Clean Code · Refactoring · Patterns · Concurrency · 45 case studies in Java — every topic, readable length

11 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — The LLD Interview & How to Approach It (Study Edition)**
   - 1.1 What this round is, and what this handbook covers
   - 1.2 How the interview runs
   - 1.3 A complete answer in five minutes: "Design a music playlist"
   - 1.4 The step-by-step answer framework
   - 1.5 From one line to a list of requirements
   - 1.6 Finding the classes
   - 1.7 Connecting the classes (preview)
   - 1.8 Keeping the design honest with SOLID
   - 1.9 The design patterns that come up in this round
   - 1.10 Code, edge cases and concurrency: how to write interview code
   - 1.11 Worked mini-example, end to end: a small parking lot
   - 1.12 Communication tips
   - 1.13 How to use this handbook, and a study plan
   - 1.14 Common mistakes
   - In the interview
   - Self-check
   - TL;DR
2. **Chapter 14 — Case Studies: Everyday Objects (Study Edition)**
   - 14.1 Design a Parking Lot
   - 14.2 Design a Vending Machine
   - 14.3 Design a Coffee Vending Machine
   - 14.4 Design an ATM
   - 14.5 Design an Elevator System
   - 14.6 Design a Traffic Signal Control System
   - 14.7 Design a Shipping Locker System
   - Chapter summary
3. **Chapter 15 — Case Studies: Booking & Inventory (Part 1) (Study Edition)**
   - Why booking problems are interview favourites
   - 15.1 Design a Library Management System
   - 15.2 Design a Movie Ticket Booking System (BookMyShow)
   - 15.3 Design a Concert Ticket Booking System
   - 15.4 Design a Hotel Management System
   - Chapter summary
4. **Chapter 16 — Case Studies: Booking & Inventory (Part 2) — Study Edition**
   - Design a Car Rental System
   - Design a Restaurant Management System
   - Design an Airline Management System
   - Design a Course Registration System
   - Chapter summary
5. **Chapter 17 — Case Studies: Games & Rules — Study Edition**
   - Design Blackjack and a Deck of Cards
   - Design Chess
   - Design Tic-Tac-Toe
   - Design Snake and Ladder
   - Design 2048
   - Chapter summary
6. **Chapter 18 — Case Studies: Platforms & Feeds (Part 1) — Study Edition**
   - 18.0 What platform problems test
   - 18.1 Design Stack Overflow
   - 18.2 Design Cricinfo
   - 18.3 Design LinkedIn
   - 18.4 Design Facebook
   - 18.5 Chapter summary
7. **Chapter 19 — Case Studies: Commerce & Marketplaces (Part 2 of Platforms) — Study Edition**
   - Design Amazon — Online Shopping System
   - Design an Online Stock Brokerage System
   - Design an Online Auction System
   - Design a Music Streaming Service (Spotify)
   - Design an Online Food Delivery Service (Swiggy) + FoodKart
   - Design a Grocery Store (POS & Checkout)
   - Chapter summary
8. **Chapter 20 — Case Studies: Pattern-Driven Components (Study Edition)**
   - 20.1 Design Splitwise
   - 20.2 Design a Notification Service
   - 20.3 Design an In-Memory Cache (and an LRU Cache from scratch)
   - 20.4 Design a Ride Sharing Service (Uber)
   - 20.5 Design a Logging Framework
   - 20.6 Design a Pub-Sub System
   - 20.7 Design a Rate Limiter
   - 20.8 Chapter summary
   - 20.9 Chapter self-check
   - 20.10 Chapter TL;DR
9. **Chapter 21 — Case Studies: Machine-Coding Round Problems (Study Edition)**
   - 21.0 How the machine-coding round works
   - 21.1 Design a Task Management System
   - 21.2 Design a Digital Wallet Service
   - 21.3 Design an Event Calendar (Flipkart)
   - 21.4 Design a LeetCode-like Coding Platform — "Coding Blox" (Flipkart)
   - 21.5 Design a Property Listing Site — "PropertyHunt" (ClearTrip)
   - 21.6 Design a Loan Ledger — "Ledger Co" (Navi)
   - 21.7 Design an Inventory Management System
   - 21.8 Design a Unix File Search (`find`)
   - Chapter summary
10. **Chapter 22 — Revision: Cheat Sheets & Glossary**
    - 22.1 How to revise
    - 22.2 The LLD interview answer framework (one page)

- 22.3 OOP & relationships cheat sheet
- 22.4 SOLID one-pager
- 22.5 Other design principles and Clean Code rules
- 22.6 Code smells → refactorings (all 24)
- 22.7 Design patterns: all 23 GoF + extras
- 22.8 Concurrency cheat sheet
- 22.9 Case-study map
- 22.10 Machine-coding and edge-case checklists
- 22.11 Glossary
- Self-check
- TL;DR

11. **Chapter 23 — Interview Question Bank**
    - How to use this bank
    - Part A — "Design X" (LLD and machine coding) questions
    - Everyday objects
    - Booking and reservations
    - Games
    - Platforms and marketplaces
    - Components and infrastructure
    - Machine-coding round favourites
    - Part B — Concept questions
    - OOP basics
    - Relationships and UML
    - SOLID
    - Design principles and clean code
    - Creational patterns
    - Structural patterns
    - Behavioral patterns
    - Concurrency
    - Java specifics for LLD rounds
    - Part C — How to run a 60–90 minute machine-coding round
    - Sources of the question lists

Chapter 1 — The LLD Interview & How to Approach It (Study Edition)

> **Study Edition.** Every topic of the full chapter, without the padding. Full code, outputs and extra examples are in the Full Edition (`README.md`).
>
> **You will learn:** what the LLD round is, a complete small answer, an 11-step framework with a time budget, how to get requirements and classes, a SOLID check, the six common patterns, how to write interview code, a worked parking lot, communication tips and a study plan.

1.1 What this round is, and what this handbook covers

What it is

The **low-level design (LLD)** round: design **one application** while an interviewer watches, in 45–60 minutes (machine coding: 90 min–2 h). You design **the classes inside the program** (parking lot, vending machine, chess, Splitwise), not servers. Analogy: system design is the **city plan**; LLD is the **floor plan of one house**.

Term	Meaning
Class / Object	A named type with fields and methods / one concrete instance of it
Responsibility	The one job a class does, in one sentence
Relationship	How two classes connect: uses, holds, owns, or extends
UML	Standard symbols for drawing classes and relationships
Design pattern	A named, reusable answer to a recurring design problem

The hard part is **explaining aloud** why each class exists and **justifying** how they connect.

What you should be able to do after this handbook

Bound one sentence into requirements plus an out-of-scope list; pick classes with **one job** each; relate every pair correctly; code the core flows until they run; name the pattern you used **and the one you rejected**.

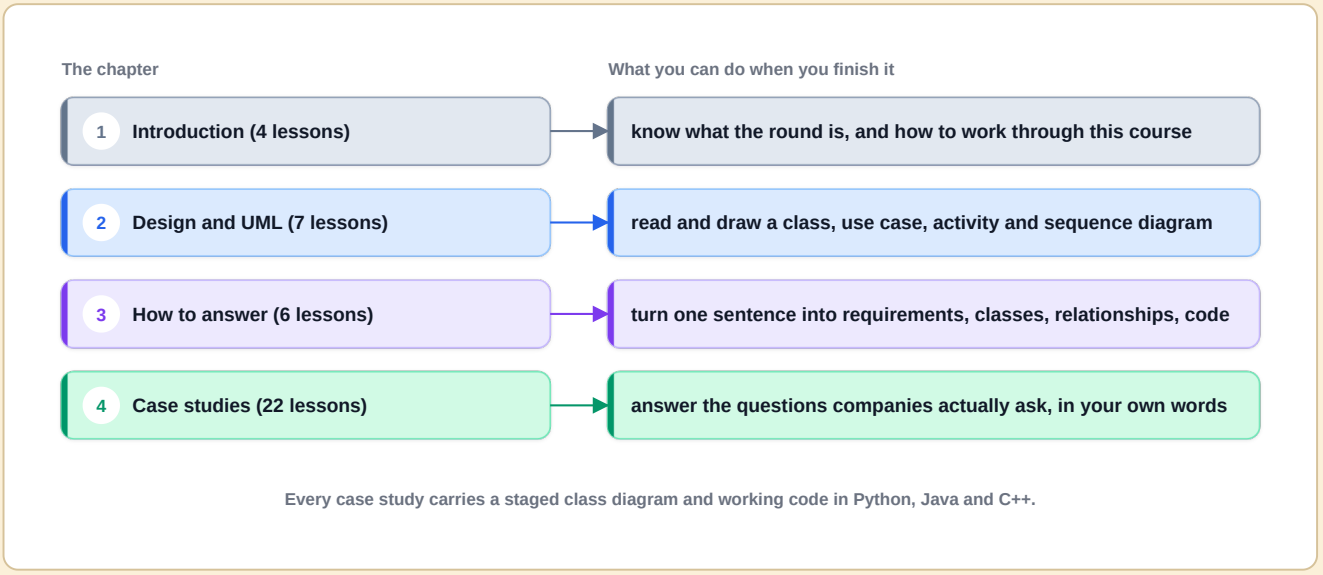


Figure — The four parts of the course material this handbook absorbs: introduction, design and UML notation, the answering method, and the case studies (course diagram)

The handbook follows this arc (round → UML → method → case studies) and adds OOP, SOLID, principles, clean code, refactoring, all 23 GoF patterns and concurrency.

What every case study in this handbook contains · Who this is for · What you need already · What this handbook will not do

- **Case-study template:** requirements with out-of-scope, actors and use cases, a staged class diagram, key flow diagrams, runnable Java with output, patterns **only where really used**, and "Extensions & follow-up questions".
- **For:** anyone facing this round. **Needed:** one OO language (examples use **Java 21**); not UML (Chapter 3), not system design (this round stays **inside one process**), not DSA beyond basic collections.
- **Won't do:** teach a language or give scripts to memorise. **The reusable part is the method.**

TL;DR

Design the classes of one application out loud, then code the main flows. Method beats memorisation.

1.2 How the interview runs

What it is

One sentence ("Design a parking lot"), then silence. Names for the round: **OOD interview** (US big tech), **LLD** (umbrella term), **machine-coding round** (commonly reported at Flipkart, Swiggy, PhonePe: **runnable code is mandatory**, 90 min–2 h).

	LLD	HLD / system design
Scope / vocabulary	One application: classes, interfaces, patterns	Many machines: servers, DBs, caches, queues
Output	Class diagram + code	Architecture + capacity numbers

OOD round vs machine-coding round

	OOD / LLD discussion	Machine coding
Time / output	45–60 min; requirements, diagram, code for 2–3 flows	~90 min coding; a **complete running program**
Storage / interaction	Rarely discussed; continuous conversation	**In-memory**; silent coding, then **demo + walkthrough**
Judged on	Bounding, responsibilities, relationships, trade-offs	Working code, **data models**, **service interfaces**, **extensibility**, readability, edge cases, sometimes thread safety
Common failure	Not converging, coding too early	Not finishing, or one giant class

How it works: the five core steps

1. **Clarify requirements** 2. **Actors and use cases** 3. **Find the classes** 4. **Connect them** (class diagram) 5. **Write the code** for 2–3 main flows.

Order matters more than speed: skip step 1 and step 3 invents classes nobody asked for.

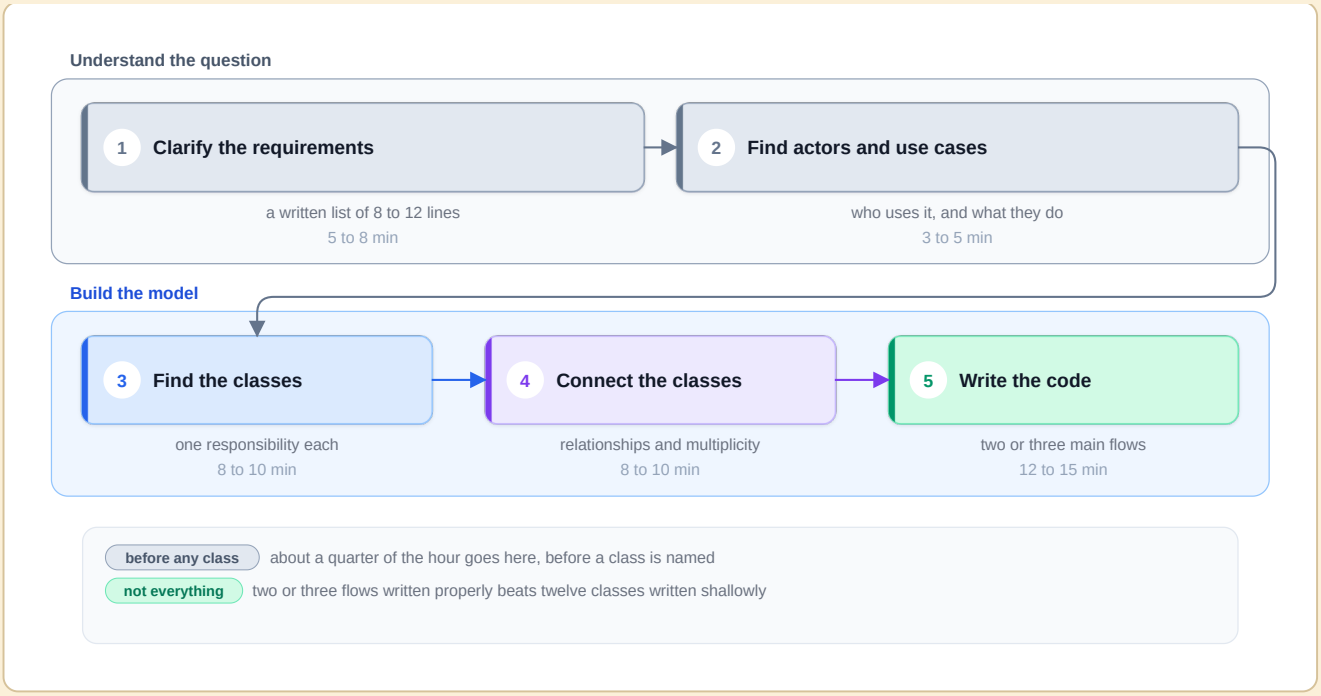


Figure — The five steps split into "understand the question" and "build the model", with a time budget for each (course diagram)

About **a quarter of the hour passes before any class is named**. **Nobody finishes the product**; you are judged on the part you built properly.

Time-boxing

45-min OOD step	Time	Output
Requirements	5–8 min	8–12 lines + out-of-scope
Actors & use cases	3–5 min	2–4 actors, 6–10 use cases
Classes	8–10 min	One responsibility each
Connect	8–10 min	Diagram with multiplicity
Code	12–15 min	2–3 main-flow methods

90-min machine coding: requirements/assumptions 5–10 · model (entities, enums, interfaces) 10–15 · core flow end to end 30–40 · remaining features + edge cases 15–20 · polish + demo 5–10 · buffer 5 (never zero). **Get something running by halfway.**

What the interviewer is actually judging

1. Did you **bound** the problem? 2. **One job** per class? 3. **Right relationships**? 4. Does **code match the diagram**? 5. Can you explain a **trade-off**?

Machine coding adds: **runs**, **modular** (new type = new class), readable, handles bad input, thread-safe (senior). Perfect UML notation is **not** graded; the model is.

A worked example of size

Parking lot: ~12 requirements → ~12 classes → ~15 relationships. Code the three that carry the main flows; name the other nine. Costliest mistake: all twelve at equal depth.

> **TL;DR**
>
> Five steps in order, a quarter of the time before naming a class, two or three flows done properly.

1.3 A complete answer in five minutes: "Design a music playlist"

Step 1: requirements

Songs are **ordered**; a listener asks for **the next song**; **shuffle** changes the order. **Out of scope (said aloud):** accounts, downloads, storage, audio.

Step 2: actors and use cases

One **actor** (someone outside the app): the listener (add song, play next, shuffle). On bigger problems **actors reveal which classes must exist**.

Step 3: classes, one sentence each

Class	Responsibility
`Song`	Title, artist, length; knows only itself
`Playlist`	Holds songs in order; stores, does not play
`PlayOrder` (interface)	Picks the next position (several rules exist)
`InOrder` / `Shuffled`	Insertion order / an order fixed up front
`Player`	Holds playlist + order, remembers the position

A rule that can vary becomes its own type. If a responsibility needs "and" to join two jobs, it is two classes.

Step 4: connect them

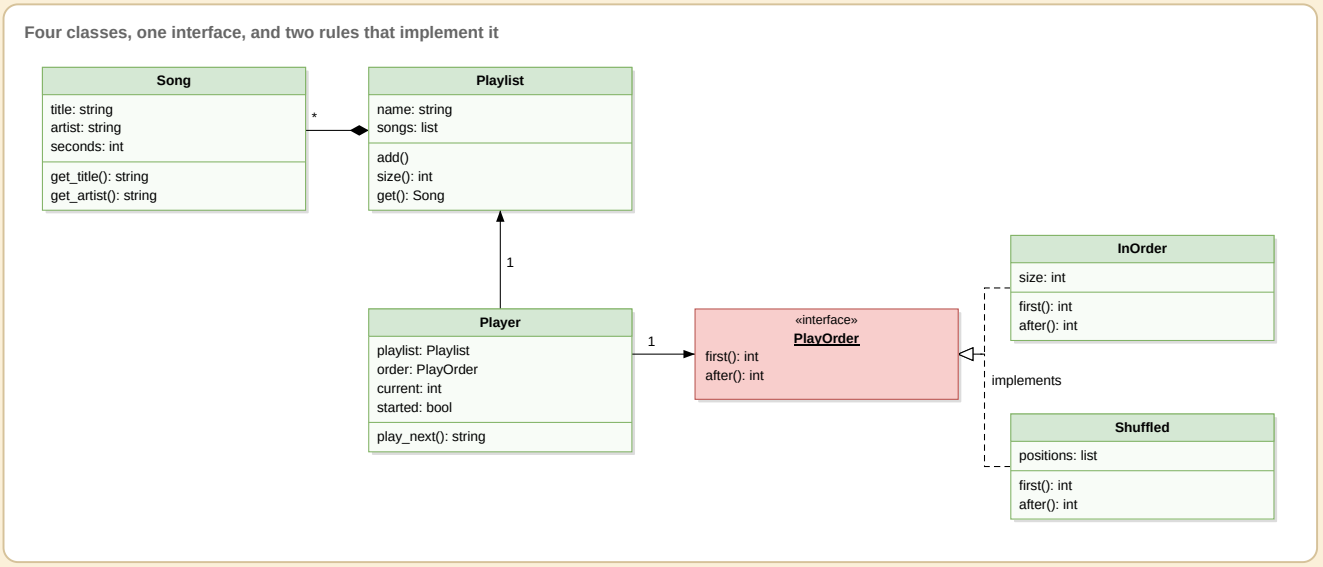
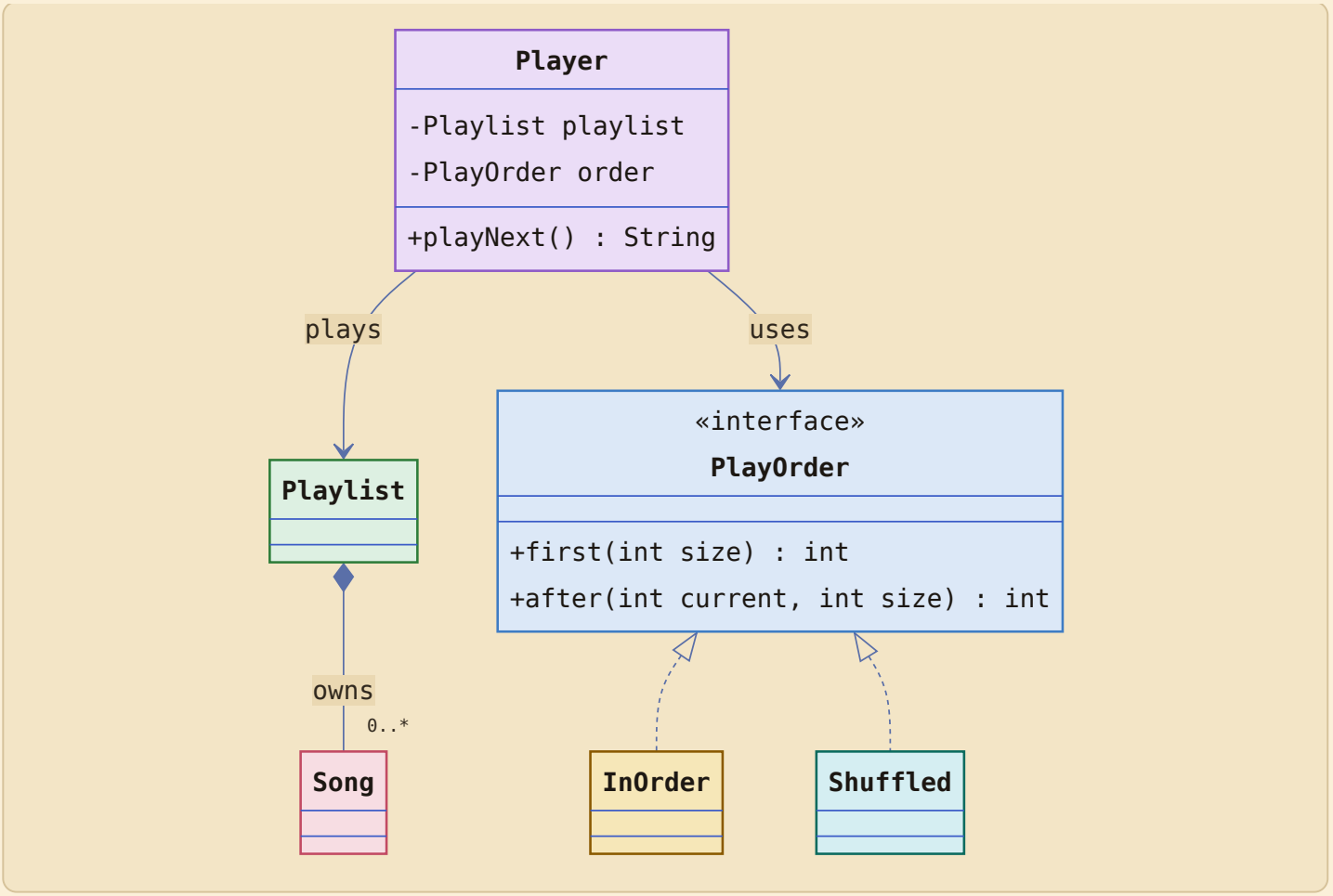


Figure — The playlist design: Playlist owns many Songs, Player uses a PlayOrder, and InOrder and Shuffled implement PlayOrder (course diagram)

Filled diamond = `Playlist` **owns** many `Song`s (composition). Plain arrow = `Player` **uses** the `PlayOrder` interface only. Dashed hollow-triangle arrows = `InOrder`/`Shuffled` **implement** it; a third rule is one more box.



Step 5: the code

```
// File: code/lld/ch01/playlist/PlayOrder.java
public interface PlayOrder {
    int NONE = -1;                          // "there is no next position"

    int first(int size);                    // first position to play, or NONE if empty
    int after(int current, int size);       // position after 'current', or NONE at the end
}
```

```
// File: code/lld/ch01/playlist/Player.java
public final class Player {
    private final Playlist playlist;
    private final PlayOrder order;          // depends on the interface, not on InOrder/Shuffled
    private int current = PlayOrder.NONE;
    private boolean started = false;
    // ... constructor

    public String playNext() {
        if (!started) {
            current = order.first(playlist.size());
            started = true;
        } else if (current != PlayOrder.NONE) {          // once at the end, stay at the end
            current = order.after(current, playlist.size());
        }
        if (current == PlayOrder.NONE) return "end of playlist";
        Song s = playlist.get(current);
        return "playing " + s.title() + " by " + s.artist();
    }
}
```

Full runnable code: code/lld/ch01/playlist/ (see the Full Edition).

`Song` is a validating **record**; `Playlist` keeps its list `private`; `Shuffled` is **given** its order (not `Random`), so output is the same every run: **deterministic = testable**. `Player` never checks which mode is active.

What to notice

- **Shuffle (one line) became a type**: classes come from requirements.
- **One job per class**: `Song` doesn't know playlists exist.
- **Open to change**: "repeat one" is a new `PlayOrder` (Strategy + Open/Closed).
- **Announce deliberate gaps** (pause, volume, queue).

TL;DR

Bounded requirements, one-sentence classes, one diagram, code for the main flow. A varying rule becomes an interface.

1.4 The step-by-step answer framework

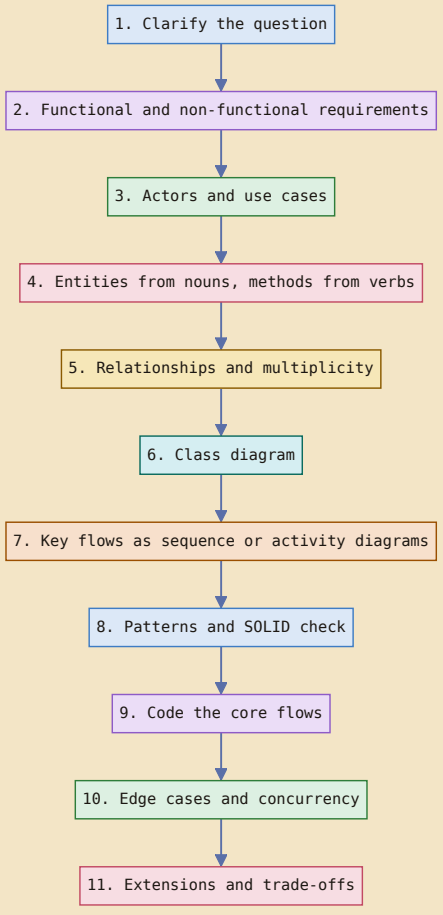
What it is

A checklist for every question, expanding the five steps into eleven.

The problem it solves

Stops freezing and premature coding; makes progress **visible** (which is scored).

How it works



Ask at step 8: varying rule (Strategy)? lifecycle (State)? many listeners (Observer)? At step 10: invalid input, full, two users on one seat?

Trade-offs

Do this	Instead of	Why
8–13 min on steps 1–3	Coding at minute 2	Unbounded scope costs more later
2–3 flows fully	12 stubs	Depth is judged
1–2 patterns with a signal	Six patterns	Over-engineering = poor judgement
Pass time in; `BigDecimal`/cents	Clock in logic; `double` money	Testable; `double` can't hold 0.10 exactly

When not to use it (rigidly)

Follow the interviewer if they steer. In machine coding, steps 5–7 are often mental; **the code is the diagram**.

TL;DR

Clarify → requirements → use cases → entities → relationships → diagram → flows → patterns/SOLID → code → edge cases → extensions. Announce each step.

1.5 From one line to a list of requirements

What it is

A **requirement** is one testable capability. **Functional** = behaviour ("customer takes a ticket"). **Non-functional** = quality (thread safety, extensibility, in-memory).

The problem it solves

Without a list the design has **no edges**, you **can't defend omissions** ("reservations were out of scope; we agreed"), and you **can't tell when you're done**.

How it works: the four questions

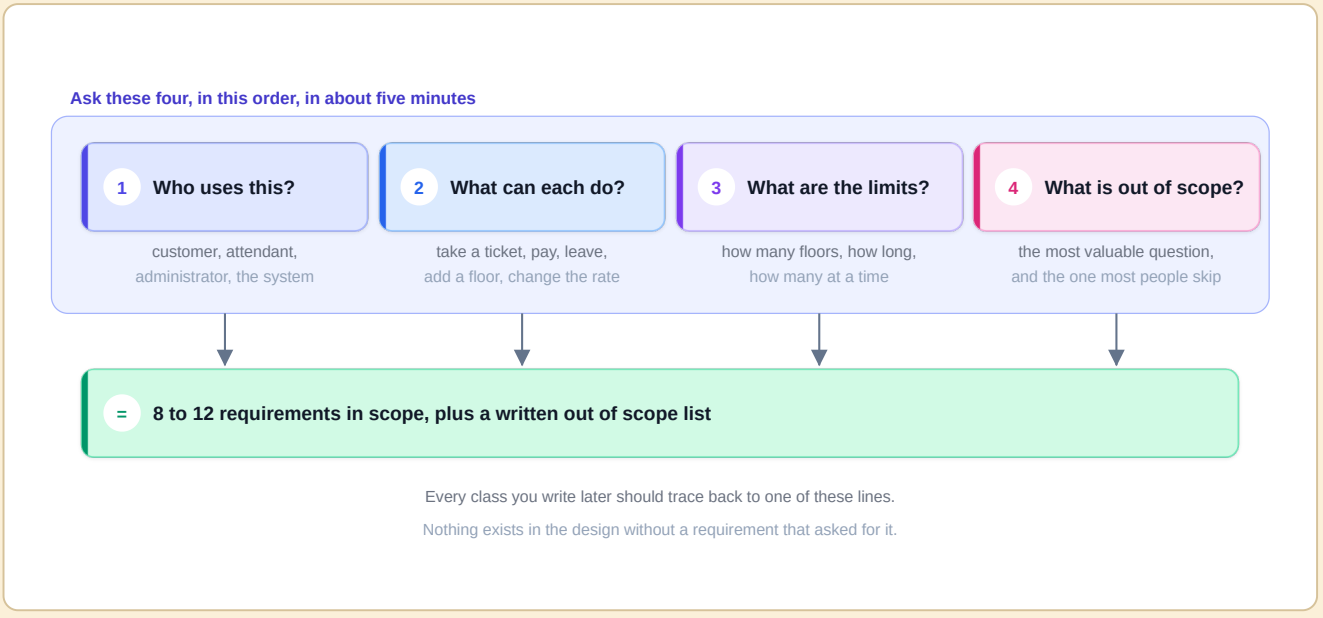


Figure — Four questions asked in order produce 8–12 in-scope requirements plus a written out-of-scope list (course diagram)

1. **Who uses it?** (actors) 2. **What can each do?** (use cases; most requirements) 3. **What are the limits?** (→ constants + validation) 4. **What is out of scope?** (the only question that makes the problem **smaller**).

Ask early: payments in scope? one site or many? users/permissions modelled? Each answer adds or removes 2–3 classes. If told "your choice": **choose, say it, write it down**.

What good looks like: the parking lot in ten lines

In scope: multiple floors of spots; spot types (compact, large, handicapped, motorbike, electric); vehicle types; ticket at entrance; pay at exit/portal/attendant; cash or card; hourly rate by stay length; free-spot display per floor; refuse entry when full; admins add floors/spots/panels and change rates. **Out of scope:** season passes, plate recognition, pre-booking, refunds, multi-site reports. **If no line asked for it, it's not in the design.**

Make each requirement checkable

Put a number on it ("at most **five** books on loan" → constant → one check) and **name the actor** ("**a member** can reserve a copy on loan" → tells you the owning class).

Use cases

Customer: take ticket, park, pay, exit. Attendant: collect cash. Admin: add floor/spot/panel, change rate. System: assign spot, update boards, refuse when full. A list is enough in 45 minutes.

Common mistakes

Saying yes to every idea (declining politely is a **strong** move); forgetting non-functional requirements; vague lines without number or actor.

In the interview

> **30-second answer:** "I ask who uses it, what each actor does, the limits, and what's out of scope. I write 8–12 numbered requirements plus an out-of-scope list, confirm them, and every class must trace back to a line."

- *"Why not just start designing?"* — No boundary means endless growth and indefensible omissions.
- *"'Your choice'?"* — I pick the simpler option and write it as a visible assumption.

Self-check

- **Q:** Which question shrinks the problem? **A:** Out of scope.
- **Q:** Make "users can book some seats" checkable. **A:** "A customer can hold at most 10 seats per show; unpaid holds release after 10 minutes."

TL;DR

Who, what, limits, out of scope → 8–12 checkable lines plus out-of-scope.

1.6 Finding the classes

What it is

Collect candidates, cut non-classes, give each survivor one job.

How it works

1. **Nouns** → candidate classes (deliberately too many). 2. **Verbs** → methods; a verb with no home = a missing class. 3. **Cut with five tests:**

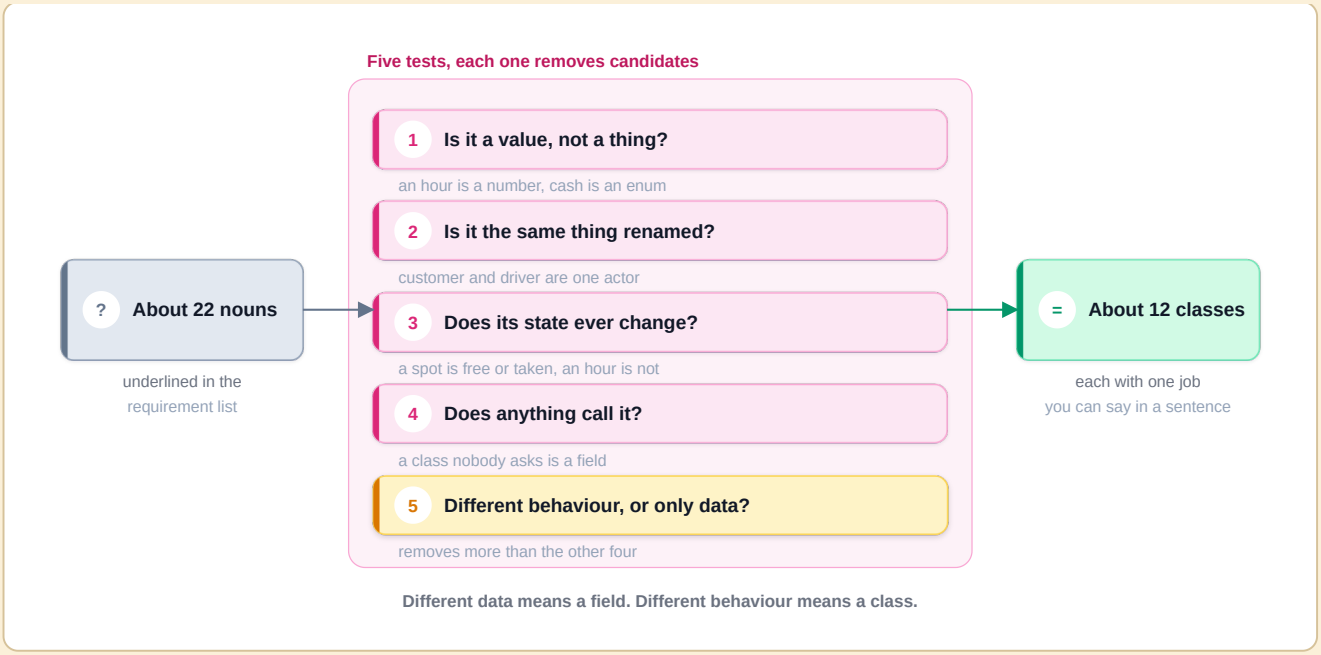


Figure — About 22 nouns pass through five tests and about 12 classes survive, each with one job (course diagram)

#	Test	Example	Result
1	A **value**?	Hour, plate, cash/card option	Field or **enum**
2	**Another name**?	Customer/driver	Merge
3	**Changing state**?	Spot free/occupied	Class
4	Anything **calls** it?	Nobody asks an `Hour`	Value
5	**Behaviour** or only **data** differs?	Compact vs large spot (data); cash vs card (behaviour)	**Data → field. Behaviour → class.**

Test 5 cuts the most: one `ParkingSpot` with a `type` field, but separate cash and card classes. Smell: subclasses that only pass a constant to the parent.

4. **One sentence per class**, e.g. `ParkingFloor` owns a level's spots and finds a free one; `ParkingSpot` tracks its occupant; `ParkingTicket` records vehicle, spot, entry time; `ParkingRate` turns duration into price; `Payment` collects money; `ParkingDisplayBoard` shows free spots. `ParkingLot` (issues tickets **and** tracks fullness) has two jobs: **name the tension aloud**.
5. **Method goes where its data is** ("find a free spot" → `ParkingFloor`). Needing three classes' data = misplaced (GRASP Information Expert; *Feature Envy*).

Common mistakes

One subclass per value; missing the **record-keeping** class (ticket/loan/booking) and the **rule** class (rate/strategy); "Manager" god classes.

Self-check

- **Q:** Car, van, truck: subclasses or enum? **A:** Enum/field unless behaviour truly differs.
- **Q:** Class with fields but no verbs? **A:** Probably a value (record/field).

TL;DR

Nouns → candidates, verbs → methods. **Different data = field; different behaviour = class.** Method where its data is.

1.7 Connecting the classes (preview)

For each pair: **uses** briefly (dependency), **holds** (association), **contains** parts that live alone (aggregation), **owns** parts that die with it (composition), or **is-a** (inheritance/realization); then **multiplicity** (1, 0..1, 1..*, *). Chapter 3 details all. Top error: inheritance where composition fits. A car is not an engine, so `Car` **has an** `Engine`.

1.8 Keeping the design honest with SOLID

What it is

The five SOLID rules (Chapter 4) used as **five quick checks** on your diagram (~5 minutes).

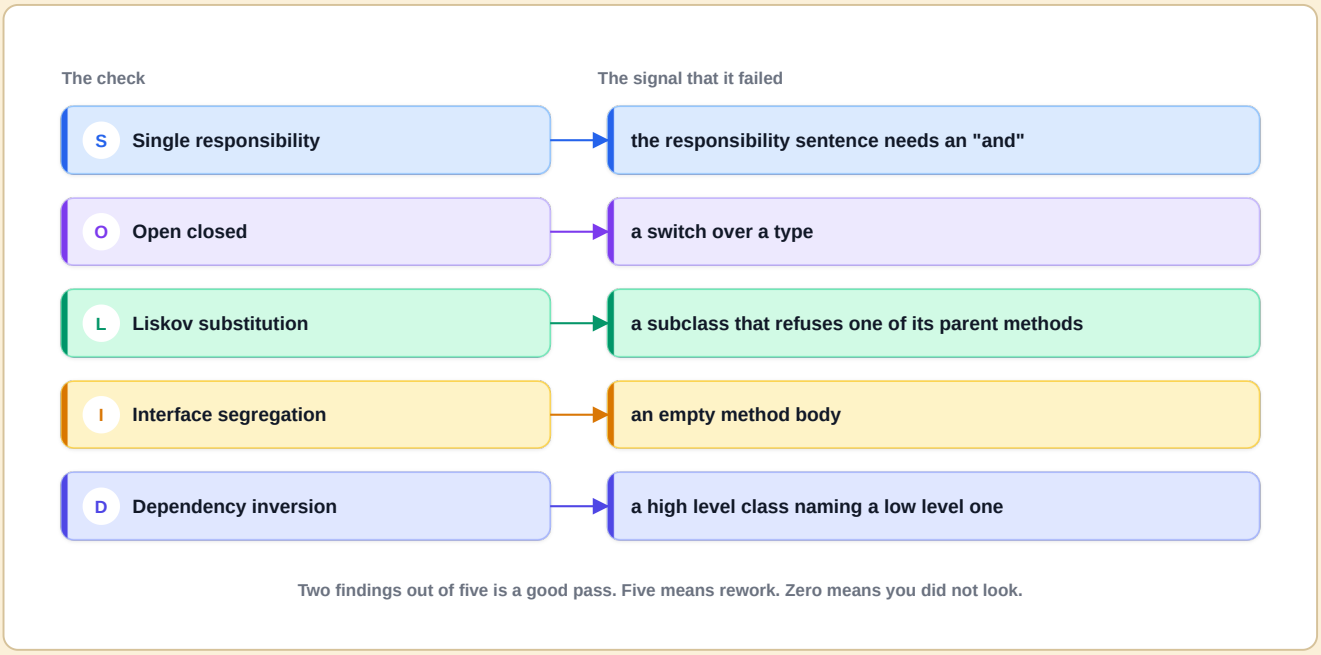


Figure — The five SOLID checks and the signal that each one has failed (course diagram)

How it works: run the five checks in this order

Check	Failure signal	Parking-lot example
S One reason to change	Sentence needs "and"	Lot that assigns spots **and** prints receipts → split
O Extend without editing	`switch` over a type	Weekend rate = new class, not new case
L Child works wherever parent does	Child throws / does nothing	`ReadOnlySpot` whose `occupy` throws
I No unused methods	Empty bodies	Split an interface half its implementers can't honour
D Depend on interfaces	High-level class does `new` of low-level	Lot depends on `Payment`, not `new CreditCardTransaction()`

The two that earn the most credit

S improves the class list; **O** is what gets probed ("trucks charged differently?" → "add a class").

When not to use it

No single-implementer interfaces "because SOLID" (unless a test double needs one); no splitting past what a reader can follow.

A five-minute pass over the parking lot

S: lot has two jobs (split or say it). O: pricing behind `ParkingRate`, passes. L: five behaviour-less spot subclasses aren't a Liskov violation, but they fail "different data → field": collapse. I, D: pass. **Two findings is good; five = rework; zero = you didn't look.**

> **TL;DR**
>
> Check S-O-L-I-D before the interviewer does; S and O matter most; don't over-apply.

1.9 The design patterns that come up in this round

What it is

A **pattern** is a named solution to a recurring problem (GoF has 23; Chapters 8–11). Six recur here. **A misapplied pattern hurts more than an unknown one**: learn the **signal**.

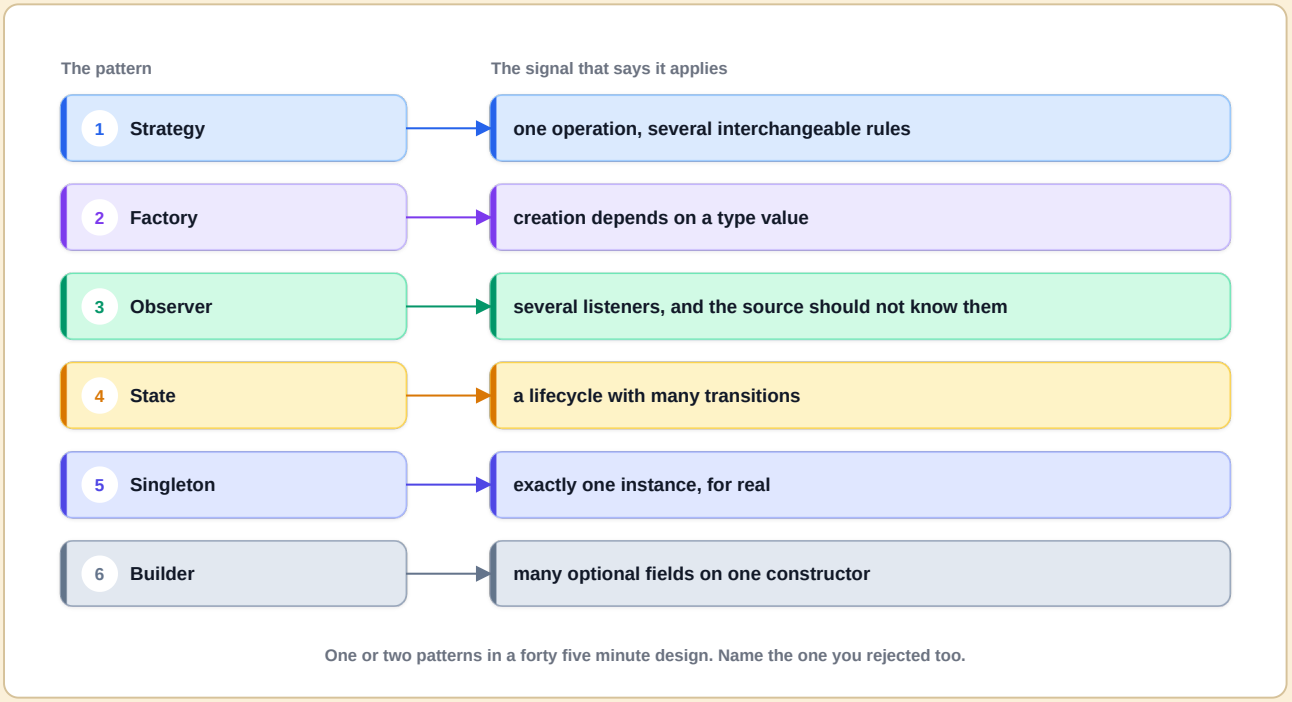


Figure — The six patterns most used in this round and the signal for each (course diagram)

What to say about each one

Pattern	Signal	What to say
Strategy	Several swappable rules for one operation	`ParkingRate` → `HourlyRate`, `WeekendRate`; a new rate is a new class, not a `switch` case
Factory	Object to build depends on a type value	Registering a new type happens in **one place**; a factory method is usually enough
Observer	One change, many listeners the source shouldn't know	The spot is the subject; boards register. One listener → direct call
State	Behaviour depends on state, fixed transitions	Illegal moves become impossible; for 3–4 states, **enum + guarded method**
Singleton	Exactly one instance	Must be **thread-safe**; **global state** hurts tests; often inject one instance instead
Builder	Many optional fields (~6+ constructor args)	Sets fields by name; a readability fix

Choosing patterns without overusing them

Skip each when its signal is absent: one rule (Strategy), one concrete class (Factory), one listener (Observer), three states and one guard (State), just wanting global access (Singleton), three required fields (Builder). **One or two patterns** per design; best is a comparison: "Rate gets Strategy; ticket doesn't get State, one guard covers three statuses."

Real-world examples

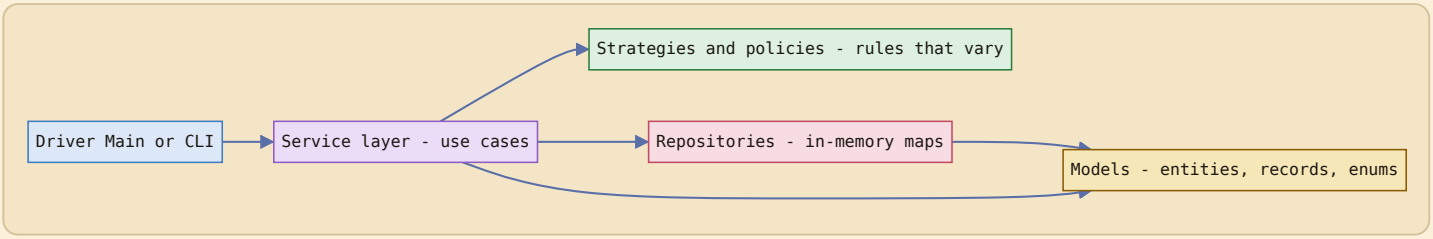
`Comparator` (Strategy); `List.of`, `Executors.newFixedThreadPool` (Factory); `ActionListener`, Spring events (Observer); order lifecycle, TCP (State); `Runtime.getRuntime()` (Singleton; Spring beans are singleton **per container**); `StringBuilder`, `HttpRequest.newBuilder()` (Builder).

TL;DR

Know six; use one or two where the signal appears; name the rejected one.

1.10 Code, edge cases and concurrency: how to write interview code

How it works: a machine-coding skeleton that scales



Models (validated entities, records, enums) → **repositories** (in-memory maps behind an interface) → **strategies** (rules that vary) → **services** (use cases: the **service interface**) → **driver** (`Main` demo).

Habits: pass time in; money as `BigDecimal` (from strings) or cents, never `double`; fail loudly (`IllegalArgumentException`, `IllegalStateException`, clear custom exceptions); `Optional` for normal "not found"; `private final` fields; domain names.

Edge cases and concurrency checklist

Input (null, negative, duplicate/unknown IDs) · **Capacity** (full lot, empty playlist) · **State** (pay twice, exit without entry) · **Time** (exit before entry, round part hours) · **Concurrency** (two threads, same seat: `synchronized`, `ReentrantLock`, `ConcurrentHashMap.computeIfAbsent`; lock the **smallest** thing that keeps the invariant; Chapters 12–13).

TL;DR

Models → repositories → strategies → services → driver; inject time, exact money, validate, guard shared state.

1.11 Worked mini-example, end to end: a small parking lot

Problem & clarifying questions

Assumed answers: single lot, one or more floors; motorbike/car/truck; MOTORBIKE/COMPACT/LARGE spots (smaller vehicles may use bigger spots); payment out of scope (we compute the fee); tiered hourly pricing, swappable; several panels act concurrently.

Requirements

1. Floors with MOTORBIKE, COMPACT, LARGE spots. 2. Motorbike → MOTORBIKE/COMPACT; car → COMPACT/LARGE; truck → LARGE; smallest first. 3. Entry assigns a spot and issues a ticket. 4. No fit → refused. 5. Exit closes ticket, frees spot, returns fee. 6. $4.00 first hour, $3.50 hours 2–3, $2.50 after; round

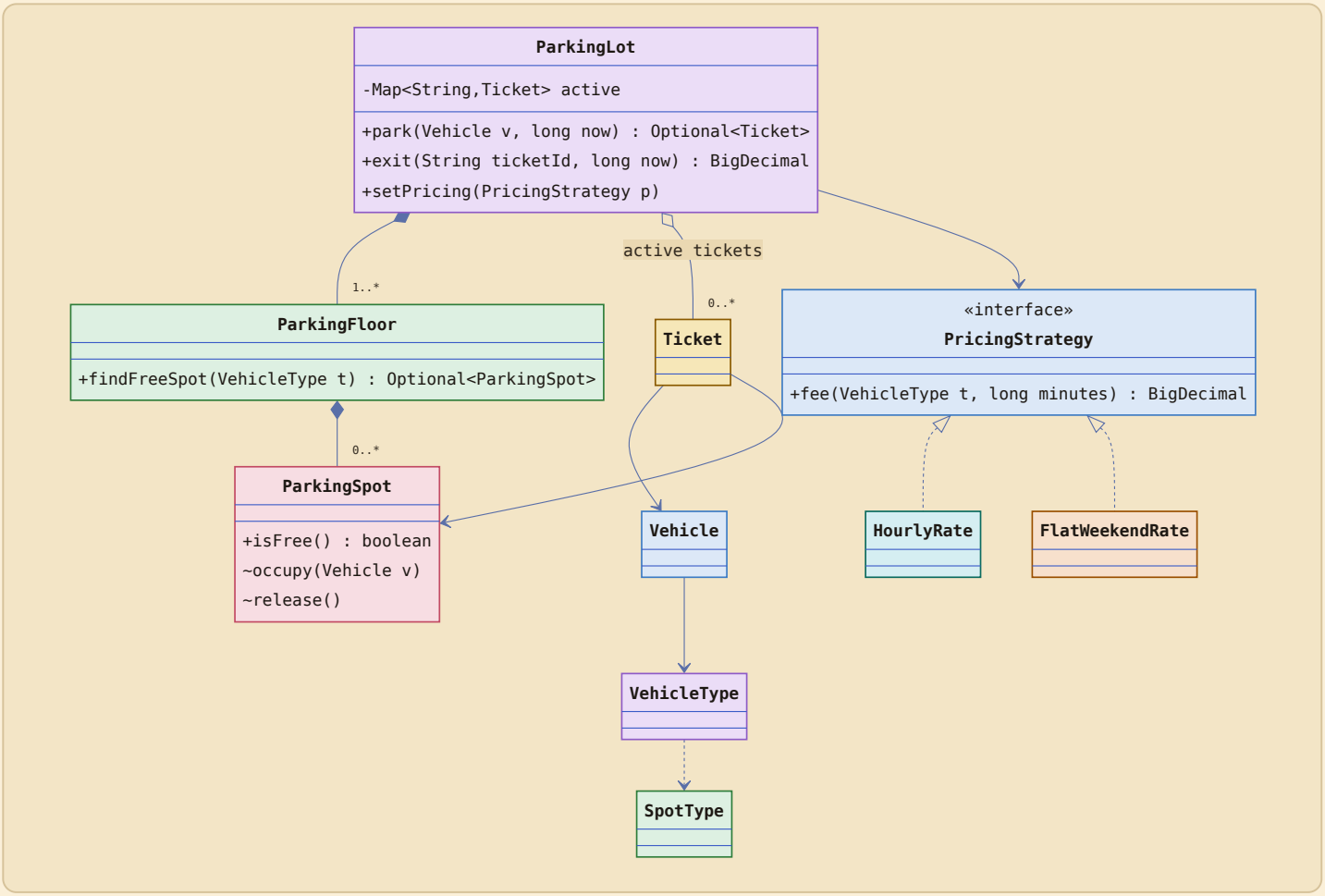
up; minimum 1 h; trucks double. 7. Pricing swappable at run time (flat $10 weekend). 8. Free-spot count per type.

Non-functional: in-memory, thread-safe, extensible, time passed in. **Out of scope:** payment collection, reservations, boards, plate recognition, multiple lots.

Use cases · Core entities

Driver: enter/exit. Admin: add spots, change pricing. System: find spot, refuse, compute fee. Types: `SpotType`, `VehicleType` (enums; vehicle knows which spots fit), `Vehicle` and `Ticket` (records; the ticket is the "connection object"), `ParkingSpot`, `ParkingFloor`, `PricingStrategy` + `HourlyRate`/`FlatWeekendRate`, and `ParkingLot` (the service). Spot sizes are an enum: **different data → field**.

Class diagram



Key flows

`park`: lot asks each floor `findFreeSpot(type)`; if found, `occupy` it and return a `Ticket`, else return empty (refuse). `exit`: remove the ticket, `release()` the spot, return `pricing.fee(type, minutes)`.

Design decisions & patterns used

Decision	Why	Rejected
Strategy pricing	Weekend rule required; open/closed	`switch` in the lot
Enums for types	Data only differs	Subclass per type
`Ticket` record	Fixed after issue (only *shallowly* immutable)	Mutable ticket
`BigDecimal`	`3.50 + 3.50` is exactly `7.00`	`double`
`synchronized` lot methods	Two panels never share a spot	Per-floor locks (extension)
No Singleton, no State	Injectable/testable; only active/closed	`getInstance()`; state classes

Java implementation

```java
// File: code/lld/ch01/parkinglot/HourlyRate.java
public final class HourlyRate implements PricingStrategy {
    // ... FIRST_HOUR = 4.00, HOURS_2_TO_3 = 3.50, LATER_HOURS = 2.50
    @Override
    public BigDecimal fee(VehicleType type, long minutesParked) {
        long hours = Math.max(1, (minutesParked + 59) / 60);    // ceiling, minimum one hour
        BigDecimal total = BigDecimal.ZERO;
        for (long h = 1; h <= hours; h++) {
            total = total.add(h == 1 ? FIRST_HOUR : (h <= 3 ? HOURS_2_TO_3 : LATER_HOURS));
        }
        return type == VehicleType.TRUCK ? total.multiply(BigDecimal.TWO) : total;
    }
}
```

```java
// File: code/lld/ch01/parkinglot/ParkingLot.java
public final class ParkingLot {
    private final Map<String, Ticket> active = new HashMap<>();
    private PricingStrategy pricing;
    // ... floors, nextTicket, constructor

    public synchronized Optional<Ticket> park(Vehicle v, long nowMinute) {
        for (ParkingFloor floor : floors) {
            Optional<ParkingSpot> spot = floor.findFreeSpot(v.type());
            if (spot.isPresent()) {
                spot.get().occupy(v);
                Ticket t = new Ticket("T" + nextTicket++, v, spot.get(), nowMinute);
                active.put(t.id(), t);
                return Optional.of(t);
            }
        }
        return Optional.empty();               // lot full for this vehicle type -> refuse entry
    }

    public synchronized BigDecimal exit(String ticketId, long nowMinute) {
        Ticket t = active.get(ticketId);
        if (t == null) throw new IllegalArgumentException("unknown or already closed ticket " + ticketId);
        if (nowMinute < t.entryMinute()) throw new IllegalArgumentException("exit before entry");
        active.remove(ticketId);               // validate first, then change state
        t.spot().release();
        return pricing.fee(t.vehicle().type(), nowMinute - t.entryMinute());
    }
    // ... freeSpots(type) and setPricing(p), both synchronized
}
```

Full runnable code: code/lld/ch01/parkinglot/ (see the Full Edition).

Output (excerpt): the third car is refused; `T1 exits after 150 min, pays $11.00` (4.00 + 3.50 + 3.50); the truck's 50 min → 1 h × 2 = `$8.00`; after `setPricing(new FlatWeekendRate())` the exit pays `$10.00`; a second exit of T1 prints `error: unknown or already closed ticket T1`.

Concurrency & edge cases

`synchronized park/exit` makes find-then-occupy **atomic** (no *check-then-act* race); `freeSpots` uses the same lock; `ParkingFloor` is only called under it. `exit` **validates before changing state**; `occupy` throws if taken (second defence) and is package-private.

Extensions & follow-up questions

- **EV charging:** `ELECTRIC` spot + `ELECTRIC_CAR`; if charging has behaviour, a `ChargingSpot` or composed component.
- **Nearest spot:** inject a `SpotSelectionStrategy`.
- **Display boards:** Observer on spot events.
- **Throughput:** per-floor/type locks, or a `ConcurrentLinkedQueue` of free spots with atomic `poll()`.
- **Cash/card:** `Payment` interface + implementations, collected by a `PaymentService` (DIP).

In the interview

~5 min requirements, 3 actors, 8 entities (enum vs subclass), 8 diagram, 15 coding `park`, `exit`, `HourlyRate`. Close by naming Strategy (used), Singleton and State (rejected), and per-floor locking.

Self-check

- **Q:** What race does `synchronized park` prevent? **A:** Two threads occupying the same free spot.
- **Q:** How did the weekend rate respect open/closed? **A:** A new class; nothing edited.

TL;DR

8 requirements, 10 types, one Strategy, one lock, an honest out-of-scope list.

1.12 Communication tips

Do: think out loud (name the step), write requirements/classes where visible, confirm before moving on, state assumptions, give trade-offs in pairs ("X over Y because..."), welcome hints, say what you leave out, watch the clock and announce the switch to code, use domain names (not `Manager`/`Helper`), and **when stuck, return to the requirements**. **Don't:** go silent, argue, talk `HashMap`s in minute two, or recite patterns without a reason.

1.13 How to use this handbook, and a study plan

Attempt first, then compare

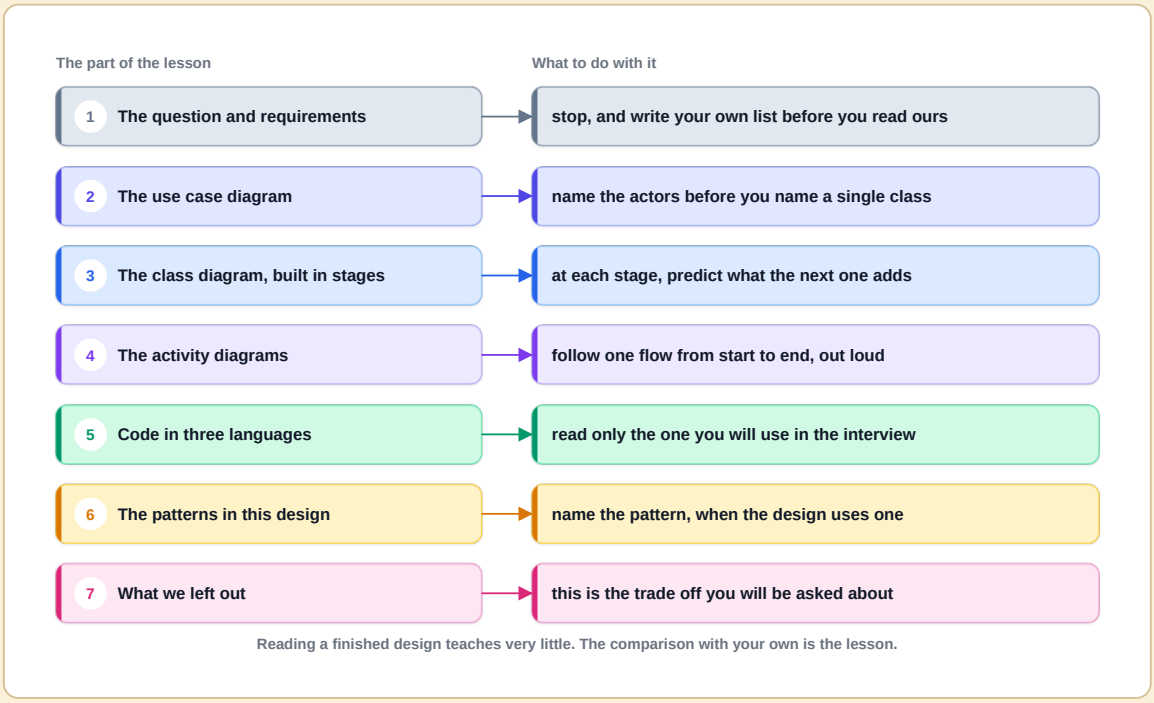


Figure — The parts of a case-study lesson and what to do at each one (course diagram)

Reading builds little skill; **attempt, then compare**. Stop (1) after the question (write requirements), (2) after the requirements (write classes; most people are **two short**: the record class and the rule class), (3) at each diagram stage (predict). Also: argue with decisions in writing, read one language, **never skip extensions**, don't invent pattern names, switch to **timed 45-minute blank-page runs** after 6–7 case studies, run `bash tools/compile_all.sh`.

Map of this handbook

1 approach · 2 OOP · 3 relationships & UML · 4 SOLID · 5 design principles · 6 clean code · 7 refactoring · 8 creational · 9 structural · 10–11 behavioural (+MVC) · 12 concurrency concepts · 13 concurrency

problems · 14 everyday objects · 15–16 booking & inventory · 17 games · 18–19 platforms · 20 components (LRU, rate limiter, Splitwise) · 21 machine coding · 22 cheat sheets.

Study plans

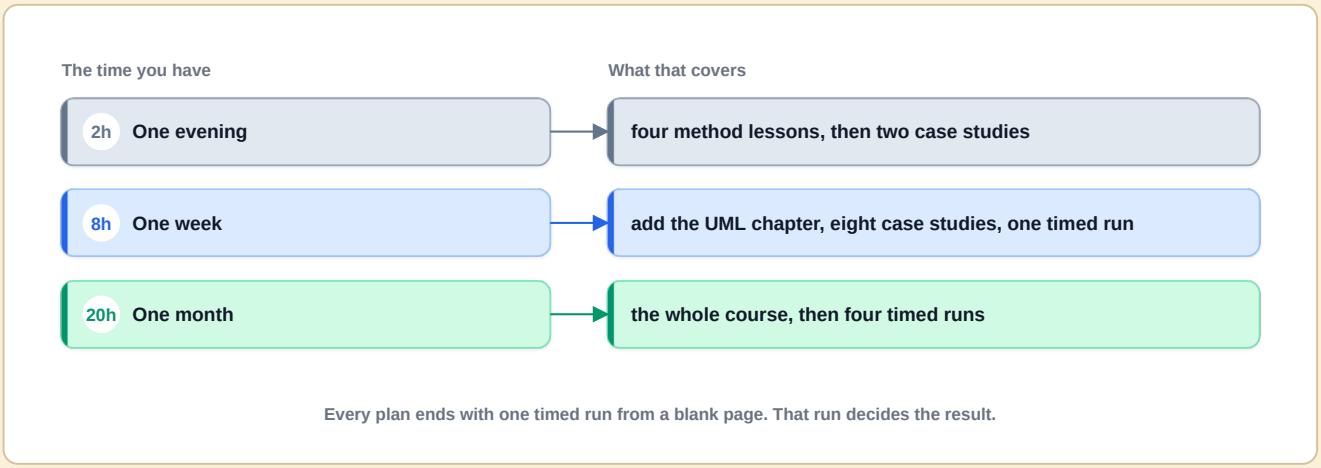


Figure — Three study plans for three amounts of time; every plan ends with one timed run from a blank page (course diagram)

- **Tomorrow (~2 h):** §1.2, §1.5, §1.6, Ch 3 relationship guide + class notation, parking lot (30 min), vending machine (25 min, a **lifecycle**).
- **One week:** Ch 1 + OOP pillars; Ch 3 diagrams; parking lot + library; ATM + vending; movie tickets + hotel; Splitwise + Amazon; **day 7: redo parking lot blank-page in 45 min**.
- **One month (~20 h):** Ch 1–4 + 3 case studies; Ch 5, 8–11 + case studies 4–10; Ch 12–13 + 11–18 + a timed run; the rest, Ch 21, three timed runs, Ch 22.
- **Beginner (6 weeks):** Ch 1–2; 3–5; 6–8; 9–11 + two case studies; 12–13 + four incl. ticket booking; Ch 21 timed runs + Ch 22.

Only six case studies? Parking lot, library, vending machine, movie tickets (concurrency), Splitwise (money), Amazon (too big: choose). Add **LRU cache** if data-structure questions are likely. **Night before:** nothing new; re-read the time budget; recite **requirements, actors and use cases, classes, relationships, code.**

1.14 Common mistakes

Mistake	Fix
Coding in minute two; no out-of-scope list; yes to every feature	8–13 min on requirements; write and agree scope
One subclass per value; god class	Data → field/enum; split on "and"
Missing record class; "Car extends Engine"	Connection with data = object; composition
Pattern stuffing; `switch` on type	1–2 patterns with signal; polymorphism
Equal depth everywhere; silent designing	2–3 flows fully; narrate
Ignoring concurrency; non-running code	Lock shared state; thin path running by halfway
`double` money; clock in logic	`BigDecimal`/cents; inject time

In the interview

30-second answer: "I clarify and write 8–12 requirements with an out-of-scope list, list actors and use cases, pick classes from the nouns with one responsibility each, connect them in a class diagram,

> sketch the key flow, run a SOLID check and pick one or two patterns where the signal is clear. Then I code the core flows, handle edge cases and concurrency, and finish with extensions and trade-offs."

- *"Different from system design?"* — That's many machines and scale; LLD is classes and code inside one application.
- *"You won't finish; what matters?"* — Bounded scope, clean responsibilities, correct relationships, code matching the diagram, one defended trade-off.

Self-check

- **Q:** Five core steps? **A:** Requirements, actors & use cases, classes, relationships, code.
- **Q:** Two classes beginners miss? **A:** The record-keeping class and the rule class.
- **Q:** Patterns in 45 minutes? **A:** One or two where the signal appears; name a rejected one.

TL;DR

- Design the classes of one app and code main flows; machine coding = same, running, ~90 min.
- A quarter of the time before naming a class; always an out-of-scope list.
- Different data → field; different behaviour → class; one sentence per class.
- SOLID as quick checks; one or two patterns with a signal.
- Inject time, exact money, guard shared state; attempt first, then timed blank-page runs.

Chapter 14 — Case Studies: Everyday Objects (Study Edition)

> **Where this fits.** Chapters 1–13 gave you the tools (OOP, UML, SOLID, clean code, the GoF patterns, concurrency). Chapters 14–21 apply them to real interview questions. This chapter starts with **machines you have used yourself** — parking lot, vending machine, coffee machine, ATM, elevator, traffic light, parcel locker — so all your attention can go to the design. This is the study edition: every topic stays, the padding goes. The Full Edition (README.md) has the complete code, outputs and stage-by-stage course diagrams.

#	Problem	The one idea the interviewer is really testing	Main patterns
14.1	Parking Lot	**Resource allocation** under concurrency: two cars must never get the same spot	Strategy (spot finding, pricing), Factory, Observer (display boards)
14.2	Vending Machine	A **state machine**: the order in which actions are allowed	State
14.3	Coffee Vending Machine	**Atomic, all-or-nothing** deduction from a shared inventory; recipes as data	Observer (low-stock alerts), data-driven recipes, sealed result types
14.4	ATM	**Security + money correctness**: authenticate, never lose money, never double-spend	State, Chain of Responsibility (cash dispenser), Command (transactions)
14.5	Elevator System	**Scheduling** (SCAN/LOOK) and **dispatching** many cars	Strategy (dispatch), State-like modes
14.6	Traffic Signal Control	**Timed state machine** with a hard safety invariant and emergency override	State, Strategy (timing), Observer (safety monitor)
14.7	Shipping Locker System	**Best-fit allocation + deadlines + a secret**: reserve a door atomically, expire holds and uncollected parcels, single-use hashed pickup codes	Strategy (door choice), Observer (SMS), enum state machines, sealed results

What these problems test

1. **State machines** — a fixed set of states plus transition rules: no dispensing before payment, no withdrawal before the PIN, never green on two crossing roads. The **State pattern** (Chapter 11) gives every button an answer in every state.
2. **Resource allocation** — spots, notes, milk, cars, locker doors handed out never twice, never below zero, fairly.
3. **Concurrency** — several gates, two ATMs on one account, a timer plus an emergency thread; the design must stay correct when calls overlap (Chapters 12–13).

> **Key insight.** The hardest part of each problem is **one method**: pick a spot, make change, deduct ingredients, pick the next stop. Find it early, name it out loud, spend your time there.

How each case study is laid out

Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns → Java implementation → Concurrency & edge cases → Extensions & follow-ups → In the interview → Self-check → TL;DR. This is also a good order to speak in during an interview.

How to read the course diagrams

The course draws class diagrams in stages (earlier classes in grey, new ones in colour). This study edition keeps only the **finished** course diagram per case plus a mermaid diagram of our Java code.

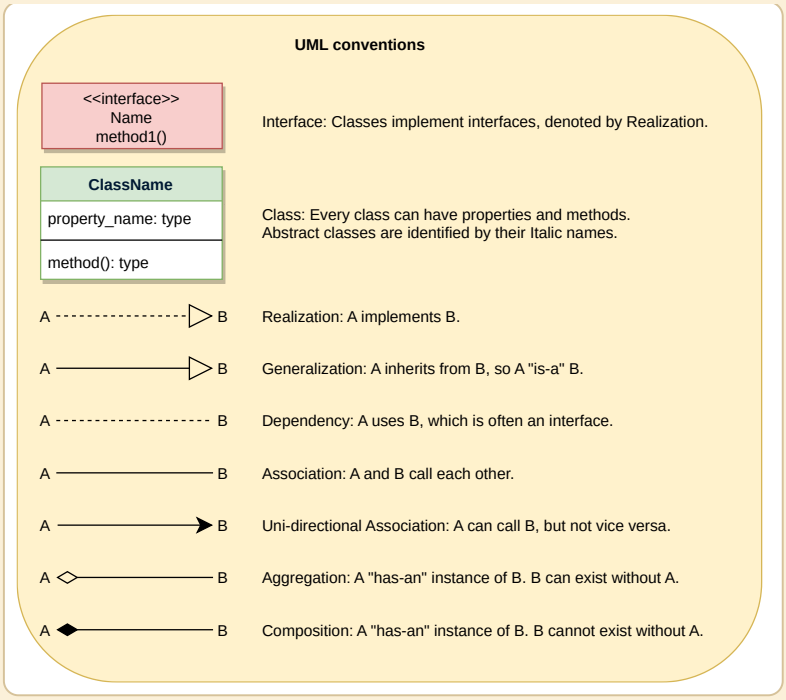


Figure — Legend for the course class diagrams: interface, class, realization, association, inheritance, composition and the other arrow kinds (course diagram)

Hollow triangle = inheritance ("is a"); **dashed line + hollow triangle** = implements an interface; **filled diamond** = composition (the part dies with the whole); **plain arrow** = association (refers to, does not own). See Chapter 3.

How to run every example in this chapter

```
cd chapters/14-case-everyday-objects/code
javac --release 21 -d /tmp/lld_14 $(find . -name '*.java')
java -cp /tmp/lld_14 lld.ch14.parkinglot.Main      # or vendingmachine, coffeemachine, atm, elevator, trafficsignal, locker
```

Java 21, standard library only. **Code-style note:** some files hold several small package-private types (e.g. `Vehicles.java`); fine except for the `auxiliaryclass` lint — in production give each type its own file.

14.1 Design a Parking Lot

A **parking lot** (car park) with many floors, many spot sizes and several ways in and out. It is the **most-asked LLD question**. It looks easy, so the interviewer can watch how you **scope** it, how you model the **fit rule** (which vehicle may use which spot), and whether you notice that **two gates can hand out the same spot** at the same moment.

Problem & clarifying questions

Question	Assumed answer
Scope?	One lot, many floors; several entrances and exits working at once.
Vehicles / spots?	Car, truck, van, motorcycle, EV / compact, large, handicapped, motorcycle, electric.
Payment?	Cash or card at the exit panel, an attendant, or a per-floor info portal.
Fee?	$4 first hour, $3.50 hours 2–3, $2.50 after; a started hour counts as full.
EV charging?	Electric spots have a pay-and-charge panel.
Reservations, passes, plate recognition?	Out of scope (extensions).

Requirements

Functional: (1) multiple floors; (2) multiple entries/exits; (3) ticket at entry, pay at exit; (4) pay at the exit panel or an attendant; (5) cash or card; (6) pay early at a per-floor **info portal** (not charged again); (7) **never exceed capacity** — "full" on the entrance panel and ground-floor board; (8) spot types; (9) **electric** spots with a pay-and-charge panel; (10) vehicle types; (11) a **display board per floor**; (12) tiered hourly fees.

Non-functional: correct under concurrency (no shared spot, no double payment); fast gate (O(log n) from a sorted free set); extensible; testable (injectable clock).

Use cases

Actors: Admin (floors, spots, attendants, rates), Customer, Parking attendant (a customer proxy who takes cash), System (messages, assigns/frees spots). **Top use cases: take ticket**, **scan ticket**, card/cash payment, admin edits, set rate.

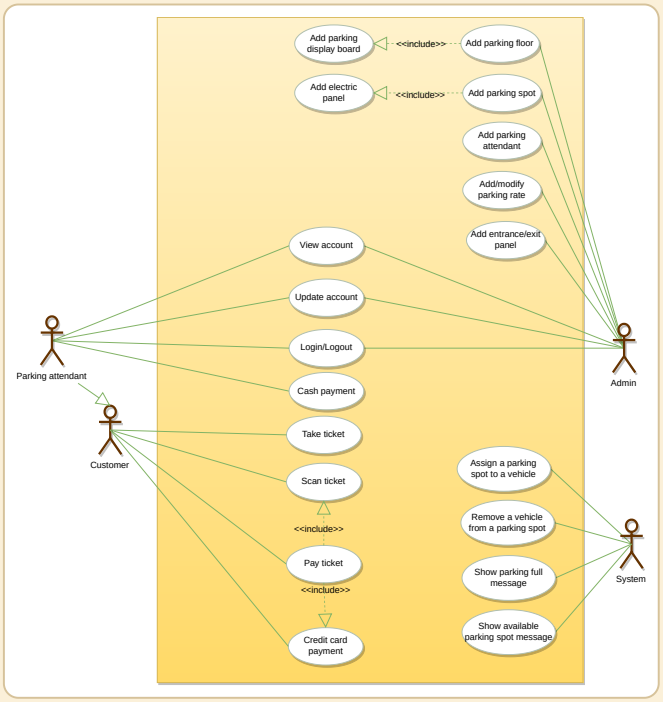


Figure — Use case diagram of the parking lot: Admin, Customer, Parking attendant and System, with their use cases (course diagram)

Core entities

Class	Responsibility
`VehicleType` (enum)	Kinds of vehicle and, for each, the spot types it fits, **best fit first**.
`SpotType` (enum)	Motorcycle, compact, large, handicapped, electric.
`Vehicle` (record)	Plate, type, accessibility permit; knows its preferred spot types.
`ParkingSpot`	One spot: id, floor, type, occupant (atomic).
`ChargingPanel`	Pay-and-charge panel owned by an electric spot.
`SpotFactory`	Builds spots; the only place that knows an electric spot needs a panel.
`ParkingFloor`	Free spots per type in a thread-safe sorted set; notifies boards.
`DisplayBoard`	Free spots per type on one floor (an observer).
`SpotAllocationStrategy`	*How* to choose a spot.
`PricingStrategy`	*How much* a stay costs.
`Ticket`	Vehicle, the **actual spot**, entry time, status ACTIVE → PAID → EXITED.
`PaymentMethod`	Cash or card.
`ParkingLot`	Ties it together: admit, price, pay, exit.
`EntrancePanel`, `ExitPanel`, `CustomerInfoPortal`	Devices customers touch; **no rules**, they call the lot.

The course also has `Account` with `Admin` and `ParkingAttendant` subclasses; they carry no parking logic, so the code omits them.

Class diagram

The course grows the design in **five stages**, each leaving something that runs (a good interview habit: if time runs out, what you drew already works):

Stage	Adds	Key decision
1	Vehicle and spot types	Spot holds **0..1** vehicle; the **fit rule lives on the vehicle** (truck/van → large; EV → electric, compact, large).
2	Floors, boards	Floor owns spots and board; board stores **counts**, not a stale "sample free spot"; one map keyed by type.
3	Ticket, rate, payment	**Ticket stores the exact spot**; abstract `Payment` with cash/card (LSP holds).
4	`ParkingLot`	Owns floors and rate; course makes it a **Singleton** and serialises assignment.
5	Panels, portal, electric panel	Panels are **thin**, forwarding to the lot; electric panel owned by its spot.

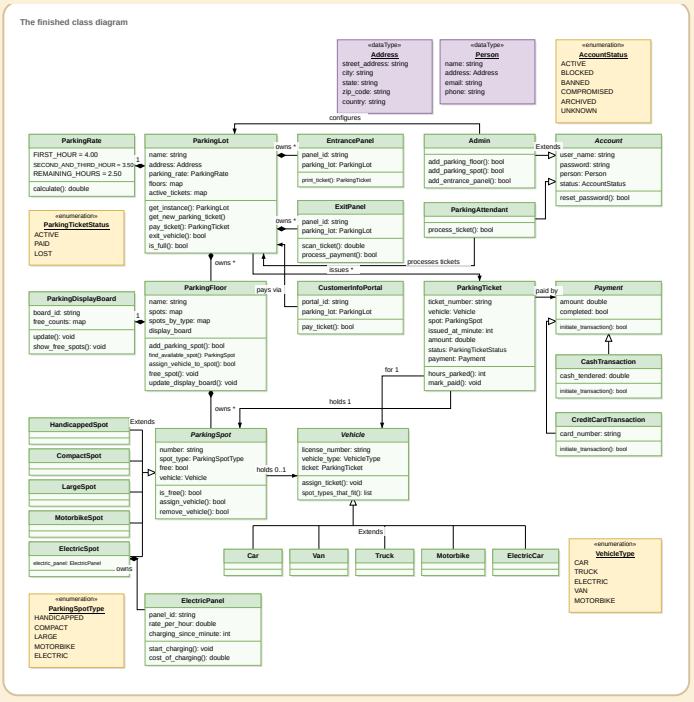
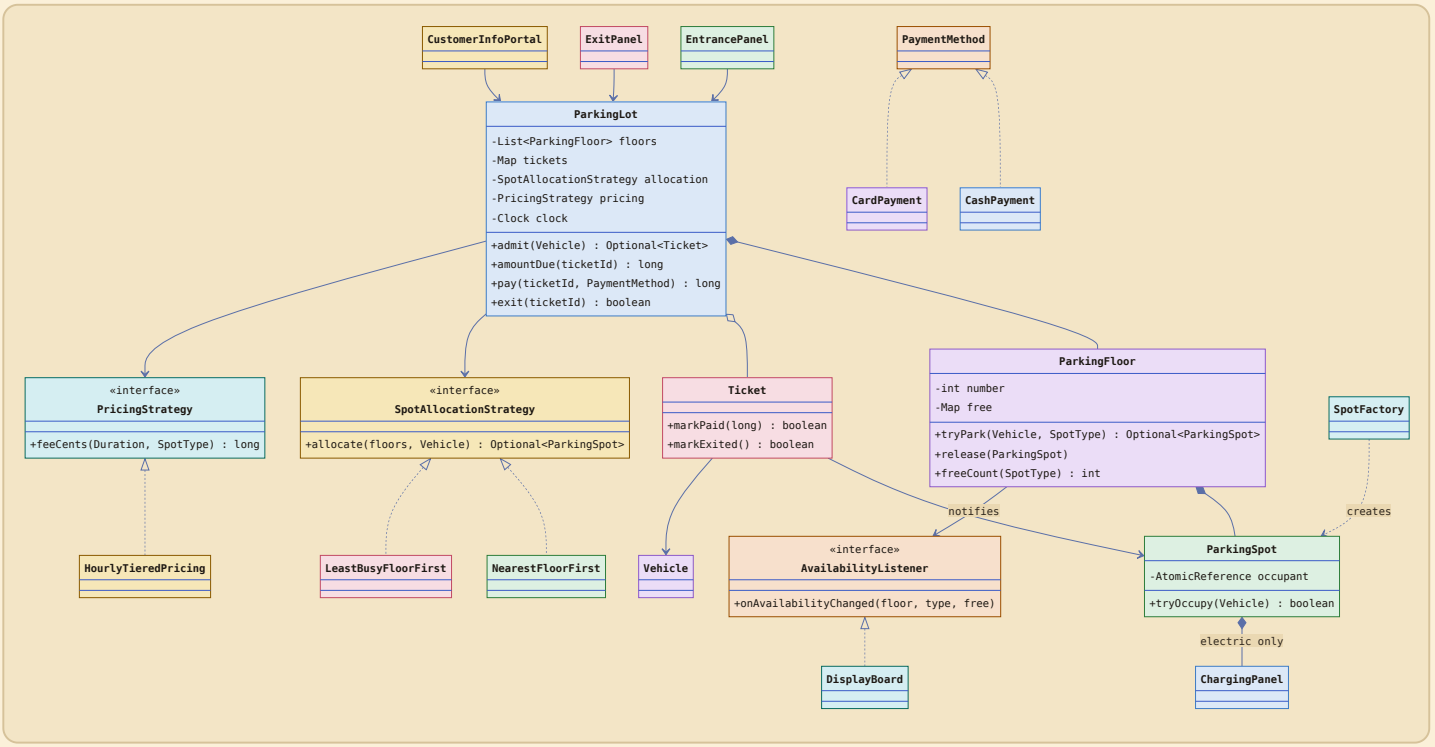


Figure — The finished class diagram of the parking lot (course diagram)

Our Java code keeps the same shape with three changes interviewers push for: spot types are an **enum** (not five empty subclasses), pricing and spot-finding are **Strategies**, and the display board is an **Observer**.



Key flows

- **Entry:** `EntrancePanel` → `lot.admit(vehicle)` → strategy tries each preferred spot type on each floor → `floor.tryPark` does `pollFirst()` on the free set (**atomic**) and notifies the board → ticket printed, or "LOT FULL".
- **Exit:** `ExitPanel` → `lot.pay(ticketId, card)` **locks the ticket**; if ACTIVE, price it and charge, then mark PAID; if already PAID at the portal, charge 0 → `lot.exit` moves PAID → EXITED and releases the spot → barrier opens.
- **Ticket life cycle:** `ACTIVE → PAID → EXITED`.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Fit rule on `VehicleType`	Information Expert, OCP	New vehicle type = one enum constant	`switch(vehicleType)` in the lot
Spot type is an **enum field**	Composition over inheritance	The 5 course subclasses had no behaviour	`CompactSpot`, `LargeSpot`…
`SpotAllocationStrategy`	**Strategy**	Nearest floor, least busy, VIP… swap freely	`if (mode == NEAREST)`
`PricingStrategy`	**Strategy**	"What if pricing changes?"; admin swaps at run time	Tiers hard-coded in `Ticket`
`SpotFactory` attaches `ChargingPanel`	**Factory**	One place knows "electric ⇒ has a panel"	Every caller remembers
Boards subscribe to floors	**Observer**	Boards, apps, bay lights update without the floor knowing them	Lot pushing into boards
Ticket holds the **spot**	—	Exit frees the exact spot	Counting occupied spots
Lot is **not** a Singleton	Dependency injection	A Singleton is a hidden global; tests can't build two lots	`ParkingLot.getInstance()`
`Clock` injected	Testability	Demo jumps hours instantly	`Instant.now()` inside

> **Honest note on "Strategy".** A cash/card subclass pair is just an abstract base class; Strategy means an object **holds** a swappable behaviour (`setPricing(...)`). The course admits the lot does two jobs; spot-finding moved to strategy and floors, and tickets could move to a `TicketRepository`.

Java implementation

Package `lld.ch14.parkinglot`. Amounts are **cents** (`long`) to avoid floating-point rounding.

```
enum VehicleType {
    MOTORCYCLE(List.of(SpotType.MOTORCYCLE)),
    CAR(List.of(SpotType.COMPACT, SpotType.LARGE)),
    ELECTRIC_CAR(List.of(SpotType.ELECTRIC, SpotType.COMPACT, SpotType.LARGE)),
    VAN(List.of(SpotType.LARGE)),
    TRUCK(List.of(SpotType.LARGE));
    // ...
}

class ParkingFloor {
    private final Map<SpotType, ConcurrentSkipListSet<ParkingSpot>> free = new EnumMap<>(SpotType.class);
    // ...
    /** pollFirst() is atomic: two threads can never receive the same spot. Lowest id = nearest. */
    Optional<ParkingSpot> tryPark(Vehicle v, SpotType type) {
        ParkingSpot spot = free.get(type).pollFirst();
        if (spot == null) return Optional.empty();
        if (!spot.tryOccupy(v)) throw new IllegalStateException("free-set out of sync for " + spot);
        notifyListeners(type);
        return Optional.of(spot);
    }
    // ...
}

final class ParkingLot {
    // ...
    Optional<Ticket> admit(Vehicle v) {
        String placeholder = "pending";
        if (ticketByPlate.putIfAbsent(v.plate(), placeholder) != null) {
            throw new IllegalStateException(v.plate() + " is already inside");
        }
        Optional<ParkingSpot> spot = allocation.allocate(floors, v);
        if (spot.isEmpty()) {
            ticketByPlate.remove(v.plate());
            return Optional.empty();
        }
        Ticket t = new Ticket("T" + nextTicket.getAndIncrement(), v, spot.get(), clock.instant());
        tickets.put(t.id(), t);
        ticketByPlate.put(v.plate(), t.id());
        return Optional.of(t);
    }
    // ...
    long pay(String ticketId, PaymentMethod method) {
        Ticket t = ticket(ticketId);
        synchronized (t) {                  // per-ticket lock: portal and exit panel cannot both charge
            long due = amountDue(ticketId);
            if (due == 0) return 0;
            method.pay(due);                // throws if the payment fails: ticket stays ACTIVE
            t.markPaid(due);
            return due;
        }
    }
    // ... exit(): markExited() (PAID -> EXITED), then floor.release(spot)
}
```

Full runnable code: code/lld/ch14/parkinglot/ (see the Full Edition).

How it works: `Vehicle.preferredSpots()` puts `HANDICAPPED` first for permit holders only. `ParkingSpot.tryOccupy` (`compareAndSet(null, v)`) is a second safety net. `NearestFloorFirst` and `LeastBusyFloorFirst` are interchangeable strategies.

Output (excerpt):

```
[entry-A] TRUCK-2 -> SORRY, LOT FULL for TRUCK
[portal] T2 due $16.00, charged $16.00
[exit-A] T2: already paid, barrier OPEN
Unpaid exit attempt: barrier opens? false
== 20 cars race for the remaining CAR spots ==
cars admitted: 5, distinct spots: 5
```

4 h 50 min rounds to 5 hours = $4 + $3.50 + $3.50 + $2.50 + $2.50 = **$16.00**; 20 racing threads get exactly 5 distinct spots.

Concurrency & edge cases

Situation	Risk	Handling
Two gates, last spot	Double booking	Atomic `pollFirst()` + CAS `tryOccupy`
Portal and exit pay the same ticket	Charged twice	`synchronized (ticket)`; a PAID ticket costs 0
Same plate enters twice	Two tickets	`ticketByPlate.putIfAbsent`
Card declined	Marked paid anyway	`method.pay` throws **before** `markPaid`
Leaves without paying	Fee lost	`exit` only from PAID; barrier stays closed
Pays at portal, stays 3 more hours	Free parking	**Grace period** (e.g. 15 min); add `paidAt`, re-price after
Board shows an old number	Out-of-order notifications	OK for display; board can **pull** `freeCount` on render
Lost ticket / 0-minute stay	No entry time / $0	Daily maximum or plate lookup / minimum one hour
Lock scope	Global lock serialises gates	Per-spot CAS and per-ticket lock only

`ConcurrentSkipListSet.size()` is not constant-time and only an estimate under change — fine for a board, never for deciding a spot is free (use `pollFirst()`).

Extensions & follow-up questions

1. **Pricing changes** (weekend, per spot type, surge) → new `PricingStrategy` (or a decorator), `setPricing`. Nothing else changes.
2. **Reservations** → `Reservation(plate, spotType, from, to)` + reserved set per floor; hold with a timeout (e.g. 30 min).
3. **Monthly passes** → pricing returning 0 for pass plates, or a `PassHolderPricing` decorator.
4. **Plate recognition** → the camera is another entrance panel calling `admit`.
5. **Database** → tables `floor`, `spot`, `ticket`, `payment`; claim with `UPDATE spot SET occupied_by=? WHERE id=? AND occupied_by IS NULL` (the DB equivalent of CAS). Show **cardinalities** (floor 1..* spots, spot 0..1 vehicle, ticket 1 spot).
6. **Many lots in a city** → a `ParkingNetwork` routing to lots — another reason for no Singleton.
7. **EV idle fees** → a charging session on `ChargingPanel` with a per-minute fee after charging completes.

Real-world example

Parking guidance systems (per-bay sensors, free counts per level) are our `DisplayBoard` observers; airports use **plate recognition**; SpotHero/ParkMobile pre-booking = reservations; Tesla **idle fees** = the EV extension.

In the interview

45 minutes: 0–5 questions/requirements → 5–10 use cases → 10–25 stages 1–4 ("fit rule on the vehicle", ticket holds the spot) → 25–38 code `admit`, `tryPark`, `pay`, `exit` → 38–45 concurrency and extensions.

> **30-second answer:** "Vehicles and spot types are enums with the fit rule on the vehicle; floors keep a concurrent sorted set of free spots per type; the lot admits, prices and releases. Tickets point at the real spot. Pricing and spot-finding are Strategies. Handing out a spot is atomic — `pollFirst` plus a CAS on the spot — so two gates never get the same spot, and payment locks only the one ticket."

- *"Why not a Singleton?"* — It is a hidden global that makes testing two lots impossible; I create one and inject it. If pressed, a lazy holder-class Singleton is thread-safe.
- *"How do you make finding a spot fast?"* — Keep only free spots, indexed by type, in a sorted set: O(log n), no scan.

Self-check

1. **Q:** Why must the ticket store the spot? **A:** So the exit frees exactly that spot; a count can't say which.

2. **Q:** What makes assignment thread-safe? **A:** `pollFirst()` gives a spot to exactly one caller; `compareAndSet` double-checks.
3. **Q:** Where does a weekend price go? **A:** A new `PricingStrategy` via `setPricing`.
4. **Q:** Paid at the portal — what does the exit charge? **A:** Nothing: `amountDue` is 0 for PAID; `exit` moves PAID → EXITED.

TL;DR

- Fit rule on the **vehicle**, spot type as an **enum**, free spots per type in a **concurrent sorted set**.
- **Ticket → spot** reference; status ACTIVE → PAID → EXITED.
- **Strategy** (pricing, spot-finding), **Factory** (electric spots), **Observer** (boards); inject the lot, no Singleton.
- Atomic `pollFirst` + CAS for spots, per-ticket lock for payment, `putIfAbsent` for plates.

14.2 Design a Vending Machine

A **vending machine** is an unattended shop: key in a slot code, pay, take the item from the tray. The interviewer skips the domain and checks one thing: **which actions are legal at which moment**. Accepting a coin with nothing chosen, or two items for one payment, are bugs no tidy code elsewhere can excuse.

Problem & clarifying questions

Question	Assumed answer
Select or pay first?	Select a slot code first, then pay.
Which money?	Fixed set: 5¢, 10¢, 25¢, $1, $5. Anything else is returned.
Card / contactless?	Out of scope (extension).
Change?	Yes. If **exact** change is impossible, refuse the sale and return everything.
Cancel?	Any time before dispensing, full refund.
Concurrency?	One customer at a time (one physical machine).
Restocking?	An operator restocks, loads coins, collects takings.

Requirements

Functional (the course's twelve): (1) many **slots**, each with a code, a **single** product and a **count**; (2) a product has a name and a **price in cents**; (3) **choose, then pay**; (4) only known coins/notes, others returned; (5) the **display** says what to do after every action; (6) when paid enough, release **item and change**; (7) if **exact change is impossible**, refuse and return every cent; (8) **cancel** with full refund until the item drops; (9) an empty slot **cannot be chosen**; (10) one payment = **exactly one** item; (11) an **operator** refills, adds change, removes takings; (12) item and cash counts are **exactly right** after every sale, cancel and refusal.

Non-functional: no illegal transitions (every button answered in every state); integer cents only; extensible (new payment device or state without editing every class).

Use cases

Actors: Customer, Operator, System. **Use cases:** select · insert money · dispense · collect · cancel · restock · load change / collect takings. "Dispense" is a **System** use case — the customer only causes it.

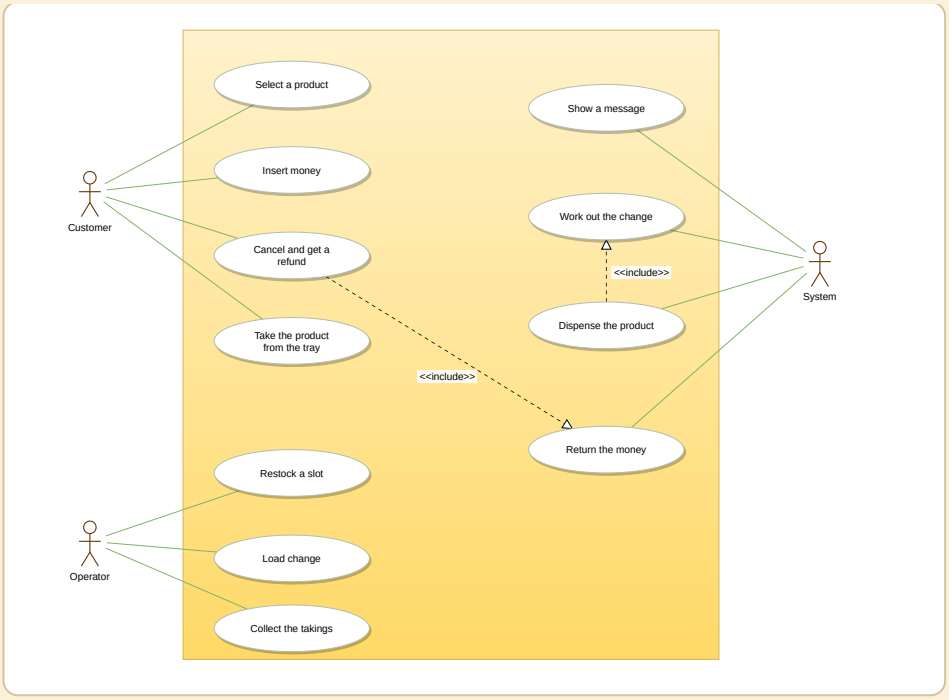


Figure — Use case diagram of the vending machine: Customer, Operator and System (course diagram)

Core entities

Class	Responsibility
`Product` (record)	Name and price in cents — *what* is sold.
`Slot`	One column: code, one product, **count** left.
`Inventory`	All slots by code.
`Denomination` (enum)	Accepted coins/notes and their cents.
`CashBox`	Counts pieces by value; plans **exact** change or reports it can't.
`VendingState` (interface)	One method per customer button: `select`, `insert`, `cancel`, `take`.
`IdleState`, `SelectedState`, `HasMoneyState`, `DispensingState`, `OutOfStockState`	The situations; each answers every button and names the next state (course calls `HasMoneyState` `PaidState`).
`Display`, `Keypad`, `CashSlot`, `Dispenser`, `Operator`	Course hardware/actor classes; in our code folded into `VendingMachine.show(...)` and its public/operator methods.
`VendingMachine`	Parts, current state, selection, balance; forwards each button to the state.

Why is the count on the slot, not the product? `Product` answers "what is it?"; `Slot` is the physical column. The same product can sit in two columns (even at different prices), and emptying one must not affect the other.

Class diagram

Course stages: (1) `Product`, `Slot` (association — a product can be in many slots), `Inventory` (composition); (2) `Coin`, `Note`, `CashBox` — **a piece goes into the box on insert**, so a refund is "pay exact change equal to the balance"; the planner returns an exact plan or "impossible"; (3) **the heart: one class per state**, each button a method on one interface — a state that forgets a button does not compile; think of a **grid** of buttons × states where every cell needs an answer; (4) `Display` and `VendingMachine` — no `if` on state; (5) `Keypad`, `CashSlot`, `Dispenser`, `Operator` point **to** the machine; `take` finishes the sale; operator actions stay **off** the state interface.

The course draws four states; our code adds `OutOfStockState` so an empty machine refuses customer buttons until restocked. Cancel and "no exact change" end identically, so there is a **single** refund method.

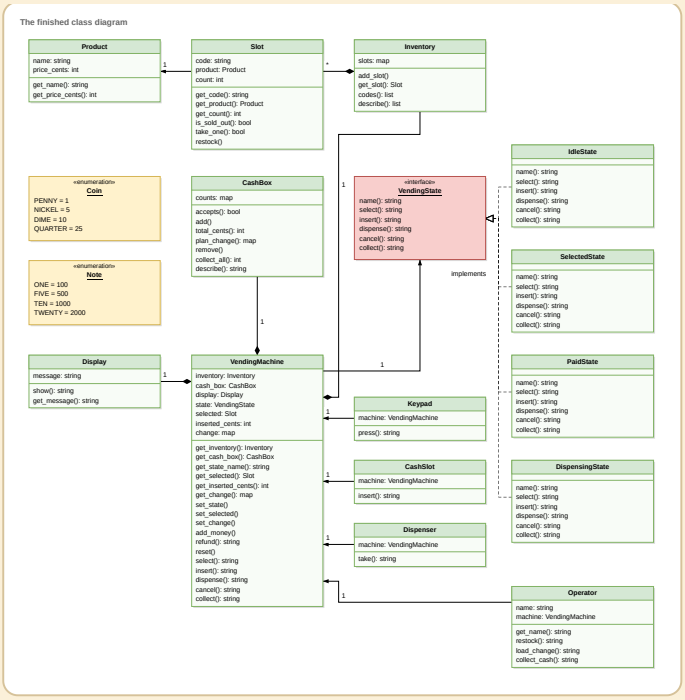
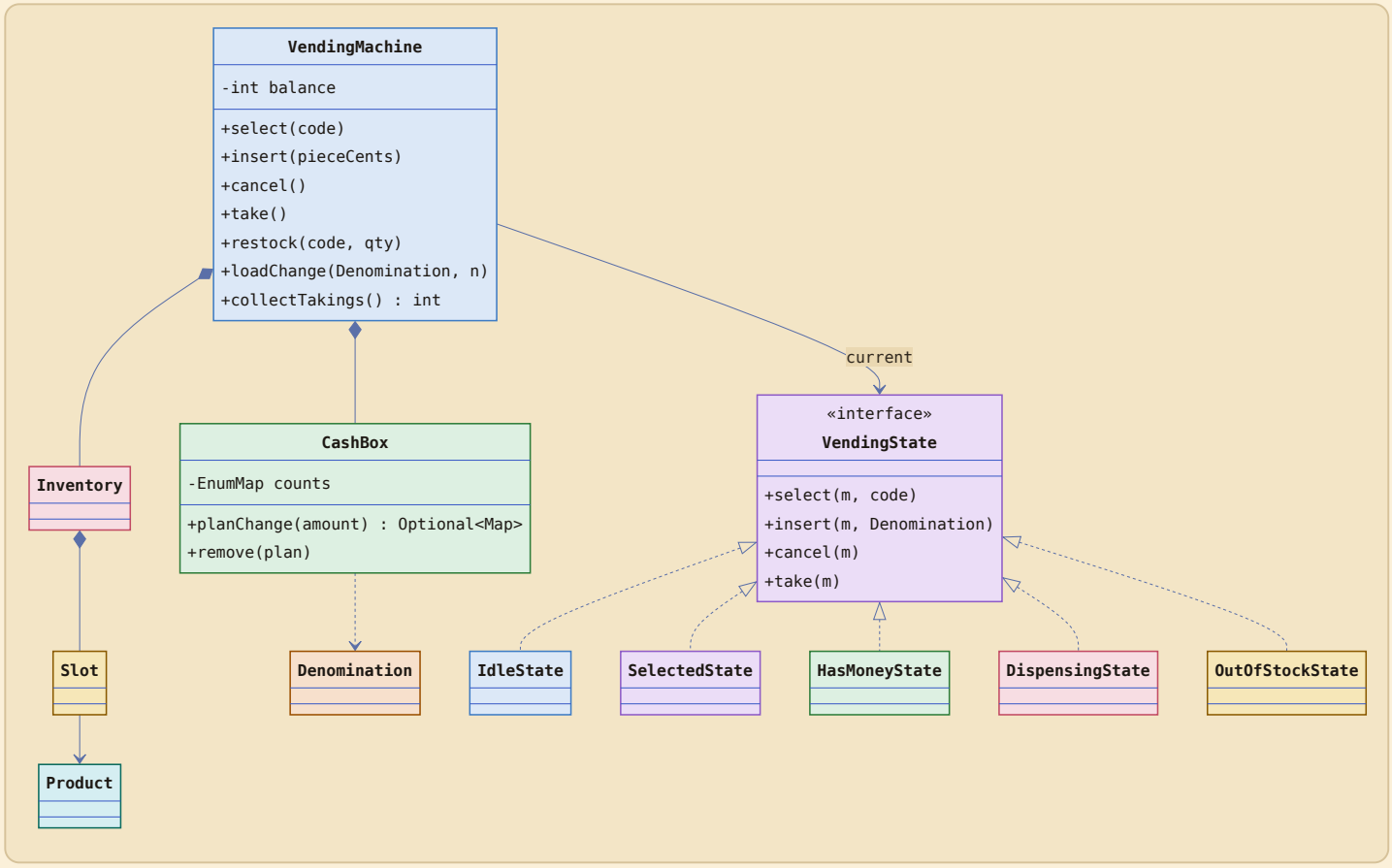
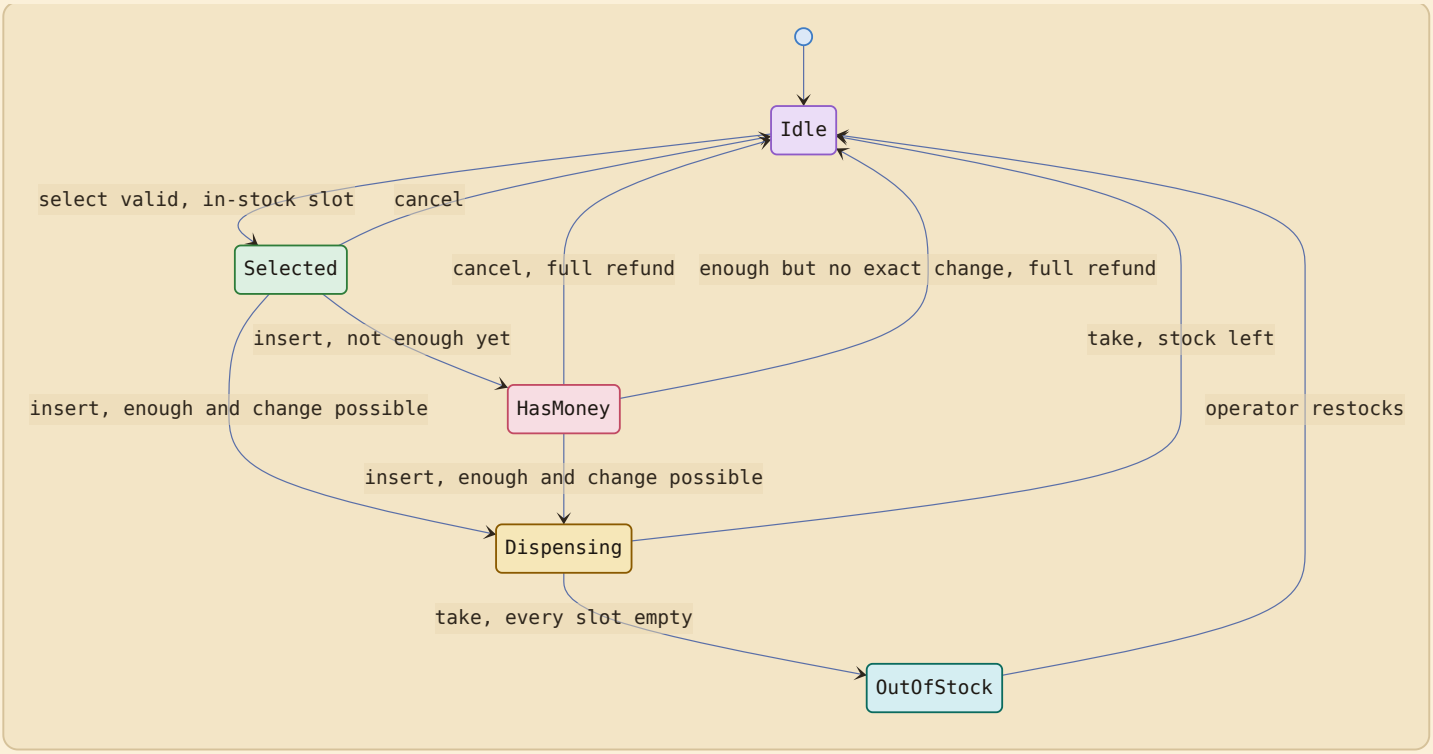


Figure — The finished class diagram of the vending machine (course diagram)



Key flows

The state machine is the design — draw it before coding.



State table (every cell answered):

Button ↓ / State →	**Idle**	**Selected**	**HasMoney**	**Dispensing**	**OutOfStock**
`select`	choose slot (reject empty/unknown)	change choice	"cancel first"	"take your product first"	"out of stock"
`insert`	return piece	accept; check paid	accept; check paid	return piece	return piece
`cancel`	"nothing to cancel"	clear choice → Idle	**refund all** → Idle	"too late"	"nothing to cancel"
`take`	"tray empty"	"pay first"	"pay the rest"	give product + change → Idle/OutOfStock	"tray empty"

Paying enough: insert → `acceptPiece` (into box, balance += value) → `checkPaid` → `planChange(balance − price)`; if a plan exists, remove it, `dispenseOne`, go to Dispensing; otherwise refund all and go Idle.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
One class per situation	**State**	Every cell answered; compiler enforces completeness	Flags + `if/else` (double-dispense bugs)
States are stateless singletons (`INSTANCE`)	Flyweight-style sharing	No data, so one instance each	`new IdleState()` each transition
Operator actions off the state interface	Interface Segregation	Restock is valid in every state	`restock` on `VendingState`
Money counted by value in an `EnumMap`	—	Which physical quarter comes back doesn't matter	List of coin objects
Inserted money straight into the box	—	Refund = make change for the balance	Separate escrow pile
Exact change by dynamic programming	Correctness	Greedy fails when short of one coin	Greedy only
Refuse if exact change impossible	Req. 7	Never short-change	"Close enough" change

> **About the change algorithm.** The course's planner is **greedy** (largest piece first) and says so: correct for standard coin sets when well stocked, but it can fail when the box is short. The fix is **dynamic programming** — "coin change with limited coins". Example: 30¢ from {1 quarter, 3 dimes}: greedy takes the quarter and is stuck at 5¢; DP returns three dimes.

Java implementation

Package `lld.ch14.vendingmachine`.

```java
/** State pattern: every button is on the interface, so every state must answer every button. */
interface VendingState {
    void select(VendingMachine m, String code);
    void insert(VendingMachine m, Denomination d);
    void cancel(VendingMachine m);
    void take(VendingMachine m);
    String name();
}

/** Some money is in, but not enough yet. */
final class HasMoneyState implements VendingState {
    static final HasMoneyState INSTANCE = new HasMoneyState();

    @Override public void select(VendingMachine m, String code) { m.show("Cancel first to change your choice."); }
    @Override public void insert(VendingMachine m, Denomination d) { m.acceptPiece(d); m.checkPaid(); }
    @Override public void cancel(VendingMachine m) { m.refundAll("Cancelled."); }
    @Override public void take(VendingMachine m) { m.show("Please pay the rest first."); }
    @Override public String name() { return "HAS_MONEY"; }
}
// ... IdleState, SelectedState, DispensingState, OutOfStockState

final class VendingMachine {
    // ...
    /** Called after every accepted piece in SELECTED / HAS_MONEY. */
    void checkPaid() {
        int price = selected.product().priceCents();
        if (balance < price) {
            show("Paid %d c, %d c to go.".formatted(balance, price - balance));
            setState(HasMoneyState.INSTANCE);
            return;
        }
        var change = cashBox.planChange(balance - price);
        if (change.isEmpty()) {                              // refuse the sale rather than short-change
            refundAll("Cannot make exact change.");
            return;
        }
        cashBox.remove(change.get());
        tray.add(selected.dispenseOne().name());
        if (!change.get().isEmpty()) tray.add("change " + change.get());
        show("Dispensed. Please take: " + tray);
        balance = 0;
        selected = null;
        setState(DispensingState.INSTANCE);
    }

    /** One refund path for "cancel" and for "cannot make change". */
    void refundAll(String reason) {
        Map<Denomination, Integer> back = cashBox.planChange(balance)
                .orElseThrow(() -> new IllegalStateException("inserted pieces must be refundable"));
        cashBox.remove(back);
        show(reason + " Refunded " + balance + " c as " + back);
        balance = 0;
        selected = null;
        setState(IdleState.INSTANCE);
    }
    // ...
}
```

Full runnable code: code/lld/ch14/vendingmachine/ (see the Full Edition).

How it works: unknown pieces are returned before reaching a state. `CashBox.planChange` is bounded coin-change DP (fewest pieces); `greedyPlan` is only for comparison. `refundAll` cannot fail — the inserted pieces are in the box. Public methods are `synchronized`.

Output (excerpt):

```
 > insert 500c   (SELECTED)
   [display] Cannot make exact change. Refunded 500 c as {FIVE_DOLLAR=1}
30c from {1 quarter, 3 dimes}: greedy=Optional.empty dp=Optional[{DIME=3}]
```

Concurrency & edge cases

Edge case	Handling
Money before selecting	`IdleState.insert` returns it
Unknown coin / foreign note	Returned before reaching the state
Empty slot selected	Refused; stays Idle
Change choice after paying	"Cancel first" (avoids price confusion)
Overpayment	Change planned **before** dispensing; refuse if impossible
Cancel after dispensing	"Too late"
Two products for one payment	Impossible: balance reset to 0, Dispensing accepts no money
Last item sold	`take` → `OutOfStock`; only a restock leaves it
Two threads pressing buttons	`synchronized` public methods
Machines sharing one stock / power failure	Slot count needs a lock or atomic decrement / non-volatile transaction log (out of scope)

Extensions & follow-up questions

1. **Contactless card** → a new input device calling the same machine (e.g. `payByCard(amount)` setting the balance); states unchanged.
2. **Maintenance state** → a new `MaintenanceState` class; existing states untouched (OCP).
3. **Different price per slot** → move price from `Product` to `Slot`.
4. **Report empty slots** → `Inventory` notifies an observer; a telemetry client sends it.
5. **Walk-away timeout** → a timer calls `cancel()` after ~60 s through the same state machine.
6. **What did you leave out?** Card payment, network reporting, refrigeration, discounts — say they are scope decisions.

Real-world example

Cashless readers (**Nayax**, **Cantaloupe ePort**) plug in as another payment device; coin mechanisms light **"exact change only"** when a tube runs low — our planner saying it cannot make change.

In the interview

45 minutes: 5 questions → 5 entities → **10 draw the state diagram and table** → 15 code the interface, two or three states and `checkPaid`/`refundAll` → 10 edge cases.

> **30-second answer:** "The question is the order of events, so I use the State pattern: Idle, Selected, HasMoney, Dispensing, OutOfStock, all implementing an interface listing every button. The machine forwards to its state. Money goes into the box on insert, so a refund is 'make exact change for the balance'; if exact change is impossible the sale is refused and everything returned. Integer cents."

- *"Why not an enum with a switch?"* — Every new state means editing every switch; a State class is forced by the compiler to answer every button.
- *"Is greedy change correct?"* — Only for well-stocked canonical coin sets; use bounded coin-change DP.

Self-check

1. **Q:** What makes the design complete? **A:** Every state implements every button, so every (state, button) cell has an answer.
2. **Q:** Why is `restock` not on `VendingState`? **A:** Not a customer action, valid in every state (ISP).
3. **Q:** A greedy failure? **A:** 30¢ with one quarter and three dimes.
4. **Q:** How is "two items for one payment" prevented? **A:** Balance reset to 0 and Dispensing returns any coins until `take`.

TL;DR

- **State pattern is the design**: five states, one interface listing every button, no `if` on state.
- Count on the **slot**; **integer cents**; money into the **box immediately**.
- Change **exact or refuse**; greedy can fail, **DP** is correct.
- One refund path; operator actions off the state interface.

14.3 Design a Coffee Vending Machine

A **coffee machine** makes drinks (espresso, americano, latte, cappuccino, mocha, hot chocolate) from **shared ingredients** (beans, water, milk, chocolate, sugar). Unlike 14.2, a drink **consumes several ingredients at once**, and many drinks draw on the same milk. The interviewer wants: (1) **recipes as data** — a new drink is a data change; (2) **atomic, all-or-nothing deduction** — a latte takes beans, water *and* milk together or nothing, never negative even with ten concurrent orders; (3) **low-stock alerts** via Observer before an ingredient runs out.

Problem & clarifying questions

Question	Assumed answer
Drinks?	Six, loaded from a menu file.
Customisation?	Extra espresso shot (+50¢), sugar in grams.
Payment?	Amount in cents; change returned (coins covered in 14.2).
Concurrent orders?	Yes — several touch screens.
Refills?	Operator, alerted at a threshold.
Alert frequency?	**Once per crossing** of the threshold, re-armed after refill.

Requirements

Functional: (1) drinks with a **price** and a **recipe**; (2) **menu** with prices and availability; (3) selection and **payment**, **change**, full refund if the drink can't be made; (4) track inventory, deduct exactly the recipe; (5) **low-stock notifications**; (6) **refills**; (7) **customisation** without changing the menu.

Non-functional: thread-safe (no over-draw, no half recipe); data-driven; observable.

Use cases

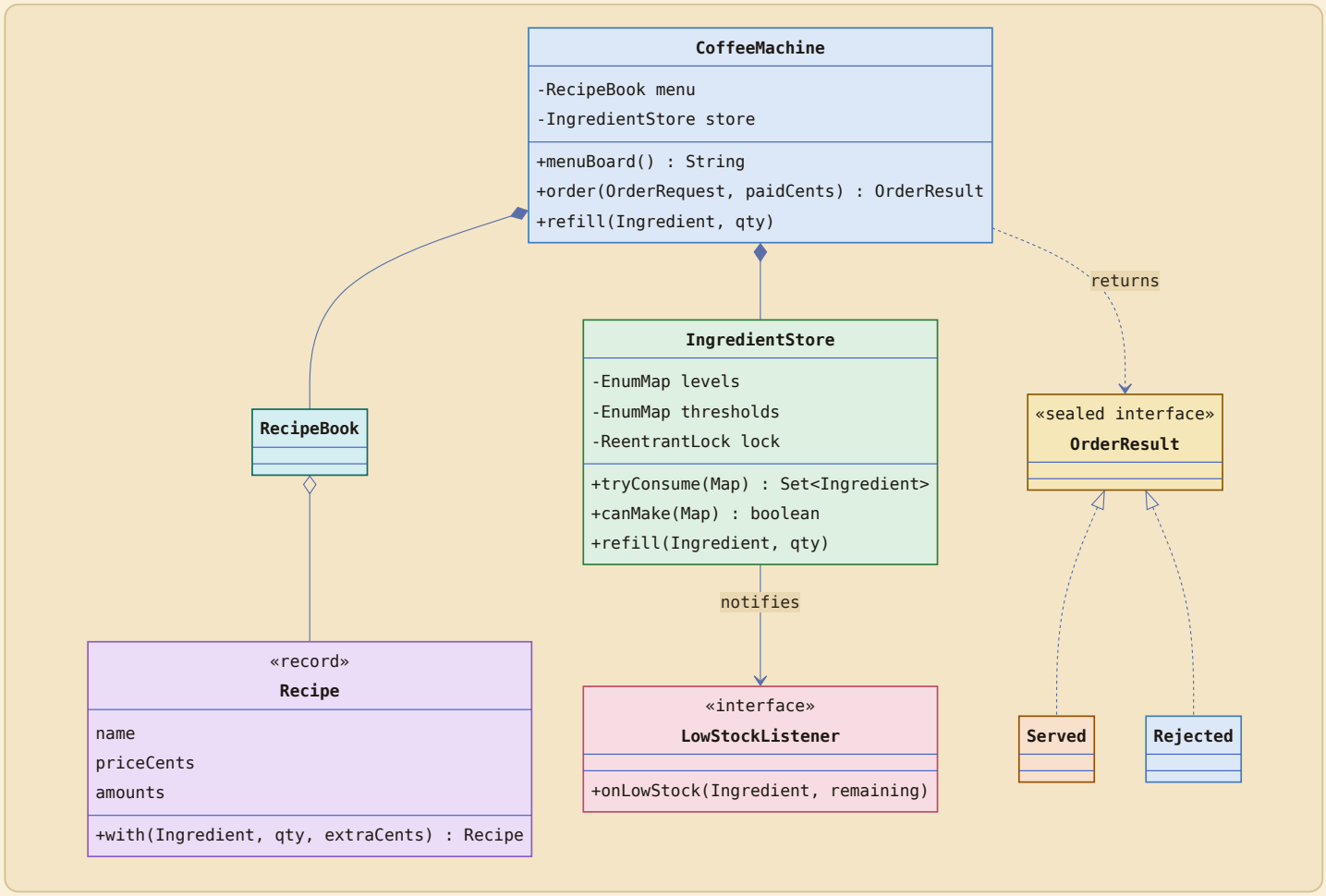
Actors: Customer (view menu, order, pay, take drink/change), Operator (refill, receive alerts), System (check stock, deduct, brew, alert). **Use cases:** view menu · order (with customisation) · pay / change / refund · refill · receive low-stock alert.

Core entities

Class	Responsibility
`Ingredient` (enum)	Beans, water, milk, chocolate, sugar — with unit (g/ml).
`Recipe` (record)	**Data**: name, price, `Map<Ingredient,Integer>`. Immutable; `with(...)` returns a customised copy.
`RecipeBook`	Parses menu text into recipes; lookup by name.
`IngredientStore`	Levels, thresholds, **one lock**; `tryConsume` = check all + deduct all atomically; alerts.
`LowStockListener`	Observer called once per threshold crossing.
`OrderRequest` (record)	Drink + extra shots + sugar grams.
`OrderResult` (sealed)	`Served(drink, price, change)` or `Rejected(reason, refund)`.
`CoffeeMachine`	Facade: menu board, `order(...)`, `refill(...)`.

Mapping to the common problem statement (*awesome-low-level-design*): `Coffee` → `Recipe`; `Ingredient` with `synchronized` updates → enum + levels in `IngredientStore`; `Payment` → `paidCents` and `OrderResult`; Singleton → injected machine. We lock the **whole recipe**, not each ingredient (per-ingredient locks can't make a multi-ingredient deduction all-or-nothing).

Class diagram



Key flows

Ordering: find recipe → apply customisation (`recipe.with(...)`) → if underpaid, `Rejected` with full refund → `tryConsume(amounts)`: lock, check **every** ingredient; if any shortage, nothing is deducted and the order is `Rejected`; else deduct all, note thresholds crossed, **unlock, then** notify listeners → `Served` with change.

Order life cycle: `Selected → Reserved (ingredients deducted atomically) → Brewing → Served`, or `Selected → Rejected` (unknown drink, underpaid, or short) with full refund.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Recipes are **data** from menu text	Data-driven, OCP	New drink = new line	`Latte extends Coffee` (class explosion)
Check-and-deduct under **one lock**	Atomicity (Ch. 12)	Stops two orders both seeing "enough milk" (check-then-act)	Separate check and deduct; per-ingredient locks
All-or-nothing	Transaction semantics	No half-made recipe	Deduct one by one and roll back
`LowStockListener`	**Observer**	Operator app, dashboard, telemetry subscribe	Store calling an SMS class
Alert **once per crossing**	—	No alert storm	Alert on every order below threshold
Listeners **outside** the lock	Concurrency hygiene	Slow listener can't block orders; no deadlock on callback	Notifying inside the lock
Customisation returns a **new** `Recipe`	Immutability	Menu recipe never corrupted	Mutating it; a Decorator chain (heavier)
Sealed `OrderResult`	Algebraic data type (Java 17+/21 switch)	Callers must handle both; exhaustive `switch`	`null` or exceptions for normal refusals
No Singleton	DI	Tests build fresh machines (demo builds two)	`getInstance()`

> **Decorator or data?** *Head First Design Patterns* models add-ons as Decorators (`new Whip(new Mocha(new DarkRoast()))`). Good when add-ons have **behaviour**. Here they only change **quantities and price**, so `recipe.with(...)` is simpler. Say both and pick one for a reason.

Java implementation

Package `lld.ch14.coffeemachine`.

```java
/** A recipe is DATA: a name, a price and the amount of each ingredient. */
record Recipe(String name, int priceCents, Map<Ingredient, Integer> amounts) {
    // ...
    /** Customisation returns a NEW recipe; the menu entry is never modified. */
    Recipe with(Ingredient extra, int qty, int extraCents) {
        if (qty == 0) return this;
        Map<Ingredient, Integer> m = new EnumMap<>(Ingredient.class);
        m.putAll(amounts);
        m.merge(extra, qty, Integer::sum);
        return new Recipe(name, priceCents + extraCents, m);
    }
}

final class IngredientStore {
    // ...
    /** Returns the ingredients that are short. Empty set = success, everything was deducted. */
    Set<Ingredient> tryConsume(Map<Ingredient, Integer> need) {
        Set<Ingredient> shortages = EnumSet.noneOf(Ingredient.class);
        List<Runnable> alerts = new ArrayList<>();
        lock.lock();
        try {
            need.forEach((i, q) -> { if (levels.get(i) < q) shortages.add(i); });
            if (!shortages.isEmpty()) return shortages;               // nothing touched
            need.forEach((i, q) -> levels.merge(i, -q, Integer::sum));
            for (Ingredient i : need.keySet()) {
                int left = levels.get(i);
                if (left <= thresholds.get(i) && alerted.add(i)) {    // alert once per crossing
                    alerts.add(() -> listeners.forEach(l -> l.onLowStock(i, left)));
                }
            }
        } finally {
            lock.unlock();
        }
        alerts.forEach(Runnable::run);   // call listeners OUTSIDE the lock
        return shortages;
    }
    // ...
}
```

Full runnable code: code/lld/ch14/coffeemachine/ (see the Full Edition).

How it works: the menu is a Java text block (`Latte, 400, COFFEE_BEANS=18, WATER=40, MILK=180`) parsed by `RecipeBook.parse`. `Recipe`'s compact constructor stores an unmodifiable `EnumMap`. `refill` clears the `alerted` flag when the level goes back above the threshold. `CoffeeMachine.order` validates drink → customises → checks payment → reserves ingredients.

Output (excerpt):

```
REJECTED out of [MILK], refund 400 c
   beans left 410g (unchanged by the rejected latte)
== 10 customers order a Latte at the same instant (milk for exactly 4) ==
   ALERT to operator: MILK low, 80ml left
served=4 rejected=6 milk left=80ml (never negative)
```

Concurrency & edge cases

Situation	Risk	Handling
Ten orders, milk for four	Negative stock (check-then-act)	Check + deduct under one lock
One ingredient short	Beans taken, milk missing	Shortages found before any deduction
Slow listener (SMS)	Orders wait on the network	Listeners after `unlock()`
Many orders below threshold	Alert storm	One alert per crossing; refill re-arms
Underpayment	Drink made, money short	Payment checked before reserving
Brew fails after deduction	Pays, gets nothing	Extension: refund, optionally refill the amounts (compensation)
Menu says available, order fails	Display/order race	Menu is advisory; `tryConsume` is the truth

One lock is right: the critical section is tiny and the machine brews one drink at a time. For a cloud inventory shared by many machines, use optimistic concurrency (version / CAS on a snapshot) or a DB transaction.

Extensions & follow-up questions

1. **New drink** → one menu line. 2. **Sizes** → `recipe.scaled(size)`. 3. **Several brewers** → a `BlockingQueue` consumed by N `Brewer` threads, one store. 4. **Remote monitoring** → another `LowStockListener`. 5. **Different brewing steps** → Template Method or `Step` objects (Command). 6. **Cash** → reuse 14.2's `CashBox`.

Real-world example

The Thermoplan Mastrena (Starbucks) stores **recipes as configuration**; Starbucks and Microsoft (2019, Azure Sphere) push recipes and collect telemetry from the cloud; connected kiosks report stock remotely — the low-stock Observer.

In the interview

40 minutes: 5 questions → 5 entities → 5 "recipes are data" and the menu format → 15 code `tryConsume` and `order` → 10 concurrency and extensions.

> **30-second answer:** "Recipes are data — ingredient-to-amount maps loaded from a menu, so a new drink is a new row. The store checks and deducts **every** ingredient of a recipe under one lock: all-or-nothing, never negative. Crossing a threshold is noted and observers are notified once, after the lock is released. Orders return a sealed Served/Rejected result; rejected orders are refunded in full."

- *"Why not an `AtomicInteger` per ingredient?"* — Each is safe alone, but a recipe updates several at once; without a common lock you get partial deductions.
- *"Why notify outside the lock?"* — A slow or re-entrant listener would block all orders or deadlock.

Self-check

1. **Q:** All-or-nothing for `tryConsume`? **A:** Every ingredient deducted, or none (shortages detected first).
2. **Q:** Which race does the lock prevent? **A:** Check-then-act: two threads see enough milk and both deduct.
3. **Q:** How is an alert storm prevented? **A:** An `EnumSet` of alerted ingredients, re-armed by refill.
4. **Q:** When use Decorator for add-ons? **A:** When they carry behaviour, not just quantities.

TL;DR

- **Recipes are data**; customisation makes a new immutable recipe.
- **One lock, check-then-deduct, all-or-nothing**.
- **Observer** alerts, **once per crossing**, **outside** the lock.
- **Sealed** `OrderResult`; full refund on rejection.

14.4 Design an ATM

An **ATM** is a cashier-less machine where a bank's customers do everyday banking outside branch hours. The interviewer checks that you keep **money correct** (never dispense without debiting, never debit without dispensing, never let two machines overdraw one account), **authenticate first**, and model the **session states** cleanly.

Problem & clarifying questions

Question	Assumed answer
Accounts / identification?	Checking and savings (daily cap); card + PIN, checked by the bank on every transaction.
Deposits available at once?	Checks **not** — they wait. Keep **total** and **available** balances.
Notes?	$100, $50, $20, $10 cassettes.
Wrong PIN?	3 wrong attempts block the card and the ATM keeps it (course leaves this out; we add it plus a journal).
Customers at a time?	One; the ATM must not shut down mid-session.

Requirements

Parts: card reader, keypad, screen, cash dispenser, deposit slot (cash/check), printer, network link to the bank.

Functional: (1) card + PIN authenticated **by the bank** first; (2) five transactions: balance inquiry, cash deposit, check deposit, withdraw, transfer; (3) the ATM **keeps the card** all session; cancel ejects it; (4) a **journal** of transactions and hardware faults; (5) a **check is held** until verified (the course holds all deposits; we credit counted cash at once); (6) **bank manager** reports.

Actors' operations: Operator — on/off key switch, refill cash, paper, ink, empty deposits. Customer — the transactions. Bank manager — total deposits/withdrawals reports, print them, check remaining cash.

Non-functional: security (no clear PINs on the ATM; limited attempts); consistency (debit and dispense together or compensate); concurrency (many ATMs, one account, no overdraft, no deadlock); auditability.

Use cases

Only Customer use cases need the PIN (Operator uses a key; Manager reads bank data). The course's activity diagrams (authenticate, withdraw, deposit check, transfer) all end by ejecting the card — **every** exit path, including declined retries, must return it.

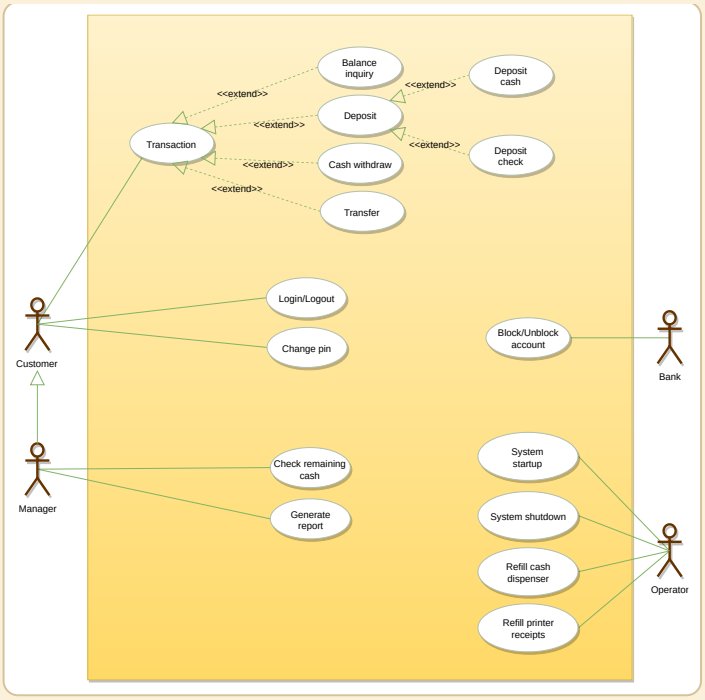


Figure — ATM use case diagram: Customer (balance, withdraw, deposit, transfer), Operator (on/off, refill cash, paper, ink, empty deposits) and Bank Manager (reports, check remaining cash) (course diagram)

Core entities

Class	Responsibility
`Card` (record)	Number and holder (course also has `Customer` with one card).
`Account` (abstract)	Id, **total** and **available** balance, a lock, withdrawal-rule hook.
`CheckingAccount`, `SavingsAccount`	Savings adds a **daily cap** — a behaviour difference, hence subclasses.
`BankService` (interface)	Authenticate, balances, withdraw, reverse, deposits, transfer.
`InMemoryBank`	Stand-in bank: PIN hashes, failed attempts, account locks, ordered-lock transfers.
`AuthResult` (sealed)	`Ok(accounts)`, `WrongPin(attemptsLeft)`, `Blocked`.
`NoteHandler` + 4 subclasses	**Chain of Responsibility** $100 → $50 → $20 → $10.
`CashDispenser`	Plans (dry run), then dispenses; can jam (fault injection).
`Transaction` (sealed) + 5 records	**Command**: one `execute(ctx)` each.
`AtmState` + 4 states	**State**: Idle → CardInserted → Authenticated, plus OutOfService.
`Atm`	Id; owns parts, session, journal; forwards buttons to state. Course hardware (`CardReader`, `Keypad`, `Screen`, `Printer`, `DepositSlot`) is folded in.

Class diagram

Course stages: (1) **money and card** — two balances (a check raises *total* now, *available* after clearing); savings overrides one method (behaviour ⇒ inheritance); (2) **the parts** — none knows what a transaction is; whether $85 can be made from loaded notes is the **dispenser's** job; the reader **keeps the card** all session; `DepositSlot` (the opening) ≠ the deposit *transaction*; (3) **five transactions**, one `execute` each; the course's `Withdraw` checks **denominations → account → dispenser** and debits only after notes are counted; (4) **the machine** owns its parts and holds the session (one customer at a time); the `Bank` owns its ATMs.

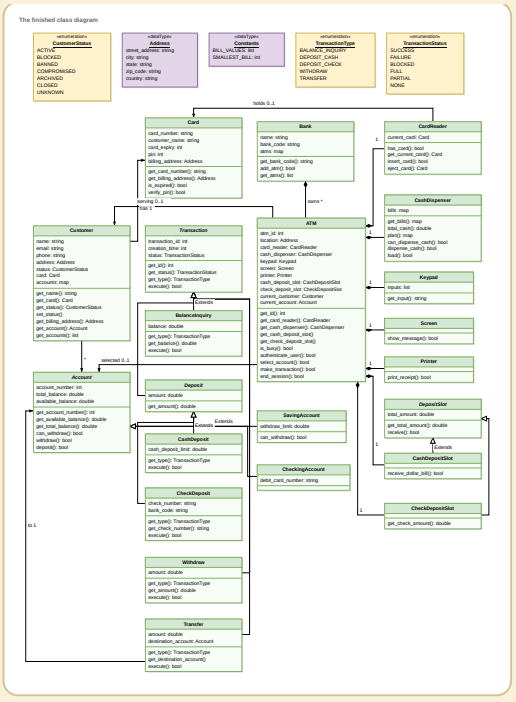
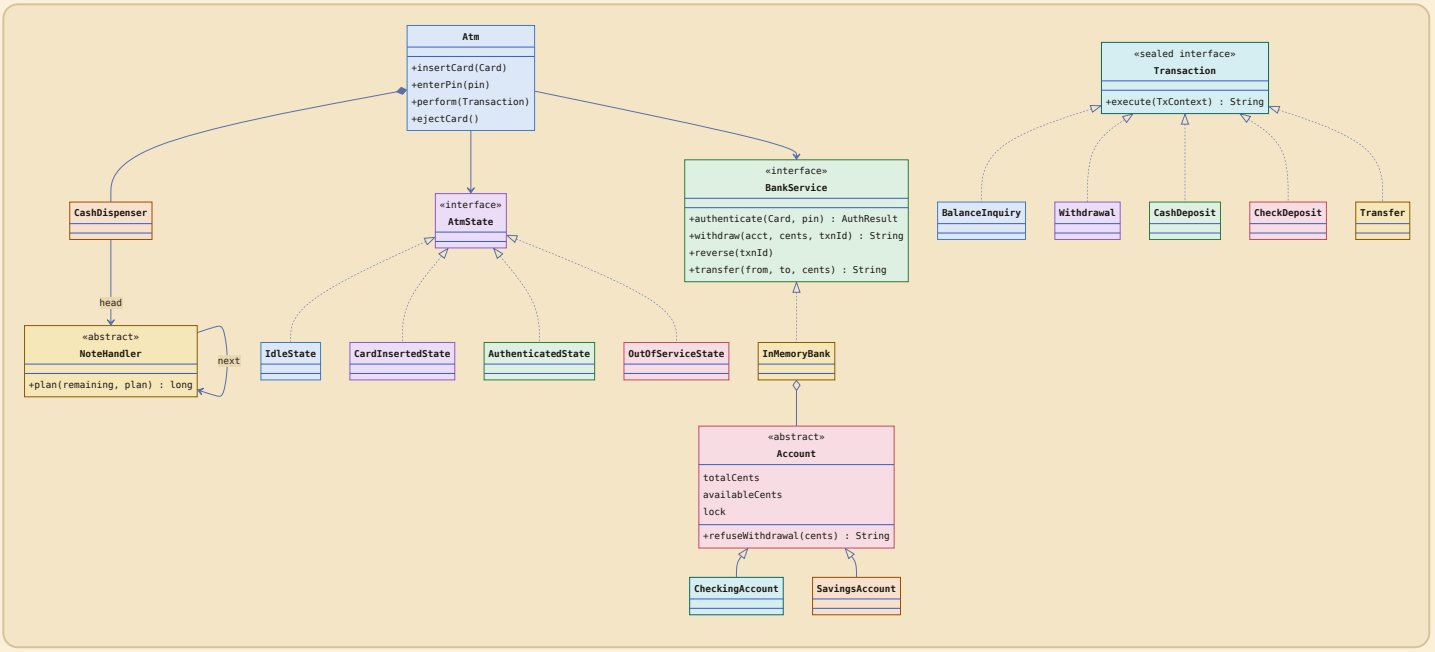


Figure — The finished class diagram of the ATM system (course diagram)



Key flows

Session states: Idle → CardInserted → Authenticated (PIN OK) → Idle (finish/cancel, card ejected). **3rd wrong PIN → Idle, card retained.** Session ends with no cash → OutOfService (card out first); key switch only when Idle; refill → Idle. **Balance inquiry:** the ATM creates a transaction, which asks the account; the result goes to the screen.

Withdrawal with compensation (our code): multiple of $10? → `plan(280)` dry run on the note chain (2×100, 1×50, 1×20, 1×10) → bank `withdraw` (lock account, check rule, debit) → `dispense(plan)` → on **jam**, `reverse(txnId)` and report "account NOT charged" → journal the result.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Session as state classes	**State**	Each button means something different per state	Booleans + nested `if`s
Dispenser as chain of note handlers	**Chain of Responsibility**	Each takes what it can, passes the rest; a $5 cassette is one new handler	Hard-coded note list
Plan first, then dispense	—	Dry run touches nothing	Removing notes while searching
Each transaction = one class	**Command**	Rules in one place; can be journaled, queued, replayed	`switch (type)` in the ATM
`Transaction`, `AuthResult` sealed	Sealed types	Exhaustive `switch`	`instanceof` chains
ATM → `BankService` interface	Dependency Inversion	In-memory, ISO 8583, or a test fake	ATM changes balances
Total vs available	Domain modelling	Check holds correct	Single balance
Savings subclass	Inheritance for **behaviour**	Overrides one hook	`if (type == SAVINGS)`

> **Debit before or after dispensing?** The course debits **after** counting notes (safer for the customer) and admits a crash in between leaves the bank short. Our code debits then dispenses, with a **compensating** `reverse(txnId)` on a jam. Real systems make the pair atomic (**hold/authorise, dispense, confirm**) and **reconcile** from the journal. This is the "power fails halfway" follow-up.

> **Cash deposits.** Many banks make machine-counted cash available immediately and hold checks. `depositCash` credits both balances, `depositCheck` only the total — a one-line policy change in the bank, not the ATM.

Java implementation

Package `lld.ch14.atm`. Amounts are cents (`long`).

```
abstract class NoteHandler {
    // ...
    /** Dry run: fills 'plan', returns the amount that nobody in the chain could cover. */
    long plan(long remaining, Map<Long, Integer> plan) {
        int use = (int) Math.min(count, remaining / noteCents);
        if (use > 0) plan.put(noteCents, use);
        long left = remaining - use * noteCents;
        return (left == 0 || next == null) ? left : next.plan(left, plan);
    }
    // ...
}

record Withdrawal(String account, long cents) implements Transaction {
    static final long SMALLEST_NOTE = 10_00;

    public String execute(TxContext c) {
        if (cents <= 0 || cents % SMALLEST_NOTE != 0) return "Withdrawal refused: amount must be a multiple of $10";
        var plan = c.dispenser().plan(cents);                                  // 1. can the notes make it?
        if (plan.isEmpty()) return "Withdrawal refused: ATM cannot make " + Money.fmt(cents) + " from its notes";
        String refusal = c.bank().withdraw(account, cents, c.txnId());         // 2. atomic debit at the bank
        if (refusal != null) return "Withdrawal refused: " + refusal;
        try {
            c.dispenser().dispense(plan.get());                                // 3. hand out the cash
        } catch (IllegalStateException hardwareFault) {
            c.bank().reverse(c.txnId());                                       // compensate: undo the debit
            return "Withdrawal failed (" + hardwareFault.getMessage() + "), account NOT charged";
        }
        return "Dispensed " + Money.fmt(cents) + " as " + describe(plan.get());
    }
    // ...
}

final class InMemoryBank implements BankService {
    // ...
    /** Locks both accounts in a fixed (id) order, so two opposite transfers cannot deadlock. */
    @Override public String transfer(String from, String to, long cents) {
        if (cents <= 0) return "amount must be positive";
        if (from.equals(to)) return "same account";
        Account a = acc(from), b = acc(to);
        Account first = from.compareTo(to) < 0 ? a : b;
        Account second = first == a ? b : a;
        first.lock.lock();
        try {
            second.lock.lock();
            try {
                if (cents > a.availableCents) return "insufficient funds";
                a.availableCents -= cents;
                a.totalCents -= cents;
                b.availableCents += cents;
                b.totalCents += cents;
                return null;
            } finally {
                second.lock.unlock();
            }
        } finally {
            first.lock.unlock();
        }
    }
    // ...
}
```

Full runnable code: code/lld/ch14/atm/ (see the Full Edition).

How it works: the bank stores a SHA-256 hash of card+PIN and counts failures; the 3rd returns `Blocked` and the ATM **retains** the card (demo only — real banks verify an encrypted PIN block in an HSM). `withdraw` checks and debits under the account's `ReentrantLock`. `finishSession` picks Idle or OutOfService only after the card is out.

Output (excerpt):

```
   [ATM-7 screen] Dispensed $280.00 as 2x$100 + 1x$50 + 1x$20 + 1x$10
   [ATM-7 screen] Balance CHK-1: available $720.00, total $1220.00
   [ATM-7 screen] Withdrawal failed (note jam), account NOT charged
== Two ATMs, one account, same instant: $300 each from $500 ==
   cash handed out $300.00, JOINT left $200.00
== 2,000 opposite transfers in parallel (ordered locking: no deadlock) ==
   successful=2000, A+B=$2000.00
```

Concurrency & edge cases

Situation	Risk	Handling
Two ATMs, one account	Overdraft (check-then-act)	Check + debit under the **account lock**
A→B and B→A at once	**Deadlock**	Lock in a global order (account id)
Jam after debit	Charged, no cash	`reverse(txnId)` + journal
Amount not makeable	Partial dispense	Dry-run `plan` before touching the bank
Greedy chain misses a combination	$60 with one $50, three $20, no $10	OK for standard cassettes; DP planner (14.2) if asked
Many wrong PINs	Brute force	3 tries, block **at the bank**, retain card
Walks away / unlinked account	Misuse	Session timeout (extension) / `sessionAccounts` check
Cash runs out	Card trapped	Finish session, then `OutOfService`; refill → Idle
Key switch mid-session	Shuts down while serving	Refused unless idle

Extensions & follow-up questions

1. **Can't make the amount** → plan fails, refused before the bank is called.
2. **Power fails mid-withdrawal** → hold at the bank, dispense, confirm; on restart read the journal and confirm or reverse; bank reconciles.
3. **Daily limit for all accounts** → in `Account` or a policy object; ATM unchanged.
4. **Other banks' cards** → a routing `BankService` (Proxy/Adapter to the card network).
5. **Cardless** → a new state path `enterOneTimeCode`; transactions unchanged.
6. **Manager reports** → aggregate the journal; cash left = `dispenser.total()`. (Receipts / PIN change → more `Transaction` records.)

Real-world example

ATMs (NCR, Diebold Nixdorf) drive devices through **CEN/XFS** interfaces, talk **ISO 8583**, and use **ISO 9564** PIN blocks verified in HSMs; after a dispense error they reverse and reconcile from the journal.

In the interview

45 minutes: 5 parts, actors, transactions → 5 total vs available → 10 state diagram → 15 code `Withdrawal.execute` and the chain → 10 concurrency and follow-ups.

> **30-second answer:** "Parts behind interfaces, the session as a State machine, five transactions as Commands with one `execute`. The ATM never touches balances; it calls a `BankService`. Accounts have total and available balances so checks can be held. Withdrawal plans notes through a Chain of Responsibility, debits atomically at the bank, dispenses, and reverses on hardware failure. The bank locks per account; transfers lock in id order to avoid deadlock."

- *"Where is the PIN checked?"* — At the bank (really an HSM, from an encrypted PIN block).
- *"Two ATMs, joint account?"* — Check and debit together under the account's lock (or one DB transaction).

Self-check

1. **Q:** Why two balances? **A:** A check raises the total at once but must not be spendable until it clears.

2. **Q:** Safe order of debit and dispense? **A:** Neither alone; use hold + confirm (or debit + compensating reverse) and reconcile from the journal.
3. **Q:** How are transfer deadlocks avoided? **A:** Always lock both accounts in the same global (id) order.
4. **Q:** What happens after the 3rd wrong PIN? **A:** The bank blocks the card (every ATM refuses it) and this ATM retains it.

TL;DR

- **State** (session), **Command** (transactions), **Chain of Responsibility** (cassettes), `BankService` interface.
- **Total vs available** balance.
- Withdrawal = **plan → atomic debit → dispense → reverse on failure**, journaled.
- **Per-account lock**, transfers in **id order**, PIN attempts limited at the bank.

14.5 Design an Elevator System

A building's cars are run as a **group** by a shared controller. Two decisions carry the interview: **which car answers a hall call** (dispatching) and **in what order one car visits its stops** (scheduling).

Problem & clarifying questions

Question	Assumed answer
Floors and cars?	Floors 0 … N-1, several cars serving all.
Buttons?	Hall UP/DOWN per floor (no UP on top, no DOWN on ground); car panel per floor plus door open/close.
Displays?	In car: floor, direction, load. Per floor: one display per car.
Scheduling?	Serve every stop ahead before turning — **LOOK**.
Dispatching?	Nearest suitable car; swappable.
Capacity?	Overloaded car doesn't move and says so.
Maintenance, emergencies?	Out-of-service cars; fire alarm recalls all to the lobby.
Real time?	Simulated: one `step()` = one action (testable without a clock).

Requirements

Functional: (1) many floors and cars; (2–3) hall and car panels; (4–5) car and floor displays; (6) a hall call **picks one car**; (7) a car button **adds a stop**; (8) **serve stops ahead** before turning; (9) **never move with doors open**; (10) **max load** — overloaded car stays and says so; (11) idle car waits where it finished; (12) **out of service** cars get no riders; (13) *(ours)* **fire recall**.

Non-functional: efficient (no useless trips to the shaft end); no starvation; safety overrides scheduling; thread-safe; deterministic stepping.

Use cases

Actors: Rider (call, ride, choose destination), Maintenance engineer (out of / back in service), System (choose car, move, doors, displays). **Use cases:** call · choose destination · dispatch · move one floor · open/close doors · report overload · take out of service. Dispatch and move belong to the System; the rider only triggers them.

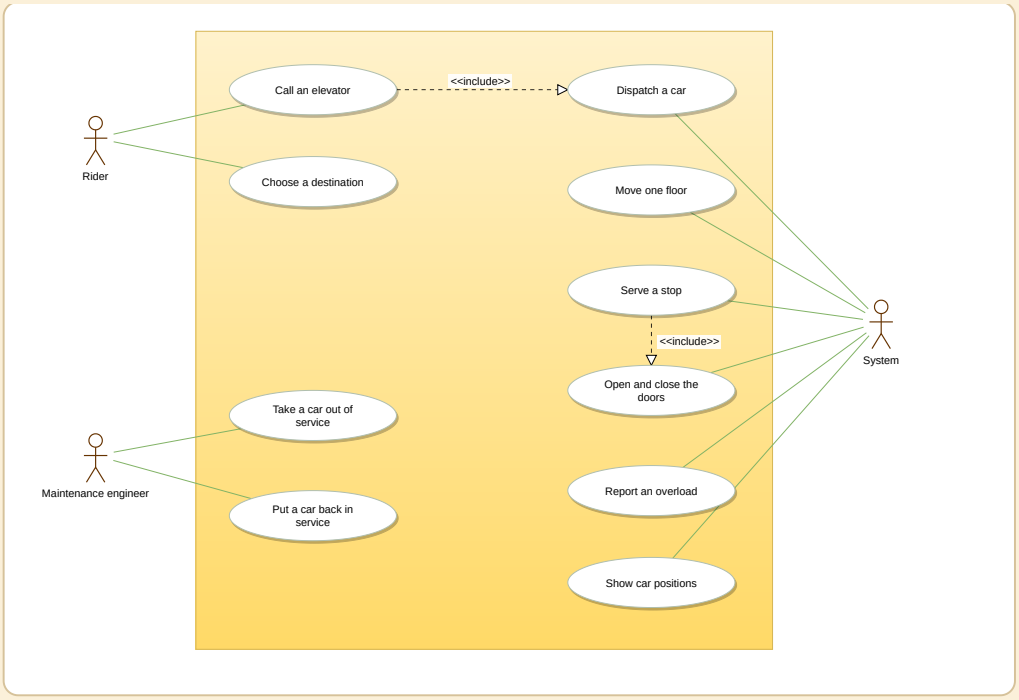


Figure — Use case diagram of the elevator system: Rider, Maintenance engineer and System (course diagram)

Core entities

Class	Responsibility
`Direction` (enum)	UP, DOWN, IDLE.
`Stop` (record)	Floor **plus** direction on leaving — "5 going up" and "5 going down" are two stops.
`StopList`	Stops in **two sorted sets** (up, down); `next(...)` implements LOOK.
`Door`	Open or closed.
`Elevator`	Floor, direction, mode, load, door, stops; `step()` does one action.
`Mode` (enum)	NORMAL, MAINTENANCE, FIRE_RECALL.
`Event` (record)	What a step did (MOVED, ARRIVED, DOORS_CLOSED, OVERLOAD, NOTHING).
`DispatchStrategy` / `NearestCarStrategy`	*Which car* answers a hall call.
`Dispatcher`	Cars + one strategy; queues calls no car can take.
`Building`	Hall lights, cars, dispatcher; ticks cars; clears lights on arrival; fire alarm.

The course also models `Button`, `HallPanel`, `ElevatorPanel`, `Display` and `Floor` one-to-one; our code folds them — say which you chose.

Class diagram

Course stages: (1) `Button` family (shared light/reset behaviour) and `Door`; (2) panels, displays, `Floor` — all **composition**; (3) **most important:** `Stop` **and** `StopList` — two sets, not one list; the next stop is reachable **without reversing**, and with nothing ahead the car heads to the **farthest** request; `Elevator` **moves**, `StopList` **chooses**; (4) `Elevator` — `step` does **one** of: close doors, refuse (overload), move one floor, stop and open; it never touches a `Floor` or another car; (5) `DispatchStrategy`, `Dispatcher`, `Building` — the only class knowing cars and floors, so it clears hall lights.

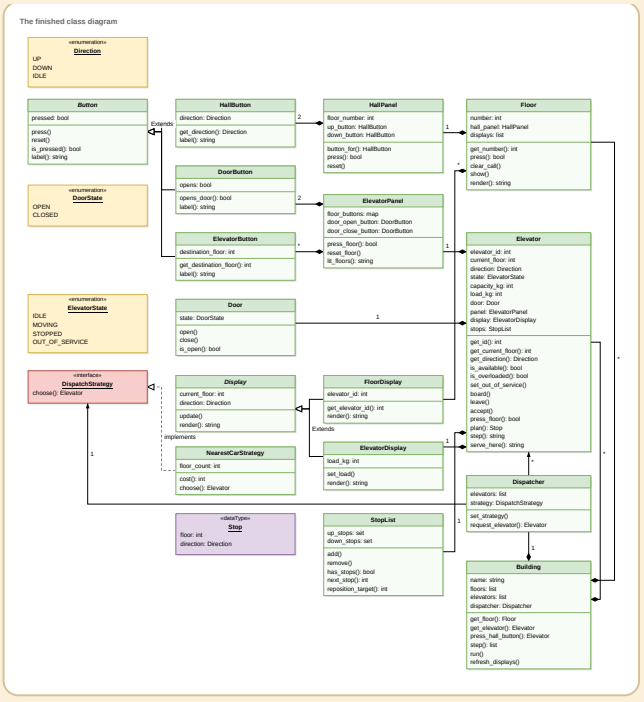
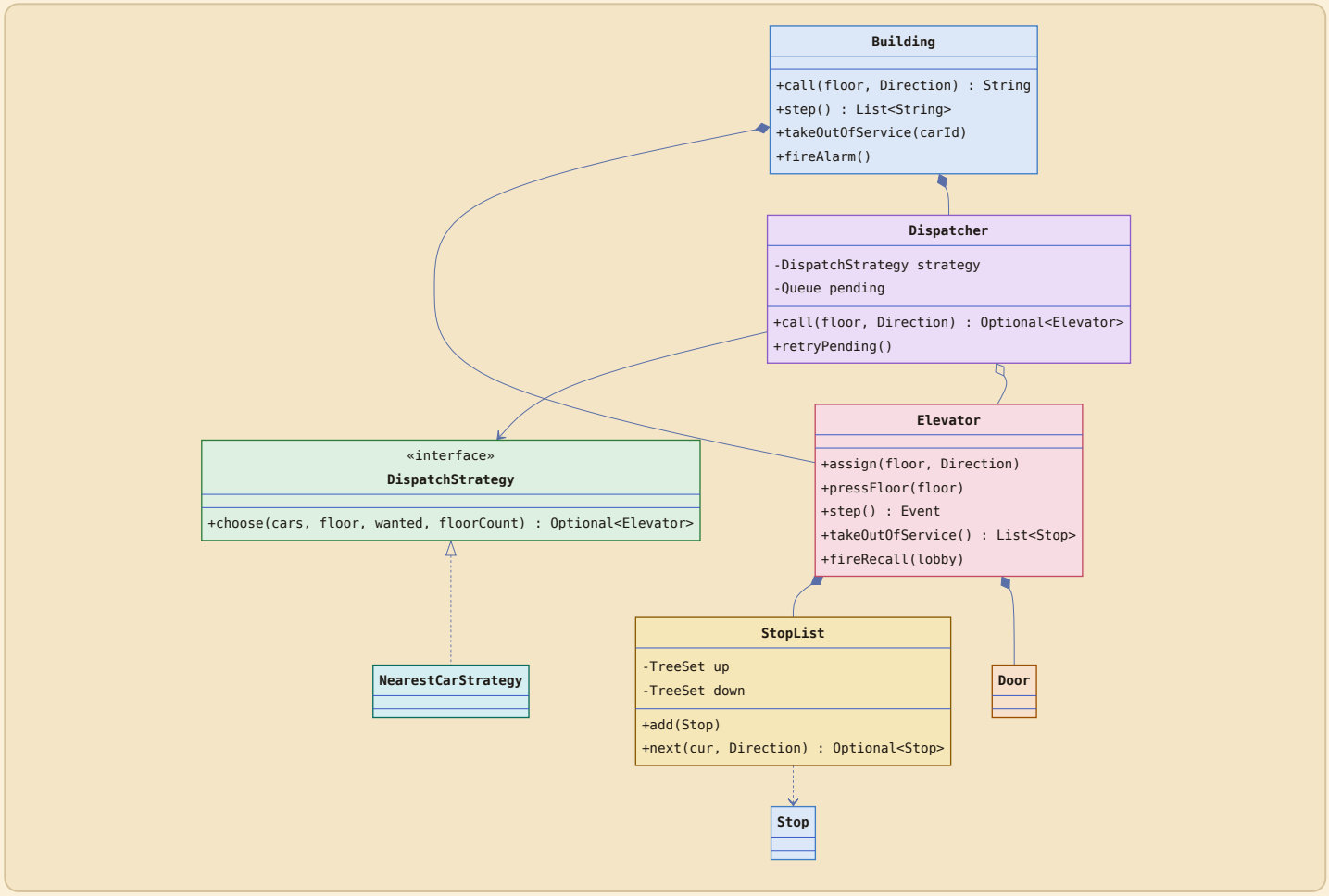


Figure — The finished class diagram of the elevator system (course diagram)



Key flows

Scheduling algorithms compared (borrowed from disk-arm scheduling — same shape: a head moving along a line of requests):

Algorithm	Rule	Good	Bad
FCFS	Arrival order	Simple	Zig-zags; very inefficient
SSTF	Nearest stop first	Short trips	**Starvation** of far floors
SCAN	Sweep to the **end of the shaft**, reverse	No starvation	Wasted travel to the ends
LOOK	Sweep only **to the last request**, reverse	No starvation, no waste	Slightly more bookkeeping
C-SCAN / C-LOOK	One direction; jump back to start	Uniform waits	Empty return trips

Our `StopList.next` (LOOK, direction-aware), going UP: (1) nearest up-stop at or above (`up.ceiling(cur)`); (2) else, if someone *above* wants DOWN, go to the **highest** such floor, then turn; (3) else turn: nearest down-stop at or below (`down.floor(cur)`); (4) else go down to the **lowest** up-stop and serve upward. DOWN mirrors it; an IDLE car picks the side of the nearest stop.

Car states: Idle → Moving → DoorsOpen (overload: stay) → Moving/Idle; Idle ⇄ Maintenance; any → FireRecall. **Hall call:** light button → strategy picks a car → `assign` → `step()` each tick → on ARRIVED, clear the light.

Dispatch cost (`NearestCarStrategy`, from the course): idle car = distance; car already coming this way **in the same direction** = distance; any other car = distance **+ two building heights**. Out-of-service cars are skipped.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`DispatchStrategy`	**Strategy**	Office rush vs quiet hotel; the *requirement* to swap justifies the interface	`if (officeMode)`
Sweep rule in `StopList`	SRP	Car drives, stop list decides	Next-stop logic in `step()`
Two sets (up/down)	Data modelling	"5 up" ≠ "5 down"	One list
`step()` = one action	Deterministic simulation	Tests = calls + expected events, no clock	Thread per car with `sleep`
Modes as an enum	Lightweight State	Only 3 modes; switch to State classes if modes grow	State classes from day one
Re-dispatch when a car leaves service	Robustness	Accepted calls never lost	Silently clearing stops
Queue calls when no car is free	—	"Not now" ≠ "rejected"	Dropping the call

Course's honest weak points: (1) **no threads** — a real controller locks the stop list (ours is `synchronized`); (2) the dispatcher **decides once**; (3) `Elevator` nearly does two jobs, hence `StopList`. Only **one** pattern (Strategy); out of scope: emergency calls, fire service, express cars, door sensors.

Java implementation

Package `lld.ch14.elevator`.

```java
final class StopList {
    private final TreeSet<Integer> up = new TreeSet<>();
    private final TreeSet<Integer> down = new TreeSet<>();
    // ...
    Optional<Stop> next(int cur, Direction dir) {
        if (isEmpty()) return Optional.empty();
        if (dir == Direction.IDLE) dir = nearestSide(cur);
        return Optional.of(dir == Direction.UP ? goingUp(cur) : goingDown(cur));
    }

    private Stop goingUp(int cur) {
        Integer f = up.ceiling(cur);
        if (f != null) return new Stop(f, Direction.UP);                          // work ahead
        if (!down.isEmpty() && down.last() > cur) return new Stop(down.last(), Direction.DOWN); // go up, then turn
        f = down.floor(cur);
        if (f != null) return new Stop(f, Direction.DOWN);                        // turn around now
        return new Stop(up.first(), Direction.UP);                                // go down to lowest up-call
    }
    // ... goingDown is the mirror image
}

final class Elevator {
    // ...
    synchronized Event step() {
        if (mode == Mode.MAINTENANCE) return nothing();   // parked: never moves
        if (door.isOpen()) {
            if (mode == Mode.FIRE_RECALL && stops.isEmpty()) return nothing(); // parked at lobby, doors open
            if (loadKg > capacityKg) {
                return new Event(Event.Kind.OVERLOAD, id, floor, direction,
                        "OVERLOAD " + loadKg + "kg > " + capacityKg + "kg, doors stay open");
            }
            door.close();
            return new Event(Event.Kind.DOORS_CLOSED, id, floor, direction, "doors close");
        }
        var next = stops.next(floor, direction);
        if (next.isEmpty()) {
            direction = Direction.IDLE;
            return nothing();
        }
        Stop target = next.get();
        if (target.floor() == floor) {                      // arrived
            stops.remove(target);
            direction = stops.isEmpty() ? Direction.IDLE : target.direction();
            door.open();
            return new Event(Event.Kind.ARRIVED, id, floor, target.direction(), "doors open");
        }
        direction = target.floor() > floor ? Direction.UP : Direction.DOWN;   // never moves with doors open
        floor += direction == Direction.UP ? 1 : -1;
        return new Event(Event.Kind.MOVED, id, floor, direction, "");
    }
    // ...
}
```

Full runnable code: code/lld/ch14/elevator/ (see the Full Edition).

How it works: `TreeSet` `ceiling`/`floor`/`first`/`last` give O(log n) "nearest stop" answers. `takeOutOfService` returns the car's stops for re-dispatch; `Dispatcher.retryPending` runs every tick.

Output (excerpt):

```
== 1. LOOK on one car: at floor 4 going up; stops 6(up) 9(car) 2(down) 1(up) ==
service order [6, 9, 2, 1]  (turns at 9 and at 1: never runs to 0 or to a floor above 9)
  t15: car2@9 OVERLOAD 775kg > 680kg, doors stay open
re-dispatch hall 4 DOWN -> car2
hall 2 UP -> queued (no car available)
```

Concurrency & edge cases

Situation	Handling
Buttons pressed while the controller steps	`synchronized` car and dispatcher methods; concurrent set of hall lights
Same hall button twice	`litHallButtons.add` false: "already lit", no second dispatch
Up and down on the same floor	Two `Stop`s in two sets
Car call for current floor, doors open	Ignored
Overload	Doors stay open, OVERLOAD shown, no movement
Car out of service with accepted calls	Re-dispatched or queued, never lost
No car available	Queued; retried every tick
Fire alarm	Stops dropped, lobby added, car buttons ignored, doors open at lobby
Starvation	LOOK serves every stop within one round trip
UP on the top floor	`IllegalArgumentException`

Extensions & follow-up questions

1. **Choosing a car** → `DispatchStrategy`: zoning, **up-peak** (idle cars to the lobby at 9 a.m.), energy saving.
2. **Simultaneous presses** → synchronized dispatcher + per-car lock (the course omits it).
3. **Testing** → presses, N steps, expected events; no sleeps.
4. **Destination dispatch** → a strategy taking (from, to), grouping nearby destinations.
5. **Re-evaluate / real time** → move unserved calls to a better car each tick; a `ScheduledExecutorService` calls `step()` every second.
6. **Emergency stop / more modes** → refactor modes into State classes.
7. **Express cars / door sensor** → served-floor sets the strategy checks; an obstruction makes `step()` reopen (same slot as overload).

Real-world example

Destination dispatch (Schindler PORT, Otis CompassPlus, KONE) = extension 4; controllers switch to **up-peak** mode; fire recall follows **Phase I emergency recall** (ASME A17.1).

In the interview

45 minutes: 5 requirements/scope → 5 entities → **15 stop list and LOOK** → 10 `step()` and dispatcher → 10 concurrency, testing, follow-ups.

> **30-second answer:** "Each car keeps stops in two sorted sets, up and down, and uses LOOK: serve stops ahead, turn at the last request, never run to the shaft end for nothing. `step()` does one action — close doors, refuse if overloaded, move one floor, or stop and open — so the system is testable without a clock. A dispatcher assigns hall calls via a `DispatchStrategy` (nearest suitable car) and queues calls when no car is free. Doors, load, maintenance and fire recall override scheduling."

- *"Why LOOK, not SSTF?"* — SSTF can starve a far floor; LOOK serves everything within one round trip.
- *"Where's the lock?"* — Per car (its monitor) and around dispatch; no global lock across cars.

Self-check

1. **Q:** SCAN vs LOOK? **A:** SCAN runs to the shaft end; LOOK reverses at the last request.
2. **Q:** Car at 4 going up; stops 6(up), 9(car), 2(down), 1(up) — order? **A:** 6, 9, 2, 1.
3. **Q:** Why is scheduling in `StopList`? **A:** SRP: the car drives, the stop list decides; each testable alone.
4. **Q:** Every car in maintenance — what happens to a call? **A:** Queued, retried every tick.
5. **Q:** When would you move modes to State classes? **A:** When many modes behave differently per action.

TL;DR

- **Two sorted sets** + **LOOK** = efficient, starvation-free scheduling.
- `step()` **= one action**; doors, load, maintenance, fire recall override scheduling.
- **Strategy** for dispatch, pending queue, re-dispatch when a car leaves service.
- Per-car lock, synchronized dispatcher, concurrent hall-light set.

14.6 Design a Traffic Signal Control System

A **traffic signal controller** decides which **approaches** (roads entering: N, S, E, W) get green, for how long, and in what order. A **phase** is a set of approaches that may be green together because their paths do not cross (e.g. N and S straight). The controller cycles phases with **yellow** (warning) and **all-red** (clearance) between them. It tests a **timed state machine** with a **hard safety rule** — *two crossing flows are never green together* — plus an **emergency override** that interrupts the cycle *safely* (never cutting a yellow short).

Problem & clarifying questions

Question	Assumed answer
Roads?	Four-way (N, S, E, W); phases configurable.
Signals?	Red, yellow, green; flashing red in fault mode.
Timing?	Fixed, or adaptive from queue sensors.
Pedestrians, turn arrows?	Out of scope (extensions).
Emergency vehicles?	Green as soon as *safely* possible, held until they pass.
Many intersections?	One controller each; central coordination is an extension.
Real time?	1 tick = 1 s; tick-driven so it can be simulated.

Requirements

Functional: (1) multiple roads; (2) red/yellow/green; (3) **configurable, traffic-adaptive** durations; (4) **safe transitions**: green → yellow → all-red → next green; (5) **emergency** priority; (6) scalable/extensible; (7) fault → **all flashing red** (all-way stop).

Non-functional: safety invariant (never conflicting greens, not even for an instant); deterministic 1 s ticks; thread safety (timer vs sensor and emergency threads); observability (listeners for light changes).

Use cases

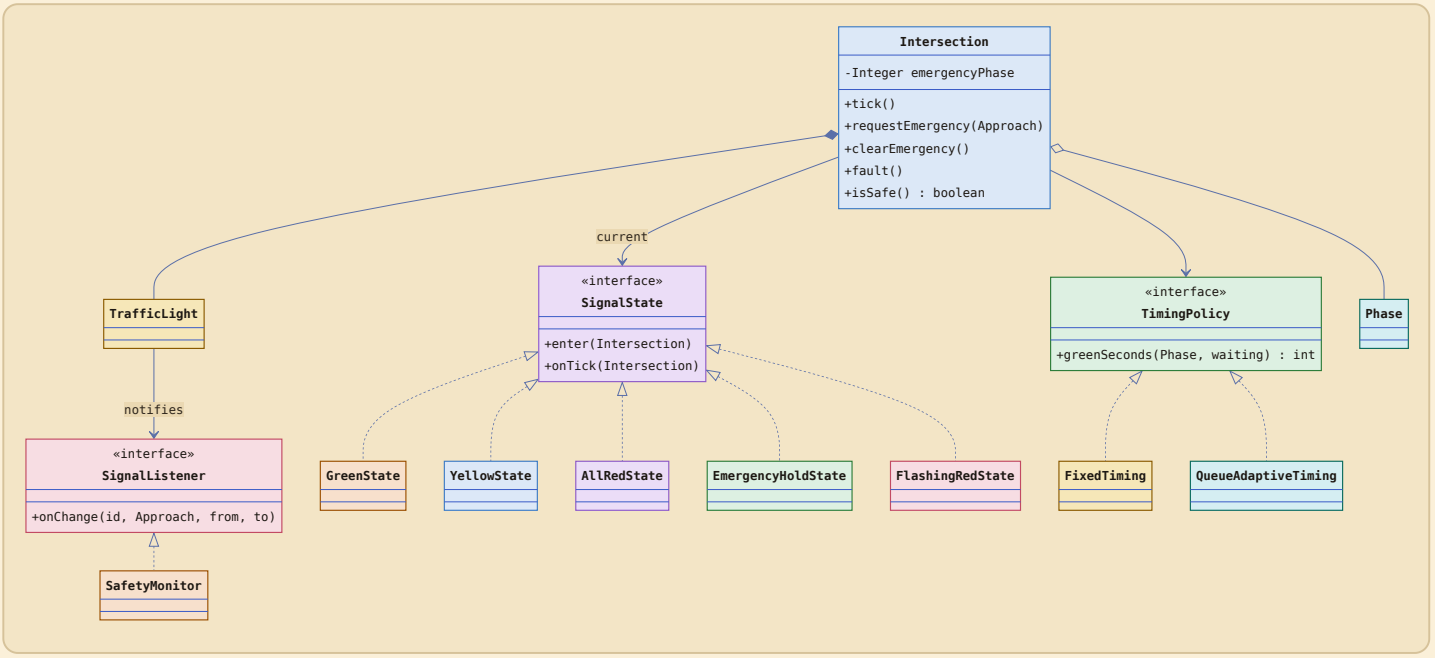
Actors → use cases: Timer → advance the cycle; Vehicle sensor → report waiting cars; Emergency vehicle → request/clear pre-emption; Traffic engineer → configure phases/timing, fault mode; Drivers see the lights.

Core entities

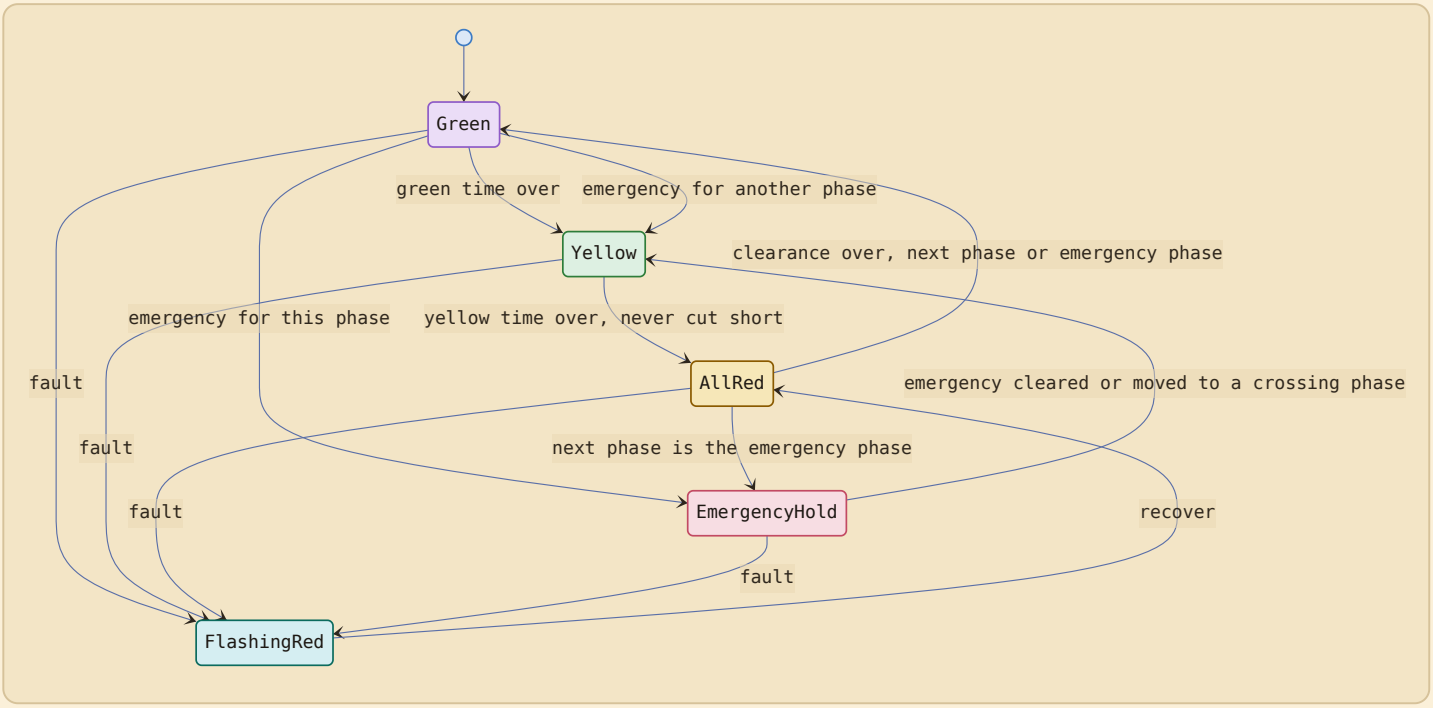
Class	Responsibility
`Signal` (enum)	RED, YELLOW, GREEN, FLASHING_RED.
`Approach` (enum)	NORTH, SOUTH, EAST, WEST.
`Phase` (record)	Name + set of approaches green together.
`TrafficLight`	One head for one approach; notifies listeners.
`SignalListener`	**Observer** for light changes.
`SafetyMonitor`	Listener checking the invariant after every change to green.
`TimingPolicy`	**Strategy**: `FixedTiming`, `QueueAdaptiveTiming`.
`SignalState` + 5 states	**State**: Green, Yellow, AllRed, EmergencyHold, FlashingRed.
`Intersection`	State context: phases, lights, timing, sensors, emergency flag; `tick()`.

Mapping to the common problem statement (*awesome-low-level-design*): `Road` → `Approach` + its `TrafficLight`; durations move into `TimingPolicy`; Singleton `TrafficController` → our `Intersection` (**not** a Singleton). We add what the reference leaves implicit: phases, all-red and the safety check.

Class diagram



Key flows



Emergency pre-emption: ambulance from WEST → `requestEmergency(WEST)` sets `emergencyPhase = East-West` → next tick N-S green ends **early** into yellow → yellow runs in full → all-red → `EmergencyHold(East-West)` → `clearEmergency()` → yellow, all-red, normal cycle.

One fixed cycle (green 5, yellow 2, all-red 1): N-S green 5 → N-S yellow 2 → all red 1 → E-W green 5 → E-W yellow 2 → all red 1 — **16 s**.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Each interval a state class (`enter`, `onTick`)	**State**	Transitions and "yellow never cut short" live in one small class each	One `switch` with counters
States **carry data** (remaining seconds), new per transition	State with per-instance data	Unlike vending's stateless states, they hold a countdown	Shared singletons with counters on the context
Green time from `TimingPolicy`	**Strategy**	Fixed, adaptive, time-of-day swap at run time	Hard-coded in `GreenState`
Light changes published	**Observer**	Displays, logs, central monitoring, safety monitor	Calling each consumer
`showPhaseGreen`: others **red first**, then green	Safety by construction	Never an instant of conflicting greens	Arbitrary order
Separate `SafetyMonitor`	Defence in depth	Like hardware conflict monitors forcing flash	Trusting controller logic alone
Emergency is a **flag** states read	Safety	Only *shortens* green; yellow and all-red run in full	Jumping straight to emergency green
No Singleton controller	Rejected	Many intersections; a `TrafficControlCenter` can own them	`getInstance()`

Java implementation

Package `lld.ch14.trafficsignal`.

```java
/** State pattern: each state knows what one second of time means for it, and which state is next. */
interface SignalState {
    default void enter(Intersection x) {}
    void onTick(Intersection x);
    String name();
}

final class GreenState implements SignalState {
    // ...
    @Override public void onTick(Intersection x) {
        Integer em = x.emergencyPhase();
        if (em != null) {                                  // pre-emption: end this green early
            x.setState(em == phase ? new EmergencyHoldState(phase) : new YellowState(phase));
            return;
        }
        if (--remaining <= 0) x.setState(new YellowState(phase));
    }
    // ...
}

final class YellowState implements SignalState {
    // ...
    @Override public void onTick(Intersection x) {           // yellow is never cut short: that is unsafe
        if (--remaining <= 0) x.setState(new AllRedState(x.nextPhaseAfter(phase)));
    }
    // ...
}

final class Intersection {
    // ...
    synchronized void tick() { clock++; state.onTick(this); }
    // ...
    void setState(SignalState s) { state = s; s.enter(this); }
    // ...
    /** Turns red first, then green: never two crossing flows green, even for an instant. */
    void showPhaseGreen(int p) {
        Phase phase = phases.get(p);
        lights.forEach((a, l) -> { if (!phase.greens().contains(a)) l.set(Signal.RED); });
        phase.greens().forEach(a -> lights.get(a).set(Signal.GREEN));
    }
    // ...
    /** Safe = every approach that is green or yellow belongs to ONE phase. */
    synchronized boolean isSafe() {
        return phases.stream().anyMatch(p -> lights.entrySet().stream()
                .filter(e -> e.getValue().signal() == Signal.GREEN || e.getValue().signal() == Signal.YELLOW)
                .allMatch(e -> p.greens().contains(e.getKey())));
    }
    // ...
}
```

Full runnable code: code/lld/ch14/trafficsignal/ (see the Full Edition).

How it works: `QueueAdaptiveTiming` gives `secondsPerCar × waiting cars`, clamped to [min, max]. `setState` calls `enter` at once (set lights, load timer). `AllRedState` picks the emergency phase if pending, else the next. `SafetyMonitor` checks `isSafe()` on every change to GREEN; a real `ScheduledExecutorService` run with 20 concurrent emergency requests shows zero violations.

Output (excerpt):

```
t=37  N:G S:G E:R W:R  GREEN(0)
t=38  emergency request: WEST
t=39  N:Y S:Y E:R W:R  YELLOW(0)
t=41  N:R S:R E:R W:R  ALL_RED
t=42  N:R S:R E:G W:G  EMERGENCY_HOLD(1)
t=50  N:F* S:F* E:F* W:F*  FLASHING_RED
light changes seen by observer: 48, safety violations: 0
```

With 12 cars waiting E-W, the adaptive policy gives a 10 s green (the maximum).

Concurrency & edge cases

Situation	Handling
Emergency between ticks	Stored under the lock; next tick acts
Emergency during yellow	Yellow completes; all-red then picks the emergency phase
Emergency for the phase already green	Straight to `EmergencyHold` — no needless yellow
Two emergencies from crossing directions	Latest wins; the hold exits via yellow + all-red. Real systems queue by priority and arrival
Emergency never cleared	Maximum hold time
Timer and sensor threads overlap	`synchronized` methods; `CopyOnWriteArrayList` listeners
Adaptive timing starves a quiet road	Minimum green; every phase still gets its turn
Bug causes conflicting greens	`SafetyMonitor` detects; real controllers force flashing red
Timer drift	`scheduleAtFixedRate`; real deployments use the hardware clock

Extensions & follow-up questions

1. **Pedestrians** → WALK during the phase's green, flashing DON'T WALK before yellow, push-button requests.
2. **Left-turn arrows** → more phases; phases are data, state machine unchanged.
3. **Green wave** → a `TrafficControlCenter` sets cycle length and **offset** per intersection.
4. **Emergency corridor** → the centre pre-empts intersections along the route.
5. **Night mode** → time-of-day `TimingPolicy`, or flashing yellow/red states.
6. **Testing** → call `tick()` N times, assert snapshots.

Real-world example

SCATS and **SCOOT** adapt timing city-wide from sensors; **emergency pre-emption** emitters still time out yellow and red first; **conflict monitors / MMUs** force flash on conflicting greens — our `SafetyMonitor`.

In the interview

40 minutes: 5 requirements → 5 entities → 10 state machine → 15 code Green/Yellow/AllRed and `tick()` → 5 concurrency and follow-ups.

30-second answer: "Phases are sets of approaches that can be green together; the controller is a timed State machine — Green, Yellow, All-Red, plus Emergency-Hold and Flashing-Red. A timer calls `tick()` each second; each state counts down and names the next. Green time comes from a swappable timing Strategy. An emergency only shortens green: yellow and all-red always run in full, then the emergency phase is held until cleared. Lights publish to observers, including a safety monitor that checks no crossing flows are ever green together."

- *"Why not jump straight to emergency green?"* — Cars in the junction need yellow and all-red to clear it.
- *"Why not a Singleton?"* — One per intersection; a city has thousands.

Self-check

1. **Q:** What is a phase? **A:** Approaches that may be green together because their paths don't cross.
2. **Q:** Why all-red? **A:** To let vehicles still in the junction clear it.
3. **Q:** Emergency during yellow? **A:** Yellow finishes; all-red then picks the emergency phase.
4. **Q:** Why do these states hold data but vending states don't? **A:** Each interval has its own countdown.

TL;DR

- **Timed State machine**: Green → Yellow → All-Red → next Green; plus Emergency-Hold and Flashing-Red.
- **Safety first**: red before green, yellow never cut short, conflict-monitor observer.

- **Strategy** for timing, **Observer** for lights, emergency as a flag.
- Tick-driven and synchronized: deterministic tests.

14.7 Design a Shipping Locker System

A **parcel locker** is a wall of numbered doors in a shop or lobby: a courier puts a parcel behind one door, the customer types a **pickup code** and that door opens (Amazon Hub Locker, InPost Paczkomat). It looks like the parking lot — hand out limited spaces of several sizes — plus three extras interviewers probe: **deadlines** (a reservation that times out, a parcel that expires), **a secret** (the code, hard to guess, never shown to the courier) and **a network** (station full → suggest another).

Problem & clarifying questions

Question	Assumed answer
Door sizes?	Small, medium, large; a parcel may be upgraded to a bigger free door.
When is the door chosen?	In advance: the carrier **reserves** it; the hold lasts 2 h, then the door is released.
How does the customer open it?	6-digit **one-time code** by SMS/app on deposit; the courier never sees it.
How long can a parcel stay?	3 days; then it **expires** and the courier takes it back.
Code guessing?	Random, single-use; 5 wrong codes in a row lock the keypad for 15 min.
Many stations?	Yes; offer the nearest one with space. Returns and cash on delivery are extensions.

Requirements

Functional: (1) numbered doors of fixed sizes; (2) **reserve** the best-fitting free door, all-or-nothing, with a hold deadline; (3) **deposit** → one-time code **sent to the customer only**; (4) **pickup** opens one door once; (5) **expiry** → customer told, courier collects for return; (6) keypad lockout; (7) nearest station with space; (8) doors out of service.

Non-functional: never one door for two parcels under concurrency; codes unpredictable, single-use, stored as a hash; deadlines correct even if the timer is late; clock and randomness injected.

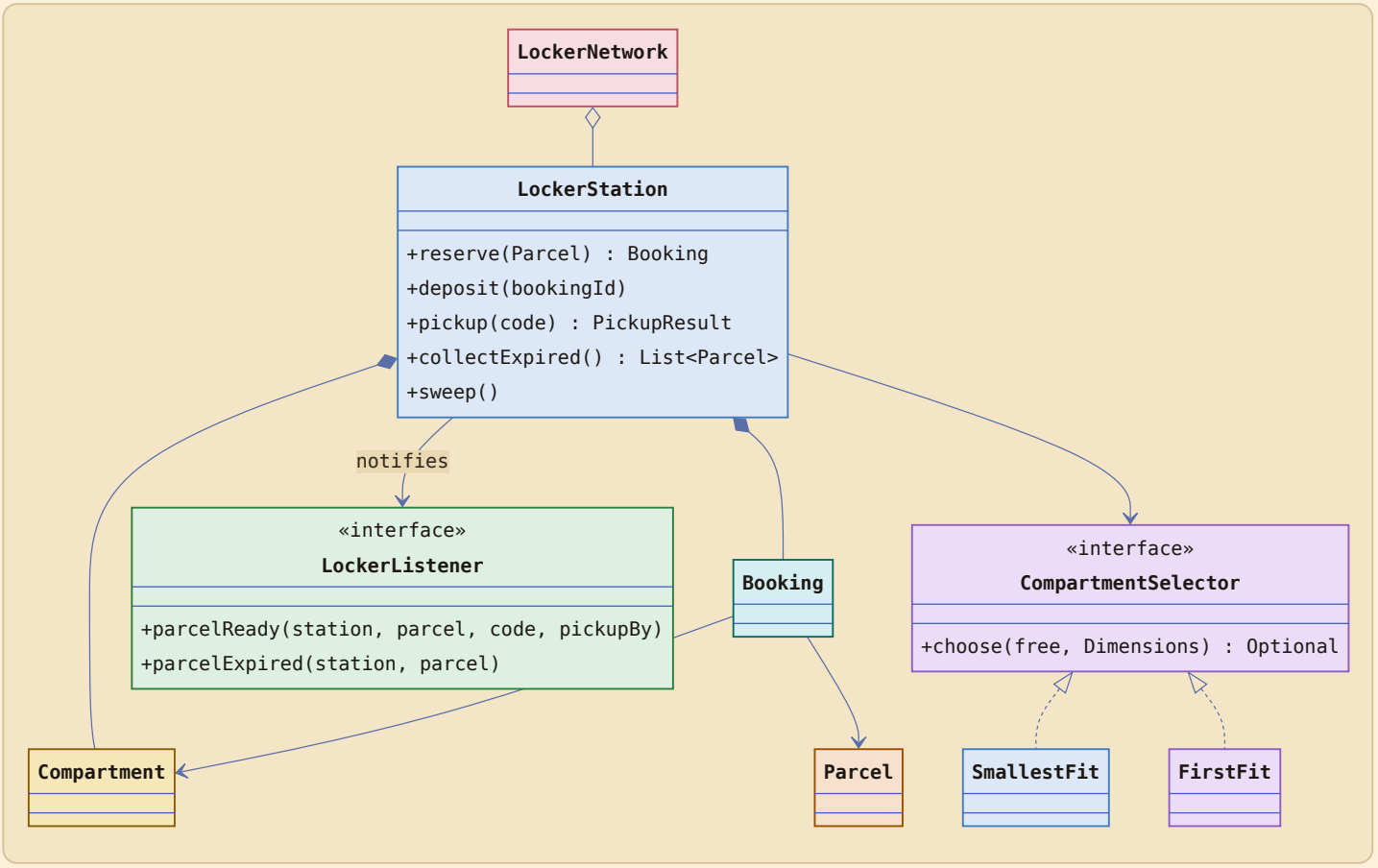
Use cases

Actors → use cases: Carrier system → reserve, find a station with space; Courier → deposit, collect expired; Customer → pick up; Technician → door out of service; Scheduler → sweep deadlines and notify.

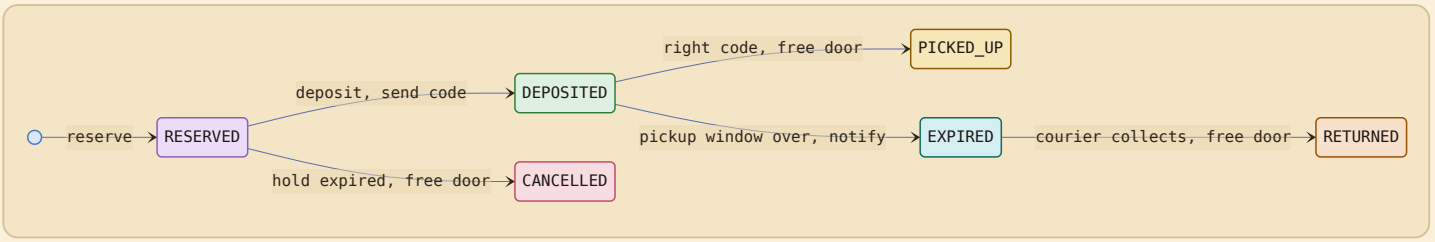
Core entities

Class	Responsibility
`Size` (enum), `Dimensions`, `Parcel` (records)	Door sizes with `fits()` (sort both boxes' sides, compare in order); parcel data.
`Compartment` + `CompartmentState`	One door: FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE; legal moves in `moveTo`.
`Booking` + `BookingStatus`	One parcel's stay: RESERVED → DEPOSITED → PICKED_UP / EXPIRED → RETURNED / CANCELLED; deadlines and code **hash**.
`PickupResult` (sealed)	`DoorOpened`, `WrongCode`, `KeypadLocked`, `ParcelExpired`.
`CompartmentSelector`	**Strategy**: `SmallestFit`, `FirstFit`.
`LockerListener`	**Observer**: parcel ready (with code), parcel expired.
`LockerStation`, `LockerNetwork`	Doors, bookings, code index, lockout under one lock; nearest station with space.

Class diagram



Key flows



Delivery: carrier `reserve(parcel)` → `SmallestFit` picks the smallest free door that fits → door RESERVED, hold 2 h → courier `deposit` → door OCCUPIED, random code, only its SHA-256 hash stored → listener sends the code to the customer **after the lock is released** → customer `pickup(code)` → hash matches → door FREE, code deleted. The door has its own small machine: FREE ↔ RESERVED → OCCUPIED → FREE, and FREE ↔ OUT_OF_SERVICE.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Door choice as `CompartmentSelector`	**Strategy**	Smallest-fit saves big doors; first-fit, eye-level-first swap in	`if (mode == …)` in `reserve`
Door state separate from booking status	Two enum state machines	Doors are reused; bookings keep deadlines and history	One field mixing both
Sealed `PickupResult`	Sealed result types	Every keypad outcome handled, compiler-checked	`null` / magic strings
Random, single-use, hashed codes	Security by design	A DB leak reveals no live code; courier never sees it	Plain codes; codes derived from the parcel id
Listeners called outside the lock	**Observer**	A slow SMS gateway can't block the keypad	SMS call inside `deposit`
Lazy expiry on every call + `sweep()` for messages	Check deadlines on read	Doors come back even if the scheduler is late	A reaper thread as the only mechanism
Injected `Clock`, `RandomGenerator`	DI	Deterministic tests	`Instant.now()`, `new Random()` inside
One lock per station	Monitor	Tens of doors, microsecond calls	Per-door CAS (needless here)

Java implementation

Package `lld.ch14.locker`.

```java
// File: lld/ch14/locker/LockerStation.java
    synchronized Booking reserve(Parcel parcel) {
        expireOverdue();
        // ... reject a parcel that already has an active booking
        List<Compartment> free = compartments.values().stream().filter(Compartment::isFree).toList();
        Compartment c = selector.choose(free, parcel.dims())
                .orElseThrow(() -> new LockerException("No free compartment fits " + parcel.dims() + " at " + id));
        c.moveTo(CompartmentState.RESERVED);
        Booking b = new Booking(id + "-B" + nextBooking++, parcel, c, now() + reservationHold.toSeconds());
        bookings.put(b.id, b);
        return b;
    }

    void deposit(String bookingId) {
        Booking b;
        String code;
        synchronized (this) {
            // ... expireOverdue(); booking must be RESERVED
            b.compartment.moveTo(CompartmentState.OCCUPIED);
            b.status = BookingStatus.DEPOSITED;
            b.pickupBy = now() + pickupWindow.toSeconds();
            do { code = "%06d".formatted(random.nextInt(1_000_000)); }       // unique among active codes here
            while (bookingByCodeHash.containsKey(hash(code)));
            b.codeHash = hash(code);
            bookingByCodeHash.put(b.codeHash, b.id);
        }
        for (LockerListener l : listeners) l.parcelReady(id, b.parcel, code, b.pickupBy);   // outside the lock
    }

    synchronized PickupResult pickup(String code) {
        expireOverdue();
        long t = now();
        if (t < lockedUntil) return new KeypadLocked(lockedUntil - t);
        String bookingId = bookingByCodeHash.get(hash(code));
        if (bookingId == null) {
            // ... count the wrong code; the 5th in a row sets lockedUntil = t + 15 min and returns KeypadLocked
            return new WrongCode(MAX_WRONG_CODES - wrongCodes);
        }
        wrongCodes = 0;
        Booking b = bookings.get(bookingId);
        if (b.status == BookingStatus.EXPIRED) return new ParcelExpired(b.parcel.id());
        b.status = BookingStatus.PICKED_UP;
        b.compartment.moveTo(CompartmentState.FREE);
        bookingByCodeHash.remove(b.codeHash);                        // a code works once
        return new DoorOpened(b.compartment.id(), b.parcel.id());
    }
    // ... expireOverdue(): RESERVED past hold -> CANCELLED (door FREE); DEPOSITED past window -> EXPIRED
```

Full runnable code: code/lld/ch14/locker/ (see the Full Edition).

How it works: `Size.fits` sorts the sides of box and door and compares in order (no need to try six rotations). `sweep()` (called by a scheduler) runs the same deadline check and sends "not collected" messages outside the lock. `LockerNetwork.nearestWithSpace` sorts stations by distance and returns the first within the radius that can fit the parcel now.

Output (excerpt):

```
P3 9x12x12cm    -> MG01-03(MEDIUM)
error: Parcel P1 already has a locker
SMS to +91-98450-00001: parcel P1 is in locker MG01, code 694696, collect by 2026-10-04T09:00:00Z
right code  -> DoorOpened[compartmentId=MG01-01, parcelId=P1]
same code again -> WrongCode[attemptsLeft=4]  (a code works once)
guess 000004 -> KeypadLocked[secondsLeft=900]
booking MG01-B4 is CANCELLED, free now: {SMALL=2, MEDIUM=1, LARGE=1}
customer types the code -> ParcelExpired[parcelId=P3]
station F: small parcel -> F-01(LARGE), then large parcel -> REJECTED
station S: small parcel -> S-02(SMALL), then large parcel -> S-01(LARGE)
reserved=20 rejected=30 distinct doors=20
```

The mug was upgraded to MEDIUM because both small doors were taken; `FirstFit` wasted the large door and had to refuse a large parcel; 50 concurrent reservations on 20 doors gave exactly 20 distinct doors.

Concurrency & edge cases

Situation	Handling
Two carriers, last door	`synchronized reserve`: choose + mark is one step
Courier late past the hold	Booking CANCELLED; `deposit` refuses; carrier re-reserves
Retried reserve for the same parcel	Rejected while a booking is active (idempotent by parcel id)
Duplicate random code	Regenerate until unique among live codes
Guessing	10^6 codes, lockout after 5 wrong, codes expire
Scheduler down	Lazy expiry still frees doors; only the SMS is late
`canFit` true but `reserve` fails	`nearestWithSpace` is a hint; reserve, then try the next station
Broken door	Only a FREE door can go OUT_OF_SERVICE (`moveTo`)

Extensions & follow-up questions

1. **Returns** → drop-off booking with its own code; courier collects later.
2. **Extend pickup window** → move `pickupBy` once, only while DEPOSITED.
3. **Cash on delivery** → a payment step before the door opens.
4. **Keep large doors for large parcels** → a selector that refuses upgrades when few large doors are left.
5. **Thousands of stations** → shard by station; geo index (geohash, R-tree) for the network search.
6. **Database** → `UPDATE compartment SET state='RESERVED' WHERE id=? AND state='FREE'` and check one row changed.

Real-world example

Amazon Hub Locker sends a pickup code and returns uncollected parcels after a limited number of days (our EXPIRED → RETURNED path); InPost's Paczkomat lockers in Poland open with a code or an app and are dense enough that "nearest locker with space" matters. In both, the system — not the courier — chooses the door.

In the interview

40 minutes: 5 questions → 5 entities → 8 the two state machines → 15 code `reserve`/`deposit`/`pickup` → 7 security, concurrency, follow-ups.

> **30-second answer:** "Doors come in a few sizes; a Strategy picks the smallest free one that fits, and choose-and-mark runs under the station lock so two carriers never get the same door. On deposit I generate a random single-use code, store only its hash and notify the customer through an observer after releasing the lock. Pickup matches the hash, frees the door and deletes the code; five wrong codes lock the keypad. Holds and pickup windows are checked on every call, so doors come back even if the scheduler is late."

- *"Why a Booking separate from the Compartment?"* — The door is reused; the booking keeps each parcel's deadlines and history.
- *"Why hash the code?"* — Anyone reading the database or logs could otherwise open doors.

Self-check

1. **Q:** How does `fits` handle a box on its side? **A:** Sort both sets of sides and compare in order.
2. **Q:** What frees a door if the scheduler never runs? **A:** Lazy expiry at the start of every call.
3. **Q:** Why notify after the `synchronized` block? **A:** A slow SMS call must not hold the station lock.
4. **Q:** Which pattern swaps smallest-fit for first-fit? **A:** Strategy (`CompartmentSelector`).

TL;DR

- **Strategy** picks the smallest free door that fits; choose-and-mark is **atomic** per station.
- Two **state machines**: door and booking (RESERVED → DEPOSITED → PICKED_UP / EXPIRED → RETURNED / CANCELLED).
- Codes **random, single-use, hashed**, sent via an **Observer** outside the lock; lockout after 5 wrong.

- Deadlines enforced **lazily**; inject clock and randomness.

Chapter summary

Problem	Core difficulty	Key data structure	Key patterns	Concurrency tool
Parking lot	Allocate spots without double booking	Concurrent sorted set of free spots per type	Strategy, Factory, Observer	`pollFirst` + CAS, per-ticket lock, `putIfAbsent`
Vending machine	Legal order of actions; exact change	`EnumMap` of coin counts; DP table	State	`synchronized` machine
Coffee machine	All-or-nothing multi-ingredient deduction	`EnumMap` levels, recipes as data	Observer, sealed results	One `ReentrantLock`; notify outside the lock
ATM	Money correctness and authentication	Total vs available balance; note chain	State, Chain of Responsibility, Command	Per-account lock; ordered locking for transfers
Elevator	Scheduling and dispatching	Two `TreeSet`s (up/down stops)	Strategy (+ modes)	Per-car lock, synchronized dispatcher
Traffic signal	Timed transitions with a safety invariant	Phases as data	State, Strategy, Observer	Synchronized tick; scheduled executor
Shipping locker	Best-fit door, holds and pickup windows, secret codes	Doors per station; bookings; code-hash index	Strategy, Observer, enum state machines, sealed results	One lock per station; lazy expiry; notify outside the lock

Chapter self-check

1. **Q:** Stateless vs data-carrying states? **A:** Vending and ATM share stateless singletons; traffic-signal states hold countdowns, new per transition.
2. **Q:** The check-then-act race in parking, coffee, ATM? **A:** Two gates one spot (atomic `pollFirst`/CAS); two orders the last milk (one lock around check+deduct); two ATMs one account (check+debit under the account lock).
3. **Q:** Where did we reject a Singleton? **A:** Parking lot, coffee machine, traffic controller — hidden global, untestable, and real deployments have many.
4. **Q:** Where does greedy fail? **A:** Change with limited coins (vending) and notes with limited cassettes (ATM); use bounded coin-change DP.
5. **Q:** Which designs are tick-driven and why? **A:** Elevator and traffic signal — deterministic, clock-free tests.
6. **Q:** Which case enforces deadlines lazily? **A:** The shipping locker — every call expires overdue holds/parcels first; the sweep only sends messages.

Chapter TL;DR

- Find the **one hard method** per problem and spend your time there.
- **State** for machines whose buttons change meaning; **Strategy** for changing rules; **Observer** for boards, alerts, monitors; **Chain of Responsibility** and **Command** for the ATM.
- Make every **check-then-act** atomic, lock at the **smallest** scope, lock in a **fixed order**, call listeners **outside** locks.
- Prefer injected instances over Singletons, integer cents over doubles, tick-driven logic over sleeps.
- Deadlines (holds, pickup windows): check on every call; secrets (pickup codes): hashed, single-use.

Chapter 15 — Case Studies: Booking & Inventory (Part 1) (Study Edition)

> **Where this fits.** Chapter 14 designed small, self-contained objects. This chapter is the first of two on **booking systems**: software that sells or lends a limited thing (a book copy, a cinema seat, a concert place, a hotel night) to many people who want it at once. Chapter 16 continues. This is the study edition: every topic stays, the padding goes; the Full Edition (README.md) has the complete code and outputs.

Why booking problems are interview favourites

A booking problem looks like simple CRUD until two people want the same thing. Then five hard questions appear, and each problem here is a different mix:

Theme	The question the interviewer is really asking	Where you will see it
Inventory	What exactly is sold or lent? A title or a copy? A seat or a seat *for one show*? A room or a room *on one night*?	`Book` vs `BookItem` (15.1), `HallSeat` vs `ShowSeat` (15.2), GA count vs numbered seat (15.3), `Room` + booked nights (15.4)
Holds / locks	How do you keep something for a customer while they pay, without keeping it forever?	Seat lock with a TTL (time-to-live) (15.2), CAS holds + reaper (15.3)
Double-booking prevention	What stops two threads (or servers) from both succeeding?	`synchronized` per item (15.1, 15.4), one monitor per show (15.2), lock-free CAS (15.3)
Payments	What if payment fails? Or succeeds but the hold was lost meanwhile?	Release on decline, refund as **compensation** (15.2, 15.3)
Cancellations	Who gets the thing next, and how much money comes back?	Waitlists (15.1, 15.3), refund policies (15.2, 15.4)

Three words used everywhere:

- **Race condition** — two threads both read "free" and both act; one result silently overwrites the other.
- **Check-then-act** — "if free, then take it". Safe only if check and act are one **atomic** step.
- **Hold** (lock, soft lock, reservation) — a temporary claim **with an expiry**, so an abandoned checkout does not block the item forever.

#	Problem	The one idea that carries the design
15.1	Library Management System	A *title* (`Book`) is not a *copy* (`BookItem`). Limits live with the member; the waitlist lives with the title.
15.2	Movie Ticket Booking (BookMyShow)	You book a `ShowSeat`, not a physical seat. Hold with TTL → pay → commit; refund if the hold was lost.
15.3	Concert Ticket Booking	Extreme contention: lock-free CAS, a GA counter, a fair virtual queue, per-user limits, a waitlist.
15.4	Hotel Management System	A status field is not availability. Each room remembers which nights are sold.

How to run every example in this chapter

```
cd chapters/15-case-booking-1/code
javac --release 21 -d /tmp/lld_15 $(find . -name '*.java')
java -cp /tmp/lld_15 lld.ch15.library.Main        # or moviebooking, concert, hotel
```

In the concurrency demos, *which* thread wins changes per run; the *number* of winners never does. Several small package-private types share a file (e.g. `Model.java`); `-Xlint:all` flags this as `[auxiliaryclass]` — a style warning, not an error.

> **Money.** Every amount is a `BigDecimal` built from a string (`new BigDecimal("0.50")`), never a `double` (0.10 is not exact in binary, so sums drift). Compare with `compareTo` / `signum()`, not `equals` (which also compares scale), and name the `RoundingMode` at every rounding step.

> **Time trick.** Every service takes a `java.time.Clock`; demos pass a `SimClock` whose `advance(Duration)` moves time forward, so "10 days later" needs no `sleep`. Production passes `Clock.systemUTC()`.

15.1 Design a Library Management System

Problem & clarifying questions

A **Library Management System** (LMS) tracks every title and physical copy, who borrowed which copy and when it is due, members and cards, reservations and fines.

Question	Answer we assume
Copies?	Many per title — each a **book item** with barcode and rack.
Limits?	**5 books** per member, **10 days** per loan, then a fine per day.
Reserve a book that is out?	Yes, with a notification when a copy returns.
Fines?	Calculated and recorded; payment out of scope. E-books, inter-library loans out of scope.

Requirements

Functional: (1) search by **title, author, subject, publication date**; (2) each book has an ISBN; each copy a **rack number**; (3) a title has **many copies**; (4) answer **"who took this copy?"** and **"which books does this member have?"**; (5) **max 5 books** per member; (6) **max 10 days** per loan; (7) **fines** for late returns; (8) **reserve** unavailable books; (9) **notify** when a reserved book is available and when a book is overdue; (10) **barcodes** on items and cards.

Non-functional: correct under concurrency (no double-lent last copy, never over 5); index-based search; swappable fine rules and channels.

Use cases

Actors: Librarian (add/edit/remove books, items, members; issue, reserve, return for a member), **Member** (search, check out, reserve, renew, return), **System** (overdue, reserved-book-back, cancelled-reservation notifications). **Top use cases:** manage books/items; search; register/cancel account; **check out**; **reserve**; **renew**; **return**.

In the course's use-case diagram, `<<include>>` means *always* performs another (checking out always clears the member's reservation), and `<<extend>>` is an *optional variation* (search by author extends search catalog).

Core entities

Class	Responsibility
`Book` (record)	The **`title`**: ISBN, title, subject, publisher, date, authors. Immutable.
`BookItem`	One **physical copy**: barcode, format, rack, price, status (`AVAILABLE`, `RESERVED`, `LOANED`, `LOST`), who it is held for.
`Author`, `Rack` (records)	Author name; rack number and location.
`Account` (sealed, abstract) → `Member`, `Librarian`	People; an account **owns** its `LibraryCard` (composition); a member carries the loan count.
`LibraryCard` (record)	Card number, barcode, issue date.
`BookLending`	One loan: barcode, member id, created, **due date**, return date.
`Fine` (record)	Amount owed for one late return.
`Search` / `Catalog`	Four searches; one index per search; knows every copy of every title.
`LendingService`	Rules: check out, return, renew, reserve, overdue notices.
`FinePolicy` / `PerDayFine`	Cost of a late return (Strategy).
`Notifier` / `ConsoleNotifier`	Where messages go.
`Library`	Facade: owns catalog and service; only librarians add stock.

All course classes are kept; `BookReservation` becomes a per-title waitlist and `Notification` the `Notifier` interface.

Class diagram

The course builds five stages: (1) the "what happened" records — `BookReservation`, `BookLending`, `Fine`, each with barcode and member id (answers requirements 4, 7, 8); (2) **`Book` vs `BookItem`** — the split that carries the design (the course has `BookItem` *inherit* `Book`; we use **composition**); (3) abstract `Account` with `Member`/`Librarian` — limits (req. 5, 6) belong to people; the card is **composition** (dies with the account); (4) `Search` interface implemented by `Catalog`; (5) `Library` owns catalog, racks, items; abstract `Notification` per channel.

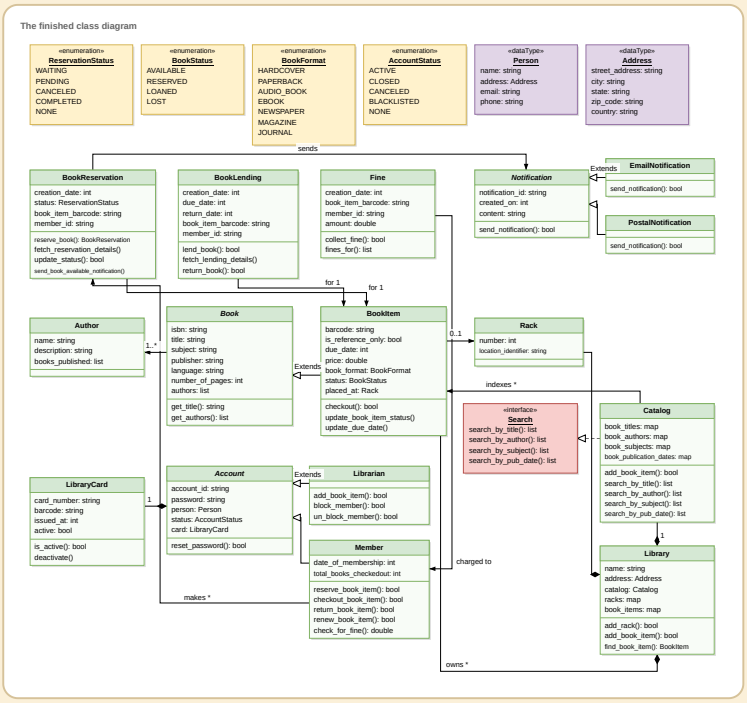
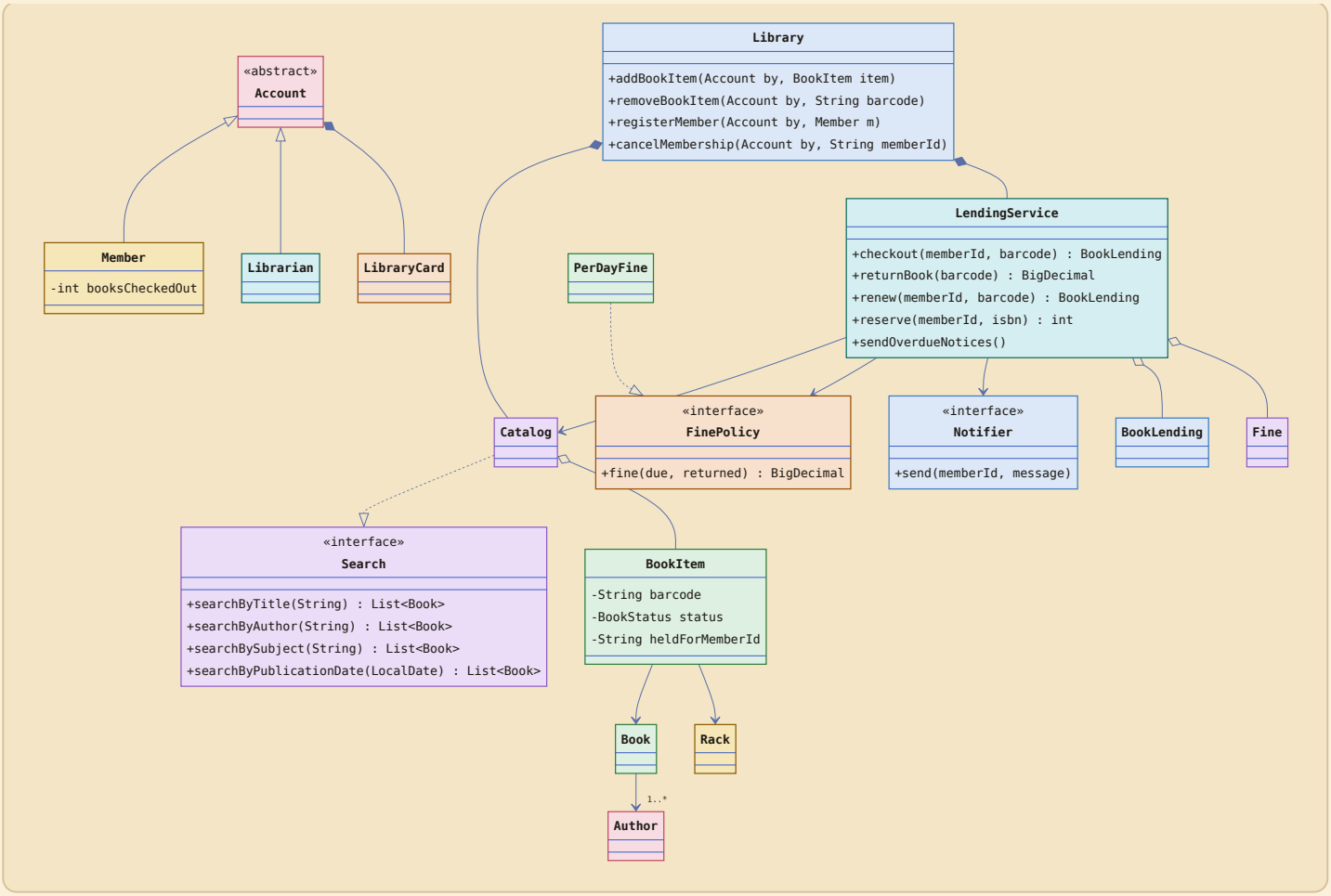


Figure — The finished library class diagram: every class from the five stages in full colour (course diagram)



Key flows

The course's three activity diagrams:

- **Check out:** scan book → can it be issued (not reserved for someone else)? → member under quota? → create the loan, mark "Loaned", increment the member's count, complete this member's reservation.
- **Return:** scan → late? charge fine → decrement count → reserved? notify the first waiting member and mark it held; else make it available.
- **Renew:** scan card and book → charge a fine if already late → someone reserved it? refuse; else new due date.

Copy state machine: `AVAILABLE → LOANED` (checkout); `LOANED → AVAILABLE` (return, nobody waiting) or `→ RESERVED` (return, waitlist non-empty); `RESERVED → LOANED` (checkout by the member it is held for); `LOANED → LOST`.

Design decisions & patterns used

Decision	Why	Alternative rejected
`Book` vs `BookItem`	Members borrow *copies*; search works on *titles*. Say it early.	One `Book` with a `copies` counter — can't say *which* copy is late or where.
Composition (`BookItem` has a `Book`)	Many copies share one immutable title; inheritance would duplicate title fields per copy; a title has no barcode, so "copy *is a* title" fails strict substitution.	`BookItem extends Book` (course) — valid for a small model; mention it.
Rules in `LendingService`, not `Member`	The course itself flags `Member` doing checkout/return/renew as two reasons to change (SRP).	Methods on `Member` (fine for a first sketch).
Reservation = FIFO waitlist per title	A member wants "a copy of Dune"; any returned copy goes to the first in line.	Per-copy reservation — wait on copy #3 while #1 sits on the shelf.
Strategy for fines (`FinePolicy`)	Caps, discounts, holidays change.	`if/else` in `returnBook`.
Observer-style `Notifier`	Lending announces; delivery is someone else's job (course's `Notification` subclasses = same idea).	Emailing from `LendingService`.
Facade (`Library`)	One entry point; role checks live there.	Exposing catalog and service to the UI.
One index per search	Every lookup is a map read; cost: a new search adds a map (a single `(field, value)` index would stay one structure).	Scanning all books.

Java implementation

Package `lld.ch15.library` (`Model`, `Catalog`, `Support`, `LendingService`, `Library`, `Main`).

```java
/**
 * The three core flows (check out, return, renew) plus reservations.
 * Lives apart from Member so a Member stays "a person", and the rules live in one place.
 *
 * Locking rule (prevents deadlock): always lock the MEMBER first, then the BOOK ITEM.
 */
public final class LendingService {
    static final int MAX_BOOKS_PER_MEMBER = 5;
    static final int MAX_LENDING_DAYS = 10;
    // ...
    BookLending checkout(String memberId, String barcode) {
        Member m = member(memberId);
        BookItem item = catalog.item(barcode);
        synchronized (m) {
            synchronized (item) {
                if (m.status() != AccountStatus.ACTIVE)
                    throw new LibraryException(memberId + " account is " + m.status());
                if (m.booksCheckedOut() >= MAX_BOOKS_PER_MEMBER)
                    throw new LibraryException(memberId + " already has " + MAX_BOOKS_PER_MEMBER + " books");
                switch (item.status()) {
                    case AVAILABLE -> { /* ok */ }
                    case RESERVED -> {
                        if (!memberId.equals(item.heldForMemberId()))
                            throw new LibraryException(barcode + " is held for another member");
                    }
                    case LOANED, LOST -> throw new LibraryException(barcode + " is " + item.status());
                }
                item.markLoaned();
                m.incrementLoans();
                LocalDate today = today();
                BookLending loan = new BookLending(barcode, memberId, today, today.plusDays(MAX_LENDING_DAYS));
                activeLoans.put(barcode, loan);
                removeFromWaitlist(item.book().isbn(), memberId);   // their reservation is now fulfilled
                return loan;
            }
        }
    }
    // ... returnBook(): same lock order, close loan, chargeFineIfLate, handToNextInLine
    /** Caller holds the item's monitor. Give the copy to the first waiting member, or shelve it. */
    private void handToNextInLine(BookItem item) {
        Deque<String> q = waitlist(item.book().isbn());
        String next;
        synchronized (q) { next = q.pollFirst(); }
        if (next == null) { item.markAvailable(); return; }
        item.holdFor(next);
        notifier.send(next, "'" + item.book().title() + "' copy " + item.barcode() + " is held for you");
    }
    // ...
}

/** Strategy: how much a late return costs. Swappable without touching LendingService. */
interface FinePolicy {
    BigDecimal fine(LocalDate dueDate, LocalDate returnDate);
}
```

Full runnable code: code/lld/ch15/library/ (see the Full Edition).

Output (excerpt):

```
  REFUSED: M1 already has 5 books
== 3. 8 members race for the ONLY copy of Dune ==
  winners=1 losers=7 (exactly one copy was lent)
  REFUSED: cannot renew B-DUNE-1: another member reserved this title
  [notify M9] fine of $1.50 for late B-DUNE-1
  [notify M2] 'Dune' copy B-DUNE-1 is held for you
  M4 REFUSED: B-DUNE-1 is held for another member
```

A sixth book is refused; one of eight racing members wins the only copy; renewal is refused while someone waits; a 3-day-late return is fined 3 × $0.50 and the copy is **held** for the first in line. Librarian housekeeping: a copy on loan can't be removed, a member with a book out can't be cancelled, a cancelled member can't borrow.

Concurrency & edge cases

- **Last copy:** `synchronized (item)` makes "available? → mark loaned" atomic; the 8-thread race always has one winner.
- **Two librarians, one member:** quota check and increment under `synchronized (member)` — never 6 books.
- **Deadlock avoidance:** always **member first, then item** (fixed global order). `returnBook` reads the loan to find the member, then locks in that order and re-checks the loan.
- **Renew vs reserve:** renewal refused if the waitlist is non-empty — checked *before* charging a fine, so no double fine.
- **Known small window:** `reserve` checks "no free copy" then queues in two steps; a copy returned in between goes to the shelf and the *next* return reaches the member. Accepted, because taking the queue lock before copy locks would reverse the order returns use — the recipe for deadlock.
- **Edge cases:** unknown barcode/member, duplicate barcodes, inactive/blacklisted accounts, removing a loaned/held copy, cancelling with books out, reserving a title with a free copy ("just check it out"), joining a waitlist twice (keeps place), returning a book not out.
- **Across servers:** use a conditional update — `UPDATE book_item SET status='LOANED' WHERE barcode=? AND status='AVAILABLE'` and check exactly one row changed.

Extensions & follow-up questions

- **"Two members reserve at the same moment?"** — The course calls it unguarded; ours appends to a FIFO deque under a lock, so they queue in a definite order.
- **"Where is the available-copies count?"** — Derive it (`availableCopies(isbn)` counts AVAILABLE copies); if slow, cache it and update under the same lock as the status.
- **"How is the fine paid?"** — Add a `PaymentService`; block checkouts while unpaid fines exceed a threshold.
- **"Hold expiry?"** — `heldUntil` on `BookItem` plus a daily job passing it on — the TTL idea of 15.2.
- **"E-books?"** — `EBOOK` items with no rack, a fixed number of concurrent licences.

Real-world example

Koha and **Evergreen** use this model (records + barcoded items, circulation rules, holds queue per title); **Libby** applies the waitlist to e-book licences.

In the interview

30–45 minutes: 3 min clarify (copies, 5 books, 10 days, fines, reservations) → 5 min use cases → 10 min entities, **starting with** **`Book`** / **`BookItem`** → 10 min code `checkout` / `returnBook` with lock ordering → 5 min reservation hand-over and fines → follow-ups.

> **30-second answer:** "The key split is `Book` (title) vs `BookItem` (a copy with barcode and rack). Members borrow copies; search works on titles via a `Catalog` with one index per search. A `LendingService` enforces 5 books and 10 days, records `BookLending`s, charges fines via a `FinePolicy` strategy, and hands returned copies to a FIFO waitlist per title. Checkout locks member then copy so the last copy can't be double-lent."

Self-check

- **Q:** Why a fixed lock order? **A:** Otherwise A holds member→waits item while B holds item→waits member: deadlock.
- **Q:** What stops a 6th book with two desks scanning? **A:** Check and increment inside `synchronized (member)`.
- **Q:** When can't a book be renewed? **A:** When another member is waiting for that title.
- **Q:** Which pattern changes fine rules without touching lending? **A:** Strategy (`FinePolicy`).

TL;DR

Title vs copy. Limits on the member, state on the copy, history in `BookLending`. Rules in a `LendingService`; fines as a Strategy; notifications behind an interface. Lock member-then-copy. Reservations are a FIFO queue per title.

15.2 Design a Movie Ticket Booking System (BookMyShow)

Problem & clarifying questions

An online movie booking system (**BookMyShow**, **Fandango**, **AMC**) lets customers find movies, pick a show, choose seats on a map, pay and get a ticket. The classic booking question: a place hierarchy (city → cinema → hall → seat), a time dimension (shows), and a genuine race (two people tapping one seat).

Question	Answer we assume
Venues?	Many cities; several halls per cinema; a hall runs **one show at a time**.
Seats?	Several per booking from a seat map; **held 5 minutes** while paying.
Payment, refunds?	Card or cash, optional **coupon**; refund up to 2 hours before the show.
Users?	Customers, admins, front-desk officers (walk-ins), guests (search only).

Requirements

Functional (course order): (1) list **cities**; (2) cinemas have **halls**, one show at a time; (3) movies have many **shows**; (4) **search** by title, language, genre, release date, city; (5) show **cinemas and shows** for a movie; (6) **book** a show; (7) **seating plan**, pick several seats; (8) distinguish **available** from booked; (9) **notify** on new movie, booking, cancellation; (10) pay by **card or cash**; (11) **no two customers reserve the same seat**; (12) **discount coupon**.

Non-functional: strong seat consistency; high read, bursty write; **hold expiry** (a gap the course names; we implement it); never pay for a seat you don't get.

Use cases

Actors: Admin (add/cancel movies and shows, block customers), FrontDeskOfficer (book/cancel for walk-ins), Customer (view, book, cancel), Guest (search only — must register), System (notifications). **Use cases:** search; create/modify/view booking; pay; apply coupon; assign seats; refund on cancellation within the allowed time. In the course diagram a **Customer is a specialised Guest** — enforced in code because `BookingService` takes a `Customer`.

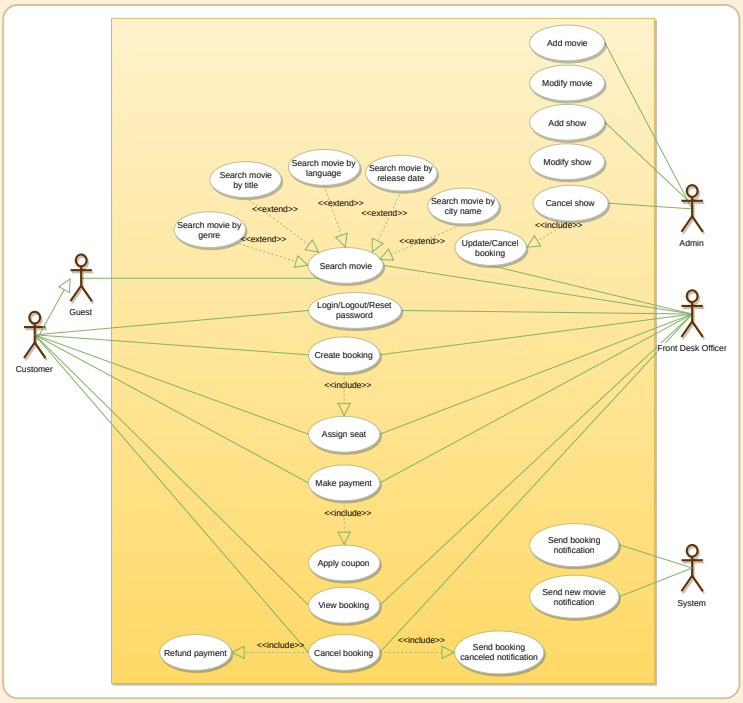


Figure — Movie booking use cases. The hollow arrow from Customer to Guest says a customer can do everything a guest can; the five "search by" use cases extend "Search movie"; "Create booking" includes "Assign seat"; "Make payment" includes "Apply coupon"; "Cancel booking" includes "Refund payment" and the cancellation notification; the admin's "Cancel show" includes updating/cancelling its bookings (course diagram)

Core entities

Class	Responsibility
`City`, `Cinema`, `CinemaHall`	Venue hierarchy; a cinema **owns** halls, refers to its city; a hall refuses an overlapping show.
`HallSeat` (record)	The bolted-down seat: id, `SeatType`. **No booking state.**
`Movie` (record)	Title, language, genre, release date, duration.
`Show`	One screening; **owns one** `ShowSeat` **per hall seat**.
`ShowSeat`	A seat *for this show*: price, `booked`. **This is what gets booked.**
`SeatLockProvider` / `InMemorySeatLockProvider`	Expiring, all-or-nothing **holds**; commit; release.
`Booking`	Customer, show, seat ids, status (`PENDING`, `CONFIRMED`, `CHECKED_IN`, `CANCELED`, `ABANDONED`), payment, coupon.
`BookingService`	hold → pay → commit; cancel; seat map.
`Payment`, `PaymentMethod` (`CardPayment`, `CashPayment`), `PaymentGateway`	Money; the gateway is the boundary to the provider.
`Coupon`	Percent off the booking amount before payment.
`CancellationPolicy` / `CutoffRefundPolicy`	Refund on cancel (Strategy).
`MovieSearch` / `Catalog`	Five searches; "by city" comes from shows.
`Customer`, `Guest`, `FrontDeskOfficer`, `Admin`; `Notifier`	Actors (`Guest` can't book); messages.

Class diagram

Six course stages: (1) city, cinema, hall, physical seat; (2) movie, show, **show seat — where the design succeeds or fails**: one physical seat is free at 3 pm and sold at 9 pm, so you reserve a `ShowSeat` (which also allows per-show prices); (3) coupon, payment types, notification types — the coupon reduces **the booking's amount**, not the payment; (4) `Booking` and `BookingService` — one seat never goes to two customers; (5) search interface and catalog; (6) people — booking restricted to customers.

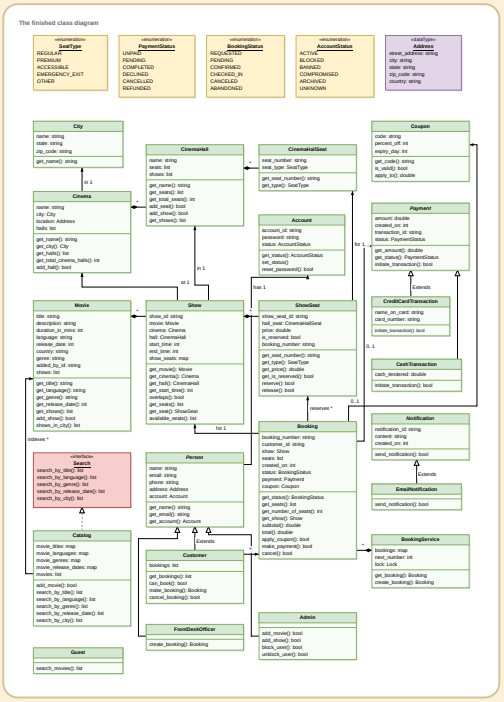
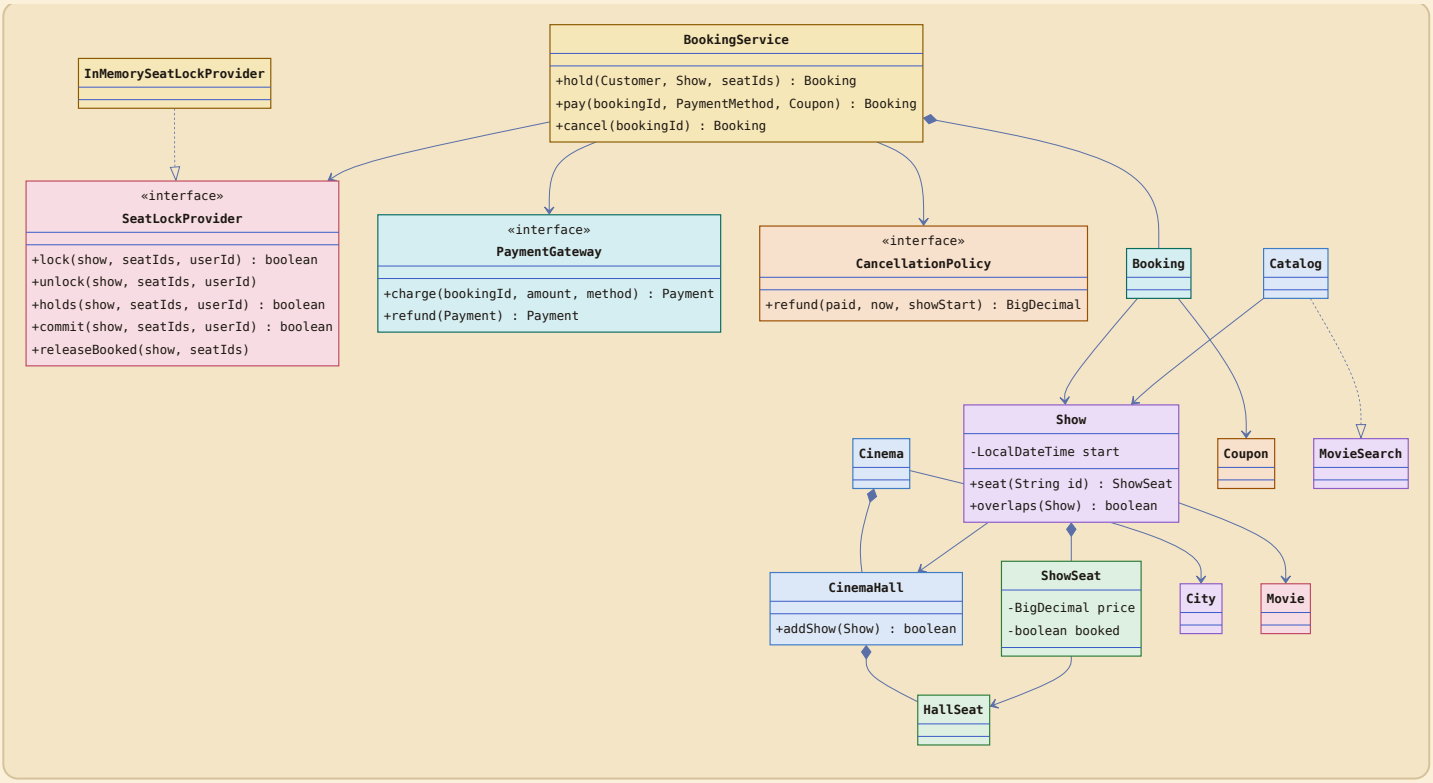


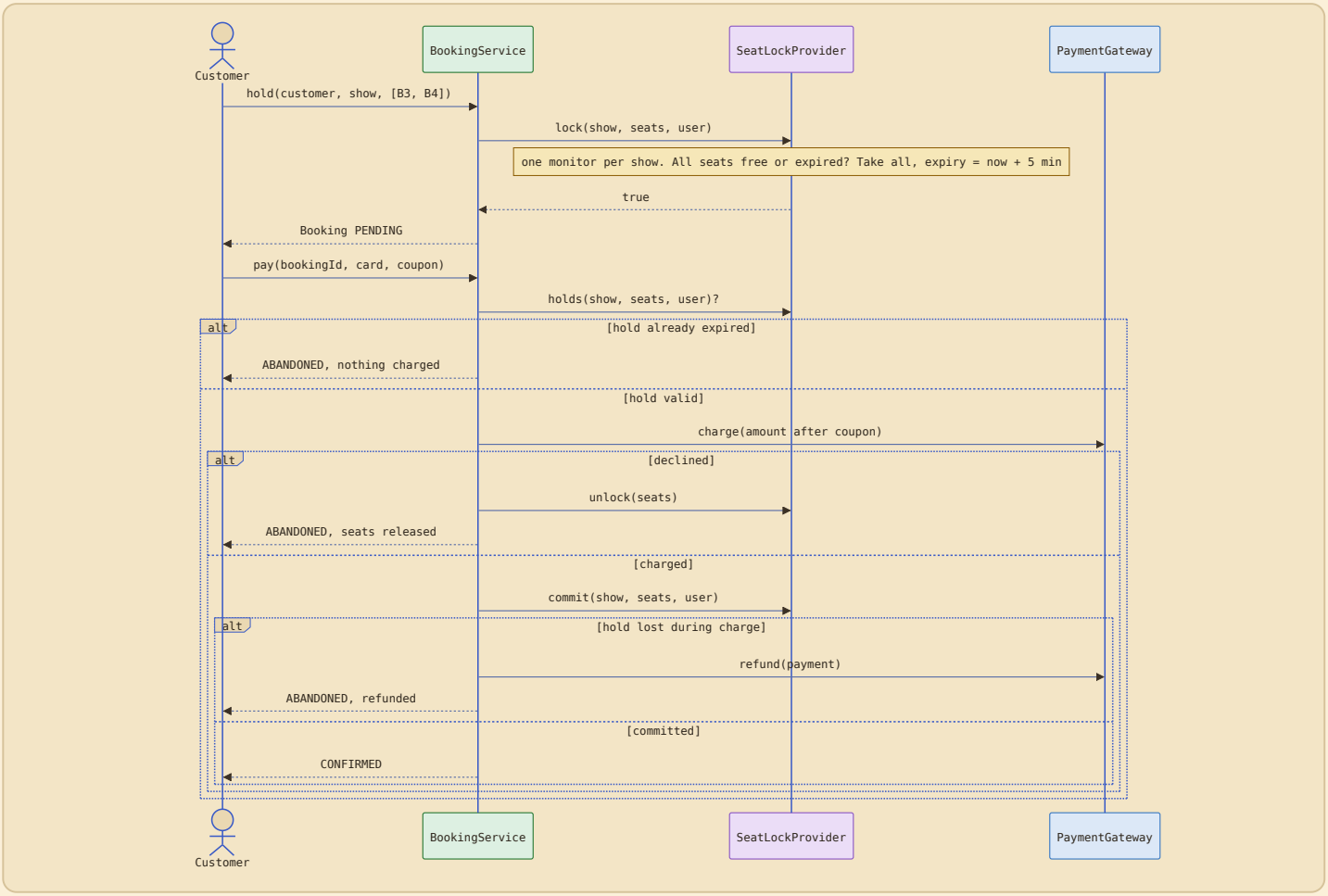
Figure — The finished movie booking class diagram (course diagram)

Our main difference from the course: a separate `SeatLockProvider` with **TTL holds** (the course's biggest self-named omission).



Key flows

The course's activity diagrams cover **make a booking** and **cancel a booking** (refund only within the allowed time). The heart is **hold → pay → commit**:



Why `commit` re-checks inside the lock: payment can take seconds; if the hold expires and someone else grabs the seat, a naive "mark booked" double-books. `commit` verifies "still held by me, not expired" **and** marks

booked atomically. If that fails after money was taken, run a **compensating action** (refund) — a card charge can't be rolled back. This is the **Saga** idea: each step has an undo.

Booking lifecycle: `PENDING → CONFIRMED` (paid and committed) `→ CHECKED_IN` (scanned) or `→ CANCELED` (refund per policy); `PENDING → ABANDONED` (expired, declined, or lost and refunded) or `→ CANCELED`. On the seat map a seat is free (`.`), **held** (`L`) or booked (`X`); "held" lives only in the lock provider and disappears on expiry.

Design decisions & patterns used

Decision	Why	Alternative rejected
`ShowSeat` per show	Same physical seat free in one show, taken in another; per-show prices.	Booking `HallSeat` — breaks with two shows a day.
`SeatLockProvider` with TTL holds	Abandoned checkouts free themselves; swap in **Redis** (`SET key value NX PX 300000`) or a DB `expires_at` column.	Holding until cancel (course's gap).
All-or-nothing lock	4 seats together, never 2: check all, then take all.	Locking one by one without rollback.
One monitor per show	Shows never block each other; within a show every decision is serialized.	One global lock (course) — correct but serializes the country.
Commit re-validates + compensation	Never pay for a seat you don't get.	Trusting the earlier check.
Idempotent `pay`	Double-click/retry can't charge twice: non-PENDING returns unchanged. Real systems add an **idempotency key**.	Charging on every call.
Strategy refunds, **Strategy-like** `PaymentMethod`	Rules and payment types vary independently.	`if (method == CARD)`.
Adapter `PaymentGateway`	Wraps Stripe/Razorpay; tests use `FakeGateway`.	Provider SDK in `BookingService`.
Coupon on the booking	Amount correct before payment.	Each payment type applying coupons.
Guest has no booking method	"Register to book" enforced by *types*: `hold` takes a `Customer`.	`Guest.book()` that throws.
"Search by city" from shows	A movie is in a city only when a show is scheduled there.	A city on `Movie`.

Java implementation

Package `lld.ch15.moviebooking` (`Venue`, `SeatLockProvider`, `Payments`, `Catalog`, `BookingService`, `Main`).

```
final class InMemorySeatLockProvider implements SeatLockProvider {
    // ...
    @Override public boolean lock(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            Instant now = clock.instant();
            for (String id : seatIds) {                          // 1. check every seat first
                if (show.seat(id).isBooked()) return false;
                SeatLock l = t.get(id);
                if (l != null && !l.expired(now) && !l.userId().equals(userId)) return false;
            }
            SeatLock mine = new SeatLock(userId, now.plus(ttl));
            for (String id : seatIds) t.put(id, mine);           // 2. then take them all
            return true;
        }
    }
    // ...
    @Override public boolean commit(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            if (!holdsLocked(t, seatIds, userId, clock.instant())) return false;
            for (String id : seatIds) { show.seat(id).setBooked(true); t.remove(id); }
            return true;
        }
    }
    // ...
}

final class BookingService {
    // ...
    Booking pay(String bookingId, PaymentMethod method, Coupon coupon) {
        Booking b = get(bookingId);
        synchronized (b) {
            if (b.status() != BookingStatus.PENDING) return b;
            if (!locks.holds(b.show(), b.seatIds(), b.customerId())) {       // fast fail, no charge
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " hold expired before payment, nothing charged");
                return b;
            }
            b.setCoupon(coupon);
            BigDecimal amount = coupon == null ? b.baseAmount() : coupon.apply(b.baseAmount());
            Payment p = gateway.charge(b.id(), amount, method);
            b.setPayment(p);
            if (p.status() != PaymentStatus.COMPLETED) {
                locks.unlock(b.show(), b.seatIds(), b.customerId());           // free seats at once
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " payment " + p.status() + ", seats released");
                return b;
            }
            if (!locks.commit(b.show(), b.seatIds(), b.customerId())) {       // hold lost during charge
                b.setPayment(gateway.refund(p));                              // COMPENSATION
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " hold expired during payment, " + Money.fmt(amount) + "
refunded");
                return b;
            }
            b.setStatus(BookingStatus.CONFIRMED);
            // ... notify CONFIRMED
            return b;
        }
    }
    // ...
}
```

Full runnable code: code/lld/ch15/moviebooking/ (see the Full Edition).

Output (excerpt):

```
  add SH2 RRR 19:00 same hall -> false
== 3. 10 customers click B3+B4 at the same instant ==
  holds granted = 1 (the other 9 saw 'not available')
  paying twice is harmless -> CONFIRMED
  [notify ASHA] BK-2 hold expired before payment, nothing charged
  [notify MEENA] BK-4 hold expired during payment, Rs.200.00 refunded
  [notify NEHA] BK-6 payment DECLINED, seats released
```

The 10% coupon turns Rs.700 into Rs.630; an overlapping show in the same hall is refused; Meena's hold expired while the bank was charging and Kiran took the seat, so she is refunded.

Concurrency & edge cases

- **Atomic check-then-act:** `lock`, `holds`, `commit`, `unlock`, `releaseBooked` for one show all run `synchronized` on that show's lock table.
- **Lazy expiry:** no background thread; an expired `SeatLock` is treated as absent the next time anyone looks (Redis TTL does the same).
- **Payment-window race:** commit re-validation + refund.
- **Lock order:** booking monitor (serialises `pay`/`cancel` — no double charge, no pay-while-cancel) → show monitor, never reversed.
- **Across many servers**, from simplest: (1) **conditional update** (course): `UPDATE show_seat SET status='HELD', held_by=?, held_until=? WHERE show_id=? AND seat_id IN (...) AND (status='FREE' OR held_until < now())`, check the row count; (2) **pessimistic** `SELECT ... FOR UPDATE`; (3) **serializable** transaction (course) — simplest to reason about, slowest under collision; (4) **optimistic** `version` column, retry on conflict (rare conflicts); (5) **unique constraint** on `(show_id, seat_id)` as the final net; (6) **Redis TTL** keys for holds, DB as truth for confirmed seats. The in-process lock only makes the common case quick; with two servers, correctness comes from the database.
- **Edge cases:** empty or duplicate seat ids (de-duplicated), unknown seat, blocked customer, overlapping show, cancelling an abandoned booking, cancelling too late (no refund).

Extensions & follow-up questions

- **Live seat map** → push seat events over WebSockets/SSE (Observer). **Extend the hold** → refresh `expiresAt` only if still held. **Dynamic pricing** → a `PricingStrategy`.
- **Paid but crashed before commit** → store PENDING with the gateway transaction id first; a reconciliation job (plus provider **webhooks**) commits or refunds.
- **Cancel a whole show** → cancel and fully refund every confirmed booking of that show.
- **Seat-gap rule** → a `SeatSelectionRule` chain (Chain of Responsibility) before locking; loyalty points → another `PaymentMethod`.

Real-world example

BookMyShow's payment-screen **countdown timer** is the hold's TTL; its **automatic refund** when the bank debits but booking fails is the compensation path.

In the interview

30–45 minutes: 4 min requirements (hold time, payment, refunds) → 4 min actors → 8 min entities, saying **why** `ShowSeat` → 12 min `SeatLockProvider` (`lock`, `commit`) and `BookingService.hold/pay/cancel` → 5 min payment failure and refund → multi-server concurrency.

> **30-second answer:** "City → Cinema → Hall → HallSeat. A Show owns a ShowSeat per hall seat — that's what's booked. Booking is hold → pay → commit: a SeatLockProvider takes all requested seats atomically with a 5-minute TTL; payment goes through a gateway interface; commit re-checks the hold and marks seats booked atomically, refunding if the hold was lost. In one process a per-show monitor prevents double booking; across servers a conditional update or unique constraint is the real guarantee, with Redis TTL keys for holds."

Follow-ups: *Why TTL?* Otherwise abandoned checkouts block seats forever. *Optimistic or pessimistic?* Pessimistic/conditional update for hot shows; optimistic when conflicts are rare.

Self-check

- **Q:** Why is `ShowSeat` separate from `HallSeat`? **A:** State and price are per show; the physical seat is shared by every show.
- **Q:** Payment succeeded but `commit` returned false? **A:** Refund (compensation), mark abandoned.
- **Q:** How are expired holds cleaned up? **A:** Lazily — any check treats an expired lock as absent.
- **Q:** What makes `pay` safe to call twice? **A:** It runs under the booking's monitor and returns immediately unless PENDING.

TL;DR

Book `ShowSeat`s. Hold (all-or-nothing, TTL) → pay → commit (re-validate atomically) → refund if lost. Coupon on the booking, gateway behind an interface, refund rule as a Strategy. Per-show lock in one JVM; conditional updates, unique constraints and Redis TTL across servers.

15.3 Design a Concert Ticket Booking System

Problem & clarifying questions

Concert ticketing (**Ticketmaster**, **Live Nation**, **SeatGeek**) has the same objects as 15.2 — we reuse them — but a different enemy: **extreme contention**. A stadium sells 50,000 tickets in minutes to a million people arriving at 10:00:00. That changes three things: (1) **fairness** — who tries first (a *virtual waiting room*); (2) **throughput** — locks that serialise everyone become the bottleneck (use *lock-free* CAS); (3) **abuse** — scalpers and bots (*per-user limits*) and fans who miss out (*waitlist*).

Question	Answer we assume
Seat types?	**VIP** and **PREMIUM** numbered seats; **GENERAL_ADMISSION** (GA) standing = a *count*.
Search?	By artist, venue, date, time (as 15.2).
Stampede?	**Virtual waiting room**: queue numbers, batches admitted in order.
Limit?	**4 tickets** per user per concert.
Hold time?	**10 minutes**.
Sold out?	Per-tier **waitlist**; released tickets offered to it first.
Notifications?	Confirmation; "a ticket freed up for you".

Requirements

Functional: show concerts with layout and availability; search; choose seats or a GA quantity and buy; **prevent double booking** under concurrency; **fair access**; secure payments; confirmations; **waitlist** for sold-out events.

Non-functional: extreme write contention; **no overselling** ever; FCFS fairness; per-user caps; self-expiring holds.

Use cases

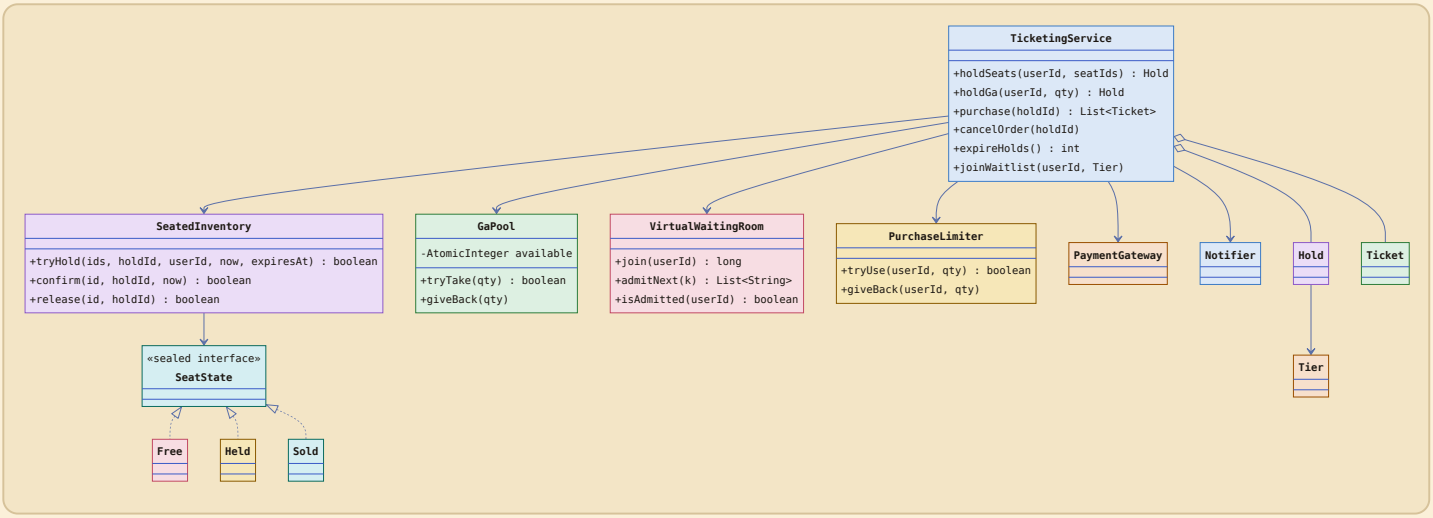
Actors: Fan (join queue, hold, buy, cancel, join waitlist), Organizer/Admin (create concert, tiers, prices, open sale), System (admit batches, expire holds, offer released tickets, notify), Payment provider. **Flow:** join waiting room → admitted → **hold** → **purchase** → tickets; cancel (refund); join waitlist; system **reaps** expired holds.

Core entities

Class	Responsibility
`Concert` (record)	Artist, venue, date/time.
`Tier` (enum)	`VIP`, `PREMIUM` (seated), `GENERAL_ADMISSION` (unseated), each with a price.
`SeatState` (sealed) → `Free`, `Held`, `Sold`	A seat's whole state as one immutable value.
`SeatedInventory`	One `AtomicReference<SeatState>` per seat; claims by **CAS**.
`GaPool`	GA capacity as an `AtomicInteger`; CAS take/give back.
`Hold` (record)	Seats or GA quantity, amount, expiry.
`Ticket` (record)	Tier and seat id or "GA".
`VirtualWaitingRoom`	Arrival-order numbers; admits FIFO batches.
`PurchaseLimiter`	Max N per user, atomic check-and-update.
`TicketingService`	hold → purchase → cancel; reaper; waitlist offers.
`PaymentGateway`, `Notifier`	External boundaries (as 15.2).

Class diagram

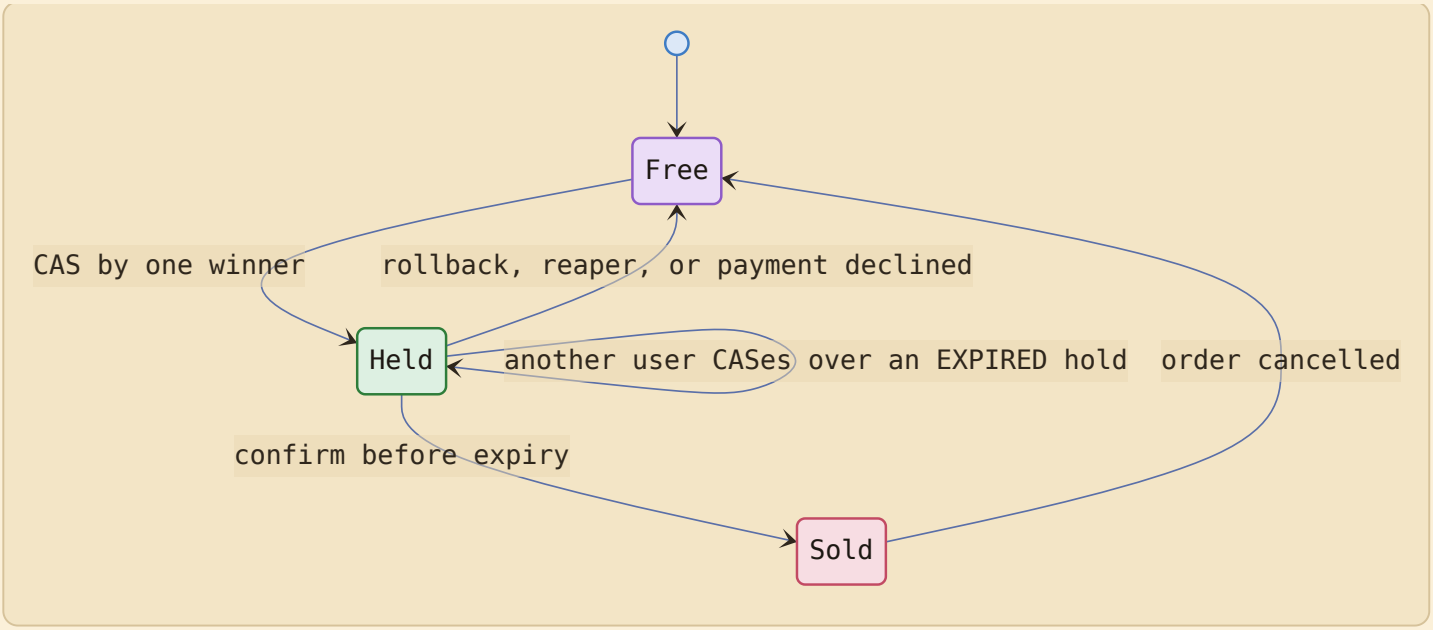
No course lesson here; the problem comes from the *awesome-low-level-design* set. Our design:



Key flows

Buying: fan joins the room (gets e.g. number 41,207; the system calls `admitNext(batch)` every few seconds) → `holdSeats`: is admitted? → `limiter.tryUse` → CAS each seat Free → Held in **sorted order**; if any CAS fails, roll back own Held seats and give back the limit (`SoldOutException`) → `purchase`: remove the hold from the active map (claims it once) → charge → CAS each seat Held → Sold → tickets.

Seat state machine — every arrow is one `compareAndSet`:



Released inventory (expired hold, cancellation) is **offered to the tier's waitlist first**: the next fan is admitted directly and given a fresh hold plus a notification; if unpaid, the reaper releases it to the next.

Design decisions & patterns used

Decision	Why	Alternative rejected
Lock-free CAS per seat	Different seats never wait; on one seat exactly one CAS wins, losers learn instantly; no thread blocks holding a lock, so no deadlock.	One monitor per concert (15.2) — at 50,000 buyers everyone queues on one lock.
Immutable sealed `SeatState`	CAS compares *references*; a fresh record per state means "what I read" is exactly "what I replace"; exhaustive pattern matching.	Mutable `status` / `heldBy` / `expiry` fields — readers see half-updated seats.
Sorted multi-seat hold + rollback	Release only seats *we* took; sorted order makes overlapping buyers contend on the same first seat, so one fails fast.	Nested locks — deadlock-prone.
GA as a counter	Standing places are interchangeable; CAS loop (`if cur < qty fail; else CAS(cur, cur-qty)`) never oversells.	40,000 `Seat` objects.
Virtual waiting room	Arrival order decides who may *try*; batches cap load; bots gain nothing. `join` is `synchronized` so number order = queue order.	"Whoever lands first" — rewards bots, melts the backend.
Per-user limit via `ConcurrentHashMap.compute`	Check-and-update for one key atomically.	`get` then `put` — a race.
TTL holds + reaper	Seats reclaimed lazily (CAS over expired `Held`); GA holds need `expireHolds()`. `holds.remove(id, h)` ensures reaper and `purchase` never both process a hold.	Holds that never expire.
Waitlist offers on release	Longest-waiting fan (FIFO), not fastest refresher.	Silent return to the pool.
Compensation on lost hold	As 15.2: release and refund.	Assuming the hold is valid.
No Singleton	The *awesome-lld* reference uses one; DI is testable and allows one service per concert.	Global Singleton.

Java implementation

Package `lld.ch15.concert` (`Model`, `Inventory`, `TicketingService`, `Main`). Demos use Java 21 **virtual threads** (`Executors.newVirtualThreadPerTaskExecutor()`), making 1,000 concurrent buyers cheap.

```java
/** A seat's whole life in one immutable value, swapped atomically with compare-and-set (CAS). */
sealed interface SeatState permits Free, Held, Sold {}
record Free() implements SeatState {}
record Held(String holdId, String userId, Instant expiresAt) implements SeatState {
    boolean expired(Instant now) { return !now.isBefore(expiresAt); }
}
record Sold(String holdId, String userId) implements SeatState {}

final class SeatedInventory {
    // ...
    private boolean tryHoldOne(String id, String holdId, String userId, Instant now, Instant expiresAt) {
        AtomicReference<SeatState> ref = seats.get(id);
        if (ref == null) throw new IllegalArgumentException("no seat " + id);
        while (true) {
            SeatState cur = ref.get();
            boolean claimable = cur instanceof Free || (cur instanceof Held h && h.expired(now));
            if (!claimable) return false;
            if (ref.compareAndSet(cur, new Held(holdId, userId, expiresAt))) return true;
            // CAS lost: someone changed the seat between get() and CAS. Re-read and decide again.
        }
    }
    // ...
}

/** General admission: no seat numbers, just a counter. A CAS loop means it can never go below zero. */
final class GaPool {
    // ...
    boolean tryTake(int qty) {
        while (true) {
            int cur = available.get();
            if (cur < qty) return false;
            if (available.compareAndSet(cur, cur - qty)) return true;
        }
    }
    // ...
}

final class PurchaseLimiter {
    // ...
    boolean tryUse(String userId, int qty) {
        boolean[] ok = {false};
        used.compute(userId, (u, cur) -> {
            int c = cur == null ? 0 : cur;
            if (c + qty > max) return cur;            // unchanged
            ok[0] = true;
            return c + qty;
        });
        return ok[0];
    }
    // ...
}

final class TicketingService {
    // ...
    List<Ticket> purchase(String holdId) {
        Hold h = holds.remove(holdId);                    // atomic claim: a hold is purchased at most once
        if (h == null) throw new IllegalStateException(holdId + " unknown, expired or already used");
        // ... expired / declined -> releaseInventory(h); then CAS each seat Held -> Sold, refund if lost
    }
    // ...
}
```

Full runnable code: code/lld/ch15/concert/ (see the Full Edition).

Output (excerpt):

```
REFUSED: eli is still in the waiting room
holds on VIP-1 = 1
sold=98 soldOut=902 gaLeft=0 (never negative)
REFUSED: ben would exceed the per-user ticket limit
[notify cai] a GA ticket freed up, held for you as H244 until 2026-06-01T09:10:00Z
expired holds reaped = 3 (VIP-1, cai's unpaid offer, dev's P-6)
```

40 concurrent CAS attempts on one seat → exactly one hold; 1,000 buyers for 98 GA places sell exactly 98; a cancellation goes straight to the waitlisted fan as a hold; money collected 85,900.00 = 98 × 800 + 3 × 2,500.

Concurrency & edge cases

- **Why CAS is correct:** `compareAndSet(expected, new)` replaces only if the value is still the object we read; otherwise re-read and decide. `tryHoldOne` retries only while the seat is still claimable.
- **ABA problem** (A → B → A fooling a CAS)? Every transition creates a *new* record and `AtomicReference` compares references, so a returned "A" is a different object — the CAS fails safely.
- **Hold claimed once:** `purchase` does `holds.remove(holdId)`, the reaper `holds.remove(id, h)` — only one wins.
- **Limit given back** on every failure path (sold out, declined, expired, lost seat).
- **Lost hold after payment:** `confirm` fails → release, give back limit, refund.
- **Fair queue scope:** fair within one process; at scale the waiting room is a separate service (often at the CDN edge) issuing signed, time-limited access tokens.
- **Across servers:** seat CAS → `UPDATE seat SET state='HELD', hold_id=?, expires_at=? WHERE id=? AND (state='FREE' OR expires_at < now())` or Redis `SET NX PX`; GA → Redis `DECRBY` in a Lua script, or `WHERE available >= ?`.

Extensions & follow-up questions

- **Best available seats** → a `SeatAllocator` strategy picks adjacent free seats, then the same CAS hold.
- **Verified fans / presale codes** → admission tokens for pre-registered fans; limiter keyed on verified identity.
- **Dynamic pricing** → a `PricingStrategy` using remaining inventory and time, frozen in the `Hold`.
- **Resale** → transfer ownership and re-issue a new barcode.
- **Waitlist for seated tiers** → same mechanism per released seat (`offerSeat`).

Real-world example

Ticketmaster uses a virtual queue ("Smart Queue"), **Verified Fan** presales and per-account limits. The 2022 *Eras Tour* presale overwhelmed the system and the general sale was cancelled — admission control matters as much as seat locking.

In the interview

30–45 minutes: say "entities like movie booking; the hard part is contention and fairness" → tiers (why GA is a counter) → TTL holds; code the seat CAS and GA CAS loop → waiting room and atomic per-user limit → waitlist and reaper → scale-out (tokens, conditional updates/Redis).

30-second answer: "Seated tiers are an `AtomicReference<SeatState>` per seat claimed by CAS — one winner, no locks. GA is an `AtomicInteger` decremented by a CAS loop, never negative. Everyone enters through a virtual waiting room admitting in arrival order, with an atomic per-user limit. Holds expire after 10 minutes, a reaper returns them, and released tickets go to a FIFO waitlist first. A hold lost during payment is refunded."

Follow-ups: *Lock-free vs locks?* Lock-free wins under high contention with tiny critical sections; locks are simpler when many things change together. *Queue service fails?* Fail closed — pause admissions. *One person, 50 accounts?* Verified identity, card fingerprinting, limits on those.

Self-check

- **Q:** Why is GA a counter? **A:** Places are interchangeable; a CAS counter is simpler and can't oversell.
- **Q:** What does `compareAndSet` guarantee? **A:** Update only if the value is still the one you read.
- **Q:** Why must `join` be synchronized? **A:** So number and queue position are assigned together, in the same order.

- **Q:** How can't a hold be both purchased and reaped? **A:** Both remove it from `holds` first; only one removal succeeds.

> **TL;DR**

Same nouns as movie booking, different enemy: contention. CAS per seat, CAS counter for GA, FIFO virtual waiting room, atomic per-user limits, TTL holds with a reaper, waitlist first for released tickets, refunds when a hold is lost.

15.4 Design a Hotel Management System

Problem & clarifying questions

A **Hotel Management System** (property management system, PMS) knows every room, takes bookings, checks guests in and out, issues key cards, records housekeeping and room service, and produces the bill. Unlike a cinema, a room is sold **night by night** — that shapes the whole design.

Question	Answer we assume
Locations, types, search?	Many `HotelLocation`s; `STANDARD`, `DELUXE`, `FAMILY_SUITE`, `BUSINESS_SUITE`; search by type and **date range**.
Cancellation?	**Full refund if more than 24 h before check-in**; otherwise none (our assumption).
Extras?	Room service, kitchen, amenities on the bill.
Payment?	**Card, check or cash**.
Keys?	Electronic cards with barcode and key id; deactivated at check-out.
Housekeeping?	Logged per room, never overwritten.
Several rooms per booking?	No — one room (a known limitation).

> **Requirements**

Functional (course): (1) book **room types**; (2) **search** and book any **available** room; (3) **who booked a room** / **which rooms a customer booked**; (4) **cancel**, full refund > 24 h before check-in; (5) **notify** before check-in and check-out; (6) **housekeeping log**; (7) **room service and food**; (8) **amenities**; (9) pay by **card, check or cash**.

Non-functional: never double-book a room-night; bill = sum of lines; actors limited to their jobs; refund rule in one place.

Use cases

Actors (course): Guest (search, book), Receptionist (rooms, bookings, check in/out), System (notifications), Manager (hires workers), Housekeeper (housekeeping records), Server (room-service records). **Top use cases:** manage rooms; search; register/cancel account; book; **check in**; **check out** (return keys); add room charge; update housekeeping.

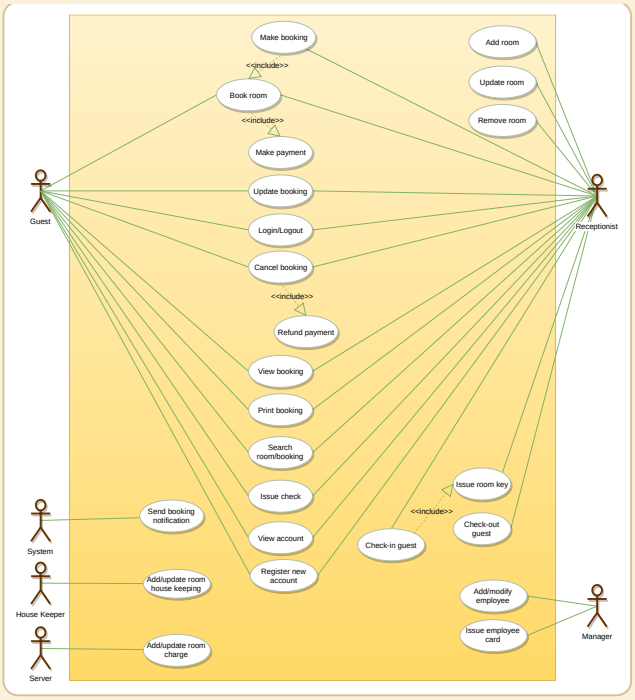


Figure — Hotel use cases: Guest and Receptionist share booking, update, cancel, view, print and search; "Make booking" includes "Book room", which includes "Make payment"; "Cancel booking" includes "Refund payment"; the receptionist's "Check-in guest" includes "Issue room key"; the System sends booking notifications; the House Keeper adds/updates housekeeping; a manager adds/modifies employees (course diagram)

Core entities

Class	Responsibility
`Hotel`	Owns locations; searches each.
`HotelLocation` implements `Search`	Owns rooms; "which rooms of this type are free for these dates?".
`Room`	Number, style, price; the **set of booked nights**; owns keys and housekeeping log; current `RoomStatus`.
`RoomStyle`, `RoomStatus`	Type; what it is doing *now* (`AVAILABLE`, `RESERVED`, `OCCUPIED`, `NOT_AVAILABLE`, `BEING_SERVICED`, `OTHER`).
`RoomKey`	Key id, barcode, issue time, master, active.
`RoomHouseKeeping` (record)	When, who, what, how long.
`RoomBooking`	Reservation number, guest id, room, dates, status, deposit; **owns its** `Invoice`; `refundDue()`.
`BookingStatus`	`REQUESTED`, `PENDING`, `CONFIRMED`, `CHECKED_IN`, `CHECKED_OUT`, `CANCELLED`, `ABANDONED`.
`Invoice`, `InvoiceItem`	Lines; total computed.
`RoomCharge` → `Amenity`, `RoomService`, `KitchenService`	Writes itself onto the invoice (Template Method).
`Payment` (sealed) → `CreditCardTransaction`, `CheckTransaction`, `CashTransaction`	Three genuinely different behaviours.
`Person` (sealed) → `Guest`, `Receptionist`, `Server`, `Housekeeper`, `Manager`	Actors; methods accept only the right type.
`FrontDesk`	Book, check in, add charge, check out, cancel, housekeeping, reminders, queries.
`Notifier`	Messages to guests.

Class diagram

Five course stages: (1) **invoice, payments, notifications** — the invoice stores **lines** and computes its total, so it can't drift from them (req. 7, 8); (2) **room, keys, housekeeping** — the key decision: **a status field is not availability** — `OCCUPIED` describes now, but guests ask about future date ranges, so each room stores its sold nights; (3) **hotel, location, search** — search sits on the **location** (a room can't list other free rooms; `Room.search()` is a common beginner mistake); (4) **booking and room charges**; (5) **people** — there is deliberately **no arrow guest ↔ booking**: the booking stores the guest's **id**, and queries filter by it.

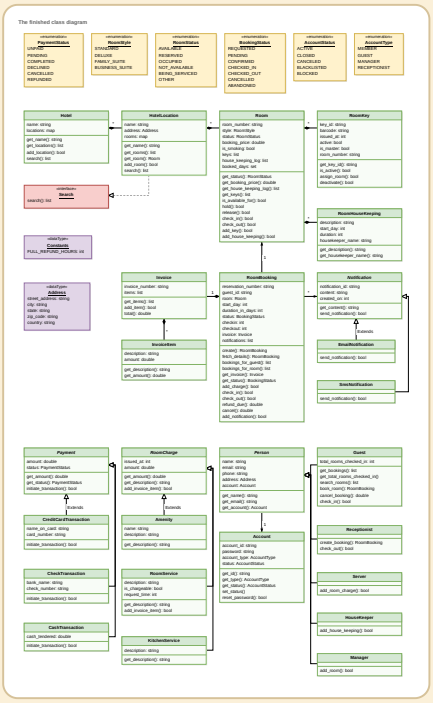
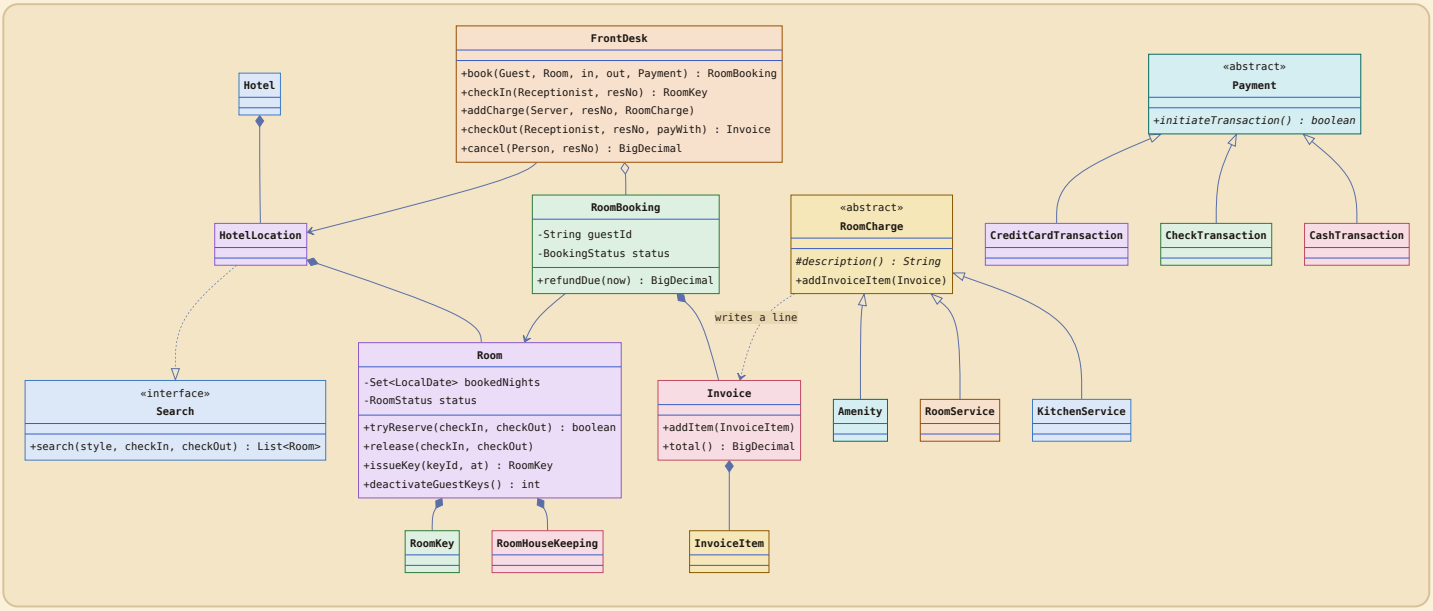


Figure — The finished hotel class diagram (course diagram)



Key flows

The course's activity diagrams: **make a booking** (search type and dates → none: tell customer; else find/register customer, book, pay, deliver details), **check in** (find booking; room not ready → wait; else issue key, status "Checked-in"), **cancel** (find booking; before the deadline → refund, else none; print/email).

A whole stay: guest `book(room 102, Jul 10–13, deposit)` → `tryReserve` under the room monitor (nights 10, 11, 12 taken) → CONFIRMED → receptionist `checkIn` (room OCCUPIED, key issued) → server `addCharge(KitchenService)` writes an invoice line → `checkOut(pay by card)` adds the nights line and subtracts the deposit, deactivates keys, room BEING_SERVICED → invoice total = sum of lines.

Booking lifecycle: `CONFIRMED → CHECKED_IN → CHECKED_OUT`, or `CONFIRMED → CANCELLED` (refund if > 24 h ahead). The room's own status cycles independently: `AVAILABLE → OCCUPIED → BEING_SERVICED → AVAILABLE` (after housekeeping).

Design decisions & patterns used

Decision	Why	Alternative rejected
Availability = set of booked nights	Answers date ranges; nights are half-open `[checkIn, checkOut)`, so leaving and arriving on the 13th don't clash.	`RoomStatus` — only knows now.
`tryReserve` `synchronized` on the room	Check-then-act made atomic; rooms don't block each other.	Hotel-wide lock.
Search on `HotelLocation`	A room can't answer "which rooms are free?".	`Room.search()`.
Invoice = lines, total computed	Total can't disagree with lines.	Stored running total.
Template Method `RoomCharge.addInvoiceItem`	`final` parent algorithm (describe → price → write line); subclasses supply only `description()`; new charge = new subclass (OCP). The course calls it its clearest Template Method; Strategy swaps a *whole* method instead.	`switch` on charge type in the invoice.
Three `Payment` subclasses	Card completes at once; **check stays `PENDING`**; cash declines if short. Each passes LSP.	One `Payment` with a `type` field.
Refund rule in one method (`refundDue`)	One place to edit; promote to a `CancellationPolicy` strategy (15.2) for several policies.	Scattered refund arithmetic.
Actor types in signatures	`addCharge(Server, …)`, `logHousekeeping(Housekeeper, …)`, `addRoom(Manager, …)` — compiler-enforced roles; a receptionist booking for a guest calls the *same* method.	Anyone calls anything.
Booking stores guest id	No two-way dependency; "rooms of this guest" is a query.	Bidirectional links.
Nights released on failed deposit / cancel	Reserve-then-pay needs an undo.	Phantom reservations.

Java implementation

Package `lld.ch15.hotel` (`Billing`, `Rooms`, `Stay`, `Main`).

```
final class Room {
    // ...
    /** Nights are [checkIn, checkOut): the checkout day itself is free for the next guest. */
    synchronized boolean isFree(LocalDate checkIn, LocalDate checkOut) {
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) if (bookedNights.contains(d)) return
false;
        return true;
    }
    synchronized boolean tryReserve(LocalDate checkIn, LocalDate checkOut) {
        if (!isFree(checkIn, checkOut)) return false;              // re-entrant monitor: same lock
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) bookedNights.add(d);
        return true;
    }
    // ...
}

abstract class RoomCharge {
    private final BigDecimal amount;
    RoomCharge(BigDecimal amount) { this.amount = amount; }
    protected abstract String description();                // the one step that varies
    final void addInvoiceItem(Invoice invoice) {            // the template: final, so the order can't change
        invoice.addItem(new InvoiceItem(description(), amount));
    }
}

final class RoomBooking {
    // ...
    /** The refund rule lives in exactly ONE method: full refund if cancelled more than 24h before check-in. */
    BigDecimal refundDue(LocalDateTime now) {
        Duration notice = Duration.between(now, checkIn.atTime(CHECK_IN_TIME));
        // compareTo, not toHours(): toHours() truncates, so 24h30m would wrongly count as "24, not more than 24".
        return notice.compareTo(FULL_REFUND_NOTICE) > 0 ? deposit() : BigDecimal.ZERO;
    }
}

final class FrontDesk {
    // ...
    /** Guest or receptionist books: same call, two actors. The room's monitor makes it race-free. */
    RoomBooking book(Guest guest, Room room, LocalDate checkIn, LocalDate checkOut, Payment deposit) {
        if (!checkIn.isBefore(checkOut)) throw new IllegalArgumentException("check-out must be after check-in");
        if (!room.tryReserve(checkIn, checkOut))
            throw new IllegalStateException("room " + room.number() + " is not free " + checkIn + ".." + checkOut);
        RoomBooking b = new RoomBooking("R" + seq.incrementAndGet(), guest.id(), room, checkIn, checkOut);
        if (!deposit.initiateTransaction()) {                  // payment failed: give the nights back
            room.release(checkIn, checkOut);
            throw new IllegalStateException("deposit " + deposit.status());
        }
        // ... store booking, notify
    }
    // ...
}
```

Full runnable code: code/lld/ch15/hotel/ (see the Full Edition).

Output (excerpt):

```
  successful bookings = 1 of 6
  [notify BEN] R3 confirmed: room 102 2026-07-13..2026-07-15
  [notify CARL] R5 cancelled by Carl, refund $0.00
  [notify ASHA] R4 cancelled by Asha, refund $250.00
  cash short: payment DECLINED, guest still checked in
    TOTAL                          $299.00
  key active after checkout? false, room 102 is BEING_SERVICED
```

Six concurrent overlapping bookings of room 101 → one wins; Ben's stay from the 13th fits after Asha's ending the 13th; Carl cancels 19 h ahead (no refund), Asha 8 days ahead (full); the bill is $5 + $24 + $30 + 3 × $120 − $120 deposit = **$299**, and a declined cash payment doesn't duplicate lines on retry.

Concurrency & edge cases

- **Double-booking a night:** `Room.tryReserve` is `synchronized` — check and add of every night are one step; the 6-thread race has one winner.
- **Search is advisory:** the room may be gone a millisecond later; `tryReserve` is the real check.
- **Booking transitions** (`checkIn`, `checkOut`, `cancel`) run under the booking's monitor; lock order booking → room.
- **The 24-hour rule, exactly:** compare `Duration`s; `toHours() > 24` truncates minutes, so 24 h 30 min would wrongly lose its refund.
- **Retry-safe checkout:** nights and deposit lines are written only after payment succeeds; `addCharge` takes the same booking monitor, so no charge slips in after totalling.
- **Edge cases:** check-out not after check-in; checking in too early; charges when not in house; cancelling after check-in; short cash (declined); a check stays `PENDING`.
- **Chain-wide inventory:** per-room nights don't scale across channels; real systems use an **inventory table** keyed by `(room_type, date)` with `UPDATE ... SET sold = sold + 1 WHERE sold < capacity`.

Extensions & follow-up questions

- **Two rooms in one booking** (course flags it) → a `Reservation` holding several `RoomStay`s; all-or-nothing across rooms by locking in room-number order, or one DB transaction.
- **Book by type, assign at check-in** → per-type-per-night counts for selling; pick the physical room at check-in.
- **Seasonal rates** → a `RatePlan`/`PricingStrategy`, one line per night.
- **Overbooking** → sell capacity × (1 + factor); "walk" extras to a partner hotel — a strategy on the availability check.
- **Booker vs occupants** (course names it missing) → `Occupant`s on the stay. **Lost key** → `deactivateGuestKeys()` + `issueKey()`.

Real-world example

Oracle OPERA sells room types by date, assigns rooms at check-in, and posts charges to the folio; key cards (VingCard, dormakaba) expire at check-out; **Booking.com** shows per-rate policies like "free cancellation until 24 hours before arrival".

In the interview

30–45 minutes: clarify (types, date search, 24 h refund, charges, payments, keys, housekeeping) → actors → entities, saying early **"a status field is not availability"** → code `tryReserve`, `book`, `checkIn`, `checkOut` → Template Method charges, payment subclasses, refund rule → follow-ups (two rooms, chain inventory, overbooking).

> **30-second answer:** "A `Hotel` has `HotelLocation`s that own `Room`s and search by type and date range. Each room keeps its booked nights, and reserving nights is a synchronized check-then-act. A `RoomBooking` owns an `Invoice` of lines; room service, kitchen and amenities are `RoomCharge` subclasses adding a line via Template Method. Payment is card, check or cash subclasses. Cancellation refunds in full if more than 24 hours before check-in — one method. Check-in issues a key; check-out totals the invoice, takes payment, kills the keys and hands the room to housekeeping."

Self-check

- **Q:** Why isn't `RoomStatus` enough? **A:** It describes now; a booking asks about future nights.
- **Q:** Why half-open `[checkIn, checkOut)`? **A:** The checkout day's night isn't used, so the next guest can arrive that day.
- **Q:** Which pattern adds a charge type without touching `Invoice`? **A:** Template Method.
- **Q:** How is "only a server adds room service" guaranteed? **A:** `addCharge` takes a `Server`; other types don't compile.

TL;DR

Rooms are sold night by night: each room keeps booked nights and reserves them atomically. Search on the location. The bill is lines written by `RoomCharge` Template Methods; three payment subclasses; one 24-hour

refund method. Keys issued at check-in, killed at check-out; housekeeping is append-only.

Chapter summary

Problem	Inventory unit	How a hold/booking is made safe	Payment failure handling	Cancellation
Library	`BookItem` (copy)	`synchronized` member → item (fixed order)	— (fines recorded)	Return → FIFO waitlist per title gets the copy
Movie tickets	`ShowSeat` (seat × show)	Per-show monitor; all-or-nothing lock with TTL; atomic commit	Declined → release; lost hold → refund (compensation)	Refund per `CancellationPolicy`; seats back on sale
Concert tickets	Numbered seat state / GA counter	Lock-free CAS; waiting room; per-user limit	Same compensation as movies	Released tickets offered to waitlist first
Hotel	Room × night	`synchronized` `Room.tryReserve` over a date range	Failed deposit → nights released	Full refund if > 24h before check-in

Five sentences to remember for any booking interview:

1. Name the *real* unit of inventory first (copy, show seat, room-night, GA count).
2. Every "if free then take" must be one atomic step — a monitor, a CAS, or a conditional `UPDATE`.
3. Temporary holds need an expiry, or abandoned checkouts block inventory forever.
4. Money can't be rolled back — failures after payment need a compensating refund.
5. In-process locks protect one server; with many servers, the database (constraints, conditional updates) or a TTL store like Redis is the real guarantee.

Chapter 16 — Case Studies: Booking & Inventory (Part 2) — Study Edition

> **Where this fits.** Chapter 15 covered the first group of booking problems. This chapter finishes the family with four more: car rental, restaurant, airline and course registration. They are one problem underneath: **a limited resource, a calendar, and many people trying to claim it at once.** Spot that skeleton and every new booking problem is quick to set up.

The shared skeleton of every booking problem

Idea	Car rental	Restaurant	Airline	Course registration
The thing being booked	a vehicle	a table	a seat on one flight instance	a seat in one section
The "calendar"	days	hourly time slots	the flight instance (route + date)	the term
The atomic check-then-claim	`Vehicle.tryBook`	`Table.tryHold`	`FlightInstance.holdSeats`	`Section.enroll`
When it is full	search another car	try the next table	another seat or flight	waitlist
Money	bill lines + late fee	check split by seat + tax	fare per seat class, paid per itinerary	none (fees are an extension)

A **resource** is anything in limited supply. A **race condition** is a bug where the result depends on which thread runs first — here, two people both "win" one seat because both checked "free?" before either wrote "taken".

How to run: `cd chapters/16-case-booking-2/code && javac --release 21 -d /tmp/lld_16 $(find . -name '*.java') && java -cp /tmp/lld_16 lld.ch16.carrental.Main` (or `restaurant`, `airline`, `courseregistration`). Where threads race, the *counts* are always the same; *which* racer wins varies. One file sometimes holds several small package-private types (legal Java; production code would split them).

> **Money.** All prices use `BigDecimal`, never `double` — a `double` cannot store 0.1 exactly, so sums drift by fractions of a cent.

Design a Car Rental System

Problem & clarifying questions

A **car rental system** rents vehicles for hours to weeks from many branches; a car may be returned to a different branch.

Question	Assumed answer
Vehicle kinds?	Car, truck, SUV, van, motorcycle.
Branches?	Many, each with its own vehicles; drop-off branch may differ from pick-up.
Price?	Daily rate per vehicle type + add-ons + a late fee per day late.
Add-ons?	Insurance, equipment (navigation, child seat, ski rack), services (roadside assistance, wifi), additional drivers.
Identification?	Unique barcode, scanned at pick-up and return.
History?	A vehicle log of every event (reserved, picked up, returned, fuelling, cleaning, damage).
Concurrency?	Two members must never get the same car on overlapping days.

Requirements

Functional: (1) rent car/truck/SUV/van/motorcycle; (2) add each vehicle with a unique barcode and details incl. parking stall; (3) answer "who took this vehicle?" and "what has this member rented?"; (4) late fee after the due date; (5) search and reserve any available vehicle; (6) notify when pick-up is near, due date is near, and when overdue; (7) read barcodes; (8) cancel reservations; (9) vehicle log; (10) rental insurance; (11) extra equipment; (12) services.

Non-functional: no double booking for overlapping days; new add-ons or pricing rules without changing reservation code (Open/Closed); auditability — every charge is a bill line, every event a log entry.

Use cases

Actors: **Receptionist** (adds/edits vehicles and workers, can reserve for a customer), **Member** (searches, reserves, picks up, returns), **Worker** (handles returns, updates the vehicle log), **System** (reminders, overdue alerts, cancellations). Top use cases: add/edit vehicle; search by type and availability; register/cancel membership; reserve; check out; return; add equipment/services; update log. Reserving is shared by member and receptionist; the log belongs to the worker.

Core entities

Class	Responsibility
`CarRentalSystem`	Owns the branches and the reservation service.
`CarRentalLocation`	One branch with its own `VehicleInventory`.
`VehicleInventory`	Indexes by barcode, type and model; "which SUVs are free on these days?".
`Vehicle`	Barcode, plate, type, parking stall, mileage, **the set of booked days**, its log.
`VehicleLogEntry`	One event (date, type, note).
`DateRange`	Half-open `[start, end)`; `end` is the due date.
`Reservation`	Member, vehicle, dates, pick-up/return branch, add-ons, drivers, bill, status; holds the business rules.
`ReservationAddOn` (+ `RentalInsurance`, `Equipment`, `Service`), `AdditionalDriver`	Things that add a charge; a named extra driver (no charge).
`Bill`, `BillItem`	The one place that knows what a rental costs.
`PricingStrategy`, `LateFeePolicy`	Pluggable base-charge and late-fee rules.
`ReservationService`	Creates reservations atomically, answers "who rented what", runs reminders.
`Notifier`, `PaymentMethod`	Ports for messages and money.

Class diagram

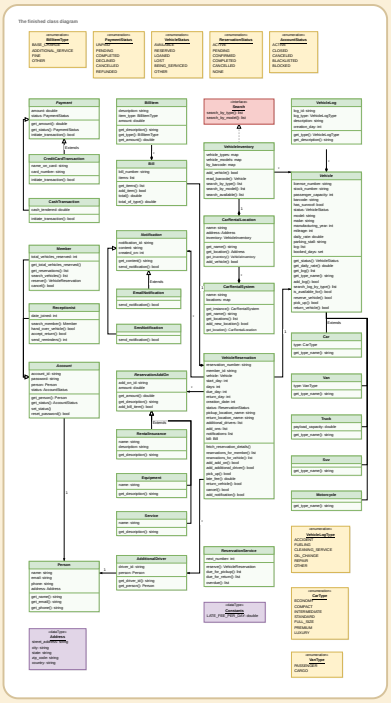
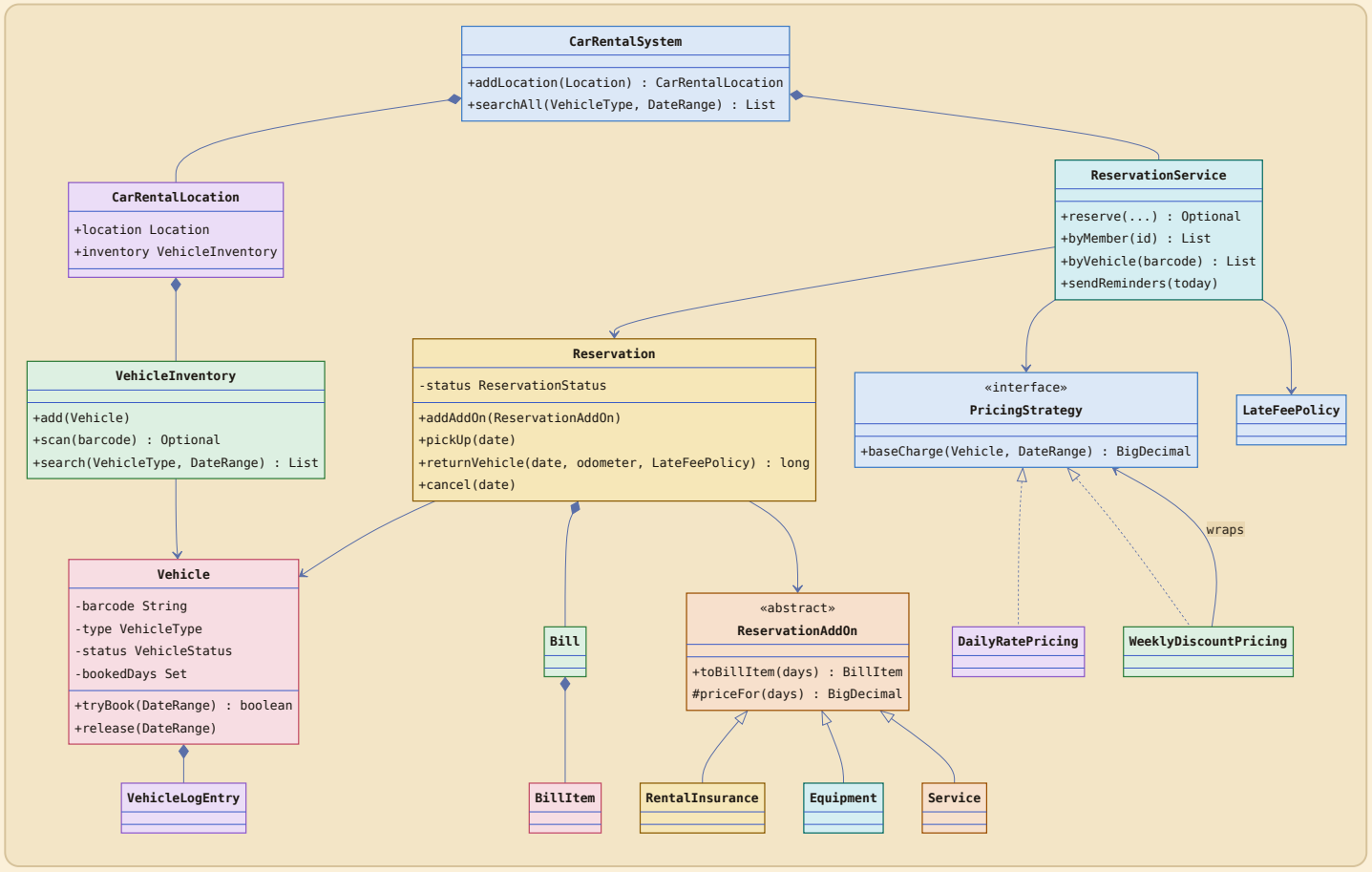


Figure — The finished class diagram of the car rental system (course diagram)

The system owns its branches and each branch owns its inventory (**composition** — the part dies with the whole); the inventory only *refers to* vehicles (**association**). A reservation owns its bill but refers to the vehicle. Our Java design differs in two ways: vehicle kinds are an enum (the five subclasses added no behaviour), and pricing is a strategy object.



Key flows

- **Pick up:** scan licence (or search by name) → check reservation → scan vehicle barcode → hand over keys; no reservation → reserve a free car on the spot.

- **Return:** worker scans the barcode → past due date adds a late fee (one line in `returnVehicle`) → check damage and fuel → settle the bill.
- **Reserve:** `ReservationService.reserve` calls `vehicle.tryBook(dates)`; if false the member picks another car; if true it prices via `PricingStrategy`, creates the reservation (bill gets a BASE line) and notifies.
- **Lifecycle (state machine):** `CONFIRMED →pickUp→ ACTIVE →returnVehicle→ COMPLETED`; `CONFIRMED →cancel→ CANCELLED` (days released on cancel and return).

Design decisions & patterns used

Decision	Why	Alternative rejected
Availability = booked days per vehicle, not a status	Members ask about future dates; a car loaned now may be free next week, a car on the lot may be promised next week.	A `RESERVED` status (can't answer date questions; one reservation per car).
Keep `VehicleStatus` too	Staff need "where is it now" (lot, loaned, in service, lost).	Dropping it.
`tryBook` checks **and** books in one `synchronized` method	Closes the check-then-act race.	`isFree()` then `book()`.
Template Method on `ReservationAddOn`	Billing written once in `final toBillItem`; subclasses give `kind` and may override the `priceFor` hook.	Type enum + `switch`.
Strategy for pricing and late fee; **Decorator** `WeeklyDiscountPricing`	Swap/stack rules without touching the reservation.	Constants; a subclass per combination.
`Bill` is the only place charges live; methods `synchronized`, `settle` idempotent	One total; no line added mid-payment; a double-click pays once.	Per-add-on running totals.
`ReservationService` separate from `Reservation`	Creating/indexing vs one reservation's rules (SRP).	Static factory.
Actors call the rules, never compute them	Member calls the service; receptionist calls the reservation.	`Receptionist.computeLateFee()`.
Course's `CarRentalSystem` **Singleton** → we pass one in	Same "one per app", but testable (dependency injection).	`getInstance()`.

Java implementation

Package `lld.ch16.carrental`. Full runnable code: code/lld/ch16/carrental/ (see the Full Edition).

```java
// Vehicle.java — check AND book inside one lock
synchronized boolean isFreeFor(DateRange r) {
    return r.dates().noneMatch(bookedDays::contains);
}

synchronized boolean tryBook(DateRange r) {
    if (!isFreeFor(r)) return false;
    r.dates().forEach(bookedDays::add);
    return true;
}

synchronized void release(DateRange r) { r.dates().forEach(bookedDays::remove); }

// Billing.java — Template Method
abstract class ReservationAddOn {
    // ...
    final BillItem toBillItem(long rentalDays) {
        return new BillItem(kind() + ": " + name, BillItemType.ADD_ON, priceFor(rentalDays));
    }

    protected abstract String kind();

    /** Hook: by default an add-on is charged per rental day. */
    protected BigDecimal priceFor(long rentalDays) { return price.multiply(BigDecimal.valueOf(rentalDays)); }
}

/** A service such as roadside assistance or wifi is a flat fee per rental. */
final class Service extends ReservationAddOn {
    Service(String name, BigDecimal flatFee) { super(name, flatFee); }
    protected String kind() { return "Service"; }
    @Override protected BigDecimal priceFor(long rentalDays) { return price; }
}

// Pricing.java — Decorator over any pricing: 10% off rentals of 7 days or more.
final class WeeklyDiscountPricing implements PricingStrategy {
    private final PricingStrategy inner;
    WeeklyDiscountPricing(PricingStrategy inner) { this.inner = inner; }
    public BigDecimal baseCharge(Vehicle v, DateRange p) {
        BigDecimal base = inner.baseCharge(v, p);
        return p.days() >= 7 ? base.multiply(new BigDecimal("0.90")).setScale(2, RoundingMode.HALF_UP) : base;
    }
}

// Reservation.java — the rules live here
synchronized long returnVehicle(LocalDate on, int odometer, LateFeePolicy lateFee) {
    require(ReservationStatus.ACTIVE, "return");
    long daysLate = Math.max(0, ChronoUnit.DAYS.between(period.end(), on));
    if (daysLate > 0) {
        bill.add(new BillItem("Late fee: " + daysLate + " day(s)", BillItemType.LATE_FEE,
                lateFee.fee(vehicle, daysLate)));
    }
    vehicle.setMileage(odometer);
    vehicle.setStatus(VehicleStatus.AVAILABLE);
    vehicle.release(period);
    // ... log RETURNED, status = COMPLETED
}

// RentalSystem.java — ReservationService
Optional<Reservation> reserve(Member m, Vehicle v, DateRange period, Location pickup, Location dropoff, LocalDate
today) {
    if (!v.tryBook(period)) return Optional.empty();
    var r = new Reservation("R" + seq.incrementAndGet(), m, v, period, pickup, dropoff, pricing.baseCharge(v,
period));
    reservations.put(r.id(), r);
    v.addLog(today, VehicleLogType.RESERVED, r.id() + " for " + period);
    notifier.send(m, "Reservation " + r.id() + " confirmed: " + v.model() + " " + period);
    return Optional.of(r);
}
```

Output excerpt:

```
3) 8 members race for VH-002 on the same dates
   successful reservations for VH-002: 1
6) Alice's bill
   Base: RAV4 x 3 days                      BASE_CHARGE   240.00
   Insurance: Collision damage waiver       ADD_ON        45.00
   Equipment: Child seat                    ADD_ON        24.00
   Service: Roadside assistance             ADD_ON        25.00
   Late fee: 2 day(s)                       LATE_FEE      80.00
   TOTAL 414.00
   paid: true, settle again (double click) -> true, no second charge
10) A 7-day Civic rental gets the weekly discount
   base charge 315.00 (7 x 50 = 350, minus 10%)
```

Eight threads race for one car and exactly one wins; the late fee is 2 × 40.00; a second `settle` does not charge again.

Concurrency & edge cases

- **Double booking** (a gap the course admits): `tryBook` is `synchronized` per vehicle, so check and mark happen as one step; per-vehicle locks let different cars book in parallel. In a database: unique key on `(vehicle_id, day)` or `SELECT … FOR UPDATE`.
- **Search is advisory** — `reserve` may still fail; show "no longer available".
- **Late return collides with the next rental** — keep a buffer day; reassign the next booking to another car of the same type.
- **Early return** releases the whole range; refunds are policy. **Cancel after pick-up**, **add-ons after the rental**, **invalid ranges**, **odometer going backwards** are all rejected.
- **Different return branch** — recorded, but the car stays in the pick-up inventory: a known gap (transfers).

Extensions & follow-up questions

- **One-way rentals?** On return at another branch move the vehicle between inventories, log a `TRANSFER`, add a one-way fee line.
- **Seasonal/dynamic pricing?** A `SeasonalPricing` strategy or another decorator; reservation unchanged.
- **Deposits?** `authorize` at pick-up, `capture` at return. **Mileage/fuel/damage?** `FINE` bill lines. **Reminders?** A daily scheduler calls `sendReminders(today)`. **Scale?** Partition by branch, `(vehicle_id, day)` unique key, search on a replica/index.

In the interview

30-second answer: "Branches own inventories indexed by barcode, type and model. Availability is a question about *dates*, so each vehicle keeps its booked days and `tryBook` checks and books atomically. A reservation owns a bill; every charge is a line. Add-ons share a template-method base; pricing and late fees are strategies."

Plan (40 min): 0–5 clarify; 5–10 actors and entities; 10–20 class diagram ("dates, not status"); 20–35 code `tryBook`, `Reservation`, `reserve`; 35–40 concurrency, late collisions, transfers. Likely follow-ups: book next month while loaned now? (yes — dates); two clicks at once? (atomic `tryBook`); new add-on? (one subclass).

Self-check

1. **Q:** Why isn't a `RESERVED` status enough for "free next week?" **A:** Status describes *now*; only per-day bookings answer date questions.
2. **Q:** Pattern and hook in `toBillItem`? **A:** Template Method; `priceFor` is the hook (`Service` makes it flat).
3. **Q:** Why one `tryBook` not `isFree` + `book`? **A:** Atomic check-then-act under the vehicle lock.
4. **Q:** How is the late fee computed? **A:** Days from `period.end` to return × `LateFeePolicy` rate, as a `LATE_FEE` line.

TL;DR

- Availability = booked **days** per vehicle; status = where it is now.

- `tryBook` = atomic check-and-book per vehicle.
- One bill per reservation; late fee = days late × rate.
- Template Method (add-ons), Strategy + Decorator (pricing). Gaps: branch transfers, buffer between rentals.

Real-world example. Hertz or Enterprise sell a *car class* ("intermediate SUV or similar"), counting availability per type per day and binding a specific vehicle at the counter — the same design one level up.

Design a Restaurant Management System

Problem & clarifying questions

A **restaurant management system** runs tables, reservations, orders, the kitchen, and the bill.

Question	Assumed answer
One restaurant or chain?	Several branches, each with its own menu, tables, staff.
Menu?	Menu → sections → items with prices.
Orders?	Waiter creates an order per table and adds meals **per seat**; meal items point at menu items.
Reservations / walk-ins?	Search tables by time and party size and reserve; cancel; reminders; show tables free now.
Layouts?	Several seating arrangements per branch ("Weekday", "Banquet").
Payment?	Card, cheque or cash.
Kitchen?	Orders go to a chef queue, cooked in order; waiter told when ready.

Requirements

Functional: (1) several branches; (2) menu per branch; (3) sections with items; (4) waiter creates an order per table, meals per seat; (5) meal items ↔ menu items; (6) free tables for walk-ins; (7) table reservations; (8) search by date/time and reserve; (9) cancel; (10) reminder notification; (11) pay by card, cheque or cash; (12) several seating arrangements; (13, added) kitchen queue, waiter notified when ready.

Non-functional: no two parties on one table for overlapping time; prices live in one place (the menu item); easy to add a payment type or chef-assignment rule.

Use cases

Actors: **Receptionist** (tables, layouts, reservations, check-in), **Waiter** (orders), **Manager** (staff, menu), **Chef** (cooks), **Cashier** (check and payment), **System** (reminders). Top use cases: add/modify tables, search tables, place/update order, create/cancel reservation, check in, pay. Each role has a small separate set — each role class gets only its own methods.

Core entities

Class	Responsibility
`Restaurant`	Owns branches; closing it deactivates every employee account.
`Branch`	Owns menu, kitchen, seating charts, reservation service; employs staff.
`Menu`, `MenuSection`, `MenuItem`	Menu tree; **price only on** `MenuItem`.
`Table`, `TableSeat`	Capacity and seats; **current status** and **held time slots**.
`TableChart`	Named arrangement; refers to tables (a table can be in several charts).
`Reservation`	Customer, party size, time, table, status; `updatePeopleCount` refuses more than the table seats.
`ReservationService`	Best-fit search, atomic reserve, cancel, check-in, reminders, walk-ins.
`Order`, `Meal`, `MealItem`	One order per visit; one meal per seat; item × quantity; status machine + listeners.
`Kitchen`, `Chef`	Kitchen picks a chef by a pluggable rule; each chef has a FIFO queue.
`Employee` + roles	`Receptionist`, `Waiter`, `Manager`, `Chef`, `Cashier`.
`Check`	Subtotal, tax, total — computed from the order, never stored.
`Payment` (`CardPayment`, `CashPayment`, `CheckPayment`)	Three behaviours.

Class diagram

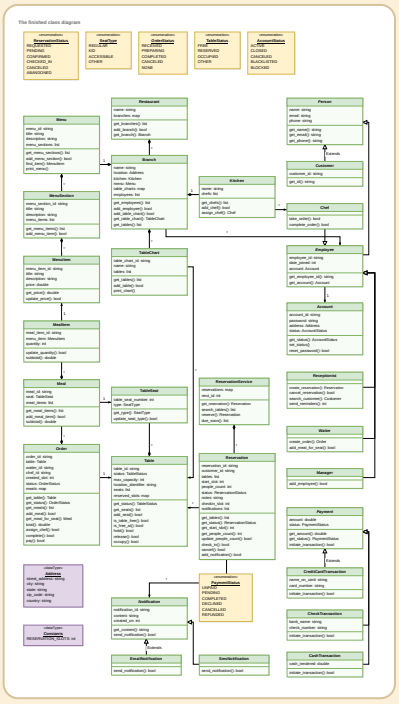
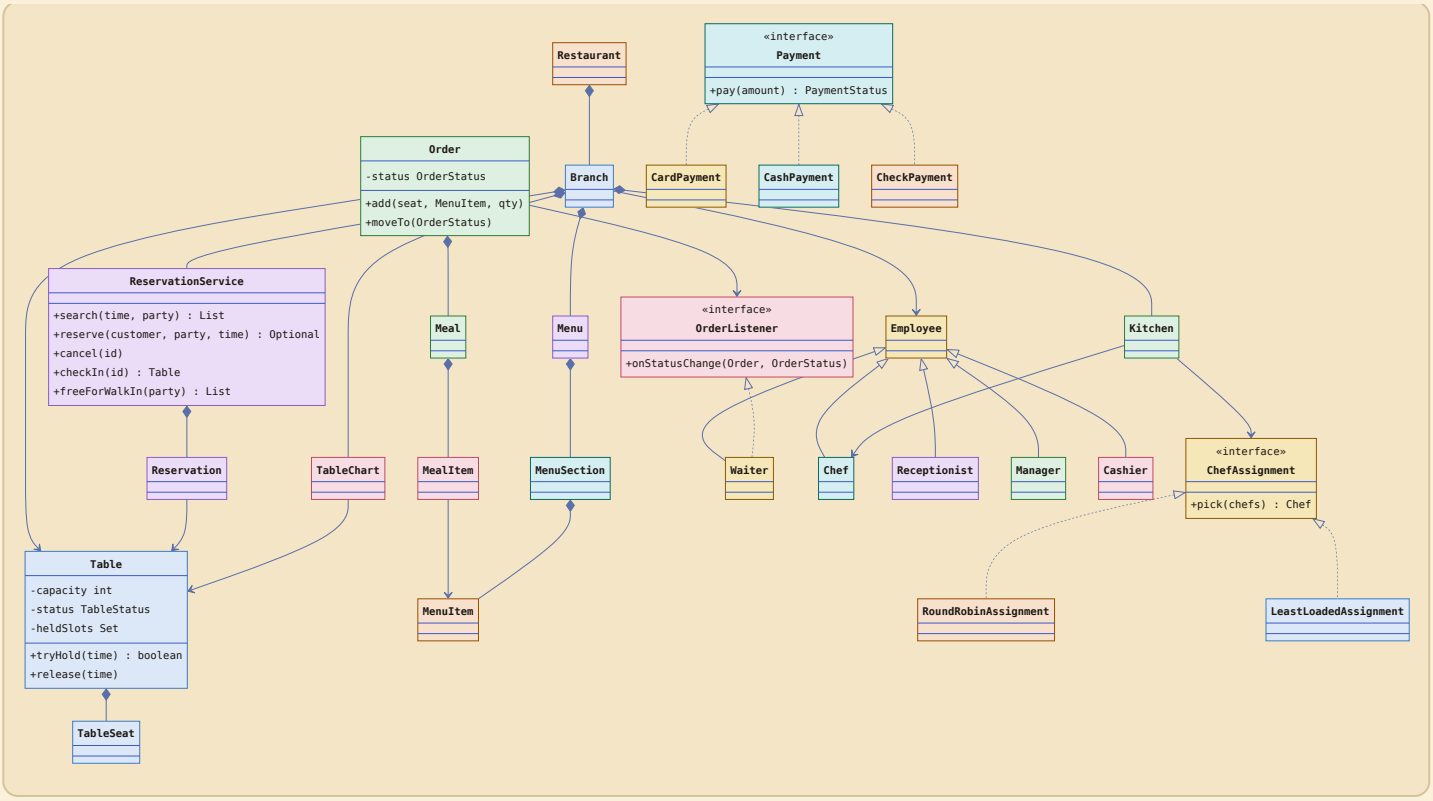


Figure — The finished class diagram of the restaurant management system (course diagram)

The branch is the hub: menu, kitchen and charts are composition; staff are associated (they can move branches). There is **no stored** `Bill` — the order already holds every priced line, so a stored total could disagree. Our `Check` is computed on demand.



Key flows

- **Place an order:** waiter opens an order for the table, adds a meal per seat, submits it to the kitchen.
- **Make a reservation:** search by time and party size → reserve and confirm, or "none available". **Cancel:** look up, cancel, release table, notify.
- **Order lifecycle:** `RECEIVED → PREPARING` (no more items) `→ READY` (waiter paged) `→ SERVED` (cashier settles); `RECEIVED → CANCELLED`.
- **Order to payment:** `Kitchen.submit` → `ChefAssignment.pick` → chef queue → `PREPARING` → `READY` → Observer pages the waiter → `Check.of(order)` → `payment.pay(total)` → `SERVED`, table `FREE`.

Design decisions & patterns used

Decision	Why	Alternative rejected
One meal per seat (`Map<seat, Meal>`)	Plate per guest; split bills are trivial.	Flat list of dishes.
Table keeps held slots as well as status	Status = now (walk-ins); slots = future bookings (a booking holds two hourly slots).	Status only.
Search on `ReservationService`, not `Table`	One table knows only itself (an old course version had an unworkable static `Table.search`).	Static search.
Best fit	Smallest table that fits keeps big tables for big parties.	First free.
Observer (`OrderListener`)	Waiter, screen, delivery app can all subscribe to "ready".	Chef calls waiter directly.
Strategy (`ChefAssignment`)	Course rotates chefs (fair, not load-aware); `LeastLoadedAssignment` is one class.	Hard-coded round-robin.
State machine in `Order.moveTo`	Illegal moves fail loudly.	Free `setStatus`.
`sealed` `Payment` with three classes	Card completes, cash may be short/gives change, cheque is pending.	One class + `if`s.
Check computed; `Customer` not an `Employee`	No second total; only staff have accounts.	Stored bill; one `Person` with a role flag.

Java implementation

Package `lld.ch16.restaurant`. Full runnable code: code/lld/ch16/restaurant/ (see the Full Edition).

```
// Tables.java — Atomic check-then-hold of two hourly slots
synchronized boolean isFreeAt(LocalDateTime start) { return slots(start).stream().noneMatch(heldSlots::contains); }

synchronized boolean tryHold(LocalDateTime start) {
    if (!isFreeAt(start)) return false;
    heldSlots.addAll(slots(start));
    return true;
}

// Reservations.java — Best fit first
List<Table> search(LocalDateTime time, int partySize) {
    return tables.stream().filter(t -> t.capacity() >= partySize && t.isFreeAt(time))
            .sorted(Comparator.comparingInt(Table::capacity)).toList();
}

Optional<Reservation> reserve(Customer c, int partySize, LocalDateTime time) {
    for (Table t : search(time, partySize)) {
        if (t.tryHold(time)) {     // another receptionist may have taken it since search(): try the next one
            var r = new Reservation("RES-" + seq.incrementAndGet(), c, partySize, time, t);
            reservations.put(r.id(), r);
            notifier.send(c, "Table " + t.id() + " booked for " + partySize + " at " + time);
            return Optional.of(r);
        }
    }
    return Optional.empty();
}

// Orders.java — state machine + Observer
void moveTo(OrderStatus next) {
    synchronized (this) {
        boolean ok = switch (status) {
            case RECEIVED -> next == OrderStatus.PREPARING || next == OrderStatus.CANCELLED;
            case PREPARING -> next == OrderStatus.READY;
            case READY -> next == OrderStatus.SERVED;
            case SERVED, CANCELLED -> false;
        };
        if (!ok) throw new IllegalStateException("order " + id + ": " + status + " -> " + next + " not allowed");
        status = next;
    }
    listeners.forEach(l -> l.onStatusChange(this, next));   // notify outside the lock
}

// Kitchen.java — Strategy: which chef gets the next order
interface ChefAssignment { Chef pick(List<Chef> chefs); }

final class LeastLoadedAssignment implements ChefAssignment {
    public Chef pick(List<Chef> chefs) { return chefs.stream().min(Comparator.comparingInt(Chef::load)).orElseThrow();
}
}

// Staff.java — Cashier
PaymentStatus settle(Check check, Payment payment) {
    // Check the order state BEFORE taking money: a refused state change after payment would leave the guest charged.
    if (check.order().status() != OrderStatus.READY)
        throw new IllegalStateException("order " + check.order().id() + " is " + check.order().status() + ", not
READY");
    var status = payment.pay(check.total());
    if (status != PaymentStatus.DECLINED) {
        check.order().moveTo(OrderStatus.SERVED);
        check.order().table().setStatus(TableStatus.FREE);
    }
    return status;
}
```

Output excerpt:

```
2) Ravi searches tables for 4 at 19:00: [T2, T3, T4]
   Asha -> T2, Ben -> T3 (T2 is held 19:00-21:00)
   Asha's party grows to 6: rejected, table T2 seats 4, not 6
6) Kitchen works the queues
   rejected: order O1 is already PREPARING
    [Wendy's pager] order O1 for table T2 is READY
7) Check for O1 (split by seat)
   seat 1: [1x Tomato soup, 1x Margherita pizza] = 18.50
   subtotal 43.00, tax 3.44, total 46.44
   O2 total 25.92 paid by check: PENDING
```

Concurrency & edge cases

- **Two receptionists, one table:** search is advisory; `tryHold` checks and holds both slots atomically; on failure try the next best fit.
- **Notify outside the lock:** calling unknown listener code while locked risks deadlock and long waits.
- **Reminder spam:** `markReminded` sends once per reservation.
- **Party larger than any table:** "no single table fits" (combining tables is an extension). **No-shows:** `NO_SHOW` status; a job frees tables 15 min after start.
- **Party grows** beyond the table → rejected. **Pay before ready** → `settle` checks `READY` before taking money. Unknown id → clear `IllegalArgumentException`; bad seat number rejected.
- **Cash short** → `DECLINED`, order stays `READY`. **Cheque** → `PENDING`.

Extensions & follow-up questions

- **Large party, two tables?** Reservation holds `List<Table>`; lock tables in a fixed order (by id) to avoid deadlock.
- **Split bill?** Already easy — per-seat `Meal.subtotal()`. **Tips?** Extra `Check` lines. **Walk-in waitlist?** FIFO per party size.
- **Kitchen stock?** Ingredient `Inventory`, recipes, mark items "86'd". **Real kitchen threads?** Each chef takes from a `BlockingQueue<Order>`. **Shifts?** A `Shift` entity; only on-shift chefs are picked.

In the interview

30-second answer: "Branch owns a menu tree, kitchen, charts and reservation service. Tables keep status for walk-ins and held slots for bookings; the service searches best-fit and holds atomically. Orders have a meal per seat, a strict state machine and an observer for 'ready'. Chefs are assigned by a strategy. The check is computed; payment is card, cash or cheque strategies."

Plan: 0–5 clarify; 5–10 six actors; 10–20 diagram ("meal per seat", "status vs slots"); 20–35 code `tryHold`, `reserve`, `Order`, `Kitchen.submit`; 35–40 races, large parties, load balancing.

Self-check

1. **Q:** Why both status and held slots? **A:** Status = now (walk-ins); slots = future times (reservations).
2. **Q:** Why is search on the service? **A:** One table only knows itself.
3. **Q:** Which pattern pages the waiter, and why notify outside the lock? **A:** Observer; foreign code under a lock risks deadlock.
4. **Q:** Why is a cheque `PENDING`? **A:** It must clear at the bank.

TL;DR

- Menu → section → item; price only on the item.
- Table = status (now) + held slots (later); best-fit, atomic hold.
- Order → meal per seat; state machine; Observer for "ready"; Strategy for chefs.
- Check is computed; card/cash/cheque strategies.

Real-world example. POS systems like Toast send orders as tickets to kitchen display screens in order (FIFO + Observer); OpenTable models inventory by time slot and party size, like `heldSlots`.

Design an Airline Management System

Problem & clarifying questions

An **airline management system** schedules flights, reserves tickets, cancels flights, and manages staff and crew.

Question	Assumed answer
Search by?	Date, source and destination airport.
Trips?	An **itinerary** of several flights, several passengers (no accounts needed).
Seat classes/types?	Economy, economy plus, preferred economy, business, first; regular, accessible, emergency exit, extra legroom.
Who manages flights?	Only the admin adds aircraft/flights/schedules and cancels flights; everyone affected is notified.
Crew?	Pilots and crew assigned to flights, can view their schedule.
Payment / notifications?	Paid per itinerary; notify on booking changes and flight status changes.

Requirements

Functional: (1) search by date, source, destination; (2) reserve tickets, build itineraries; (3) view schedules and available seats; (4) several passengers per itinerary; (5) only the admin adds aircraft, flights, schedules and cancels flights (all notified); (6) customers cancel a reservation or itinerary; (7) assign pilots and crew; (8) take payment; (9) notify on booking create/modify and flight status change.

Non-functional: a seat on an instance is sold at most once; a multi-passenger booking gets all seats or none (**atomic**); fare rules change without touching booking code.

Use cases

Actors: **Admin**, **Front desk officer** (books for walk-ins), **Customer**, **Pilot/Crew** (view assignments), **System** (notifications). Top use cases: search; create/modify/view reservation; assign seats; pay; update schedule; assign crew. The officer's booking is the same operation as the customer's, triggered by another actor.

Core entities

The key is **four "seat-ish" and "flight-ish" ideas**:

Idea	Class	Example
The route	`Flight`	GA212 London → Tokyo, 14 h, base fare 500
The route on one date	`FlightInstance`	GA212 on 8 Oct 2026 at 10:00
The physical seat	`Seat`	12A, extra legroom, economy plus
That seat on that instance	`FlightSeat`	12A on GA212 8 Oct — booked by PNR147

Others: `Airport`; `Aircraft` (registration, model, seats); `WeeklySchedule` ("every Tuesday 10:00"); `Passenger` (a name on a ticket — **not** an account); `Customer` (books and pays); `Admin`, `FrontDeskOfficer` (admin-only methods need an `Admin` argument); `CrewMember`; `FlightReservation` (one leg: instance, passenger → seat, amount, status); `Itinerary` (legs, connection rule, the unit paid); `FareStrategy`; `Airline` (Facade, keeps the index instance → reservations).

Class diagram

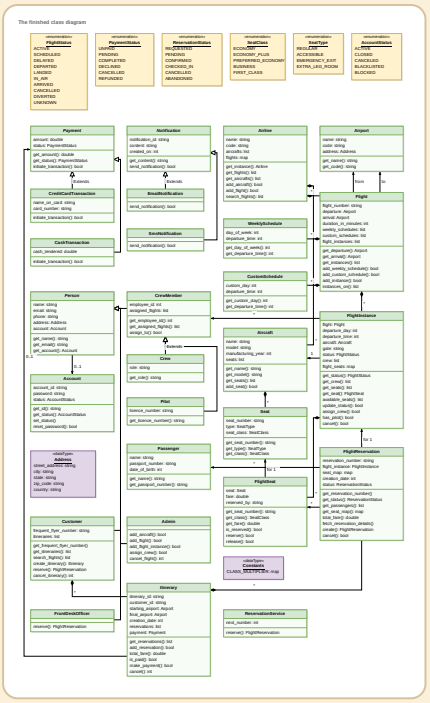
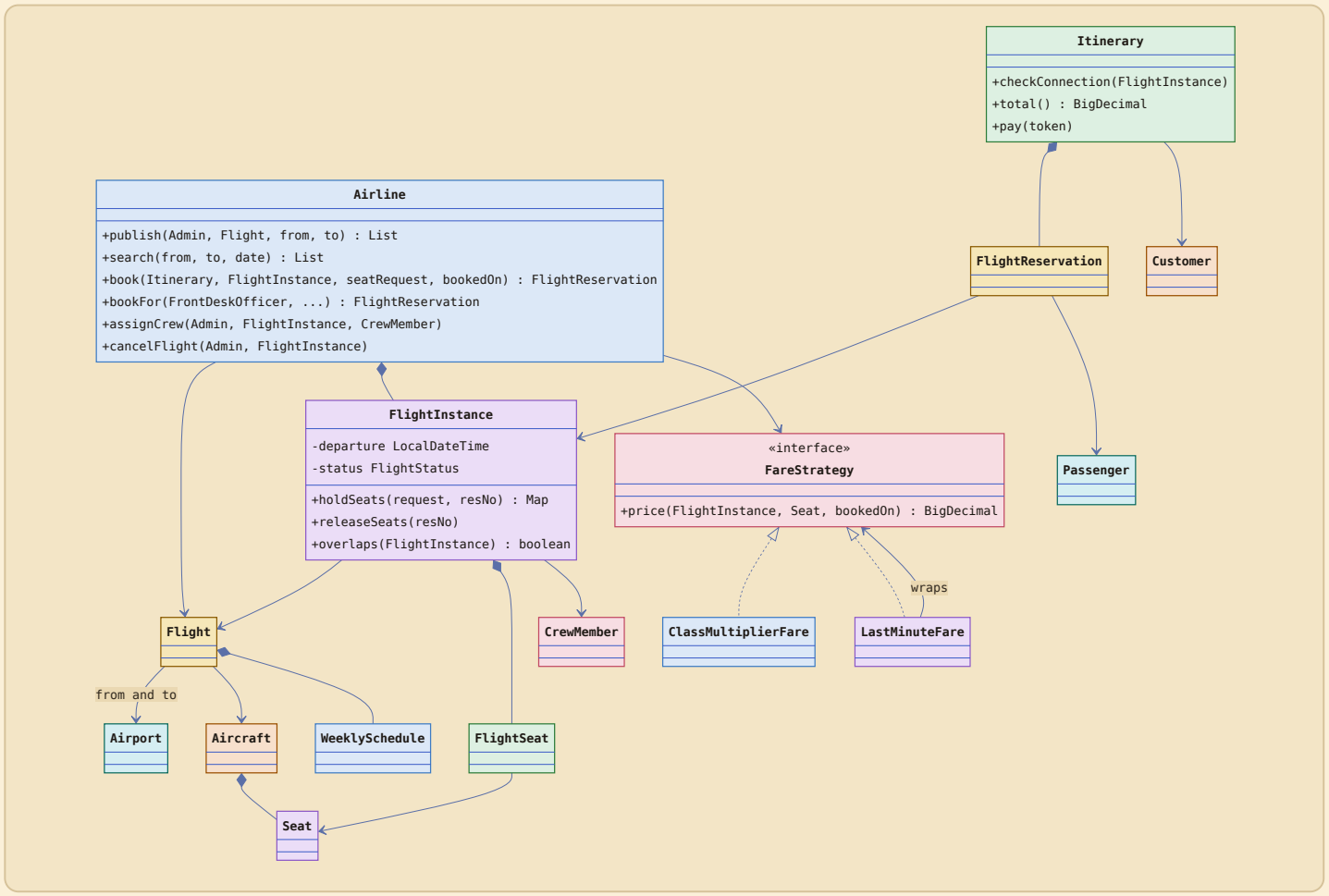


Figure — The finished class diagram of the airline management system (course diagram)

Ownership runs one way: schedules and instances belong to their flight, flight seats to their instance; airports, aircraft and crew are only referenced. `Passenger` sits outside the account tree — a passenger may never log in.



Key flows

- **Reserve:** search → choose seats and passengers → pay → confirmation. In code: `checkConnection` (same airport, ≥ 45 min after the previous leg lands) → `holdSeats` all-or-nothing → price each seat → add leg

→ index instance → reservation → notify.

- **Cancel reservation:** validate → cancel → release seats → notify.
- **Admin cancels a flight:** under the instance lock set `CANCELLED` and copy the index → cancel each reservation (releases seats) → notify each customer with the refund → notify every crew member.

Design decisions & patterns used

Decision	Why	Alternative rejected
Flight / FlightInstance / Seat / FlightSeat	12A is free Tuesday, taken Thursday: booking and fare belong to the seat *on an instance*.	`booked` flag on `Seat`.
`holdSeats` all-or-nothing, `synchronized` per instance	A family gets all seats or none; flights sell in parallel.	One-by-one holds; a global lock.
`Passenger` not an account	Customers book for others.	`Passenger extends Person`.
Itinerary is paid; **connection rule on `Itinerary`**	Course gap: no connecting-time check.	Pay per leg; no check.
Index instance → reservations	Course gap: cancelling notified crew but not passengers.	Scanning every itinerary.
Strategy + Decorator fares	Multipliers 1.0/1.3/1.5/2.5/4.0; `LastMinuteFare` +25% within 3 days.	Price on the seat.
Crew overlap check	No pilot on overlapping instances.	Trust the admin.
Facade `Airline`; admin ops take `Admin`	Compiler enforces "only the admin"; `bookFor` = `book` by another actor.	Rules in actors.
Course Singleton → construct and pass in	Testable.	`getInstance()`.

Java implementation

Package `lld.ch16.airline`. Full runnable code: code/lld/ch16/airline/ (see the Full Edition).

```java
// FlightInstance.java — All-or-nothing: a family of three either gets all three seats or none.
synchronized Map<Passenger, Seat> holdSeats(Map<Passenger, String> request, String reservationNumber) {
    if (status != FlightStatus.SCHEDULED && status != FlightStatus.DELAYED)
        throw new IllegalStateException(key() + " is " + status);
    for (var seatNo : request.values()) {
        var fs = seats.get(seatNo);
        if (fs == null) throw new IllegalArgumentException("no seat " + seatNo + " on " + key());
        if (!fs.isFree()) return Map.of();
    }
    if (request.values().stream().distinct().count() != request.size())
        throw new IllegalArgumentException("two passengers asked for the same seat");
    Map<Passenger, Seat> out = new LinkedHashMap<>();
    request.forEach((p, no) -> { var fs = seats.get(no); fs.assign(reservationNumber); out.put(p, fs.seat()); });
    return out;
}

// Booking.java — Decorator on any fare: +25% when booked less than 3 days before departure.
public BigDecimal price(FlightInstance fi, Seat seat, LocalDate bookedOn) {
    var p = inner.price(fi, seat, bookedOn);
    long daysBefore = ChronoUnit.DAYS.between(bookedOn, fi.departure().toLocalDate());
    return daysBefore < 3 ? p.multiply(new BigDecimal("1.25")).setScale(2, RoundingMode.HALF_UP) : p;
}

// Booking.java — Itinerary: the next leg must leave from where the last one landed, with time to change.
synchronized void checkConnection(FlightInstance next) {
    if (legs.isEmpty()) return;
    var last = legs.getLast().instance();
    if (!last.flight().to().equals(next.flight().from()))
        throw new IllegalArgumentException("leg must depart from " + last.flight().to().code());
    if (next.departure().isBefore(last.arrival().plus(MIN_CONNECTION)))
        throw new IllegalArgumentException("connection too short: " + next.key() + " leaves before " + last.key() + "
lands + 45 min");
}

// Airline.java
FlightReservation book(Itinerary it, FlightInstance fi, Map<Passenger, String> seatRequest, LocalDate bookedOn) {
    it.checkConnection(fi);
    String resNo = "PNR" + seq.incrementAndGet();
    FlightReservation r;
    synchronized (fi) {        // hold the seats AND index the reservation as one step (see cancelFlight)
        var seatMap = fi.holdSeats(seatRequest, resNo);
        if (seatMap.isEmpty()) return null;
        var amount = seatMap.values().stream().map(s -> fares.price(fi, s, bookedOn)).reduce(BigDecimal.ZERO,
BigDecimal::add);
        r = new FlightReservation(resNo, fi, seatMap, amount);
        owner.put(r, it);
        byInstance.computeIfAbsent(fi, k -> new CopyOnWriteArrayList<>()).add(r);
    }
    it.addLeg(r);
    // ... notify customer, return r
}

void cancelFlight(Admin by, FlightInstance fi) {
    List<FlightReservation> affected;
    synchronized (fi) {        // same lock as the hold+index step in book(): no booking can slip past the cancel
        fi.setStatus(FlightStatus.CANCELLED);
        affected = List.copyOf(byInstance.getOrDefault(fi, List.of()));
    }
    for (var r : affected) {
        if (r.cancel()) {
            // ... notify the itinerary's customer with the refund
        }
    }
    fi.crew().forEach(c -> notifier.send(c.id() + "@crew", "Stand down: " + fi.key() + " cancelled"));
}
```

Output excerpt:

```
   itinerary total 3300.00 (2 x 400 economy + 2 x 1250 business)
   rejected: connection too short: GA212@2026-10-06 leaves before GA100@2026-10-07 lands + 45 min
   successful bookings of 10C: 1, free seats left: 3
   economy-plus fare 500 x 1.3 = 650.00, charged 812.50
   rejected: Capt. Kim already flies GA212@2026-10-08 at that time
    [email priya@example.com] GA212@2026-10-08 is cancelled. Reservation PNR103 refunded 2500.00. Itinerary IT-101 now
totals 800.00
```

20 threads race for seat 10C and exactly one wins; 650 + 25% last-minute = 812.50; the cancellation reaches every passenger through the index.

Concurrency & edge cases

- **Selling one seat twice:** `holdSeats` checks every seat, then assigns all, under the instance lock.
- **Booking while the admin cancels:** hold + index and set-`CANCELLED` + copy use the same `synchronized (fi)` — every reservation is either in the copy (cancelled and notified) or refused.
- **Double payment:** `Itinerary.pay` is idempotent. **Idempotent cancel:** `FlightReservation.cancel()` returns `false` if already cancelled — no double refund.
- **Duplicate seat in one request**, **booking a cancelled/departed flight**, **crew on overlapping instances** — all rejected.
- **Time zones:** real systems use `ZonedDateTime`/`Instant` per airport. **Holds during payment:** 10–15 min `heldUntil` on `FlightSeat` plus a sweeper.

Extensions & follow-up questions

- **Overbooking?** Per-class `overbookLimit`; assign physical seats at check-in.
- **Check-in/boarding passes, baggage, change fees, codeshares (`MarketingFlight`)** — small new entities.
- **Delays?** `DELAYED` + new departure; notify through the index; re-check connections.
- **Scale?** One row per seat with `UPDATE … WHERE reservation IS NULL` or optimistic locking; cache search.

In the interview

> **30-second answer:** "`Flight` is the route, `FlightInstance` one departure; `Seat` is the physical chair, `FlightSeat` that chair on that departure, where booking and fare live. Seats are held all-or-nothing per instance. An itinerary groups legs, checks connections and is paid. Fares are a strategy with a last-minute decorator. An instance → reservations index lets a cancelled flight notify every passenger."

Plan: 0–5 clarify; 5–10 actors; 10–20 the four-way distinction and diagram; 20–35 code `holdSeats`, `book`, `checkConnection`, `cancelFlight`; 35–40 holds, overbooking, time zones. Expected follow-up: "How do passengers learn of a cancellation?" — "Through the instance → reservations index; without it you only reach the crew."

Self-check

1. **Q:** Where does "booked" live? **A:** On `FlightSeat` — per instance, not per aircraft.
2. **Q:** All seats or none? **A:** `holdSeats` checks all, then assigns all, in one synchronized block.
3. **Q:** `LastMinuteFare`? **A:** +25% within 3 days; a Decorator over any `FareStrategy`.
4. **Q:** Two course gaps closed? **A:** Passenger notification on cancel (index) and connection-time check (`Itinerary.checkConnection`).

TL;DR

- Flight ≠ FlightInstance; Seat ≠ FlightSeat.
- Hold seats all-or-nothing under the instance's lock.
- Itinerary = legs + connection rule + payment.
- Fares: Strategy + Decorator; instance → reservations index for cancellations.

Real-world example. Airlines separate a flight number from a leg on a date and control inventory per leg and booking class; a PNR groups passengers and segments — like `FlightReservation`/`Itinerary`. On cancellation

they find every PNR on the leg, which is why the index matters.

Design a Course Registration System

Problem & clarifying questions

A **course registration system** lets students sign up for classes each term — a harsh concurrency problem: at 09:00 thousands click "register" for a few popular sections.

Terms: a **course** is a catalogue entry (CS201, 4 credits); a **section** is one offering with instructor, time and capacity (CS201-A, Mon/Wed 10:00–11:30, 30 seats) — students register for **sections** (like Flight vs FlightInstance); a **prerequisite** must be completed first; a **waitlist** is a FIFO queue for a full section; a **registration window** is when a student may register (seniors often first).

Question	Assumed answer
Courses or sections?	Sections.
Prerequisites?	Must be completed.
Capacity?	Per section; when full, a bounded waitlist.
Promotion?	Automatic on drop, FIFO, after re-checking the timetable.
Limits?	Credit limit; no time clashes; no two sections of one course.
Windows?	Open by class year; close together.
Concurrency?	Never exceed capacity; no double registration from two tabs.

Requirements

Functional: (1) register for sections and view them; (2) course has code, name, credits; section has instructor, time slot, limit; (3) search by code or name; (4) never exceed capacity, else waitlist (if room); (5) enforce prerequisites, credit limit, time conflicts, one section per course, window; (6) on drop, promote the waitlist head and notify.

Non-functional: safe concurrent registration; rules easy to add/remove ("majors only"); extensible (fees, holds, cross-listing).

Use cases

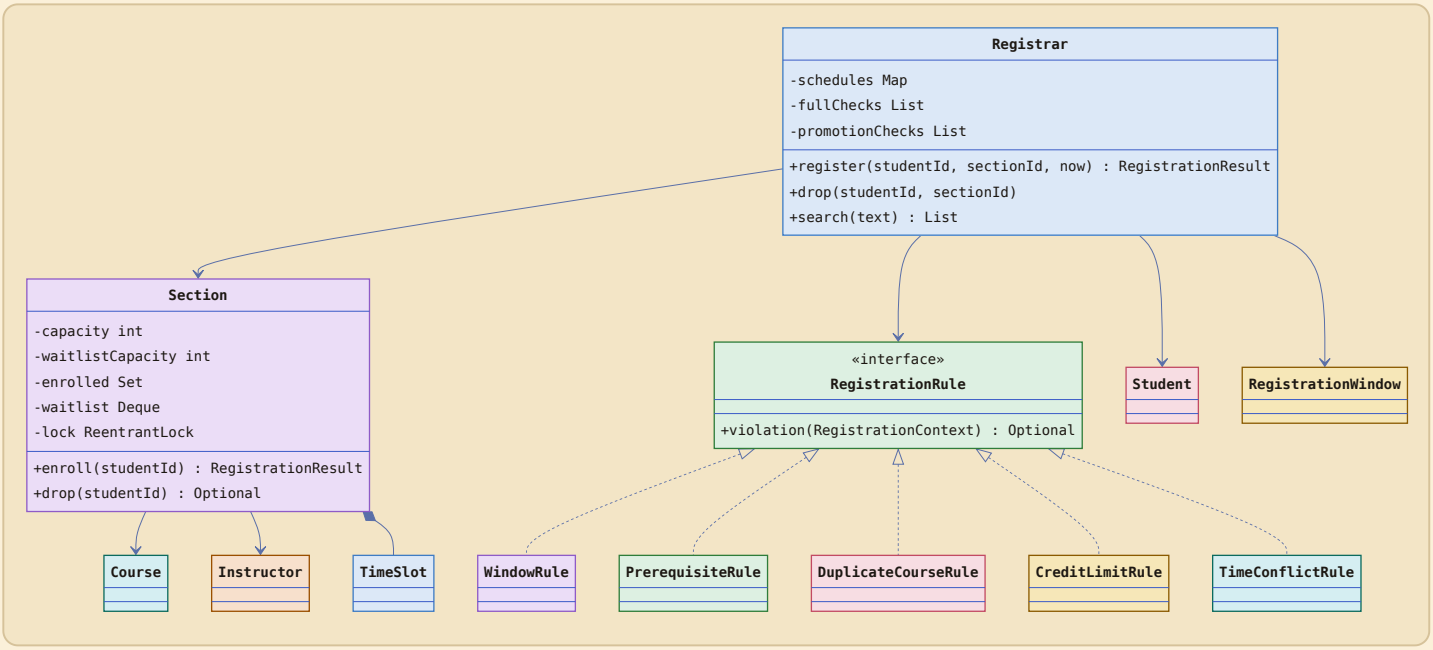
Student — search, register, drop, view schedule, join waitlist. **Registrar/Admin** — create courses/sections, set capacity and windows. **Instructor** — view roster. **System** — enforce rules, promote from waitlist, notify (a drop triggers promotion).

Core entities

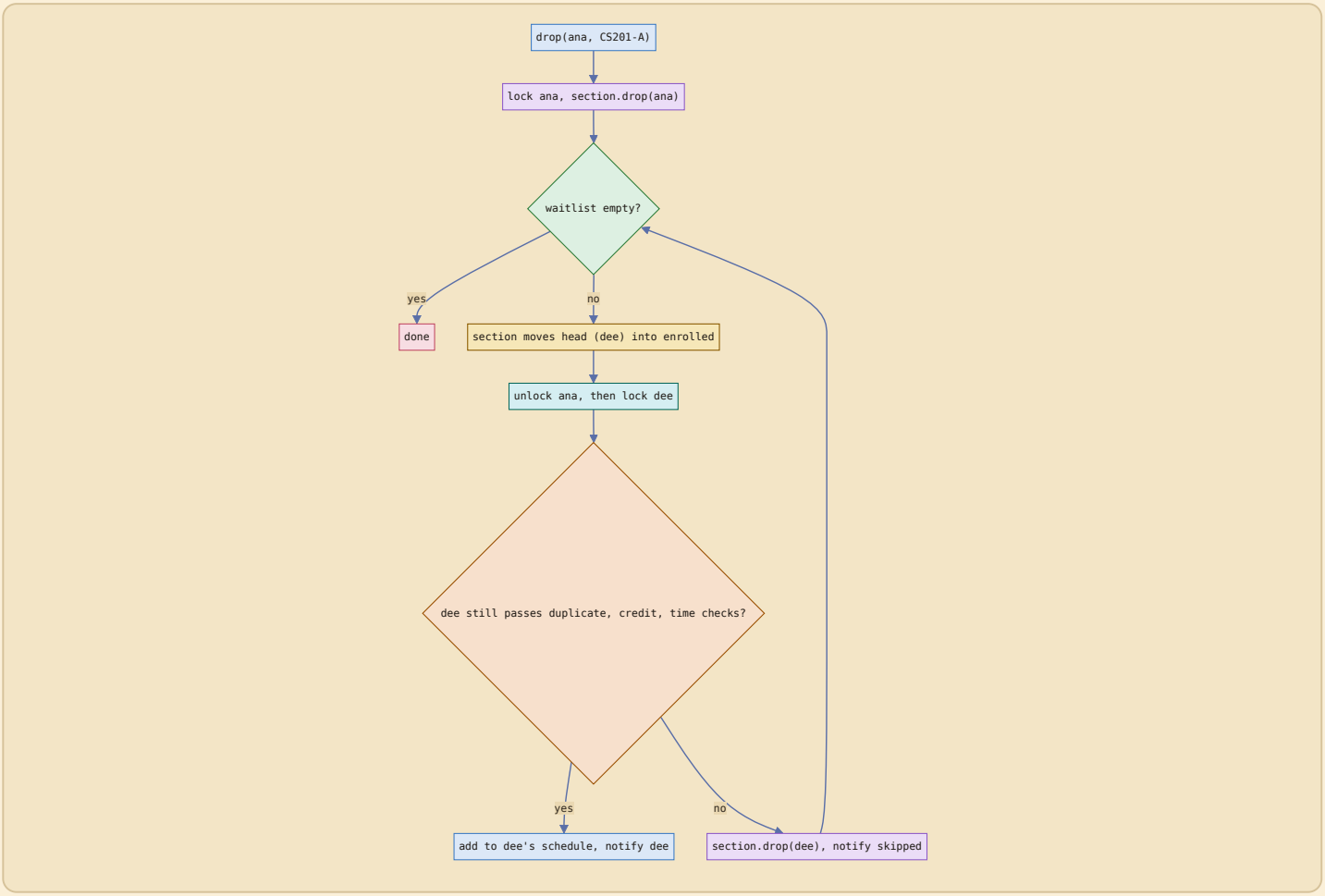
Class	Responsibility
`Course`	Code, title, credits, prerequisite codes.
`Section`	Instructor, slot, capacity, enrolled set, waitlist. **The only place a seat is taken**, under its own lock.
`TimeSlot`	Days + start/end; knows overlap.
`Instructor`, `Student`	Who teaches; year, completed courses, credit limit.
`RegistrationWindow`	Opening per class year, common close.
`RegistrationRule` (+ `WindowRule`, `PrerequisiteRule`, `DuplicateCourseRule`, `CreditLimitRule`, `TimeConflictRule`)	One rule each; returns a violation reason.
`RegistrationContext`	Everything a rule may look at.
`RegistrationResult`, `Outcome`	`ENROLLED` / `WAITLISTED` / `REJECTED` + message.
`Registrar`	Runs the rule chain, takes the seat, drops and promotes, keeps schedules, defines the lock order.

Class diagram

There is no course lesson for this problem; it comes from the *awesome-low-level-design* problem set (register, view, search, capacity, concurrency, extensibility) — all covered, with instructor and capacity on `Section`.



Key flows



- **Register:** lock the student's schedule → run window, prerequisites, duplicate, credits, time conflict → first failure = `REJECTED` + reason; else `section.enroll` (section lock) → `ENROLLED` (add to schedule), `WAITLISTED at position n`, or `REJECTED` (both full).
- **Student status in a section:** `ENROLLED`, `WAITLISTED` (→ `ENROLLED` on promotion, or `SKIPPED` if the re-check fails), `REJECTED`; drop leaves.
- **Drop and promote:** as in the flowchart above.

Design decisions & patterns used

Decision	Why	Alternative rejected
Course vs Section	Capacity, time, instructor belong to an offering.	Capacity on `Course`.
Seat taken only in `Section.enroll`, `ReentrantLock` per section	Atomic check-capacity-then-add; sections don't block each other.	`synchronized` registrar (the whole university in one line).
Per-student lock around rules + enroll	Credit/time checks read the schedule; two tabs would both pass without it.	Section lock only.
Lock order: student, then section	Deadlock needs opposite orders; promotion takes the next student's lock only *after* releasing the dropper's.	Promote while holding the dropper's lock.
Rules as objects (Specification / Chain of Responsibility)	A new rule = new class + list entry (Open/Closed); promotion reuses a subset.	One long `if` method.
Bounded FIFO `Deque` waitlist	Fair, bounded.	Unbounded list.
Re-check on promotion	Timetable may have changed while waiting.	Promote blindly.
`now` passed in	Testable windows.	`LocalDateTime.now()` in rules.
GitHub problem's **Singleton** → one `Registrar` passed in	Testable.	`getInstance()`.

Java implementation

Package `lld.ch16.courseregistration`. Full runnable code: code/lld/ch16/courseregistration/ (see the Full Edition).

```
// Section.java — The only place a seat is taken: check-and-add in one critical section.
RegistrationResult enroll(String studentId) {
    lock.lock();
    try {
        if (enrolled.contains(studentId)) return RegistrationResult.rejected("already enrolled in " + id);
        if (waitlist.contains(studentId)) return RegistrationResult.rejected("already on the waitlist of " + id);
        if (enrolled.size() < capacity) {
            enrolled.add(studentId);
            return new RegistrationResult(Outcome.ENROLLED, "enrolled in " + id + " (" + enrolled.size() + "/" +
capacity + ")");
        }
        if (waitlist.size() < waitlistCapacity) {
            waitlist.addLast(studentId);
            return new RegistrationResult(Outcome.WAITLISTED, "waitlisted for " + id + " at position " +
waitlist.size());
        }
        return RegistrationResult.rejected(id + " and its waitlist are full");
    } finally {
        lock.unlock();
    }
}

// Rules.java
interface RegistrationRule { Optional<String> violation(RegistrationContext c); }

final class CreditLimitRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        int current = c.currentSchedule().stream().mapToInt(s -> s.course().credits()).sum();
        int after = current + c.section().course().credits();
        return after <= c.student().maxCredits() ? Optional.empty()
                : Optional.of("credit limit " + c.student().maxCredits() + " exceeded (" + after + ")");
    }
}

// Registrar.java — lock order is ALWAYS student-then-section
RegistrationResult register(String studentId, String sectionId, LocalDateTime now) {
    // ... look up student, section (reject if unknown)
    var schedule = schedules.get(studentId);
    synchronized (schedule) {                                   // 1st lock: the student
        var ctx = new RegistrationContext(student, section, Set.copyOf(schedule), now, window);
        var v = firstViolation(fullChecks, ctx);
        if (v.isPresent()) return RegistrationResult.rejected(v.get());
        var result = section.enroll(studentId);                 // 2nd lock: the section
        if (result.outcome() == Outcome.ENROLLED) schedule.add(section);
        return result;
    }
}

void drop(String studentId, String sectionId) {
    // ...
    synchronized (schedule) {
        promoted = section.drop(studentId);
        schedule.remove(section);
    }
    notifier.accept(studentId + " dropped " + sectionId);
    // Finish promotions OUTSIDE the dropper's lock, one student lock at a time (no nested student locks).
    while (promoted.isPresent()) promoted = finishPromotion(promoted.get(), section);
}
```

`finishPromotion` locks the promoted student, re-runs `promotionChecks` (duplicate, credit, time), and either adds the section to their schedule or drops them and returns the next promotion.

Output excerpt:

```
   ana  MATH201-A         -> REJECTED: time conflict with CS201-A [MONDAY, WEDNESDAY] 10:00-11:30
   ana  CS301-A           -> REJECTED: missing prerequisites [CS201, MATH201]
   fay  CS201-A           -> REJECTED: CS201-A and its waitlist are full
   ben  MATH201-A         -> REJECTED: credit limit 4 exceeded (7)
    [notify] dee promoted from waitlist into CS201-A
5) 50 first-years grab 30 seats (+10 waitlist) in CS101-A at 09:00 sharp
   ENROLLED=30 WAITLISTED=10 REJECTED=10, roster size 30
6) One student double-clicks: two threads, same request
   {ENROLLED=1, REJECTED=1}, gus schedule [CS201-B]
```

The 09:00 rush counts are exact every run; the double-click succeeds exactly once.

Concurrency & edge cases

- **Overselling** — section lock around "size < capacity → add". **Double-click / two tabs** — per-student lock; the second request hits `DuplicateCourseRule` or "already enrolled".
- **Deadlock** — fixed order student → section; never two student locks at once.
- **Waitlisted in one section, enrolled in another of the same course** — caught by the promotion re-check, which skips to the next student.
- **Window closes while waitlisted** — promotion doesn't re-check the window; add `WindowRule` to `promotionChecks` if that is the policy.
- Dropping a section you're not in → `IllegalArgumentException`; windows are half-open (`opens ≤ now < closes`).
- **Beyond one JVM:** `UPDATE section SET enrolled = enrolled + 1 WHERE id = ? AND enrolled < capacity` (atomic conditional update) or optimistic locking; a queue in front of hot sections.

Extensions & follow-up questions

- **Co-requisites / "prereq OR permission"?** Composite expression tree (`And`, `Or`, `CourseCompleted`, `Permission`) in `PrerequisiteRule`.
- **Holds?** A `HoldRule`. **Priority waitlist?** `PriorityQueue` by (year, time joined). **Major seat buckets?** Split capacity in `Section.enroll`.
- **Lecture + lab?** Lock both sections by id order, check both, enrol both. **Notifications/fees?** Swap the `Consumer<String>` notifier; a `Bill` per student per term.

In the interview

> **30-second answer:** "Students register for sections, not courses. A section owns capacity and a bounded FIFO waitlist and takes seats only inside its own lock. The registrar runs a chain of rule objects under the student's lock, then asks the section for a seat. Lock order is always student then section, so no deadlocks. On drop the waitlist head is promoted after a re-check."

Plan: 0–5 clarify; 5–12 entities and diagram; 12–30 code `Section.enroll/drop`, the rule interface and two rules, `register`; 30–40 concurrency, lock order, DB conditional update.

Self-check

1. **Q:** Which two locks does `register` take, in what order? **A:** Student's schedule, then the section — always.
2. **Q:** Why not `synchronized` on the whole registrar? **A:** Correct but serialises the whole university.
3. **Q:** Why re-check on promotion? **A:** The timetable may have changed while waiting.
4. **Q:** Capacity with many servers? **A:** Atomic conditional update (`… WHERE enrolled < capacity`) or optimistic locking.

TL;DR

- Course (catalogue) vs Section (capacity, slot, waitlist).
- Rules are small objects run as a chain.
- Seats taken only in `Section.enroll`; student lock first, section lock second.
- Drop → promote head → re-check → notify. At scale: DB conditional update.

Real-world example. Student-information systems such as Ellucian Banner run priority windows by class standing, check prerequisites and clashes, and keep section waitlists; registration-day slowdowns are the 09:00 "thundering herd" this design handles.

Chapter summary

Problem	Key insight	Atomic claim	Patterns
Car rental	Availability is about **dates**, not status	`Vehicle.tryBook(range)`	Template Method, Strategy + Decorator, state machine
Restaurant	**Meal per seat**; table = status + slots	`Table.tryHold(time)`	Observer, Strategy (chefs, payment), state machine
Airline	**Flight ≠ FlightInstance, Seat ≠ FlightSeat**	`FlightInstance.holdSeats(map)` (all or nothing)	Strategy + Decorator, Facade, cancellation index
Course registration	**Course ≠ Section**; rules as objects	`Section.enroll` inside a student lock	Specification / Chain of rules, lock ordering

Find the calendar, find the atomic check-then-claim, and put the rules on the object that owns the data.

Chapter 17 — Case Studies: Games & Rules — Study Edition

> **Where this fits.** Chapters 14–16 modelled the real world. In games the "world" is a set of exact **rules** — every rule must live *somewhere*, and the interviewer will ask where. That is why games are a favourite interview topic.

What games test in an LLD interview

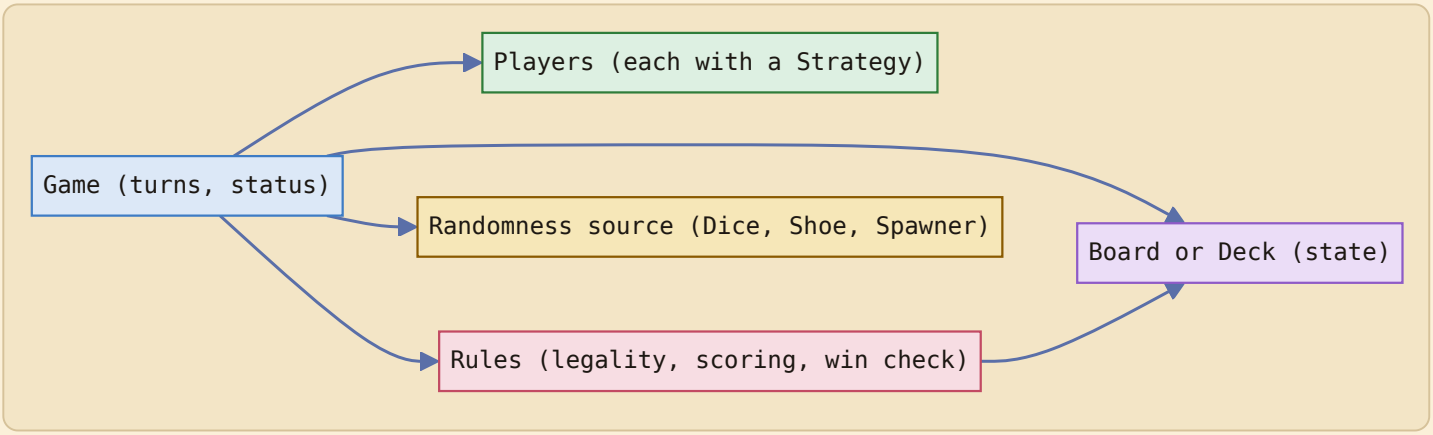
Skill	What it means	Where you see it
Rule encapsulation	Each rule lives in one obvious class.	`BlackjackScoring`; chess `Rules`; 2048 `LineMerger`.
Turn management	Whose turn, what may they do, what happens at the end of a turn.	Turn index (tic-tac-toe), player queue (snake and ladder), `turn` colour (chess), rounds (blackjack).
Polymorphic pieces	One method call, many behaviours (*polymorphism*).	Chess `Piece.targets(...)`.
Pluggable strategies	Varying behaviour behind an interface (**Strategy**, Ch. 10).	`PlayerStrategy`, `DealerPolicy`, `MoveStrategy`, `Dice`, `OverflowRule`, `WinStrategy`, `TileSpawner`.
Deterministic testing	Randomness is *injected*: `new Random(seed)` or a scripted source (stacked shoe, loaded dice).	Every `Main` prints the same output every run.
State and status	Games move through states; illegal actions in the wrong state are rejected.	`GameStatus`, `GameState` enums.

> **Key idea: seeded randomness.** `java.util.Random` is *pseudo-random*: the same **seed** always gives the same sequence, and `Collections.shuffle(list, rnd)` is deterministic for a given `Random`. Rule: **never call `new Random()` or `Math.random()` inside the game — accept a `Random` (or a `Dice` / `Spawner`) from outside.** Production passes an unseeded (or `SecureRandom`) source; tests pass a seed. This is plain Dependency Injection (Ch. 5).

Problems: 17.1 Blackjack and a deck of cards · 17.2 Chess · 17.3 Tic-Tac-Toe · 17.4 Snake and Ladder · 17.5 2048.

How to run: `cd chapters/17-case-games/code && javac --release 21 -d /tmp/lld_17 $(find . -name '*.java') && java -cp /tmp/lld_17 lld.ch17.blackjack.Main` (or `chess`, `tictactoe`, `snakeladder`, `game2048`). All randomness is seeded or scripted, so output is exact.

A reusable skeleton for any board/card game — name these five parts early:



Part	Question it answers	Typical class names
State	What does the table look like now?	`Board`, `Deck`, `Shoe`, `Hand`, `Cell`
Actors	Who acts, how do they decide?	`Player` + `PlayerStrategy` / `MoveStrategy`
Rules	Allowed? What does it do? Who won?	`Rules`, `DealerPolicy`, `OverflowRule`, `WinStrategy`
Controller	Whose turn? Which state?	`Game`, `GameStatus`
Randomness	Where does chance come from?	`Dice`, `Shoe`, `TileSpawner` — always injected

---

Design Blackjack and a Deck of Cards

Problem & clarifying questions

Blackjack: every player plays only against the **dealer** (the casino employee playing for the house); everyone tries to get close to **21** without going over (**busting** loses at once). Often asked as "design a deck of cards, then blackjack" — the first half checks the deck stays **generic**, the second checks rule placement.

Question	Assumed answer
Decks?	A **shoe** of several 52-card decks (casinos use 6–8); one deck = shoe of 1.
Players?	One or more, each against the dealer.
Options?	Hit, stand, double down, split (once), insurance. No surrender/side bets.
Dealer rule?	Hit ≤ 16, stand ≥ 17; "hit soft 17" pluggable.
Payouts?	Win 1:1, blackjack 3:2, push returns the bet, insurance 2:1.
Money?	Whole **chips** (`int`), even ante. Never `double`.
UI, accounts?	Out of scope (the course's `Account`/`Person` would attach to `Player`).

Requirements

Background rules: 13 ranks × 4 suits; two cards each; the dealer shows the **up card** and hides the **hole card**. J/Q/K = 10, **ace = 1 or 11**. **Soft** total = an ace counted as 11; **hard** = all aces as 1 (A+5 = soft 16 / hard 6). **Blackjack** (natural) = two cards totalling 21.

Functional:

1. Place an ante to play.
2. Deal two cards each (dealer: one up, one hole).
3. Dealer shows an **ace** → offer **insurance** (half the ante, pays **2:1** on dealer blackjack); the dealer then **peeks**; dealer blackjack ends the round (insurance wins, main bets lose, player blackjack pushes).
4. Below 21 the player chooses **hit**, **stand**, **double down** (double the bet, exactly one card, stop) or **split** (equal ranks only, second bet = ante, two hands, one card each, no re-split).
5. Over 21 → immediate loss.
6. Dealer plays by the fixed rule.
7. Dealer bust → every live hand wins; else higher total wins; equal = **push**. Blackjack pays **3:2**, other wins 1:1.
8. **Even money**: blackjack vs a dealer ace may take 1:1 at once (= insurance + blackjack; explained, not built).

Non-functional: generic `Card`/`Deck`; deterministic tests; pluggable dealer and player rules; whole-chip `int` money — deliberately not `BigDecimal`: an even ante keeps every payout whole (3:2 on 10 = 15). If fractions appear, use `BigDecimal` + `RoundingMode`.

Use cases

Player — bets; accepts/declines insurance, split, double, even money; hits or stands (log in / join game belong to a lobby service). **Dealer** — deals, plays by the fixed rule, collects and pays. Use cases: create hands, place bet, player plays, dealer plays, insurance / even money / examine hole card (ace up only), split, resolution; dealing and payout are `<<include>>`d in every round.

Core entities

Class	Responsibility
`Suit`, `Rank`	Fixed values; `Rank` stores *face value* 1–13 (ace 1, K 13).
`Card` (record)	Rank + suit; knows **nothing** about blackjack.
`Deck<T>`	Generic shuffleable stack; `standard52()` factory.
`Shoe`	Several decks; deals; reshuffles at the **cut card**; `Shoe.stacked(...)` for tests.
`BlackjackScoring`	Blackjack value of a rank (K = 10, ace = 1 hard) — kept off `Card`.
`Hand`	Cards + **its own bet**; hard/soft/best total, bust, blackjack, split, double.
`Action`, `PlayerStrategy`	HIT/STAND/DOUBLE/SPLIT; `BasicStrategy`, `MimicDealerStrategy` (a UI human would be a third).
`DealerPolicy`	`standOnAll17()`, `hitSoft17()`.
`Player`	Name, chip balance, strategy.
The dealer	No class here: a `Hand` in `BlackjackGame` + the `DealerPolicy` (the course has `Dealer` beside `Player` under `BasePlayer`).
`BlackjackGame`	Runs a round: bets, deal, insurance + peek, player turns, dealer turn, settlement.

Class diagram

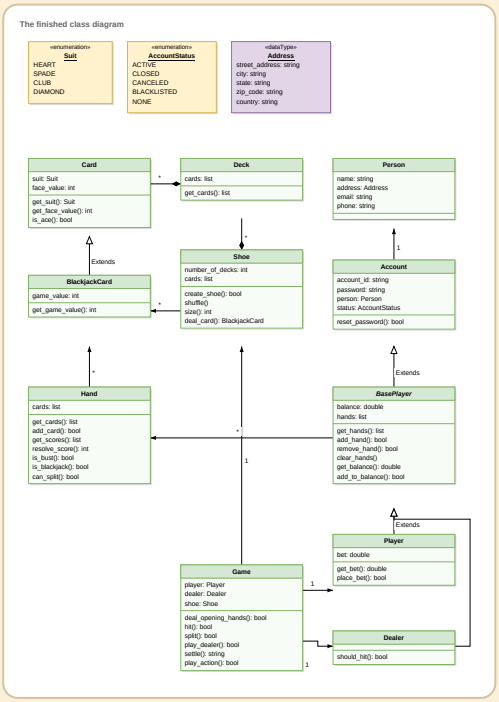
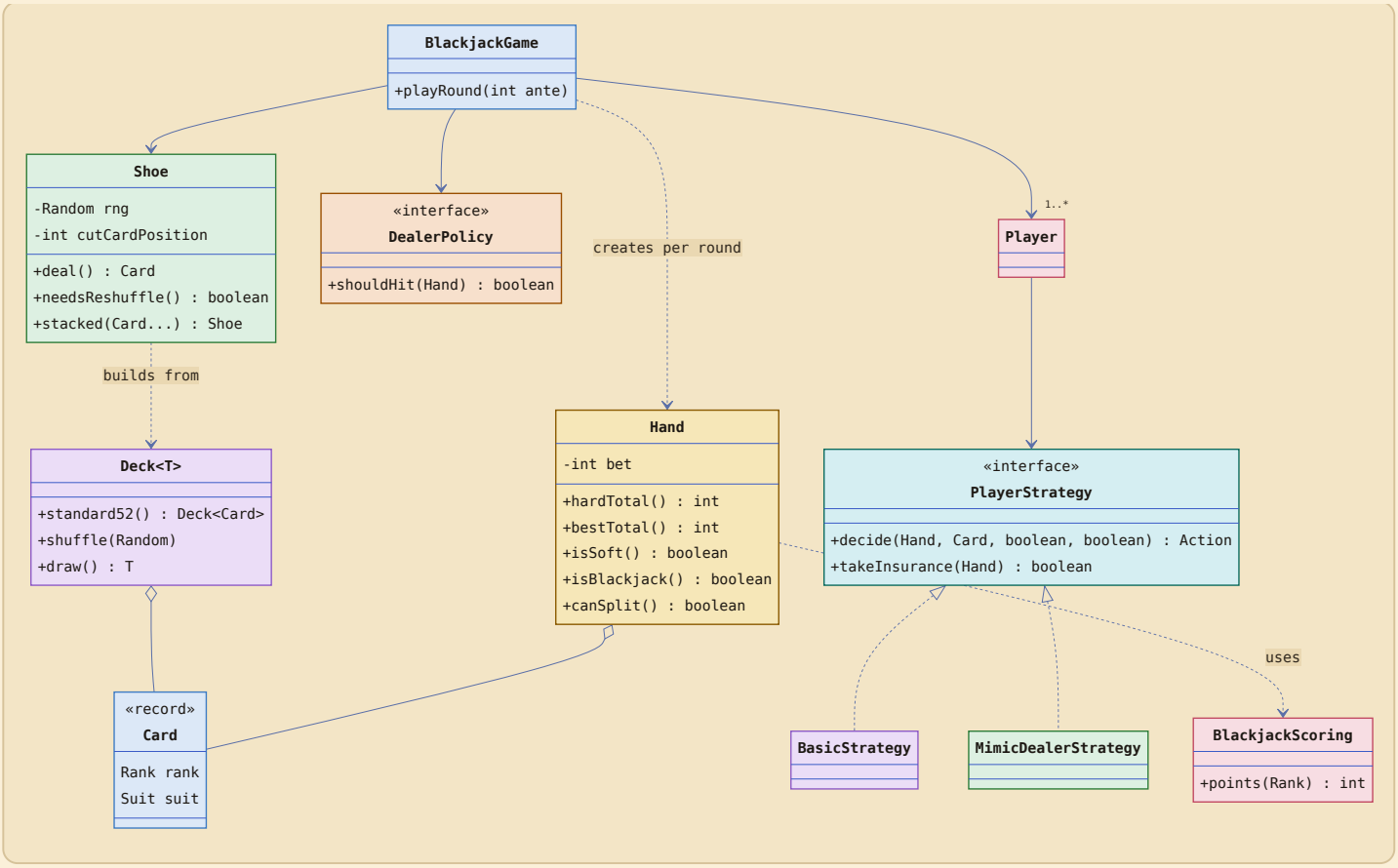


Figure — Course class diagram: Card, BlackjackCard, Deck, Shoe, Hand, BasePlayer, Player, Dealer, Game, Account (course diagram)

In the course, `BlackjackCard extends Card` adds the *game value* (K: face 13, worth 10); a `Shoe` is composed of `Deck`s; `Player` and `Dealer` share `BasePlayer` (balance, `*` hands because of splits). We keep the core idea — **the game value must not live on the generic card** — using a scoring function, and make player and dealer rules strategies:



Key flows

One round: debit antes → deal two each → if up card is an ace, offer insurance → if up card is ace or ten and dealer has blackjack, settle and stop → else each hand plays (`decide` until stand, bust, 21 or double) → dealer draws while `shouldHit` (skipped if all hands bust) → settle. States: `Betting → Dealing → (Insurance) → Peek → PlayerTurns → DealerTurn → Settled`.

Scoring a hand (the ace problem). At most one ace can ever count as 11 (two would be 22). So: (1) sum all cards with aces as 1 = **hard total**; (2) if there is an ace and `hard + 10 ≤ 21`, the hand is **soft** and best = `hard + 10`; (3) else best = hard. O(n), no search, and the soft flag comes free (needed for "hit soft 17").

Design decisions & patterns used

Decision	Why	Alternative rejected
`Card` is a game-agnostic **record**.	Poker, bridge, solitaire reuse it; K = 10 is a rule of this game.	`getValue()` returning 10 for K on `Card`.
Value in `BlackjackScoring`, not a `BlackjackCard` subclass.	Records can't be extended; one card type in the shoe.	The course's subclass — also valid; say both.
Generic `Deck<T>`.	Uno/tarot reuse shuffle/draw.	Deck hard-wired to `Card`.
Strategy for `PlayerStrategy`, `DealerPolicy`.	UI, bots, scripts plug in; S17 vs H17 is one line (Open/Closed).	`if (isHuman)`, `if (casino == X)`.
Factory methods (`standard52`, `stacked`, `standOnAll17`); injected `Random`.	Named constructors; exact tests.	Flag constructors; hidden randomness.
`hardTotal` / `bestTotal` / `isSoft`.	The course returns every possible total; hard + soft flag carries the same information.	List of totals to filter.
Hand stores its **own bet**.	Split and double change bets per hand.	One bet per player.
Game **validates** the strategy's choice.	A buggy client can't double without money.	Trusting the client.

Honest weakness. `playRound` owns the whole sequence *and* the payouts; each new option adds a branch. A larger design uses a **State** machine (one class per round state, like the Ch. 14 vending machine) and a `PayoutTable`. Say it out loud.

Java implementation

Package `lld.ch17.blackjack`. Full runnable code: code/lld/ch17/blackjack/ (see the Full Edition).

```
/** A game-agnostic playing card: just a rank and a suit. Immutable, so it is safe to share. */
public record Card(Rank rank, Suit suit) {
    @Override public String toString() { return rank.symbol() + suit.letter(); }
}

// BlackjackScoring.java — Points with an ace counted as 1 (the "hard" value).
public static int points(Rank rank) {
    return switch (rank) {
        case ACE -> 1;
        case JACK, QUEEN, KING -> 10;
        default -> rank.faceValue();
    };
}

// Hand.java
public int hardTotal() {
    int sum = 0;
    for (Card c : cards) sum += BlackjackScoring.points(c.rank());
    return sum;
}
public boolean isSoft() { return hasAce() && hardTotal() + 10 <= 21; }
public int bestTotal() { return isSoft() ? hardTotal() + 10 : hardTotal(); }
public boolean isBust() { return bestTotal() > 21; }
public boolean isBlackjack() { return !fromSplit && cards.size() == 2 && bestTotal() == 21; }

// DealerPolicy.java
@FunctionalInterface
public interface DealerPolicy {
    boolean shouldHit(Hand dealerHand);

    /** Classic rule: draw on 16 or less, stand on every 17 ("S17"). */
    static DealerPolicy standOnAll17() { return h -> h.bestTotal() <= 16; }

    /** Variant: also draw on a soft 17 such as A+6 ("H17"), which favours the house. */
    static DealerPolicy hitSoft17() { return h -> h.bestTotal() <= 16 || (h.bestTotal() == 17 && h.isSoft()); }
}

// BlackjackGame.java — validate the strategy's choice, then settle
Action a = p.strategy().decide(hand, up, canDouble, canSplit);
if ((a == Action.DOUBLE && !canDouble) || (a == Action.SPLIT && !canSplit)) {
    throw new IllegalStateException(label + " chose a move that is not allowed: " + a);
}
// ...
private void settle(Player p, Hand h, Hand dealer) {
    String who = "  " + p.name() + " " + h + " vs dealer " + dealer.bestTotal() + ": ";
    if (h.isBust()) { log.accept(who + "LOSE " + h.bet()); return; }
    if (h.isBlackjack()) {                                  // dealer blackjack was handled by the peek
        p.credit(h.bet() + h.bet() * 3 / 2);
        log.accept(who + "BLACKJACK pays 3:2 (+" + h.bet() * 3 / 2 + ")");
        return;
    }
    int mine = h.bestTotal(), theirs = dealer.bestTotal();
    if (dealer.isBust() || mine > theirs) { p.credit(h.bet() * 2); log.accept(who + "WIN +" + h.bet()); }
    else if (mine == theirs) { p.credit(h.bet()); log.accept(who + "PUSH"); }
    else log.accept(who + "LOSE " + h.bet());
}
```

Output excerpt:

```
  [AS 6H] = soft 17   (hard total 7)
  [AS 6H 10C] = 17   (hard total 17)
  [AS AH 9C] = soft 21   (hard total 11)
== 2. Stacked shoe: split 8s, then double both hands ==
  Alice splits -> [8H 3C] = 11 and [8D 2S] = 10
  Alice hand 1 doubles (bet now 20): [8H 3C 9H] = 20
  Dealer reveals and plays: [6S 10C 7S] = bust 23
  Balances: Alice=140
== 3. Stacked shoe: dealer shows an ace, Bob buys insurance ==
  Dealer peeks: [AS KD] = BLACKJACK -> round over
  Bob insurance pays 2:1 (+10)
  Bob: LOSE 10
  Balances: Bob=100
```

A stacked shoe forces exact scenarios; Bob's insurance hedges his loss to break even.

Concurrency & edge cases

- **One table = one thread** (or actor); tables share only the wallet, which needs atomic debit/credit — debit *before* dealing.
- **Odd antes** (3:2 on 5 = 7.5) → we require even antes; casinos round down or use smaller chips — decide and say it.
- **Split aces**: 21 after a split is *not* blackjack (`fromSplit`). **Shoe exhaustion**: cut card reshuffles between rounds; a stacked shoe throws instead.
- **Can't afford double/split** → `canDouble=false`, and the action is rejected if chosen. **Peek** only on ace or ten.

Extensions & follow-up questions

- **Even money?** Same as insurance with a blackjack (always nets +1× ante); add `takeEvenMoney(hand)` and settle 1:1 before the peek.
- **Surrender?** `Action.SURRENDER` as first decision; refund half.
- **Testing the dealer rule?** Stacked shoe, deal A+6, assert `hitSoft17()` draws and `standOnAll17()` stands.
- **Uno deck?** `Deck<UnoCard>`. **Security?** Cut card limits counting; never send the hole card early.

Real-world example

Online casino tables split a certified **RNG/shuffle service** (regulators such as the UK Gambling Commission require independent testing), a **game engine** enforcing table rules (S17/H17, 3:2 or 6:5), and a **wallet** debiting atomically — rule variants are configuration.

In the interview

30-second answer: "A generic `Card` and `Deck<T>`. A `Shoe` combines decks with an injected `Random`, plus a stacked constructor for tests. The blackjack value lives in the game's scoring, not on `Card`. `Hand` scores itself: hard total with aces as 1, soft if one ace can be 11. Player decisions and the dealer rule are strategies. `BlackjackGame` runs bets, deal, insurance and peek, player turns, dealer turn, and settles 3:2, 1:1 or push."

Likely follow-ups: *Why not `int value` on `Card`?* — value is game-specific. *Best hand value?* — hard + 10 if an ace fits. *Testable?* — seeded `Random`, stacked shoe.

Self-check

1. **Q:** Why does `Hand` store the bet? **A:** After a split each hand has its own bet; a double changes only that hand.
2. **Q:** A+A+9? **A:** Hard 11; one ace as 11 → soft 21.
3. **Q:** When does the dealer peek? **A:** Only on an ace or 10-value up card.
4. **Q:** Is 21 after a split a blackjack? **A:** No — only the original two cards.

TL;DR

- Keep `Card` / `Deck` generic; the value rule lives in the game.

- Hard total + 10 if an ace fits → soft. Blackjack = original two cards totalling 21.
- Strategies for player and dealer; inject randomness; stacked shoe for tests.
- Round = bet → deal → (insurance + peek) → player turns → dealer → settle (3:2, 1:1, push).

Design Chess

Problem & clarifying questions

Chess: two players, 8×8 board, 16 pieces each (**8 pawns, 2 rooks, 2 knights, 2 bishops, 1 queen, 1 king**); goal is **checkmate**. We design an **online** two-player game.

Question	Assumed answer
Full rules?	Yes: all moves, check, checkmate, stalemate, castling, en passant, promotion.
Sides?	Random (`Game.withRandomSides`, injected `Random`); **white first**; alternate.
Take-backs?	Not in rated games (course); `undo()` for analysis and tests.
Endings?	Checkmate, stalemate (draw), resignation, forfeit.
Move log?	Yes.
Timers, rating, AI?	Out of scope.

Requirements

Functional:

1. Two players, a colour each, white first, alternating.
2. Move only your **own** piece; reject out-of-turn moves, impossible moves, and moves leaving your king in check.
3. All special moves:
   - **Castling** — king moves two squares toward a rook, rook jumps over; only if neither moved, squares between empty, king not in check and not passing through or landing on an attacked square.
   - **En passant** — a pawn that advanced two squares beside an enemy pawn may be captured as if it moved one, only on the very next move.
   - **Promotion** — a pawn on the last rank becomes Q, R, B or N.
4. Detect **check**, **checkmate** (in check, no legal move — attacker wins) and **stalemate** (not in check, no legal move — draw).
5. **Resignation** and **forfeit** (losing without choosing, e.g. leaving or timeout).
6. **Log** every move.

Non-functional: new piece types without touching existing ones (Open/Closed); correctness before speed; safe when two player connections call concurrently.

Use cases

Player — create/join games, **make moves**, resign/forfeit, manage account. **Admin** — add, modify, block members. "Make move" is extended per piece type — our `Piece` polymorphism. The move activity `Game.move` follows: valid for the piece? → own king safe? → capture → promote → castling moves the rook → checkmate/stalemate ends the game.

Core entities

Class	Responsibility
`Color`	WHITE/BLACK, `opposite()`.
`Position` (record)	Square a–h/1–8; validates; parses "e4" (course: *Box*).
`Piece` (abstract, sealed)	Colour + `targets(board, from)`; `attacks(...)` for check detection.
`King`, `Knight`, `Pawn`	Step pieces and the pawn rule.
`SlidingPiece` → `Rook`, `Bishop`, `Queen`	One "slide until blocked" loop; subclasses list directions.
`Board`	8×8 grid; `copy()`, `kingOf`, `isAttacked`, `render`; no turns or history.
`Move` + `MoveType`	from, to, type (NORMAL, CASTLE_KINGSIDE, CASTLE_QUEENSIDE, EN_PASSANT, PROMOTION), promotion piece.
`CastlingRights` (record)	Four permissions and how a move changes them.
`Rules`	Stateless: pseudo-legal, legal (king-safety filter), special moves, `apply`, `isInCheck`.
`Game`	Turn, status, winner, en passant square, castling rights, log, undo (Command + Memento), resign, forfeit, random sides. Thread-safe.
`Player`, `GameStatus`, `IllegalMoveException`	Supporting types.

Class diagram

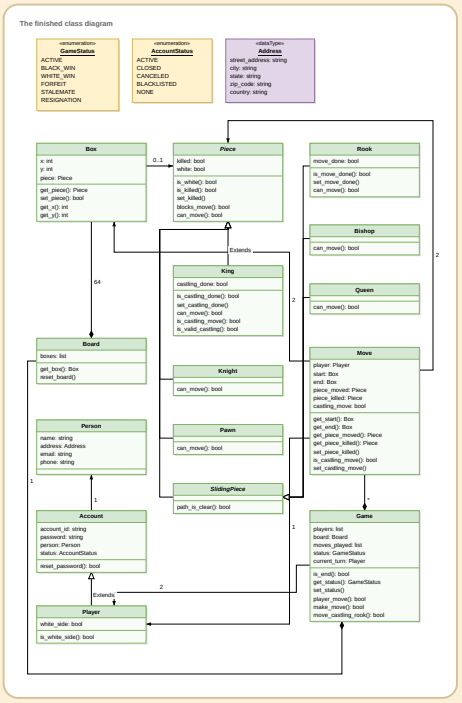
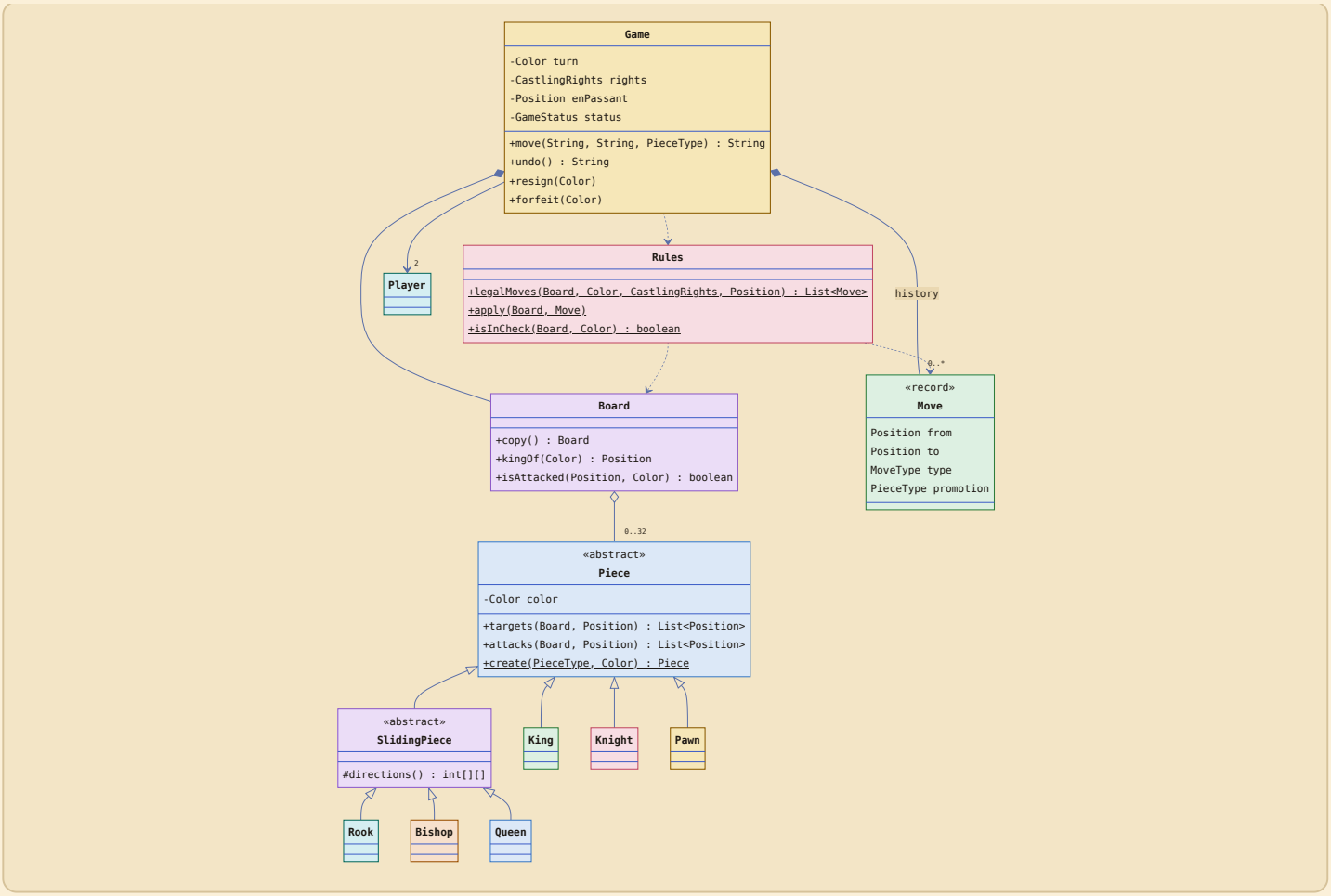


Figure — Course class diagram: Box, Board, Piece hierarchy with SlidingPiece, Move, Player, Account, Game, GameStatus (course diagram)

In the course, `Board` is **composed** of 64 `Box`es; a box refers to a piece by **association** (a captured piece isn't destroyed, so the log is replayable). `SlidingPiece` writes the "path is clear" check once; `Move` records piece moved and captured; course `GameStatus`: ACTIVE, BLACK_WIN, WHITE_WIN, FORFEIT, STALEMATE, RESIGNATION. We fix two limits the course admits — history rules (castling, en passant) had nowhere to live, and checkmate/stalemate weren't detected — by adding `Rules` and a `MoveType` on `Move`.



Key flows

Making a move: check status, piece and colour → `Rules.legalMoves` → match or throw `IllegalMoveException` → save a `Snapshot` (memento) → `Rules.apply` → update rights, en passant, turn → check the opponent → notation (`Qh5xf7#`). **Status:** `ACTIVE` → `CHECKMATE` / `STALEMATE` / `RESIGNATION` / `FORFEIT` (undo can reactivate).

Detecting check, checkmate, stalemate:

1. **Pseudo-legal moves** — the pieces' movement rules + castling, en passant, promotions.
2. **Legal moves** — apply each to a *copy* of the board, keep it only if the mover's king is **not attacked**. This one filter handles pins, moving into check and ignoring check.
3. **King attacked?** — every enemy piece's `attacks(...)`; pawns attack diagonally even onto empty squares.
4. After a move, if the **opponent** has **zero legal moves**: in check = **checkmate**, else **stalemate**.

This one-ply search is all the *rules* need (deeper search is an *engine's* job). Cost: ~200 pseudo-legal moves × an attack scan over ≤ 16 pieces — trivial; engines use bitboards.

Design decisions & patterns used

Decision	Why	Alternative rejected
Polymorphic pieces, template-like `SlidingPiece`.	Each rule in its own class; path loop written once.	Giant `switch (pieceType)`.
Sealed hierarchy.	Closed set; exhaustive `switch`.	Open hierarchy — fine for variants.
Immutable pieces; `Board.copy()` copies references.	Cheap, safe trial moves and snapshots.	Mutable `hasMoved` flags undo must restore.
History rules in `Rules` / `Game`.	Castling and en passant depend on earlier moves.	`King` reading the game log.
`Move` carries a **`MoveType`**.	Castling moves two pieces; en passant captures off-square.	Special cases in `Game` (course's `move_castling_rook`).
Command + Memento undo.	Restoring a snapshot is always correct for special moves.	Hand-written inverse per move type (bug source).
Factory method `Piece.create`.	Set-up and promotion.	Scattered `new`.
`synchronized` `Game`.	A move and a resignation can't interleave.	No locking.

Memento vs incremental undo. A snapshot is 64 references (~0.5 KB) — nothing for a human game. An engine searching millions of positions stores only what changed (captured piece, old rights, old en passant) and reverses it. Know both.

Java implementation

Package `lld.ch17.chess`. Full runnable code: code/lld/ch17/chess/ (see the Full Edition).

```java
public abstract sealed class Piece permits King, Knight, Pawn, SlidingPiece {
    // ...
    /** Squares this piece may move to by its ordinary rules (captures included, own-king safety NOT checked). */
    public abstract List<Position> targets(Board board, Position from);

    /** Squares this piece attacks. Same as targets for every piece except the pawn. */
    public List<Position> attacks(Board board, Position from) { return targets(board, from); }
    // ...
}

public abstract sealed class SlidingPiece extends Piece permits Rook, Bishop, Queen {
    // ...
    protected abstract int[][] directions();

    @Override public final List<Position> targets(Board board, Position from) {
        List<Position> out = new ArrayList<>();
        for (int[] d : directions()) {
            int f = from.file() + d[0], r = from.rank() + d[1];
            while (Position.onBoard(f, r)) {
                Position p = new Position(f, r);
                Piece occupant = board.get(p);
                if (occupant == null) {
                    out.add(p);
                } else {
                    if (occupant.color() != color()) out.add(p);   // capture, then stop
                    break;
                }
                f += d[0];
                r += d[1];
            }
        }
        return out;
    }
}

// Rules.java
public static List<Move> legalMoves(Board board, Color side, CastlingRights rights, Position enPassant) {
    List<Move> legal = new ArrayList<>();
    for (Move m : pseudoLegalMoves(board, side, rights, enPassant)) {
        Board trial = board.copy();
        apply(trial, m);
        if (!isInCheck(trial, side)) legal.add(m);
    }
    return legal;
}

public static void apply(Board board, Move m) {
    Piece piece = board.get(m.from());
    board.set(m.from(), null);
    board.set(m.to(), piece);
    switch (m.type()) {
        case EN_PASSANT -> board.set(new Position(m.to().file(), m.from().rank()), null);   // the passed pawn
        case PROMOTION -> board.set(m.to(), Piece.create(m.promotion(), piece.color()));
        case CASTLE_KINGSIDE -> moveRook(board, m.from().rank(), 7, 5);
        case CASTLE_QUEENSIDE -> moveRook(board, m.from().rank(), 0, 3);
        case NORMAL -> { }
    }
}

// Game.java — inside play(): end-of-game detection
boolean check = Rules.isInCheck(board, turn);
boolean noMoves = Rules.legalMoves(board, turn, rights, enPassant).isEmpty();
if (noMoves) {
    status = check ? GameStatus.CHECKMATE : GameStatus.STALEMATE;
    winner = check ? mover.color() : null;
}

public synchronized String undo() {
    if (history.isEmpty()) throw new IllegalStateException("nothing to undo");
    HistoryEntry last = history.pop();
    Snapshot s = last.before();
    board = s.board(); turn = s.turn(); rights = s.rights(); enPassant = s.enPassant();
    status = s.status(); winner = s.winner();
```

```
    return last.notation();
}
```

`Pawn` overrides `attacks` to return both forward diagonals; `Rules.addCastling` checks rights, empty squares and attacked squares; `CastlingRights.after` clears rights when the king or a rook moves or a corner rook is captured.

Output excerpt:

```
e7-e5 rejected: it is WHITE's turn
a1-a4 rejected: ROOK on a1 cannot move to a4
h5-f7 played as Qh5xf7#
Status: CHECKMATE, winner WHITE
undo Qh5xf7# -> status ACTIVE, WHITE to move, 43 legal moves
c7-d8 played as c7xd8=Q+
b2-a1 rejected: b2-a1 would leave the BLACK king in check
Status: STALEMATE, black in check? false, black legal moves: 0
e1-g1 rejected: cannot castle e1-g1: the king would pass through the attacked square f1
e8-g8 rejected: cannot castle e8-g8: the king is in check
```

Validation, Scholar's mate and undo, a checking promotion, Sam Loyd's 10-move stalemate, and castling refusals with reasons (`Rules.whyCannotCastle`).

Concurrency & edge cases

- **Two players, one game:** `synchronized` public methods; each game has its own monitor, so games don't block each other.
- **Out of turn / game over / empty square** rejected before rule work. **Pins, discovered checks** handled by the copy-and-test filter.
- **Castling through check** tested for the king's crossing and landing squares (the queenside rook's crossing square b1/b8 may be attacked — legal). **Rights** lost when king or rook moves or the rook is **captured** on its corner.
- **En passant expires** after one move. **Promotion** defaults to a queen; underpromotion supported.
- **Not implemented, but name them:** threefold repetition, 50-move rule, insufficient material, clocks, draw offers.

Extensions & follow-up questions

- **Threefold repetition?** `Map<PositionKey, Integer>` keyed on (board, side, rights, en passant); engines use Zobrist hashing.
- **50-move rule?** Half-move clock, reset on pawn moves/captures. **Clocks?** A `Clock` per player (Observer on flag-fall).
- **AI?** A `MoveStrategy` over `legalMoves()`; minimax + alpha-beta. **Chess960/new piece?** New `Piece` subclass + start factory.
- **Spectators/UI?** Observer `GameListener.onMove`; the course's `GameController`/`GameView` is MVC (Ch. 11).

Real-world example

Lichess and Chess.com treat the **server** as the source of truth. Games export as PGN (a move log) and positions as FEN (board, side to move, castling rights, en passant — our `Snapshot` fields); takebacks are a server-side undo.

In the interview

30-second answer: "`Board` is an 8×8 grid of immutable `Piece`s. Each piece overrides `targets`; rook/bishop/queen share a `SlidingPiece` loop. A stateless `Rules` class generates pseudo-legal moves, adds castling, en passant and promotion using history kept in `Game`, and filters out moves that leave your king attacked. If the opponent then has no legal move, it's checkmate if in check, else stalemate. `Game` handles turns, status, the log, and undo via Command plus Memento."

Likely follow-ups: *Checkmate?* — no legal moves + in check. *Where does castling live?* — `Rules`; `MoveType.CASTLE_*` tells `apply` to move the rook. *Why is Box→Piece not composition?* — a captured piece isn't destroyed. *Undo with promotion?* — the memento holds the old board with the pawn.

Self-check

1. **Q:** Why does `Pawn` override `attacks()`? **A:** It moves straight but attacks diagonally, even onto empty squares.
2. **Q:** Checkmate vs stalemate? **A:** Both = no legal move; in check = loss, not in check = draw.
3. **Q:** Three castling conditions? **A:** King and rook unmoved; squares between empty; king not in, through, or into check.
4. **Q:** Why immutable pieces? **A:** Board copies and snapshots can share them safely.

TL;DR

- Polymorphic `targets()`; `SlidingPiece` shares the path loop; pawn overrides `attacks()`.
- `Rules`: pseudo-legal → add specials → drop moves leaving own king attacked.
- No legal moves: in check = checkmate, else stalemate.
- History rules live in `Game` / `Rules`. Undo = Command + Memento. `synchronized` game.

Design Tic-Tac-Toe

Problem & clarifying questions

Players take turns placing X or O; first to fill a **row, column or diagonal** wins; a full board with no winner is a **draw**. Interviewers generalise to **N×N** and ask for **fast win detection**.

Question	Assumed answer
Board size?	Any N ≥ 3.
Win?	A full line of N marks: row, column, or one of the two main diagonals.
Players?	Two by default; more allowed, each with its own mark.
Human or computer?	Both, via a pluggable **move strategy** (human, random, rule-based, scripted).
Invalid moves?	Rejected; same player tries again.

Requirements

Functional: (1) N×N board and 2+ players with unique marks; (2) turns, current player picks an empty cell; (3) validate inside and empty; (4) detect **win** or **draw** after each move; (5) announce result and show board.

Non-functional: win check **O(1)** per move (not O(N²)); new player types without touching `Game`; seeded random bots and scripted players; games share no state.

Use cases

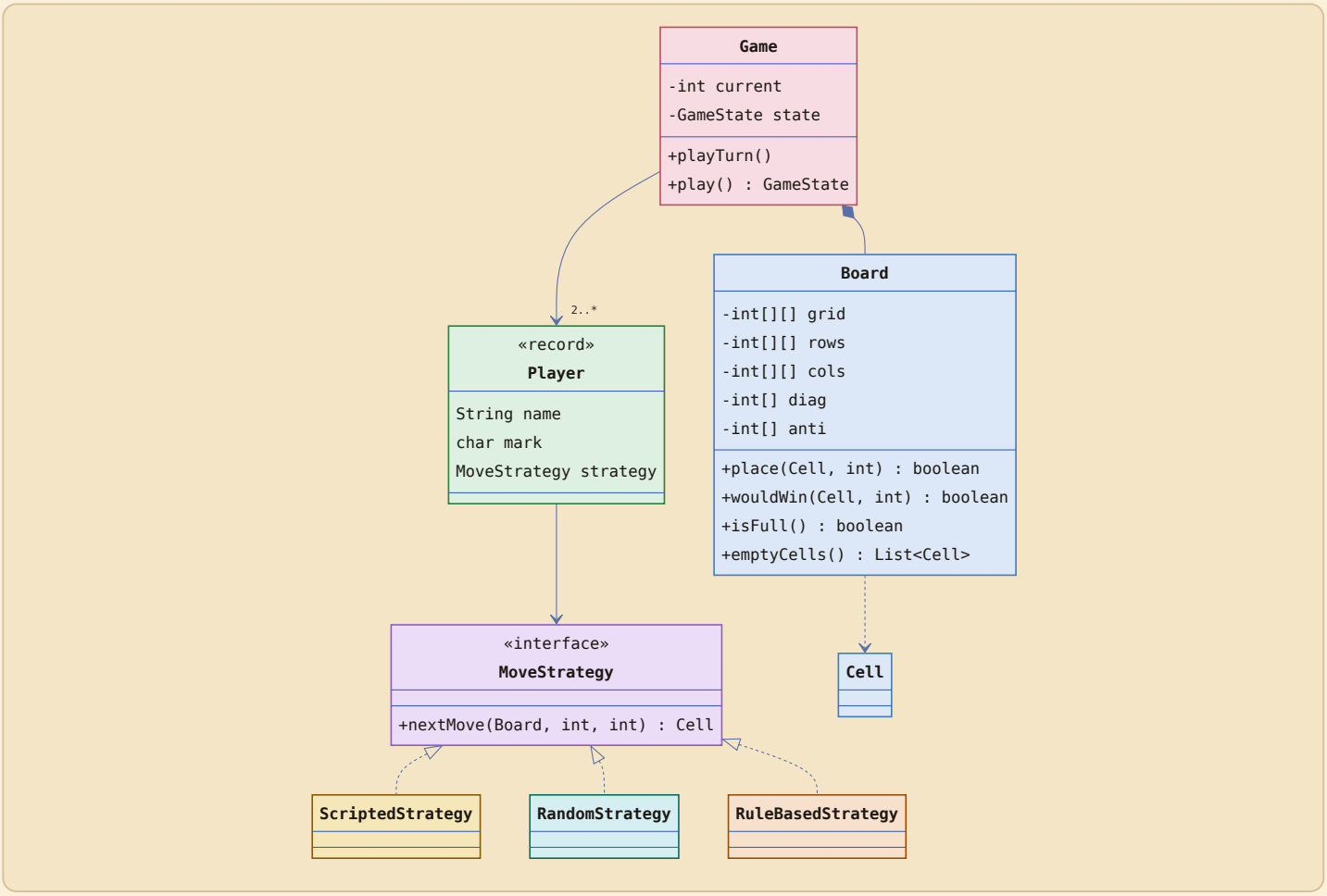
Player (human or bot): start game (choose N and players), make move, view board, see result. The `Game` validates moves, switches turns, declares the result.

Core entities

Class	Responsibility
`Cell` (record)	Row + column.
`Board`	Grid + **per-player line counters**; `place` returns "won?" in O(1); `wouldWin`, `isFull`, `emptyCells`, `render`.
`MoveStrategy`	`ScriptedStrategy`, `RandomStrategy`, `RuleBasedStrategy`.
`Player` (record), `GameState`	Name, mark, strategy; IN_PROGRESS, WON, DRAW.
`Game`	Turn order, validation, state, winner.

Class diagram

No course lesson; from the *awesome-low-level-design* problem set (3×3, X/O turns, line wins, full board draws, validate, announce) — covered and generalised to N×N.



Key flows

O(1) win detection. A move at `(r, c)` can only complete **row r, column c, and maybe the two diagonals**. Keep `rows[p][r]`, `cols[p][c]`, `diag[p]` (`r == c`) and `anti[p]` (`r + c == N-1`); increment the ones the cell belongs to; if any reaches **N**, player `p` wins (only `p`'s marks bump `p`'s counters). Four increments: **O(1) time, O(P·N) memory**.

> **The 2-player trick:** one counter per line, +1 for X and −1 for O, `|count| == N` wins — half the memory but only for two players.

The same counters give **"would this move win?"**: a line already holding `N-1` of `p`'s marks. The rule-based bot uses it to win, else block.

One turn: `nextMove` → `place` → occupied/off-board throws, `Game` logs and the same player goes again; else `WON`, `DRAW` if full, or next player. States: `IN_PROGRESS → WON | DRAW`.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for moves.	Human, random, rule-based, network, minimax plug in.	`if (player.isComputer())`.
Counters inside `Board`.	O(1), next to the data.	Scanning — O(N²), duplicated for "would win".
`place` returns "won?".	A win can only appear right after a move.	A `checkWinner()` callers forget.
Board throws, Game catches.	Board guards its invariant; game sets policy (retry).	Silently ignoring bad moves.
`current = (current+1) % size`.	Any number of players.	X/O toggle.

Java implementation

Package `lld.ch17.tictactoe`. Full runnable code: code/lld/ch17/tictactoe/ (see the Full Edition).

```java
// Board.java — Places a mark and returns true if this move completed a line. O(1).
boolean place(Cell c, int player) {
    if (!inside(c)) throw new IllegalArgumentException("cell " + c + " is off the board");
    if (grid[c.row()][c.col()] != -1) throw new IllegalArgumentException("cell " + c + " is already taken");
    grid[c.row()][c.col()] = player;
    filled++;
    int r = c.row(), k = c.col();
    boolean win = ++rows[player][r] == n;
    win |= ++cols[player][k] == n;
    if (r == k) win |= ++diag[player] == n;
    if (r + k == n - 1) win |= ++anti[player] == n;
    return win;
}

public boolean wouldWin(Cell c, int player) {
    int r = c.row(), k = c.col();
    return rows[player][r] == n - 1 || cols[player][k] == n - 1
            || (r == k && diag[player] == n - 1) || (r + k == n - 1 && anti[player] == n - 1);
}

public boolean isFull() { return filled == n * n; }

/** Win if you can, else block an opponent's win, else take the centre, else the first corner, else anything. */
public final class RuleBasedStrategy implements MoveStrategy {
    @Override public Cell nextMove(Board board, int me, int playerCount) {
        List<Cell> empty = board.emptyCells();
        for (Cell c : empty) if (board.wouldWin(c, me)) return c;
        for (int other = 0; other < playerCount; other++) {
            if (other == me) continue;
            for (Cell c : empty) if (board.wouldWin(c, other)) return c;
        }
        // ... centre, then corners, then empty.get(0)
    }
}

// Game.java
public void playTurn() {
    if (state != GameState.IN_PROGRESS) throw new IllegalStateException("game is over");
    Player p = players.get(current);
    Cell cell = p.strategy().nextMove(board, current, players.size());
    boolean won;
    try {
        won = board.place(cell, current);
    } catch (IllegalArgumentException e) {
        log.accept("  " + p.name() + " tried " + cell + ": rejected (" + e.getMessage() + "), try again");
        return;
    }
    log.accept("  " + p.name() + " (" + p.mark() + ") -> " + cell);
    if (won) { state = GameState.WON; winner = p; }
    else if (board.isFull()) state = GameState.DRAW;
    else current = (current + 1) % players.size();
}
```

Output excerpt:

```
== 1. 3x3 scripted game (with one invalid move) ==
  Xena (X) -> (0,0)
  Omar tried (0,0): rejected (cell (0,0) is already taken), try again
  ...
    X O O
    . X .
    . . X
  Result: Xena wins
== 3. 4x4 rule-based vs rule-based ==
  Result: draw
```

Concurrency & edge cases

- **Many games:** each `Game` owns its `Board`; keep them in a `ConcurrentHashMap<gameId, Game>`, one move at a time per game.
- **Two clicks at once:** `synchronized` `makeMove(player, cell)` that checks the caller is the current player.

- Move after game over → `IllegalStateException`; off-board/occupied → retry; scripted strategy out of moves → `null`, fail fast; N ≥ 3.
- **Centre of an odd board** is on both diagonals → two independent `if`s.

Extensions & follow-up questions

- **K in a row on N×N (Gomoku)?** Counters fail; walk out from the new cell in 4 directions counting own marks — O(K).
- **Unbeatable AI?** Minimax (3×3 has ≤ 9! orders), alpha-beta or memoisation for bigger boards — another `MoveStrategy`.
- **Undo?** Move stack; clear the cell and **decrement** counters. **Online?** Server holds the `Game`, broadcasts state (Observer). **Replay?** Move list + `ScriptedStrategy`.

Real-world example

Google Search's built-in tic-tac-toe offers difficulty levels — different players behind the same rules, the Strategy split here.

In the interview

> **30-second answer:** "`Board` is N×N plus per-player counters for each row, column and both diagonals. A move bumps at most four; if one hits N the player wins — O(1). `Game` holds a player list and turn index, validates moves, and ends on a win or full board. Each `Player` has a `MoveStrategy` — human, seeded random, or rule-based that wins if it can and blocks otherwise."

Time plan: spend 10 minutes on the counter idea — draw it.

Self-check

1. **Q:** Why check only four lines? **A:** A cell is in one row, one column and at most two diagonals.
2. **Q:** Anti-diagonal test? **A:** `row + col == N - 1`.
3. **Q:** Why per-player counters, not +1/−1? **A:** Work for 3+ players and give "would win" for anyone.
4. **Q:** Draw detection? **A:** No win and filled cells = N^2.

TL;DR

> - Per-player row/column/diagonal counters → O(1) win; `N-1` → O(1) "would win".
> - `Game` = players + turn index + state; invalid moves retried.
> - Players via `MoveStrategy`; K-in-a-row needs a directional walk instead.

Design Snake and Ladder

Problem & clarifying questions

A race on a numbered board (usually 1–100): roll, move forward; a **ladder foot** takes you up, a **snake head** down; first to the last square wins. No decisions — the design is about a **clean rules engine**, **turn management** and **testable randomness**.

Question	Assumed answer
Board?	Configurable (default 100, given snakes/ladders); validated.
Dice?	Configurable count and faces (default one d6).
Players?	Two or more, starting off the board (square 0).
Overshoot?	Configurable: **exact finish** or **bounce back**.
Rolling a six?	Roll again; three sixes cancels the turn (on/off).
End?	**First to finish** or **rank everyone**.
Several games?	Yes, independent sessions.

Requirements

Functional: (1) numbered board (~100) with predefined snakes and ladders; (2) several players with one token, off the board; (3) roll and move by the total; (4) snake head → tail, ladder foot → top; (5) overshoot rule; (6) optional bonus on a six with the three-sixes penalty; (7) continue until the win strategy says over; (8) many independent sessions.

Non-functional: deterministic dice (seeded or loaded); pluggable rules without editing the loop; invalid boards rejected **at construction**.

Use cases

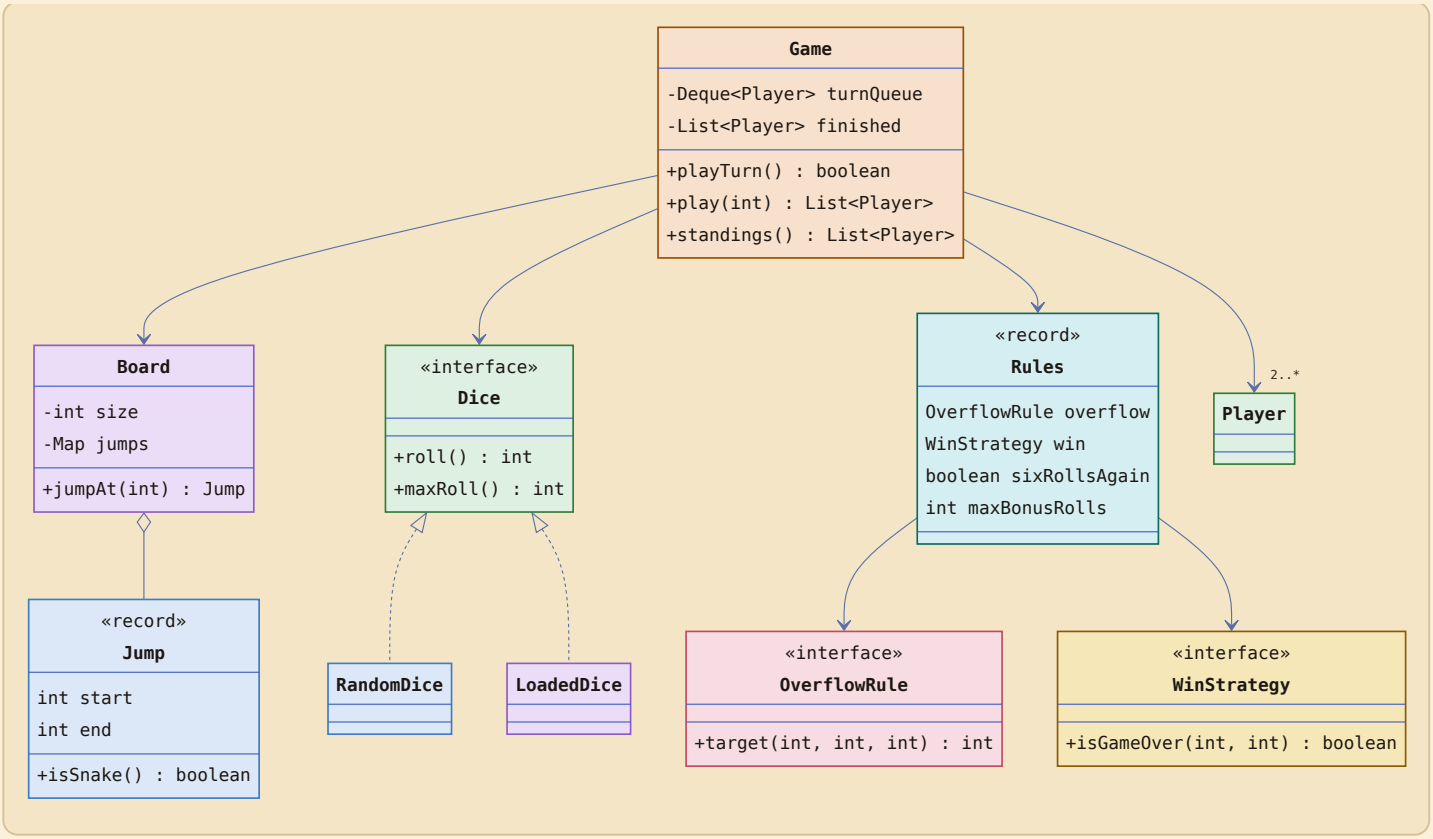
Player (rolls, or the system rolls) and **Game admin/host** (configures board, dice, rules; starts). Use cases: configure board, add players, start, take turn (roll → move → snake/ladder → maybe bonus), declare winner/standings.

Core entities

Class	Responsibility
`Jump` (record)	Snake or ladder: `start → end`; one class, `isSnake()` = `end < start`.
`Board`	Size + `Map<start, Jump>` (O(1)); validates config.
`Dice`	`roll()`, `maxRoll()`; `RandomDice` (seeded), `LoadedDice` (scripted).
`OverflowRule`	`exactFinish()` or `bounceBack()`.
`WinStrategy`	`firstToFinish()` or `rankAll()`.
`Rules` (record)	A variant: overflow, win, six-rolls-again, max bonus rolls.
`Player`, `Game`	Name + position; turn queue, `playTurn`, `play(maxTurns)`, standings, event log.

Class diagram

No course lesson; from the *awesome-low-level-design* problem set (board, snakes and ladders, tokens, turns, win at the last cell, many concurrent sessions — its reference uses a singleton `GameManager`, a thread per session). All covered.



Key flows

One turn: take the next player from the queue → roll → a max roll with bonus enabled that is the 3rd in a row voids the turn (back to the start square) → else `OverflowRule` gives the target → apply a jump starting there → bonus roll if earned and not finished → on the last square add to `finished`, else re-queue → stop when `WinStrategy` says so.

Board validation (once, in the constructor): (1) no jump to itself; (2) no jump starting on square 1 or the last square; (3) no two jumps with the same start; (4) no jump ending on another jump's start — forbids chains and therefore infinite loops.

Design decisions & patterns used

Decision	Why	Alternative rejected
One `Jump` for snakes and ladders.	Same behaviour; direction is data.	Duplicate `Snake` / `Ladder` classes.
`Map<Integer, Jump>`.	O(1); fine for sparse/huge boards.	Array of 101 (also fine).
Strategy for `Dice`, `OverflowRule`, `WinStrategy`.	House rules vary; lambdas keep them tiny.	Flags and `if`s in the loop.
`Rules` record.	One object = one variant ("classic", "kids").	Many `Game` constructor params.
Queue of players.	Finished players just aren't re-queued.	Index arithmetic skipping finishers.
`Consumer<String>` event log.	Engine doesn't print; full design uses **Observer** (`GameListener`).	`System.out` in the engine.
Validate in the constructor; `play(maxTurns)` guard.	Fail fast; a bad rule set can't hang a thread.	Mid-game loop detection; `while (true)`.

Java implementation

Package `lld.ch17.snakeladder`. Full runnable code: code/lld/ch17/snakeladder/ (see the Full Edition).

```java
// Board.java — validate once, up front
public Board(int size, Collection<Jump> all) {
    if (size < 10) throw new IllegalArgumentException("board too small");
    this.size = size;
    for (Jump j : all) {
        if (j.start() == j.end()) throw new IllegalArgumentException("jump to itself: " + j);
        if (j.start() <= 1 || j.start() >= size) throw new IllegalArgumentException("jump cannot start on first/last
square: " + j);
        if (j.end() < 1 || j.end() > size) throw new IllegalArgumentException("jump ends off board: " + j);
        if (jumps.putIfAbsent(j.start(), j) != null) throw new IllegalArgumentException("two jumps start at " +
j.start());
    }
    for (Jump j : jumps.values()) {   // no chains: an end square may not be another jump's start (also rules out
loops)
        if (jumps.containsKey(j.end())) throw new IllegalArgumentException("jump " + j + " ends on another jump's
start");
    }
}

// OverflowRule.java
static OverflowRule exactFinish() { return (from, roll, size) -> from + roll > size ? from : from + roll; }
static OverflowRule bounceBack() { return (from, roll, size) -> from + roll > size ? size - (from + roll - size) :
from + roll; }

// WinStrategy.java
static WinStrategy firstToFinish() { return (finished, total) -> finished >= 1; }
static WinStrategy rankAll() { return (finished, total) -> finished >= total - 1; }

// Rules.java
public record Rules(OverflowRule overflow, WinStrategy win, boolean sixRollsAgain, int maxBonusRolls) {
    public static Rules classic() { return new Rules(OverflowRule.exactFinish(), WinStrategy.firstToFinish(), true,
3); }
}

// Game.java
public boolean playTurn() {
    if (isOver()) throw new IllegalStateException("game is over");
    Player p = turnQueue.poll();
    takeTurn(p);
    if (p.position() == board.size()) {
        finished.add(p);
        log.accept("  *** " + p.name() + " finishes in place " + finished.size() + " ***");
    } else {
        turnQueue.offer(p);
    }
    return isOver();
}

private void takeTurn(Player p) {
    int start = p.position();
    int bonus = 0;
    // ...
    while (true) {
        int roll = dice.roll();
        // ...
        boolean max = rules.sixRollsAgain() && roll == dice.maxRoll();
        if (max && ++bonus == rules.maxBonusRolls()) {
            p.moveTo(start);                      // too many sixes in a row: the whole turn is cancelled
            // ...
            break;
        }
        move(p, roll, line);
        if (!max || p.position() == board.size()) break;
        // ...
    }
    // ...
}
```

`move` asks the overflow rule for the target, then looks up `board.jumpAt(target)` and moves to its end.

Output excerpt:

```
Ann @0: rolls 4 -> 4 (ladder to 14)
Bob @0: rolls 6 -> 6, bonus roll: rolls 3 -> 9 (ladder to 31)
Ann @14: rolls 3 -> 17 (snake to 7)
Bob @31: rolls 6 -> 37, bonus roll: rolls 6 -> 43, bonus roll: rolls 6 -> 3 sixes in a row, turn void, back to 31
rejected: jump Jump[start=10, end=30] ends on another jump's start
winners by seed: {1=Ann, 2=Bob, 3=Cid, 4=Bob, 5=Cid, 6=Cid, 7=Ann, 8=Bob}
same as running each session alone? yes
```

Eight seeded sessions on eight threads match their solo results — sessions share no mutable state.

Concurrency & edge cases

- **Many sessions:** each `Game` owns players, queue and **its own** `Random` (a shared one is contended and breaks per-game reproducibility).
- Rolling after the end → `IllegalStateException`. A finished player is never re-queued. A bonus roll that finishes stops immediately.
- **Three sixes** voids the turn and returns the token to the turn's start (even past a ladder). **Bounce-back onto a snake** is applied.
- **Endless games** impossible with a valid board and fair dice, but `maxTurns` guards custom rules.

Extensions & follow-up questions

- **Special cells?** Generalise `Jump` into a `CellEffect` interface (`apply(player, game)`).
- **Two dice?** `new RandomDice(2, 6, rng)`; 2d6 isn't uniform (7 most likely). **Roll a 6 to enter?** An `EntryRule`. **Pause/resume?** Serialize positions, queue, dice state.
- **UI/spectators?** Observer events (`PlayerMoved`, `SnakeBitten`, …).

Real-world example

Apps such as Ludo King (with a Snakes & Ladders mode) isolate each match, roll dice where players can't tamper (the server), and push moves as events.

In the interview

30-second answer: "`Board` holds size and a map from square to `Jump` — snake or ladder, same behaviour — validated on construction. `Dice` is a strategy: seeded for play, loaded for tests. `Rules` bundles overflow (exact or bounce), win strategy (first or rank all) and the bonus-six rule. `Game` keeps a queue of players; each turn rolls, applies overflow, then any jump, and re-queues the player unless they finished."

Self-check

1. **Q:** Why is one `Jump` class enough? **A:** Both mean "land on start, go to end"; direction is data.
2. **Q:** How is an infinite loop prevented? **A:** No jump may end on another jump's start.
3. **Q:** Testing "three sixes voids the turn"? **A:** `LoadedDice(6, 6, 6)`; assert the position is unchanged.
4. **Q:** Why a queue? **A:** Finishers simply aren't put back.

TL;DR

- Board = size + `Map<start, Jump>`; validate once (no jump on last square, no chains/loops).
- Dice, overflow, win and bonus rules are strategies; seeded or loaded dice.
- Queue-based turns with a `maxTurns` guard; events to a listener, not `System.out`.

Design 2048

Problem & clarifying questions

2048: single-player 4×4 puzzle. Each move slides all tiles one way; two **equal** tiles that collide **merge** into their sum, added to the **score**. After each move that changes the board a new tile (usually 2,

sometimes 4) appears in a random empty cell. Win on a 2048 tile; **over** when no move changes the board.

No course lesson; the common machine-coding statement adds: start with **two 2s**; with **three** equal tiles the pair **nearest the wall** merges, with **four** the first two and last two; input **0 = left, 1 = right, 2 = up, 3 = down**.

Question	Assumed answer
Size?	N (default 4).
New tile?	One per successful move: 2 (p = 0.9) else 4, uniform empty cell; start with two 2s.
Merge rule?	Each tile merges **at most once per move**, from the wall you move toward.
No-change move?	Not a turn: no tile, no score.
Win?	Target tile (2048, configurable); may continue.
Over?	No empty cell **and** no equal horizontal/vertical neighbours.

Requirements

Functional: (1) N×N with two 2s; (2) four directions: slide, merge once, score; (3) spawn a 2 or 4 only if the board changed; (4) detect WON, allow continuing; (5) detect LOST; (6) accept 0/1/2/3 (`Direction.fromKey`).

Non-functional: deterministic spawns via a seeded `Random`; merge logic written **once**, not four times; pluggable spawner (e.g. an "evil" one for hard cases).

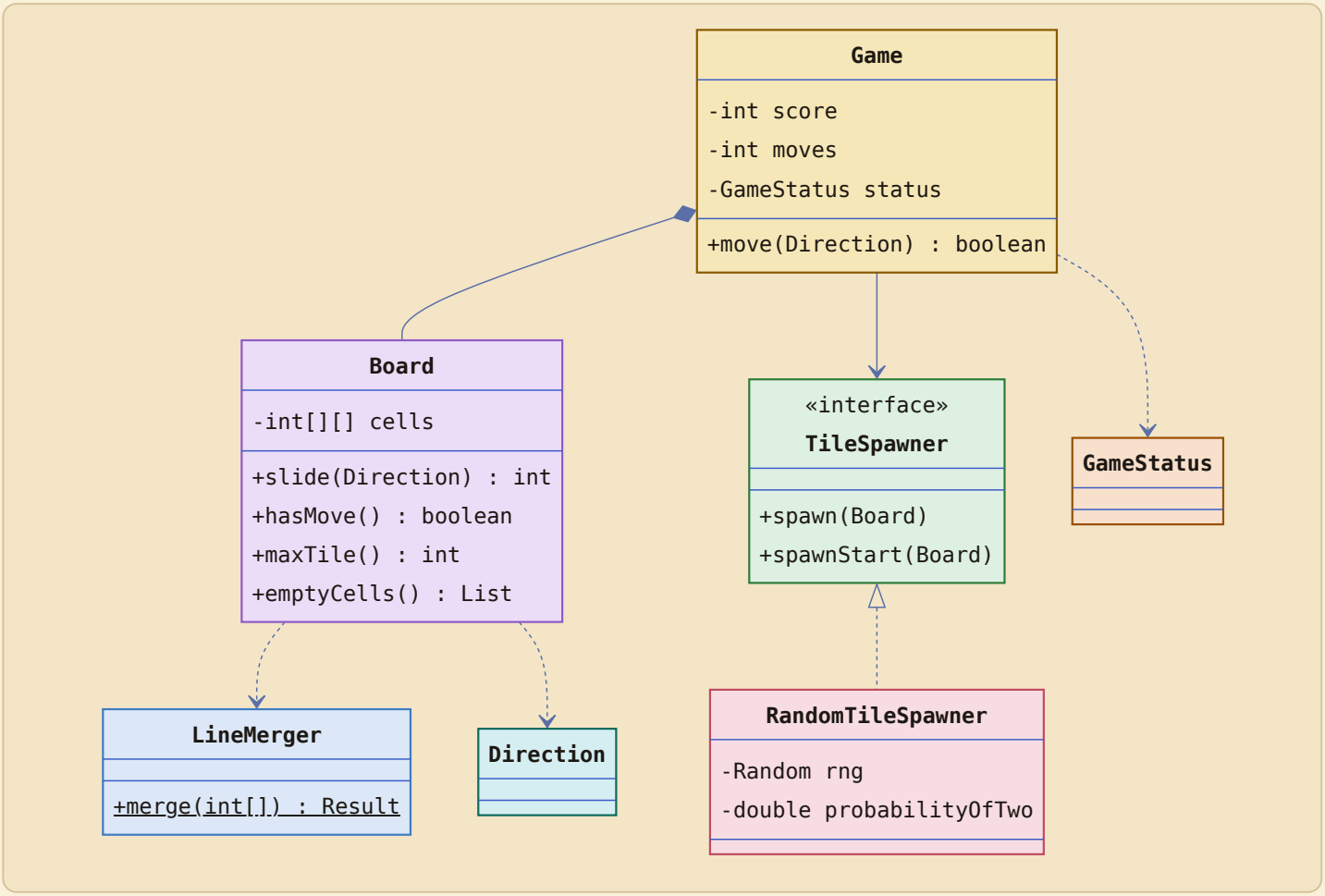
Use cases

Player: start new game, make a move (includes slide and merge, spawn tile, update score), see win, see game over, restart.

Core entities

Class	Responsibility
`Direction`	LEFT, RIGHT, UP, DOWN; `fromKey(0..3)`.
`LineMerger`	Pure function: slide and merge **one line toward index 0**; new line + points.
`Board`	N×N values; maps directions to lines; `slide`, `hasMove`, `maxTile`, `emptyCells`, `render`.
`TileSpawner` / `RandomTileSpawner`	Seeded, 90% twos; `spawnStart` places the two starting 2s.
`GameStatus`, `Game`	PLAYING, WON, LOST; score, moves, status; slide + spawn + status.

Class diagram



Key flows

Merging one line toward index 0: walk from the wall, skip zeros; keep one **pending** tile; if the next equals it, write the double, add points, clear pending (so it **can't merge again**); if different, write pending and make the new tile pending; finally write any pending tile. Examples (left): `[2,2,2,2] → [4,4,0,0]` (not one 8), `[2,2,2,0] → [4,2,0,0]` (pair nearest the wall), `[2,2,4,0] → [4,4,0,0]` (new 4 doesn't re-merge), `[4,0,4,8] → [8,8,0,0]`.

One function for four directions — `coords(direction, i, j)` maps "line i, position j from the wall" to a cell:

Direction	Line i is	j = 0 is
LEFT	row i	column 0
RIGHT	row i	column N-1
UP	column i	row 0
DOWN	column i	row N-1

A move: `slide` merges every line; nothing changed → return `false`, no spawn; else add points, spawn, update status. States: `PLAYING → WON` (target first reached) `→ PLAYING` (keep going); `PLAYING/WON → LOST` (no move left).

Game-over check: a move exists if any cell is empty or two neighbours are equal; comparing each cell with its right and lower neighbour is enough. O(N²).

Design decisions & patterns used

Decision	Why	Alternative rejected
One pure `LineMerger` + direction mapping.	Tricky rule written and tested once (DRY).	Four copies of nested loops — a bug farm.
`LineMerger` is **pure**.	Trivial unit tests.	Merge mixed with board mutation.
Strategy `TileSpawner`.	Seeded tests, evil spawner, variants.	`Math.random()` in `Board`.
No change = no turn.	Matches the real game; no spawn on a no-op.	Always spawning — unfair.
WON set once; keep playing.	Like the original game.	Ending at 2048.

Java implementation

Package `lld.ch17.game2048`. Full runnable code: code/lld/ch17/game2048/ (see the Full Edition).

```java
// LineMerger.java
public static Result merge(int[] in) {
    int n = in.length;
    int[] out = new int[n];
    int write = 0, points = 0;
    int pending = 0;                        // tile waiting for a possible merge partner
    for (int v : in) {
        if (v == 0) continue;
        if (pending == 0) {
            pending = v;
        } else if (pending == v) {
            out[write++] = v * 2;           // merge, and the new tile cannot merge again this move
            points += v * 2;
            pending = 0;
        } else {
            out[write++] = pending;
            pending = v;
        }
    }
    if (pending != 0) out[write] = pending;
    return new Result(out, points);
}

// Board.java — returns the points earned, or -1 if nothing moved
int slide(Direction d) {
    boolean changed = false;
    int points = 0;
    for (int i = 0; i < n; i++) {
        int[] line = new int[n];
        for (int j = 0; j < n; j++) line[j] = read(d, i, j);       // j = 0 is the cell next to the wall
        LineMerger.Result res = LineMerger.merge(line);
        for (int j = 0; j < n; j++) {
            if (read(d, i, j) != res.line()[j]) changed = true;
            write(d, i, j, res.line()[j]);
        }
        points += res.points();
    }
    return changed ? points : -1;
}

private int[] coords(Direction d, int i, int j) {
    return switch (d) {
        case LEFT -> new int[]{i, j};
        case RIGHT -> new int[]{i, n - 1 - j};
        case UP -> new int[]{j, i};
        case DOWN -> new int[]{n - 1 - j, i};
    };
}

public boolean hasMove() {
    for (int r = 0; r < n; r++)
        for (int c = 0; c < n; c++) {
            if (cells[r][c] == 0) return true;
            if (c + 1 < n && cells[r][c] == cells[r][c + 1]) return true;
            if (r + 1 < n && cells[r][c] == cells[r + 1][c]) return true;
        }
    return false;
}

// Game.java
public boolean move(Direction d) {
    if (status == GameStatus.LOST) throw new IllegalStateException("game over");
    int points = board.slide(d);
    if (points < 0) return false;          // nothing moved: no spawn, no turn
    score += points;
    moves++;
    spawner.spawn(board);                  // at least one cell is empty, because something moved or merged
    updateStatus();
    return true;
}
```

Output excerpt:

```
[2, 2, 2, 2] -> [4, 4, 0, 0]  +8
[2, 2, 2, 0] -> [4, 2, 0, 0]  +4
[2, 2, 4, 0] -> [4, 4, 0, 0]  +4
move LEFT changed board? false, moves=0
checkerboard, no empty cell, no equal neighbours -> LOST
reached 128 after 85 moves (status WON), score 852
game over after 232 moves, score 2584, max tile 256
```

A seeded corner-strategy game reaches the 128 target and keeps playing until the board locks.

Concurrency & edge cases

- Single-player, single-threaded; if a UI thread and an input thread both call `move`, make it `synchronized` (or post to the UI event queue).
- **Key repeat:** a no-op move returns `false` — harmless.
- **Spawn needs an empty cell:** guaranteed after a change (tiles move into gaps or merge, freeing a cell); the spawner still checks.
- **Integer overflow:** unreachable on 4×4 (max tile 2^17 = 131072). **Merge order:** `[2,2,2]` left → `[4,2,0]`.

Extensions & follow-up questions

- **Undo?** Memento: push board + score before each successful move.
- **Sizes/targets?** Already parameters (`new Game(5, 4096, spawner)`).
- **AI?** Expectimax (max over moves, average over spawns) as a move-choosing strategy.
- **Animations?** `LineMerger` also returns tile origins for the view (Observer, MVC). **Leaderboard?** A separate service.

Real-world example

2048 (Gabriele Cirulli, 2014, open source) separates a grid model, a game manager and an animating view; new tiles are 2 at 90%, 4 at 10%.

In the interview

> **30-second answer:** "One pure function slides and merges a single line toward index 0: skip zeros, merge equal neighbours once, score the merges. The board maps each direction to lines in the right order, reusing the function four times. After a move that changes the board, a seeded `TileSpawner` adds a 2 (90%) or 4. Won when the target appears; over when no empty cell and no equal neighbours."

Time plan: give `LineMerger` + mapping 15 minutes; write its tests first.

Self-check

1. **Q:** `[4,4,8,8]` moving left? **A:** `[8,16,0,0]`, 24 points.
2. **Q:** Why no spawn on a no-op? **A:** It isn't a turn; spawning would punish harmless key presses.
3. **Q:** Avoiding four copies of slide code? **A:** Map directions to lines, one merge function.
4. **Q:** Repeatable tests? **A:** Seeded spawner or `Board.of(...)`.

TL;DR

- One pure `LineMerger` (merge once, from the wall) + direction→line mapping.
- Spawn only after a real change; 2 (90%) or 4; seeded spawner.
- WON once at the target; LOST with no empty cell and no equal neighbours.

Chapter summary

Game	Core rule object	Pluggable strategies	Randomness for tests	Signature technique
Blackjack	`Hand` + `BlackjackScoring`	`PlayerStrategy`, `DealerPolicy`	Seeded `Shoe`, stacked shoe	Soft/hard totals; generic `Deck<T>`
Chess	`Rules` (legal-move filter)	(bot = move strategy)	None needed	Polymorphic pieces; checkmate = in check + no legal move; Command + Memento
Tic-Tac-Toe	`Board` counters	`MoveStrategy`	Seeded `RandomStrategy`, scripted	O(1) win with counters
Snake & Ladder	`Board` + `Rules`	`Dice`, `OverflowRule`, `WinStrategy`	`RandomDice`, `LoadedDice`	Validated config; queue-based turns
2048	`LineMerger`	`TileSpawner`	Seeded spawner, `Board.of`	One merge function for 4 directions

Patterns used: Strategy (everywhere), Factory Method (`Deck.standard52`, `Piece.create`, `Board.of`), template-style base (`SlidingPiece`), Command + Memento (chess undo), State (status enums; full State pattern suggested for blackjack rounds), Observer (event-log hooks; suggested for UIs).

Three habits interviewers look for: (1) **put every rule in exactly one place** and say where; (2) **inject randomness** so the design is testable; (3) **model states explicitly** and reject actions invalid in the current state.

Chapter 18 — Case Studies: Platforms & Feeds (Part 1) — Study Edition

> **Where this fits.** Chapters 14–17 designed *things*, *bookings* and *games*. This chapter and the next design **platforms**: websites where many people create content, react to each other and build up a history. The objects are people, posts, votes, relationships and feeds. This is the study edition: every topic of the full chapter, without the padding.

#	Problem	What makes it interesting	Main patterns and ideas
18.0	What platform problems test	The toolkit reused in every social design	Facade, Observer, Strategy, services for two-sided operations, inverted index
18.1	Stack Overflow	Reputation unlocks privileges; one vote per member; moderation; bounties	Shared `Post` base, Strategy (reputation), Observer (badges), privilege table
18.2	Cricinfo (+ Udaan "Cricket Match Dashboard")	A real sport's rules; live scorecards; stats from balls	Template Method (stat queries), Observer (live feeds), "derive, don't store"
18.3	LinkedIn	Connections and degrees, invitations, messaging, jobs, search, feed	Service for two-sided operations, State (invitation), BFS, Strategy (feed ranking)
18.4	Facebook	Friends vs followers, privacy lists, pages, groups, feed, suggestions	One visibility policy for feed + search, sealed `Audience`, pull feed, suggestions

Running the code: `cd chapters/18-case-platforms-1/code && javac --release 21 -d /tmp/lld_18 $(find . -name '*.java') && java -cp /tmp/lld_18 lld.ch18.stackoverflow.Main` (or `cricinfo`, `cricketdashboard`, `linkedin`, `facebook`). Java 21, standard library only. A file may hold several small package-private types (e.g. `Posts.java` holds `Post`, `Question`, `Answer`, `Comment`, `Tag`, `Bounty`) to keep a design readable in one place.

18.0 What platform problems test

What it is

A **platform problem** asks you to design a site where *users produce the content* — Stack Overflow, Cricinfo, LinkedIn, Facebook. There is no single "machine" object; instead there is a **rich domain model**: many small classes (profile, post, vote, tag, badge, invitation, group…) each with a little data and a little behaviour. Analogy: a vending machine is a toaster; a platform is a town — people, buildings, entry rules, a notice board (feed), a post office (notifications) and a library catalogue (search).

The problem it solves

Interviewers use platform problems to test six skills at once:

Skill tested	What it means	Where you practise it
Rich domain model	Nouns → classes with clear jobs; knowing which noun is *just data* (a record) and which has *behaviour*	Cricinfo's `Over` knows a wide is not one of its six balls
User roles	Guest, member, moderator, admin, commentator may do different things	Stack Overflow privileges; Facebook admin blocks members
Feeds	A personalised, ranked list of recent content from people and pages you are linked to	LinkedIn and Facebook feed services with a ranking Strategy
Notifications	Telling a user something happened, through **channels** (in-app, email, push)	Observer + Strategy in every design
Search	Finding things by words, using an **inverted index** (word → items containing it)	Question, member and privacy-aware post search
Reputation / trust	Numbers summarising past behaviour that unlock abilities	Reputation and badges; endorsements; page ratings

How it works — the reusable toolkit

1. **A facade in front** (Chapter 9): one class offers `ask`, `vote`, `post`, `invite` and hides the rest; it checks "is this account active / allowed?" before anything changes.
2. **Shared content behaviour goes in a base class** (`Post`, `SocialContent`): voting and liking written once.
3. **Store *who*, not *how many*.** Likes and votes are a **set of member ids**: blocks double votes, allows "unlike", and the count is `set.size()`.
4. **Derive, don't store.** If a number can be computed from data you keep (follower count, batter's runs, page rating), compute it. A stored copy will one day disagree. If too slow, keep a **cache** you can always rebuild.
5. **An operation that changes two objects lives in a service** (a class whose job is to perform an operation). Accepting a connection changes *both* members, so neither member owns it.
6. **Notifications through Observer + Strategy.** The domain publishes events; observers react; the delivery channel is a Strategy, so a new channel is a new class, not a new `if`.
7. **Search is its own class** — a `SearchIndex`, not a method on the entity. Every key maps to a *collection*, even for one match, so the second "Sharma" never breaks it.
8. **Privacy and permissions are asked in one place.** Write `canSee(viewer, post)` once and call it from both feed and search.

Trade-offs

Choice	Benefit	Cost
Rich model (many small classes)	Each rule has one home; easy to test	More classes to explain in 45 minutes
Sets of voter ids instead of counters	No double votes, reversible	Memory grows; real systems use a votes table + cached count
Derive stats from raw events	Never inconsistent	Slow at scale; needs a cache
Services for two-sided operations	Symmetry guaranteed in one place	One more class; overdone, entities become "anemic"

When not to use it

For a tiny problem ("just a like button"), a set on the post is enough — no services, no observers.

Common mistakes

- Search as a static `Question.search()`, so the entity must know every other question.
- `likeCount` as a constructor argument — a post can be created "with 1,000 likes".
- Forgetting that privacy also applies inside search and the feed.
- `acceptInvitation` on the sender (or receiver), updating only one side.

Real-world examples

Kafka was built at LinkedIn to move activity events between systems (industrial Observer). **Facebook's TAO** serves the social graph at huge scale. **Stack Overflow** publishes its reputation-to-privilege table — exactly the `Privilege` enum in 18.1.

In the interview

> **30-second answer:** "Platform problems test whether I can model a domain with many small classes, put each rule in one place, and handle roles, feeds, notifications and search. I'll start with requirements and actors, then core entities, then the tricky flows — voting, accepting requests, privacy — and I'll say which operations need a service because they touch two objects."

- *Why a set instead of a counter?* — A number does not record *who* liked, so it can neither block a repeat nor undo one; the set gives the count for free.
- *Where does notification code go?* — The domain raises an event; a notification service with pluggable channels delivers it.

Self-check

- **Q:** What is an inverted index? **A:** A map from each word to the items containing it.
- **Q:** Why is "accept a friend request" not a method on `Member`? **A:** It changes two members; a service updates both together so the halves cannot drift.
- **Q:** Two notification channels and the pattern that makes adding a third easy? **A:** Email and push; Strategy (`NotificationChannel`).

TL;DR

Platform problems = rich domain model + roles + feed + notifications + search + reputation. Share behaviour in a content base class, store *who* not *how many*, derive numbers, put two-sided operations in services, notify via Observer + Strategy, keep search and privacy each in one place.

18.1 Design Stack Overflow

Stack Overflow is a Q&A community for programmers. Members **upvote/downvote** questions and answers, edit posts wiki-style, and earn **reputation** (points) and **badges** (named awards). Higher reputation unlocks **privileges**. An upvote on an answer is worth +10; the course says +5 for a question upvote, but since November 2019 it is +10 as well, and the code uses today's value.

Problem & clarifying questions

Question to ask	Answer we assume
Who uses it?	Guests, members, moderators, admins, and the system (notifications, badges)
What can a guest do?	Search and read only
What can be voted on?	Questions, answers, comments (comments: upvote only)
Reputation per vote?	Upvote +10 (question or answer); downvote −2 to the author, −1 to whoever downvotes an *answer* (downvoting a question is free); accepted answer +15 (+2 to the asker)
Privileges?	Vote up 15, comment everywhere 50, set bounty 75, vote down 125, create tag 1,500, edit 2,000, close votes 3,000, delete votes 10,000
Closing?	3 close votes from privileged members, or one moderator. Moderators also reopen, delete, undelete
Bounty?	50–500 points staked to attract answers, runs 7 days (+24 h grace on the real site), goes to the answer the sponsor picks. Deducted at once, **never refunded**; if unawarded, the site may auto-award half to a qualifying new answer
Search?	By keyword and tag, plus "most used tags"
Scale?	Read-heavy; votes must never double-count

Requirements

Functional (the course's thirteen): (1) guests search and view, must join to post or vote; (2) members ask; (3) answer **open** questions; (4) comment on questions and answers; (5) upvote questions, answers and comments; (6) **flag** posts for moderators; (7) add a **bounty** to their own question; (8) earn **badges**; (9) vote to **close**, and a moderator can close or reopen alone; (10) add **tags**; (11) vote to **delete** off-topic or low-quality questions; (12) moderators close and undelete; (13) report the **most used tags**. Added from the real site: downvotes, accepting an answer, reputation privileges, blocking accounts.

Non-functional: one vote per member per post, reversible reputation; search and top tags must not scan every question; concurrent votes each counted once; no voting on your own post and blocked accounts can do nothing; reputation, badge and channel rules change without touching voting code.

Use cases

Actors: **Guest** (search, view), **Member** (ask, answer, comment, vote, flag, bounty, tag, delete/undelete own posts), **Moderator** (close, reopen, delete, undelete *any* question), **Admin** (block/unblock), **System** (notifications, badges). In the course's use-case diagram, "Add question" `<<include>>`s "Add tag" and a bounty is an `<<extend>>`. The activity diagram for asking checks, per tag, "does it exist?" and if not "does the member have enough reputation to create it?". Our `ask()` follows the same order.

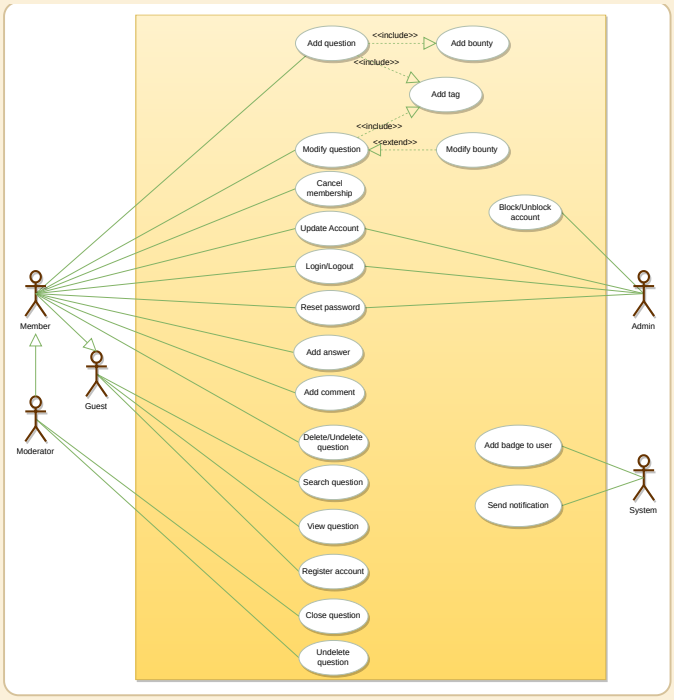


Figure — Use cases per actor: Guest, Member, Moderator, Admin, System (course diagram)

Core entities

Class	Responsibility
`Member` (+ `Moderator`, `Admin`)	Identity, atomic reputation, badges, status; `hasPrivilege(p)`
`Privilege` (enum)	Reputation needed per action — a data table, not scattered `if`s
`Post` (sealed, abstract)	Author, body, votes per voter, flags, comments
`Question`	Title, tags, answers, status, closing remark, close/delete voters, bounty, views
`Answer` / `Comment`	Answer belongs to one question and can be *accepted*; comment belongs to a question or answer (vote/flag counts derived from sets)
`Tag`	Name + description; counts its own uses
`Bounty`	Staked reputation, sponsor, expiry, winner
`Badge` (record)	Name + description
`ReputationPolicy` / `DefaultReputationPolicy`	Strategy: points per event
`ReputationListener` / `BadgeAwarder`	Observer: awards badges on reputation change
`Notifier` + `NotificationChannel` / `EmailChannel`	Pluggable delivery
`Search` / `QuestionIndex`	Inverted index by word and tag
`StackOverflow`	Facade: status + privilege checks, delegation, reputation

Class diagram

The course builds the diagram in five stages:

1. **`Badge`**, **`Tag`**, **`Notification`** (reqs 8, 10, 13). `Tag` counts its own uses, so "top tags" is a quick read. `EmailNotification` extends abstract `Notification` (passes the substitution test).
2. **`Photo`**, **`Bounty`** (req 7). The bounty owns the staked amount. The course says it can go back to the sponsor; on the real site it is **not** refundable, so our code deducts it at once and never returns it.
3. **The posts** — the heart. Voting and flagging work the *same* on three classes, so that lives in abstract `Post`; what they *contain* differs, so there are three subclasses, not one class with a "kind" field.

`Post` remembers **who** voted. Filled diamonds (composition): deleting a question deletes its answers, comments, photos, bounty. `Tag` is a plain association — it outlives any question.

4. **Search** — `Search` interface + `QuestionIndex`, not a static method on `Question`.
5. **Accounts** — `Guest` ← `Member` ← `Moderator`/`Admin`. Requirement 1 is enforced by *shape*: a guest has no `createQuestion` method. `Member` owns its badges.

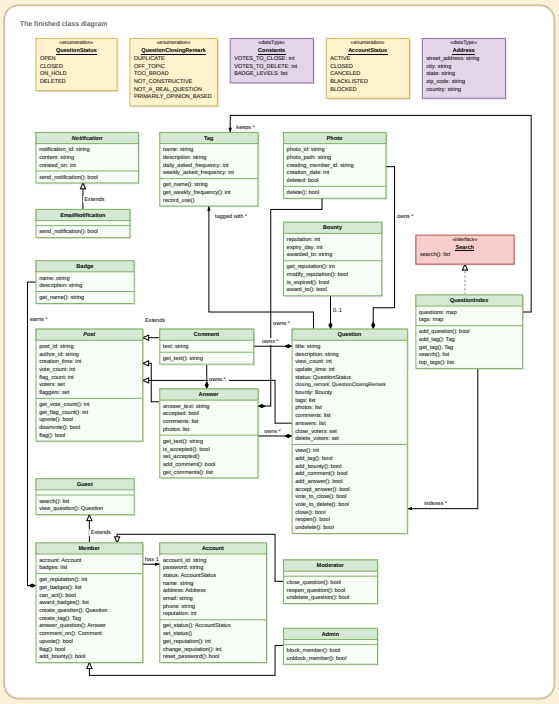
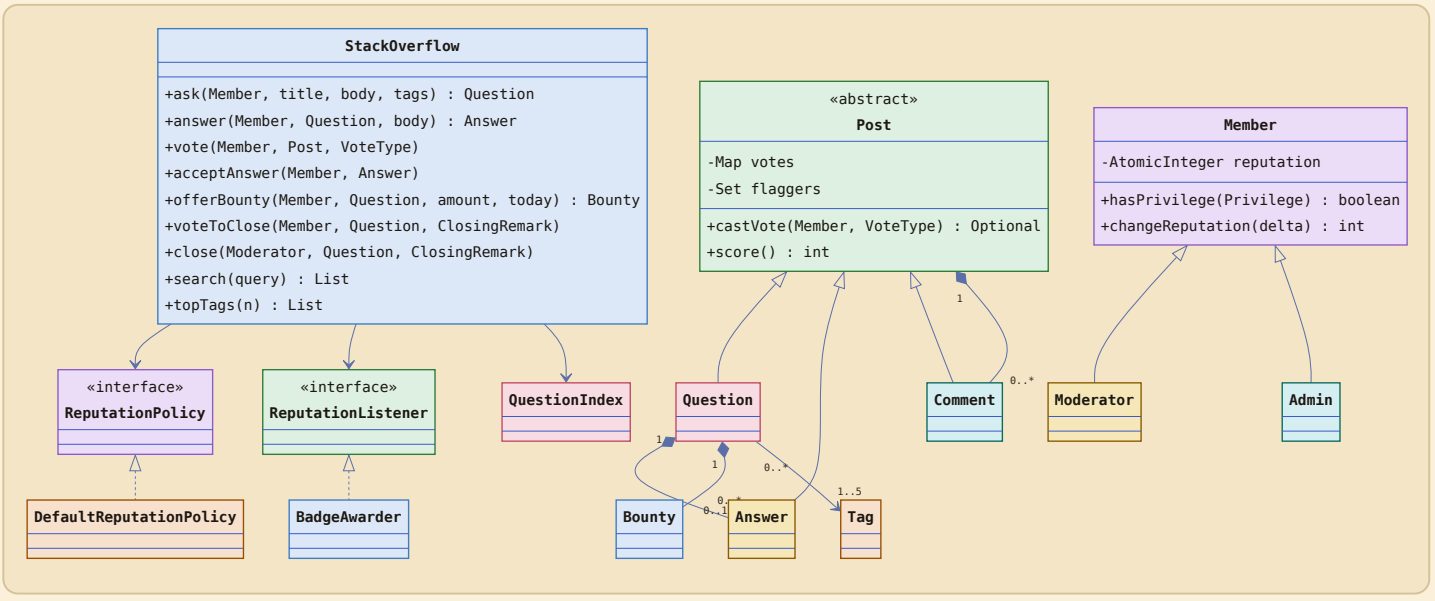


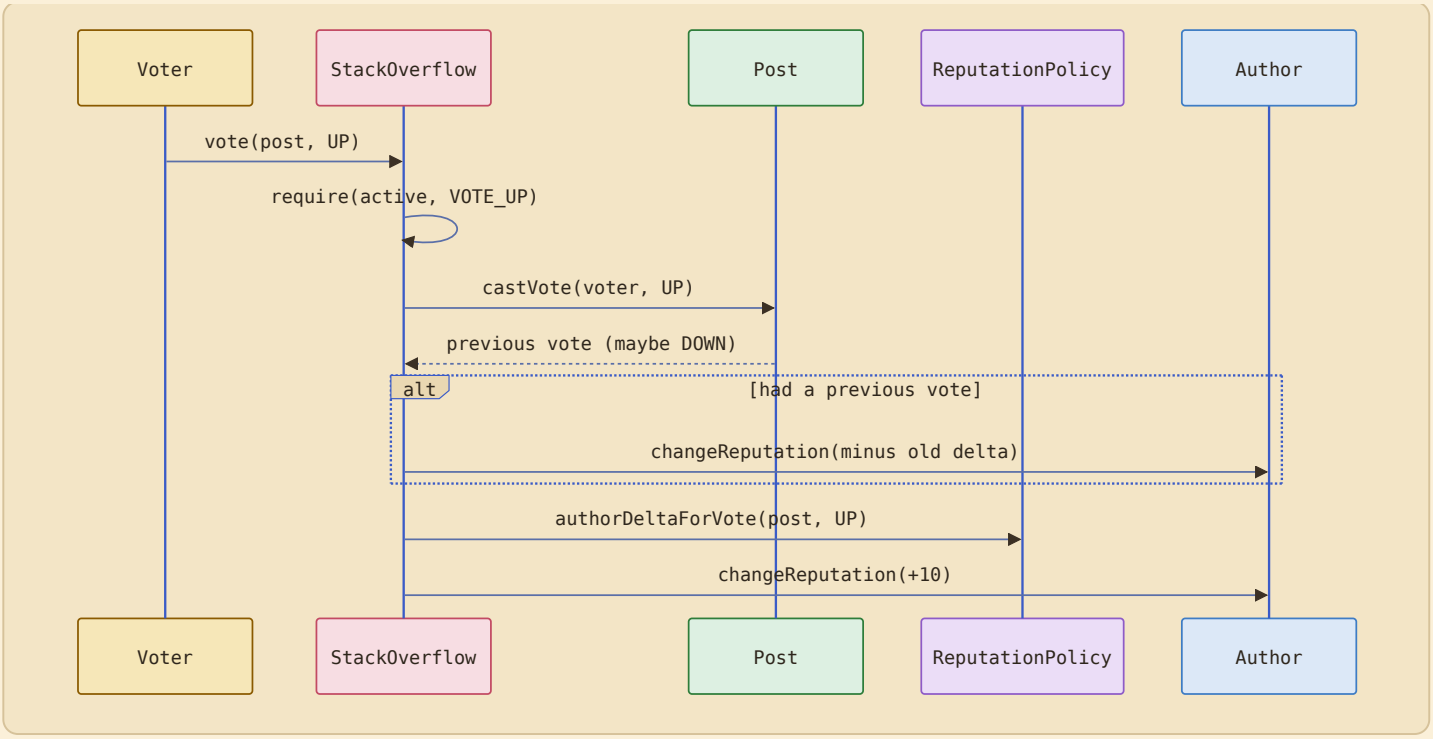
Figure — The finished Stack Overflow class diagram (course diagram)

Our Java design keeps this structure and adds a reputation **Strategy**, a badge **Observer** and a data-driven **privilege table**. A guest is simply "no `Member` object": every write needs a `Member` argument. We also add the downvote cost the course leaves out; edit history and duplicate links are extensions; chat is out of scope.



Key flows

Voting (with a changed vote). The facade checks privilege, asks the post to record the vote; the post returns the voter's *previous* vote so the facade can reverse its reputation effect before applying the new one; listeners (badges) are told.



Question lifecycle: OPEN → CLOSED (3rd close vote or moderator) → OPEN (moderator reopen); CLOSED → DELETED (3rd delete vote or moderator); OPEN → DELETED (moderator); DELETED → CLOSED (undelete). No new answers while closed.

Bounty: `offerBounty` checks `SET_BOUNTY` and the 50–500 range, attaches it to the open question and deducts the amount immediately; `awardBounty` (sponsor only, not their own answer, before expiry) credits the winner.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`Post` base for question/answer/comment	Inheritance, `sealed`	Voting, flagging, commenting identical	One class + `type` field → `if (type == …)` everywhere
Votes as `Map<voterId, VoteType>`	Store who, not how many	Blocks double votes; allows reversal	`int voteCount`
`ReputationPolicy`	**Strategy**	Scoring rules change (they did in 2019)	Hard-coded numbers in `vote()`
`ReputationListener` + `BadgeAwarder`	**Observer**	Badges, notifications react without the facade knowing	Calling `awardBadges()` inside `vote()`
`Privilege` enum with thresholds	Table-driven	One place for rules	`if (rep >= 125)` scattered
Moderator-only methods take `Moderator`	Type-level permission	Cannot call `close(…)` without a moderator	Forgettable runtime role check
`QuestionIndex` separate	SRP	Swap for Elasticsearch behind `Search`	Static `Question.search()`
`Tag` counts its uses	Aggregate at write time	Top tags = sort a small collection	Scan all questions
`StackOverflow` facade	**Facade**	One entry that validates, delegates, logs	UI calling entities directly
Pattern-matching `switch` over sealed `Post`	Exhaustive dispatch	Compiler forces every post kind	`instanceof` chains

Java implementation

Package `lld.ch18.stackoverflow` (`Types.java`, `Accounts.java`, `Posts.java`, `Reputation.java`, `QuestionIndex.java`, `StackOverflow.java`, `Main.java`). The core: vote storage, the policy, the badge observer and the facade's `vote`.

```
// File: lld/ch18/stackoverflow/Posts.java (excerpt)
abstract sealed class Post permits Question, Answer, Comment {
    // ...
    private final Map<Long, VoteType> votes = new HashMap<>();
    private final Set<Long> flaggers = new HashSet<>();
    // ...
    /** Records or changes a vote and returns the voter's previous vote, if any. */
    synchronized Optional<VoteType> castVote(Member voter, VoteType type) {
        if (voter.id() == author.id()) throw new IllegalStateException("you cannot vote on your own post");
        VoteType previous = votes.put(voter.id(), type);
        if (previous == type) throw new IllegalStateException(voter.name() + " already voted " + type + " on this
post");
        return Optional.ofNullable(previous);
    }

    synchronized int score() {
        return votes.values().stream().mapToInt(v -> v == VoteType.UP ? 1 : -1).sum();
    }
    // ...
}

// File: lld/ch18/stackoverflow/Reputation.java (excerpt)
final class DefaultReputationPolicy implements ReputationPolicy {
    @Override public int authorDeltaForVote(Post post, VoteType type) {
        return switch (post) {
            case Question q -> type == VoteType.UP ? 10 : -2;   // +5 before November 2019
            case Answer a -> type == VoteType.UP ? 10 : -2;
            case Comment c -> 0;                       // comment votes earn no reputation
        };
    }
    @Override public int voterDeltaForVote(Post post, VoteType type) {
        return post instanceof Answer && type == VoteType.DOWN ? -1 : 0;
    }
    @Override public int acceptedAnswerBonus() { return 15; }
    @Override public int acceptorBonus() { return 2; }
}

final class BadgeAwarder implements ReputationListener {
    // ... levels: 15 Contributor, 50 Helpful, 125 Trusted
    @Override public void onReputationChanged(Member m, int oldValue, int newValue, String reason) {
        for (Level level : levels) {
            if (oldValue < level.reputation() && newValue >= level.reputation() && m.addBadge(level.badge())) {
                notifier.send(m, "You earned the '" + level.badge().name() + "' badge");
            }
        }
    }
}

// File: lld/ch18/stackoverflow/StackOverflow.java (excerpt)
    void vote(Member voter, Post post, VoteType type) {
        require(voter, type == VoteType.UP ? Privilege.VOTE_UP : Privilege.VOTE_DOWN);
        if (post instanceof Comment && type == VoteType.DOWN) throw new IllegalStateException("comments can only be
upvoted");
        Optional<VoteType> previous = post.castVote(voter, type);
        log(voter.name() + " voted " + type + " on " + post.label() + " -> score " + post.score());
        previous.ifPresent(p -> {                                  // undo the old vote's effect first
            adjust(post.author(), -policy.authorDeltaForVote(post, p), "vote changed on " + post.label());
            adjust(voter, -policy.voterDeltaForVote(post, p), "vote changed");
        });
        adjust(post.author(), policy.authorDeltaForVote(post, type), type + " on " + post.label());
        adjust(voter, policy.voterDeltaForVote(post, type), "downvoted an answer");
    }
```

Full runnable code: code/lld/ch18/stackoverflow/ (see the Full Edition).

Output (excerpt) — a changed vote reverses the old effect first:

```
  REJECTED: alice needs 1500 reputation for CREATE_TAG (has 200)
erin voted DOWN on answer #103 by carol -> score -1
    rep carol  -2 -> 3198 (DOWN on answer #103 by carol)
    rep erin   -1 -> 3099 (downvoted an answer)
erin voted UP on answer #103 by carol -> score 1
    rep carol  +2 -> 3200 (vote changed on answer #103 by carol)
    rep erin   +1 -> 3100 (vote changed)
    rep carol  +10 -> 3210 (UP on answer #103 by carol)
erin voted to close question #107 as DUPLICATE -> CLOSED
  REJECTED: question is CLOSED, no new answers
```

Key points: `votes.put` both records the vote and returns the previous one (voting the same way twice throws). The sealed `Post` makes the policy's `switch` exhaustive. `Member.changeReputation` uses `AtomicInteger.updateAndGet` with a floor of 1. `BadgeAwarder` fires only when a threshold is *crossed*, and `Set.add` stops duplicate badges. Alice (200 rep) cannot create the new tag `hashmap`. The third close vote closes the question; a moderator reopens it.

Concurrency & edge cases

- **Simultaneous votes:** `castVote`, `score`, `flag` are `synchronized` per post; reputation is `AtomicInteger`, so both changes apply.
- **Vote and reputation not in one lock:** a reader may briefly see the vote before the reputation — acceptable. In a database: one transaction plus a **unique constraint** on `(post_id, voter_id)`.
- **Reputation floor of 1** makes reversal not perfectly symmetric (same quirk as the real site); keep a ledger of events if exactness matters.
- **Accepting a different answer:** `Question.accept` returns the previous one so its +15 is removed; the asker's +2 is given once.
- **Self-actions** (vote own post, award own bounty) rejected; **closed** questions take no answers, **deleted** ones are hidden from search; **blocked** members fail `requireActive`; **bounty** award refused after expiry.

Extensions & follow-up questions

- **Stop double voting** — the per-post voter map; in a DB, a unique key on `(post_id, voter_id)`.
- **Edit history** — immutable `Revision`s appended per edit (close to **Memento**).
- **Daily reputation cap (200/day)** — a `CappedReputationPolicy` **decorator**; no voting code changes.
- **Duplicate link** — `duplicateOf` on `Question` set when closed as `DUPLICATE`.
- **Review queues** — flagged posts go to a `ReviewQueue` where reviewers vote keep/delete.
- **Hot questions** — ranking Strategy over recent votes/answers/views with time decay.
- **Tag frequency per day/week** — counters per time bucket rolled over by a scheduled job.

Real-world example

The numbers are real stackoverflow.com rules (privileges page: 15, 50, 75, 125, 1,500, 2,000, 3,000, 10,000). The 2019 change of question upvotes from +5 to +10 touches only the `ReputationPolicy`; lowering close votes from 5 to 3 after a 2019 experiment is one constant in our code.

In the interview

Minutes	What to do
0–5	Actors and reputation rules; requirements
5–10	Use cases; "guest has no posting method"
10–20	Class diagram: `Post` + 3 subclasses, `Member` hierarchy, `Tag`, `Bounty`, index, facade
20–30	`vote()` and `acceptAnswer()` with reversal; question states
30–40	Code `Post.castVote`, the policy, facade `vote`
40–45	Concurrency and extensions

30-second answer: "Questions, answers and comments share voting and flagging, so they extend one `Post` that stores votes per voter — that stops double voting and lets me reverse a changed vote. Reputation

rules are a Strategy; badges are an Observer on reputation changes. Privileges come from a reputation table; moderator actions take a `Moderator` parameter. Search is a separate inverted index, and tags count their own uses for 'top tags'."

- *Why not store `voteCount` only?* — It cannot prevent duplicates or support changes; derive it from the voter map.
- *Is reputation awarded inside `upvote`?* — No; the facade asks the policy, so posts know nothing about scoring (the course's suggested improvement).
- *Scale search?* — Keep `Search`, replace `QuestionIndex` with a search engine fed by events.

Self-check

- **Q:** Which requirement is enforced by class shape? **A:** Guests cannot post or vote — no `Member` object, and every write needs one.
- **Q:** Why is `Post` abstract and sealed? **A:** A bare post is never created; sealed makes switches over post kinds complete.
- **Q:** Erin changes a downvote on an answer to an upvote — reputation? **A:** Old effect reversed (author +2, Erin +1), new applied (author +10).
- **Q:** How do comments differ in voting? **A:** Upvote only, and they earn no reputation.

TL;DR

Stack Overflow = `Post` base (votes per voter, flags, comments) + `Question`/`Answer`/`Comment` + reputation **Strategy** + badge **Observer** + reputation **privilege table** + moderator-typed methods + inverted-index search + a facade that checks, delegates and scores.

18.2 Design Cricinfo

Cricinfo (ESPNcricinfo) is a cricket site with **live ball-by-ball commentary**, a database of historic matches, and news. This design is graded on whether you model the *rules*, not just the nouns. Cricket primer:

Term	Meaning
Delivery / ball	One ball bowled by the **bowler** to the **batter** on strike
Over	Six **legal** deliveries by one bowler. A **wide** or **no ball** gives a penalty run and is re-bowled — it does not count toward the six
Striker / non-striker	The striker faces; they change ends on an odd number of runs (1, 3, 5) and after every over
Runs	Off the bat (batter's) or **extras** — wides, no balls, **byes**, **leg byes** (the team's, not the batter's)
Wicket	Bowled, caught, LBW, stumped, hit wicket, run out, obstructing the field. **A run out is not the bowler's**
Innings	One team's turn to bat; ends all out (10 wickets), overs used up, or target reached
Formats	**T20** (20 overs/side), **ODI** (50 overs/side), **Test** (no limit, two innings per side)
Squad / playing eleven	A team names a squad per tournament, then picks exactly **eleven** from it per match

Problem & clarifying questions

Question	Answer we assume
Formats?	T20, ODI, Test
Who enters data?	**Admin** (scorer) records each ball; **Commentator** adds text
Real rules?	Yes: six legal balls, extras, wicket credit, strike rotation, bowler limits
Live updates?	Push to subscribers after every ball
Stats?	Per player, match, tournament, plus global ("highest wicket taker ever")
Tournaments?	Yes, with squads and points table
Out of scope?	Duckworth–Lewis–Stern, follow-on and declarations in Tests, powerplays, free hits, leg/off-side wides, partnerships, net run rate, news (our code adds a simple `declare()`)

Requirements

Functional (the course's eight): (1) track every team and its matches; (2) live ball-by-ball commentary; (3) follow international rules; (4) a squad per tournament; (5) a playing eleven per match from the squad; (6) stats for players, matches, tournaments; (7) global stat questions ("most wickets ever", "most Test hundreds"); (8) ODI, Test and T20.

Non-functional: stats exactly match the ball record (a correction fixes every derived number); live updates within seconds to millions; two scorers must not interleave balls; a new stat, format or subscriber is a new class.

Use cases

Admin adds/modifies teams, players, tournaments, matches, stadiums, umpires, referees and records balls (some stats system-generated); **Commentator** adds commentary. In the use-case diagram "Add match" is *extended* by the three formats and "Add ball" *includes* its runs and wicket. Activity "record a ball": choose ball type (normal, wide, no ball, wicket) → wicket details if any → a run record per run type → commentary → save.

Core entities

Class	Responsibility
`Person` (record)	Name, country; roles *hold* a person
`Player`; `MatchOfficial` → `Umpire`, `Referee`, `Commentator`	Roles; officials share "assign to match"
`Team` ⊇ `TournamentSquad` ⊇ `Playing11`	Everyone ever picked ⊇ one tournament ⊇ exactly eleven, in batting order
`Tournament`	Owns squads, lists matches, points table
`BallOutcome` (record)	Bat runs, extra type/runs, boundary?, wicket type, who was out, commentary; validates the laws
`Ball`, `Wicket` (records) + `WicketType`	Recorded delivery; `creditedToBowler()`
`Over`	Six **legal** deliveries; runs conceded; maiden?
`Innings`	Owns overs; decides striker, over end, new batter, bowler limits, innings end
`Match` (abstract) → `T20`, `ODI`, `Test`	Formats override only overs per innings and innings per side
`Result` (sealed) → `Win`, `Tie`, `Draw`	Outcome
`MatchObserver` → `CommentaryFeed`, `LiveScorecard`	Live subscribers
`PlayerStats`, `StatsService`, `StatQuery` → `MostRuns`, `MostHundreds`, `MostWickets`, `MostSixes`	Derived stats; Template Method queries

Class diagram

The course builds it in six stages:

1. **People** — `Player` *holds* a `Person` (composition over inheritance: one human can play, then commentate). Officials share assignment in `MatchOfficial`.
2. **Teams, squads, eleven** — three nested sets. `Playing11` keeps its squad so it rejects outsiders and a twelfth pick: requirement 5 lives in code.
3. **The delivery** — three hidden rules: an over is six *legal* balls (two wides → eight deliveries); a **run out is not the bowler's wicket**; byes and leg byes are the team's. `Ball` owns runs, wicket, commentary; it only *refers* to bowler and batter.
4. **The innings** — owns the overs, so it alone decides when an over ends. `score` uses runs/wickets and overs.balls notation.
5. **Match and formats** — only two numbers differ (overs per innings, innings per match), so `Match` is abstract and each format overrides two short methods. Test returns 0 overs ("no limit") and four innings total.
6. **Stats** — **worked out from the balls**, never stored beside them (a stored counter drifts after a correction). Fifties/hundreds counted per innings (`countInnings`) beside `countBall`. Each global question is a small `StatQuery` subclass.

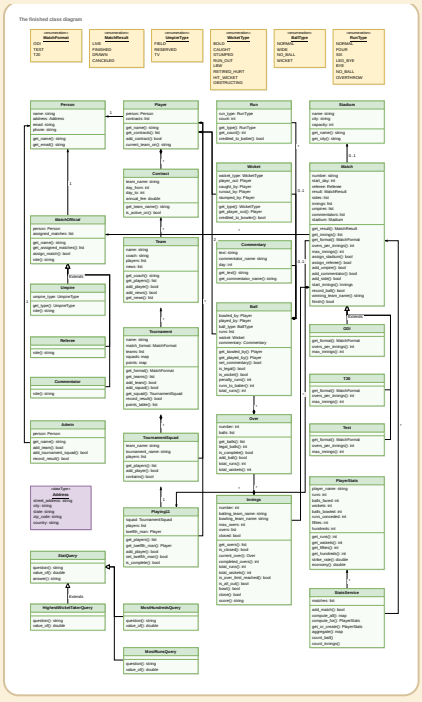
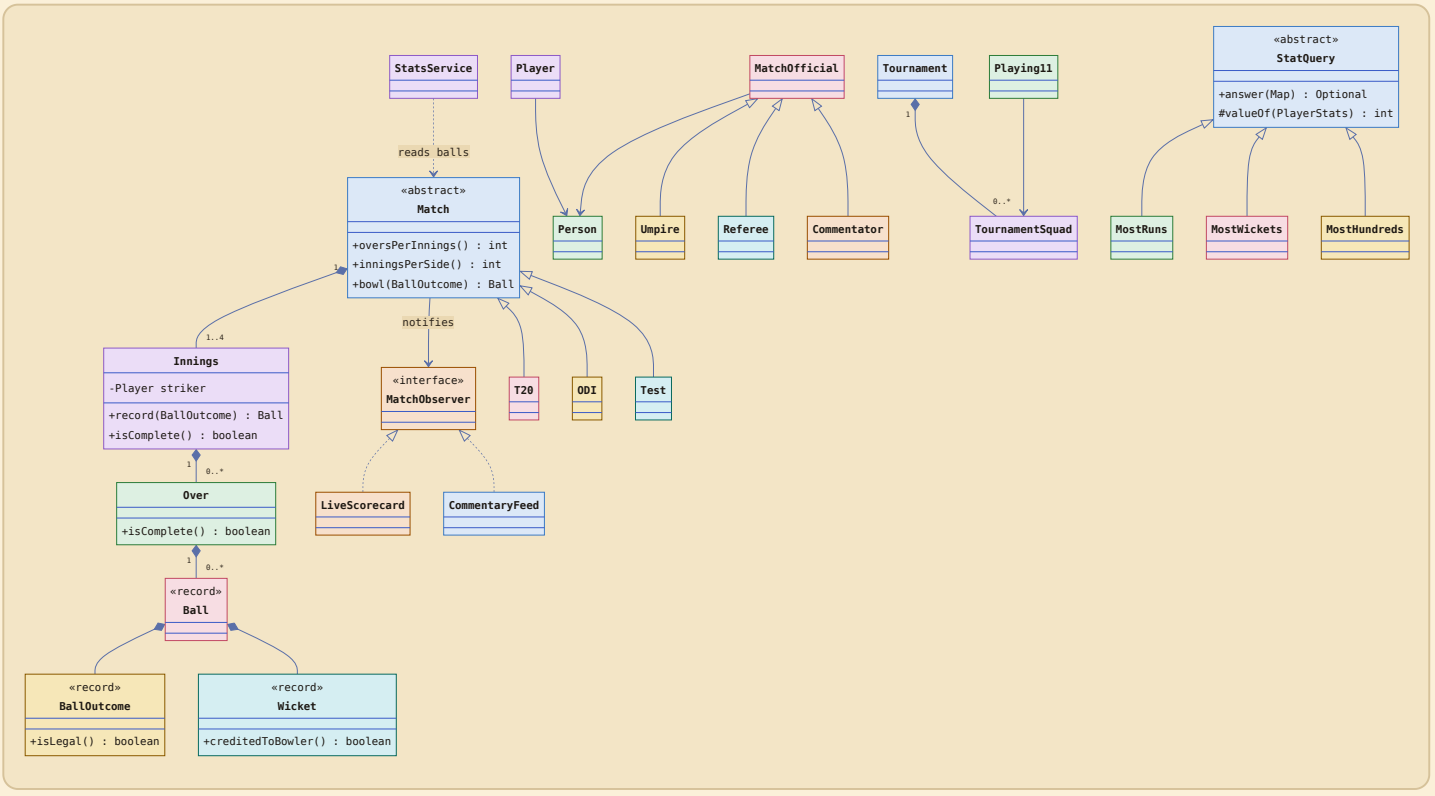


Figure — The finished Cricinfo class diagram (course diagram)

Our Java design adds what the course flags as missing: **strike rotation**, bowler rules (not two overs in a row; 4 overs per bowler in T20, 10 in ODI), which dismissals are legal off a wide/no ball, result calculation, and **Observers**. A `BallOutcome` record replaces separate `Run` objects.



Key flows

Recording a ball: scorer → `Match.bowl(outcome)` → `Innings.record`: build the `Ball` with current bowler and striker, add to the over, add runs; wicket → new batter; odd runs run → swap ends; over complete → swap ends. The match then notifies `onBall`, and `onOverEnd` / `onInningsEnd` if finished; after the last innings or target reached, `finish()` computes the `Result` and calls `onMatchEnd`.

Innings end (`Innings.isComplete`): all out (wickets = players − 1) → final innings and runs ≥ target → over limit reached and last over complete → declared.

Match status: SCHEDULED → LIVE → FINISHED; LIVE → DRAWN (Test out of time); SCHEDULED → CANCELED (washed out).

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Roles hold a `Person`	Composition over inheritance	One human, many roles	`Player extends Person`
`Match` abstract; formats override two numbers	Template-style base (not a named pattern here)	Only what differs is overridden	Three copies of `Match`
`StatQuery.answer()` final, `valueOf()` abstract	**Template Method**	New statistic = one tiny subclass (OCP)	`switch (questionType)`
Stats derived from balls	Single source of truth	Corrections fix every number	Stored counters that drift
`Innings` owns over rule and strike	Information Expert	Class with the data decides	UI deciding when an over ends
`BallOutcome` validates in constructor	Fail fast, immutable	Bowled off a no ball never enters the record	Checking later in stats
`MatchObserver` (default methods)	**Observer**	Feeds subscribe without the match knowing	Match printing itself
`Result` sealed records	Algebraic data type	Exhaustive `switch`	Strings like "Team A won"
`Match.bowl` `synchronized`	Monitor lock per match	Scorers cannot interleave	Unsynchronised updates

Java implementation

Package `lld.ch18.cricinfo` (`People`, `Teams`, `Delivery`, `Innings`, `Match`, `Observers`, `Stats`, `Main`). The demo shows both eleven refusals, a scripted first over (a wide and a no ball make **eight deliveries**; a bowled that counts for the bowler; a run out that does not), then a seeded simulation of a T20 and an ODI.

```java
// File: lld/ch18/cricinfo/Delivery.java (excerpt)
enum WicketType {
    BOWLED, CAUGHT, LBW, STUMPED, HIT_WICKET, RUN_OUT, OBSTRUCTING_THE_FIELD;

    /** A run out (or obstructing the field) is a wicket for the fielding side, NOT for the bowler. */
    boolean creditedToBowler() { return this != RUN_OUT && this != OBSTRUCTING_THE_FIELD; }
}
// ... record BallOutcome: isLegal() { return extra != Extra.WIDE && extra != Extra.NO_BALL; }
    /** Byes and leg byes are not the bowler's fault; wides and no balls are. */
    int runsChargedToBowler() { return batRuns + (isLegal() ? 0 : extraRuns); }
// ...
/** An over is SIX LEGAL deliveries, not six balls. */
final class Over {
    // ...
    int legalBalls() { return (int) balls.stream().filter(b -> b.outcome().isLegal()).count(); }
    boolean isComplete() { return legalBalls() == LEGAL_BALLS; }
    // ...
}

// File: lld/ch18/cricinfo/Innings.java (excerpt)
    Ball record(BallOutcome o) {
        Over over = currentOver();
        if (over == null || over.isComplete()) throw new IllegalStateException("start a new over first");
        if (isComplete()) throw new IllegalStateException("innings is complete");

        int deliveryNo = over.legalBalls() + 1;          // a wide or no ball is re-bowled, so it shares the next
number
        Wicket wicket = null;
        if (o.wicket() != null) wicket = new Wicket(o.wicket(), o.nonStrikerOut() ? nonStriker : striker,
over.bowler());
        Ball ball = new Ball(over.number(), deliveryNo, over.bowler(), striker, o, wicket);
        over.add(ball);
        balls.add(ball);
        runs += o.totalRuns();

        if (o.runsRun() % 2 == 1) swapEnds();              // odd runs: batters crossed
        if (wicket != null) {
            wickets++;
            if (!allOut()) {
                Player newBatter = batting.battingOrder().get(nextBatter++);
                if (wicket.batter().equals(striker)) striker = newBatter; else nonStriker = newBatter;
            }
        }
        if (over.isComplete()) swapEnds();                  // ends change after every over
        return ball;
    }

// File: lld/ch18/cricinfo/Stats.java (excerpt)
abstract class StatQuery {
    record Leader(Player player, int value) {}

    abstract String question();
    abstract int valueOf(PlayerStats s);

    final Optional<Leader> answer(Map<Player, PlayerStats> stats) {
        Leader best = null;
        for (PlayerStats s : stats.values()) {
            int v = valueOf(s);
            if (v > 0 && (best == null || v > best.value()
                    || (v == best.value() && s.player.name().compareTo(best.player().name()) < 0))) {
                best = new Leader(s.player, v);
            }
        }
        return Optional.ofNullable(best);
    }
}

final class MostWickets extends StatQuery {
    @Override String question() { return "Highest wicket taker"; }
    @Override int valueOf(PlayerStats s) { return s.wickets; }
}
```

Full runnable code: code/lld/ch18/cricinfo/ (see the Full Edition).

Output (excerpt) — the first over has eight lines but ends at `0.6`:

```
  0.1  Tarun to Arun, FOUR  [Ravi: driven through cover]
  0.2  Tarun to Arun, wide  [Ravi: sprayed down leg]
  0.2  Tarun to Arun, 1 run(s)  [Ravi: pushed to mid-on]
  0.3  Tarun to Bala, no ball  [Ravi: overstepped, free hit next]
  0.3  Tarun to Bala, OUT BOWLED (Bala)  [Ravi: through the gate!]
  0.4  Tarun to Chetan, OUT RUN_OUT (Chetan) - not the bowler's wicket  [Ravi: a second run was never there]
  0.5  Tarun to Arun, no run  [Ravi: defended]
  0.6  Tarun to Arun, SIX  [Ravi: launched over long-on]
  REJECTED: Tarun cannot bowl two overs in a row
```

Key points: ball numbers are "completed overs . ball"; an extra carries the number of the ball it replaces. Chetan is run out after one completed run: `record` swaps ends for the single first, then replaces the dismissed batter, so Dev arrives at the right end; Tarun's figures show only **1** wicket. Five bowlers rotate, so each bowls exactly 4 overs (`maxOversPerBowler() = 20 / 5`). The chase target is set automatically (`first-innings total + 1`). The T20 ends "Lions won by 23 runs"; the ODI "Lions won by 5 wickets". Global queries are one-line subclasses; "Most runs in ODIs" reuses the service with a format filter.

Machine-coding variant: Cricket Match Dashboard (Udaan)

A **machine-coding round** asks for a *working, runnable* program in 60–90 minutes, judged on correctness, readability and extensibility. This problem was set by Udaan (Indian B2B e-commerce) and appears in the kumaransg/LLD collection.

Problem statement (in our words)

- **Input:** players per team, number of overs, each team's batting order, then each over's balls: runs `0`–`6`, `W`, `Wd` or `Nb`.
- **After every over:** each batter's runs, 4s, 6s, balls (with `*` on the two at the crease), then total/wickets and overs.
- **Rules:** strike changes on 1s and 3s and at over end; wides and no balls add one run and are re-bowled; innings ends all out, overs done, or target passed.
- **At the end:** winner and margin (runs or wickets). **Good to have:** bowler figures, extras, strike rates.

How the full Cricinfo design shrinks to a 90-minute solution

Cricinfo class	Machine-coding version	Why it is enough
`BallOutcome`	`BallEvent` sealed interface (`Runs`, `Wide`, `NoBall`, `Wicket`) with `parse(token)` factory	Only four token kinds
`Innings`	`TeamInnings` — same responsibilities, ints instead of objects	Same rules, no bowler data
Scorecard from balls	`Batter` counters + `ScorecardPrinter`	Printer separate so another view can replace it
`Match` + `Result`	`MatchRunner`	One format, two innings

The lesson: keep the *same responsibilities in the same places* (innings owns strike and the over rule; parsing, scoring and printing separate), but use the simplest data that satisfies the input.

Code

Package `lld.ch18.cricketdashboard` (`Model.java`, `Main.java`; `Main` feeds the sample input as a text block).

```java
// File: lld/ch18/cricketdashboard/Model.java (excerpt)
sealed interface BallEvent {
    record Runs(int runs) implements BallEvent {}
    record Wide() implements BallEvent {}
    record NoBall(int batRuns) implements BallEvent {}
    record Wicket() implements BallEvent {}
    // ... static BallEvent parse(String token): "W", "Wd", "Nb..", or 0-6
    default boolean isLegal() { return !(this instanceof Wide || this instanceof NoBall); }
}
// ...
    void play(BallEvent e) {
        if (isOver()) throw new IllegalStateException("innings already finished");
        Batter s = order.get(striker);
        switch (e) {
            case BallEvent.Wide w -> { total += 1; extras += 1; }
            case BallEvent.NoBall nb -> {
                total += 1 + nb.batRuns(); extras += 1;
                s.faced(nb.batRuns());               // a no ball counts as a ball faced by the batter
                if (nb.batRuns() % 2 == 1) swap();
            }
            case BallEvent.Wicket w -> {
                legalBalls++;
                s.balls++;
                s.out = true;
                wickets++;
                if (!allOut()) striker = next++;     // new batter takes strike
            }
            case BallEvent.Runs r -> {
                legalBalls++;
                total += r.runs();
                s.faced(r.runs());
                if (r.runs() % 2 == 1) swap();       // 1s and 3s change strike
            }
        }
        if (e.isLegal() && legalBalls % 6 == 0 && !isOver()) swap();  // end of over
    }

    boolean isOver() {
        return allOut() || legalBalls == maxOvers * 6 || (target != null && total >= target);
    }
```

Full runnable code: code/lld/ch18/cricketdashboard/ (see the Full Edition).

```
Scorecard for Team 2:
Player   Score  4s  6s Balls     SR
P6          10   1   1     3  333.3
P7*          1   0   0     1  100.0
P8           0   0   0     1    0.0
P9*          1   0   0     1  100.0
P10          0   0   0     0      -
Total: 12/2   Extras: 0
Overs: 1
...
Result: Team 1 won the match by 4 runs
```

Notes on the sample and on correctness

- Our output matches the published sample except where the sample is inconsistent: after team 2's first over it prints `12/1`, but the over has **two** `W`s, so `12/2` is correct; and it later shows P6 with 2 balls after 3. Pointing this out calmly is a plus.
- Team 2's second over stops after four balls: 5 players → all out at 4 wickets ("Overs: 1.4").
- The over loop counts **legal** balls, so a wide makes an over seven tokens long.
- Extras and strike rate are printed; bowler figures would need bowler names in the input (add a `Bowler` class).

Concurrency & edge cases

- **Two scorers:** `bowl`, `startOver`, `startInnings` are `synchronized` per match; different matches never block each other.

- **Many readers:** observers in a `CopyOnWriteArrayList`. At web scale, publish each ball to a broker and push via WebSockets/SSE; the scorecard is a cached view rebuilt from balls.
- **Rejected sequences:** over while one is in progress, same bowler twice in a row, bowler over quota, bowling after the innings, bowler from batting side, twelfth player, non-squad player.
- **Impossible dismissals:** bowled/caught/LBW/stumped/hit wicket off a no ball; bowled/caught/LBW off a wide.
- **Wicket on the last ball:** new batter comes in, then ends change. **Tie** → `Result.Tie`. **Tests:** 0 = no over limit; `declare()` and `declareDraw()`.

Extensions & follow-up questions

- **Who is on strike?** — `Innings` tracks it (odd runs and over end swap).
- **Stats too slow** — a per-player running total updated by an Observer, rebuilt from balls after corrections; balls stay the truth.
- **Duckworth–Lewis–Stern** — a `TargetCalculator` Strategy (default: opponent total + 1).
- **Free hit** — innings remembers "next legal ball is a free hit"; only run-out-type dismissals allowed.
- **Partnerships / fall of wickets, net run rate** — derive from balls and innings totals.
- **Push notifications** — a new `MatchObserver` firing on wickets and 50/100 milestones.

Real-world example

ESPNcricinfo (founded 1993, bought by ESPN in 2007) is the reference for cricket stats; its **Statsguru** answers requirement-7 questions across all internationals. Live pages update ball by ball with commentary like our `CommentaryFeed`, for millions of phones — hence push-based Observers, brokers and caches.

In the interview

Minutes	What to do
0–5	Formats, actors; say early **"an over ends after six *legal* balls"**
5–10	Team ⊇ squad ⊇ eleven; people hold a `Person`
10–20	Match (abstract) → innings → over → ball; formats differ in two numbers
20–30	Ball flow, strike rotation, **no bowler credit for a run out**, byes not the batter's
30–40	Derived stats; Template Method queries; Observers
40–45	One scorer lock per match, broadcast to readers; extensions

> **30-second answer:** "I model cricket's rules, not just its nouns. An over is six legal balls; a wide or no ball is re-bowled. The innings owns the overs, so only it decides when an over ends, who is on strike and when the innings is over. Formats differ only in overs per innings and innings per match. Stats are derived from balls so corrections propagate, each stat question is a Template Method subclass, and live pages subscribe as observers."

- *Why not store `wickets` on `Player`?* — It drifts from the ball record and cannot be filtered by format; derive it.
- *Add "most catches"?* — Fielder on `Wicket`, count it in `PlayerStats`, `MostCatches extends StatQuery`.
- *Is `Match`/formats a Template Method?* — No; only `StatQuery` is a named pattern here — formats are plain abstract-class specialisation.

Self-check

- **Q:** A wide and a no ball in an over — how many deliveries? **A:** Eight.
- **Q:** Who gets a run-out wicket? **A:** The fielding side; not in the bowler's figures.
- **Q:** Are leg byes the batter's runs? **A:** No — team extras, not charged to the bowler either.
- **Q:** What distinguishes `T20`, `ODI`, `Test`? **A:** `oversPerInnings()` (20, 50, 0) and `inningsPerSide()` (1, 1, 2).
- **Q:** Correct team 2 score after over 1 in the Udaan sample? **A:** 12/2.

TL;DR

Cricinfo = people hold a `Person`; team ⊇ squad ⊇ eleven; `Match` (abstract, formats override two numbers) → `Innings` (owns overs, strike, end rules) → `Over` (six **legal** balls) → `Ball`. Run outs are not the bowler's; byes are not the batter's. Stats derived from balls; stat questions are **Template Method** subclasses; live pages are **Observers**. The Udaan dashboard is the same responsibilities in a 90-minute, input-driven form.

18.3 Design LinkedIn

LinkedIn is a networking site for **working lives**: profiles with skills, jobs and schooling, and a home **feed** of updates from your network. It is like Facebook with career features: **connections** (mutual), **follows** (one-sided), **endorsements**, **recommendations**, **company pages** and **job postings**.

Problem & clarifying questions

Question	Answer we assume
Is a connection mutual?	Yes: an **invitation** is accepted or rejected. Following is one-sided
Degrees?	1st, 2nd, 3rd; beyond that "out of network"
Messaging?	Only connections (non-connections need paid *InMail*, out of scope)
Endorsements?	Only 1st-degree connections, once per skill
Recommendations?	*Requested* from a connection; only that person writes it
Profile stats?	Views, connections, search appearances
Companies/jobs?	Members create pages; page admins post jobs; apply once per job
Feed?	Posts from connections and followed members, ranked (course omits it; we add one)

Requirements

Functional (the course's twelve): (1) profile with details, jobs, schooling, skills, accomplishments; (2) search members, companies (and jobs); (3) send/accept/reject connection requests; (4) request recommendations; (5) profile stats; (6) posts to connections; (7) comment on posts, like/share posts **and comments**; (8) messages; (9) notifications for new messages, invitations, comments on your posts; (10) company pages and job postings; (11) create and join groups; (12) follow members or companies.

Non-functional: a connection always appears on **both** members; fast reads (feed, search, degrees); blocked members' content disappears and no spamming strangers; new channels and ranking rules plug in.

Use cases

Actors: **Member**, **Admin** (block/unblock), **System** (notifications). Top cases: profile, search and connect, follow/unfollow, message connections, post/like/comment, notify. Activity diagrams: *add experience* (enter details, optional media, verify, save or ask for missing data) and *send message* (recipients, body, media, verify, deliver, email + push to each recipient).

Core entities

Class	Responsibility
`Member`, `Admin`	Identity, profile, connection set (changed only by the service), follows, posts, inbox, notifications
`Profile`	Owns `Experience`, `Education`, `Skill`, `Accomplishment`, `Recommendation`, `ProfileStats`
`Skill`	Name + set of endorsers
`Recommendation`	Born empty on request; only the named author writes it (REQUESTED → WRITTEN)
`ProfileStats`	Counters bumped by whoever witnesses the event
`SocialContent` → `Post`, `Comment`	Likes as a set; posts own comments and may be shares
`Message` (record)	Not social content — nobody likes a message (text only in our code)
`ConnectionInvitation` + `InvitationStatus`	PENDING → ACCEPTED / REJECTED / CANCELED
`ConnectionService`	Invite, accept, reject, withdraw; BFS degree; mutuals; people-you-may-know
`Company`, `JobPosting`, `JobApplication`, `Group`	Pages, jobs (apply once), groups (owner cannot leave)
`Notification`, `NotificationService`, channels	One class with a *type* field; delivery Strategy
`SearchIndex`	Word → set of members / companies / jobs
`FeedService` + `FeedRanker`	Pull feed with pluggable ranking
`LinkedIn`	Facade

Class diagram

Six course stages:

1. **Profile** — each part is a small class because other code uses it. `Recommendation` is created *without* text (requirement 4 is about asking). The profile never bumps its own `ProfileStats` — whoever sees the event does (an object cannot count events it never sees). All composition.
2. **Posts, comments, messages** — *comments get likes too*, so both extend `SocialContent` holding author, text and **liker ids**. `Message` stays apart.
3. **Invitations and notifications** — an invitation **changes state over time**, so it is its own object that can remember "no". Notification kind is a **field**: **subclass when behaviour differs; field when only data differs.**
4. **Companies, jobs, groups** — a page owns its postings; followers are ids; a group's creator can never leave.
5. **Actors and `ConnectionService`** — accepting edits **two** members, so it belongs to neither; one service keeps connections **symmetric**. Following is one-sided: just a set on the follower.
6. **Search** — the course's real bug: the index stored *one* member per name and broke on the second Sharma. Rule: **an index value is always a collection.** Keys are lowercase words.

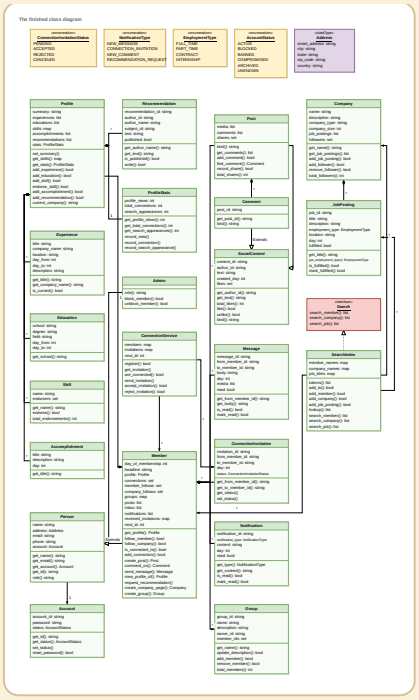
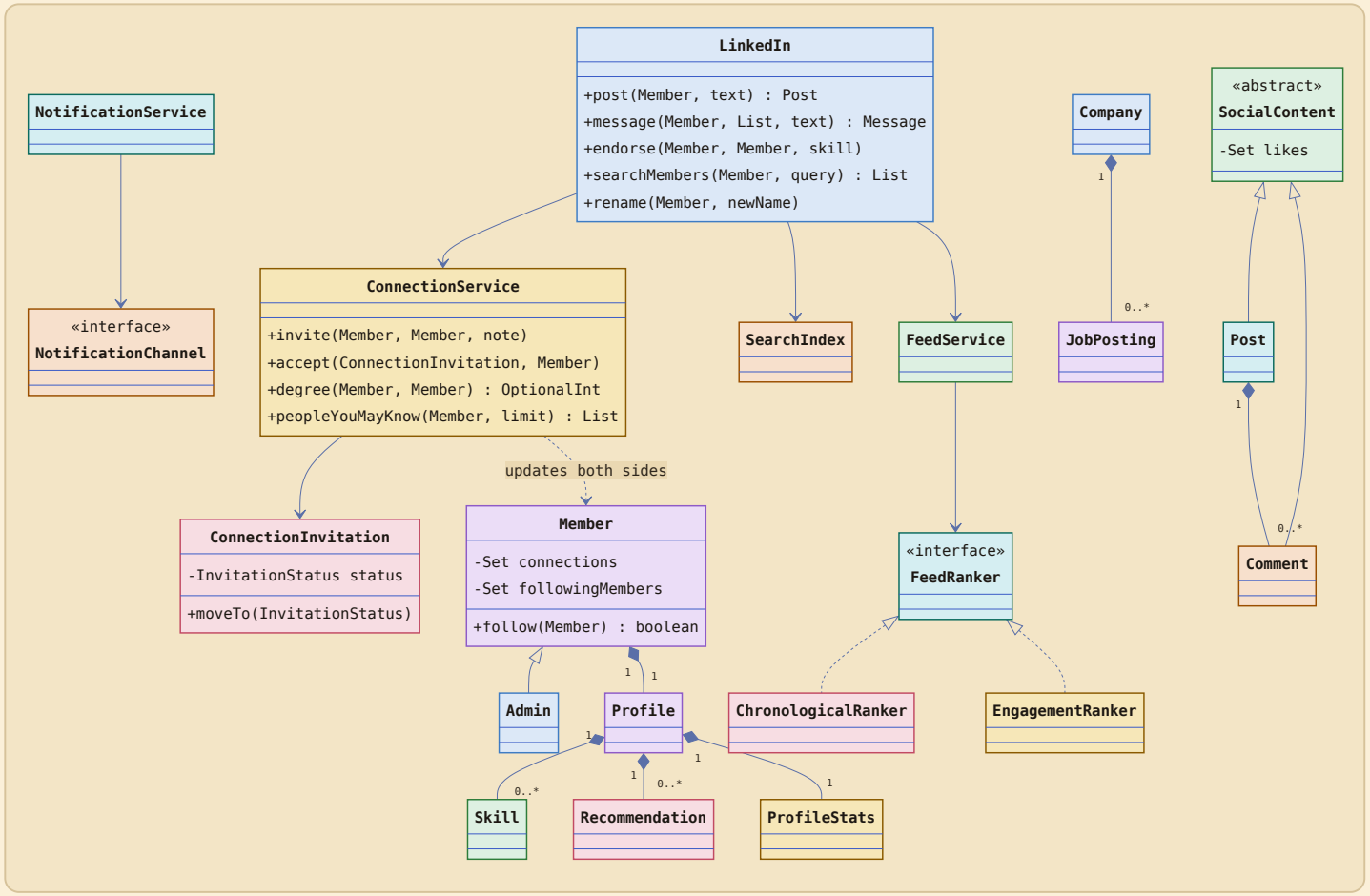


Figure — The finished LinkedIn class diagram (course diagram)

The course omits the feed and ranking, suggestions, degrees, job applications, privacy controls and member-to-member blocking, and admits two weak spots: no feed (pull at load vs copy on write — depends on typical connection count) and a search index that goes stale on rename. **Our Java design** fills these: a pull **feed** with a ranking Strategy, **BFS degrees**, **people you may know**, a **rename that updates the index**, job **applications**, and a connection count derived from the set.



Key flows

Invite and accept: `invite` checks not-self, not-connected, and nothing pending *either way*, creates a PENDING invitation and notifies the receiver. `accept` locks the graph, moves the invitation to ACCEPTED,

and adds each member to the other's set together.

Invitation states: PENDING → ACCEPTED (receiver accepts) / REJECTED (receiver rejects) / CANCELED (sender withdraws); all final.

Degree of connection: BFS (all 1-hop neighbours, then 2-hop…) from the viewer, stopping at depth 3: Asha → Meera 1st, Karan 2nd, Zoya 3rd, Ravi out of network.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`ConnectionService.accept` updates both under one lock	Service for two-object operation	Symmetry has one guardian	`member.accept(inv)` editing the other
Following = set on follower	Simplicity	One-sided, no agreement	Reusing invitations
Invitation with explicit states	**State** (enum + allowed transitions)	Accept-twice / accept-after-reject rejected	Boolean `accepted`
`SocialContent` base	Inheritance	Comments are liked too	Likes only on `Post`
One `Notification` with `type`	Field when only data differs	Delivery identical	Subclass per event
`NotificationChannel`	**Strategy**	In-app, email, push interchangeable	`if (email) … else …`
`FeedRanker`	**Strategy**	"Recent" vs "Top" at run time	Sorting inside the service
Index key → set	Robust inverted index	Both Sharmas found	Key → single member
`rename` tells the index	Sync derived data at the write point	Search never stale	Nightly rebuild
Count = `connections().size()`	Derive, don't store	Cannot disagree	Separate counter

Java implementation

Package `lld.ch18.linkedin` (`Profile`, `Content`, `Network`, `Pages`, `Services`, `LinkedIn`, `Main`). The heart is `Network.java`:

```java
// File: lld/ch18/linkedin/Network.java (excerpt)
enum InvitationStatus {
    PENDING, ACCEPTED, REJECTED, CANCELED;
    boolean canMoveTo(InvitationStatus next) { return this == PENDING && next != PENDING; }
}
// ... ConnectionInvitation:
    synchronized void moveTo(InvitationStatus next) {
        if (!status.canMoveTo(next)) throw new IllegalStateException("invitation is already " + status);
        status = next;
    }
// ...
final class ConnectionService {
    // ...
    synchronized ConnectionInvitation invite(Member from, Member to, String note) {
        if (from.id() == to.id()) throw new IllegalArgumentException("cannot connect to yourself");
        if (from.isConnectedTo(to)) throw new IllegalStateException(from + " and " + to + " are already connected");
        if (pendingBetween(from, to)) throw new IllegalStateException("an invitation between " + from + " and " + to +
" is already pending");
        ConnectionInvitation inv = new ConnectionInvitation(from, to, note);
        invitations.put(inv.id(), inv);
        notifications.send(to, NotificationType.CONNECTION_INVITATION, from + " wants to connect: \"" + note + "\"");
        return inv;
    }

    /** One lock for the whole graph change: both sides are updated together or not at all. */
    synchronized void accept(ConnectionInvitation inv, Member by) {
        if (by.id() != inv.to().id()) throw new IllegalStateException("only " + inv.to() + " can accept");
        inv.moveTo(InvitationStatus.ACCEPTED);
        inv.from().addConnection(inv.to().id());
        inv.to().addConnection(inv.from().id());
    }
    // ...
    /** Degree of connection (1st, 2nd, 3rd) by breadth-first search; empty means "out of network". */
    OptionalInt degree(Member from, Member to) {
        if (from.id() == to.id()) return OptionalInt.of(0);
        Set<Long> seen = new HashSet<>(Set.of(from.id()));
        Deque<long[]> queue = new ArrayDeque<>();                // {memberId, depth}
        queue.add(new long[]{from.id(), 0});
        while (!queue.isEmpty()) {
            long[] cur = queue.poll();
            if (cur[1] == 3) continue;                             // LinkedIn stops at 3rd degree
            for (long next : lookup.apply(cur[0]).connections()) {
                if (next == to.id()) return OptionalInt.of((int) cur[1] + 1);
                if (seen.add(next)) queue.add(new long[]{next, cur[1] + 1});
            }
        }
        return OptionalInt.empty();
    }
    // ... mutualConnections, peopleYouMayKnow
}
```

Full runnable code: code/lld/ch18/linkedin/ (see the Full Edition).

```
  REJECTED: an invitation between Ravi Sharma and Asha Sharma is already pending
degree asha->meera=1, asha->karan=2, asha->zoya=3, asha->ravi=out of network
people asha may know: [Karan Mehta]
search 'sharma' (by meera): [Asha Sharma, Ravi Sharma]
search 'khan' after rename: [], search 'ali': [Zoya Ali]
feed(meera) hides blocked Karan's share: ["Tip: prefer composition over inheritance." by Asha Sharma]
```

Key points: Ravi's counter-invite is refused (pending *either* direction). A second accept fails because only PENDING can move. `peopleYouMayKnow` counts mutuals for 2nd-degree members, skips pending invitations, sorts by count then name. `endorse` needs a 1st-degree connection and rejects repeats via the endorser set; `Recommendation.write` accepts only the named author, once. `rename` removes old words before adding new ones. `FeedService` is a **pull** feed: Karan sees Asha's tip because he *follows* her.

Concurrency & edge cases

- **Symmetric connections:** all graph changes are `synchronized` on the one service. Distributed: write both edges in one transaction, or one row keyed by the ordered id pair.

- **Double invitations** either way refused while pending.
- **Likes, endorsements, applications:** id-keyed sets/maps with `synchronized` mutators.
- `Member` **collections** are concurrent and getters return copies.
- **Stale search:** handled by `rename` — "whatever changes the data tells the index" (events in production).
- **Blocked members:** writes refused, posts vanish from feeds. **Validation:** experience ending before it starts, blank posts rejected.

Extensions & follow-up questions

- **Feed for 900 million members — fan-out on write (push)**: append post ids to each connection's precomputed feed (fast reads, costly for huge followings). **Fan-out on read (pull)**: assemble on open (ours; cheap writes, slower reads). **Hybrid**: push for ordinary members, pull for very popular accounts, merged at read.
- **People you may know at scale** — compute offline (friends-of-friends + shared companies, schools), store, read live.
- **Connection limit** — 30,000; enforce in `accept`.
- **InMail** — a premium credit balance spent by `message`.
- **Who viewed my profile** — store view events, derive the count.
- **Job recommendations** — a Strategy matching skills and title to postings.

Real-world example

LinkedIn shows 1st/2nd/3rd-degree badges in search (our BFS depth) and "People You May Know" from shared connections. Messaging non-connections usually needs paid **InMail**. Its event stream gave rise to **Apache Kafka**, open-sourced in 2011.

In the interview

Minutes	What to do
0–5	Connect vs follow, degrees, who can message and endorse
5–15	Profile composition, `SocialContent`, invitation with state
15–25	**Say it:** accept changes two objects → `ConnectionService`; follow needs no service
25–35	Search (key → list), BFS degrees, people you may know
35–45	Feed pull/push/hybrid, notification Strategy, concurrency

30-second answer: "Members own profiles made of small parts. Posts and comments share a base class because both are liked, with likes as a set. A connection request is an object with states. Accepting changes both members, so it lives in a `ConnectionService` that updates both sides under one lock; following is just a set. Search is an inverted index mapping each word to a list. Degrees come from a BFS to depth 3. The feed is pull with a pluggable ranking Strategy, moving to a push/pull hybrid at scale."

- *Why not store `connectionCount`?* — It is `connections.size()`; a copy drifts.
- *2nd-degree badge for every search result quickly?* — Cache each member's 1st-degree set; a result is 2nd-degree if its set intersects yours.

Self-check

- **Q:** Why is accepting not a `Member` method? **A:** It updates two members; a service keeps both consistent.
- **Q:** Connect vs follow? **A:** Mutual with acceptance vs one-sided and immediate.
- **Q:** Why does `Recommendation` start without text? **A:** It is requested first; the author fills it later.
- **Q:** Which feed model is `FeedService`, and its cost? **A:** Pull; cheap writes but every read scans connections' posts.

TL;DR

LinkedIn = profile composition + `SocialContent` (likes as sets) + invitation **State** + `ConnectionService` for the two-sided accept (following is a set) + BFS degrees + people-you-may-know + inverted-index search (key → list) + notification and feed-ranking **Strategies** + a facade.

18.4 Design Facebook

Facebook: **friends** (mutual), **follows** of people and **pages**, **groups**, posts. New challenges versus LinkedIn: **privacy** (who may see each post), a real **news feed**, **pages** with ratings, public/private **groups**, and **friend suggestions**.

Problem & clarifying questions

Question	Answer we assume
Friend vs follow?	Friendship mutual (request + accept); following one-sided
Who sees a post?	Author picks an **audience**: Public, Friends, custom **privacy list**, Only me
Privacy list?	A named subset of the author's friends ("Close friends")
Pages?	Created by members; followed; **recommendations** (1–5 rating + text)
Groups?	Public (join at once) or private (owner approves; posts visible only to members)
Search?	Members, groups, pages by name; posts by word — respecting privacy
Feed?	Friends, followed members/pages, joined groups — privacy-filtered, ranked
Extended?	Suggest people you may know

Requirements

Functional (twelve + extended): (1) profile — details, jobs, schooling, places; (2) search members, groups, pages; (3) friend requests; (4) follow without friendship; (5) create groups/pages, join groups, follow pages; (6) posts to friends; (7) comments; posts **and comments** liked or shared; (8) privacy lists; (9) messages; (10) page recommendations; (11) notifications for messages, friend requests, comments; (12) search posts by word; (13) **extended:** suggest connections.

Non-functional: privacy never leaked (feed, search, comments, likes, shares); fast feeds for hundreds of friends; symmetric friend graph; stable (non-reshuffling) suggestions and feeds.

Use cases

Actors: **Member**, **Admin** (block/unblock), **System** (notifications). Top cases: profile, search, friend requests, follow/unfollow, message friends, post/like/comment on visible posts, notifications, create page, recommend page. Activity *create post*: text, optional media, **choose the privacy list**, verify, publish to that audience, notify.

Core entities

Class	Responsibility
`Member`, `Admin`	Identity, profile, follows, groups, posts, privacy lists, inbox, notifications
`Profile` (+ `Work`, `Education`, `Place`)	Defaults for every field
`PrivacyList`	Owner + named set of friend ids
`Audience` (sealed) → `Public`, `Friends`, `Custom(list)`, `OnlyMe`	Who may see a post
`SocialContent` → `Post`, `Comment`	Likes as set; posts own comments; may belong to a group/page or be a share
`FriendRequest` + `FriendService`	Friend graph, `areFriends`, `pendingBetween` (both ways)
`VisibilityService`	The single `canSee(viewer, post)`
`Page` (+ `PageType`, `PageRecommendation`)	Followers, one recommendation per member, average rating
`Group` (+ `GroupPrivacy`)	Members, join requests, owner approval, posts
`NotificationService` + `PushChannel` / `EmailChannel`	Routes each type to channels
`SearchIndex`	Names; post words — `searchPosts` takes the viewer
`NewsFeedService` + `FeedRanker`; `SuggestionService` + `Suggestion`	Feed; friends-of-friends
`Facebook`	Facade

Class diagram

Five course stages plus the extended requirement:

1. **Profile** — `Work`, `Education`, `Place` in lists; every field has a **default** so a new account already works.
2. **Posts, privacy, messages, notifications** — fixes three old bugs: **store likers, not a like count** (an old constructor took the count); **comments get likes too**; **the post checks its own privacy list** — but it cannot know friendships (stage 5 supplies that). `Post` → `PrivacyList` is `0..1`; one list serves many posts. `Notification` abstract with a subclass per delivery method.
3. **Pages and groups** — *no member-count field* (an old version stored both count and members). Rule: **if a number can be computed by counting a collection, keep the collection and count it.**
4. **Actors and friend graph** — an old invitation forgot the *sender*. Acceptance changes two members → `FriendService`, which owns the graph and answers "friends?" and "pending?". Following is a set, no service.
5. **Search** — **`searchPost` takes the viewer**: privacy applies in search. Visibility needs the post's list *and* the friend graph; neither answers for the other. Keys map to collections.
6. **Extended — suggestions** — friends of friends **scored by mutual friends**; never yourself, a friend, or anyone with a request pending **in either direction**; sort by mutual count then id so equal scores don't reshuffle. Arrives as two new classes, no edits (OCP).

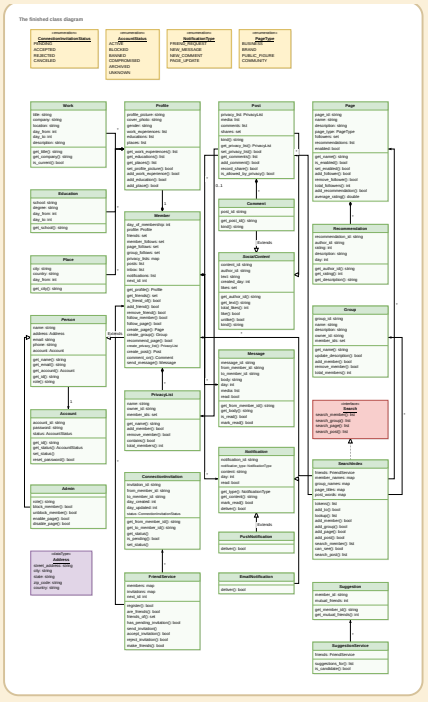
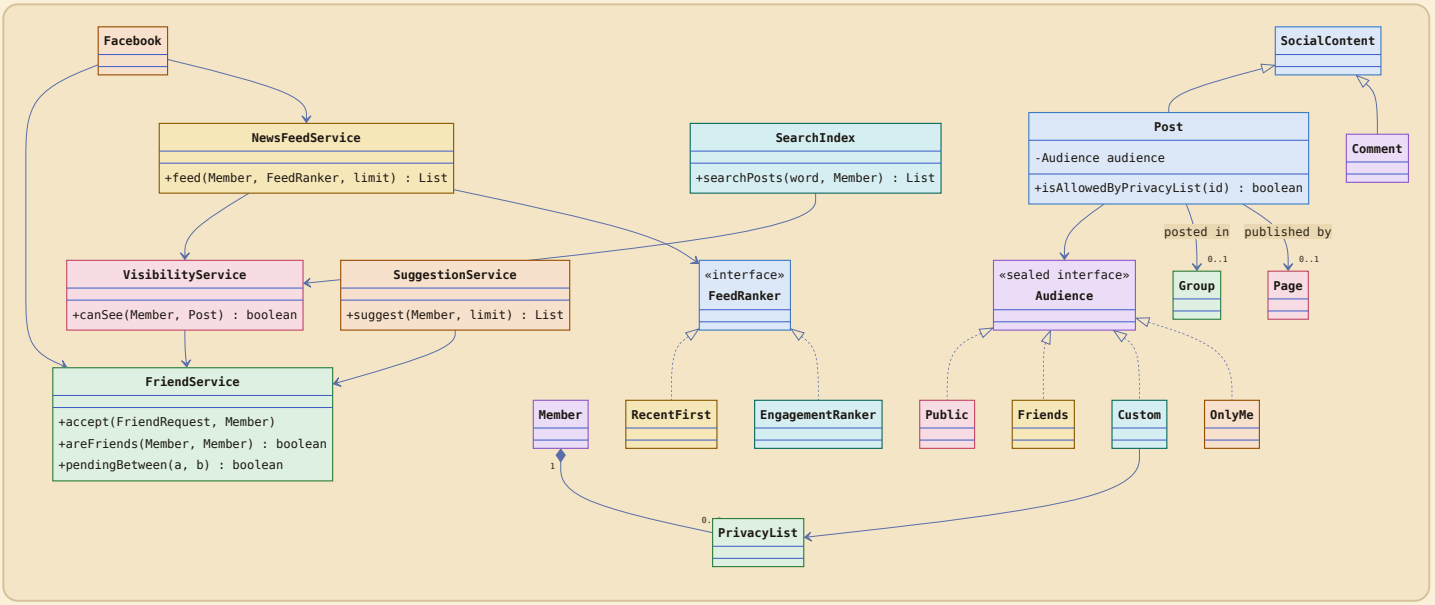


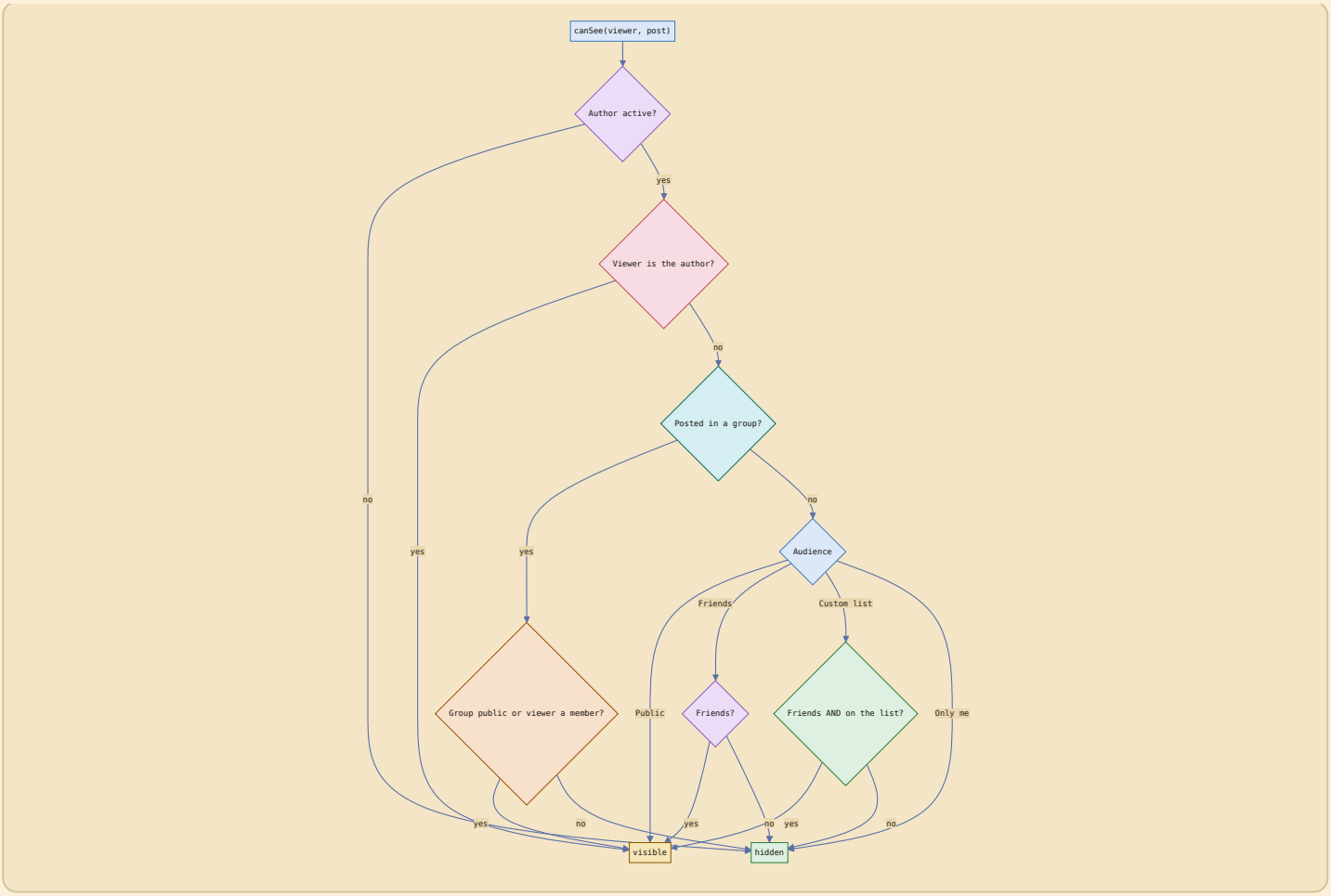
Figure — The finished Facebook class diagram (course diagram)

Our Java design adds a sealed `Audience`, one `VisibilityService` used by feed, search, comments, likes and shares, a `NewsFeedService` with ranking Strategies, private groups with approval, and page posts in the feed. The course deliberately omits the feed, member blocking, non-like reactions, tagging, events, marketplace, stories and albums with separate privacy; we add the feed and sketch the rest under Extensions.



Key flows

Can this viewer see this post? (feed, search, comment, like, share):



News feed (pull): friends + followed members + followed pages + joined groups → collect posts, drop own and duplicates → filter with `canSee` → sort with the ranker (recency, or engagement × affinity / age) → top N.

Friend suggestions: for each friend F, for each friend C of F: skip if C is me, already a friend, or pending either way; else `mutual[C] += 1`; sort by mutual desc, then id; take top N.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
`VisibilityService.canSee` everywhere	Single policy point (Specification-like)	Search and feed cannot leak	Checks per screen
Sealed `Audience` records	Algebraic data type, exhaustive `switch`	New audience forces every decision to handle it	Int/string codes
`Post.isAllowedByPrivacyList` + `FriendService.areFriends`	Information Expert	Each answers what it knows	Post holding the whole graph
`FriendService` owns the graph	Service for two-sided ops	Symmetry; "pending either way" in one place	`member.acceptFriend()`
Likes as a set; no stored counts	Store who; derive, don't store	No fake/double likes; counts can't drift	`likeCount`, constructor counts
`NotificationService` routes types → channels	**Strategy** + config map	Friend requests push + email; comments push only	`if/else` on type
`FeedRanker`	**Strategy**	Recent vs ranked	Ranking inside the feed
`SuggestionService` added without edits	OCP	Extended requirement = two new classes	Fields on `Member`
`Facebook`	**Facade**	One entry, status checks	UI calling every service

Java implementation

Package `lld.ch18.facebook` (`Model`, `PagesAndGroups`, `Services`, `Facebook`, `Main`).

```
// File: lld/ch18/facebook/Model.java (excerpt)
sealed interface Audience {
    record Public() implements Audience {}
    record Friends() implements Audience {}
    record Custom(PrivacyList list) implements Audience {}
    record OnlyMe() implements Audience {}
}
// ... in Post:
    /** The half of the privacy rule the post CAN answer on its own. */
    boolean isAllowedByPrivacyList(long viewerId) {
        return !(audience instanceof Audience.Custom c) || c.list().contains(viewerId);
    }

// File: lld/ch18/facebook/Services.java (excerpt)
final class VisibilityService {
    private final FriendService friends;
    VisibilityService(FriendService friends) { this.friends = friends; }

    boolean canSee(Member viewer, Post post) {
        Member author = post.author();
        if (author.status() != AccountStatus.ACTIVE) return false;
        if (viewer.id() == author.id()) return true;
        if (post.group().isPresent()) {
            Group g = post.group().get();
            return g.isPublic() || g.isMember(viewer.id());
        }
        return switch (post.audience()) {
            case Audience.Public p -> true;
            case Audience.Friends f -> friends.areFriends(viewer, author);
            case Audience.Custom c -> friends.areFriends(viewer, author) && post.isAllowedByPrivacyList(viewer.id());
            case Audience.OnlyMe o -> false;
        };
    }
}
// ...
    List<Suggestion> suggest(Member me, int limit) {
        Set<Long> mine = friends.friendsOf(me.id());
        Map<Long, Integer> mutual = new HashMap<>();
        for (long f : mine) {
            for (long candidate : friends.friendsOf(f)) {
                if (candidate == me.id() || mine.contains(candidate) || friends.pendingBetween(me.id(), candidate))
continue;
                mutual.merge(candidate, 1, Integer::sum);
            }
        }
        return mutual.entrySet().stream()
                .map(e -> new Suggestion(members.apply(e.getKey()), e.getValue()))
                .sorted(Comparator.comparingInt(Suggestion::mutualFriends).reversed()
                        .thenComparingLong(s -> s.member().id()))           // stable tie-break
                .limit(limit)
                .toList();
    }
```

Full runnable code: code/lld/ch18/facebook/ (see the Full Edition).

```
  Alice Rao  public=true  friends=true  closeFriends=true
  Bob Das    public=true  friends=true  closeFriends=true
  Erin Shah  public=true  friends=true  closeFriends=false
  Frank Lee  public=true  friends=false closeFriends=false
search posts 'party' as Bob:  ["Planning a surprise party for Erin" by Alice Rao]
search posts 'party' as Erin: []
before: [Dave Paul(2 mutual), Grace Lee(1 mutual)]
after inviting Grace: [Dave Paul(2 mutual)]
```

Key points: the visibility table is the course's key scenario — Bob is a friend *and* on "Close friends", Erin only a friend, Frank neither. Search respects privacy because `searchPosts` takes the viewer. `createPrivacyList` refuses non-friends. Comments, likes and shares also call `canSee` (otherwise "comment" is a side door). Pages allow one recommendation per member (`putIfAbsent`); count and average are computed. A private group's join waits for approval and its posts are hidden from non-members, even in search. Once

Frank is blocked his posts vanish from feeds. Notification routing is a `Map<NotificationType, List<NotificationChannel>>`.

Concurrency & edge cases

- **Friend graph:** `sendRequest`, `accept`, `reject`, `unfriend` `synchronized`; concurrent adjacency sets. In a DB, one row keyed by `(min(id), max(id))` makes symmetry automatic.
- **Pending both ways:** `pendingBetween(a, b)` for requests and suggestions.
- **Privacy changes after posting:** visibility computed at read time, so unfriending hides posts at once (a push feed must still re-check `canSee` on read).
- **Shares:** need to see the original; production should cap the share's audience at the original's.
- **Deterministic ordering:** suggestions tie-break by id, feeds by creation time; a logical `Tick` clock makes output repeatable.
- **Blocked/deleted authors:** `canSee` returns false.

Extensions & follow-up questions

- **Scale the feed** — fan-out on write vs on read; mix (push for normal users, pull for celebrities), always re-check privacy.
- **Better ranking** — affinity, content type, recency decay behind `FeedRanker`: early **EdgeRank** (affinity × weight × time decay).
- **Suggestions at scale** — precompute offline (the original diagram's `memberSuggestions` field), add shared groups/schools/work.
- **Member blocking** — per-member block set checked both ways by `canSee` and `sendRequest`.
- **Reactions** — `Map<memberId, Reaction>`, like Stack Overflow's vote map.
- **Tagging** — tagged ids on `Post`, notify, optionally widen audience.
- **Events, stories, marketplace, albums** — new `SocialContent` types with their own `Audience`.

Real-world example

Facebook launched **News Feed** in 2006. Posts carry an audience selector, a profile is capped at **5,000 friends** (then followers or pages), and the graph is served by **TAO**, a distributed store for objects and associations.

In the interview

Minutes	What to do
0–5	Friend vs follow, audiences, pages, groups, feed
5–15	Profile, `SocialContent`, `PrivacyList`, `Audience`
15–25	`FriendService` owns the graph; follow is a set
25–35	**Strongest point:** `canSee` consults post list + friend graph; every read path calls it
35–45	Feed models, suggestions, concurrency

> **30-second answer:** "Posts and comments share a likes set. A post has an audience — public, friends, a custom list, or only me. The post checks its own list, but only the friend service knows friendships, so one `VisibilityService.canSee` asks both, and every read path — feed, search, comment, like, share — calls it. The friend service owns the graph because accepting changes two members; following is one-sided. The feed gathers from friends, follows, pages and groups, filters by `canSee`, and ranks with a Strategy. Suggestions are friends of friends by mutual count, excluding pending requests either way, with a stable tie-break."

- *How could a private post leak?* — Any read path that forgets privacy: search, feed, comments, shares, notifications.
- *Why the second sort key?* — Equal mutual counts would swap between loads; a self-reshuffling list looks broken.

Self-check

- **Q:** Erin is a friend but not on "Close friends" — can she see a close-friends post? **A:** No; friend *and* on the list.
- **Q:** Why does `searchPosts` take a viewer? **A:** Privacy applies inside search.
- **Q:** Who is excluded from suggestions? **A:** Yourself, existing friends, anyone pending either way.
- **Q:** Push vs pull feed weaknesses? **A:** Push is expensive for huge audiences; pull reads are slower.

TL;DR

Facebook = LinkedIn's structure + **privacy**: sealed `Audience`, privacy lists of friends, one `VisibilityService.canSee` (post's list AND friend graph) called by feed, search, comments, likes and shares. `FriendService` owns the symmetric graph; following is a set. Pages and groups store collections, not counts. The feed gathers, filters, ranks with a Strategy; suggestions are friends-of-friends by mutual count, excluding pending both ways, stable tie-break.

18.5 Chapter summary

The four designs side by side

Aspect	Stack Overflow	Cricinfo	LinkedIn	Facebook
Core content	Question / answer / comment	Ball / over / innings / match	Post / comment / message	Post / comment / message / page & group posts
User relationship	None (votes, reputation)	Team membership, officials	Connection + follow	Friendship + follow
Two-sided op → service	—	—	`ConnectionService.accept`	`FriendService.accept`
Roles	Reputation privileges + Moderator/Admin	Admin scorer, Commentator	Member / Admin	Member / Admin + audiences
Derive, don't store	Score, tag counts	All stats from balls	Connection, follower counts	Member/follower counts, ratings
Strategy	Reputation policy, channels	(Target calculator, extension)	Feed ranker, channels	Feed ranker, channels
Observer	Reputation listeners	Match observers	Notifications	Notifications
Template Method	—	`StatQuery`	—	—
Search	Words and tags	Stat queries	Members, companies, jobs	Members, groups, pages, privacy-aware posts

Common mistakes across all four

- Counters instead of sets — cannot stop duplicates.
- Stored counts beside the collections they summarise — they drift.
- Two-sided operations on one entity — asymmetric graphs.
- Privacy or permission checks in one read path but not another.
- Index values that are a single object instead of a list.
- Modelling nouns but not rules (six *legal* balls; run outs; byes).

Chapter TL;DR

Platforms are rich domain models. Share behaviour in a content base class; keep votes and likes as sets of ids; derive every number you can; put operations that change two objects in a service; notify through Observer with Strategy channels; rank feeds with a Strategy; keep search in an inverted index mapping words to lists; and write each permission or privacy rule once, then call it from every read path.

Chapter 19 — Case Studies: Commerce & Marketplaces (Part 2 of Platforms) — Study Edition

> **Where this fits.** Chapter 18 designed platforms where people *talk*. This chapter designs platforms where people *buy, sell and trade*. Three pressures shape every design: **money must be exact, state must follow rules, and shared things must not be double-sold.** This is the study edition: every topic of the full chapter, without the padding.

#	Problem	The one idea that makes it work	Main patterns
19.1	Amazon — online shopping	An order is an immutable copy of the cart, moved through a **state machine**	State machine (enum), Strategy (payment), Observer, Facade
19.2	Stock brokerage + exchange matching engine	Four order types differ in only **two questions**; a book matches by **price, then time**	Template Method, Observer, Strategy (price policy)
19.3	Online auction	"Read highest bid, compare, replace" is **one atomic step**	Monitor lock, Observer, Specification
19.4	Music streaming (Spotify)	Play order is a **pluggable strategy**; plan rules live **in the plan**	Strategy, Observer, enum-with-behaviour
19.5	Food delivery (Swiggy) + FoodKart	Capacity and riders are **claimed atomically**; selection is a **strategy**	Strategy, Observer, State machine, CAS
19.6	Grocery store (POS & checkout)	A **pure pricing engine**: best offer per product, then a coupon spread over items so tax and refunds stay exact	Strategy (offers, tax, payment), Facade, Observer, enum state machine

Running the code: `cd chapters/19-case-platforms-2/code && javac --release 21 -d /tmp/lld_19 $(find . -name '*.java') && java -cp /tmp/lld_19 lld.ch19.amazon.Main` (or `brokerage`, `stockexchange`, `auction`, `music`, `fooddelivery`, `foodkart`, `grocery`). Java 21, standard library only; the stock-exchange `Main` also runs a small test suite. One file may hold several small package-private types.

Four rules that apply to every design in this chapter

1. **Money is `BigDecimal`, never `double`.** A `double` cannot store 0.1 exactly (`0.1 + 0.2` = `0.30000000000000004`). Build it from a `String` (`new BigDecimal("0.10")`) and compare with `compareTo`, not `equals` — `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` is `false` because the scales differ.
2. **Status changes follow a state machine** — a fixed list of states plus allowed moves, kept *inside the enum*, so an illegal move ("cancel a delivered order") throws at once.
3. **Check-then-act on shared stock must be atomic.** "One left? Take it" is two steps; two threads can both see "yes". Do both under one lock, or with one compare-and-set (CAS: "set to B only if it is still A").
4. **Copy prices into orders.** Later catalog or menu price changes must not change old orders.

Design Amazon — Online Shopping System

Problem & clarifying questions

We design Amazon's **online retail part**: people list products, others find them, fill a cart, pay and track the parcel.

Question	Assumed answer
Buy without an account?	No. Browsing and carts are open; ordering needs a **registered member**
Who sells?	Members can list products (a marketplace)
Payment?	Credit card (clears at once) or bank transfer (clears later)
Cancel?	Yes, until shipped
Notifications?	On every order or shipment status change
Returns, wish lists, payouts, recommendations, multi-warehouse stock?	Out of scope — say so out loud

Requirements

Functional: (1) members add products to sell; (2) anyone searches by **name** or **category**; (3) anyone views, only members buy; (4) add/remove/change cart items; (5) check out the whole cart; (6) rate and review; (7) shipping address; (8) cancel an unshipped order; (9) notified on each order/shipment status change; (10) pay by card or bank transfer; (11) track shipment and order state.

Non-functional: two buyers never both get the last unit; exact `BigDecimal` money; orders and shipments keep a **history** of states; a new payment method or channel must not change checkout.

Use cases

Actors: **Admin** (block/unblock accounts, manage categories), **Guest** (search, cart, register), **Member** (guest + order + sell), **System** (notifications). In the use-case diagram, "Checkout" *includes* "Make payment" and "Add shipping address"; search by name/category *extend* "Search product".

Core entities

Class	Responsibility
`ProductCategory`	Named group; exists without products
`Product`	Price, seller, stock; **owns** reviews; guards stock (`reserve` / `release`)
`ProductReview`	Rating 1–5 + text; stores reviewer **by id** (no `Product ↔ Member` dependency)
`Search` / `Catalog`	Name-word and category indexes, both updated in `addProduct`; points to, never owns, products
`ShoppingCart` / `Item`	Mutable cart owning its lines; `Item` is an immutable record (product id, name, qty, unit price)
`Order` / `OrderLog`	Immutable **copy** of cart lines + address + status on legal edges; one log entry per change
`Payment` (`CreditCardPayment`, `BankTransferPayment`)	Card clears now; transfer returns `PENDING`
`Shipment` / `ShipmentLog`	Parcel + tracking history; last entry = "where is it now"
Notification channel (`EmailChannel`, `SmsChannel`)	One delivery way (course: abstract `Notification` with subclasses; same idea)
`Account`, `Customer` → `Guest`, `Member`, `Admin`	`Customer` has only what *anyone* may do; ordering lives on `Member`
`OrderService`	Turns a cart into an order; drives the state machine

Class diagram

The course builds it in five stages (products → catalog → payment/shipment/notification → cart/order → accounts). Filled diamonds are composition (`Product` owns reviews, cart owns items, order owns log and

shipment); plain arrows are associations (order only *refers to* its payment); `Guest` and `Member` extend `Customer`.

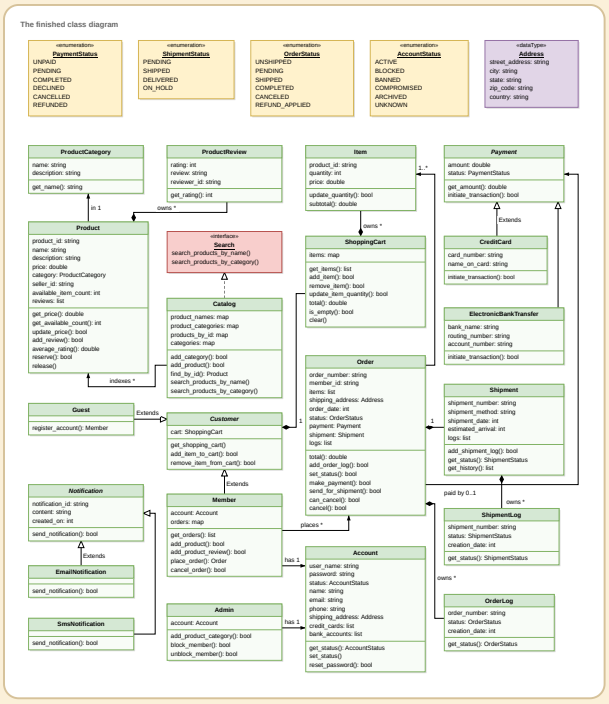
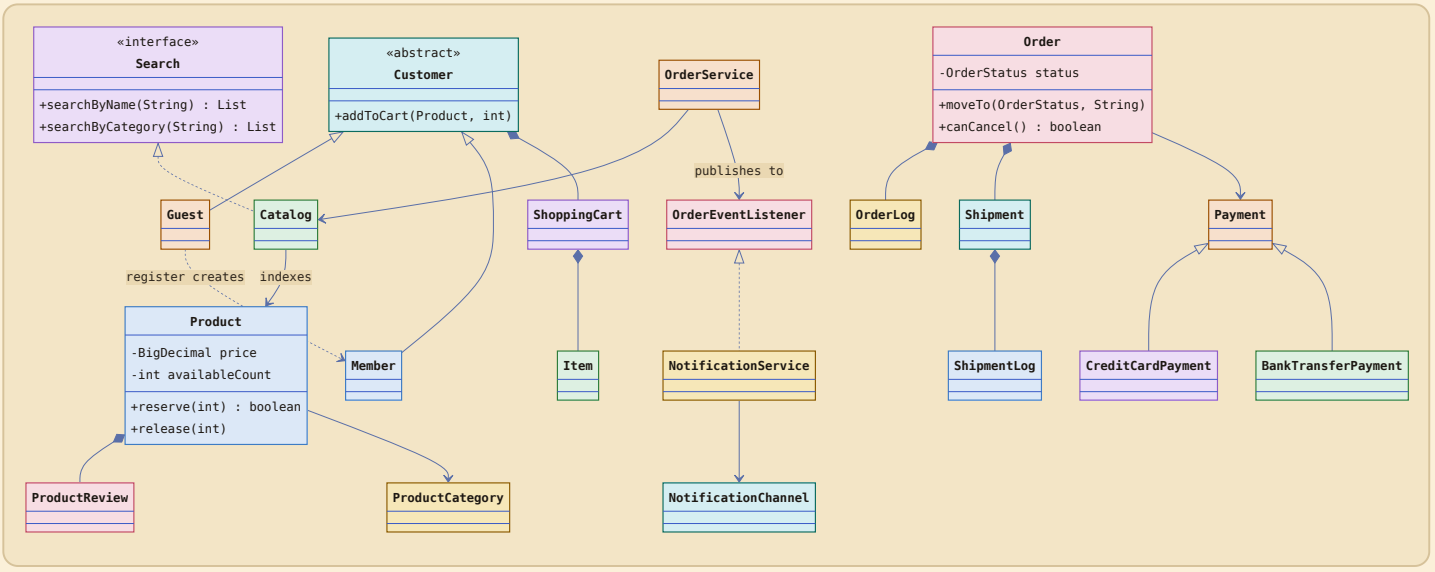


Figure — The finished class diagram of the online shopping system (course diagram)



Key flows

The course's activity diagram is the shopping loop (search, view, add, update cart, "more shopping?", checkout); its sequence diagrams cover search, add-to-cart, and **checkout**: get cart items → billing/shipping info → create order → verify items → pay → create shipment → return status (with error paths).

Checkout (`OrderService.placeOrder`) follows that order, but stock is **reserved** during verification, not just checked: member active and cart not empty → reserve each line (any failure releases what was reserved and fails) → new `Order` copying the lines → `payment.process()`: COMPLETED → `UNSHIPPED` + notify; PENDING (transfer) → notify PENDING; DECLINED → release stock, `CANCELED`.

Order state machine (`OrderStatus.next()`): PENDING → UNSHIPPED (paid) or CANCELED; UNSHIPPED → SHIPPED or CANCELED; SHIPPED → COMPLETED; CANCELED → REFUND_APPLIED. Requirement 8 is one line — `canCancel()` returns `status.canMoveTo(CANCELED)`; there is no `SHIPPED → CANCELED` edge.

Design decisions & patterns used

Decision	Why	Alternative rejected
Ordering on `Member`, not `Customer`	A guest *is a* customer; inheriting `placeOrder` it must reject breaks **LSP**	`Customer.placeOrder()` that throws for guests
`Guest.register()` returns a new `Member` carrying the cart	The cart survives registration	Mutating a guest (Java objects can't change class)
`Order` copies cart lines	A finished purchase must not change	Shared `Item` objects
Stock reserved at order time	No two buyers get the last unit (abandoned orders hold stock → timer release)	Check, pay, then decrement (race window)
State machine inside `OrderStatus`	One owner of the rules	Scattered `if`s
Sealed abstract `Payment`, a subclass per method (**Strategy**)	Methods *behave* differently; new method = new subclass (OCP); that is why it is not an enum	`enum PaymentType` + `switch`
Payment passed as `Function<BigDecimal, Payment>`	Total known only after building the order	Creating payment before the total
Observer for notifications	New channel = new `NotificationChannel`	Email/SMS code inside `OrderService`
`Shipment` keeps a log; reviews store `reviewerId`	History for tracking/disputes; no two-way dependency	Single status field; review holding `Member`
`OrderService` as **Facade**	Hides stock, payment, shipment, notification	Callers orchestrating five objects

Java implementation

Package `lld.ch19.amazon`: `Money.java`, `Catalog.java`, `Order.java`, `Payments.java`, `Accounts.java`, `OrderService.java`, `Main.java`.

```java
// File: lld/ch19/amazon/Order.java (excerpt)
enum OrderStatus {
    PENDING, UNSHIPPED, SHIPPED, COMPLETED, CANCELED, REFUND_APPLIED;

    private Set<OrderStatus> next() {
        return switch (this) {
            case PENDING -> EnumSet.of(UNSHIPPED, CANCELED);      // waiting for payment
            case UNSHIPPED -> EnumSet.of(SHIPPED, CANCELED);      // paid, not yet handed to carrier
            case SHIPPED -> EnumSet.of(COMPLETED);                // too late to cancel
            case CANCELED -> EnumSet.of(REFUND_APPLIED);          // money goes back if it was taken
            case COMPLETED, REFUND_APPLIED -> EnumSet.noneOf(OrderStatus.class);
        };
    }

    boolean canMoveTo(OrderStatus target) { return next().contains(target); }
}

// File: lld/ch19/amazon/Catalog.java (excerpt, in Product)
    synchronized boolean reserve(int quantity) {
        if (quantity <= 0 || quantity > availableCount) return false;
        availableCount -= quantity;
        return true;
    }

// File: lld/ch19/amazon/OrderService.java (excerpt)
    Order placeOrder(Member member, Address address, Function<BigDecimal, Payment> paymentMethod) {
        if (!member.canBuy()) throw new IllegalStateException("account " + member.account().status());
        List<Item> items = member.cart().snapshot();
        if (items.isEmpty()) throw new IllegalStateException("cart is empty");

        List<Item> reserved = new ArrayList<>();
        for (Item i : items) {
            Product p = catalog.find(i.productId()).orElseThrow();
            if (!p.reserve(i.quantity())) {
                reserved.forEach(r -> catalog.find(r.productId()).orElseThrow().release(r.quantity()));
                throw new IllegalStateException("out of stock: " + i.name());
            }
            reserved.add(i);
        }

        Order order = new Order("ORD-" + orderSeq.incrementAndGet(), member.id(), items, address);
        orders.put(order.orderNumber(), order);

        Payment payment = paymentMethod.apply(order.total());
        order.attachPayment(payment);
        switch (payment.process()) {
            case COMPLETED -> { transition(order, OrderStatus.UNSHIPPED, "paid by " + payment.describe());
member.cart().clear(); }
            case PENDING -> { publish(order, OrderStatus.PENDING); member.cart().clear(); }
            default -> {                                  // declined: give the stock back, keep the cart
                releaseStock(order);
                transition(order, OrderStatus.CANCELED, "payment declined");
            }
        }
        return order;
    }
```

Full runnable code: code/lld/ch19/amazon/ (see the Full Edition).

```
== 6. Illegal transition is rejected by the state machine ==
  ORD-1002: REFUND_APPLIED -> SHIPPED is not allowed
== 9. Ten members race for the last copy ==
  winners=1 losers=9 stock left=0
```

The demo also shows: name search matches every query word (`usb cable` finds "USB-C Cable"); the guest's cart survives registration; each status change goes to email + SMS; the cancel path `PENDING → UNSHIPPED → CANCELED → REFUND_APPLIED` returns stock; a blocked member is rejected; ten threads released together by a `CountDownLatch` produce exactly one winner.

Concurrency & edge cases

Risk	How the code prevents it
Last unit, two buyers	`Product.reserve` is `synchronized` (check + decrement in one step)
Multi-line cart, one line out of stock	`placeOrder` releases everything it reserved (all-or-nothing)
Cancel and ship at the same time	Both lock the `Order` around check-then-move
Payment declined	Stock released, order `CANCELED`, cart kept for retry
Bank transfer never clears	A timeout job cancels old `PENDING` orders
Price changes after adding to cart	`Item` stores unit price; production re-validates at checkout
Blocked member	`Member.canBuy()` checks `ACTIVE`
Deadlock	Each lock held briefly, never while waiting for another in opposite order

Real-world example

Amazon offers *Cancel items* only before the order enters shipping — our missing `SHIPPED → CANCELED` edge. Its 2007 **Dynamo** paper chose availability so "add to cart" always works during failures, merging conflicting cart versions later. At scale, cart, catalog and orders are separate services.

Extensions & follow-up questions

- **New payment (gift card, UPI)** — `GiftCardPayment extends Payment` (add to `permits`); `OrderService` untouched.
- **Last unit, distributed** — `UPDATE ... SET stock = stock - 1 WHERE id = ? AND stock >= 1`, or optimistic locking with a version column.
- **Abandoned orders** — reservation expiry; a job cancels `PENDING` orders after ~30 minutes.
- **Returns** — `COMPLETED → RETURN_REQUESTED → RETURNED → REFUND_APPLIED` (local enum change).
- **Multiple warehouses** — `Inventory` per warehouse; a Strategy picks one per line. **Split shipments** — one order, many `Shipment`s.
- **Search at scale** — Elasticsearch/OpenSearch fed by product events behind `Search`. **Payouts, recommendations, wish lists** — modules listening to order events.

In the interview

Plan: clarify (3 min) → actors/use cases (5) → class diagram, **saying the LSP sentence**: "Ordering is on `Member`, not `Customer`, otherwise every guest inherits a method it is not allowed to use" (10) → state machine, no cancel-after-ship edge (5) → code `placeOrder` and `cancel` (10) → concurrency and extensions (5).

> **30-second answer:** "Catalog with name and category indexes; a cart per customer; ordering only on `Member`; checkout reserves stock for every line atomically, copies the lines into an immutable order, then takes payment through a `Payment` strategy. The order moves through a state machine that forbids cancelling after shipping, and every status change is published to a notification observer."

- *Tax and discounts?* — A `PricingRule` chain (Chain of Responsibility or list of Strategies) applied when building the order; the order stores final line prices.
- *Double-click checkout?* — Client idempotency key; return the existing order for a repeated key.

Self-check

1. **Q:** Why can't `placeOrder` live on `Customer`? **A:** Guests extend `Customer` and must not order; inheriting a method you must refuse violates LSP.
2. **Q:** Two faults of "check stock, pay, then decrement"? **A:** A race lets two buyers pass the check; payment can succeed for goods no longer there.
3. **Q:** Which edges allow a cancel? **A:** `PENDING → CANCELED` and `UNSHIPPED → CANCELED`.
4. **Q:** Why `compareTo` on `BigDecimal`? **A:** `equals` also compares scale (`2.0` ≠ `2.00`).

TL;DR

- Guests browse and fill carts; **only members order** (LSP).
- Checkout = **reserve all lines atomically → copy into an immutable order → pay (Strategy) → publish (Observer)**.
- `OrderStatus` is a **state machine**; cancel after ship has no edge. Money in `BigDecimal`; histories in logs; stock guarded by a lock.

Design an Online Stock Brokerage System

Two halves, both asked in interviews:

- **Part A — the brokerage** (the course): members, money in/out, watchlists, four order types, lots and positions, statements.
- **Part B — the exchange's matching engine** (a Geektrust machine-coding problem kept in kumaransg/LLD as "Navi"): an order book matching by **price-time priority**, implemented and tested.

A **broker** holds your cash and shares and forwards orders; an **exchange** is where orders from all brokers meet and match. The code keeps them apart.

Problem & clarifying questions

An online brokerage lets members trade without a human broker, track transactions, see charts, and get alerts when a stock moves by a chosen amount — faster and cheaper, with real-time prices.

Question	Assumed answer
Order types?	Market, limit, stop-loss, stop-limit
Partial fills?	Yes, at several prices
Separate purchases tracked?	Yes — each is a **lot** (tax depends on which lot you sell)
Money in/out?	Check, wire, electronic bank transfer, each with a per-transfer limit
Reports?	Quarterly statements and a yearly tax statement
Notifications?	When an order executes
Margin, shorting, options, dividends/splits, streaming quotes, settlement delay, wash-sale rule?	Out of scope

Requirements

Functional (brokerage): (1) buy and sell; (2) **multiple watchlists**; (3) **market, limit, stop-loss, stop-limit** orders; (4) **multiple lots** per stock kept apart; (5) **quarterly** and **yearly tax** statements; (6) money via **check, wire, electronic transfer**; (7) notify on execution.

Functional (matching engine, Navi): (8) input lines `<id> <time> <symbol> <buy/sell> <price> <qty>` (e.g. `#1 09:45 BAC sell 240.12 100`) from a file given as argument, trades to console; (9) match each new order against the opposite side — lower sells and higher buys first, **earlier first** at the same price; (10) trade when buy ≥ sell, recorded at the **sell order's price**; (11) print `<buy_id> <price> <qty> <sell_id>`; unmatched quantity rests.

Non-functional: money never over-committed (ten $1,000 orders cannot all pass a $1,000 check); deterministic matching; engine safe under concurrency and conserves quantity.

Use cases

Admin (block/unblock), **Member** (search, trade, watchlists), **System** (order notifications, fetch quotes). Top cases: register/cancel membership, manage watchlists, search by symbol, place/cancel orders, deposit/withdraw.

Core entities

Class	Responsibility
`MoneyTransfer` → `CheckTransfer`, `WireTransfer`, `ElectronicBankTransfer`	Differ only in name and clearing days (5/1/3); the 100,000 limit lives once in the base
`Notifier` / `EmailNotifier`	"Order filled" messages (an `SmsNotifier` would be a second implementation)
`Order` (abstract) + `MarketOrder`, `LimitOrder`, `StopLossOrder`, `StopLimitOrder`	Shared fields; each subclass answers the **two questions**; `StopLimitOrder extends StopLossOrder`
`OrderPart`	One fill (e.g. 70 @ 10.00 + 30 @ 10.05)
`Stock`, `StockInventory`	Symbol + latest price; inventory is the **single source of truth** for prices
`StockExchange`, `OrderListener`	Holds open orders, fills on new orders/ticks, calls back `onFill` / `onCancelled` (never knows `Member`)
`StockLot`, `StockPosition`, `LotSale`	One purchase; all lots of a symbol, sold FIFO unless a lot is named; per-lot realised gain
`Watchlist`	Named set of **symbols** (never stale)
`Trade`, `Statement` → `QuarterlyStatement`, `TaxStatement`	Fills as the member sees them; statements share day-range filtering
`Member`, `Admin`	Cash, **reserved funds**, positions, watchlists, trades; implements `OrderListener`
(engine) `OrderBook`, `MatchingEngine`, `Trade`, `ExecutionPricePolicy`	Price levels with FIFO queues; routing and locking; trade price

Class diagram

Course stages: transfers and notifications → **the order hierarchy** (the part that earns most credit) → stock, inventory, exchange → lots and positions → watchlists, trades, statements → members and admins. `Member` extends `Account` and implements `OrderListener`; it owns positions, watchlists and trades.

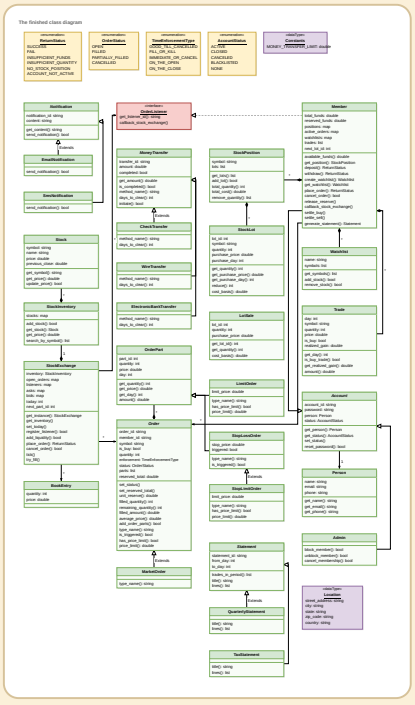
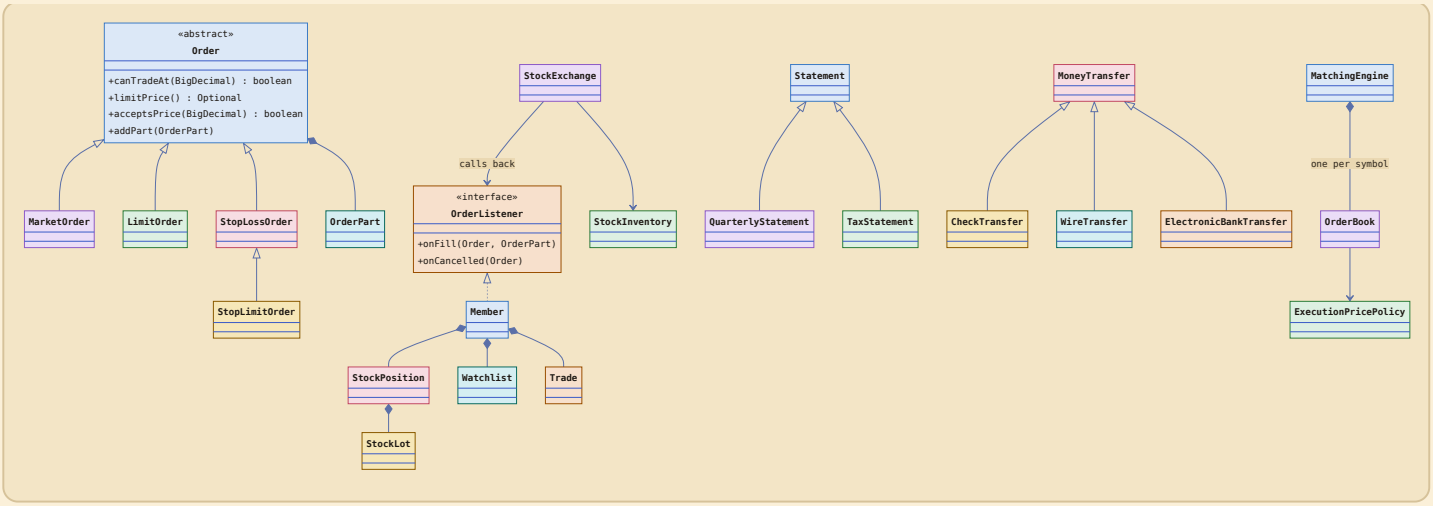


Figure — The finished class diagram of the online stock brokerage (course diagram)



Key flows

Buy order (course activity diagram): pick stock and type, enter quantity, limit and time enforcement → check funds → reserve money → send to exchange → "order placed". **Sell order** mirrors it, choosing **which lots** and checking owned shares instead of money.

The four order types share everything except two answers:

Type	Q1: may it trade at this price?	Q2: worst acceptable price	Plain words
Market	Always	None	"Now, at whatever the market gives"
Limit	Always	The limit	"Buy at 9.50 or less / sell at 10.90 or more"
Stop-loss	Only after price **reaches the stop**, then forever	None (becomes market)	"If it falls to 10.00, get me out"
Stop-limit	Only after the stop	The limit	"If it rises to 305, buy — but pay no more than 306"

A SELL stop triggers when price ≤ stop; a BUY stop when price ≥ stop; once triggered it "latches".

Fill: member reserves (funds or free shares) → exchange checks `canTradeAt`, fills level by level while `acceptsPrice` → `onFill` moves cash, releases the reservation, adds a lot or sells FIFO. Later `tick(symbol, price)` re-checks open orders (stops trigger here).

Matching engine — price-time priority. Per symbol, an `OrderBook` with two sides, each a `TreeMap` price → FIFO `ArrayDeque` (bids highest-first, asks lowest-first). For an incoming order: look at the best opposite price; stop if it does not **cross**; else fill the oldest order at that level for `min(remaining, resting)`; remove filled orders and empty levels; repeat; rest any remainder at the back of its own level.

Order	Book before	What happens
#1 sell 240.12 × 100	empty	rests
#2 sell 237.45 × 90	asks 240.12	rests; best ask 237.45
#3 buy 238.10 × 110	asks 237.45×90, 240.12×100	trade `#3 237.45 90 #2`; 20 left; 240.12 too high; rests 20 @ 238.10
#4 buy 237.80 × 10	best ask 240.12	no cross; rests
#5 buy 237.80 × 40	same	rests **behind #4**
#6 sell 236.00 × 50	bids 238.10×20, 237.80×(10, 40)	fills #3 20, #4 10, #5 20 — all at 236.00 (sell price)

Design decisions & patterns used

Decision	Why	Alternative rejected
One abstract `Order` + two abstract questions	Only "may I trade?" and "worst price?" differ. The course calls it plain polymorphism — not State (no order changes kind); its only named patterns are Observer and Singleton (Template Method/Strategy labels here are ours)	Four copied classes; one class with `type` + `switch`
`StopLimitOrder extends StopLossOrder`	Inherits the trigger, overrides only `limitPrice()`	Duplicated trigger logic
`OrderPart` per fill	Each fill has its own price and becomes a lot	Average price only (loses tax info)
Observer `OrderListener`	Exchange never imports `Member`	Exchange updating balances
Reserved funds / shares	Stops over-commitment; money actually moves only in `onFill`, called by the exchange (course's `callbackStockExchange`)	Balance check only at fill time
Lots, FIFO default	Tax depends on which shares were sold	One average-cost position
Template Method `Statement`	`render` final (shared filter); subclasses fill `body`	Two copies of the filter
`MoneyTransfer` base holds the limit	One rule, three methods	Limit copied per subclass
Singleton?	Course makes `StockExchange` a Singleton; we inject one instance (testable)	Global `getInstance()`
Engine: `TreeMap<price, ArrayDeque>` per side	Best price O(log n); FIFO gives time priority	One `PriorityQueue` (O(n) cancel, no visible levels)
Engine: lock per symbol	Symbols match in parallel; same symbol serialised	Global lock or no lock
Engine: `ExecutionPricePolicy` (**Strategy**)	Navi: sell price; real exchanges: resting order's price	Hard-coding one rule

Java implementation

Part A — Brokerage (`lld.ch19.brokerage`)

Files: `Orders.java`, `Market.java`, `Holdings.java`, `Accounts.java`, `Main.java`. The simulated exchange offers 70 shares per price level, 5 cents apart, 3 levels, so fills come in parts.

```java
// File: lld/ch19/brokerage/Orders.java (excerpt)
abstract class Order {
    // ... id, memberId, symbol, side, quantity, timeInForce, parts, status
    /** Question 1. Market and limit orders always say yes; stop orders wait for the trigger. */
    abstract boolean canTradeAt(BigDecimal marketPrice);

    /** Question 2. Empty means "any price" (market behaviour). */
    abstract Optional<BigDecimal> limitPrice();

    /** Is a fill at this price acceptable? BUY: price <= limit. SELL: price >= limit. */
    final boolean acceptsPrice(BigDecimal price) {
        return limitPrice().map(l -> side == Side.BUY ? price.compareTo(l) <= 0 : price.compareTo(l) >= 0)
                           .orElse(true);
    }
    // ...
}

final class MarketOrder extends Order {
    // ...
    @Override boolean canTradeAt(BigDecimal p) { return true; }
    @Override Optional<BigDecimal> limitPrice() { return Optional.empty(); }
    // ...
}
// ... LimitOrder: canTradeAt -> true, limitPrice -> Optional.of(limit)
class StopLossOrder extends Order {
    // ...
    @Override final synchronized boolean canTradeAt(BigDecimal p) {
        if (!triggered) {
            triggered = side() == Side.SELL ? p.compareTo(stopPrice) <= 0 : p.compareTo(stopPrice) >= 0;
        }
        return triggered;
    }
    @Override Optional<BigDecimal> limitPrice() { return Optional.empty(); }
    // ...
}

/** A stop order that becomes a LIMIT order (not a market order) once triggered. */
final class StopLimitOrder extends StopLossOrder {
    // ...
    @Override Optional<BigDecimal> limitPrice() { return Optional.of(limit); }
    // ...
}

// File: lld/ch19/brokerage/Accounts.java (excerpt, in Member)
    private synchronized ReturnStatus reserveFor(Order o) {
        if (status != AccountStatus.ACTIVE) return ReturnStatus.ACCOUNT_NOT_ACTIVE;
        if (o.side() == Side.BUY) {
            BigDecimal perShare = o.limitPrice().orElse(inventory.priceOf(o.symbol()));
            BigDecimal need = Money.times(perShare, o.quantity());
            if (need.compareTo(availableFunds()) > 0) return ReturnStatus.INSUFFICIENT_FUNDS;
            reservedFunds.put(o.id(), need);           // released part by part as fills arrive
            return ReturnStatus.SUCCESS;
        }
        // ... sells: free shares = position - shares promised to open sells of this symbol
    }
```

```
== 3. Market buy 100 AAPL fills in two parts (two lots) ==
    [email alice@mail.test] filled 70 AAPL @ 10.00 (O1 PARTIALLY_FILLED)
    [email alice@mail.test] filled 30 AAPL @ 10.05 (O1 FILLED)
Tax statement for Alice (days 1-365)
  AAPL L1: 70 sh bought @ 10.00 sold @ 11.00 gain $70.00
  AAPL L2: 30 sh bought @ 10.05 sold @ 10.95 gain $27.00
  AAPL L3: 20 sh bought @ 9.40 sold @ 10.95 gain $31.00
  AAPL L3: 30 sh bought @ 9.40 sold @ 9.95 gain $16.50
  realised gain: $144.50
```

Reading the output: a 200,000 check fails the 100,000 limit. The market order reserved $1,000.00 but cost $1,001.50 — a market order can spend more than reserved (the course's warning). A limit buy waits for the 9.40 tick; a 120-share sell goes FIFO across three lots; the stop does nothing at 10.50 and fires at 9.95; the stop-limit triggers at 310 but its 306 limit blocks the fill until 305.50; a second $2,000 order is refused by the reservation and cancelling returns the money; the tax statement shows gain **per lot**.

Full runnable code: code/lld/ch19/brokerage/ (see the Full Edition).

Part B — Matching engine (`lld.ch19.stockexchange`)

Files: `OrderBook.java`, `MatchingEngine.java` (routing, `synchronized (book)` per symbol, sequence taken inside the lock, trades published outside it), `MatchingEngineTests.java` (nine tests), `Main.java`.

```
// File: lld/ch19/stockexchange/OrderBook.java (excerpt)
interface ExecutionPricePolicy {
    BigDecimal price(Order buy, Order sell, Order incoming);

    ExecutionPricePolicy SELL_PRICE = (buy, sell, incoming) -> sell.price();
    ExecutionPricePolicy RESTING_PRICE = (buy, sell, incoming) -> incoming == buy ? sell.price() : buy.price();
}

final class OrderBook {
    // ...
    private final NavigableMap<BigDecimal, Deque<Order>> bids = new TreeMap<>(Collections.reverseOrder());
    private final NavigableMap<BigDecimal, Deque<Order>> asks = new TreeMap<>();
    // ...
    /** Match the incoming order against the other side, then rest whatever is left. */
    List<Trade> add(Order incoming) {
        List<Trade> trades = new ArrayList<>();
        NavigableMap<BigDecimal, Deque<Order>> opposite = incoming.side() == Side.BUY ? asks : bids;

        while (!incoming.isDone() && !opposite.isEmpty()) {
            Map.Entry<BigDecimal, Deque<Order>> best = opposite.firstEntry();
            if (!crosses(incoming, best.getKey())) break;          // best price does not meet ours: stop
            Deque<Order> queue = best.getValue();
            Iterator<Order> it = queue.iterator();
            while (!incoming.isDone() && it.hasNext()) {
                Order restingOrder = it.next();                    // oldest first = time priority
                int qty = Math.min(incoming.remainingQty(), restingOrder.remainingQty());
                Order buy = incoming.side() == Side.BUY ? incoming : restingOrder;
                Order sell = incoming.side() == Side.SELL ? incoming : restingOrder;
                trades.add(new Trade(buy.id(), pricePolicy.price(buy, sell, incoming), qty, sell.id()));
                incoming.fill(qty);
                restingOrder.fill(qty);
                if (restingOrder.isDone()) {
                    it.remove();
                    resting.remove(restingOrder.id());
                }
            }
            if (queue.isEmpty()) opposite.pollFirstEntry();
        }
        if (!incoming.isDone()) rest(incoming);
        return trades;
    }
    // ... cancel, bestBid, bestAsk, crosses, rest
}
```

```
== Trades (execution price = sell order's price) ==
#3 237.45 90 #2
#3 236.00 20 #6
#4 236.00 10 #6
#5 236.00 20 #6
== Same input, real-exchange rule (price of the resting order) ==
#3 237.45 90 #2
#3 238.10 20 #6
#4 237.80 10 #6
#5 237.80 20 #6
  9/9 passed
```

The first block matches the problem's expected output exactly; the second prices each trade at the *resting* buyer's price. Tests cover price priority, time priority, no-cross, partial fill, cancel, symbol isolation, validation, and concurrency (8 threads × 500 orders: quantity conserved, book never crossed).

Full runnable code: code/lld/ch19/stockexchange/ (see the Full Edition).

Concurrency & edge cases

Risk	Where	Prevention
Over-spending	Brokerage	`reserveFor` (synchronized on member) subtracts all reservations first
Selling shares twice	Brokerage	Free shares = position − remaining open sells of that symbol
Member/exchange deadlock	Brokerage	Member reserves under its lock, submits outside it; only exchange → member calls
Market order costs more than reserved	Brokerage	Known gap; reserve a buffer (+5%) or a price band
Time in force not enforced	Brokerage	Enum has GTC, FOK, IOC, on-the-open, on-the-close but nothing reads it; belongs in the exchange's fill loop
Same-price orders "together"	Engine	Lock acquisition order = time priority; `sequence` taken inside the lock
Cancel racing a match	Engine	Same per-symbol lock: matched or cancelled, never both
Bad input (duplicate id, zero qty, negative price)	Engine	Validated before registering
Slow trade publishing	Engine	Published after releasing the lock
Floating-point prices	Both	`BigDecimal`; `TreeMap` compares with `compareTo`, so `237.8` = `237.80`

Real-world example

Zerodha offers **SL** (stop-loss limit) and **SL-M** (stop-loss market) — our `StopLimitOrder` and `StopLossOrder`; orders go to exchanges like NSE with price-time priority books. **LMAX** (Martin Fowler's "The LMAX Architecture") runs matching on a **single thread** fed by a ring buffer (the Disruptor) — "one serialised writer per book" taken to its end. US tax rules treat shares as sold FIFO unless you name lots, hence `StockPosition.sell`'s optional lot id.

Extensions & follow-up questions

- **Trailing stop** — new subclass moving its stop with the best price; only `canTradeAt` changes.
- **IOC / FOK** — IOC cancels the remainder after one pass; FOK checks available quantity first and cancels if short.
- **Price alerts** — a `PriceAlert` observer on ticks, firing once past the threshold.
- **Modify order** — reducing quantity keeps priority; price change or increase = cancel + new.
- **Faster engine** — single writer per symbol via a queue, pre-allocated objects, `long` price ticks.
- **Market orders in the engine** — limit at +∞ / 0, never rest the remainder. **Settlement** (T+1) — `unsettledCash`.

In the interview

Brokerage plan: clarify (3) → actors (3) → **key sentence**: *"The four order types differ in exactly two things: whether they may trade yet, and the worst price they accept"* (8) → lots and tax (4) → reserved funds (3) → code the two questions and `StockPosition.sell` (10) → Observer, statements, extensions (5). **Navi, 90 minutes:** parse lines → `OrderBook` with two `TreeMap`s of `ArrayDeque` → matching loop → print trades → tests (sample, price, time, partial, cancel). Pass the sample first, then refactor.

> **30-second answer:** "One abstract `Order` with four tiny subclasses answering 'may I trade now?' and 'what is my worst price?'. Fills are `OrderPart`s and each purchase becomes a lot, sold FIFO. Money is reserved at placement so a member cannot over-commit; the exchange calls back through an `OrderListener`. The matching engine keeps, per symbol, two `TreeMap`s of price levels with FIFO queues — best price first, oldest first — under one lock per symbol."

- *Book data structure and complexity?* — `TreeMap` levels: best level and new level O(log L); FIFO append O(1), O(1) per fill; cancel O(log L) + O(k) in the level (O(1) with a linked handle).

- *Trade price?* — Navi: the sell price. Real continuous trading: the resting order's price (incoming may get "price improvement").

Self-check

1. **Q:** SELL stop at 10.00; price 10.50 → 9.95 → 10.20. When does it trade? **A:** At 9.95, and it stays triggered.
2. **Q:** What does stop-limit add? **A:** A worst price after triggering — it becomes a limit order.
3. **Q:** Why did one market order create two lots? **A:** It filled at two prices; a lot is defined by its purchase price.
4. **Q:** Bids b1 then b2 at 50.00; a sell for 7 at 49. Trades? **A:** b1 gets 5, b2 gets 2 (time priority).
5. **Q:** What is a "crossed" book? **A:** Best bid ≥ best ask — those orders should already have traded.

TL;DR

- **Brokerage:** abstract `Order`, four subclasses answering *may I trade now?* and *worst price?*; fills are `OrderPart`s; purchases are **lots**; money **reserved** at placement; **Observer** callback; statements are a **Template Method**.
- **Matching engine:** per symbol, two `TreeMap<price, FIFO queue>`; best price, then oldest; rest the remainder; one lock per symbol; trade price is a pluggable policy.

Design an Online Auction System

Problem & clarifying questions

An online auction (like eBay) lets a **seller** list an item with a starting price and end time; **bidders** outbid each other; at the end the highest bidder wins — if the hidden **reserve price** was reached.

Question	Assumed answer
Listing contents?	Name, description, category, starting price, minimum increment, optional reserve, duration
Valid bid?	First ≥ starting price; later ≥ current highest + increment
Seller bids? Leader outbids self?	No and no
When does it end?	At end time; a late bid is refused even if the closing job hasn't run
Who is told what?	Outbid bidder immediately; winner and seller at close; seller if unsold
Proxy bidding, anti-sniping, payments?	Extensions

Requirements

Functional: (1) register and log in (login is the usual auth service, not coded); (2) create listings; (3) search by name, category, price range; (4) bid on **active** listings; (5) keep the **current highest bid** and notify; (6) close at end time — **highest bidder wins** if the reserve is met. **Non-functional:** (7) many simultaneous bids stay consistent (one true highest); (8) new bid rules, channels or search criteria without rewriting the core.

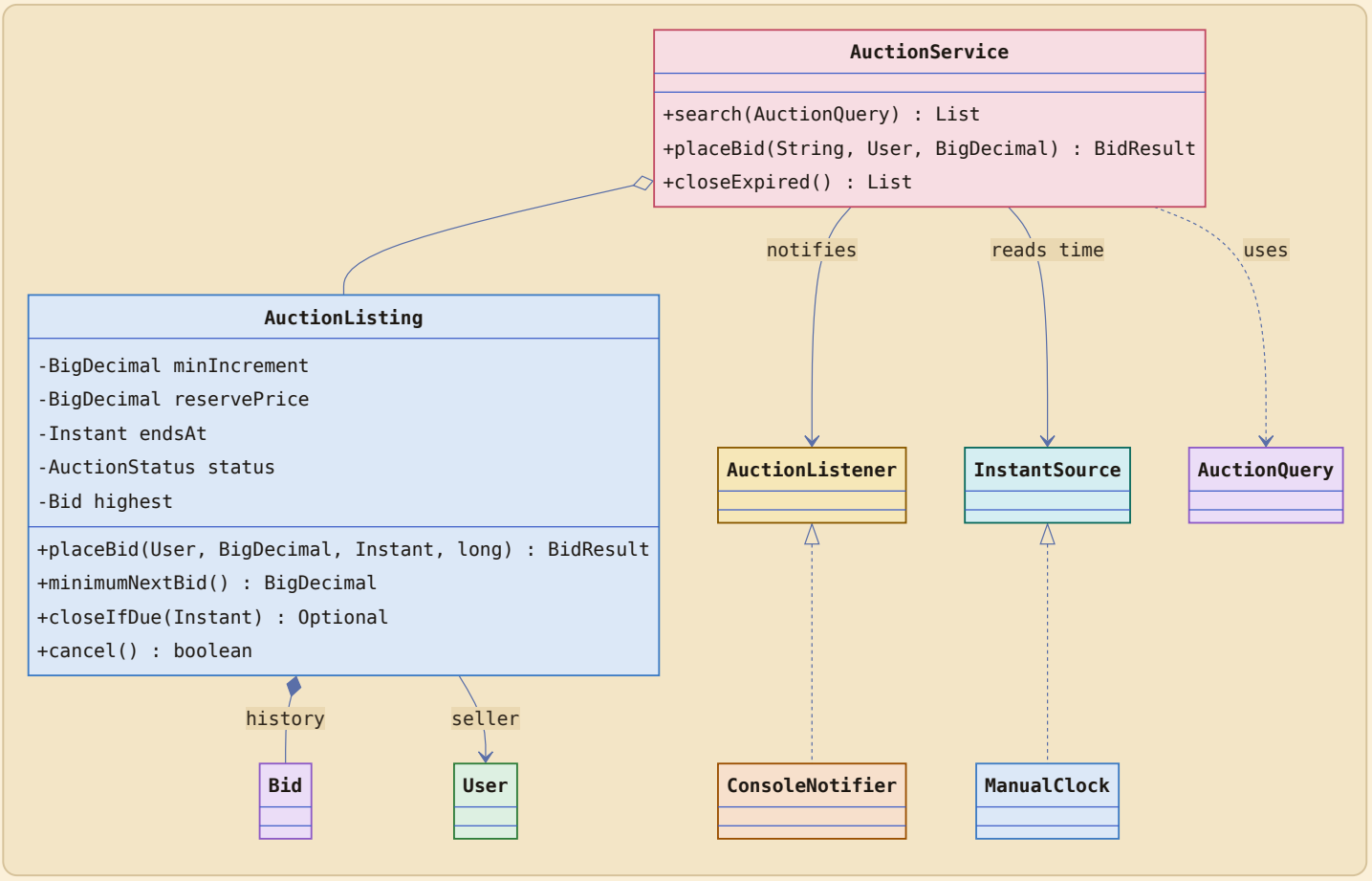
Use cases

Actor	Use cases
Seller	Create listing, cancel (only while nobody has bid), see result
Bidder	Search, view listing and history, bid, be told when outbid or won
System (scheduler)	Activate, close expired, declare winner, notify

Core entities

Class	Responsibility
`User`, `Bid`	Records; bid = bidder, `BigDecimal` amount, time, sequence
`AuctionStatus`	`SCHEDULED → ACTIVE → CLOSED`, or `CANCELLED`
`AuctionListing`	Item, seller, prices, times, history, highest bid; **owns the bidding rules** under its own lock
`BidOutcome` / `BidResult` / `CloseResult`	Why accepted/refused, next minimum, accepted bid, who was outbid; winner and reserve met
`AuctionQuery`	Fluent search; each criterion a `Predicate` (**Specification**)
`AuctionListener` / `ConsoleNotifier`	Observer: new high bid, outbid, won, unsold
`AuctionService`	Facade; calls listeners **outside** listing locks
`ManualClock`	An `InstantSource` moved forward in demos and tests

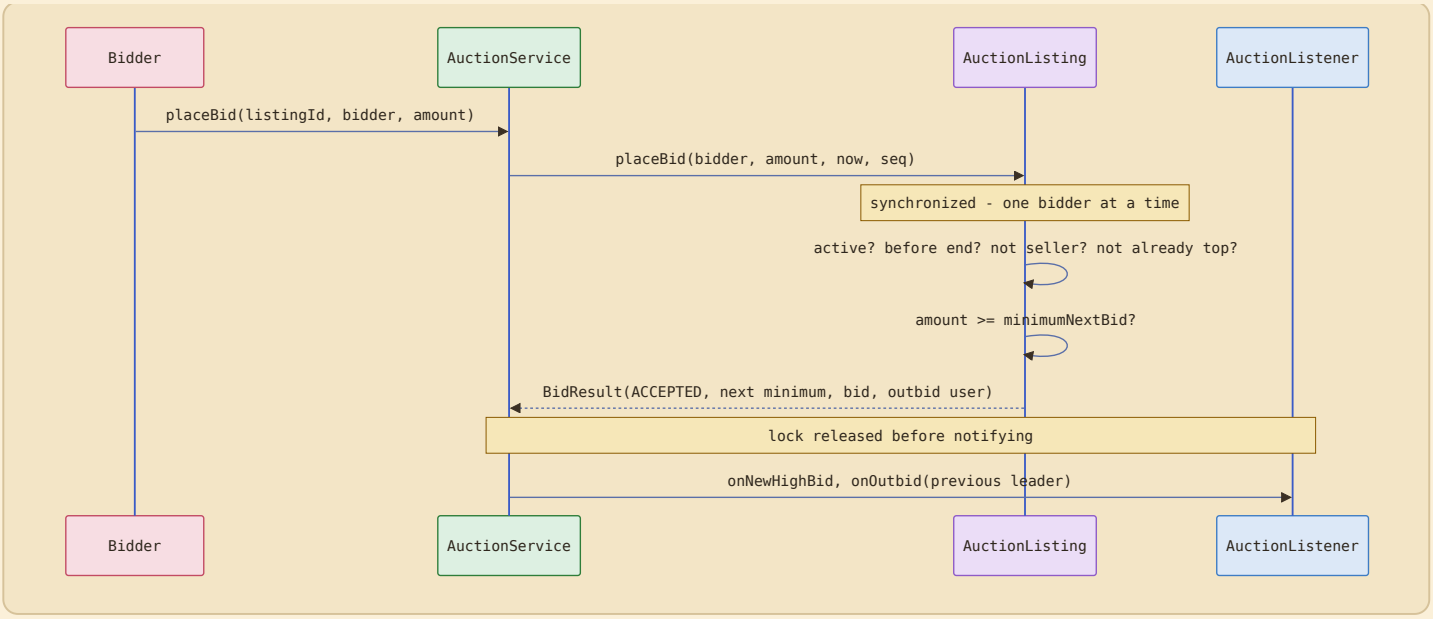
Class diagram



Key flows

Lifecycle: SCHEDULED → ACTIVE (start time) or CANCELLED (seller withdraws); ACTIVE → CANCELLED (withdraw with no bids) or CLOSED (end time). Activation is **lazy**: every method first calls `refresh(now)`, so no thread is needed to flip the state.

Placing a bid (the critical section):



Closing: a scheduler (a `ScheduledExecutorService` in production) calls `closeExpired()`; `closeIfDue(now)` atomically moves `ACTIVE → CLOSED` and returns a winner only if the reserve is met; notifications go out after locks are released.

Design decisions & patterns used

Decision	Why	Alternative rejected
Bid rules inside `AuctionListing` under its lock (a *monitor*: all methods `synchronized`)	Compare-and-replace is atomic; per-listing lock, so auctions don't wait for each other	Global lock; or `AtomicReference<Bid>` + CAS (valid, but seller/leader/time/increment rules are easier under one lock)
Refusals returned (`BidOutcome`), not thrown	A low bid is normal business; caller gets the next minimum	Exception per low bid
Observer called outside the lock	Slow notifications don't block bidders; `BidResult` carries a **snapshot** taken inside the lock	Notifying inside `synchronized` (slow, deadlock risk)
Minimum increment	No 1-cent wars; predictable next minimum	Any higher amount
Reserve price checked only at close	Seller protection without revealing it	Rejecting bids below reserve
Injected `InstantSource`	Testable time (`ManualClock`)	`Instant.now()` everywhere
Specification for search	Each criterion a `Predicate`, combined with `and`	An ever-growing `search(...)` parameter list
Not a Singleton	awesome-low-level-design uses a Singleton `AuctionSystem`; we pass one instance so tests get fresh ones	Global shared state

Java implementation

Package `lld.ch19.auction`: `Auction.java`, `AuctionService.java`, `Main.java`.

```java
// File: lld/ch19/auction/Auction.java (excerpt, in AuctionListing)
    synchronized BidResult placeBid(User bidder, BigDecimal rawAmount, Instant now, long seq) {
        refresh(now);
        BigDecimal amount = money(rawAmount);
        if (status != AuctionStatus.ACTIVE) return refuse(BidOutcome.AUCTION_NOT_ACTIVE);
        if (!now.isBefore(endsAt)) return refuse(BidOutcome.AUCTION_ENDED);  // closing job may not have run yet
        if (bidder.id().equals(seller.id())) return refuse(BidOutcome.SELLER_CANNOT_BID);
        if (highest != null && highest.bidder().id().equals(bidder.id())) return refuse(BidOutcome.ALREADY_HIGHEST);
        if (amount.compareTo(minimumNextBid()) < 0) return refuse(BidOutcome.TOO_LOW);

        Optional<User> outbid = Optional.ofNullable(highest).map(Bid::bidder);
        highest = new Bid(bidder, amount, now, seq);
        bids.add(highest);
        return new BidResult(BidOutcome.ACCEPTED, minimumNextBid(), Optional.of(highest), outbid);
    }

    /** First bid must reach the starting price; later bids must beat the leader by the increment. */
    synchronized BigDecimal minimumNextBid() {
        return highest == null ? startingPrice : highest.amount().add(minIncrement);
    }

    /** ACTIVE -> CLOSED once time is up. Returns empty if not yet due (or already closed). */
    synchronized Optional<CloseResult> closeIfDue(Instant now) {
        refresh(now);
        if (status != AuctionStatus.ACTIVE || now.isBefore(endsAt)) return Optional.empty();
        status = AuctionStatus.CLOSED;
        boolean reserveMet = highest != null && highest.amount().compareTo(reservePrice) >= 0;
        return Optional.of(new CloseResult(this, reserveMet ? Optional.of(highest) : Optional.empty(), reserveMet));
    }

// File: lld/ch19/auction/AuctionService.java (excerpt)
    BidResult placeBid(String listingId, User bidder, BigDecimal amount) {
        AuctionListing a = listings.get(listingId);
        if (a == null) throw new IllegalArgumentException("no listing " + listingId);
        BidResult r = a.placeBid(bidder, amount, clock.instant(), bidSeq.incrementAndGet());
        r.accepted().ifPresent(top -> {                 // snapshot taken inside the lock
            listeners.forEach(l -> l.onNewHighBid(a, top));
            r.outbid().ifPresent(u -> listeners.forEach(l -> l.onOutbid(a, u, top.amount())));
        });
        return r;
    }
```

Full runnable code: code/lld/ch19/auction/ (see the Full Edition).

```
  Ana bids 90 (below start) -> TOO_LOW (next minimum 100.00)
  Ana bids 120 (already top) -> ALREADY_HIGHEST (next minimum 110.00)
  Bob bids 105 (increment is 10) -> TOO_LOW (next minimum 110.00)
  Sam bids on own item -> SELLER_CANNOT_BID (next minimum 120.00)
  accepted=28 (varies run to run), highest=650.00 by bidder49, history strictly increasing by >= increment: true
  late bid -> AUCTION_ENDED (next minimum 660.00)
```

Every refusal rule fires; outbid notices print before the bidder's own result line (the service notifies before returning). The watch closes unsold (60 < 500 reserve). Of 50 concurrent bids the accepted count varies, but the highest is always 650.00 and the history always rises by at least the increment — the invariant the lock protects.

Concurrency & edge cases

Case	Handling
Same amount at the same time	First through the lock wins; the second is `TOO_LOW`
Bid exactly at end time	`!now.isBefore(endsAt)` → `AUCTION_ENDED` (end is exclusive)
Close job runs late / races a bid	Late bids refused anyway; both `synchronized`, so the bid lands before or after the close
Slow or failing listener	Outside the lock; production publishes to a retrying queue
Seller cancels after bids	Refused
Money precision	Scale 2 `BigDecimal`, `compareTo`
Very hot auction	Single writer per auction (partition by id, e.g. Kafka), or `UPDATE ... SET high = ? WHERE id = ? AND high < ?`

Real-world example

eBay uses **proxy bidding** (it bids for you up to your maximum, by the increment), hidden **reserve prices**, and fixed end times — hence "sniping". Other sites use a **soft close** (late bids extend the end) to defeat it.

Extensions & follow-up questions

- **Proxy bidding** — per-bidder `maxAmount`; after each accepted bid, inside the lock, the strongest proxy responds with `min(its max, competitor + increment)`; ties to the earlier proxy (`Bid.seq`).
- **Anti-sniping** — soft close: a bid in the last N minutes extends `endsAt` (non-final, same lock).
- **Buy It Now** — fixed price closes the auction if paid before the first bid.
- **Dutch / sealed / Vickrey** — an `AuctionRules` strategy (`validate`, `winner`, `price`); Vickrey: highest wins, pays second-highest.
- **Payment after winning** — `AWAITING_PAYMENT`, 48 hours, else offer to the second bidder.
- **Scale the close job** — a priority queue by `endsAt`.

In the interview

Plan: bid rules (4 min) → entities and lifecycle (8) → code `placeBid` under the listing lock and **say why the lock is per listing** (12) → closing and notifications outside the lock (6) → concurrency and extensions (10).

> **30-second answer:** "Each listing owns its bid rules and highest bid, and `placeBid` is synchronized on the listing, so compare-and-replace is atomic and different auctions never contend. Refusals return a reason and the next minimum. The service notifies outbid users and, at close, the winner and seller, always after releasing the lock. Time comes from an injected clock, so closing and late bids are testable."

- *Why not `AtomicReference` + CAS?* — The rule also checks seller, leader and time; one small lock is simpler and still fast per listing.
- *Close on time across servers?* — A scheduler with a time-ordered index; `closeIfDue` is idempotent, so running twice is harmless.
- *Live price for thousands?* — Push `onNewHighBid` to pub/sub and WebSockets; viewers read a cache.

Self-check

1. **Q:** When is a bid `TOO_LOW`? **A:** Below the starting price with no bids, or below highest + increment.
2. **Q:** Why notify after the listing call returns? **A:** Short lock; no slowdown or deadlock from notification code.
3. **Q:** Bid at exactly the end instant? **A:** Refused — the end is exclusive.
4. **Q:** Where does the reserve matter? **A:** Only at close.
5. **Q:** What is the Specification pattern here? **A:** Search criteria as composable `Predicate`s.

TL;DR

- One **lock per listing**; all bid rules inside it; refusals are values, not exceptions.
- `SCHEDULED → ACTIVE → CLOSED | CANCELLED` ; activation lazy, closing idempotent.
- **Observer** after the lock; **injected clock**; **Specification** for search.

Design a Music Streaming Service (Spotify)

Problem & clarifying questions

Users search a big catalog, build playlists and play songs with play, pause, skip, previous, seek, shuffle and repeat. Free and paid plans differ; the service suggests music.

Question	Assumed answer
Search?	Songs, albums, artists
Playlists?	Users create, reorder, edit
Controls?	Play, pause, resume, skip, previous, seek; shuffle; repeat off / all / one
Plans?	FREE (ads, limited skips/hour, no downloads), PREMIUM (no ads, unlimited skips, offline)
Recommendations?	A history-based stub (real ones are ML)
Audio bytes, DRM, CDNs?	Out of scope — the player models *what* plays

Requirements

Functional: (1) browse/search songs, albums, artists; (2) playlists; (3) accounts (auth is the usual service); (4) play, pause, skip, seek (+ previous, shuffle, repeat); (5) recommendations from history; (6) FREE vs PREMIUM limits. **Non-functional:** (7) concurrent users on a read-mostly, thread-safe catalog; (8) large scale (indexes in production); (9) extensible — sharing, offline, new play orders and recommenders.

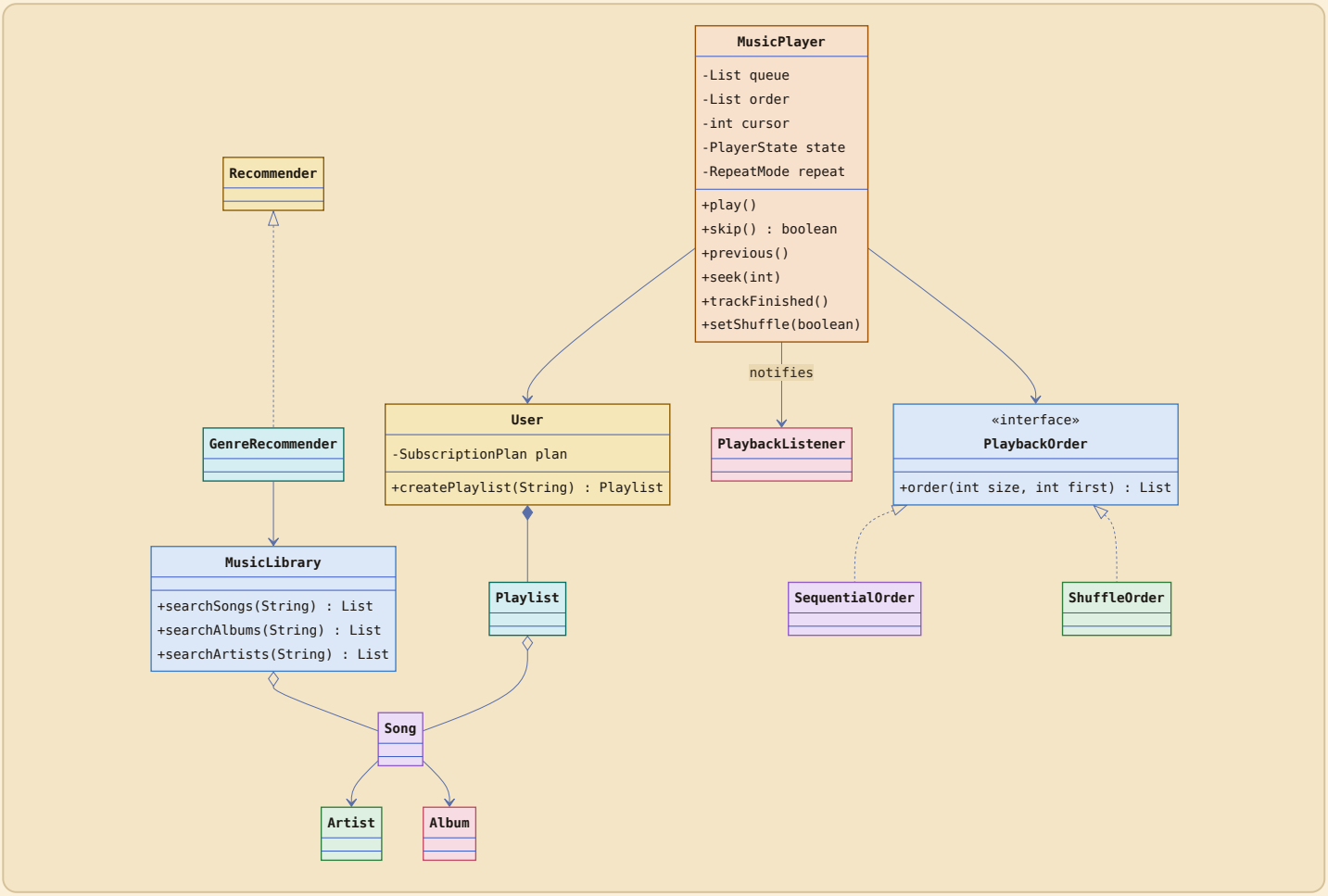
Use cases

Actor	Use cases
Listener	Search, controls, shuffle/repeat, playlists, upgrade plan
Artist / label	Add songs and albums
System	Record plays (history, royalties), ads for free users, recommend

Core entities

Class	Responsibility
`Artist`, `Album`, `Song`, `Genre`	Immutable catalog records
`MusicLibrary`	Thread-safe catalog with search by title, album, artist
`SubscriptionPlan`	Enum **carrying the rules**: price, skips/hour, ads, offline
`User`, `Playlist`	Plan, playlists, history; ordered song list (add, remove, move)
`PlaybackOrder` → `SequentialOrder`, `ShuffleOrder`	Strategy: a permutation starting from the current song
`RepeatMode`, `PlayerState`	`OFF/ALL/ONE`; `STOPPED/PLAYING/PAUSED`
`MusicPlayer`	One session: queue, order, cursor, position, state, skip window, ad counter
`PlaybackListener`	Observer on "song started"
`Recommender` → `GenreRecommender`	Strategy: unplayed songs from favourite genres

Class diagram



Key flows

Player states: STOPPED → PLAYING (play) ⇄ PAUSED (pause/resume); PLAYING → PLAYING on skip or track end with more songs; PLAYING → STOPPED at end of queue with repeat OFF.

Song ends or Next pressed: natural end with repeat ONE → replay; else more songs in the order → cursor + 1; else repeat ALL/ONE → build a new order (reshuffle if shuffling) and play the first; else STOPPED. Pressing Next on repeat-ONE moves on (repeat-one only affects natural ends). **Skip** first checks the plan's rolling one-hour limit: a deque of skip timestamps, drop those older than an hour, compare with `skipsPerHour`.

Shuffle keeps the current song: a new permutation starts with the playing song and shuffles the rest (`Collections.shuffle(list, random)`); turning it off rebuilds the sequential order from the current song. The queue never changes — only the **order of indexes** — so nothing is lost.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for play order	Sequential, shuffle, "smart shuffle" interchangeable	`if (shuffle)` in skip, previous, wrap
Order = indexes into an unchanged queue	Shuffle off restores the original	Physically shuffling the queue
Seeded `Random`	Repeatable demos/tests	`Math.random()`
Plan rules in `SubscriptionPlan` enum	No `if (plan == FREE)`; new plan = new constant	Boolean flags on `User`
Observer for plays	History, royalties, recommendations react to "song started"	Player calling each subsystem
Strategy for recommendations	Genre today, collaborative filtering tomorrow	One algorithm in `User`
One `MusicPlayer` per session, all `synchronized`	Phone + desktop may send commands at once; library uses concurrent maps	One shared player
State as enum, not State pattern	Three trivial states; GoF State pays off only for substantial per-state behaviour	Three state classes
No Singletons	awesome-low-level-design makes `MusicLibrary`, `UserManager`, `MusicRecommender` Singletons; we inject	Global `getInstance()`

Java implementation

Package `lld.ch19.music`: `Catalog.java`, `Player.java`, `Main.java`.

```java
// File: lld/ch19/music/Catalog.java (excerpt)
enum SubscriptionPlan {
    FREE(new BigDecimal("0.00"), 6, true, false),
    PREMIUM(new BigDecimal("10.99"), Integer.MAX_VALUE, false, true);
    // ... monthlyPrice, skipsPerHour, ads, offlineDownloads
}

// File: lld/ch19/music/Player.java (excerpt)
/** Strategy: in what order should the queue be played? */
interface PlaybackOrder {
    /** A permutation of 0..size-1 that starts with {@code first}. */
    List<Integer> order(int size, int first);
}

final class ShuffleOrder implements PlaybackOrder {
    private final Random random;
    ShuffleOrder(long seed) { this.random = new Random(seed); }   // seeded: repeatable demo and tests
    @Override public List<Integer> order(int size, int first) {
        List<Integer> rest = new ArrayList<>(IntStream.range(0, size).filter(i -> i != first).boxed().toList());
        Collections.shuffle(rest, random);
        rest.add(0, first);                    // the song you are hearing keeps playing
        return rest;
    }
}
// ... in MusicPlayer:
    synchronized boolean skip() {
        if (state == PlayerState.STOPPED) return false;          // nothing to skip
        Instant now = clock.instant();
        while (!recentSkips.isEmpty() && recentSkips.peekFirst().isBefore(now.minus(Duration.ofHours(1))))
recentSkips.pollFirst();
        if (recentSkips.size() >= user.plan().skipsPerHour) {
            System.out.println("  [player] skip limit reached on " + user.plan() + " plan");
            return false;
        }
        recentSkips.addLast(now);
        advance(false);
        return true;
    }

    synchronized void setShuffle(boolean on) {
        int now = order.isEmpty() ? 0 : order.get(cursor);
        playbackOrder = on ? new ShuffleOrder(42) : new SequentialOrder();
        order = playbackOrder.order(queue.size(), now);
        cursor = 0;
    }
```

Full runnable code: code/lld/ch19/music/ (see the Full Edition).

```
== 4. Shuffle keeps the current song, reorders the rest ==
  now Blue in Green - Miles Davis, up next [Love of My Life - Queen, So What - Miles Davis, Bohemian Rhapsody - Queen,
Hello - Adele]
  [player] skip limit reached on FREE plan
  recommended: [You're My Best Friend - Queen, All Blues - Miles Davis, When We Were Young - Adele]
```

Repeat-ONE replays "Bohemian Rhapsody" on natural end; on FREE an ad plays every three songs, repeat-ALL wraps and reshuffles, and the seventh skip in the hour is refused; PREMIUM has no ads and with repeat OFF the queue ends; the recommender suggests *unheard* songs from the most-played genres.

Concurrency & edge cases

Case	Handling
Two devices press "next" together	`synchronized` player: applied one after the other
Catalog updated during search	`ConcurrentHashMap`, weakly consistent iteration
Playlist edited while playing	`load` copies it (`List.copyOf`) — the queue is a snapshot
Seek out of range	Clamped to `[0, duration]`
Previous after 3 seconds	Restarts the song; within 3 s goes back one
Skip when stopped	Refused, not counted
Plan change mid-session	`plan` is `volatile`
Empty queue	`play()` throws `IllegalStateException`

Real-world example

Spotify's free tier has limited skips (e.g. six per hour on mobile) and ads; Premium removes both and allows downloads — our enum's rules. In 2014 Spotify engineers wrote that truly random shuffle "felt" non-random when an artist repeated, so they spread same-artist songs apart — a new `PlaybackOrder` in our design. Discover Weekly is the industrial `Recommender`.

Extensions & follow-up questions

- **Play next / add to queue** — a separate user queue consumed before the context order.
- **Offline** — `offlineDownloads`, a `DownloadManager`, and a local/remote `AudioSource` Strategy.
- **Collaborative playlists** — collaborators + edit log, changes pushed via Observer.
- **Crossfade / gapless** — a `TransitionStrategy` around `trackFinished`.
- **Royalties** — a `PlaybackListener` counting a play after a listening threshold.
- **Search at scale** — inverted index with prefix and fuzzy matching. **Smart shuffle** — a `PlaybackOrder` spacing out artists.

In the interview

Plan: controls, plans, scope (4 min) → catalog + search (6) → user, playlist, plan (5) → **player**: queue + order + cursor, state machine, shuffle/repeat (15) → Observer, recommender, concurrency, extensions (10).

> **30-second answer:** "Immutable catalog records in a thread-safe library. Each user has a player session holding the queue, a play order produced by a `PlaybackOrder` strategy, a cursor and a repeat mode. Shuffle rebuilds the order around the current song. Plan limits live in the plan enum. A playback listener records history, which feeds a pluggable recommender."

- *How does shuffle avoid repeats?* — It is a permutation (each index once), not "random next".
- *Previous in shuffle mode?* — Walk back along the same order list.

Self-check

1. **Q:** Why store the order as indexes? **A:** Shuffle off restores the original order.
2. **Q:** Repeat-ONE and Next? **A:** Moves on; replay only on natural end.
3. **Q:** How is "6 skips per hour" enforced? **A:** Deque of skip times; drop entries older than an hour; refuse at the limit.
4. **Q:** Why an enum, not the State pattern? **A:** The states have almost no distinct behaviour.

TL;DR

- Catalog = immutable records in a concurrent library.
- Player = **queue + order (Strategy) + cursor + repeat mode + state**; shuffle keeps the current song.
- Plan rules in the **enum**; plays via **Observer**; recommendations via **Strategy**.

Design an Online Food Delivery Service (Swiggy) + FoodKart

Two shapes:

- **Part A — Swiggy-style platform** (awesome-low-level-design): restaurants and menus, cart, tracked orders, payments, **rider assignment** by a pluggable strategy. We add **kitchen capacity** (max items cooking at once), the twist of a reported Flipkart food-ordering variant (not part of FoodKart).
- **Part B — FoodKart** (Flipkart machine coding, kumaransg/LLD): command-driven — users and pincodes, one-dish restaurants, ratings, `show_restaurant` by rating or price, `place_order` — plus a **restaurant selection strategy** filtered by capacity.

Problem & clarifying questions

Question	Assumed answer
Mix restaurants in one cart?	No — one restaurant per cart
Who confirms?	The restaurant; it refuses if the kitchen is full
When is capacity freed?	When food is ready, not when delivered
Rider choice?	A strategy: nearest, or fair share (fewest deliveries today)
No rider free?	Order waits in a FIFO queue; the next freed rider takes the oldest
Customer cancel?	Only before cooking (PENDING or CONFIRMED)
Payments?	UPI, card, cash on delivery (nothing to refund)
GPS, ETA, batching, surge?	Extensions

Requirements

Functional (Part A): (1) browse restaurants, view menus, order; (2) restaurants manage menus, prices, availability; (3) agents accept and fulfil; (4) track status with notifications; (5) several payment methods; (6) real-time notifications to customers, restaurants, agents.

Functional (Part B — FoodKart): (7) `register_user(name, gender, phone, pincode)`, phone unique; (8) `login_user(phone)` switches context; (9) `register_restaurant(name, pincodes "A/B", dish, price, quantity)` by the logged-in owner — **one** dish, **many** pincodes; (10) `update_quantity`, `update_location` — owner only; (11) `rate_restaurant(name, 1..5, optional comment)`, rating = average; (12) `show_restaurant(rating|price)` — **serviceable** (my pincode **and** quantity > 0), **descending**; (13) `place_order(name, qty)`, bonus order history; (14) *variant:* order without naming a restaurant, chosen by a **selection strategy** among those delivering here **with** capacity for the whole quantity.

Non-functional: no double-booked kitchen capacity or riders under concurrency; no negative stock, no rider on two orders; in-memory, demoable driver, graceful failures.

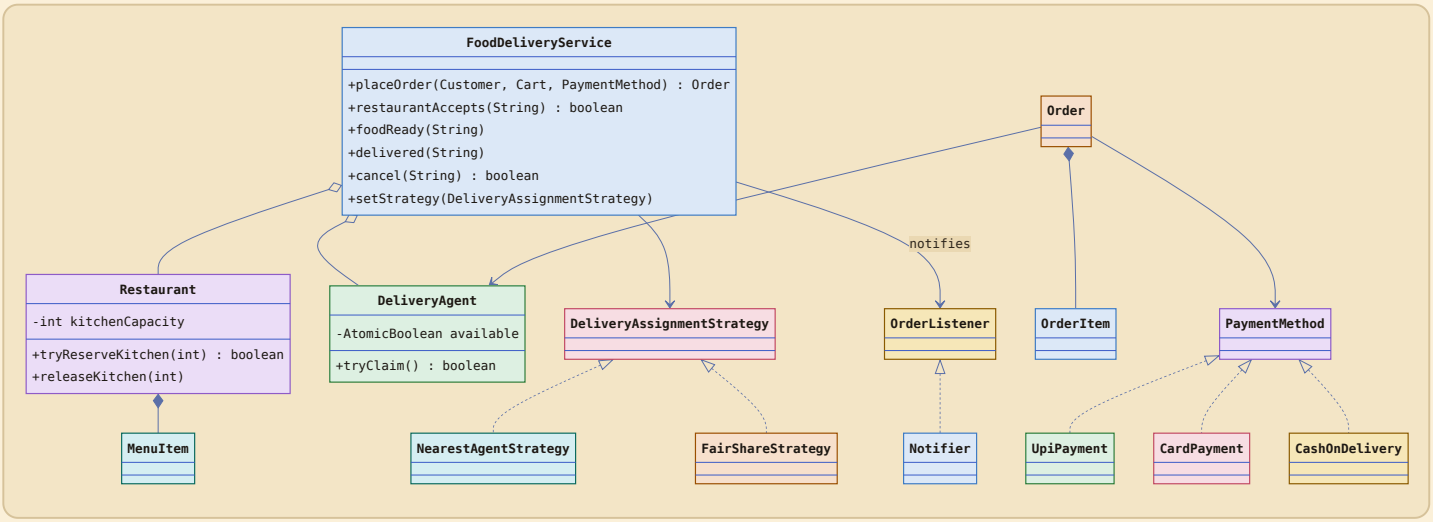
Use cases

Actor	Use cases
Customer	Browse nearby, cart, pay, order, track, cancel, rate
Restaurant	Manage menu, accept/refuse, preparing, ready
Delivery agent	Get assigned, pick up, deliver
System	Fees, rider assignment, waiting queue, notifications

Core entities

Class	Responsibility
`Location`, `Customer`, `MenuItem`	Point + `distanceTo`; customer details; name, price, available flag
`Restaurant`	Menu, open flag, **kitchen capacity**, `tryReserveKitchen` / `releaseKitchen`
`Cart`	One restaurant + quantities; refuses a second restaurant
`OrderItem`	Snapshot of name, **unit price at order time**, quantity
`OrderStatus`	PENDING → CONFIRMED → PREPARING → READY_FOR_PICKUP → OUT_FOR_DELIVERY → DELIVERED, or CANCELLED
`Order`	Customer, restaurant, items, fee, payment, history, agent
`PaymentMethod` → `UpiPayment`, `CardPayment`, `CashOnDelivery`	Sealed; COD not paid upfront
`DeliveryAgent`	Location, `AtomicBoolean available`, deliveries today; `tryClaim()` is a CAS
`DeliveryAssignmentStrategy` → `NearestAgentStrategy`, `FairShareStrategy`	Ranks free riders
`OrderListener` / `Notifier`, `FoodDeliveryService`	Observer; facade with the waiting-for-rider queue
(FoodKart) `User`, `Restaurant`, `Review`, `FoodOrder`, `SortBy`, `RestaurantSelectionStrategy`, `FoodKartService`	`Restaurant.take(q)` is the atomic capacity check

Class diagram



FoodKart (Part B): `FoodKartService` (current user; `registerUser`, `loginUser`, `registerRestaurant`, `updateQuantity`, `updateLocation`, `rateRestaurant`, `showRestaurants(SortBy)`, `placeOrder(name, qty)`, `placeOrder(qty, strategy)`, `orderHistory`) holds `User`s, `Restaurant`s (pincodes, one dish, price, quantity, `take`, `serviceableFor`, `rating`; owns `Review`s) and `FoodOrder`s; `RestaurantSelectionStrategy` → `HighestRatingStrategy`, `LowestPriceStrategy`.

Key flows

Order states: PENDING → CONFIRMED (accepted, kitchen slots reserved) or CANCELLED (kitchen full / customer); CONFIRMED → PREPARING or CANCELLED (slots released); PREPARING → READY_FOR_PICKUP (slots released) → OUT_FOR_DELIVERY (rider assigned) → DELIVERED. Real apps split pickup into "rider assigned / arrived / picked up" — a local enum change.

Food ready → rider: `foodReady` marks READY_FOR_PICKUP, releases slots, adds the order to the queue; while the queue is not empty: the strategy ranks free agents, `tryClaim()` (CAS true → false) on each in turn; claimed → poll, assign, OUT_FOR_DELIVERY, notify; none → stop, order waits. `delivered()` frees the rider and runs the same loop.

FoodKart `place_order(name, qty)`: logged in? → restaurant exists? → delivers to my pincode? → `take(qty)` atomically → record history → "Order Placed Successfully." else "Cannot place order". **Selection variant:** filter by pincode and `quantity ≥ qty` → strategy picks → `take(qty)`; if a racer took it, drop that restaurant and ask again (bounded retries).

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for riders; `rank` returns a list	Swappable at run time (`setStrategy`); the list lets us try the next rider if the first was just taken	Returning one rider
CAS on the rider (`compareAndSet(true,false)`)	Exactly one claimer wins, even from outside the `synchronized` dispatcher (another dispatcher, a "rider accepts" button)	`isAvailable()` then set
Kitchen capacity reserved on accept, released on ready	"Cannot accept while the kitchen is full"	Unlimited orders
FIFO waiting queue	No free rider is normal at peak; fair	Failing the order
State machine in `OrderStatus`	Cancel after cooking impossible	Scattered `if`s
`OrderItem` copies price	Later menu changes can't alter orders	Referencing `MenuItem`
Sealed `PaymentMethod` records	Closed set; COD overrides `paidUpfront()`	`String paymentType`
Observer `OrderListener`	Customer, restaurant, rider apps react to the same events	Service printing itself
One restaurant per cart	Single kitchen, pickup, fee	Split orders
FoodKart: `SortBy` enum holds its comparator	`sorted(sortBy.comparator)`, descending, name tie-break	`if (sort.equals("rating"))`
FoodKart: owner recorded; `take(q)` synchronized	Only the owner updates; two users can't take the last plates	Anyone updates; get + set

Java implementation

Part A — Food delivery platform (`lld.ch19.fooddelivery`)

Files: `Model.java`, `FoodDeliveryService.java`, `Main.java`.

```java
// File: lld/ch19/fooddelivery/Model.java (excerpt)
    synchronized boolean tryReserveKitchen(int items) {
        if (itemsInKitchen + items > kitchenCapacity) return false;
        itemsInKitchen += items;
        return true;
    }
// ... in DeliveryAgent:
    boolean tryClaim() { return available.compareAndSet(true, false); }

    void completeDelivery() { deliveriesToday.incrementAndGet(); available.set(true); }

// File: lld/ch19/fooddelivery/FoodDeliveryService.java (excerpt)
interface DeliveryAssignmentStrategy {
    List<DeliveryAgent> rank(Order order, List<DeliveryAgent> free);
}
// ...
/** Fewest deliveries today first (spreads earnings), nearest as tie-breaker. */
final class FairShareStrategy implements DeliveryAssignmentStrategy {
    @Override public List<DeliveryAgent> rank(Order o, List<DeliveryAgent> free) {
        Location r = o.restaurant().location();
        return free.stream().sorted(Comparator.comparingInt(DeliveryAgent::deliveriesToday)
                .thenComparingDouble(a -> a.location().distanceTo(r))).toList();
    }
}
// ...
    private synchronized void dispatchWaiting() {
        Order o;
        while ((o = awaitingRider.peek()) != null) {
            List<DeliveryAgent> free = agents.stream().filter(DeliveryAgent::isAvailable).toList();
            Optional<DeliveryAgent> got = Optional.empty();
            for (DeliveryAgent a : strategy.rank(o, free)) if (a.tryClaim()) { got = Optional.of(a); break; }
            if (got.isEmpty()) {
                System.out.println("    (" + o.id() + " waiting for a free rider)");
                return;
            }
            awaitingRider.poll();
            o.assign(got.get());
            transition(o, OrderStatus.OUT_FOR_DELIVERY);
        }
    }
```

Full runnable code: code/lld/ch19/fooddelivery/ (see the Full Edition).

```
== 4. Kitchen capacity: 4 items max at Dosa Corner ==
    (Dosa Corner kitchen full: needs 2 item slots, free 1)
    [OD103 CANCELLED] -> customer Asha
    (OD104 waiting for a free rider)
    [OD101 DELIVERED] -> customer Asha
    [OD104 OUT_FOR_DELIVERY] -> customer Vikram: Ravi is on the way
  o6 rider: Meena (fewer deliveries today)
```

The far grill (beyond 10 km) is hidden; a second restaurant in the cart is refused; fee = 20 + 8 × 1.4 km = 31.20 and a later dosa price change does not alter the order; an order out for delivery cannot be cancelled while a confirmed card order can (and is refunded); with both riders at one spot, fair share picks Meena (1 delivery) over Ravi (2).

Part B — FoodKart (`lld.ch19.foodkart`)

Files: `FoodKart.java`, `Main.java` (replays the sample test case, then extras).

```java
// File: lld/ch19/foodkart/FoodKart.java (excerpt)
    synchronized boolean take(int q) {
        if (q <= 0 || q > quantity) return false;
        quantity -= q;
        return true;
    }

    /** Serviceable = delivers to this pincode AND has non-zero quantity (from the problem statement). */
    boolean serviceableFor(String pincode) { return pincodes.contains(pincode) && quantity() > 0; }
// ...
enum SortBy {
    RATING(Comparator.comparingDouble(Restaurant::rating).reversed()),
    PRICE(Comparator.comparing(Restaurant::price).reversed());

    final Comparator<Restaurant> comparator;
    SortBy(Comparator<Restaurant> c) { comparator = c.thenComparing(Restaurant::name); }
}

/** Strategy: when the user does not name a restaurant, which one should we pick? */
interface RestaurantSelectionStrategy {
    Optional<Restaurant> select(List<Restaurant> canFulfil);
}
// ... HighestRatingStrategy, LowestPriceStrategy
    String placeOrder(int qty, RestaurantSelectionStrategy strategy) {
        User u = requireLogin();
        List<Restaurant> canFulfil = restaurants.values().stream()
                .filter(r -> r.pincodes().contains(u.pincode()) && r.quantity() >= qty).toList();
        for (int attempt = 0; attempt < 3; attempt++) {           // retry if a racer took the capacity
            Optional<Restaurant> pick = strategy.select(canFulfil);
            if (pick.isEmpty()) return "Cannot place order";
            if (pick.get().take(qty)) {
                record(u, pick.get(), qty);
                return "Order Placed Successfully from " + pick.get().name() + ".";
            }
            canFulfil = canFulfil.stream().filter(r -> r != pick.get()).toList();
        }
        return "Cannot place order";
    }
```

Full runnable code: code/lld/ch19/foodkart/ (see the Full Edition).

```
show_restaurant(price):
  Food Court-2, Burger
  Food Court-1, NI Thali
place_order(Food Court-1, 2): Order Placed Successfully.
place_order(Food Court-2, 7): Cannot place order
highest rating: Order Placed Successfully from Food Court-2.
lowest price  : Order Placed Successfully from Food Court-4.
```

The sample block reproduces every expected line (price-sorted Burger 120 before Thali 100; owner updates to `Burger - 8`). Extras show owner-only updates, rating validation, a comment-less rating and a non-serviceable pincode. The variant: the two strategies choose **different** restaurants, Food Court-1 (3 plates) is never picked for 4, and once nobody has 4 left the order is cleanly refused.

Concurrency & edge cases

Case	Handling
Two claims for one rider	`tryClaim()` CAS: one wins, the loser tries the next ranked rider
Two dispatch loops	`dispatchWaiting` is `synchronized` (FIFO kept)
Kitchen over-booked	`tryReserveKitchen` `synchronized`
Cancel races accept	Both lock the `Order`; accept re-checks PENDING; slots released only if reserved
Item sold out after adding	`placeOrder` re-checks `available`
Restaurant closes	Hidden in `browse`; `placeOrder` refuses
COD cancelled	No refund
FoodKart: no login / duplicates / rating out of 1–5	`requireLogin()` throws; `putIfAbsent` rejects; `Review` compact constructor rejects
FoodKart: capacity taken after selection	`take()` fails; restaurant dropped; strategy asked again

Real-world example

Swiggy and Zomato show the same states (placed, confirmed, preparing, rider assigned/picked up, delivered) and ask you to **reset your cart** when adding from a second restaurant. Rider dispatch uses distance, load and fairness and may batch two orders — variations of `DeliveryAssignmentStrategy`. FoodKart is a Flipkart 90-minute, in-memory, driver-class exercise rewarding clean separation over clever algorithms.

Extensions & follow-up questions

- **Batching** — a strategy accepting a rider already carrying one order when the second pickup is on the way (agent capacity 2).
- **ETA** — an `EtaEstimator`: prep time by load + travel time.
- **Surge / distance fees** — a `DeliveryFeePolicy` Strategy.
- **Rider rejects** — rider back to available, order to the queue head, rider excluded for it.
- **Multi-dish FoodKart** — `Map<String, MenuItem>`; `take` per item, all-or-nothing.
- **Order split across restaurants** — the strategy returns a set covering all items; reserve in each, roll back on failure.
- **Live tracking** — rider location events forwarded by an Observer over WebSocket.

In the interview

Design round (45 min): clarify (5) → entities + one-restaurant cart (8) → **state machine** (7) → **assignment strategy + CAS claim + waiting queue** (12) → payments/notifications (5) → concurrency and extensions (8). **FoodKart (90 min):** model classes → one service method per command → driver replaying the sample → `SortBy` → owner checks and validation → selection strategy → history; runnable at every step, exact output lines.

> **30-second answer:** "Restaurants own their menus and a kitchen capacity reserved on accept and released when food is ready. Orders copy item prices and move through a state machine from PENDING to DELIVERED. When food is ready the order joins a FIFO queue; a pluggable assignment strategy ranks free riders and we claim one with a compare-and-set, so a rider can never get two orders. Every status change is published to observers for the customer, restaurant and rider apps."

- *Why release capacity on ready?* — The kitchen is free once food leaves; delivery is the rider's constraint.
- *Why does the strategy return a list?* — The top rider may be claimed between ranking and claiming; fall through without re-ranking.
- *Scale across cities?* — Partition by zone; one dispatcher (single writer) per zone.

Self-check

1. **Q:** Which states allow a customer cancel? **A:** PENDING and CONFIRMED — after that the food cost is spent.

2. **Q:** What stops two orders getting one rider? **A:** `AtomicBoolean.compareAndSet(true, false)` in `tryClaim()`.
3. **Q:** FoodKart "serviceable"? **A:** Delivers to the user's pincode and quantity > 0.
4. **Q:** Why is Burger (120) before NI Thali (100) by price? **A:** The statement asks for descending order.
5. **Q:** Patterns used? **A:** Strategy (assignment, selection, payment), Observer, state machine, Facade.

TL;DR

- One-restaurant cart; order copies prices; **state machine** PENDING → … → DELIVERED.
- **Kitchen capacity** reserved on accept, released on ready; **riders** claimed with **CAS**, chosen by a swappable **strategy**, FIFO queue when none are free.
- FoodKart: pincode serviceability, one dish per restaurant, average rating, `SortBy` enum, atomic `take`, capacity-filtered **selection strategy**.

Design a Grocery Store (POS & Checkout)

A **point-of-sale (POS) system** runs a supermarket till: scan, running total, offers, payment in one or more ways, receipt, stock down. Three things make it different from the Amazon cart: **weighed items** (produce priced per kg from the scale), **interacting offers** (buy-2-get-1, "3 for ₹80", % off produce, a basket coupon — the combining rule decides price, tax and refunds), and **the till never blocks a physical sale** (if the system says 0 but the customer holds the carton, the count is wrong). Money: rupees in `BigDecimal` (`Rs` in the output).

Problem & clarifying questions

Question	Assumed answer
Item identification?	Barcode (EAN-13) for packaged goods; 4-digit **PLU** + scale weight for produce.
Tax?	Shelf prices tax-inclusive (India/UK/EU); support US-style tax-on-top too.
Offers and stacking?	One item offer per product — the **best** for the customer; then one basket coupon on what is still owed.
Payment?	Cash (change), card, UPI; **split tender**.
Changes of mind?	Void a line; back to scanning if nothing paid; cancel (reverse card/UPI).
Returns?	With the receipt; refund the **net paid**, never more units than bought.
Stock?	Down on sale, up on return; alerts; never refuse a sale because of the count.

Requirements

Functional: lookup by barcode/PLU, weighed pricing; sale with voidable lines; item offers + basket coupons with stacking rules; tax per category (inclusive/exclusive); split payment; immutable receipt; returns against a receipt; stock updates with low/negative alerts. **Non-functional:** exact money (`BigDecimal`, `HALF_UP`, parts add up); several registers share catalog/stock/receipts safely; deterministic pricing; auditable voids and scan-time prices.

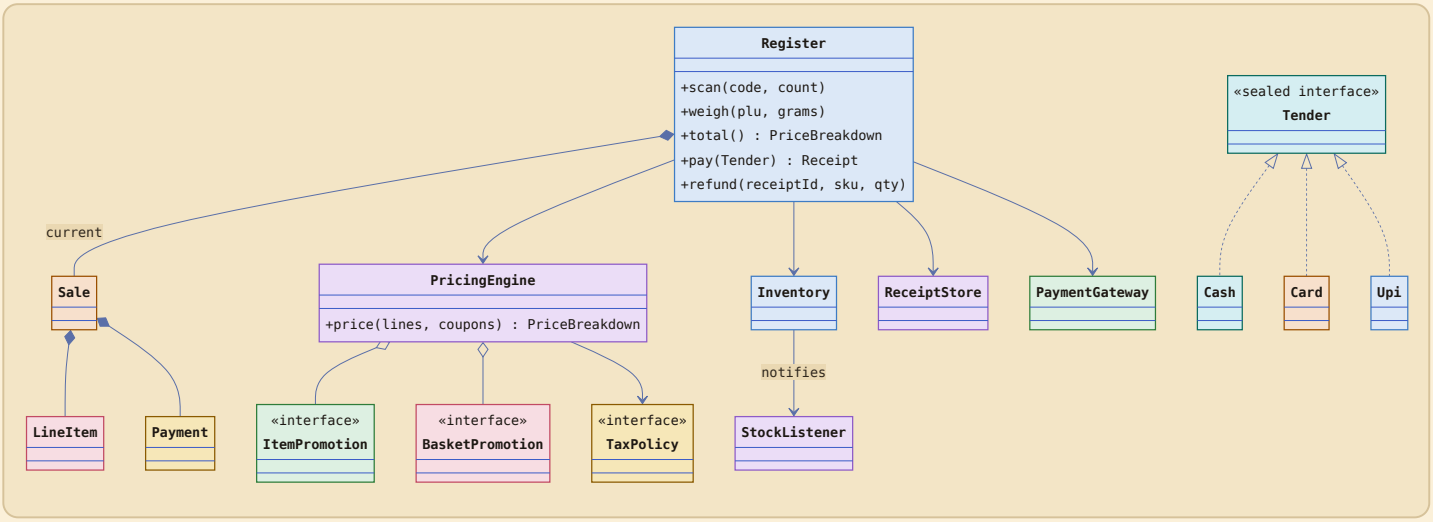
Use cases

Cashier / self-checkout (start, scan, weigh, void, coupon, total, pay, cancel), **Customer** (pay, return), **Supervisor** (approve voids/returns, reprice), **Payment network** (authorise, reverse), **Purchasing** (stock alerts).

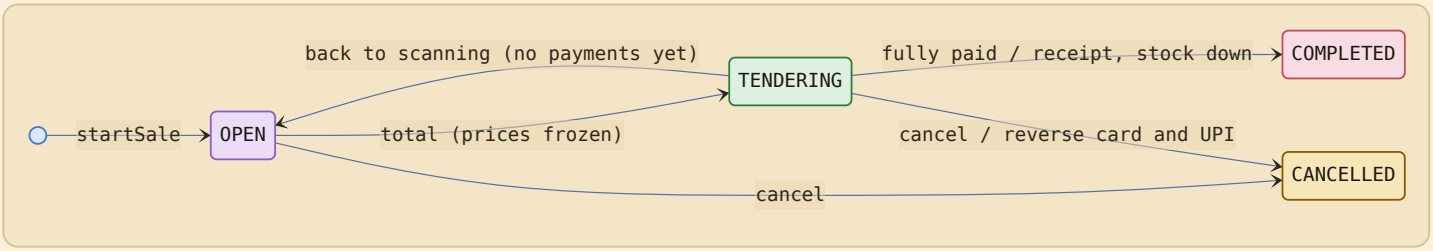
Core entities

Class	Responsibility
`Product`, `Catalog`	SKU, code, department, EACH/PER_KG, price, `TaxCategory`; lookup; reprice replaces the record.
`LineItem`, `Sale` + `SaleStatus`	One scan (price copied in); OPEN → TENDERING → COMPLETED / CANCELLED.
`ItemPromotion` (`BuyXGetYFree`, `MultiBuy`, `PercentOffDepartment`), `BasketPromotion` (`SpendThresholdCoupon`)	**Strategy** for offers.
`TaxPolicy` (`TaxInclusive`, `TaxExclusive`)	**Strategy** for tax.
`PricingEngine` → `PriceBreakdown`	Pure function: lines + coupons → discounts, net per SKU, tax, total.
`Tender` (sealed: `Cash`, `Card`, `Upi`), `Payment`, `PaymentGateway`	Tenders; applied amount; port to card/UPI networks.
`Receipt`, `ReceiptStore`	Immutable proof; atomic returns ledger.
`Inventory`, `StockListener`	Shared stock; **Observer** alerts.
`Register`	**Facade** for one lane.

Class diagram



Key flows



Pricing one basket (the hard method): skip voids → group by SKU → each SKU gets its **best** item offer → net per SKU → best basket coupon on the sum → **spread** the coupon over SKUs in proportion (remainder to the largest) → tax per category on discounted amounts → total (+ tax if added on top).

Split payment: `total()` freezes ₹588.23 → declined card throws, nothing recorded → UPI ₹400 → cash ₹500 → change ₹311.77 → receipt stored, stock reduced.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Offers behind `ItemPromotion` / `BasketPromotion`	**Strategy**	New offer = new class	`switch` over offer types
Best offer per SKU, coupon after	Explicit stacking policy	Written once, explainable at the till	Apply all in list order (order-dependent, can go negative)
Pure `PricingEngine`	Stateless service	Same basket, same price; shared by all registers	Mutating discounts into lines while scanning
Freeze at `total()`	Snapshot	Price/offer changes mid-payment can't change the amount due	Re-price per payment
Spread the coupon over SKUs	Exact allocation	Tax per category and per-item refunds need it	One unallocated number
`TaxPolicy`	**Strategy**	Inclusive vs exclusive per store	Two engines
Sealed `Tender`, `Register` facade	Sealed types, **Facade**	Compiler-checked receipt switch; UI sees one API	Strings; UI calling every part
Stock may go negative; edge-triggered alerts	**Observer**	Shelf is the truth; one alert per crossing	Refusing the sale; alert spam

Java implementation

Package `lld.ch19.grocery`.

```
// File: lld/ch19/grocery/Pricing.java
    PriceBreakdown price(List<LineItem> lines, Set<String> coupons) {
        // ... group non-voided lines by SKU into SkuTotal(product, quantity, gross)
        for (SkuTotal t : bySku.values()) {
            Optional<Discount> best = itemOffers.stream()
                    .map(p -> p.evaluate(t.product(), t.quantity()))
                    .flatMap(Optional::stream)
                    .max(Comparator.comparing(Discount::amount));
            BigDecimal off = best.map(d -> d.amount().min(t.gross())).orElse(Money.ZERO);
            best.ifPresent(d -> discounts.add(new Discount(d.description(), d.sku(), off)));
            net.put(t.product().sku(), t.gross().subtract(off));
        }
        BigDecimal subtotal = sum(net.values());
        Optional<Discount> basket = basketOffers.stream()
                .map(b -> b.evaluate(subtotal, coupons))
                .flatMap(Optional::stream)
                .max(Comparator.comparing(Discount::amount));
        basket.ifPresent(d -> { discounts.add(d); spread(d.amount(), net); });
        // ... tax per TaxCategory on the discounted nets via taxPolicy; total = nets (+ tax if added on top)
    }
```

```
// File: lld/ch19/grocery/Register.java
    /** delta < 0 for a sale, > 0 for a return. merge() is atomic per key, so registers never lose updates. */
    void adjust(String sku, BigDecimal delta) {
        BigDecimal after = onHand.merge(sku, delta, BigDecimal::add);
        BigDecimal before = after.subtract(delta);
        BigDecimal point = reorderPoint.getOrDefault(sku, BigDecimal.ZERO);
        // edge-triggered: alert only when THIS change crosses the line, so each crossing alerts exactly once
        if (before.compareTo(point) > 0 && after.compareTo(point) <= 0)
            listeners.forEach(l -> l.stockAlert(sku, after, "LOW - reorder"));
        if (before.signum() >= 0 && after.signum() < 0)
            listeners.forEach(l -> l.stockAlert(sku, after, "NEGATIVE - recount the shelf"));
    }
```

Full runnable code: code/lld/ch19/grocery/ (see the Full Edition).

How it works: `scan` checks the EAN-13 check digit before lookup; `weigh` turns grams into exact kg (`movePointLeft(3)`). Cash above the amount due becomes change; card/UPI must be authorised and cannot exceed

the amount due. Returns: `ReceiptStore.claimReturn` uses `compute` (atomic check-and-add), and the refund is `share(after) − share(before)` of the SKU's net paid, so partial returns add up exactly.

Output (excerpt):

```
  error: Bad barcode 8901000000018: check digit wrong, rescan
   10% off PRODUCE: Bananas                        -Rs 7.47
   Buy 2 get 1 free: Chips 50g                    -Rs 20.00
   3 for Rs 80.00: Biscuits                       -Rs 10.00
   Coupon SAVE50                                  -Rs 50.00
 TOTAL                                            Rs 588.23
   incl. tax STANDARD @18%                         Rs 40.07
 Change                                           Rs 311.77
  return 1 of 3 chips -> refund Rs 12.29
  return 2 more chips -> refund Rs 24.58  (all three add up to the Rs 36.87 paid)
  inclusive total Rs 588.23 vs exclusive total Rs 644.73 (tax {EXEMPT=0.00, REDUCED=9.22, STANDARD=47.28})
  paneer on hand: -21  (sold 120 + 1 earlier; the till never refuses a real item)
  alert: PNR LOW - reorder (on hand 20)
  alert: PNR NEGATIVE - recount the shelf (on hand -1)
```

Chips qualified for both "buy 2 get 1" (₹20) and "3 for ₹50" (₹10): only the better applied. Returns refund the net paid, not the ₹20 shelf price. Four concurrent registers took paneer below zero without refusing a sale; each alert fired once.

Concurrency & edge cases

Case	Handling
Registers sell the same SKU at once	`ConcurrentHashMap.merge` (atomic per key)
Count says 0, item in hand	Sell; stock negative; one recount alert
Two desks refund the same line	`compute` in `claimReturn`; second over-claim refused
Price/offer change mid-sale	Line keeps scan price; breakdown frozen at `total()`
Card declined / cancel after card	Nothing recorded / non-cash payments reversed
Rounding in coupon split and refunds	Remainder to the largest SKU; refunds as differences of cumulative shares
One register, one cashier	`Register` / `Sale` single-threaded; shared parts are the thread-safe ones

Real-world example

Tills key produce by **PLU** codes (4011 = yellow bananas) and read an integrated scale; EAN-13 barcodes end in a check digit, so a smudged code is rejected instead of misread; tax-inclusive receipts (GST in India, VAT in the UK) show the tax contained, US receipts add sales tax — our two `TaxPolicy` strategies; cycle counts fix negative stock.

Extensions & follow-up questions

- **Loyalty / member prices** → a `CustomerContext` for the engine; points on the net paid.
- **Pre-packed weighed labels** → variable-measure barcodes decoded into a line.
- **Bundles (pasta + sauce)** → multi-SKU promotion; best-per-SKU becomes a small optimiser.
- **Refund to original tender** → keep tender splits on the receipt.
- **Offline mode** → queue authorisations under a floor limit; cache catalog and offers per register.
- **End of day** → `Shift` with float and expected cash.

In the interview

45 minutes: clarify (5) → entities + facade (8) → sale state machine (5) → **pricing engine and stacking** (12) → split payment + receipt (7) → returns, stock, concurrency (8).

30-second answer: "A Register facade drives one Sale through OPEN, TENDERING, COMPLETED or CANCELLED; lines copy the price at scan; produce is priced by weight. A pure pricing engine gives each SKU its single best offer, applies a basket coupon, spreads it over SKUs so tax and refunds are exact, and computes tax through an inclusive or exclusive policy. Prices freeze at total; tenders are sealed;

cash gives change; card and UPI go through a gateway. On completion an immutable receipt is stored and shared stock drops atomically with edge-triggered alerts — the till never refuses a real item."

- *Why spread the coupon?* Tax differs per category and returns must refund what was paid for that item.
- *Why stateless pricing?* Shared by all registers; reproducible for audits and returns.

Self-check

1. **Q:** Two offers match the chips — which applies? **A:** The larger discount only.
2. **Q:** Why is a one-chip refund ₹12.29? **A:** Net paid after the offer and the coupon share.
3. **Q:** What stops a double refund? **A:** `compute` makes check-and-add atomic.
4. **Q:** Why allow negative stock? **A:** The item is physically there; the count is wrong.

TL;DR

- **Register facade** → **Sale** state machine; scan-time prices; produce by **weight**.
- **Pure pricing engine**: best offer per SKU → coupon → spread → tax via **TaxPolicy**.
- Freeze at total; **split tender**; returns refund the **net paid**, claimed atomically; stock atomic, may go negative, **edge-triggered** alerts.

Chapter summary

Problem	Hard part	The move that solves it
Amazon	Guests vs members; last-item races; cancel rules	Ordering only on `Member` (LSP); atomic all-or-nothing reservation; order state machine
Brokerage	Four order types; tax lots; over-spending	Two abstract questions on `Order`; `StockLot` + FIFO `LotSale`; reserved funds
Stock exchange	Fair matching	Per-symbol `TreeMap<price, FIFO queue>`; price then time; one lock per symbol
Auction	Concurrent bids	All bid rules in one per-listing critical section; notify after unlocking
Music streaming	Shuffle/repeat without losing order	Strategy-produced permutation of indexes; plan rules in the enum
Food delivery / FoodKart	Capacity and riders under concurrency	Atomic kitchen reservation; CAS-claimed riders; strategies for assignment and selection
Grocery POS	Weighed items, stacking offers, split tender, returns	Pure pricing engine (best offer per SKU, coupon spread, tax per category); frozen price; atomic return claims; edge-triggered stock alerts

Recurring patterns: Strategy (payments, play order, rider assignment, restaurant selection, trade price, grocery offers and tax), Observer (notifications, fills, plays), state machines in enums (orders, auctions, player), Template Method (statements), Facade (every `*Service`), and one concurrency rule — **make check-then-act atomic** (lock or compare-and-set).

Sources: DesignGurus *Grokking the Object-Oriented Design Interview* (Amazon, stock brokerage); awesome-low-level-design (auction, music streaming, food delivery); kumaransg/LLD machine-coding statements (Navi stock exchange, Flipkart FoodKart); grocery POS designed from first principles (GS1 EAN-13, IFPS PLU codes). All Java code is original.

Chapter 20 — Case Studies: Pattern-Driven Components (Study Edition)

> **Goal.** These problems are not whole products but **building blocks** most systems contain: a ledger that splits money, a notification sender, a cache, a rider–driver matcher, a logger, a message broker and a rate limiter. Each is small enough to finish in an interview and each is **anchored on one or two design patterns**. Name the pattern and the change it absorbs, and half the question is answered.

Why components get their own chapter. A component has a small public surface (a few methods) and many hidden rules. The interviewer keeps asking "what if this rule changes?"; a good design answers "one new class, nothing else touched".

#	Component	Anchor pattern(s)	The change the pattern absorbs
20.1	Splitwise (expense sharing)	**Strategy** (split rules)	A new way to split a bill
20.2	Notification service	**Template Method** (retry loop), **Strategy** (send rules), **Observer** (outcomes), Simple Factory (channels)	A new channel, a new reason not to send, a new watcher of results
20.3	In-memory cache (+ LRU from scratch)	**Strategy** (eviction), **Observer** (removal listener), Builder	A new eviction rule
20.4	Ride sharing (Uber)	**Strategy** (matching, pricing, payment), **Decorator** (surge), State machine (trip)	A new matching rule, surge on/off, a new payment method
20.5	Logging framework	**Chain of Responsibility** (logger hierarchy), **Singleton** (LogManager), Strategy (formatter), Decorator (async appender)	A new sink, a new format, async writing
20.6	Pub-Sub system	**Observer / Publish-Subscribe**, append-only log with offsets	New subscribers, replay, slow consumers
20.7	Rate limiter	**Strategy** (algorithm), Simple Factory, Composite	A new algorithm, a new tier, combined limits

Each case follows the Chapters 14–21 template (problem → requirements → entities → diagrams → flows → decisions → code → edge cases → interview).

Running the code. Java 21, standard library only, under `code/lld/ch20/<name>/`, each package with a `Main`. From `chapters/20-case-components/code`:

```
javac --release 21 -d /tmp/lld_20 $(find . -name '*.java')
java -cp /tmp/lld_20 lld.ch20.splitwise.Main      # also: notification, cache, lrucache,
                                                  # ridesharing, carpool, logging, pubsub, ratelimiter
```

Money is always `BigDecimal`. A `double` cannot hold 0.1 exactly (`0.1 + 0.2` prints `0.30000000000000004`). Two rules: compare with `compareTo` (not `equals`, which also compares scale, so `2.0` ≠ `2.00`), and always give a rounding mode when dividing (`divide(x, 2, RoundingMode.HALF_EVEN)`), or 1/3 throws `ArithmeticException`.

Reading the course diagrams. Figures marked "(course diagram)" use one arrow legend throughout:

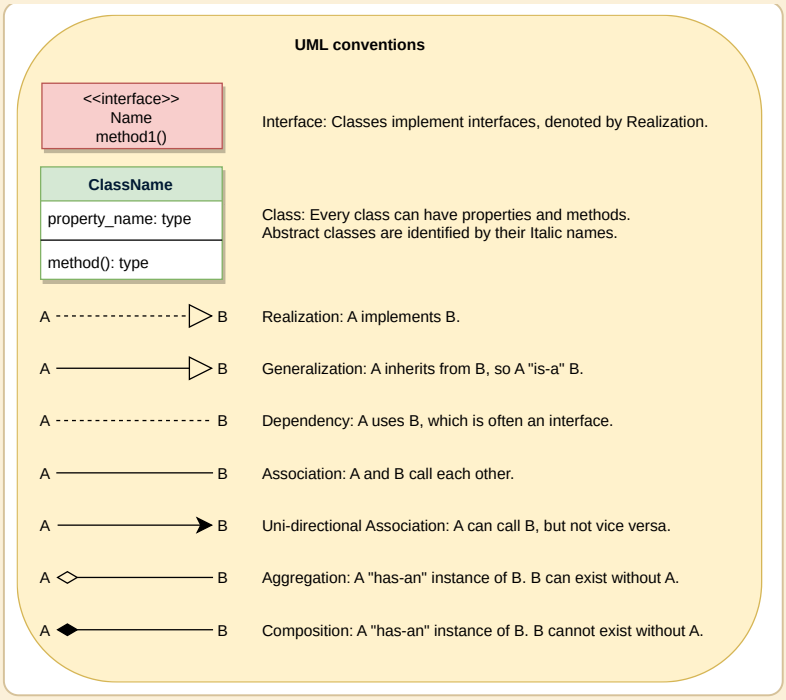


Figure — UML conventions used in the course class diagrams: interface, class, realization (implements), generalization (inherits), dependency, association, one-way association, aggregation and composition (course diagram)

The mermaid diagrams are drawn from **this chapter's Java code**.

20.1 Design Splitwise

Problem & clarifying questions

What it is. Splitwise remembers who paid for what when friends share costs and tells everyone who owes whom — the shared notebook for a weekend trip where Alice paid the hotel, Bob dinner, Carol the cab. The hard part is **not** the UI: it is deciding *which class owns the money arithmetic*, so the books balance after every operation.

Question	Assumed answer
Which split types?	Equal, exact, percentage. More later.
Must shares add up exactly?	Yes; otherwise the expense is rejected and no balance changes.
Rounding?	2-decimal money; no cent may be lost.
Pay only for yourself?	No: at least two people (payer counts) must be involved.
Settle up / simplify debts?	Yes / yes (on request, fewer transfers).
Currencies, receipts, undo?	Out of scope (say so).

Requirements

Functional: everything in the table, plus: users (name, email), groups, expense history per group, balance with one person and overall net. Invalid expense → **nothing changes**.

Non-functional: money correctness above all; a new split type changes no existing class (OCP); thread-safe.

Use cases

Actors: **Member** (adds expenses, settles up, asks what they owe), **Group creator**, **System** (computes shares, validates, updates the ledger, simplifies). *Add an expense* **includes** (always performs) *work out each share*, *check the shares add up*, *update the ledger*.

Core entities

Class	Responsibility
`User` (record)	Identity only: id, name, email.
`Group`	Name + member ids + expense ids. **Holds no balances.**
`SplitStrategy` + `EqualSplit`, `ExactSplit`, `PercentSplit`	Turn a total into one `Share` per person.
`Share` (record)	(person, amount) value object.
`Expense`	Description, total, payer, shares; **validates itself once** at creation.
`Ledger`	**One signed number per pair** of people; one mutator `adjust`.
`DebtSimplifier` / `Payment`	Greedy minimum-cash-flow → list of suggested transfers.
`Splitwise`	Facade; orders the steps, contains no arithmetic.
`Money`	2-decimal `BigDecimal` helpers.

Why the group holds no money: a person in two groups would have "Bob owes Alice" stored twice, and the copies would drift. One ledger = single source of truth. **`Group → User` is association, not composition:** deleting a group must not delete its people.

Class diagram

The course builds it in five stages, each leaving a working slice (copy this order in an interview): (1) `User`, `Group`; (2) `SplitStrategy` + rules + `Share`; (3) self-checking `Expense`; (4) `Ledger`; (5) `Splitwise` facade + `DebtSimplifier`.

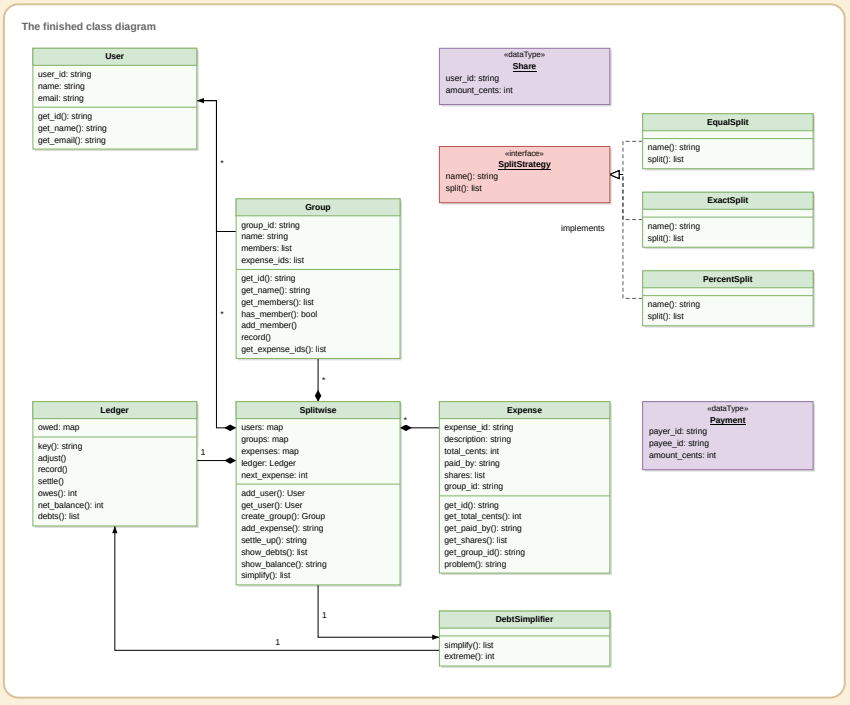
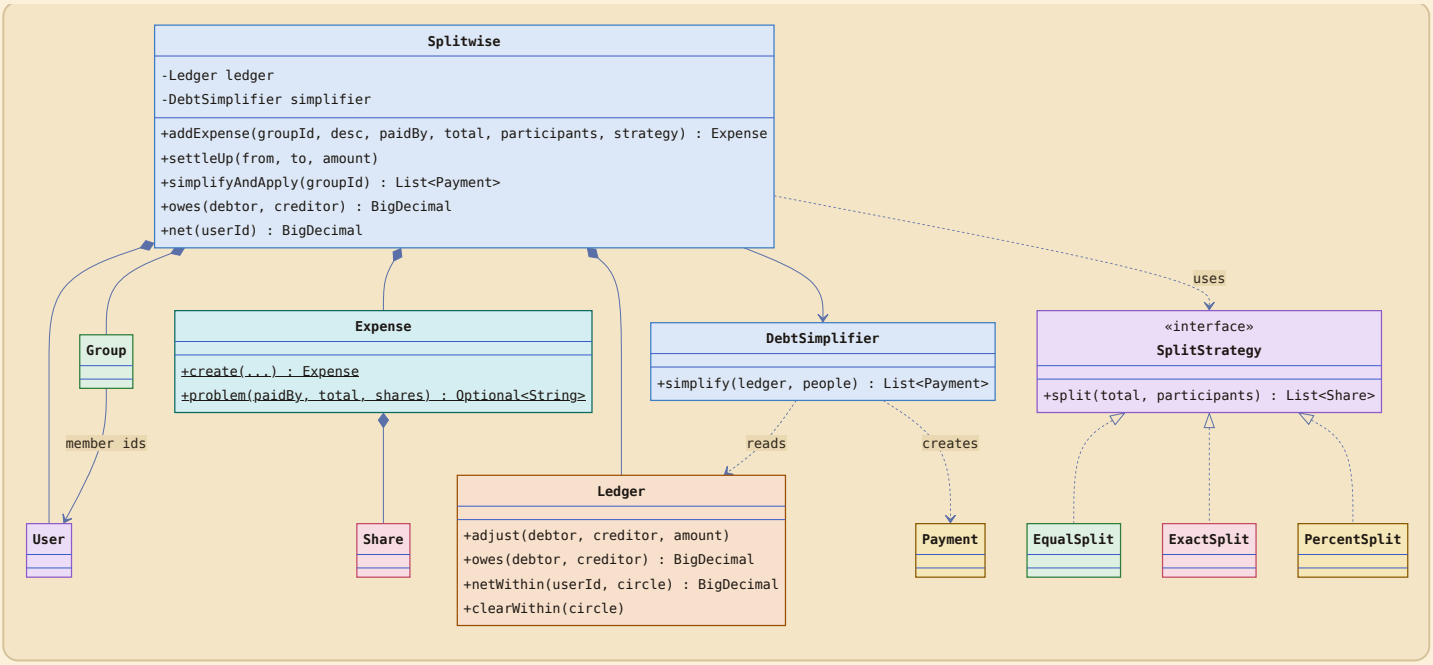


Figure — The finished Splitwise class diagram: service owning users, groups, expenses and the ledger, with the split strategies and the simplifier (course diagram)



Key flows

Adding an expense: strategy computes shares → `Expense.create` validates (≥ 2 people, sum = total) → only then, for each share, `ledger.adjust(sharer, payer, amount)`. If validation fails, an `IllegalArgumentException` with the reason is thrown and nothing is recorded.

Simplifying debts (greedy minimum cash flow):

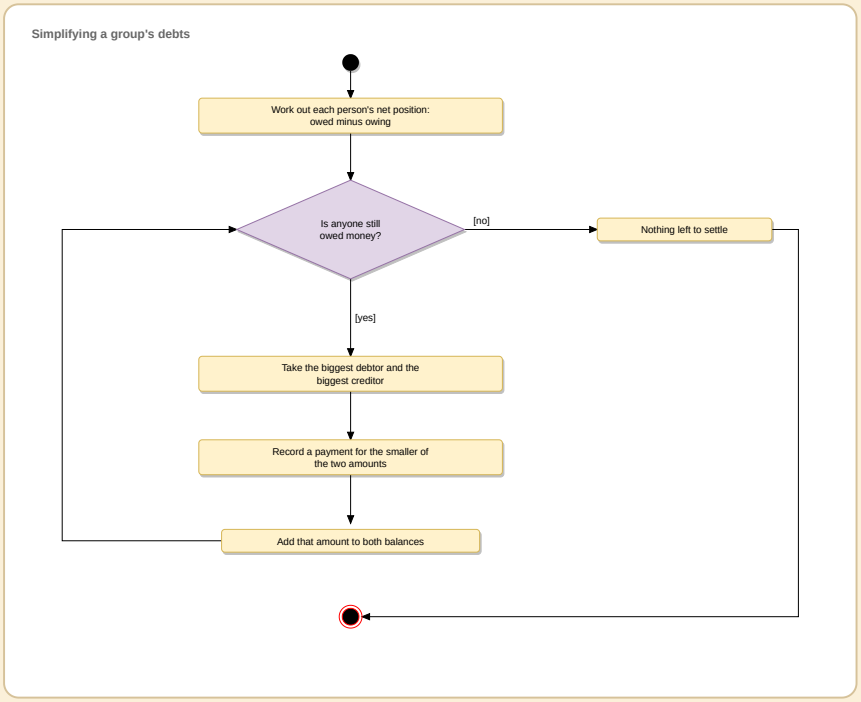


Figure — Simplifying debts: compute net positions, then the largest debtor repeatedly pays the largest creditor until every balance is zero (course diagram)

1. Net position per member = owed to them − they owe. Positive = creditor; nets sum to zero.
2. Creditors in one max-heap, debtors in another (ties by name → deterministic).
3. Pop biggest creditor C and debtor D; x = min(C, D); suggest "D pays C x".
4. Push back whichever side has a remainder; stop when a heap is empty.

Each round zeroes at least one person → **at most n − 1 payments**. It is **not always the true minimum** (that is NP-hard — it means finding the most zero-sum subgroups). Say so before you are asked.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for split rules	A new rule = a new class; all return `List<Share>`, so nothing downstream knows which ran.	`if (type == EQUAL) … else if …` in the service.
Expense validates in a **static factory**	Checked once; an invalid expense can never exist.	Checking in the ledger on every read.
`problem()` returns the **reason**	User sees *why* ("shares add up to 15.00 but the total is 30.00").	`boolean isValid()`.
One signed number per unordered pair	A debt can't disagree with itself; reverse debts cancel automatically.	Two maps "A owes B" / "B owes A".
One mutator `adjust`	Expenses, settle-ups, simplification share one cancel rule.	Separate `addDebt`, `settle`, `reduce`.
`DebtSimplifier` separate	SRP: keeping balances vs planning settlements.	Algorithm inside the ledger.
`BigDecimal`, spare cents handed out	100.00/3 → 33.34 + 33.33 + 33.33; percent split: last person absorbs remainder.	`double` (the course's whole-cents `long` is equally correct).
`Splitwise` is a **Facade**	One entry point that orders steps and decides nothing.	UI calling ledger/strategies directly.

Only one GoF pattern (Strategy) — fine: good designs use the pattern that absorbs the likely change, not a collection.

Java implementation

Key excerpts (the split strategy, the self-validating expense, the ledger's single mutator, and the facade's ordering):

```java
// File: code/lld/ch20/splitwise/SplitStrategy.java
interface SplitStrategy {
    List<Share> split(BigDecimal total, List<String> participants);
}

/** Equal split. Spare cents are handed out one at a time, in participant order. */
final class EqualSplit implements SplitStrategy {
    @Override
    public List<Share> split(BigDecimal total, List<String> participants) {
        if (participants.isEmpty()) throw new IllegalArgumentException("nobody to split between");
        BigDecimal n = BigDecimal.valueOf(participants.size());
        BigDecimal base = total.divide(n, 2, RoundingMode.DOWN);           // 100.00/3 -> 33.33
        BigDecimal leftover = total.subtract(base.multiply(n));           // 0.01
        int spareCents = leftover.divide(Money.CENT).intValueExact();     // 1
        List<Share> shares = new ArrayList<>();
        for (int i = 0; i < participants.size(); i++) {
            BigDecimal amount = i < spareCents ? base.add(Money.CENT) : base;
            shares.add(new Share(participants.get(i), amount));
        }
        return shares;
    }
}
// ... ExactSplit, PercentSplit (last person absorbs the remainder)

// File: code/lld/ch20/splitwise/Expense.java
    static Optional<String> problem(String paidBy, BigDecimal total, List<Share> shares) {
        if (total.signum() <= 0) return Optional.of("total must be positive");
        // ... negative / duplicate share checks
        Set<String> people = new HashSet<>(sharers);
        people.add(paidBy); // the payer counts as one of the two people even if they owe no share
        if (people.size() < 2) return Optional.of("an expense needs at least two people");
        BigDecimal sum = shares.stream().map(Share::amount).reduce(BigDecimal.ZERO, BigDecimal::add);
        if (sum.compareTo(total) != 0)
            return Optional.of("shares add up to " + sum + " but the total is " + total);
        return Optional.empty();
    }

// File: code/lld/ch20/splitwise/Ledger.java  (key (lo, hi), lo < hi; positive = "hi owes lo")
    /** The ONLY mutator: 'debtor' now owes 'creditor' 'amount' more. Opposite debts cancel. */
    void adjust(String debtor, String creditor, BigDecimal amount) {
        if (debtor.equals(creditor) || amount.signum() == 0) return;
        Pair key = Pair.of(debtor, creditor);
        BigDecimal delta = creditor.equals(key.lo()) ? amount : amount.negate();
        BigDecimal updated = balances.getOrDefault(key, Money.ZERO).add(delta);
        if (updated.signum() == 0) balances.remove(key); else balances.put(key, updated);
    }

// File: code/lld/ch20/splitwise/Splitwise.java
        List<Share> shares = strategy.split(total, participants);       // may throw
        Expense e = Expense.create("e" + ids.incrementAndGet(), description, paidBy, total, shares, groupId);
        for (Share s : e.shares()) ledger.adjust(s.userId(), paidBy, s.amount()); // payer's own share is a no-op
    // ... settleUp: refuse paying more than is owed, then ledger.adjust(to, from, amount) - the reverse debt cancels
```

Full runnable code: code/lld/ch20/splitwise/ (see the Full Edition).

Notes: `Money.of("10.005")` throws (`RoundingMode.UNNECESSARY` refuses silent rounding); `PercentSplit` rounds with `HALF_EVEN` (banker's rounding); `DebtSimplifier` uses two `PriorityQueue`s.

Output (excerpt):

```
Hotel shares: [alice=33.34, bob=33.33, carol=33.33]
  refused: shares add up to 15.00 but the total is 30.00
  suggested: dave pays alice 16.68
  suggested: carol pays alice 13.33
  suggested: bob pays alice 9.99
```

Four pairwise debts became three payments; every net (alice +40.00, bob −9.99, carol −13.33, dave −16.68) is unchanged.

Related machine-coding problem: The Ledger Co. (Navi)

The Geektrust/Navi round "Ledger Co." also keeps money records, but for **bank loans** (simple interest, monthly EMI, lump-sum prepayments). It is built in **Chapter 21 (The Ledger Co.)**. Both keep **one authoritative record** and compute every answer from it, and both use `BigDecimal` with a named rounding mode; the rules differ (*who owes whom* vs *how much is repaid by month N*).

Concurrency & edge cases

- **Concurrent expenses:** every public `Splitwise` method is `synchronized` (build → validate → adjust is atomic); at scale lock per group/pair or use a DB transaction.
- **Payer not a sharer** (Alice buys Bob's lunch) is valid; the payer's own share is a no-op `adjust`.
- **Overpaying on settle-up**, unknown users, duplicate/negative shares, zero totals: refused before anything changes.
- **Simplify touches other groups:** one number per *pair* means simplifying a group rewrites all debts between its members, even ones from another group. If that is unacceptable, keep per-group ledgers plus a derived global view.

Extensions & follow-up questions

- **Split by shares (2:1:1)?** New `ShareRatioSplit`; nothing else changes.
- **Delete/edit an expense?** Re-add the inverse, or better make the ledger an **append-only journal** (event sourcing): delete = append a reversing entry.
- **Is greedy optimal?** No. Nets A −6, B −5, C +2, D +4, E +5: greedy needs 4 payments; optimal 3 (B→E 5, A→D 4, A→C 2).
- **Is `Splitwise` doing too much?** At scale, yes: give users, groups, expenses and ledger their own repositories. Multiple currencies: `Money(amount, currency)` + one ledger per currency.

Real-world example

Splitwise really offers "simplify debts" for groups (the net-position idea). Bill-splitting in payment apps solves rounding the same way: one person absorbs the leftover cent.

In the interview

> **30-second answer:** "Users and groups hold no money. Each split rule is a `SplitStrategy` returning shares. An `Expense` validates itself once: at least two people, shares add to the total. The `Ledger` keeps one signed number per pair and has a single `adjust`, so debts cancel automatically; settling up is an adjust the other way. Simplification: net positions + greedy biggest-debtor-pays-biggest-creditor, at most n − 1 payments, not always optimal."

Tip: say "one class per split rule" early; spend your time on the ledger.

Self-check

1. Why does `Group` hold no balances? → Each debt must live in one place; two copies drift.
2. Split 10.00 equally among 3. → 3.34, 3.33, 3.33.
3. Max payments from the greedy simplifier for n people? → n − 1.
4. Why `compareTo` for `BigDecimal`? → `equals` compares scale: `2.0` ≠ `2.00`.
5. Alice pays 20.00, Bob owes all of it — valid? → Yes, two people are involved.

TL;DR

- **Strategy** for split rules; **Expense validates itself** once.
- **Ledger = one signed number per pair**, one mutator; settle-up = reverse adjust.
- **Simplify** = net positions + greedy; ≤ n − 1 payments; optimal is NP-hard.
- **BigDecimal**, explicit rounding, no lost cents.

20.2 Design a Notification Service

Problem & clarifying questions

What it is. The component that sends emails, SMS and push alerts ("Your order has shipped", "482913 is your login code"). Analogy: a post-office counter that picks the delivery method, respects "do not disturb", retries if the courier fails, and keeps a receipt. **Why it is a design question:** it starts as one `sendEmail()` and grows fourteen `if`s (SMS, quiet hours, promo limits…). The exercise is *where each addition should have gone instead*.

Question	Assumed answer
Channels?	Email, SMS, push; more later.
Who picks the channel?	The user, **per kind** of notification, in preference order.
Wording?	Templates with placeholders: `Hi {name}, it arrives {eta}.`
Quiet hours?	Per user, may wrap midnight (22–07); urgent (OTP) ignores them.
Rate limits?	≤ N per user per kind per hour.
Failures?	Retry a channel a bounded number of times, then fall back to the next channel; all fail → FAILED.
Audit?	Every attempt recorded: SENT / FAILED / SUPPRESSED + reason.
Sync or async?	Caller must not wait for a slow gateway.

Requirements

Functional: the table above, plus: user has email, phone, device token; FAILED when no channel succeeds; **a new channel or a new reason to hold back needs no edit to the service.**

Non-functional: caller not blocked (async); thread-safe; testable without real time (injected clock); a broken listener never breaks delivery.

Use cases

Actors: **Product code** (asks), **Person** (channels, quiet hours), **Operator** (adds channels, watches outcomes), **Service** (renders, checks rules, tries channels, retries, records).

Core entities

Class	Responsibility
`User` / `Preferences`	Who someone is vs what they accept (ordered channels per kind, quiet hours with wrap logic in **one** place). Separate because they change for different reasons (SRP).
`Template` / `Message`	Subject + body with `{holes}`; `render(values)` → identical `Message` for every channel.
`NotificationRequest`	kind, user, values, priority (NORMAL / URGENT).
`Gateway`	External provider (SMTP, SMS gateway, push service); swappable and fakeable.
`NotificationChannel` (abstract) + Email/Sms/Push	**Template Method**: final `deliver()` has the retry-with-backoff loop; subclasses give `addressOf(user)`.
`ChannelFactory`	**Simple Factory**: type → channel.
`RetryPolicy`	Max attempts + initial backoff (doubles).
`SendRule` + `QuietHoursRule`, `RateLimitRule`	**Strategy**: one reason not to send, returned as text.
`DeliveryRecord` / `DeliveryListener`	Outcome record; **Observer** (`LoggingListener`, `MetricsListener`).
`TimeSource`	Where "now" comes from (manual clock in tests).
`NotificationService`	Thin coordinator: rules → preferences → render → channels in order → `finish()`.

Class diagram

Course build stages: (1) `User` + `Preferences`; (2) `Template` + `Message` + request; (3) channel hierarchy; (4) `SendRule`s; (5) service + listeners.

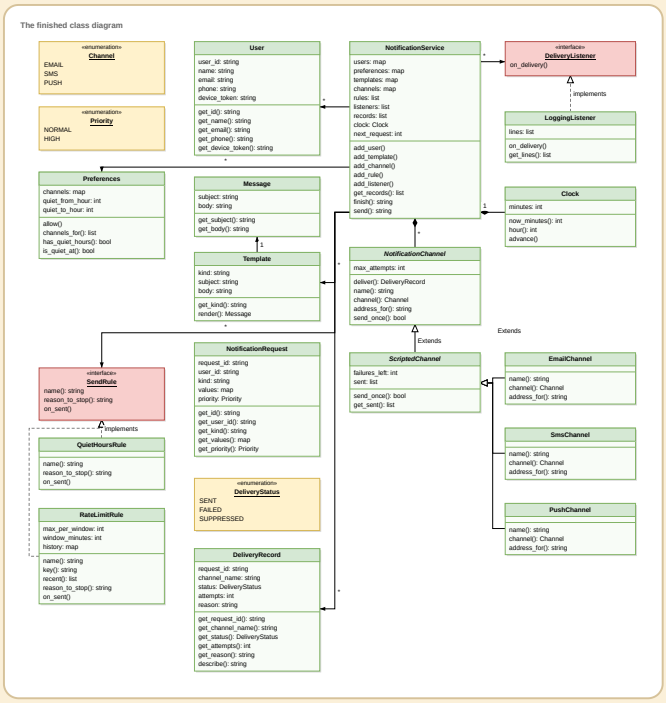
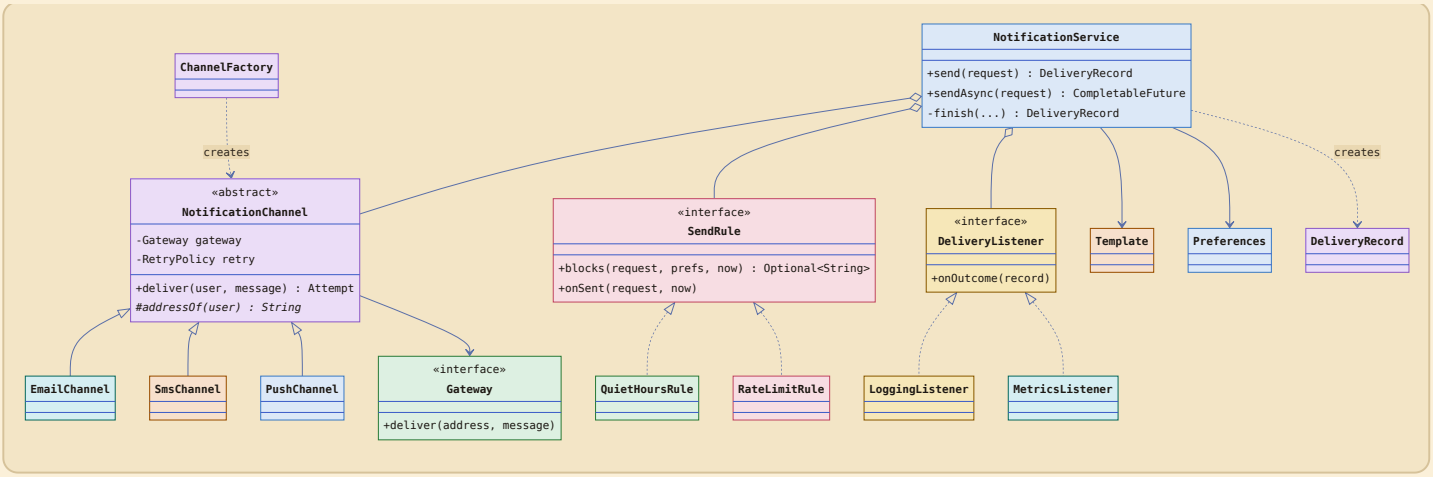


Figure — The finished notification service class diagram: the service in the middle, the channel hierarchy with the retry in the parent, the send rules, templates, preferences and the delivery listeners (course diagram)



Key flows

Sending one notification: ask every rule (any reason → SUPPRESSED); no accepted channel → SUPPRESSED; render the template **once**; try each accepted channel in the user's order (each with its own retry loop); first success → `onSent` to rules, SENT; all fail → FAILED. Every path exits through `finish()`.

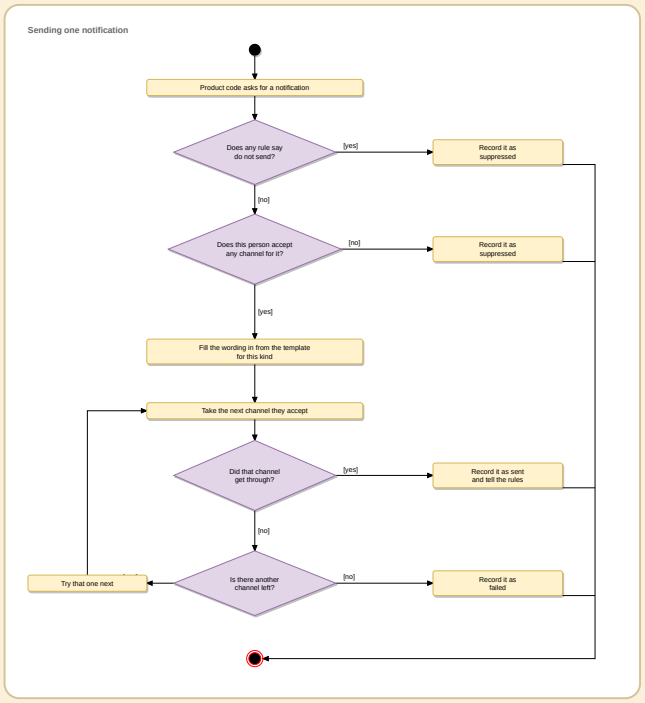


Figure — Sending one notification: ask every rule, render once, try each accepted channel in turn, and record sent / failed / suppressed (course diagram)

Design decisions & patterns used

Pattern / decision	Where	Why
Template Method	`final deliver()`; subclasses fill `addressOf()`	Retry loop is identical for all channels; only a *step* varies. Strategy would replace the whole method.
Strategy	`SendRule`	Each "reason not to send" is a class; a new rule (unsubscribed, dedupe) = new class + one wiring line.
Observer	`DeliveryListener`	Logs, dashboards, alerts want outcomes; the service doesn't know who listens.
Simple Factory	`ChannelFactory`	One place maps type → class.
Adapter (target)	`Gateway`	One adapter per vendor SDK; tests use a fake.
Rule returns **reason text**	`SendRule.blocks`	Reason is recorded.
Urgency handled **inside** the rule	`QuietHoursRule`	Service needn't know which rules urgent ignores.
Rate limit counts only **successful** sends	`onSent` hook	Suppressed/failed messages don't use up the allowance.
Single exit `finish()`	Service	Every path stores a record and notifies listeners.
Render **once**	Service	Same text everywhere; wording change = one edit.

Three patterns, three *separate* axes of change: another channel (Template Method + Factory), another reason to hold back (Strategy), another interested party (Observer).

Java implementation

```java
// File: code/lld/ch20/notification/Channels.java
abstract class NotificationChannel {
    private final Gateway gateway;
    private final RetryPolicy retry;
    // ... constructor
    abstract ChannelType type();

    /** Hook: where this channel reaches the user, or null if the user has no such address. */
    protected abstract String addressOf(User user);

    final Attempt deliver(User user, Message message) {
        String address = addressOf(user);
        if (address == null || address.isBlank()) return new Attempt(false, 0, "no " + type() + " address");
        String lastError = null;
        long backoff = retry.initialBackoff().toMillis();
        for (int attempt = 1; attempt <= retry.maxAttempts(); attempt++) {
            try {
                gateway.deliver(address, message);
                return new Attempt(true, attempt, null);
            } catch (Exception e) {
                lastError = e.getMessage();
                if (attempt < retry.maxAttempts()) sleep(backoff);
                backoff *= 2;
            }
        }
        return new Attempt(false, retry.maxAttempts(), lastError);
    }
    // ...
}

final class SmsChannel extends NotificationChannel {
    SmsChannel(Gateway g, RetryPolicy r) { super(g, r); }
    ChannelType type() { return ChannelType.SMS; }
    protected String addressOf(User u) { return u.phone(); }
}

// File: code/lld/ch20/notification/NotificationService.java  (inside send)
        for (SendRule rule : rules) {
            Optional<String> reason = rule.blocks(req, prefs, now);
            if (reason.isPresent()) return finish(req, DeliveryStatus.SUPPRESSED, null, 0, reason.get());
        }
        // ... no accepted channel -> SUPPRESSED; render the template ONCE (missing value -> FAILED via finish)
        for (ChannelType type : accepted) {
            NotificationChannel channel = channels.get(type);
            if (channel == null) continue;
            Attempt a = channel.deliver(user, message);
            totalAttempts += a.attempts();
            if (a.success()) {
                rules.forEach(r -> r.onSent(req, now));
                return finish(req, DeliveryStatus.SENT, type, totalAttempts, null);
            }
            lastError = type + ": " + a.error();
        }
        return finish(req, DeliveryStatus.FAILED, null, totalAttempts, "all channels failed, last " + lastError);
```

Full runnable code: code/lld/ch20/notification/ (see the Full Edition).

Notes: `Preferences.isQuiet` handles wrap-around (start ≤ end → range; else "after start **or** before end"). A missing template value is an error, not a raw `{name}` in an inbox. Backoff doubles (the course retried immediately — a named weakness). `RateLimitRule` prunes old timestamps (the course's version leaked them). A throwing listener is caught in `finish()`. `sendAsync` uses a pool; production uses a durable queue.

Output (excerpt):

```
== 2. SMS fails 3 times -> retried, then falls back to EMAIL
    SMTP -> asha@mail.com : Order A-17 shipped | Hi Asha, it arrives Tuesday.
  [log] SENT       ORDER_SHIPPED user=u1 channel=EMAIL attempts=4
  [log] SUPPRESSED PROMO         user=u1 channel=- attempts=0 reason=quiet hours
  [log] SUPPRESSED PROMO         user=u1 channel=- attempts=0 reason=rate limit 2 per 60 min
```

Concurrency & edge cases

- Shared state: `ConcurrentHashMap` for users/prefs/templates, `CopyOnWriteArrayList` for listeners/history, rate-limit deque locked per key.
- **Check-then-act gap:** two concurrent sends can both pass `blocks()` before `onSent()`. Fix: reserve a slot atomically at check time (release on failure) or serialize per user.
- **Retries block worker threads** with `sleep`; real systems re-enqueue with a delay (retry topic).
- **Idempotency:** a retry after a timeout may deliver twice → pass an idempotency key; receivers tolerate duplicates.
- Missing address → 0-attempt failure, next channel. No template → FAILED; no accepted channel → SUPPRESSED.
- **Time zones:** quiet hours in the *user's* zone (`ZoneId` in preferences).

Extensions & follow-up questions

- **Add WhatsApp:** one subclass, one enum value, one `Gateway` adapter.
- **Too many messages:** a new `SendRule` (daily cap, dedupe), not a new `if`.
- **Campaign to a million users:** fan-out stage onto a queue + many workers calling `send`; rate-limit per provider.
- Priorities (separate queues), localization (templates by kind + locale), webhooks for receipts, a **circuit breaker** per provider.

Real-world example

Push goes through **FCM** (Android) and **APNs** (iOS) to a *device token*; email via SES/SendGrid, SMS via Twilio — each its own API, hence `Gateway`. **AWS SNS** fans one message out to many channels. LinkedIn's **Air Traffic Controller** decides whether/when/how to notify each member — `SendRule` at company scale. Phone "Do Not Disturb" = quiet hours with an urgent bypass.

In the interview

> **30-second answer:** "Channels sit behind an abstract class whose final `deliver` holds retry-with-backoff (Template Method); each channel supplies only its address. Each do-not-send condition is a `SendRule` (Strategy) returning a reason. Preferences give an ordered channel list per kind; we render once and try channels in order, falling back on failure. Every outcome goes through one `finish` that records and notifies listeners (Observer). For scale, a queue sits in front."

Follow-ups: *Why not Strategy for channels?* → Channels share an algorithm and vary in one step: Template Method (injecting `RetryPolicy` + a `Sender` strategy is a valid composition alternative). *Duplicates on retry?* → Idempotency keys; at-least-once delivery.

Self-check

1. Which pattern puts the retry loop in one place? → Template Method (`final deliver()`).
2. Why `Optional<String>` from `blocks`? → The suppression reason is recorded.
3. Why not count suppressed sends against the rate limit? → The user never got them.
4. Quiet hours 22–07: is 06:00 quiet? 12:00? → Yes; no.
5. What must every exit of `send` do? → Go through `finish()`.

TL;DR

- **Template Method** channels (retry once in the parent); **Strategy** send rules returning reasons; **Observer** outcomes; **Factory** channels; **Adapter** providers.
- Render once, try channels in order, fall back, single `finish()` exit.
- Scale with a queue; watch idempotency and the rate-limit check-then-act gap.

20.3 Design an In-Memory Cache (and an LRU Cache from scratch)

Problem & clarifying questions

What it is. A small, fast store of recently used values, so you don't re-fetch them from a slow place (DB, another service). Analogy: the few books on your desk vs the library. When the desk is full, one book must go back — **which one** is the eviction policy, the rule most likely to change. Expect the question: *which data structure gives O(1) get and put?* (O(1) = time doesn't grow with the number of entries.)

Question	Assumed answer
Capacity in?	Number of entries.
Eviction?	LRU by default; swappable for FIFO or LFU without changing the cache.
TTL?	Optional per entry; an expired read is a miss and removes it.
Stats / notifications?	Hits, misses, evictions, expirations; subscribers learn about removals and the cause.
Thread-safe?	Yes (the course left it out; we add it).
Load on miss / write-back?	Out of scope (extensions).

Requirements

Functional: the table above, plus: `put` overwrites; a full cache evicts before inserting; O(1) get/put; invalidate one key or clear all; **every entry leaves through one code path**.

Non-functional: O(1); thread-safe; testable TTL (injected clock); listeners can't slow down or deadlock the cache.

Use cases

Actors: **Application** (read, store, remove), **Operator** (chooses rule, reads stats), **Cache** (evicts, expires, counts, notifies).

Core entities

Class	Responsibility
`Node<K,V>`	key, value, `expiresAtMillis`, `frequency`, own `prev`/`next`.
`DoublyLinkedList<K,V>`	Order, newest at front, two **sentinel** nodes (no null checks).
`EvictionPolicy<K,V>`	**Strategy**: `onInsert`, `onAccess`, `onRemove`, `victim()`. Owns the *order*.
`LruPolicy`, `FifoPolicy`, `LfuPolicy`	The three rules; LFU is O(1) with chained frequency buckets.
`EvictionPolicies` + `PolicyType`	**Simple Factory**.
`Cache<K,V>` + `Cache.Builder`	Map + policy + clock + stats + listeners; one lock; **one removal method**. **Builder** for config.
`CacheStats`, `RemovalReason`, `EvictionListener`	`LongAdder` counters; EVICTED/EXPIRED/REMOVED/CLEARED; **Observer**.
`ShardedCache<K,V>`	N caches chosen by key hash → less lock contention.

The core idea: two structures, each answering one question.

Question	Structure	Cost
"Where is key k?"	`HashMap<K, Node>`	O(1)
"Who leaves next?"	Doubly linked list by recency	O(1) at the ends
"Move this entry to the front"	Node has own links; the map hands it over	O(1), no search

A map can't name the oldest entry; a list can't find a key without walking. **Doubly** linked because unlinking a middle node needs its predecessor.

Class diagram

Course build order: (1) `Node` + list; (2) `EvictionPolicy` + LRU; (3) FIFO + LFU with no cache change; (4) `Cache` with clock, stats, TTL; (5) listeners. **Who owns what:** the cache owns the map (and nodes); the policy owns the *order*. Neither reaches into the other — that is why FIFO/LFU need no `Cache` change.

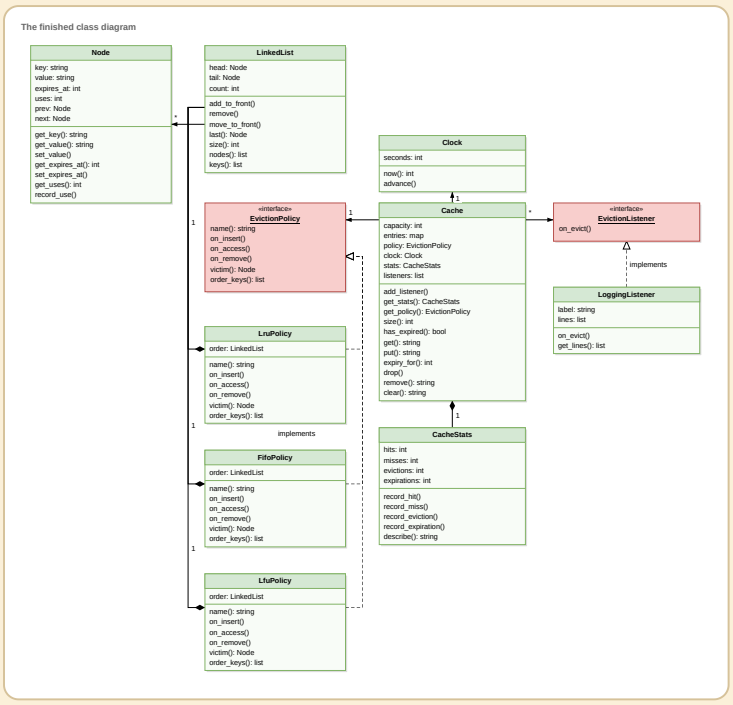
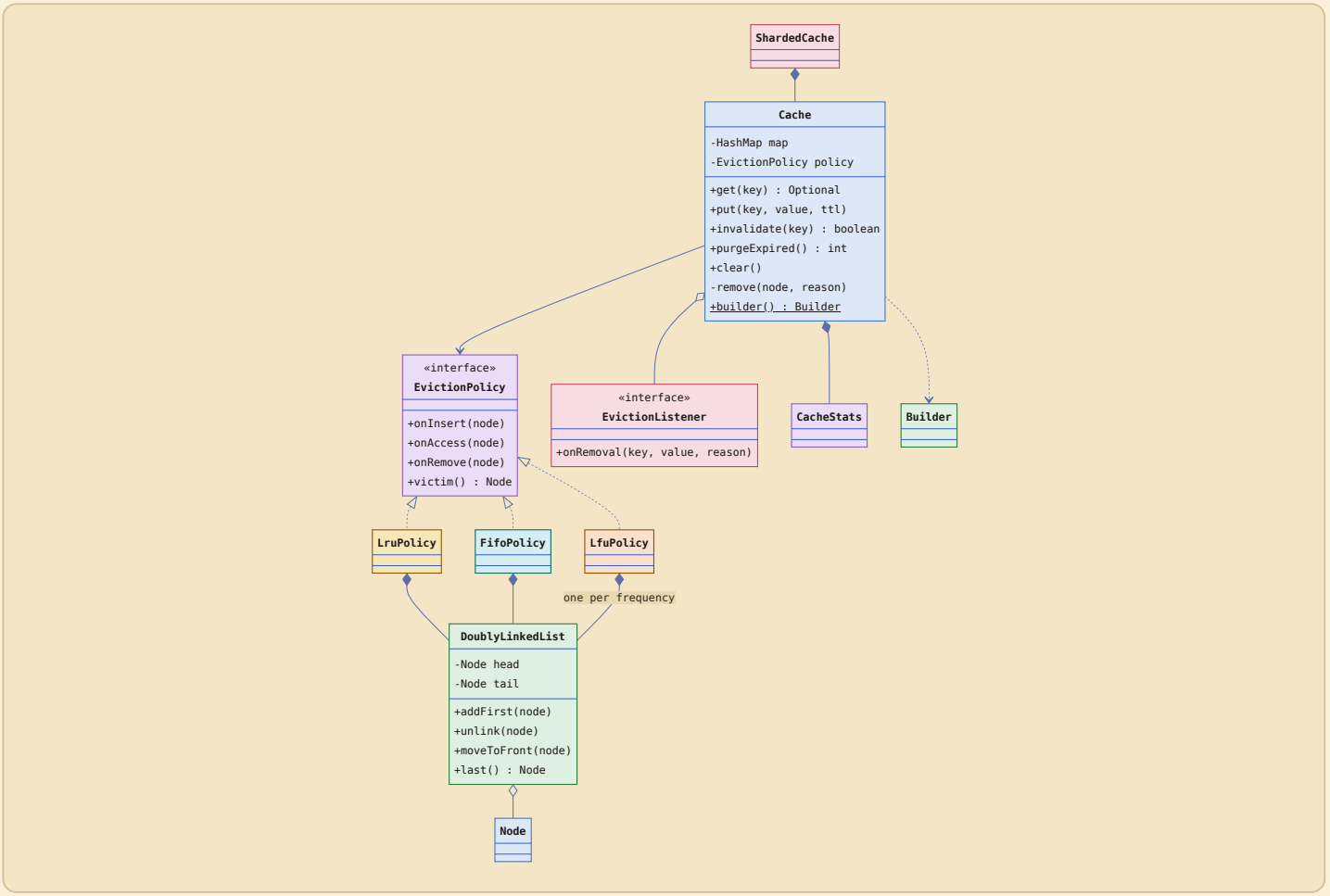


Figure — The finished cache class diagram: Cache with its map, policy, clock, stats and listeners; the three eviction policies behind one interface (course diagram)



Key flows

Read: absent → miss; present but expired → miss + removal (EXPIRED); else hit, `policy.onAccess`, return.
Store: replace if present (counts as a use); otherwise, only if full, ask `policy.victim()` and remove it through the single removal path, then insert. Listeners are called after the lock is released.

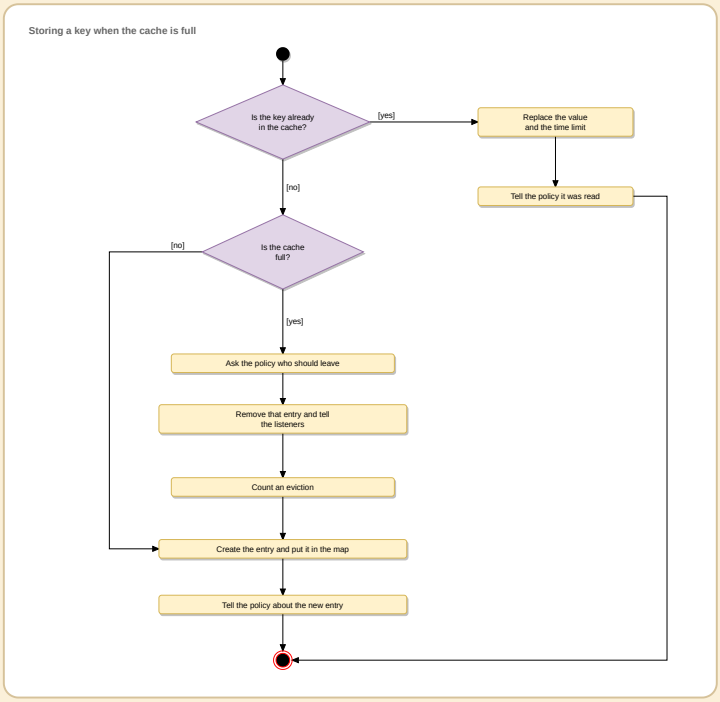


Figure — Storing a key: replace if present; otherwise, if full, ask the policy for a victim and remove it through the single removal path, then insert (course diagram)

How the three policies differ:

Event	LRU	FIFO	LFU (O(1) buckets)
`onInsert`	add to front	add to front	frequency = 1, bucket 1 (always first in chain)
`onAccess`	move to front	**nothing**	move bucket f → f+1 (created right after f); drop f if empty
`onRemove`	unlink	unlink	unlink; drop bucket if empty
`victim()`	back of list	back of list	back of the **first** bucket (LRU among least frequent)

FIFO is LRU with one empty method — evidence of good interface boundaries. The course's LFU scanned all entries (O(n)); ours keeps buckets in a **doubly linked chain in increasing frequency**, so the lowest bucket is always first and every operation is O(1) (Shah, Mitra & Matani, 2010).

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for eviction	The rule most likely to change.	`switch (policyType)` in get/put.
Observer for removals, with reason	Logging, metrics, cleanup don't belong in the cache.	Hard-coded callbacks.
One removal method `remove(node, reason)`	Eviction, expiry, removal, clear all update map, policy, stats alike.	Four copies of clean-up code.
Sentinel nodes	Add/unlink are always the same pointer writes.	Null checks everywhere.
Injected clock (`LongSupplier`)	TTL tests run instantly.	`System.currentTimeMillis()` → tests must sleep.
Lazy expiry + optional sweeper	Dropped on read; `purgeExpired()` reclaims memory.	Timer per entry.
One `ReentrantLock`, listeners **after unlock**	LRU `get` writes (moves a node), so a read-write lock doesn't help; slow/re-entrant listeners can't block or deadlock.	`synchronized` around listener calls.
Builder	Defaults + validation once.	Telescoping constructors.
Sharding	N locks; threads on different shards never wait.	One global lock.

Java implementation

```java
// File: code/lld/ch20/cache/EvictionPolicy.java
interface EvictionPolicy<K, V> {
    void onInsert(Node<K, V> node);
    void onAccess(Node<K, V> node);
    void onRemove(Node<K, V> node);
    Node<K, V> victim();
}

final class LruPolicy<K, V> implements EvictionPolicy<K, V> {
    private final DoublyLinkedList<K, V> order = new DoublyLinkedList<>();
    public void onInsert(Node<K, V> n) { order.addFirst(n); }
    public void onAccess(Node<K, V> n) { order.moveToFront(n); }
    public void onRemove(Node<K, V> n) { order.unlink(n); }
    public Node<K, V> victim() { return order.last(); }
}
// ... FifoPolicy (onAccess does nothing), LfuPolicy (chained frequency buckets)

// File: code/lld/ch20/cache/Cache.java
    void put(K key, V value, Duration ttl) {
        List<Removal<K, V>> removed = new ArrayList<>(1);
        lock.lock();
        try {
            long expiresAt = ttl == null ? 0 : clockMillis.getAsLong() + ttl.toMillis();
            Node<K, V> existing = map.get(key);
            if (existing != null) {          // replace: counts as a use
                existing.value = value;
                existing.expiresAtMillis = expiresAt;
                policy.onAccess(existing);
                return;
            }
            if (map.size() >= capacity) removed.add(remove(policy.victim(), RemovalReason.EVICTED));
            Node<K, V> n = new Node<>(key, value, expiresAt);
            map.put(key, n);
            policy.onInsert(n);
        } finally {
            lock.unlock();
            notifyListeners(removed);
        }
    }
    // ... get, invalidate, purgeExpired, clear follow the same lock / collect / notify-after-unlock shape

    /** THE single exit path for every entry, whatever the reason. Caller holds the lock. */
    private Removal<K, V> remove(Node<K, V> n, RemovalReason reason) {
        map.remove(n.key);
        policy.onRemove(n);
        if (reason == RemovalReason.EVICTED) stats.evictions.increment();
        if (reason == RemovalReason.EXPIRED) stats.expirations.increment();
        return new Removal<>(n.key, n.value, reason);
    }
```

Full runnable code: code/lld/ch20/cache/ (see the Full Edition).

Output (excerpt) — same accesses (b read 3×, c once, a twice), three different victims:

```
== LRU (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: b=2 left, EVICTED
== FIFO (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: a=1 left, EVICTED
== LFU (capacity 3): put a,b,c; get b,b,b,c,a,a; put d
  listener: c=3 left, EVICTED
  stats: hits=2 misses=1 evictions=0 expirations=2 hitRate=67%
```

LRU Cache from scratch (the classic interview question)

Often asked directly: **"Implement an LRU cache with O(1) get and put"** (LeetCode 146). Same two structures, inline: HashMap for lookup; doubly linked list with `head` / `tail` sentinels (most recent after `head`). `get`: unlink + add at front. `put`: if present update + move to front; else if full remove `tail.prev` from list **and** map (why the node stores its key), then add. Java shortcut: `LinkedHashMap(accessOrder = true)` + `removeEldestEntry` — mention it, but write the manual version when asked "from scratch".

```java
// File: code/lld/ch20/lrucache/Main.java
        synchronized V get(K key) {
            Node n = index.get(key);
            if (n == null) return null;
            unlink(n);
            addFront(n);                // now the most recently used
            return n.value;
        }

        synchronized void put(K key, V value) {
            Node n = index.get(key);
            if (n != null) {            // update + mark as recently used
                n.value = value;
                unlink(n);
                addFront(n);
                return;
            }
            if (index.size() == capacity) {
                Node lru = tail.prev;   // least recently used lives just before the tail sentinel
                unlink(lru);
                index.remove(lru.key);
            }
            n = new Node(key, value);
            addFront(n);
            index.put(key, n);
        }
        // ... addFront / unlink: four and two pointer writes
    // ... jdkLru(capacity): new LinkedHashMap<>(16, 0.75f, true) overriding removeEldestEntry -> size() > capacity
```

Full runnable code: code/lld/ch20/lrucache/ (see the Full Edition). The demo prints LeetCode 146's expected values (1, miss, miss, 3, 4); this generic version returns `null` for a miss instead of `-1`.

Approach	get / put	Thread-safe	When to use
HashMap + own doubly linked list	O(1) / O(1)	Only with a lock	Interviews; custom behaviour
`LinkedHashMap(accessOrder=true)` + `removeEldestEntry`	O(1) / O(1)	No	Quick single-threaded caches
Caffeine / Guava	amortized O(1)	Yes	Production Java

Concurrency & edge cases

- **A read lock is not enough:** LRU `get` rewrites pointers, so it needs the exclusive lock.
- **Contention:** `ShardedCache` = 8 locks; trade-off: LRU order is only per shard.
- **Cache stampede** (many misses for one key all hit the DB): let the first thread load and others wait on its result (`computeIfAbsent` / a `CompletableFuture` per key) — Chapter 13's TTL cache does this with `ConcurrentHashMap.compute`.
- **TTL precisely:** expires *at* the deadline (`now >= expiresAt`); lazy — a dead entry holds a slot until read, swept, or evicted (then reported `EVICTED`). `put` without TTL on an existing key **clears** the old TTL.
- Capacity ≤ 0 refused by the builder; nulls unsupported (Caffeine too); use a monotonic clock (`nanoTime`) in production.

Extensions & follow-up questions

- **LFU:** one new policy; naive LFU is O(n), chained buckets O(1).
- **Size in bytes:** a `Weigher` strategy; evict until under max weight.
- **Load on miss:** *read-through* (`get(key, loader)`), *write-through* (cache + DB together), *write-back* (flush later; faster, may lose data), *cache-aside* (app does check → DB → put).
- **Distributed:** consistent hashing (high-level design). **Best policy?** Workload-dependent; Caffeine's *Window TinyLFU* mixes recency and frequency.

Real-world example

`LinkedHashMap` access order is the JDK's LRU. **Guava/Caffeine** expose the same knobs: `maximumSize`, `expireAfterWrite`/`expireAfterAccess`, `removalListener` (with causes like `EXPIRED`, `SIZE`, `EXPLICIT`, `REPLACED`), `recordStats()`. **Redis** picks eviction by config (`maxmemory-policy`: `allkeys-lru`, `allkeys-lfu`, `volatile-ttl`, `noeviction`…) and approximates LRU by sampling keys.

In the interview

> **30-second answer:** "A HashMap key → node for O(1) lookup, and a doubly linked list with sentinels for O(1) ordering; the node carries its own links so we can unlink it directly. The eviction rule sits behind `EvictionPolicy` — LRU moves on access, FIFO doesn't, LFU uses frequency buckets for O(1). Every exit goes through one `remove` that updates stats; listeners are notified after the lock is released. One lock is correct; sharding reduces contention."

Follow-up: *Why does the node store the key?* → To delete it from the map on eviction.

Self-check

1. Which two structures give O(1) LRU? → HashMap (key → node) + doubly linked list.
2. What do sentinels buy? → No null checks; add/remove always the same writes.
3. FIFO vs LRU in code? → `onAccess` does nothing.
4. Why can't a read-write lock speed up LRU `get`? → `get` modifies the list.
5. Cache stampede fix? → One thread loads; others wait on the same future.

TL;DR

- **HashMap + doubly linked list (sentinels)** = O(1) get/put.
- **Strategy** eviction (LRU / FIFO / O(1) LFU); **Observer** removals; **Builder** config.
- **One removal path**; lazy TTL + sweeper; injected clock.
- One lock (gets are writes), listeners outside it, **shard** to scale; know `LinkedHashMap`, Caffeine, Redis policies.

20.4 Design a Ride Sharing Service (Uber)

Problem & clarifying questions

What it is. Connects a rider with a nearby driver — a radio taxi dispatcher who calls the nearest free cab, then the next if refused, and the meter shows the fare. Two decisions are the whole question: **which**

driver is asked (matching) and **what the trip costs** (pricing). Either should change with no edits elsewhere.

Question	Assumed answer
Vehicle types?	Economy, premium, van; rider asks for one.
Who gets a request?	Online, free drivers of the right type within range.
Offer one at a time or broadcast?	One at a time, nearest first; decline → next; none left → cancel.
Trip states?	Requested → accepted → started → ended; cancel only from requested/accepted. Illegal actions do nothing.
Fare / surge?	Base + per km + per minute, with a minimum, per type; surge multiplies the same fare.
Ratings / payment?	Each side rates once after a finished trip (average shown); charge rider's method (cash, wallet, card).
Map?	Grid with Manhattan distance (production: a routing engine giving road distance + duration).
Pool, scheduled rides, tracking, payouts?	Out of scope.

Requirements

Functional: the table above, plus: driver has one vehicle (plate, seats, type) and goes online/offline; rider cancels only before start; trip history for both sides.

Non-functional: a driver can **never** be on two trips (even under concurrency); rules swappable; deterministic tie-breaks.

Use cases

Actors: **Rider** (request, cancel, rate), **Driver** (online, accept/decline, start/end, rate), **System** (find, offer, price, surge, history).

Core entities

Class	Responsibility
`Location`, `VehicleType`, `Vehicle`	Grid point (Manhattan `distanceTo`), car type and details.
`Account` (abstract) → `Rider`, `Driver`	Shared id/name/phone, **rating as total + count**, finished trips. Rider + payment method; Driver + vehicle, location, online, **atomic busy flag** (`tryClaim`).
`TripStatus` / `Trip`	State machine table; trip holds **ranked candidate list + cursor**; every action checks state.
`PricingStrategy`, `StandardPricing` + `RateCard`	**Strategy**: base + km + min with floor, per type.
`SurgePricing`	**Decorator** wrapping any `PricingStrategy`.
`MatchingStrategy`	**Strategy**: `NearestDriverMatcher`, `BestRatedMatcher`.
`PaymentMethod`	**Strategy**: `CashPayment`, `WalletPayment`.
`TripListener`	**Observer** of trip events.
`RideSharingService`	Facade; owns accounts and trips; knows only the order of steps.

Rating as total + count: only the average is shown, and adding a rating is O(1). `Rider`/`Driver` extend `Account` because they pass the LSP "is-a" test.

Class diagram

Course stages: (1) accounts, vehicle, location; (2) `Trip` + `TripStatus`; (3) pricing + surge wrapper; (4) matching; (5) the service. In the course figure, the arrow from `SurgePricing` back to `PricingStrategy` ("has-a of my own type") is what makes it a Decorator rather than another Strategy.

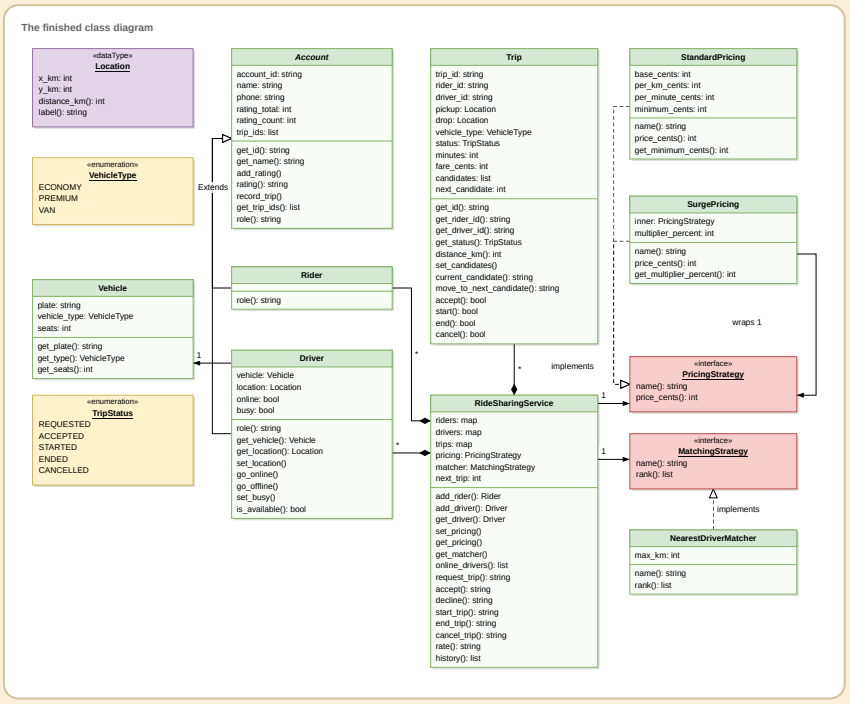
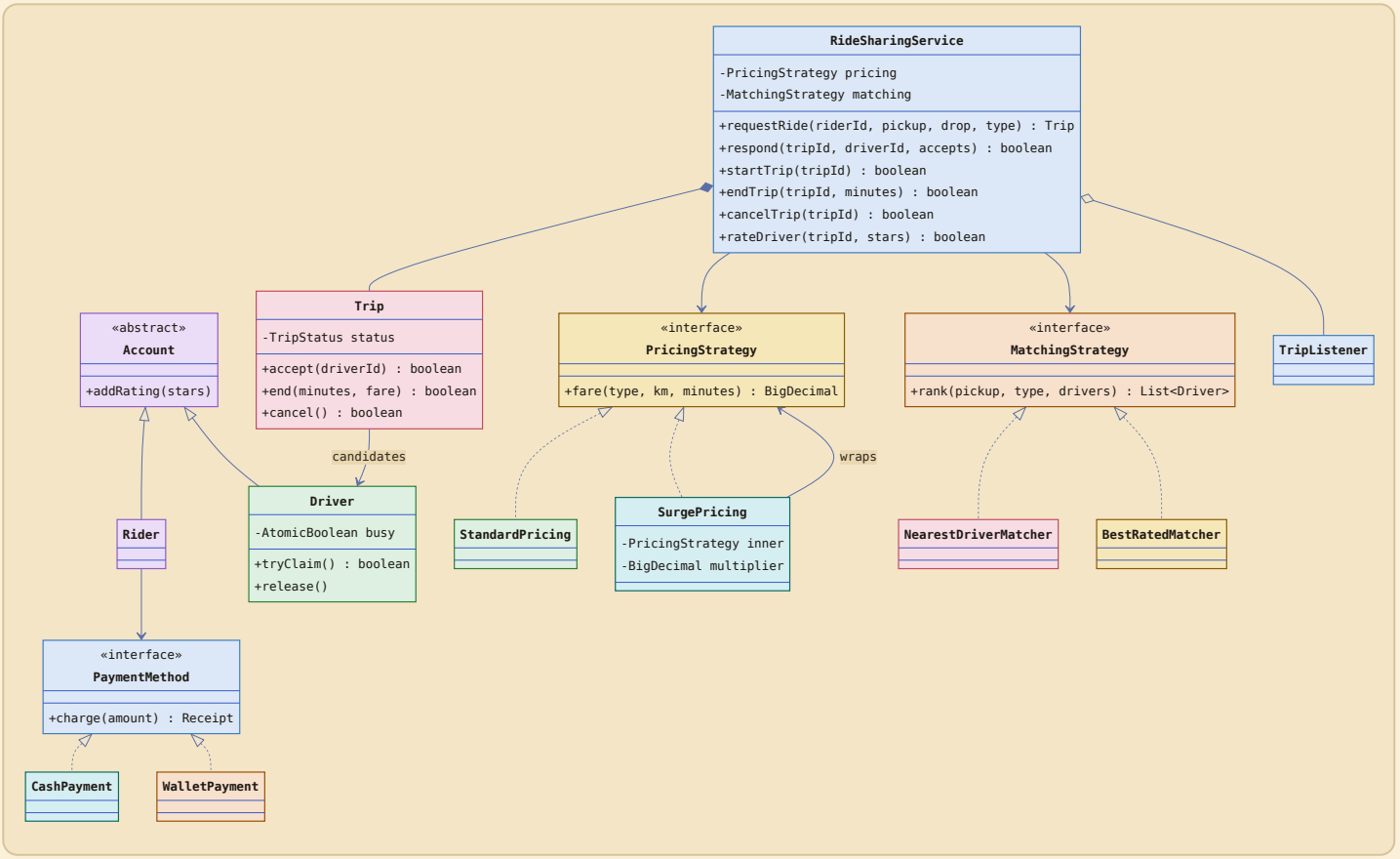
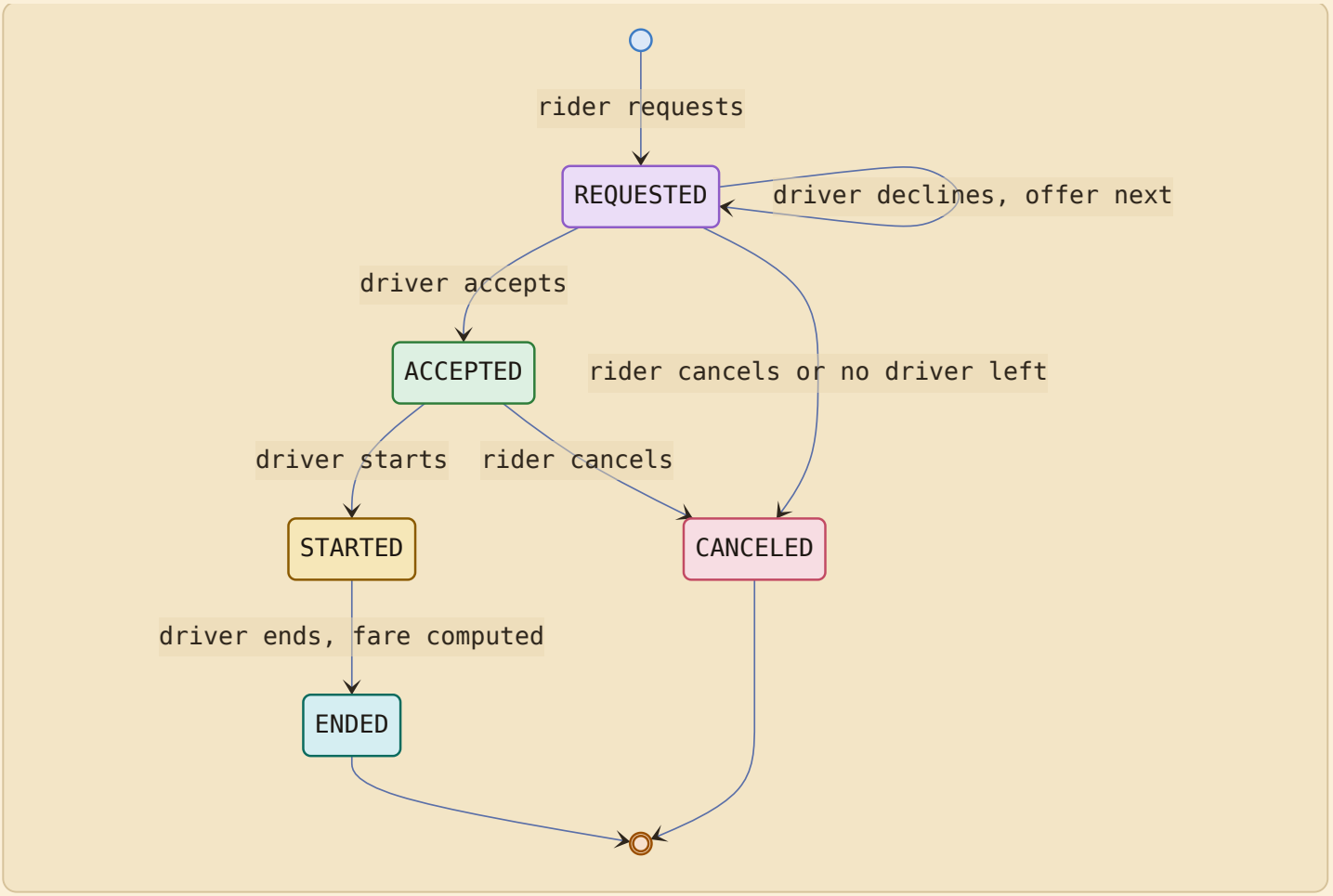


Figure — The finished ride sharing class diagram: the service owning riders, drivers and trips; PricingStrategy with StandardPricing and the SurgePricing wrapper; MatchingStrategy with the nearest-driver matcher (course diagram)



Key flows

Trip lifecycle (a state machine = fixed states + the only legal moves):



Finding a driver: rank free drivers of the right type nearest first; offer to the first; decline → move the cursor; nobody left → cancel. **Ending:** refuse if not started; else one `pricing.fare(...)` call (however many decorators wrap it), charge the rider, move the driver to the drop point, release.

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for matching	Cities rank differently (nearest, best-rated…).	Sorting inside the service.
Matcher returns a **ranked list** stored on the trip	Decline = move cursor; no re-search; remembers who refused.	Returning one driver.
Strategy pricing + **Decorator** surge	Surge wraps *any* rule; off = remove the wrapper; decorators stack (surge + airport fee + discount).	`if (surge)` in pricing.
State machine via `TripStatus.canMoveTo`	One table; illegal moves return `false`. Full State pattern only pays when per-state behaviour is large.	Scattered `if (status == …)`.
Atomic claim (`AtomicBoolean.compareAndSet`)	Two trips racing for a driver: exactly one wins; loser treats it as decline.	Check `isAvailable()` then set: race.
Move driver to **drop point** at end	Next match must use the real position.	Forgetting (real bug class).
Observer for trip events	Apps, push, analytics listen.	Service calling them directly.
Tie-break by driver id; `BigDecimal` `HALF_UP`	Reproducible tests; exact fares (course uses whole cents and surge as a percentage, 150 = ×1.5).	`double`.

Java implementation

```
// File: code/lld/ch20/ridesharing/Trip.java
enum TripStatus {
    REQUESTED, ACCEPTED, STARTED, ENDED, CANCELED;

    private static final Map<TripStatus, Set<TripStatus>> NEXT = Map.of(
            REQUESTED, EnumSet.of(ACCEPTED, CANCELED),
            ACCEPTED, EnumSet.of(STARTED, CANCELED),
            STARTED, EnumSet.of(ENDED),
            ENDED, EnumSet.noneOf(TripStatus.class),
            CANCELED, EnumSet.noneOf(TripStatus.class));

    boolean canMoveTo(TripStatus next) { return NEXT.get(this).contains(next); }
}

// File: code/lld/ch20/ridesharing/Strategies.java
/** Decorator: wraps ANY pricing rule and multiplies its answer. Remove the wrapper to switch surge off. */
final class SurgePricing implements PricingStrategy {
    private final PricingStrategy inner;
    private final BigDecimal multiplier;
    // ... constructor
    @Override
    public BigDecimal fare(VehicleType type, int km, int minutes) {
        return inner.fare(type, km, minutes).multiply(multiplier).setScale(2, RoundingMode.HALF_UP);
    }
}

// File: code/lld/ch20/ridesharing/Model.java  (Driver)
    boolean tryClaim() { return online && busy.compareAndSet(false, true); }

// File: code/lld/ch20/ridesharing/RideSharingService.java
    /** Driver answers the offer. A lost race for the driver counts as a decline. */
    boolean respond(String tripId, String driverId, boolean accepts) {
        Trip trip = trips.get(tripId);
        Driver current = trip.currentCandidate();
        if (current == null || !current.id().equals(driverId) || trip.status() != TripStatus.REQUESTED) return false;
        if (accepts && current.tryClaim()) {
            if (trip.accept(driverId)) {
                publish(trip, "accepted by " + driverId);
                return true;
            }
            current.release(); // trip was canceled meanwhile
            return false;
        }
        publish(trip, (accepts ? "could not claim " : "declined by ") + driverId);
        offerOrCancel(trip, trip.nextCandidate());
        return false;
    }
    // ... endTrip: price -> trip.end -> charge rider -> d.moveTo(drop) -> d.release() -> history
```

Full runnable code: code/lld/ch20/ridesharing/ (see the Full Edition).

`Trip` methods are `synchronized`, so a rider's cancel and a driver's accept can't interleave. `endTrip` relocates the driver **before** releasing them, so no matcher ever sees a free driver at the old spot.

Output (excerpt):

```
  [T1] declined by D1
  [T1] accepted by D2
  [T1] ended: 7 km, 12 min, fare 13.90 (wallet debited 13.90, left 36.10)
  [T2] ended: 1 km, 3 min, fare 7.50 (collect 7.50 in cash)
  [T4] cancel refused in state STARTED
  [T5] no driver left - canceled
  trips accepted = 10 (never more than 10 drivers)
```

Trip 1: 2.50 + 7 × 1.20 + 12 × 0.25 = **13.90**. Trip 2: 4.45 → minimum 5.00 → surge × 1.5 = **7.50**. 40 concurrent requests vs 10 drivers → exactly 10 accepted.

Machine-coding variant: carpool / "offer and select a ride"

A Flipkart-style round (kumaransg/LLD collection) with **no dispatcher**: users *offer* spare seats on a route and others *select* a ride with a **selection strategy**.

- `add_user`, `add_vehicle` (several per user); `offer_ride(user, origin, seats, vehicle, destination)` **fails if that vehicle already has an active offer**.
- `select_ride(user, origin, destination, seats, strategy)` — seats 1 or 2; **Most Vacant** or **Preferred Vehicle** ("Activa").
- `end_ride`, `print_ride_stats` — taken/offered per user, counting only **ended** rides.

Design: `Ride` with an atomic `book`; `RideSelectionStrategy` (`MostVacant`, `PreferredVehicle`); `RideService` filters active rides by route and seats, excludes the user's own offers, and lets the strategy choose.

```
// File: code/lld/ch20/carpool/Main.java
    interface RideSelectionStrategy {
        Optional<Ride> select(List<Ride> candidates);
    }

    static final class MostVacant implements RideSelectionStrategy {
        public Optional<Ride> select(List<Ride> c) { return
c.stream().max(Comparator.comparingInt(Ride::availableSeats)); }
    }
    // ... PreferredVehicle(model): first ride whose vehicle model matches
    // ... selectRide: filter ACTIVE rides on the same route, not offered by this user, with enough seats;
    // ... strategy.select(matching), then ride.book(user, seats) (synchronized check-and-book)
```

Full runnable code: code/lld/ch20/carpool/ (see the Full Edition). The demo replays the problem's sample (answers 2(c), 2(b), none, 2(d), none; then the stats).

	Uber-style (main design)	Carpool variant
Who supplies the ride	Professional drivers	Any user, per offered ride
Who chooses	System offers to drivers	Rider picks with a selection strategy
Key pattern	Strategy + Decorator + state machine	Strategy (selection)
Key invariant	A driver on ≤ 1 trip	A vehicle has ≤ 1 active offer; seats never negative

Concurrency & edge cases

- **Double booking:** prevented by `tryClaim()` CAS (the 40/10 race proves it).
- **Cancel vs accept:** both via `synchronized` `Trip` methods; if cancel wins, `accept` fails and the driver is released.
- **Stale candidate list:** computed once; a claimed/offline driver fails `tryClaim` → next. A nearer driver freed later isn't noticed (real systems re-match).
- **Offer timeouts:** no answer in N seconds = decline (scheduled task per offer).
- No drivers → canceled immediately. Illegal transitions → `false`, state unchanged. Wallet too low → `Receipt(paid=false)`. One rating per side, 1–5 stars.

Extensions & follow-up questions

- **Rank by rating:** swap in `BestRatedMatcher`.
- **Airport fee, promo, tolls:** more decorators: `new PromoDiscount(new AirportFee(new SurgePricing(standard, 1.5)))`.
- **How is surge computed?** Demand (open requests) vs supply (free drivers) per area and window → multiplier; the pricing design doesn't change.
- **Nearby drivers fast:** geospatial index — cells (geohash, S2, Uber's H3 hexagons) → map cell → drivers; search pickup cell + neighbours.
- **Pool rides:** honest answer: `Trip` becomes many riders/stops and the matcher considers detours. Live tracking: latest location per driver, published via pub-sub (20.6). Scheduled rides: a scheduler starts matching shortly before.

Real-world example

Uber shows surge ("dynamic pricing") when demand exceeds supply and gives a driver a short window to accept before the next — the ranked-list-with-cursor idea. Uber open-sourced **H3**, a hexagonal grid used for pricing and dispatch. Ola, Lyft and Grab follow the same request → match → accept → trip → rate lifecycle.

In the interview

> **30-second answer:** "Two rules behind interfaces: `MatchingStrategy` returns a ranked list of free drivers with the right car; `PricingStrategy` computes base + distance + time with a floor. Surge is a Decorator wrapping any pricing rule. The trip is a small state machine; every action checks the transition table and returns false if illegal. A decline moves a cursor along the stored ranking. Drivers are claimed with compare-and-set, so one driver never gets two trips."

Tip: name matching and pricing in the first minutes; spend real time on surge — *wrapping* the rule instead of adding a flag separates strong answers. *Strategy vs Decorator?* → Strategy replaces the rule; Decorator wraps it and stays in the call path (holds an `inner` of the same interface).

Self-check

1. Why a ranked list, not one driver? → Decline = next candidate, no re-search.
2. What makes `SurgePricing` a Decorator? → It is a `PricingStrategy` wrapping another and adjusting its result.
3. Can a rider cancel a STARTED trip? → No; `STARTED` only → `ENDED`.
4. Which Java class makes the claim atomic? → `AtomicBoolean.compareAndSet(false, true)`.
5. Carpool: when does `offer_ride` fail? → The vehicle already has an active offer.

TL;DR

> - **Strategy** ×3 (matching, pricing, payment) + **Decorator** (surge) + **state machine** (trip).
> - Ranked list + cursor; decline = next. **Atomic claim** prevents double booking; relocate driver at trip end.
> - Ratings = total + count; `BigDecimal` fares with a minimum.
> - Carpool: users offer seats; riders select with Most Vacant / Preferred Vehicle.

20.5 Design a Logging Framework

Problem & clarifying questions

What it is. Code writes messages ("payment 991 authorised") with a **level** (severity), and the framework sends them to **destinations** (console, file, log server) in a chosen **format** — a mini Log4j 2 / Logback /

`java.util.logging`. Analogy: a company mail room that routes notes by department and urgency stamp to inboxes, in the right envelope.

Question	Assumed answer
Levels?	TRACE < DEBUG < INFO < WARN < ERROR < FATAL (and OFF).
Per-module config?	Loggers named `app.payment`, `app.orders` form a **dot hierarchy**; a child inherits the nearest ancestor's level.
Destinations / formats?	Console, file, in-memory; pattern and JSON; both pluggable; each destination has its own minimum level.
Propagation?	Event goes to the logger's destinations then **up** to ancestors', unless additivity is off.
Async / thread-safe?	Optional async (app never waits for disk); yes.
Config files, rolling files, shipping?	Out of scope.

Requirements

Functional: the table above, plus: same name → same logger; exceptions attachable; lazy messages; async without changing appenders.

Non-functional: very cheap when disabled; thread-safe; logging never crashes the app; testable (injected clock).

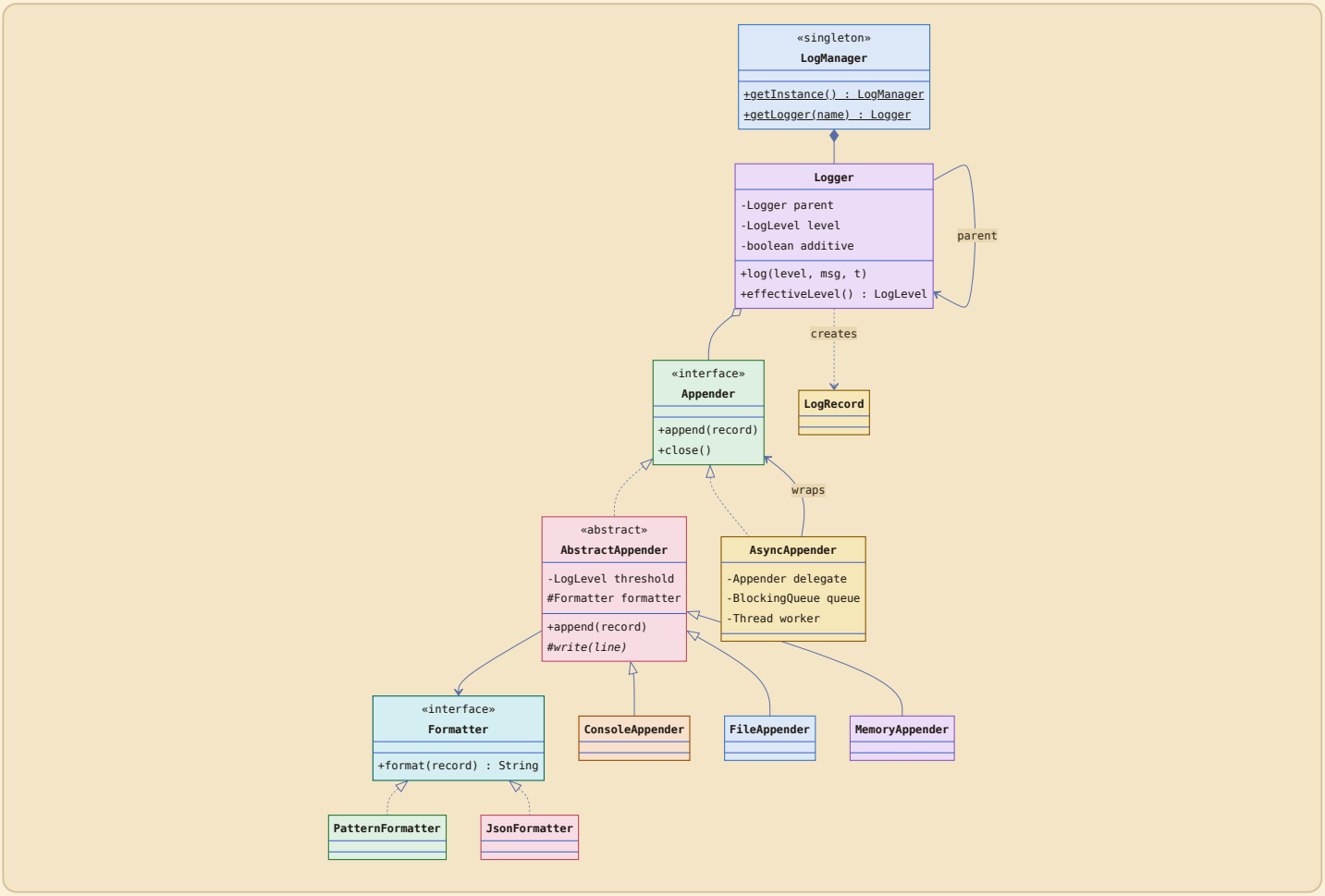
Use cases

Actors: **Developer code** (logs), **Operator** (configures levels, appenders, formats), **Framework** (filters, formats, routes, writes). Use cases: log a message, get a logger by name, set a level for a subtree, add a destination, choose a format, switch to async, stop propagation for a subtree.

Core entities

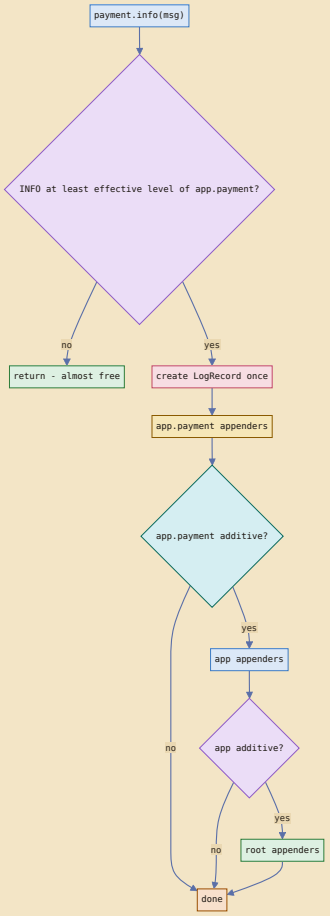
Class	Responsibility
`LogLevel`	Ordered severities; `isAtLeast(threshold)`.
`LogRecord`	One immutable event: time, level, logger, thread, message, error.
`Logger`	Named node: parent, optional level, appenders, additivity; `log()` walks up.
`LogManager`	**Singleton** registry; `getLogger(name)` creates parents first, one instance per name; holds the clock.
`Appender` / `AbstractAppender`	A sink; **Template Method** base checks threshold + formats, subclasses only `write(line)`.
`ConsoleAppender`, `FileAppender`, `MemoryAppender`	Concrete sinks.
`AsyncAppender`	**Decorator**: bounded queue + one writer thread around any appender.
`Formatter`	**Strategy**: `PatternFormatter`, `JsonFormatter`.

Class diagram



Key flows

Logging one event = Chain of Responsibility up the hierarchy (a request passes along a chain of handlers; each may handle it and/or pass it on):



The Log4j/Logback rule we copy: the **level check happens once, at the logger where the call was made**; parents' levels are not re-checked, only each **appender's own threshold** filters. So a DEBUG event from `app.payment` (set to DEBUG) still reaches root's console even though root is INFO.

Async: the app thread calls `append` → `queue.offer` (never blocks) → returns; a writer thread `take()`s and calls the slow delegate. `close()` enqueues a poison record, then `join()`s until the queue is drained.

Design decisions & patterns used

Pattern / decision	Where	Why
Chain of Responsibility	`Logger.log`: this → parent → … → root, stops at `additive == false`	Configure sinks once near the root; subtrees add or stop.
Singleton (holder idiom)	`LogManager`	One registry → same logger per name; lazy + thread-safe with no locking (JVM initializes the nested class once). Double-checked locking is easy to get wrong.
Static factory	`getLogger(name)`	Caches instances, creates parents first; `new Logger` would break the hierarchy.
Strategy	`Formatter`	Any sink × any format; no `JsonFileAppender` class explosion.
Template Method	final `append` → `write`	Threshold + formatting once.
Decorator	`AsyncAppender`	Async is an add-on, not `AsyncFileAppender`, `AsyncConsoleAppender` …
Level check first; lazy `Supplier<String>`	`isEnabled`, `debug(Supplier)`	Disabled logging = one comparison.
Appender failures caught	`Logger.log`	Logging never crashes the app.
Parent resolved **outside** `computeIfAbsent`	`LogManager.loggerFor`	A `ConcurrentHashMap` mapping function must not modify the same map (`IllegalStateException: Recursive update`).

Another common "CoR logger" answer chains *level handlers* (`ErrorHandler → WarnHandler → InfoHandler`). It works as a toy, but real frameworks put the chain on the **logger hierarchy**, because that is what operators configure. If the interviewer expects the level-handler version, explain both.

Java implementation

```
// File: code/lld/ch20/logging/Logger.java
    // ... effectiveLevel() walks up to the nearest logger with a level; isEnabled(); debug(Supplier) for lazy
messages
    void log(LogLevel lvl, String msg, Throwable t) {
        if (!isEnabled(lvl)) return;                    // cheap check first
        LogRecord r = new LogRecord(manager.now(), lvl, name, Thread.currentThread().getName(), msg, t);
        for (Logger l = this; l != null; l = l.parent) { // the chain
            for (Appender a : l.appenders) {
                try {
                    a.append(r);
                } catch (RuntimeException e) {
                    System.err.println("appender failed: " + e); // logging must never crash the app
                }
            }
            if (!l.additive) break;
        }
    }

/** Singleton registry of loggers (initialization-on-demand holder idiom: lazy and thread-safe). */
final class LogManager {
    // ...
    private static final class Holder { static final LogManager INSTANCE = new LogManager(); }

    static LogManager getInstance() { return Holder.INSTANCE; }

    private Logger loggerFor(String name) {
        if (name == null || name.isEmpty() || name.equals("root")) return root;
        Logger existing = loggers.get(name);
        if (existing != null) return existing;
        int dot = name.lastIndexOf('.');
        Logger parent = dot < 0 ? root : loggerFor(name.substring(0, dot)); // resolved OUTSIDE computeIfAbsent
        return loggers.computeIfAbsent(name, n -> new Logger(n, parent, this));
    }
    // ...
}

// File: code/lld/ch20/logging/Appenders.java
    @Override
    public final void append(LogRecord r) {
        if (r.level().isAtLeast(threshold)) write(formatter.format(r));
    }
    // ...
    @Override
    public void append(LogRecord r) {
        if (closed || !queue.offer(r)) dropped.incrementAndGet(); // full or closed: count it, never block
    }
```

Full runnable code: code/lld/ch20/logging/ (see the Full Edition).

`LogRecord` is created once and shared (immutable, thread-safe). Sinks synchronize `write` so lines never interleave. `AsyncAppender`'s promise is precise: `written + dropped` always equals the number of `append` calls; a throwing delegate is caught inside the writer loop so the writer never dies (which would leave `close()` waiting forever).

Output (excerpt):

```
2026-09-28 10:15:30.123 DEBUG [main] app.payment - card token=tok_42 (built lazily)
== 4. Additivity off: 'app.payment' events stop at its own appenders
  (nothing printed on console above) audit sink now has 2 lines
== 6. Async appender: 4 threads x 1000 events, then close() flushes
  written=4000 dropped=0 total=4000
```

Concurrency & edge cases

- Appenders in `CopyOnWriteArrayList`; level and additivity `volatile`; each sink's `write` synchronized.
- **Async queue full:** drop-and-count (never block). Alternatives: block (back-pressure, may stall the app) or drop low levels only — Logback's `AsyncAppender` by default discards TRACE/DEBUG/INFO at 80% full.
- **Shutdown:** without `close()` queued events are lost → JVM shutdown hook.

- **Recursive logging** from an appender can loop forever → thread-local guard.
- **Security:** never interpret messages — Log4Shell (CVE-2021-44228, Dec 2021) came from Log4j 2 evaluating `${jndi:...}` in logged strings. Mask card numbers, tokens, passwords.

Extensions & follow-up questions

- **Rolling files:** `RollingFileAppender` + `RollingPolicy` (size/time) Strategy.
- **Config file:** parse into `LogManager` calls; watch it for runtime changes.
- **Filters:** a `Filter` chain per appender — another Chain of Responsibility.
- **MDC / request id:** thread-local map copied into each record. Ship logs: network appender behind `AsyncAppender`, batching. Structured logging: key-value fields rendered as JSON.

Real-world example

SLF4J is the facade; **Logback** and **Log4j 2** implement exactly this model (dot-named hierarchy, inherited levels, appenders with layouts, *additivity*). Log4j 2 async loggers use the LMAX Disruptor ring buffer. `java.util.logging`: `Logger`, `Handler` (appender), `Formatter`, `Level`, `setUseParentHandlers(false)` (additivity off).

In the interview

> **30-second answer:** "Loggers form a dot hierarchy under a root managed by a singleton `LogManager`. A log call checks the inherited effective level, builds one immutable record, and passes it up the chain of loggers to their appenders — Chain of Responsibility — until additivity stops it. Each appender has its own threshold and a pluggable formatter (Strategy). `AsyncAppender` is a Decorator with a bounded queue and a writer thread."

Follow-ups: *Why is a disabled debug call cheap?* → One comparison before any string; `Supplier` for expensive ones. *Queued logs on crash?* → Lost unless flushed; close on shutdown.

Self-check

1. `app.payment` unset, `app` WARN, root INFO → effective level? → WARN.
2. Additivity=false? → Stops the event reaching ancestors' appenders.
3. Pattern making any appender async? → Decorator.
4. Why is `Formatter` separate from `Appender`? → Formats and sinks combine freely (Strategy).
5. How does `close()` lose nothing queued? → Poison pill + `join()`; refused records are counted as dropped.

TL;DR

- Named **logger hierarchy**, inherited level, one cheap check up front.
- **Chain of Responsibility** to the root (additivity stops it); **Singleton** `LogManager`.
- **Strategy** formatters, **Template Method** appender base, **Decorator** async.
- Immutable record, thread-safe sinks, never crash, never block on a full queue.

20.6 Design a Pub-Sub System

Problem & clarifying questions

What it is. Parts of a system talk **without knowing each other**: a *publisher* sends to a named *topic*; every *subscriber* of it receives the message. Analogy: a newspaper printed once, delivered to every subscriber,

new readers welcome any time. It is the **Observer pattern as infrastructure**: a **broker** in the middle decouples by topic, time and often machine. We design a simplified, Kafka-like, in-process broker.

Question	Assumed answer
Delivery to how many?	Every subscription gets every message (fan-out), in publish order.
Guarantee?	**At-least-once**: ack only after processing; failures retried; duplicates possible.
Slow subscriber?	Must not slow others or the publisher.
Keep messages?	Yes: append-only log with **offsets** → late subscribers and replay.
Poison message?	After N attempts → dead-letter topic.
Push or pull?	Push, one dispatcher per subscription.
Partitions, consumer groups, disk, retention?	Out of scope.

Requirements

Functional: create topics; publish (key, payload) → an **offset** (position in the log); subscribe from EARLIEST or LATEST; in-order delivery; commit only after the callback returns; retry, then dead-letter; **seek** to replay; unsubscribe; clean shutdown.

Non-functional: thread-safe publish; subscribers isolated; no busy-waiting; order preserved.

Use cases

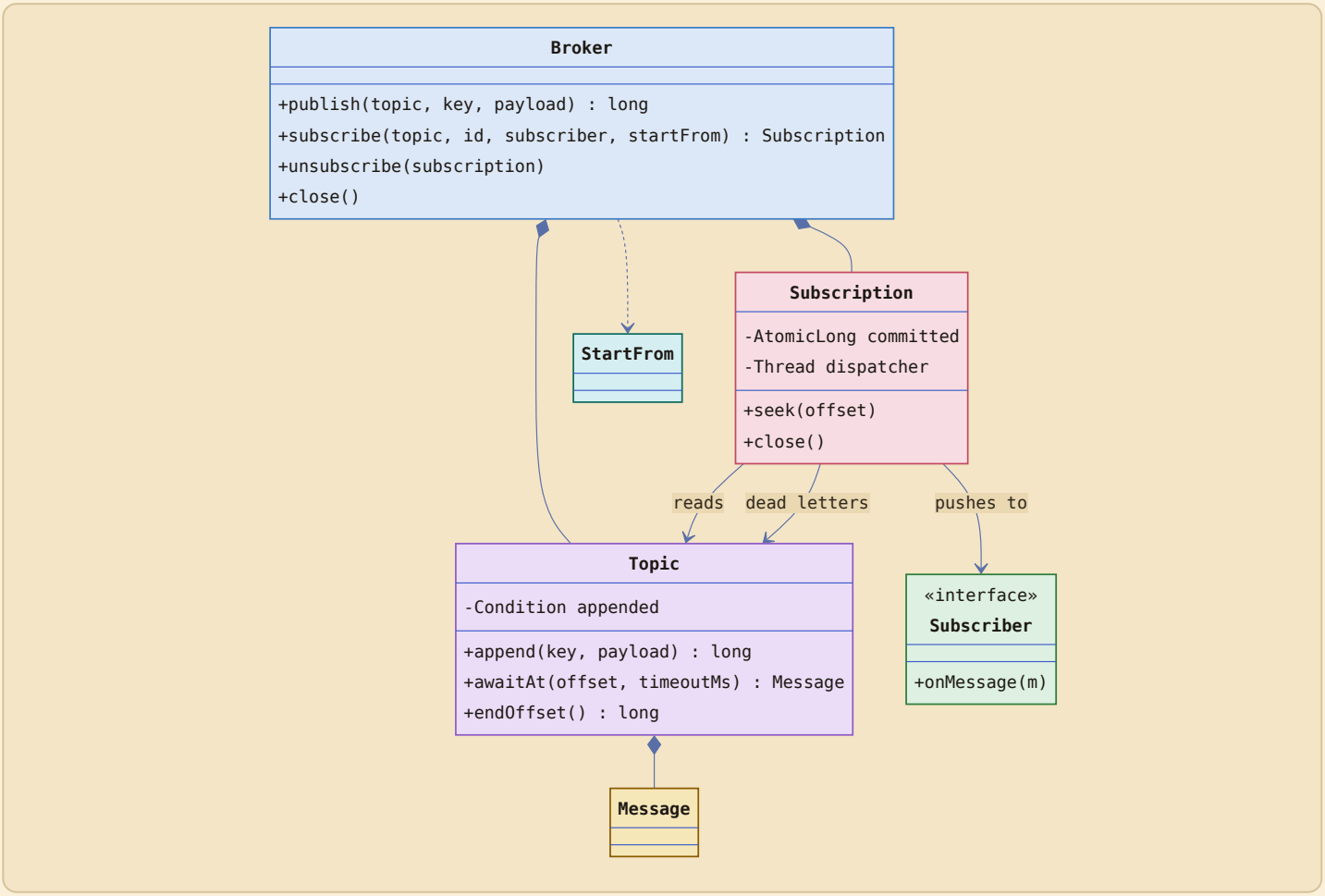
Actors: **Publisher**, **Subscriber** (sometimes fails), **Operator** (creates topics, replays, inspects dead letters), **Broker** (stores, dispatches, retries).

Core entities

Class	Responsibility
`Message`	topic, offset, key, payload; immutable.
`Topic`	Append-only `List<Message>` guarded by a `ReentrantLock` + `Condition` "appended".
`Subscriber`	Callback `onMessage`; throwing = negative ack.
`Subscription`	Committed offset (`AtomicLong`), **own dispatcher thread**, retry + dead-letter, `seek`.
`StartFrom`	EARLIEST / LATEST.
`Broker`	Registry, publish, subscribe, unsubscribe, DLQ, `awaitIdle` (tests), close.

Key idea: messages are not removed when consumed. Each subscription only remembers *how far it has read*. That gives late joiners, replay (`seek`) and independent progress for free — Kafka's model. (RabbitMQ-style queues delete on ack.)

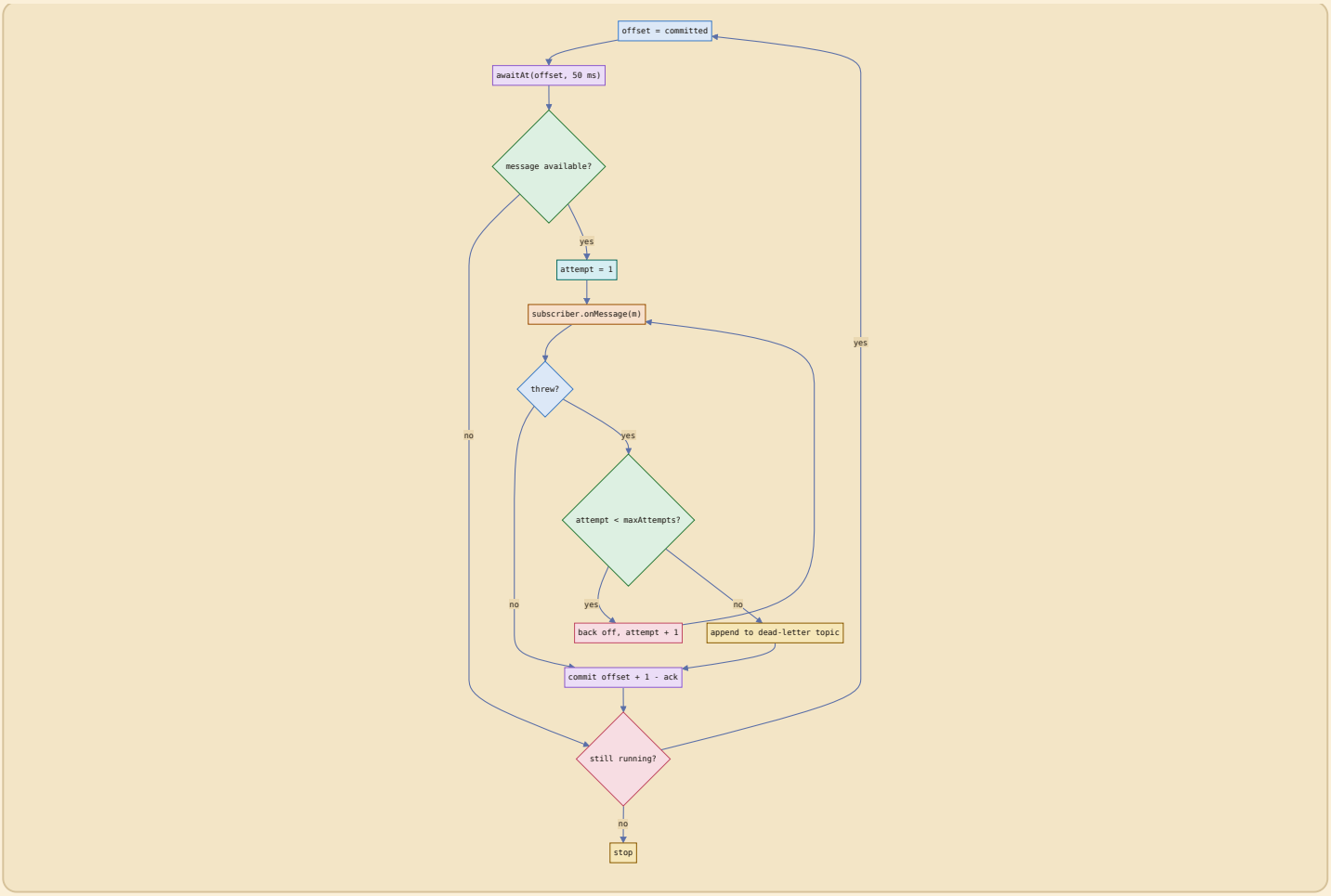
Class diagram



Key flows

Publish and fan-out: dispatchers sleep on the topic's `Condition`; `publish` appends (offset N) and `signalAll()`s; each dispatcher independently delivers message N and commits N+1 — a slow one delays only itself.

One subscription's dispatcher loop (at-least-once with dead letters):



Design decisions & patterns used

Decision	Why	Alternative rejected
Publish-Subscribe (Observer via broker)	Add subscribers with zero publisher changes.	Publisher calling consumers directly.
Append-only log + offsets	Replay, late joiners, independent progress; order = log order.	Delete on delivery; per-subscriber copies.
One dispatcher per subscription (virtual threads)	Slow subscriber delays only itself; one deliverer keeps order; virtual threads make it cheap.	One shared thread (slowest blocks all); pool per message (loses order).
`Lock` + `Condition` with timeout, in a `while`	No busy-waiting; timeout notices `close()` / `seek()`; `while` handles spurious wake-ups.	`Thread.sleep` polling.
Ack after processing	At-least-once: a crash re-delivers.	Commit first = at-most-once.
Dead-letter topic	A poison message can't block forever.	Infinite retries.
`compareAndSet` on commit	A concurrent `seek()` wins.	Plain `set`.

Delivery guarantees:

Guarantee	How	Risk
At-most-once	Commit, then process	Crash loses the message
At-least-once (ours)	Process, then commit	Duplicates → subscribers must be **idempotent**
Exactly-once	At-least-once + dedup / transactions committing output and offset together	Complex; Kafka offers it within its own transactions

Java implementation

```
// File: code/lld/ch20/pubsub/Topic.java
    // ... append(): under the lock, add to the log and appended.signalAll()
    /** Returns the message at 'offset', waiting up to the timeout for it to be published. */
    Message awaitAt(long offset, long timeoutMs) throws InterruptedException {
        lock.lock();
        try {
            long nanos = TimeUnit.MILLISECONDS.toNanos(timeoutMs);
            while (offset >= log.size()) {         // loop: guards against spurious wake-ups
                if (nanos <= 0) return null;
                nanos = appended.awaitNanos(nanos);
            }
            return log.get((int) offset);
        } finally {
            lock.unlock();
        }
    }

// File: code/lld/ch20/pubsub/Broker.java  (Subscription)
    private void loop() {
        try {
            while (running) {
                long offset = committed.get();
                Message m = topic.awaitAt(offset, 50);
                if (m == null) continue;              // timed out: re-check 'running' and any seek()
                deliver(m);
                committed.compareAndSet(offset, offset + 1); // ack; a concurrent seek() wins
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    private void deliver(Message m) throws InterruptedException {
        for (int attempt = 1; attempt <= maxAttempts; attempt++) {
            try {
                subscriber.onMessage(m);
                return;
            } catch (Exception e) {
                if (attempt == maxAttempts) {
                    deadLetters.append(id, m.topic() + "@" + m.offset() + " " + m.payload() + " -> " +
e.getMessage());
                    return;                           // skip it so one poison message cannot block the rest
                }
                Thread.sleep(5L * attempt);           // simple backoff
            }
        }
    }
    // ...
    void seek(long offset) { committed.set(offset); }
```

Full runnable code: code/lld/ch20/pubsub/ (see the Full Edition).

Output (excerpt):

```
inventory [0:order#1, 1:order#2, 3:order#3]  (offset 1 retried, offset 2 dead-lettered)
dead letter: orders@2 BAD-JSON -> cannot parse
live after replay [4:order#4, 3:order#3, 4:order#4] (order#4 seen twice: at-least-once)
```

Concurrency & edge cases

- Publishers: `append` under a lock → unique, gap-free offsets; messages immutable.
- **Unbounded log:** real brokers apply **retention** (by age/size); a lagging subscriber may then fall off the end → detect "offset out of range" and reset.
- **Subscriber blocks forever:** stalls only its own subscription; add a processing timeout. Java 21 caveat: a virtual thread blocking inside `synchronized` *pins* its carrier thread — prefer `ReentrantLock` in subscriber code, or platform threads.
- **Duplicates:** subscribers must be idempotent (remember processed ids).
- **Shutdown:** `close()` flips `running` and joins; the 50 ms timeout bounds it.

Extensions & follow-up questions

- **Consumer groups:** split a topic into **partitions**; same key → same partition (`hash(key) % n`) keeps per-key order; each group member owns partitions; offset per (group, partition). That is how Kafka scales.
- **Persist:** segment files + offset index; committed offsets in a special topic (`__consumer_offsets`).
- **Pull:** `poll(id, max)` + `commit(offset)` → back-pressure for free.
- Filtering by predicate, wildcard topics (`orders.*`), delayed retry topics (`orders.retry.5s`) instead of sleeping.

Real-world example

Apache Kafka is this model at scale (partitioned append-only logs, consumer offsets, consumer groups, per-partition order, retention by time/size). **Google Cloud Pub/Sub** and **AWS SNS + SQS** offer managed fan-out with acks and **dead-letter** queues; Spring Kafka's `DeadLetterPublishingRecoverer` is our `__dead_letters`. In-JVM: Guava `EventBus`, Spring `ApplicationEventPublisher`.

In the interview

> **30-second answer:** "Each topic is an append-only log; publishing appends and returns an offset. Each subscription keeps a committed offset and its own dispatcher thread that waits on a condition, calls the subscriber, and commits only after success — at-least-once, in order. Failures retry with backoff, then go to a dead-letter topic. Messages aren't deleted on read, so late subscribers start at the beginning and anyone can replay. To scale: partitions by key, consumer groups."

Follow-ups: *Slow consumer?* → One dispatcher per subscription; the log is shared, progress isn't. *Exactly-once?* → Hard; at-least-once + idempotent consumers. *Order?* → One writer assigns offsets, one dispatcher per subscription; with partitions, order is per key.

Self-check

1. What is an offset? → A message's position in the log; subscribers remember the next one.
2. Commit before or after processing for at-least-once? → After.
3. Why a dead-letter topic? → One poison message can't block a subscription forever.
4. Why does replay work here but not in a delete-on-ack queue? → Messages stay; seek moves the offset.
5. Why `while` around `Condition.await`? → Spurious wake-ups.

TL;DR

- **Pub-sub = Observer through a broker.**
- **Append-only log + per-subscription offsets** → fan-out, late joiners, replay.
- **One dispatcher per subscription** (isolation + order); **ack after processing** = at-least-once.
- Retry, then **dead-letter**; subscribers **idempotent**; scale with partitions and consumer groups.

20.7 Design a Rate Limiter

Problem & clarifying questions

`What it is.` Decides whether a request is allowed now, so one client can't overwhelm a service. Analogy: a turnstile allowing 10 people per minute per ticket type; over the limit: "come back in 6 seconds" — HTTP **429 Too Many Requests** with `Retry-After`. **Chapter 13 (the rate-limiter / token-bucket problem) already built the concurrency core** (a `synchronized` and a lock-free CAS token bucket, algorithm comparison). Here we

design the **objects around it**: pluggable algorithms, rules per API and tier, a factory, combined limits, a decision object.

Question	Assumed answer
Limit by?	(user, API); IP or API key is the same design, different key.
Different limits?	Per API and tier (FREE / PREMIUM), plus a default.
Algorithms?	Token bucket, fixed window, sliding log, sliding counter — per rule.
Several at once?	Yes: "2 per 10 s **and** 3 per minute" on `/login`.
Response?	allowed + remaining + retry-after.
Distributed?	Single process here; distributed in extensions.

Requirements

Functional: `check(userId, api)` → `Decision(allowed, remaining, retryAfter)`; rules per API + tier with a default; **one limiter per (user, api)**, created on first use; pluggable algorithms; combined rules must all allow.

Non-functional: ~O(1) per check; exactly N allowed under any concurrency; injectable clock; bounded memory.

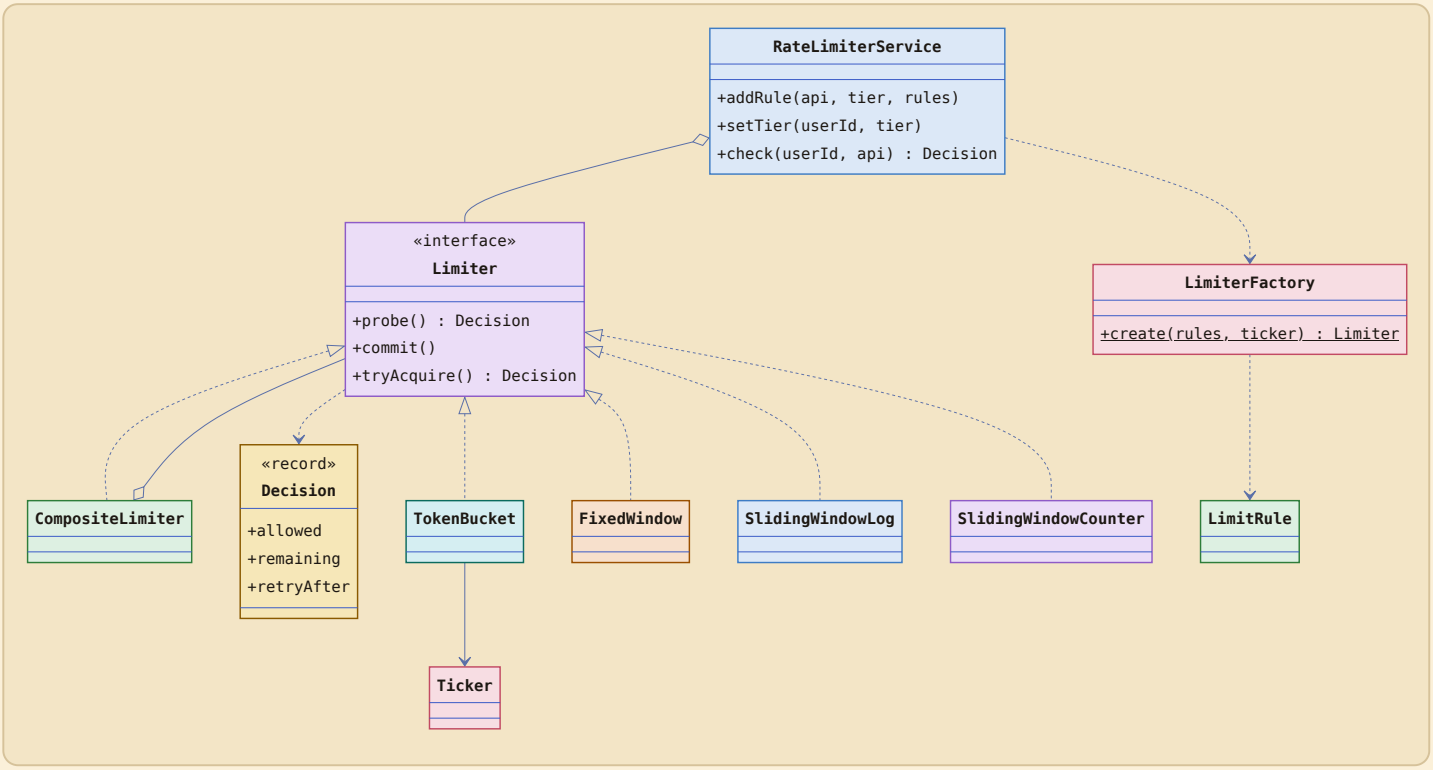
Use cases

Actors: **Client** (sends requests), **API gateway / filter** (calls `check` before the handler), **Operator** (configures rules and tiers).

Core entities

Class	Responsibility
`Limiter`	**Strategy**: `probe()` (would one more pass? consumes nothing), `commit()`, `tryAcquire()` = both atomically. One per key.
`TokenBucket`, `FixedWindow`, `SlidingWindowLog`, `SlidingWindowCounter`	The algorithms (`synchronized`).
`CompositeLimiter`	**Composite**: all children must allow; probe all, then commit all, under one lock.
`Decision`, `LimitRule`, `Algorithm`, `Tier`	Result; config (algorithm, limit, window); enums.
`LimiterFactory`	**Simple Factory**: rule(s) → limiter (single or composite).
`RateLimiterService`	**Facade**: resolves tier + rules, one limiter per (user, api) in a `ConcurrentHashMap`.
`Ticker`	Injected nanosecond clock.

Class diagram



Key flows

Gateway → `check("ann", "/search")` → `limiters.computeIfAbsent(key)` (first time: factory builds the limiter from the FREE-tier rules) → `tryAcquire()` → allowed: call the handler, 200; denied: **429 + Retry-After**.

The four algorithms (limit N per window W):

Algorithm	State per key	Bursts	Accuracy	Notes
Token bucket	tokens, last refill	Up to capacity	Good	Most common for APIs
Fixed window counter	window start, count	**Up to 2N across a boundary**	Poor at edges	Simplest
Sliding window log	one timestamp per request	None beyond N	Exact	O(N) memory per key
Sliding window counter	2 counters	Approximate	Good	previous × overlap + current
(Leaky bucket)	a queue	Constant output rate	—	Traffic shaping; not implemented

Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy (`Limiter`)	Algorithms differ in accuracy/memory; new one = new class.	`switch` in `check`.
Simple Factory	One place turns config into objects.	`new TokenBucket` in the service.
Composite, probe all then commit all	"10/s and 1000/day" looks like one limiter; a request denied by one child uses up nothing in another.	Calling each child's `tryAcquire` in turn (a later denial leaves earlier children charged).
Facade service	Gateway asks one question.	Gateway juggling tiers/rules.
`computeIfAbsent` per key	Exactly one limiter even if the first two requests race.	`get` then `put`.
`Decision` with remaining/retryAfter	Maps to `X-RateLimit-Remaining`, `Retry-After`.	`boolean`.
Injected `Ticker`; `synchronized` probe+commit	Tests never sleep; refill + check + take is one atomic step.	`nanoTime()` inside; unsynchronized fields.

Java implementation

```java
// File: code/lld/ch20/ratelimiter/Limiters.java
interface Limiter {
    Decision probe();

    void commit();

    /** Check and take in ONE atomic step (probe + commit under this limiter's lock). */
    default Decision tryAcquire() {
        synchronized (this) {
            Decision d = probe();
            if (d.allowed()) commit();
            return d;
        }
    }
}

/** Token bucket: bursts up to 'capacity', refills at 'perSecond'. Lazy refill, no background thread. */
final class TokenBucket implements Limiter {
    // ... capacity, refillPerNano, ticker, tokens, last
    @Override
    public synchronized Decision probe() {
        long now = ticker.nanos();
        tokens = Math.min(capacity, tokens + (now - last) * refillPerNano); // refilling is safe to do early
        last = now;
        if (tokens >= 1) return Decision.allow((long) (tokens - 1));
        long waitNanos = refillPerNano == 0 ? Long.MAX_VALUE : (long) Math.ceil((1 - tokens) / refillPerNano);
        return Decision.deny(Duration.ofNanos(waitNanos));
    }

    @Override
    public synchronized void commit() { tokens -= 1; }
}
// ... FixedWindow, SlidingWindowLog, SlidingWindowCounter

final class CompositeLimiter implements Limiter {
    private final List<Limiter> children;
    // ...
    @Override
    public synchronized Decision probe() {
        long remaining = Long.MAX_VALUE;
        for (Limiter l : children) {
            Decision d;
            synchronized (l) { d = l.probe(); }
            if (!d.allowed()) return d;           // first denial wins; nothing has been consumed
            remaining = Math.min(remaining, d.remaining());
        }
        return Decision.allow(remaining);
    }

    @Override
    public synchronized void commit() {
        for (Limiter l : children) synchronized (l) { l.commit(); }
    }
}

// File: code/lld/ch20/ratelimiter/RateLimiterService.java
    Decision check(String userId, String api) {
        // computeIfAbsent creates exactly ONE limiter per (user, api), even under a race
        Limiter limiter = limiters.computeIfAbsent(new Key(userId, api), k -> {
            Tier tier = tiers.getOrDefault(userId, Tier.FREE);
            List<LimitRule> r = rules.getOrDefault(api, Map.of()).getOrDefault(tier, defaultRules);
            return LimiterFactory.create(r, ticker);
        });
        return limiter.tryAcquire();
    }
```

Full runnable code: code/lld/ch20/ratelimiter/ (see the Full Edition).

Composite children are created for it alone by the factory and never shared, so the composite's lock makes probe-all + commit-all atomic.

Output (excerpt):

```
  t=0.0 ann /search #4 -> 429 Too Many Requests (retry after 1000 ms)
== 4. Boundary burst: limit 5 per 10 s, 5 calls at t=9.9 and 5 at t=10.1
  FIXED_WINDOW     allowed 10 of 10 within 0.2 s
  SLIDING_LOG      allowed 5 of 10 within 0.2 s
  SLIDING_COUNTER  allowed 5 of 10 within 0.2 s
  allowed = 50 (exactly 50, never more)
```

The classic point: **fixed window lets 10 through in 0.2 s** across a boundary; both sliding algorithms hold at 5. 640 concurrent attempts on a 50-token bucket → exactly 50.

Concurrency & edge cases

- Contention is per key (each limiter `synchronized`; map is `ConcurrentHashMap`), so different users never block each other.
- **Naive composite bug:** child 1 allows (and consumes), child 2 denies → child 1 is charged for a request that never ran. The demo's "3 per minute first" case shows the +11s call would wrongly be refused for ~49 s. Probe-then-commit fixes it (acquire-and-refund is the harder alternative).
- **Memory:** one limiter per key forever → evict idle limiters (expire-after-access cache, as in 20.3) with idle timeout ≥ the longest window — a fresh limiter then behaves identically.
- Monotonic clock; rule changes at runtime need clearing or versioning the limiter map.

Extensions & follow-up questions

- **Distributed:** shared state in Redis with an atomic Lua script (or `INCR` + `EXPIRE` for fixed windows); costs a network hop — mitigate with local token pre-allocation or sticky routing.
- **Leaky bucket:** bounded queue drained at a constant rate (protects fragile downstreams).
- Cost per request (`tryAcquire(permits)`), soft limits (warn at 80%), key by IP when logged out; runs in an API gateway or servlet filter / Spring `HandlerInterceptor`.

Real-world example

Stripe described combining a token-bucket request limiter, a concurrent-requests limiter and load shedders on Redis — the composite idea. **GitHub** returns `X-RateLimit-Limit/Remaining/Reset` (our `Decision`). HTTP 429 is RFC 6585. **NGINX** `limit_req` uses leaky bucket. Java: Guava `RateLimiter`, Resilience4j, **Bucket4j**. Cloudflare uses the sliding-window-counter approximation at the edge.

In the interview

30-second answer: "A `Limiter` interface with interchangeable algorithms — token bucket by default, sliding window when exactness matters. A factory builds limiters from rules chosen by API and tier; a composite combines '10 per second and 1000 per day'. The service keeps one limiter per (user, api) via `computeIfAbsent` and returns remaining and retry-after for the 429. Each limiter makes refill-check-take atomic; for many servers, move state into Redis with an atomic script."

Follow-ups: *Why is fixed window risky?* → Up to 2N across a boundary. *Token bucket vs sliding log?* → Bucket: bursts, O(1) memory; log: exact, stores every timestamp.

Self-check

1. Burst up to capacity, then a steady rate? → Token bucket.
2. Fixed window of 5 across a boundary in 0.2 s? → Up to 10.
3. Pattern making two rules look like one limiter? → Composite.
4. Why must refill + check + take be atomic? → Two threads could both take the last token.
5. Why probe every child before committing? → A request denied by one rule must not use a slot in another.

TL;DR

- **Strategy** algorithms, **Factory** from rules, **Composite** (probe all, commit all), **Facade** service.
- One limiter per (user, api) via `computeIfAbsent`; `Decision` → 200 or 429 + Retry-After.

- Token bucket (bursty, O(1)), fixed window (2× edge burst), sliding log (exact, O(N)), sliding counter (approx, O(1)).
- Atomic refill-check-take (Chapter 13); Redis + Lua when distributed.

20.8 Chapter summary

Component	Anchor pattern(s)	The one sentence to remember
Splitwise	Strategy	Split rules return shares; ledger = one signed number per pair; simplify with net positions + greedy (≤ n − 1).
Ledger Co. (full case in Chapter 21)	—	A loan ledger: same "one authoritative record + `BigDecimal`" idea, different rules.
Notification service	Template Method + Strategy + Observer	Retry once in the parent, one class per reason not to send, one exit that tells listeners.
In-memory cache	Strategy + Observer	HashMap + doubly linked list; policy owns the order; one removal path.
LRU cache	— (data structure)	Map to node, move to front on use, evict before tail; or `LinkedHashMap(accessOrder=true)`.
Ride sharing	Strategy + Decorator + state machine	Ranked list for declines, surge wraps pricing, atomic driver claim.
Carpool variant	Strategy	Offer seats per vehicle; select with Most Vacant / Preferred Vehicle.
Logging framework	Chain of Responsibility + Singleton + Decorator	Hierarchy with inherited levels; propagate up until additivity stops; async is a wrapper.
Pub-sub	Observer via broker	Append-only log + offsets; ack after processing; dead letters.
Rate limiter	Strategy + Factory + Composite	One limiter per key; swappable algorithms; 429 + Retry-After.

Common thread: the part most likely to change sits behind an interface, and the service only orders the steps. "Now add X" → a new class plus a one-line wiring change.

20.9 Chapter self-check

1. Two patterns in more than half the designs? → Strategy and Observer.
2. Strategy vs Decorator? → Strategy *replaces* a rule; Decorator *wraps* one and adds to its result.
3. Strategy vs Template Method? → Strategy swaps the whole algorithm (composition); Template Method fixes it and lets subclasses fill steps (inheritance).
4. Which designs used a "single exit", and why? → Notification (`finish`) and cache (`remove`): records, stats and listeners can't be skipped.
5. Three uses of CAS / `computeIfAbsent` for correctness? → Driver claim (`AtomicBoolean`), subscription commit (`AtomicLong.compareAndSet`), limiter creation (`computeIfAbsent`).

20.10 Chapter TL;DR

- Interviews probe the **rule most likely to change** — put it behind an interface.
- **Strategy** everywhere; **Observer** for "tell someone"; **Decorator** to add behaviour (surge, async); **Template Method** for a shared algorithm with one varying step; **Chain of Responsibility** for the logger hierarchy; **Composite** for combined limits.
- `BigDecimal` money; injected clocks; the smallest correct concurrency tool (one lock, CAS, `computeIfAbsent`, per-key locks, sharding).

- Volunteer the honest limits: greedy isn't optimal, LRU needs a write lock, at-least-once duplicates, fixed windows burst at edges.

Chapter 21 — Case Studies: Machine-Coding Round Problems (Study Edition)

> **Goal.** Chapters 14–20 taught whiteboard design. This chapter trains the **machine-coding round**: a written problem, 90–120 minutes, and **running code** the interviewer feeds their own inputs into. You get the scoring rubric, a skeleton to type from memory, a time plan, and eight real problems solved end to end in Java.

Each problem follows the usual template (problem → requirements → use cases → entities → diagram → flows → decisions → code → edge cases → extensions → interview → self-check → TL;DR). Code is Java 21, standard library only, one package per problem under `code/lld/ch21/<name>/`. From `chapters/21-case-machine-coding/code`:

```
javac --release 21 -d /tmp/lld_21 $(find . -name '*.java')
java -cp /tmp/lld_21 lld.ch21.taskmgmt.Main       # or wallet, calendar, codingblox, propertyhunt, ledger, inventory, filesearch
```

#	Problem	Source	The one idea that makes it work	Patterns
21.1	Task Management System	awesome-low-level-design	Status *state machine* + composable filters	State (enum), Specification, Strategy, Observer
21.2	Digital Wallet Service	awesome-low-level-design	`BigDecimal` money, idempotency keys, lock ordering	Strategy (offers), Facade
21.3	Event Calendar	Flipkart (kumaransg/LLD)	Half-open intervals + a boundary *sweep* for free slots	Strategy, Repository
21.4	Coding Blox	Flipkart (kumaransg/LLD)	Command dispatcher + level-based scoring	Command, Strategy, Repository
21.5	PropertyHunt	ClearTrip (kumaransg/LLD)	Optional criteria → list of predicates; unit normalisation	Builder, Specification, Strategy
21.6	Ledger Co (loans & EMIs)	Navi (kumaransg/LLD, Geektrust)	Derive balances from facts, don't store them	Strategy (interest), Command
21.7	Inventory Management	Common (Amazon/Flipkart-style)	Append-only movement ledger + per-SKU lock	Strategy (allocation), Observer (reorder)
21.8	Unix File Search (`find`)	Common OOD/machine-coding question	Specification filters + Composite AND/OR/NOT; Builder for criteria	Specification, Composite, Builder

(Food ordering "FoodKart" is solved in Chapter 19.)

21.0 How the machine-coding round works

What it is

An OO design interview where **working code is the answer**: a 1–2 page statement, your IDE, 90–120 minutes, no database/UI/framework — data in maps and lists, driven by `main`, a command list or tests. Unlike system design (servers, databases, queues) it never leaves one program; its vocabulary is **classes**. Analogy: whiteboard design *describes* a house; machine coding *builds a shed* while the inspector leans on every wall. Common at Flipkart, Swiggy, PhonePe, Navi, Razorpay, CRED, Meesho, Cleartrip.

How it is judged

Criterion	What is checked → how to score well
Working, demo-able code	Compiles, runs the sample, survives *their* inputs → Thin end-to-end path in the first hour; never finish with non-compiling code
Functional completeness	Every mandatory requirement; bonus after → Implement in the statement's order
Clean models	One job per entity, private fields, invariants inside the entity → Enums for fixed sets, records for values, classes for changing things
Separation of concerns	Parsing, logic, storage apart; no `System.out` in services → Driver → Service → Repository
Extensibility	"Add a new offer/sort/rule" = add a class → Strategy / Specification for "can be X or Y"
Edge cases	Unknown ids, duplicates, invalid input, wrong user → Custom exceptions; driver catches per command and **keeps going**
Readability	Names, small methods, no copy-paste → Nouns → classes, verbs → methods
Concurrency (when asked)	Wallets, inventory, booking → Say where the lock is and why; demo with a pool
Tests (bonus)	Extra credit → An `assert`-style self-check in `main` suffices

The five-step method, and how the clock changes it

Whiteboard method (Chapter 1): **Requirements → Actors & use cases → Classes → Relationships → Code** (roughly 5–8 / 3–5 / 8–10 / 8–10 / 12–15 min in a 45-minute round, code covering only 2–3 key methods). **Machine coding flips the proportions:** steps 1–4 take ~20 minutes, code the rest. A parking lot has ~12 classes but only 3 carry the main flows — build those, name the rest. Also judged: **scope**, **one job per class**, **relationships** (hold/own/extend), **code and model agree**, **a stated trade-off** — never UML notation. Top failures: **nothing runs at the end**, and a **god class** `Manager`.

A reusable project skeleton

```
src/
  model/          entities & value objects      (Task, Wallet, Loan, Property ...)   - enums, records, small classes
  repository/     storage contracts + in-memory (TaskRepository, InMemoryTaskRepository)
  service/        business rules, one per area  (TaskService, WalletService)          - no printing, no parsing
  strategy/       pluggable rules               (Offer, AllocationStrategy, SortBy)
  exception/      domain exceptions             (NotFoundException, InsufficientFundsException)
  driver/         Main + CommandProcessor       (parses input, calls services, prints output)
```

Layer	Knows about	Must NOT know about	Why
model	itself, other models	services, storage, console	Entities guard invariants (`Task.changeStatus` rejects `TODO → DONE`).
repository	model	services, console	Swap `HashMap` for a DB touching only this layer (DIP).
service	model, repository & strategy *interfaces*	concrete repositories, console	The part the interviewer reads most carefully.
strategy	model	services	Each "X or Y" rule → interface + small classes (OCP).
exception	—	—	One class per failure kind, readable message.
driver	services	repository internals	Parsing/printing replaceable; logic isn't.

(This book uses one package per problem; in the round, create the sub-packages.) Reusable blocks: a **generic in-memory repository** (Coding Blox), an `AtomicInteger` **ID generator**, and a **command processor** `Map<String, Consumer<String[]>>` (Command pattern, Chapter 10).

The 90-minute time plan

Minutes	Activity	Output
0–10	Read twice, ask questions, mark mandatory vs bonus, paste sample input into the driver	Numbered requirement list
10–20	Nouns → entities, verbs → methods; pick the 1–2 patterns the statement begs for	Packages, enums, records
20–60	Mandatory requirements **in order**, running after each	Sample input → sample output
60–75	Edge cases, validation, custom exceptions	Driver prints errors and continues
75–85	Bonus (only if all mandatory work); thread-safety for money/stock	Bonus commands
85–90	Clean names, delete dead code, final run, 2-minute walkthrough	Compiling code, demo, story

Run the program at least every 15 minutes — a compile error at minute 88 is a disaster, at minute 30 a 20-second fix.

What to say in the first two minutes

1. "Everything in memory behind a repository interface, so a DB can be added later."
2. "I'll drive it from `main` with the sample commands and print errors without crashing."
3. "Mandatory is A, B, C; bonus D, E. A–C first."
4. "Where does it need to be thread-safe?" (Wallets and inventory: always.)

Common mistakes in this round

Mistake	Do this instead
20 minutes on a perfect class diagram	Enums/records first; the diagram lives in names
One `Manager` with 15 maps	One service per area + repositories
`System.out` inside services	Services return data; the driver prints
`double` for money (`0.1 + 0.2 != 0.3`)	`BigDecimal` with fixed scale (21.2) or whole units (`long` paise)
Crashing on the first bad command	Catch domain exceptions in the driver loop
Bonus before mandatory	Mandatory in order; bonus last
Every class in full detail	Core flow first; stub or name the rest
Unseeded `Random`; `LocalDate.now()`	Seed it / Strategy (21.4); inject a `Clock` (21.1, 21.2)

Real-world examples

Standard Spring Boot layering: controllers = driver, `@Service` = rules, `@Repository` = storage; in-memory repositories are the usual unit-test fakes.

In the interview

> **30-second answer:** "I read the statement twice, list mandatory and bonus requirements and clarify ambiguities. I set up model / repository / service / strategy / exception / driver packages, put the sample input in the driver, and implement requirements one by one, running after each. Services return data and throw domain exceptions; the driver prints and continues. Then edge cases, thread-safety where state is shared, and bonus features."

Why an interface for an in-memory repository? → Services depend on an abstraction; JDBC/Redis can replace it.

Self-check

- **Q:** The six packages? → **A:** model, repository, service, strategy, exception, driver.

- **Q:** Where does `System.out.println` live? → **A:** Only in the driver.
- **Q:** Minute 60, one mandatory requirement missing, bonus tempting? → **A:** Finish the mandatory one.
- **Q:** Why inject a `Clock`? → **A:** Time-based logic gives the same output every run.

> **TL;DR**

Working code first, clean layering second, patterns where the statement asks for variation, graceful errors everywhere. **model → repository → service → strategy → exception → driver**. Run often.

21.1 Design a Task Management System

Problem & clarifying questions

The core of Jira/Asana/Trello: create, update, delete tasks (title, description, due date, priority, status), assign, move through statuses, comment, search/filter (priority, due date, assignee), reminders, task history, safe concurrent access.

Question	Assumed answer
Statuses and transitions?	TODO, IN_PROGRESS, BLOCKED, DONE, CANCELLED; only listed transitions (TODO can't jump to DONE; CANCELLED final; DONE can re-open).
Assignees?	One (nullable).
"Reminder" in a console app?	A method a scheduler calls: notify owners of open tasks due within N days or overdue.
Past due dates?	Not on creation.
Notifications?	A listener interface; the demo prints.
"Task history"?	Per-task audit log (built); a user's finished tasks is just a search: `assignedTo(u).and(withStatus(DONE))`.

> **Requirements**

Functional: register users; create/update/delete tasks; assign; change status via allowed transitions; comment; search with any filter combination (assignee, status, min priority, due before, open only, title contains) + sort (due date, priority, id); reminders; per-task history. **Non-functional:** thread-safe; new filters/sorts/channels without editing code; deterministic (injected `Clock`).

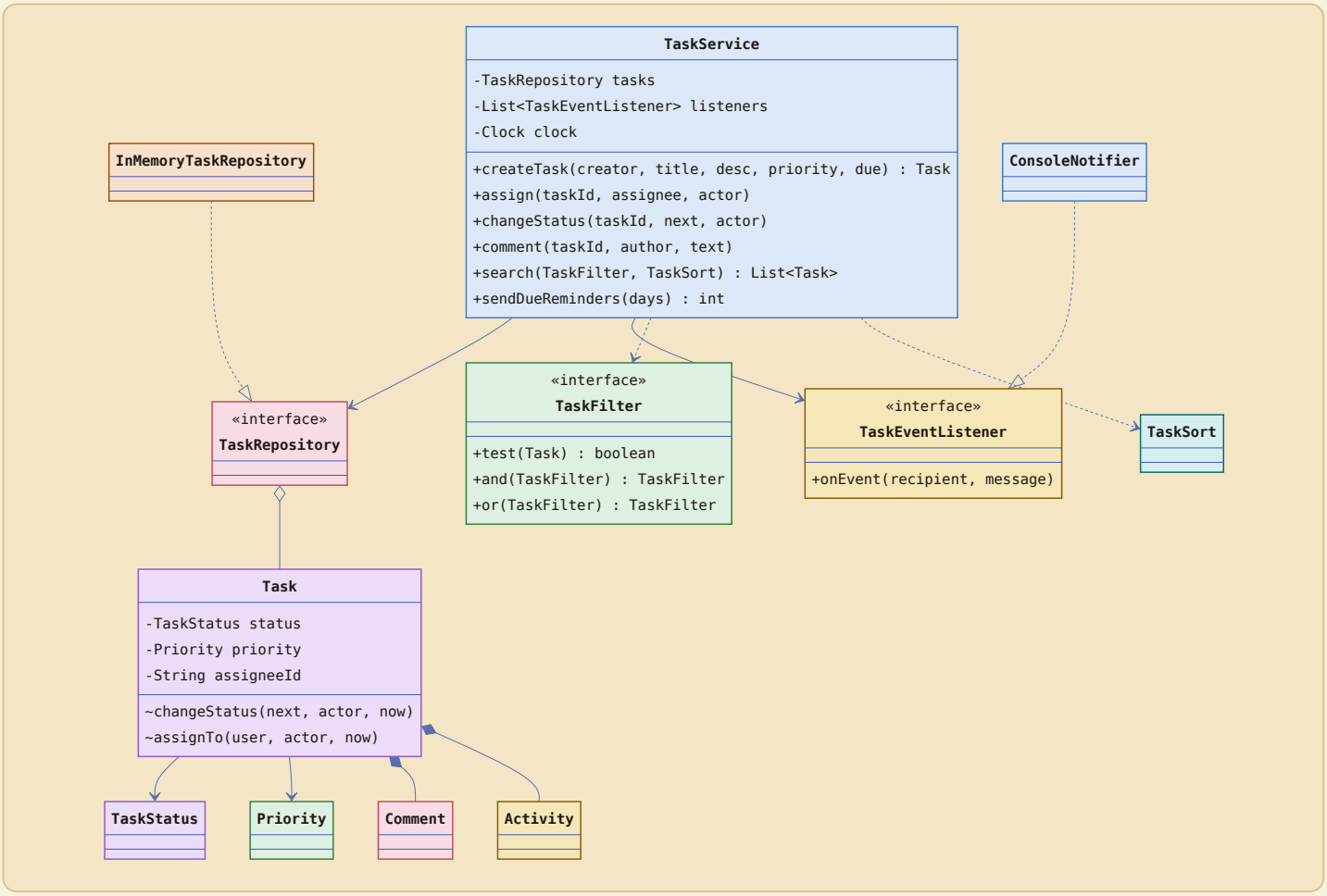
Use cases

Task creator (create, update, delete, assign, comment, search), **Assignee** (change status, comment, see own tasks), **Scheduler** (due-date reminders), **Notification channel** (receives events).

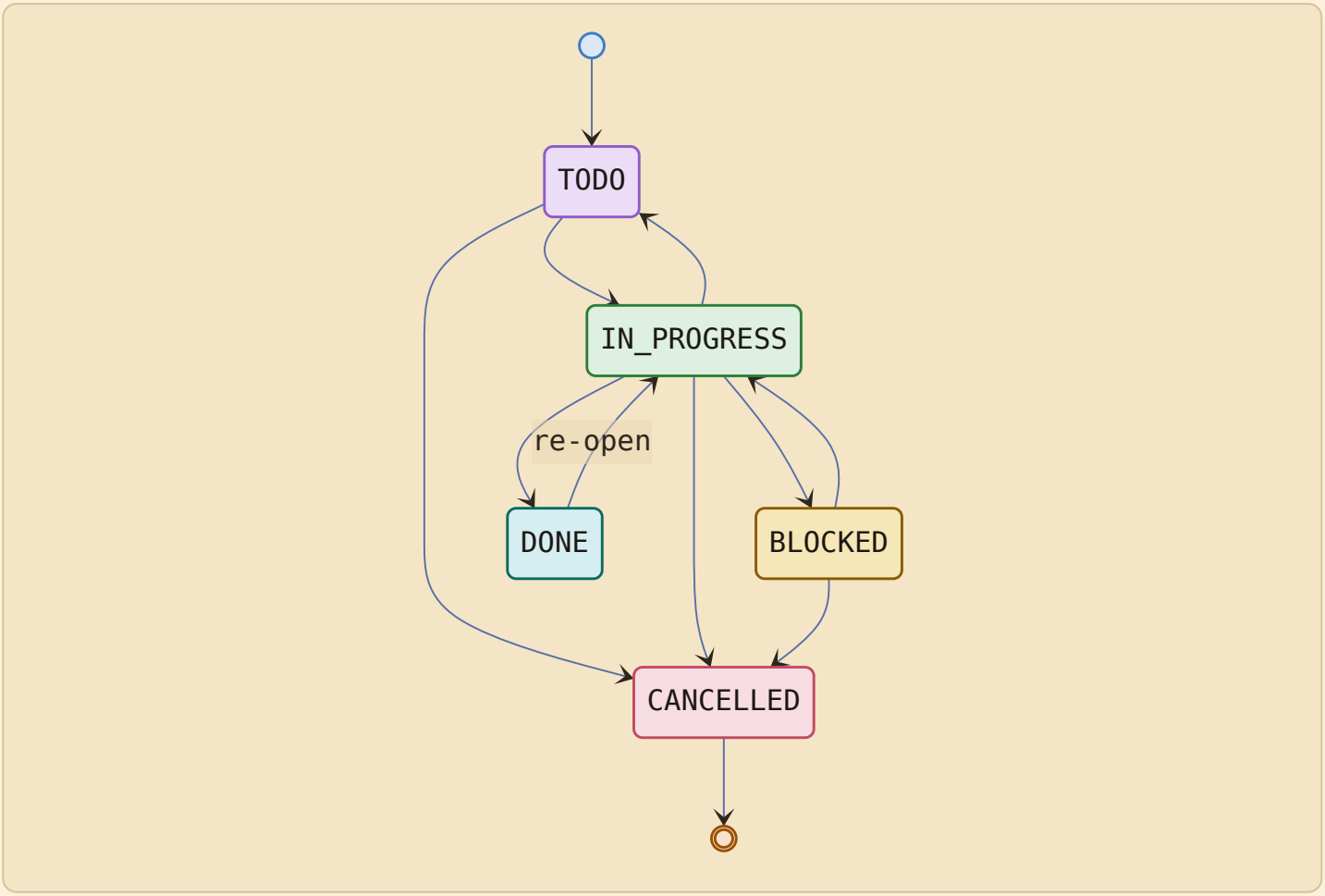
Core entities

Class	Responsibility
`Task`	Data; enforces legal status changes; records its own history; mutators `synchronized`.
`TaskStatus` (enum)	Statuses + allowed next statuses (tiny state machine).
`Priority` (enum)	LOW < MEDIUM < HIGH < CRITICAL (enum order compares).
`User`, `Comment`, `Activity`	Immutable records.
`TaskRepository` / `InMemoryTaskRepository`	Storage contract + `ConcurrentHashMap`.
`TaskFilter`	Composable predicate (**Specification**) with factories and `and` / `or`.
`TaskSort` (enum)	Named comparators (**Strategy**); `BY_ID` sorts `T2` before `T10`.
`TaskEventListener` / `ConsoleNotifier`	**Observer**.
`TaskService`	**Facade**: every use case, validation, clock, events.
`NotFoundException`, `InvalidTransitionException`	Clear failures.

Class diagram



Key flows



Change status: service finds the task → `task.changeStatus(next, actor, now)` → illegal: `InvalidTransitionException` (driver prints it); legal: log activity, set status, notify the creator.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Transitions in the `TaskStatus` enum	State (enum)	One place for rules; `Task` asks `canMoveTo`.	Class per status — overkill with no per-state behaviour.
`TaskFilter extends Predicate<Task>` + `and/or`	Specification	Any combination; new filter = one factory method.	`search(assignee, status, priority, …)` with nullable params.
`TaskSort` enum of comparators	Strategy	Sort order is data.	`switch` on a string.
Listener list	Observer	Email/Slack/push without touching the service.	`System.out` in the service.
`TaskService` single entry	Facade	Simple driver API.	Driver using repositories.
Injected `Clock`	DI	Deterministic reminders/tests.	`LocalDate.now()` scattered.
`synchronized` `Task` mutators	Monitor per entity	Different tasks never block each other.	One global service lock.

Java implementation

```
// File: lld/ch21/taskmgmt/TaskStatus.java
public enum TaskStatus {
    TODO, IN_PROGRESS, BLOCKED, DONE, CANCELLED;

    public Set<TaskStatus> allowedNext() {
        return switch (this) {
            case TODO -> EnumSet.of(IN_PROGRESS, CANCELLED);
            case IN_PROGRESS -> EnumSet.of(BLOCKED, DONE, TODO, CANCELLED);
            case BLOCKED -> EnumSet.of(IN_PROGRESS, CANCELLED);
            case DONE -> EnumSet.of(IN_PROGRESS);          // re-open
            case CANCELLED -> EnumSet.noneOf(TaskStatus.class);
        };
    }

    public boolean canMoveTo(TaskStatus next) { return allowedNext().contains(next); }

    public boolean isOpen() { return this != DONE && this != CANCELLED; }
}

// File: lld/ch21/taskmgmt/Task.java
    synchronized void changeStatus(TaskStatus next, String actor, LocalDateTime now) {
        if (!status.canMoveTo(next)) throw new InvalidTransitionException(id, status, next);
        log(now, actor, "status " + status + " -> " + next);
        status = next;
    }

// File: lld/ch21/taskmgmt/TaskFilter.java
@FunctionalInterface
public interface TaskFilter extends Predicate<Task> {

    static TaskFilter all() { return t -> true; }
    static TaskFilter assignedTo(String userId) { return t -> userId.equals(t.assigneeId()); }
    static TaskFilter withStatus(TaskStatus s) { return t -> t.status() == s; }
    static TaskFilter atLeast(Priority p) { return t -> t.priority().compareTo(p) >= 0; }
    // ... dueOnOrBefore, open, titleContains

    default TaskFilter and(TaskFilter other) { return t -> test(t) && other.test(t); }
    default TaskFilter or(TaskFilter other) { return t -> test(t) || other.test(t); }
}
```

Full runnable code: code/lld/ch21/taskmgmt/ (see the Full Edition).

Output (excerpt):

```
== reminders (due within 3 days)
  [notify carol] Reminder: T4 'Review PR #42' is due 2026-09-28
  [notify bob] Reminder: T3 'Upgrade to Java 21' is due 2026-10-01
== edge cases
  error: Task T2 cannot move from TODO to DONE
  error: Due date is in the past: 2026-09-27
  error: Cannot assign a CANCELLED task T4
```

Self-assignment sends no notification; reminders consider only open tasks; the illegal jump is rejected with a clear message.

Concurrency & edge cases

- Per-task `synchronized` mutators and getters; `ConcurrentHashMap` storage with `findAll()` returning a copy; listeners in `CopyOnWriteArrayList`; ids from `AtomicInteger`.
- Handled: unknown ids, blank title, past due date, illegal transition, assigning a closed task.
- **Not handled (say so):** lost updates on text fields (last writer wins) → optimistic locking with a `version` field.

Extensions & follow-up questions

- **Sub-tasks / dependencies:** `parentId` + `blockedBy`; forbid DONE while a blocker is open.
- **Custom workflows (Jira):** move the transition table into an injected `Workflow` (`Map<Status, Set<Status>>`) per project.

- Also: watchers (`Set<String>`), recurring tasks, `remindAt` + a `ScheduledExecutorService`, secondary indexes or Elasticsearch for search at scale.

Real-world example

Jira projects have *workflows* (statuses + allowed transitions that notify watchers) and JQL search like `assignee = currentUser() AND status != Done ORDER BY priority DESC` — compose filters, then sort, as `TaskFilter` + `TaskSort` do.

In the interview

Present: (1) statuses and transitions — shows you think in invariants; (2) `Task` and `TaskService` API; (3) filters as composable predicates; (4) observer; (5) run the demo. *Why not the full State pattern?* → Statuses have no behaviour besides legal transitions, so an enum table is simpler and still closed to illegal moves.

Self-check

- **Q:** Where is "TODO cannot go to DONE" coded? → **A:** `TaskStatus.allowedNext()`, enforced by `Task.changeStatus`.
- **Q:** Add a "created by X" filter? → **A:** `static TaskFilter createdBy(String u) { return t -> u.equals(t.createdBy()); }` — nothing else changes.
- **Q:** Why a lock per `Task`, not in the service? → **A:** Edits to different tasks never contend.

TL;DR

Task guards its own state machine and history; service = facade; filters = Specification, sorts = Strategy, notifications = Observer; thread-safety per task.

21.2 Design a Digital Wallet Service

Problem & clarifying questions

A wallet like Paytm, PhonePe or a PayPal balance: wallets, top-up, transfer, statement. The awesome-low-level-design statement also lists accounts, payment methods, external transfers, multi-currency, security and **concurrent transactions with consistency**. **Cashback** is added here (common in variants; shows Strategy and double-entry). The interviewer focuses on **money arithmetic**, **no double-spending under concurrency**, and **no double-charging on retries** (idempotency).

Question	Assumed answer
Money type?	`BigDecimal`, scale 2; > 2 decimals rejected, not rounded.
Negative balance?	No.
Client retries after a timeout?	Each request has an **idempotency key**; a repeat returns the original result and moves no money.
Offers?	"10% on first transfer, max 50" and "flat 25 on transfers ≥ 1000"; no stacking — best wins.
Cashback source?	A funded `PROMO` **wallet**; cashback = ordinary transfer PROMO → sender, so money is never created. Budget gone → no cashback, transfer stands.
Multi-currency?	Out of scope (extensions).

Requirements

Functional: create wallet; top up; atomic transfer; statement with running balance; best cashback from the PROMO wallet; idempotent top-up and transfer. **Non-functional:** no lost updates, negative balances or deadlocks; **total money conserved** (only top-ups add money); exact decimals; new offers without touching transfer code.

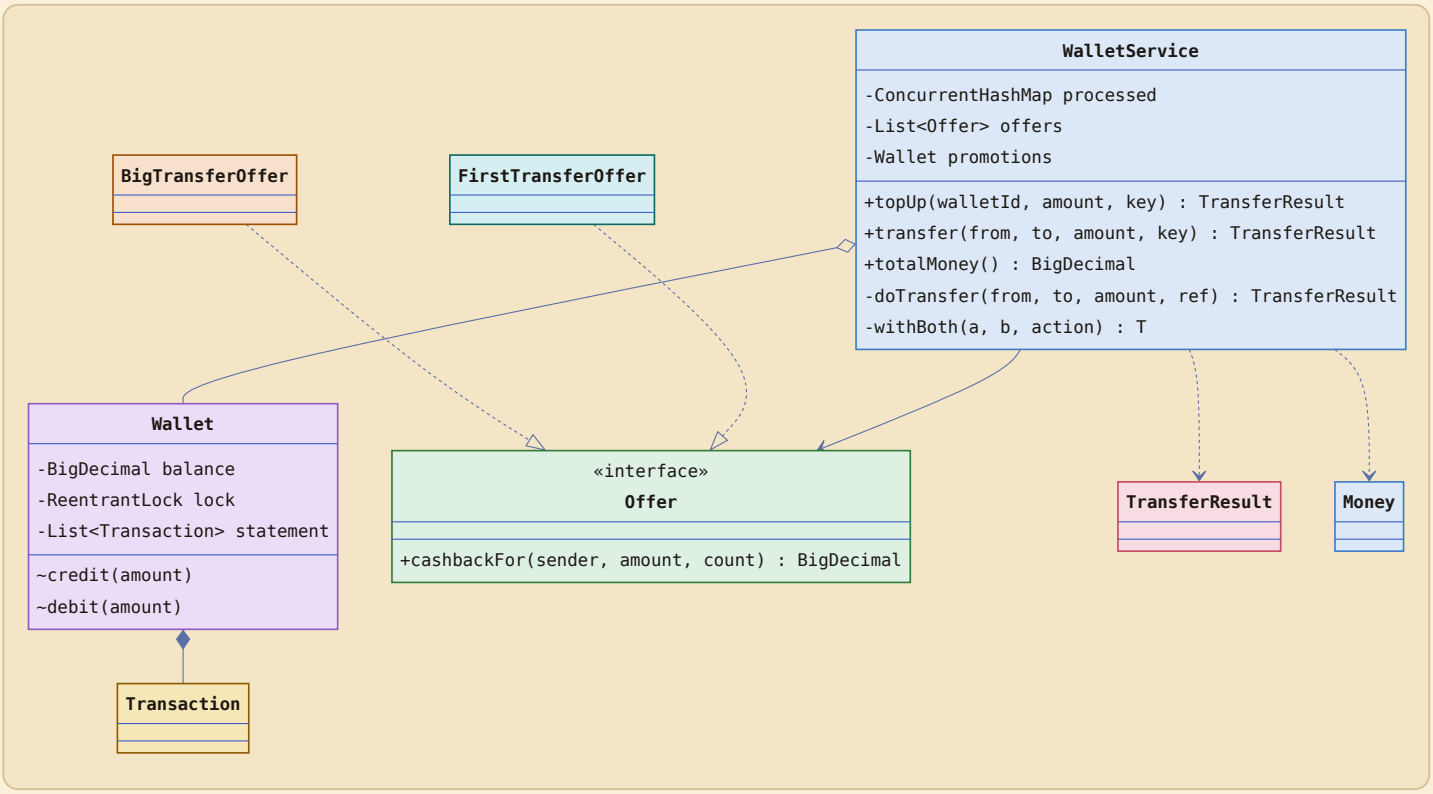
Use cases

User (create wallet, top up, transfer, statement), **Bank/card gateway** (top-up source), **Marketing** (defines offers, funds PROMO), **Client app** (retries with the same key).

Core entities

Class	Responsibility
`Wallet`	Balance + own statement + its `ReentrantLock`; `debit` refuses to go below zero.
`Transaction` (record)	Immutable line: type, amount, balance after, counterparty, reference.
`TxnType`	TOP_UP, TRANSFER_OUT/IN, CASHBACK_OUT (on PROMO)/IN.
`TransferResult` (record)	Stored for idempotent replays.
`Money`	Validates/normalises amounts.
`Offer` + `FirstTransferOffer`, `BigTransferOffer`	**Strategy**: cashback for a completed transfer.
`WalletService`	**Facade**: wallets, lock ordering, idempotency, offers.
`InsufficientFundsException`, `WalletNotFoundException`	Failures.

Class diagram



Key flows

Transfer: validate → `processed.computeIfAbsent(key, doTransfer)` (a known key returns the stored result) → lock both wallets **in id order** → debit, credit, record TRANSFER_OUT/IN → unlock → best offer → second locked step PROMO → sender (PROMO sorts before W1), recording CASHBACK_OUT/IN → store result.

Design decisions & patterns used

Decision	Why	Alternative rejected
`BigDecimal` scale 2, reject > 2 decimals	Exact money; no silent rounding of `1.005`.	`double` (wrong); `long` paise is also fine — say so.
Lock per wallet, acquired in id order	A→B and B→A would deadlock if each locked its source first; a global order makes a cycle impossible.	One global lock (serialises every transfer).
Debit before credit, under both locks	`debit` throws before any change → atomic.	Credit first + compensation.
Idempotency via `computeIfAbsent`	Check + run is one atomic step per key; if it throws, nothing is stored and the client may retry.	`containsKey` then `put` (race).
Cashback from a funded `PROMO` wallet	Double-entry: every credit has a debit; total changes only on top-ups; campaigns can't overspend.	Crediting from thin air.
Cashback as a second step, after the transfer locks	At most two locks at once; transfers don't queue on PROMO.	Locking three wallets per transfer.
`Offer` interface, best wins	New promotions = new classes; no-stacking in one method.	`if/else` on codes.

`computeIfAbsent` **detail:** the call is atomic, the function runs only if absent, an exception records nothing, and other updates to the *same map* may block while it runs — so keep the function short and never update the map inside it (ours only touches wallets).

Java implementation

```
// File: lld/ch21/wallet/WalletService.java
    public TransferResult transfer(String fromId, String toId, BigDecimal amount, String idempotencyKey) {
        if (fromId.equals(toId)) throw new IllegalArgumentException("Cannot transfer to the same wallet");
        BigDecimal amt = Money.requirePositive(amount);
        Wallet from = get(fromId), to = get(toId);
        return processed.computeIfAbsent(idempotencyKey, key -> doTransfer(from, to, amt, key));
    }

    private TransferResult doTransfer(Wallet from, Wallet to, BigDecimal amt, String ref) {
        // Step 1: the transfer itself, under both wallet locks.
        int n = withBoth(from, to, () -> {
            from.debit(amt);                        // throws before any change if funds are short
            to.credit(amt);
            from.record(txn(ref, from, TxnType.TRANSFER_OUT, amt, to.id()));
            to.record(txn(ref, to, TxnType.TRANSFER_IN, amt, from.id()));
            return transferCounts.computeIfAbsent(from.id(), k -> new AtomicInteger()).incrementAndGet();
        });
        // Step 2: cashback is a second, ordinary transfer PROMO -> sender (same lock-ordering rule).
        BigDecimal offered = bestOffer(from, amt, n);
        // ... withBoth(promotions, from, ...): if the budget covers it, debit PROMO, credit sender,
        // ... record CASHBACK_OUT / CASHBACK_IN; otherwise cashback = 0
        return new TransferResult(ref, from.id(), to.id(), amt, cashback);
    }

    /** Locks two wallets, always the smaller id first, so two opposite operations can never deadlock. */
    private static <T> T withBoth(Wallet a, Wallet b, Supplier<T> action) {
        Wallet first = a.id().compareTo(b.id()) < 0 ? a : b;
        Wallet second = first == a ? b : a;
        first.lock().lock();
        try {
            second.lock().lock();
            try { return action.get(); } finally { second.lock().unlock(); }
        } finally { first.lock().unlock(); }
    }
```

Full runnable code: code/lld/ch21/wallet/ (see the Full Edition).

Output (excerpt):

```
  TransferResult[reference=req-3, from=W1, to=W2, amount=200.00, cashback=20.00]
  retry returns same result? true
  error: At most 2 decimal places: 1.005
  failed req-5 can be retried later: 10.00
== concurrency: 1000 opposite transfers on 8 threads
  total before=2400.50 after=2400.50 conserved=true
```

Retried `req-3` moved no money; failed `req-5` stored nothing, so its key worked later; the total was conserved with no deadlock.

Concurrency & edge cases

- Every read-modify-write of a balance holds the wallet's lock; all two-lock paths go through `withBoth`, so the order can't be forgotten; never more than two locks held.
- **Same key, different body** (client bug): today the stored result is returned; production also stores a request hash and rejects mismatches.
- Cashback runs after the transfer's locks are released, so briefly a transfer is visible without its cashback — fine for a reward.

Extensions & follow-up questions

- **Multi-currency:** `Currency` per wallet + `FxRateProvider`; store both amounts and the rate. **Payment methods / payouts:** `PaymentMethod.charge` before crediting; `PENDING_PAYOUT` + `PayoutGateway`. **Refunds:** post a compensating transaction, never edit history.
- **Distributed:** row locks (`SELECT ... FOR UPDATE`) or optimistic versions; idempotency keys under a unique constraint.
- **Slow transfers blocking the map:** `putIfAbsent(key, new CompletableFuture<>())`; winner runs outside the map, duplicates `join()`; remove the key on failure.
- **Limits (daily cap, KYC):** validators in a Chain of Responsibility before `doTransfer`.
- **Why not `AtomicReference<BigDecimal>` CAS?** A transfer changes two balances together; CAS on two variables isn't atomic.

Real-world example

Stripe's `Idempotency-Key` header returns the saved result of the first attempt on a retry instead of charging twice. Paytm/PhonePe cashback rules ("X% up to a cap on your first transaction") are `FirstTransferOffer`.

In the interview

Lead with the three risks (exact money, concurrency, retries — and "cashback comes out of a budget wallet"). Show `Money`, then `doTransfer` with lock ordering (draw the A→B / B→A deadlock), then idempotency; run the concurrent demo and point at `conserved=true`. Offers are a 2-minute Strategy mention.

Self-check

- **Q:** Why `new BigDecimal("0.1")` over `new BigDecimal(0.1)`? → **A:** The double constructor captures the binary approximation; the String one is exact.
- **Q:** W1→W2 and W2→W1 at once — why no deadlock? → **A:** Both lock W1 first.
- **Q:** A transfer fails for insufficient funds; reuse the key? → **A:** Yes — `computeIfAbsent` stores nothing when the function throws.
- **Q:** Where does "max 3 transfers per minute" go? → **A:** A validator before `doTransfer`.

TL;DR

`BigDecimal` money, lock both wallets in a global order, debit before credit, idempotency key → at most once, Strategy offers paid from a funded promotions wallet; prove it with a conserved-total concurrent demo.

21.3 Design an Event Calendar (Flipkart)

Problem & clarifying questions

Flipkart statement (kumaransg/LLD `Event_calendar_flipkart`): users with **working hours**; **teams** (a user in at most one); one-day **events** whose participants are users and/or teams; a user can be in **one event at a time**. For teams the creator says how many **representatives (N)** each must send; the system blocks N *available* members, else the event isn't created. Required: create user, team, event; **a user's events in a time range** (including as representative); **suggest free slots** for users/teams on a day. Bonus: **update event time**, **recurring events**. No DB/UI; "do not make assumptions, ask".

Question	Answer used
Does 14:00–15:00 clash with 15:00–16:00?	No: **half-open** `[start, end)`.
Must events fit every attendee's working hours?	Yes.
Which members represent a team?	First N free members in team order (pluggable strategy).
Direct attendee also in an invited team?	Not picked again as representative.
Free for a team with N?	At least N members working and not busy.
Recurring series with one clash?	Reject the whole series.

The sample is inconsistent. It says A + T1 (N = 1) are free "10AM to 3PM and 4PM to 7PM", but T1's members start at 11:00 and 11:30, so by the rules the answer is `11:00-15:00, 16:00-19:00` (what our program prints). Pointing this out politely is exactly what "ask, don't assume" tests. (It also says "Onboard 5 Users" but lists six.)

Requirements

Functional: create user (working hours); create team (one team per user); create event, rejected if anyone is outside hours, busy, or a team lacks N free members; a user's events in a range; free slots; bonus: atomic update time; recurring daily/weekly. **Non-functional:** "is everyone free?" + booking must be **atomic**; representative choice replaceable.

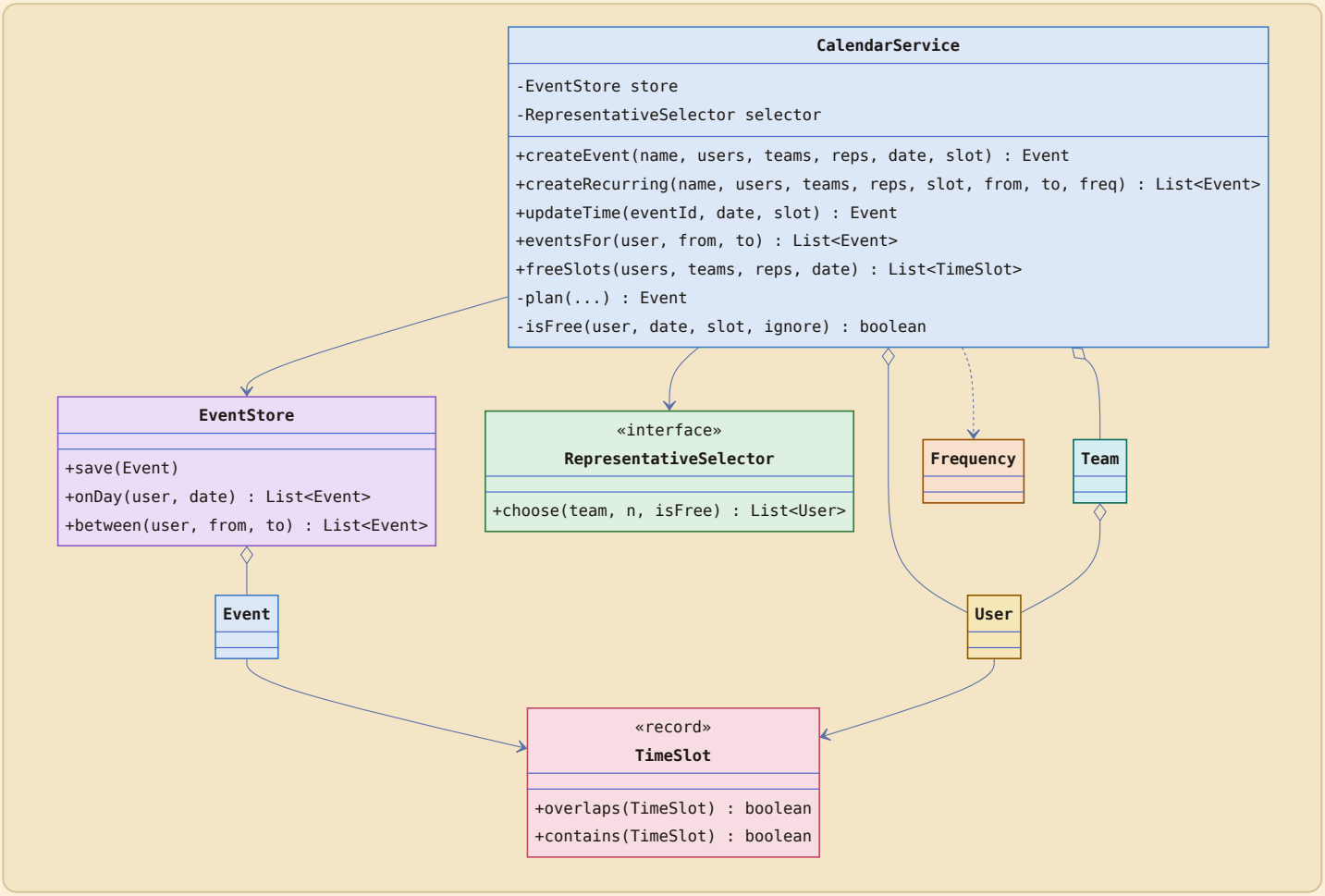
Use cases

Admin (onboard user, create team), **Organiser** (create/recurring/move event, find free slots), **Attendee** (own events in a range).

Core entities

Class	Responsibility
`TimeSlot` (record)	Half-open `[start, end)`; `overlaps`, `contains`; validates start < end.
`User`, `Team` (records)	Name + working hours; name + ordered members.
`Event` (record)	One occurrence: date, slot, request (users, teams, N), actual `attendees`, optional `seriesId`.
`Frequency`	DAILY / WEEKLY, knows the next date.
`EventStore`	Repository + index `user → date → events` (`TreeMap.subMap` range queries).
`RepresentativeSelector`	**Strategy**: pick N free members.
`CalendarService`	Use cases; planning separated from saving.
`CalendarException`	Readable rejection reason.

Class diagram



Key flows

Create event = plan, then save: reject if no participants; each direct user must be inside working hours with no overlapping event; for each team the selector picks N free members not already attending (fewer → reject "team has only k free"); then save and index per attendee.

Free-slot sweep: collect every interesting time that day — each person's working-hour start/end and each of their events' start/end — and sort. Nothing changes between two consecutive cuts, so test each piece once (all direct users free, each team has ≥ N free) and merge neighbouring good pieces. For A + T1 the cuts are 10:00, 11:00, 11:30, 15:00, 16:00, 18:30, 19:00, 19:30 → 11:00–15:00 and 16:00–19:00.

Design decisions & patterns used

Decision	Why	Alternative rejected
Half-open intervals	Back-to-back meetings don't clash; `overlaps` = `a.start < b.end && b.start < a.end`.	Closed intervals.
`plan()` separate from `save()`	Side-effect-free plan reused by create, recurring (plan all, then save all) and update (plan ignoring the old event, then swap) → all-or-nothing for free.	Save as you go + rollback.
`RepresentativeSelector`	Fairness / least-busy pluggable.	Hard-coded loop.
Index `user → TreeMap<date, events>`	Conflict checks read one user's one day; ranges use `subMap`.	Scanning all events.
Boundary sweep	Exact, any N, no minute loops.	Checking every 15/30 min (misses 11:30).
`synchronized` service methods	Check-then-book atomic.	Per-user locks (needs lock ordering; a scale-up).

Java implementation

```java
// File: lld/ch21/calendar/TimeSlot.java
/** Half-open interval [start, end) inside one day. 14:00-15:00 and 15:00-16:00 do not overlap. */
public record TimeSlot(LocalTime start, LocalTime end) {
    // ... compact constructor rejects start >= end
    public boolean overlaps(TimeSlot o) { return start.isBefore(o.end) && o.start.isBefore(end); }
    public boolean contains(TimeSlot o) { return !o.start.isBefore(start) && !o.end.isAfter(end); }
    // ...
}

// File: lld/ch21/calendar/CalendarService.java  (freeSlots: the sweep)
        TreeSet<LocalTime> cuts = new TreeSet<>();
        for (User u : everyone) {
            cuts.add(u.workingHours().start());
            cuts.add(u.workingHours().end());
            for (Event e : store.onDay(u.name(), date)) { cuts.add(e.slot().start()); cuts.add(e.slot().end()); }
        }
        List<TimeSlot> result = new ArrayList<>();
        LocalTime prev = null;
        for (LocalTime cut : cuts) {
            if (prev != null) {
                TimeSlot piece = new TimeSlot(prev, cut);
                boolean ok = direct.stream().allMatch(u -> isFree(u, date, piece, null))
                        && tms.stream().allMatch(t -> t.members().stream()
                                .filter(m -> isFree(m, date, piece, null)).count() >= reps);
                if (ok) {
                    int last = result.size() - 1;
                    if (last >= 0 && result.get(last).end().equals(piece.start()))
                        result.set(last, new TimeSlot(result.get(last).start(), piece.end()));
                    else result.add(piece);
                }
            }
            prev = cut;
        }
        return result;
    // ...
    private boolean isFree(User u, LocalDate date, TimeSlot slot, Event ignore) {
        if (!u.workingHours().contains(slot)) return false;
        return store.onDay(u.name(), date).stream()
                .filter(e -> ignore == null || !e.id().equals(ignore.id()))
                .noneMatch(e -> e.slot().overlaps(slot));
    }
```

Full runnable code: code/lld/ch21/calendar/ (see the Full Edition).

Output (excerpt):

```
  FAILED: Event2: C is not available 2026-09-29 14:00-15:00
  ok: Event3 2026-09-28 15:00-16:00 [C, E, B, D]
  FAILED: Event5: F is not available 2026-09-28 10:00-11:00
== 5. free slots today for A + T1 (1 representative)
  [11:00-15:00, 16:00-19:00]
== bonus: update event time
  FAILED: Event1: team T1 has only 1 free member(s), needs 2
  FAILED: Sync@2026-09-28: A is not available 2026-09-28 15:30-16:30
```

The refused update keeps the old booking; the "Sync" series is rejected as a whole.

Concurrency & edge cases

- All service methods `synchronized` → "is free?" + save is atomic.
- Handled: start ≥ end, unknown user/team, user already in a team, no participants, N < 1, any clash in a series, moving an event onto a slot overlapping *itself* (ignored).
- Out of scope: time zones (real calendars store UTC instants + the user's zone), multi-day events (split into per-day slots).

Extensions & follow-up questions

- **Invites:** `InviteStatus {PENDING, ACCEPTED, DECLINED}`; declined attendees don't block.

- **Fair representatives:** `LeastBusySelector`.
- **Recurring at scale:** store one rule (iCalendar RRULE) and expand lazily per query.
- **Rooms:** a room is a "user" with building hours. **Big teams:** count arrays or a segment tree over minutes.

Real-world example

Google Calendar's "Find a time" and Outlook's Scheduling Assistant intersect attendees' busy blocks the same way, and store recurring meetings as an RRULE plus exceptions.

In the interview

Draw a timeline: free time = working time minus busy time, intersected. One sentence on half-open intervals. Code `TimeSlot`, `plan()`, `createEvent` first (first three requirements), then `eventsFor`, then the sweep. Mention the sample inconsistency. Point out that the bonus features fall out of `plan()`.

Self-check

- **Q:** Why does `plan()` take an `ignore` event? → **A:** When moving an event, its own booking mustn't conflict with the new time.
- **Q:** Why is all-or-nothing recurrence easy? → **A:** Plan every occurrence first, save only if all succeed.
- **Q:** Half-open overlap test? → **A:** `a.start < b.end && b.start < a.end`.
- **Q:** Why a `TreeMap` per user? → **A:** Range query = `subMap(from, to)`.

TL;DR

Half-open `TimeSlot`s, per-user day index, side-effect-free `plan()` reused by create/recurring/update, pluggable representative selector, boundary sweep for free slots — and ask about the contradictory sample.

21.4 Design a LeetCode-like Coding Platform — "Coding Blox" (Flipkart)

Problem & clarifying questions

Flipkart SDE-2 statement (kumaransg/LLD `leetcode-lld-flipkart-coding-blox`). Users sign up, create contests and attend others'. Questions and contests have a level (LOW/MEDIUM/HIGH); a contest uses **all questions of its level**. Command-driven:

Command	Meaning
`CreateUser <name>`	Default score **1500**.
`CreateQuestion <level> <score>` / `ListQuestion [level]`	Ids auto-increment from 1.
`CreateContest <name> <level> <creator>` / `ListContest [level]`	Creator auto-attends.
`AttendContest <contestId> <user>`	Register.
`RunContest <contestId> <creator>`	Creator only. Simulate solved questions (random), sum `contestPoints`, then: LOW → `score + (points − 50)`, MEDIUM → `score + (points − 30)`, HIGH → `score + points`.
`LeaderBoard <asc/desc>`	Users sorted by score.
Bonus `ContestHistory` / `WithdrawContest`	Per-contest results; leave before the run (not the creator).

Expectations: your own sample data, driver or tests, modular OO with interfaces, graceful edge cases, DRY.

Question	Answer used
How many questions solved?	Random 1..all, behind a **seeded** `SolveSimulator`.
Run twice / attend after run?	No (errors) — ratings never applied twice.
Tie-break / empty level?	By name / can't create a contest with no questions of its level.

Requirements

Functional: all commands above including both bonus ones. **Non-functional:** commands from a built-in list; errors printed, processing continues; scoring rule and solving logic replaceable; rating updates safe if two contests finish at once.

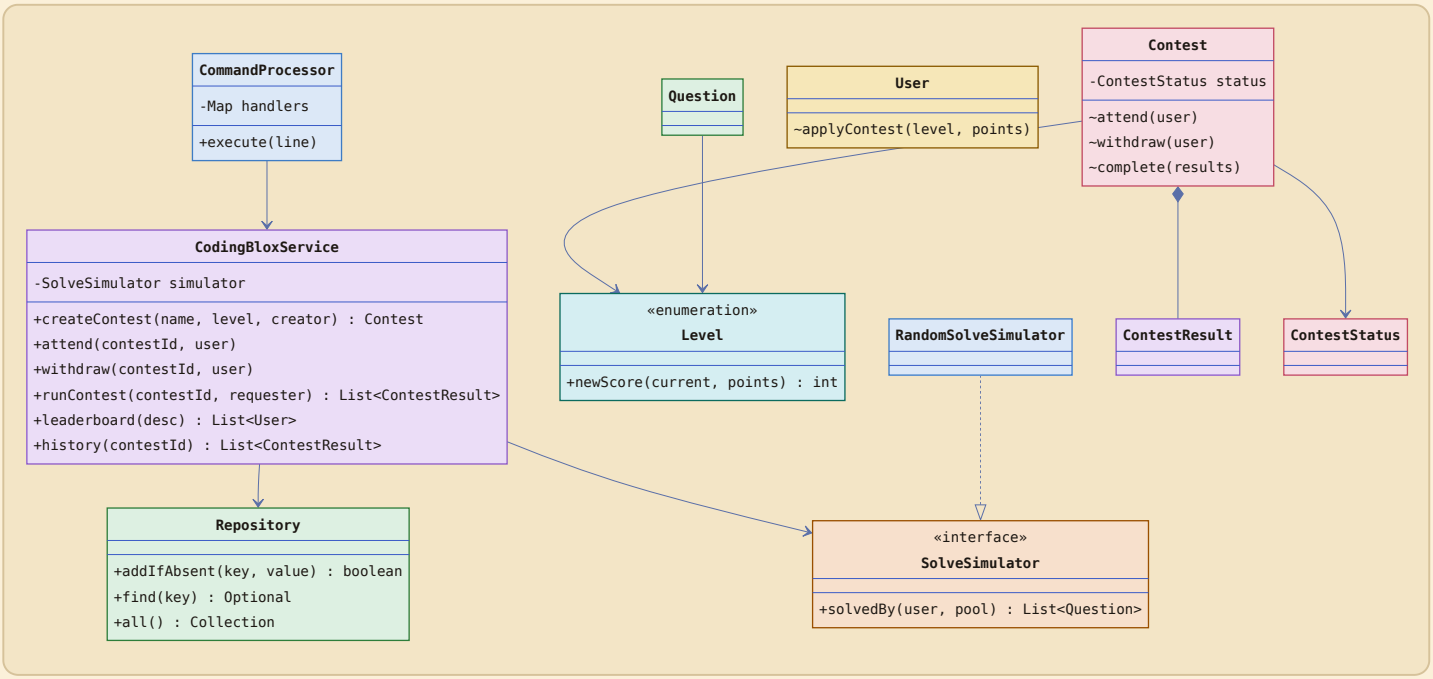
Use cases

User (sign up, attend/withdraw, leaderboard, history), **Contest creator** (create, run), **Admin / problem setter** (questions), **Judge** (simulated: who solved what).

Core entities

Class	Responsibility
`Level` (enum)	LOW/MEDIUM/HIGH with penalty; `newScore(current, points)` is the rating rule.
`User`	Name + score; `applyContest` updates atomically.
`Question`, `ContestResult` (records)	id/level/score; user/points/solved ids.
`Contest` + `ContestStatus`	Participants, OPEN/COMPLETED, results; guards "open only" rules.
`SolveSimulator` / `RandomSolveSimulator`	**Strategy** for "who solved what".
`Repository<K,V>`	Generic in-memory store reused three times.
`CodingBloxService`	Business rules per command.
`CommandProcessor`	Parses a line, dispatches via a map, prints.
`BloxException`	Readable failure.

Class diagram



Key flows

RunContest: requester must be the creator → pool = questions of the contest level → for each participant, `simulator.solvedBy` and sum points → `contest.complete(results)` (throws if already COMPLETED) → only then apply ratings. Contest lifecycle: **OPEN** (attend/withdraw allowed) → **COMPLETED** on `RunContest` by the creator.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Command name → handler map	Command	New command = one `put`; parsing/printing out of the service.	Parsing in the service / giant `switch`.
Scoring rule on `Level`	Strategy (per-constant data)	A new level brings its own rule.	`if (level == LOW)…` in `runContest`.
`SolveSimulator` interface	Strategy	Seeded for demo, scripted for tests, real judge later.	`new Random()` in the service.
`Contest.complete` checks status	Entity guards invariant	"Run once" enforced where state lives.	Check in the driver.
Generic `Repository<K,V>`	Repository	Three stores, one class (DRY).	Three copy-pasted maps.
Results → `complete` → ratings	Safe ordering	If `complete` throws, no rating changed.	Updating ratings in the loop.

Java implementation

```
// File: lld/ch21/codingblox/Level.java
public enum Level {
    LOW(50), MEDIUM(30), HIGH(0);

    private final int penalty;
    Level(int penalty) { this.penalty = penalty; }

    /** newScore = currentScore + (contestPoints - penalty), exactly as the statement defines. */
    public int newScore(int current, int contestPoints) { return current + contestPoints - penalty; }
    // ... parse(String)
}

// File: lld/ch21/codingblox/CodingBloxService.java
    public List<ContestResult> runContest(int contestId, String requester) {
        Contest c = contest(contestId);
        if (!c.creator().equals(requester)) throw new BloxException(requester + " did not create contest " +
contestId);
        List<Question> pool = listQuestions(c.level());
        List<ContestResult> results = new ArrayList<>();
        for (String name : c.participants()) {
            List<Question> solved = simulator.solvedBy(name, pool);
            int points = solved.stream().mapToInt(Question::score).sum();
            results.add(new ContestResult(name, points, solved.stream().map(Question::id).toList()));
        }
        c.complete(results);                                // throws if already run: scores never double-apply
        for (ContestResult r : results) user(r.user()).applyContest(c.level(), r.points());
        return results;
    }

// File: lld/ch21/codingblox/CommandProcessor.java
        handlers.put("runcontest", a -> svc.runContest(Integer.parseInt(arg(a, 1)), arg(a, 2))
                .forEach(r -> out("  " + r.user() + " solved " + r.solvedIds() + " -> " + r.points() + " pts")));
        // ... one entry per command
```

Full runnable code: code/lld/ch21/codingblox/ (see the Full Edition).

Output (excerpt):

```
> RunContest 1 Ross
  Ross solved [2] -> 20 pts
  Monica solved [1, 2] -> 30 pts
  Joey solved [1, 2] -> 30 pts
> RunContest 1 Ross
ERROR: Contest 1 has already run
> LeaderBoard score desc
  Monica : 1540
  Chandler : 1510
  Joey : 1480
  Ross : 1470
```

Contest 1 is LOW: Ross 1500 + (20 − 50) = 1470; Joey/Monica 1480. Contest 2 is MEDIUM: Monica 1480 + (90 − 30) = 1540, Chandler 1500 + (40 − 30) = 1510.

Concurrency & edge cases

- `ConcurrentHashMap` repositories; `synchronized` `Contest` methods; `User.applyContest` under the user's lock, so two contests finishing together can't lose an update.
- **Known gap (say it):** a user attending between the participant copy and `complete` gets no result → add a `RUNNING` status switched atomically with handing back the participant list.
- Handled: duplicates, unknown ids/levels, attend/withdraw after the run, creator withdraw, non-creator run, bad args, unknown command.

Extensions & follow-up questions

- Fixed number of questions: a `QuestionSelectionStrategy` at creation. Real submissions: a judge + `Submission(user, question, verdict, time)`, ICPC-style scoring.
- Elo-style ratings: a `RatingStrategy` over all participants' results. Scheduled contests: `SCHEDULED → RUNNING → COMPLETED` + scheduler.
- Leaderboard at scale: `TreeSet` by (score, name) or a Redis sorted set updated on each change.

Real-world example

LeetCode weekly contests recompute each participant's rating from their relative rank after the contest; HackerRank and Codeforces share the shape: a problem pool, registration, ratings only after finalisation.

In the interview

Paste the sample commands first; build CreateUser → CreateQuestion → CreateContest → AttendContest → RunContest → LeaderBoard, running after each. Name the three moves: command map, scoring on the enum, seeded randomness behind an interface. Bonus commands are tiny because `Contest` stores results and guards status.

Self-check

- **Q:** Why update ratings only after `complete`? → **A:** A second run throws before any rating changes.
- **Q:** Add `VERY_HIGH` with +20 bonus? → **A:** An enum constant with its own rule (e.g. negative penalty).
- **Q:** Deterministic `RunContest` in tests? → **A:** Inject a scripted or seeded `SolveSimulator`.
- **Q:** Where is "creator always attends" enforced? → **A:** The `Contest` constructor.

TL;DR

Command map in the driver, rules in the service, invariants in `Contest`, scoring on `Level`, randomness behind a seeded `SolveSimulator`; complete first, then update ratings.

21.5 Design a Property Listing Site — "PropertyHunt" (ClearTrip)

Problem & clarifying questions

ClearTrip statement (kumaransg/LLD `PropertyHunt`; the longer screenshot version is followed). Users **register**, **list** properties to sell or rent, **view own listings**, **shortlist** others' properties, **view shortlisted** (sold ones show as SOLD), **search** available properties by any combination of **location** (case-

insensitive), **price range**, **type** (sell/rent), **size range**, **rooms** (1/2/3BHK), **sort by** price or size; **mark sold** (owner only); **view sold**. Ranges `X-Y` are inclusive; a single `X` means "at least X" for size and "at most X" for price. **Bonus:** sq ft / sq m / sq yd; **nearby** locations shown last; prices like `40k`, `60L`, `2Cr`; comma-separated values for location and rooms.

Question	Answer used
Input format?	One command per line, fields split by `
Shortlist my own property?	No (the text readme's sample does it — a conflict to ask about).
Shortlist a sold property?	No; but one sold *after* shortlisting stays, shown as SOLD.
Internal units?	Price in whole rupees (`long`), size in sq ft (`double`).
Ordering with a location?	Exact matches, then nearby, then the chosen sort.
Mark sold records buyer/price?	Status only; a `Sale` record is an extension.

The sample `bellandur` search returns a *sarjapur* property — consistent only because sarjapur declares `nearby bellandur|…` (the text readme drops that field).

Requirements

Functional: RegisterUser, ListProperty, Search, Shortlist, ViewShortlisted, ViewListed, MarkSold, ViewSold + all four bonus items. **Non-functional:** filters combine freely; new filters/units without rewriting search; clear errors; concurrent-safe.

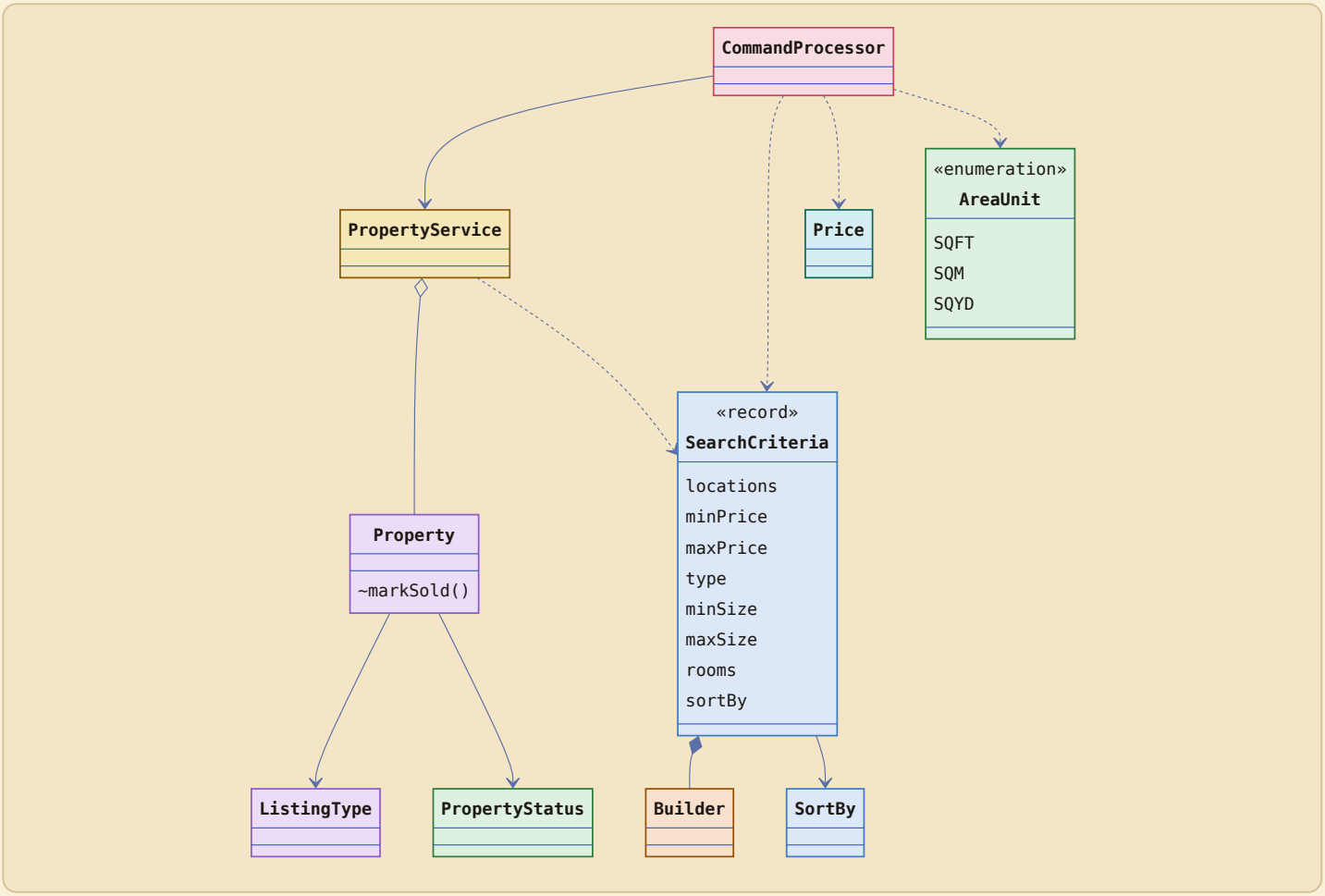
Use cases

Seller / landlord (list, view listed, mark sold, view sold), **Buyer / tenant** (search, shortlist, view shortlisted). Roles are per property, not per account.

Core entities

Class	Responsibility
`Property`	Listing data, owner, nearby locations, status; `markSold` guards double sale.
`ListingType`, `PropertyStatus`, `SortBy`	SELL/RENT, AVAILABLE/SOLD, sort comparators (**Strategy**).
`Price`	Parse `40k/60L/2Cr/1.5Cr` ↔ rupees.
`AreaUnit`	SQFT/SQM/SQYD → sq ft; parses "70 sqm".
`SearchCriteria` + `Builder`	All-optional search parameters (**Builder**).
`PropertyService`	Use cases; owner-vs-buyer rules.
`CommandProcessor`, `HuntException`	Parses `

Class diagram



Key flows

Search: parse fields → normalise with `Price.parse` / `AreaUnit.parseSize` → `SearchCriteria.Builder` → service builds a **list of predicates** (AVAILABLE, location or nearby, price, type, size, rooms) → reduce with `Predicate.and` → sort by (location rank, price or size, id) → driver prints the table. Property lifecycle: AVAILABLE → SOLD (owner only).

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
All-optional criteria with a Builder	Builder	Eight optional fields; eight nulls are unreadable.	Telescoping constructors.
Each criterion → `Predicate`, reduced with `and`	Specification	Any combination; new filter = one line.	`if` per combination.
`SortBy` enum of comparators	Strategy	"Sort by rooms" = one constant.	`switch` in search.
Normalise at the boundary (rupees, sq ft)	—	Business code compares plain numbers.	Comparing strings like "90L".
Location rank 0/1/2	—	One function filters *and* orders (exact, nearby, excluded).	Two searches merged.
Shortlist = set of ids per user	—	Status read live, so SOLD appears automatically.	Copied objects go stale.

Java implementation

```
// File: lld/ch21/propertyhunt/PropertyService.java
    public List<Property> search(String user, SearchCriteria c) {
        requireUser(user);
        Set<String> locs = c.locations() == null ? null : lower(c.locations());
        List<Predicate<Property>> filters = new ArrayList<>();
        filters.add(p -> p.status() == PropertyStatus.AVAILABLE);
        if (locs != null) filters.add(p -> locationRank(p, locs) < 2);
        if (c.minPrice() != null) filters.add(p -> p.price() >= c.minPrice());
        if (c.maxPrice() != null) filters.add(p -> p.price() <= c.maxPrice());
        if (c.type() != null) filters.add(p -> p.type() == c.type());
        if (c.minSize() != null) filters.add(p -> p.sizeSqft() >= c.minSize());
        if (c.maxSize() != null) filters.add(p -> p.sizeSqft() <= c.maxSize());
        if (c.rooms() != null) filters.add(p -> c.rooms().contains(p.rooms()));
        Predicate<Property> all = filters.stream().reduce(x -> true, Predicate::and);

        Comparator<Property> order = locs == null ? c.sortBy().comparator
                : Comparator.<Property>comparingInt(p -> locationRank(p, locs)).thenComparing(c.sortBy().comparator);
        return properties.values().stream().filter(all).sorted(order).toList();
    }
    // ... shortlist(): rejects own, sold and duplicate properties; stores only the id
```

Full runnable code: code/lld/ch21/propertyhunt/ (see the Full Edition).

Output (excerpt):

```
-> Search|Jack|bellandur|null|null|null|null|price
<- 1   A 3BHK for sale      bellandur  90L    1800sqft  3BHK  Sale
<- 2   A 2BHK for rent      sarjapur   40k    1500sqft  2BHK  Rent
-> ViewShortlisted|Jill
<- 1   A 3BHK for sale      bellandur  90L    1800sqft  3BHK  Sale  SOLD
<- 2   A 2BHK for rent      sarjapur   40k    1500sqft  2BHK  Rent  AVAILABLE
-> ListProperty|Jill|Bad|x|abc|sell|100 sqft|1
ERROR: Bad price: abc
```

Nearby sarjapur comes after exact bellandur; a sold property leaves searches but stays (SOLD) in shortlists.

Concurrency & edge cases

- `ConcurrentHashMap`s, concurrent shortlist sets, `synchronized` `markSold` (no double sale).
- A search racing a sale may still show it — fine for listings; a purchase flow re-checks under a lock.
- Handled: unregistered user, unknown property, own/duplicate/sold shortlist, non-owner mark sold, bad price/size/unit/type.

Extensions & follow-up questions

- Descending / multi-key sort: direction flag + `.reversed()`, or chained `thenComparing`.
- Fuzzy locations: a `LocationResolver`. "Within 5 km": lat/long + haversine; at scale a geo-index (geohash, R-tree) or a search engine.
- Millions of listings: secondary indexes or Elasticsearch/OpenSearch. Price-drop alerts: Observer on `Property`.

Real-world example

99acres, MagicBricks and NoBroker filter by locality, budget, BHK, buy/rent and area, suggest nearby localities, and quote prices in lakhs and crores — hence parsing `60L` and `2Cr`.

In the interview

Start with model and units (`Price.parse("90L")`, `AreaUnit`), then register → list → search with *one* filter → add predicates → sort. Call out the Builder + predicate list as the extensibility story and "normalise at the boundary".

Self-check

- **Q:** Buyer searches in sq m, seller listed in sq yd? → **A:** Both converted to sq ft at parse time.

- **Q:** Why shortlist ids, not objects? → **A:** Status is read live.
- **Q:** A single number in the price field means? → **A:** "At most X" (size: "at least X").
- **Q:** Add a "furnished" filter? → **A:** A `Property` field, an optional criterion, one predicate line.

> **TL;DR**

Normalise price and size at the edges, turn each optional criterion into a predicate and AND them, sort by (location rank, key, id), keep shortlists as ids.

21.6 Design a Loan Ledger — "Ledger Co" (Navi)

Problem & clarifying questions

The Geektrust "Ledger Co" challenge (kumaransg/LLD `ledger_company_navi`, used by Navi). Banks lend; borrowers repay in **EMIs** (Equated Monthly Instalments, same amount each month) and may pay a **lump sum** any time.

Command	Meaning
`LOAN BANK BORROWER P N R`	Principal `P`, `N` years, `R`% simple interest per year.
`PAYMENT BANK BORROWER LUMP_SUM EMI_NO`	Lump sum paid right after EMI number `EMI_NO`.
`BALANCE BANK BORROWER EMI_NO`	Print `BANK BORROWER AMOUNT_PAID EMIS_LEFT` after EMI `EMI_NO`.

Rules: `I = P × N × R / 100`; `A = P + I`; EMI = `A / (N × 12)` **rounded up** (86.67 → 87, 100.10 → 101); paid after EMI k = `k × EMI` + every lump sum with EMI number ≤ k; EMIs left = remaining ÷ EMI **rounded up**; the last EMI shrinks to what is owed. A lump sum reduces the *number* of EMIs, never the amount. Only `BALANCE` prints.

Question	Answer used
Paid can exceed `A`?	No — capped at `A` (EMIs left = 0).
Whole-number interest?	Assumed rounded up (say so); official samples are whole either way.
One loan per bank-borrower?	Yes; a second `LOAN` is an error.
`EMI_NO` beyond tenure?	Error; `0` is valid.

Requirements

Functional: LOAN, PAYMENT, BALANCE as above; reproduce the official samples. **Non-functional:** input from a list/file; a bad line doesn't stop the rest; interest rule replaceable; exact `BigDecimal` arithmetic.

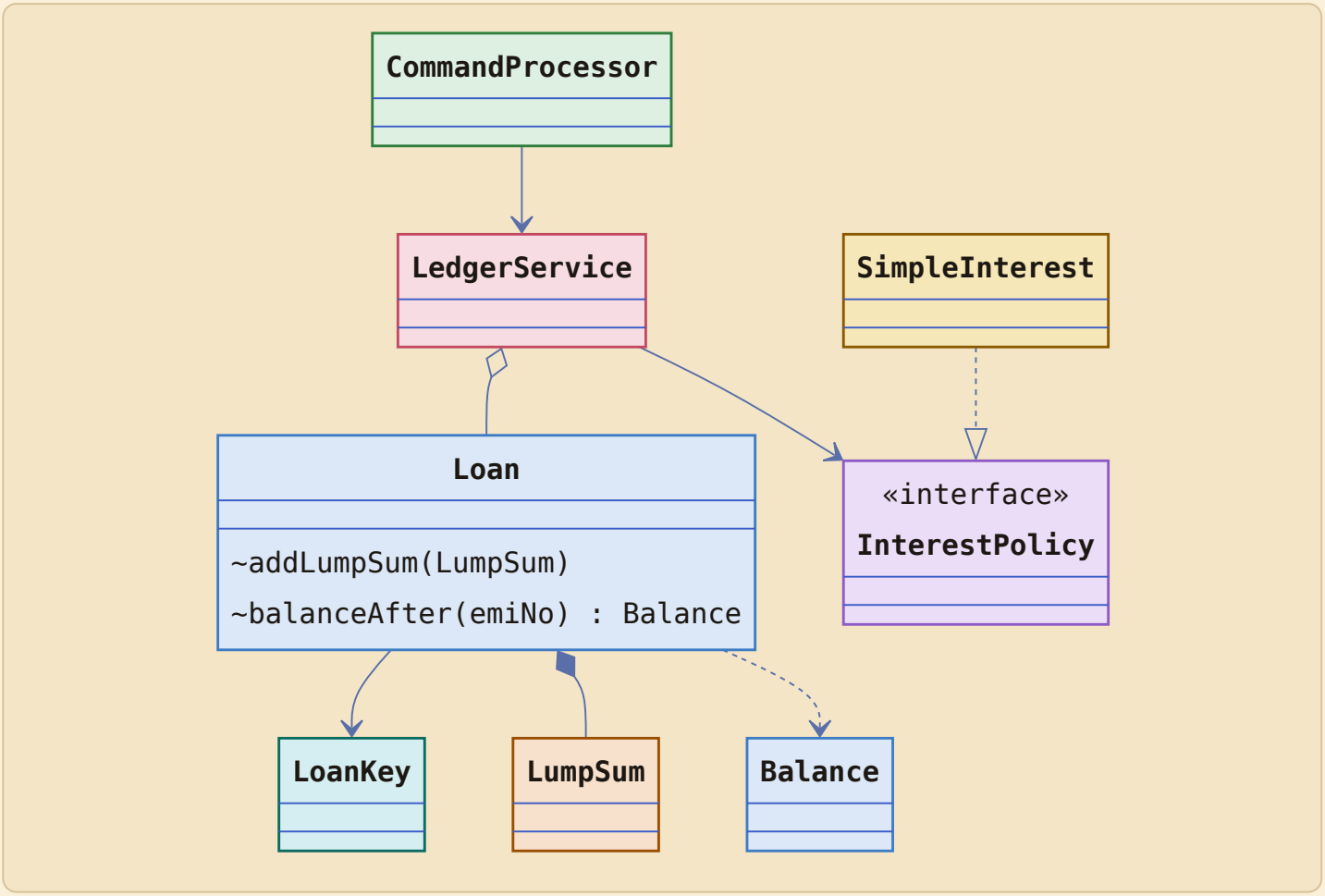
Use cases

Bank (records a loan), **Borrower** (pays lump sums), **Anyone** (balance after EMI k).

Core entities

Class	Responsibility
`LoanKey` (record)	(bank, borrower) — a perfect map key (records give `equals/hashCode`).
`Loan`	Computes `A` and EMI once; stores lump sums; derives a `Balance` for any EMI.
`LumpSum`, `Balance` (records)	amount + after-EMI; output line.
`InterestPolicy` / `SimpleInterest`	**Strategy** for total interest.
`LedgerService`, `CommandProcessor`, `LedgerException`	Loans store; parsing and validation; readable errors.

Class diagram



Key flows

Worked example (Sample 2, Shelly): P = 15000, N = 2, R = 9 → I = 2700, A = 17700, EMI = ⌈737.5⌉ = 738. After 12 EMIs + a 7000 lump sum: paid 15856, remaining 1844, EMIs left ⌈2.499⌉ = 3 → `UON Shelly 15856 3`.

Design decisions & patterns used

Decision	Why	Alternative rejected
Derive balance from facts (A, EMI, lump sums) per query	Queries may ask about *any* EMI, even before a later payment.	A mutable `remaining` field — wrong for earlier EMIs.
`InterestPolicy` Strategy	Reducing-balance or compound later without touching `Loan`.	Formula hard-coded.
`LoanKey` record as key	Value semantics for free.	`"IDIDI#Dale"` strings.
`BigDecimal` + `RoundingMode.CEILING`	Exact "always up".	`Math.ceil` on doubles (4.000000001 → 5).
Driver returns `List<String>`	Easy to compare with expected output.	Printing in the service.

Java implementation

```
// File: lld/ch21/ledger/Loan.java
    Loan(LoanKey key, BigDecimal principal, int years, BigDecimal rate, InterestPolicy policy) {
        if (principal.signum() <= 0 || years <= 0 || rate.signum() < 0)
            throw new LedgerException("Invalid loan terms for " + key);
        this.key = key;
        this.tenureMonths = years * 12;
        this.totalRepayable = principal.add(policy.interest(principal, BigDecimal.valueOf(years), rate));
        this.emi = totalRepayable.divide(BigDecimal.valueOf(tenureMonths), 0, RoundingMode.CEILING);
    }
    // ... addLumpSum validates amount > 0 and EMI number in range

    /** Amount paid after `emiNo` EMIs (lump sums made at or before that EMI count), capped at A. */
    synchronized Balance balanceAfter(int emiNo) {
        if (emiNo < 0 || emiNo > tenureMonths) throw new LedgerException("EMI number out of range: " + emiNo);
        BigDecimal lumps = lumpSums.stream().filter(l -> l.afterEmi() <= emiNo)
                .map(LumpSum::amount).reduce(BigDecimal.ZERO, BigDecimal::add);
        BigDecimal paid = emi.multiply(BigDecimal.valueOf(emiNo)).add(lumps).min(totalRepayable);
        BigDecimal remaining = totalRepayable.subtract(paid);
        int left = remaining.divide(emi, 0, RoundingMode.CEILING).intValueExact();
        return new Balance(key.bank(), key.borrower(), paid, left);
    }
```

Full runnable code: code/lld/ch21/ledger/ (see the Full Edition).

Output (excerpt) — the Sample 1 and 2 blocks match the official Geektrust expected outputs:

```
== Sample 2
IDIDI Dale 1326 9
IDIDI Dale 3652 4
UON Shelly 15856 3
MBI Harry 9044 10
== Edge cases
SBI Asha 1120 0
ERROR EMI number out of range: 13  [BALANCE SBI Asha 13]
ERROR Not a number: ten  [PAYMENT SBI Asha ten 1]
```

Asha's 2000 lump sum overpays a 1120 loan, so paid is capped at 1120 and 0 EMIs remain.

Concurrency & edge cases

- `ConcurrentHashMap` + `putIfAbsent` makes "one loan per pair" atomic; `synchronized` `Loan` methods keep PAYMENT and BALANCE apart.
- Handled: duplicate/unknown loan, EMI out of range, invalid terms, overpayment (capped), bad arguments, unknown command.

Extensions & follow-up questions

- **Reducing-balance EMI:** new policy, `EMI = P·r·(1+r)^n / ((1+r)^n − 1)`; `Loan` API unchanged.
- **Lump sum that reduces the EMI instead:** a `PrepaymentPolicy` (REDUCE_TENURE / REDUCE_EMI).
- **Late payments:** record dated payment events and derive EMIs paid. **File input:** `Files.readAllLines` into the same `run`.

Real-world example

Navi, banks and NBFCs show an EMI schedule and allow part-prepayment, usually choosing between fewer EMIs (this puzzle's rule) or a smaller EMI. Real loans use reducing-balance interest, not flat simple interest — worth raising.

In the interview

Show the three formulas and compute one sample by hand (proves you understood the rounding); code `Loan.balanceAfter` (the heart), the service, the command loop; run both samples. Emphasise "state = facts, balances = derived".

Self-check

- **Q:** Why ceil the EMI? → **A:** A floor leaves a shortfall; rounding up guarantees the EMIs cover `A`.

- **Q:** Does `BALANCE … 5` include a PAYMENT after EMI 10? → **A:** No; only `afterEmi ≤ 5`.
- **Q:** Why compute the balance rather than store it? → **A:** Queries arrive for any EMI in any order.
- **Q:** Compound interest? → **A:** A new `InterestPolicy` injected into `LedgerService`.

> **TL;DR**

EMI = ⌈(P + PNR/100) / 12N⌉. Store loan facts and lump sums; derive paid and EMIs-left on demand with ceiling rounding; interest is a Strategy; the driver is a tiny command loop.

21.7 Design an Inventory Management System

Problem & clarifying questions

A common e-commerce / quick-commerce question: track **SKUs** (stock keeping units — one id per sellable item) across **warehouses**, record every **stock movement** (received, shipped, transferred, adjusted after a physical count), fulfil orders by choosing warehouses, and raise **reorder alerts**.

Question	Answer used
Stock per warehouse?	Yes: (SKU, warehouse) → quantity.
Split orders?	Yes; warehouse order is a strategy (nearest city first / most stock first).
Partial shipments?	No — all-or-nothing.
Reorder alert when?	*Total* stock ≤ reorder point; once per dip, re-armed on recovery.
Reorder how much?	`max − current`.
Negative stock?	Never.

> **Requirements**

Functional: add products (reorder point, max level) and warehouses; receive; ship by strategy; transfer; cycle-count adjust; stock per warehouse and total; SKU movement history; reorder alerts. **Non-functional: no overselling** under concurrency; different products in parallel; full audit trail.

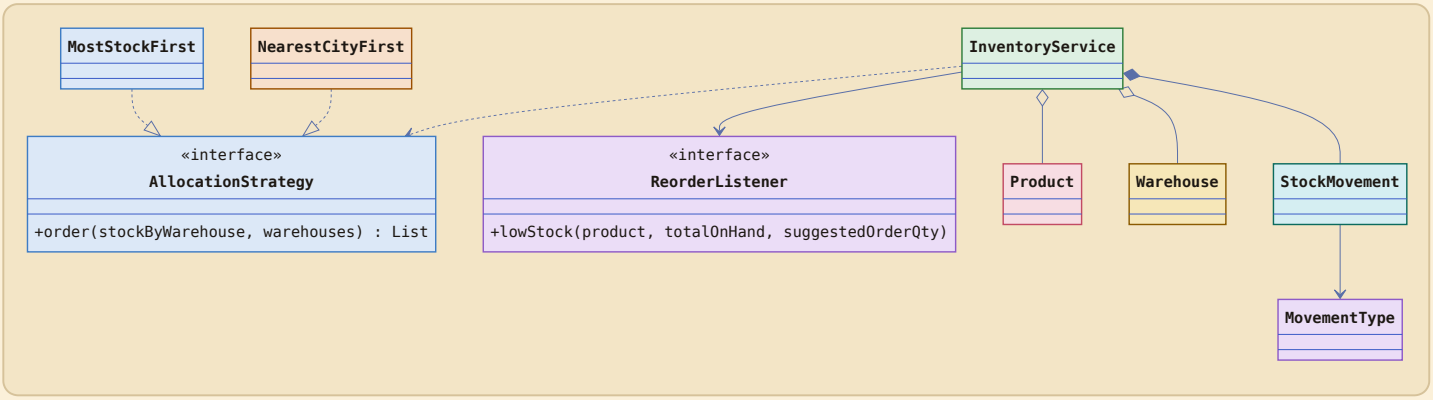
Use cases

Warehouse staff (receive, transfer, adjust), **Order service** (ship), **Purchasing** (reorder alerts), **Manager** (stock and history).

Core entities

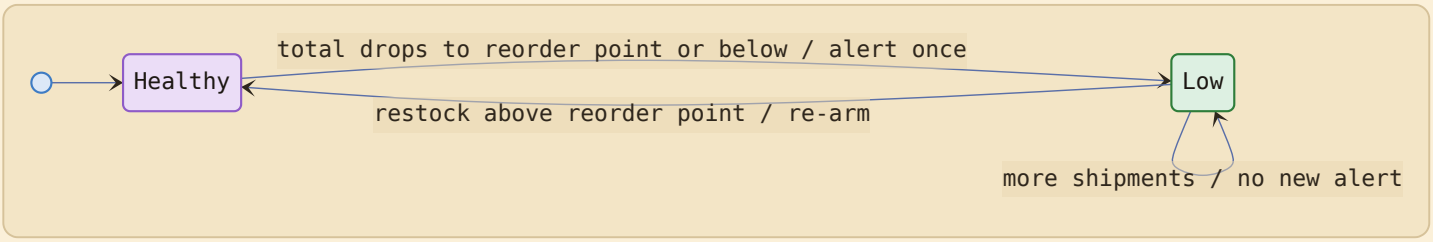
Class	Responsibility
`Product`, `Warehouse` (records)	SKU, name, reorder point, max level; id, city.
`StockMovement` + `MovementType`	Append-only ledger line: INBOUND, OUTBOUND, TRANSFER, ADJUSTMENT.
`Allocation` (record)	"take q from warehouse w".
`AllocationStrategy` + `NearestCityFirst`, `MostStockFirst`	**Strategy**: order to drain warehouses.
`ReorderListener`	**Observer** for low stock.
`InventoryService`, `InventoryException`	Levels, per-SKU locks, movements, alerts; readable errors.

Class diagram



Key flows

Ship: lock the SKU → total < qty? reject, nothing changed → strategy orders a *snapshot* of the levels → build the full plan → apply it, writing OUTBOUND movements → `checkReorder` → unlock.



Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
One lock per SKU	Lock striping by key	Covers all of a SKU's warehouses with no lock-ordering puzzle; different SKUs never contend.	Lock per (SKU, warehouse) — needs ordering; one global lock.
Plan, then apply	—	Check total and build the full plan before changing anything → all-or-nothing.	Deduct and roll back.
`AllocationStrategy`	Strategy	Speed vs fewer split parcels as small classes.	`if (mode == …)`.
Append-only movement ledger	Event log / audit	Every change explainable; levels rebuildable; adjustments record the delta.	Quantities with no history.
Once-per-dip `ReorderListener`	Observer + edge-triggering	Notified once on crossing, not every sale.	Alert on every movement (spam).
Strategy gets a snapshot	Defensive copy	Can't modify live stock.	Passing the live map.

Java implementation

```
// File: lld/ch21/inventory/InventoryService.java
    /** All-or-nothing: either the whole quantity is allocated across warehouses, or nothing moves. */
    public List<Allocation> ship(String sku, int qty, AllocationStrategy strategy, String orderRef) {
        positive(qty);
        return withSku(sku, () -> {
            Map<String, Integer> levels = stock.get(sku);
            int total = levels.values().stream().mapToInt(Integer::intValue).sum();
            if (total < qty) throw new InventoryException("Only " + total + " of " + sku + " available, order needs "
+ qty);
            List<Allocation> plan = new ArrayList<>();
            int remaining = qty;
            for (String wh : strategy.order(Map.copyOf(levels), warehouses)) {
                int take = Math.min(remaining, levels.get(wh));
                if (take > 0) { plan.add(new Allocation(wh, take)); remaining -= take; }
                if (remaining == 0) break;
            }
            for (Allocation a : plan) {
                change(sku, a.warehouseId(), -a.quantity());
                record(MovementType.OUTBOUND, sku, a.warehouseId(), null, -a.quantity(), orderRef);
            }
            return plan;
        });
    }
    // ...
    /** Fires once when stock crosses down to the reorder point; re-arms when stock recovers. */
    private void checkReorder(String sku) {
        Product p = products.get(sku);
        int total = totalUnlocked(sku);
        if (total <= p.reorderPoint()) {
            if (alerted.add(sku)) listeners.forEach(l -> l.lowStock(p, total, p.maxLevel() - total));
        } else {
            alerted.remove(sku);
        }
    }
    // ... withSku(sku, action): look up the SKU's ReentrantLock, lock, run, unlock
```

Full runnable code: code/lld/ch21/inventory/ (see the Full Edition).

Output (excerpt):

```
== ship 32 phones to a Bengaluru customer (nearest city first)
  [Allocation[warehouseId=BLR-1, quantity=30], Allocation[warehouseId=BLR-2, quantity=2]]
== ship 25 more (most stock first) -> crosses reorder point
  [ALERT] PHONE low: 18 left, reorder 82 to reach 100
  ship 1 more (no duplicate alert): [Allocation[warehouseId=BLR-1, quantity=1]]
== concurrency: 200 one-unit orders race for 50 cases
  shipped=50 rejected=150 left=0
```

The alert fires once at 18 (≤ 20), not again at 17; exactly 50 of 200 concurrent orders ship — no overselling.

Concurrency & edge cases

- Check → plan → apply under the SKU's `ReentrantLock`; different SKUs in parallel.
- **Ledger** = `ConcurrentLinkedQueue` (lock-free appends, order kept, no `ConcurrentModificationException`); a `CopyOnWriteArrayList` would copy the whole log on every movement. **Listeners** = `CopyOnWriteArrayList` (read often, changed rarely); they run under the SKU lock — hand slow ones (email) to an executor.
- Handled: unknown SKU/warehouse, bad quantities, same-warehouse transfer, orders larger than stock, zero-change adjustments (not logged).

Extensions & follow-up questions

- **Reservations (cart holds):** `onHand` and `reserved`; available = onHand − reserved; holds expire; shipping converts a reservation.
- **Batches / expiry:** FEFO (first-expiry-first-out) is just another `AllocationStrategy`.
- **Auto purchase orders:** a listener creating a pending PO for `max − current`.

- **Database / many services:** optimistic updates (`UPDATE ... SET qty = qty - ? WHERE qty >= ? AND version = ?`); ledger as an event stream (Kafka).

Real-world example

Amazon and Flipkart choose the fulfilment centre by proximity and stock and may split an order (`AllocationStrategy`); Shopify has multi-location fulfilment priorities; quick-commerce dark stores (Blinkit, Zepto, Instamart) depend on accurate per-store stock and fast reorder alerts.

In the interview

Start with the data shape: `stock[sku][warehouse] = qty` + an append-only movement list. Code receive and ship (plan-then-apply, per-SKU lock) first — most of the marks; then transfer/adjust, the once-per-dip observer, allocation strategies; run the 200-thread demo.

Self-check

- **Q:** Why check the total before planning? → **A:** Reject short orders before touching any warehouse.
- **Q:** Why is a per-SKU lock enough for multi-warehouse orders? → **A:** An order has one SKU; its lock covers all its warehouses.
- **Q:** Add FEFO? → **A:** Batch-level stock + a new `AllocationStrategy`; `ship` unchanged.
- **Q:** Why log adjustments as movements? → **A:** Shrinkage is auditable and levels rebuildable.

TL;DR

Stock per (SKU, warehouse), every change a ledger movement, a lock per SKU prevents overselling, allocation is a Strategy, low-stock alerts are an edge-triggered Observer.

21.8 Design a Unix File Search (`find`)

Problem & clarifying questions

Build a small `find` (e.g. `find /var/log -type f -size +10M -mtime +7`): a file-system model, **filters**, a way to **combine** them with AND / OR / NOT, and a clean API for search options. The trap to avoid: one method with ten optional parameters. Terms: a **predicate** is a yes/no test on one entry; a **glob** is a wildcard name pattern (`*`, `?`, `[0-9]`); **depth** = levels below the start (start = 0).

Question	Answer used
Real disk?	In-memory tree (deterministic); `java.nio` adapter is an extension.
Tests?	name / iname glob, type, size, mtime, owner, empty.
Combining?	AND, OR, NOT, parentheses; precedence NOT > AND > OR; side-by-side tests = AND.
Size semantics?	GNU: round **up** to the unit, then compare — `-size -1M` matches only empty files. Files only.
Options?	min/max depth, limit, **prune** (skip e.g. `node_modules`).
API?	Fluent **Builder** + a `find` command-line parser.
Order / huge trees?	Pre-order DFS, children by name; stream results; no recursion; parallel variant.

Requirements

Functional: model files/dirs (name, owner, mtime, size); search with any AND/OR/NOT combination; depth, limit, prune; parse `find` lines; clear errors. **Non-functional:** Open/Closed (new test = one class); streaming, deep trees; searching while files are added must not throw.

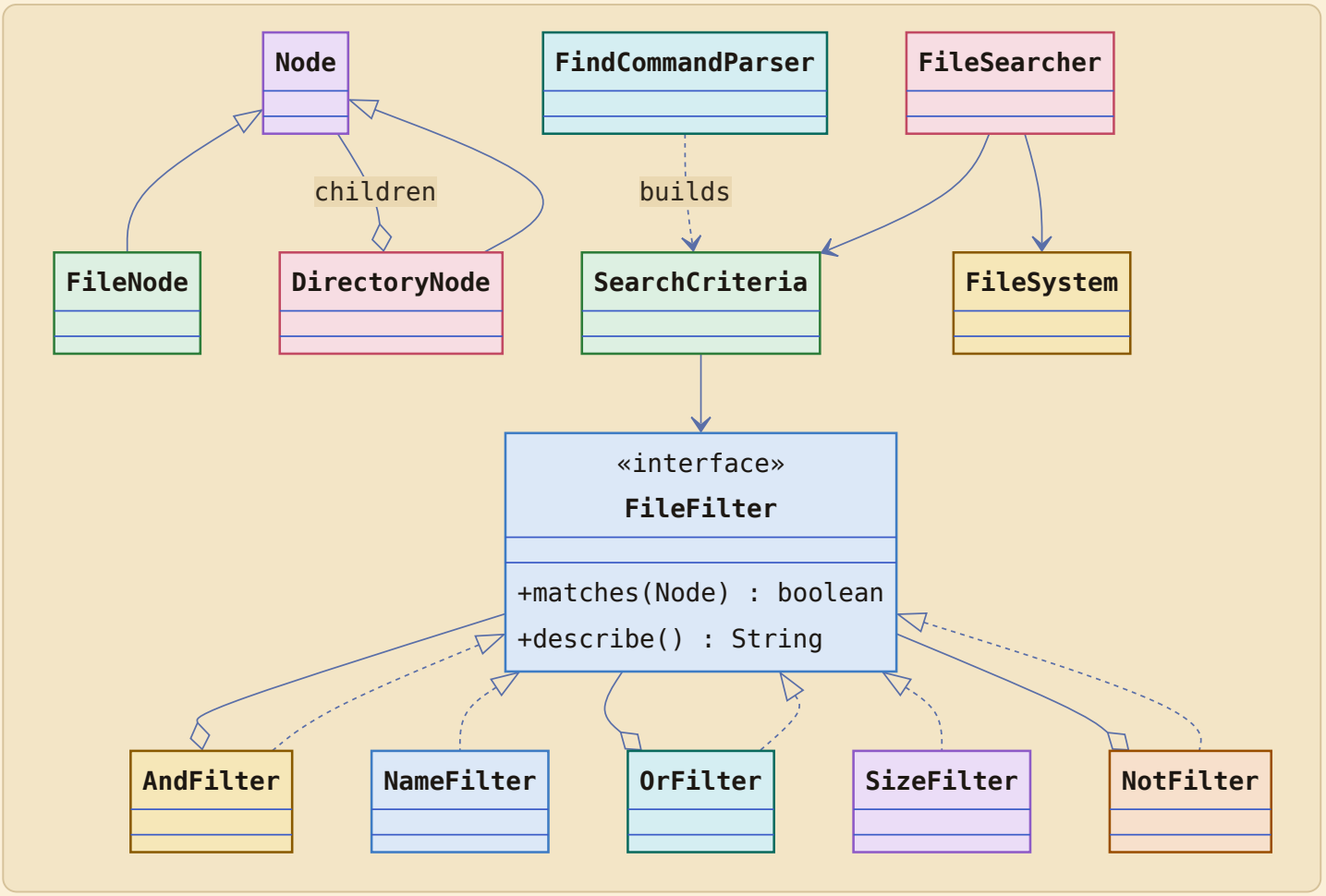
Use cases

Developer (Builder, search, stream), **Shell user** (type a command, get paths or an error), **Ops script** (big old logs, empty directories).

Core entities

Class	Responsibility
`Node` (sealed) → `FileNode`, `DirectoryNode`	**Composite** file tree; `path()`, `size()`, `totalSize()`; children in a sorted `ConcurrentSkipListMap`.
`FileSystem`	Root, `mkdirs`, `createFile`, `resolve`.
`FileFilter`	**Specification**: `matches`, `describe`, default `and` / `or` / `negate`.
`AndFilter`, `OrFilter`, `NotFilter`	**Composite** filter tree.
`Filters`	Leaf tests (name, iname, type, size, mtime, owner, empty, all).
`SearchCriteria` + `Builder`	Immutable, validated search description.
`FileSearcher`	Iterative DFS, streams to a `Consumer`; fork/join variant.
`FindCommandParser`	Recursive descent → `SearchCriteria`.

Class diagram



Two Composites: the **file tree** and the **filter tree**; the searcher walks one and asks the other one question per entry.

Key flows

`find /home ( -name '*.jpg' -o -name '*.png' ) ! -size +1M` parses to **AND(OR(name *.jpg, name *.png), NOT(size > 1M))**.

Search loop: push (start, 0) → pop → if depth ≥ minDepth and filter matches, emit → if directory, depth < maxDepth and not pruned, push children in reverse name order → repeat until empty or limit.

Precedence: `-o` < AND (`-a` or nothing) < `!` < `( )`. So `-name a -o -name b -type f` = `a OR (b AND file)`.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
One small class per test	**Specification**	New test = one record; loop unchanged	`search(name, ext, minSize, …)` with `if`s
And/Or/Not hold filters	**Composite**	Arbitrary nesting; printable, inspectable, parser output	Lambdas only (can't print; cf. §21.1)
`Node` / `FileNode` / `DirectoryNode`	**Composite**	Uniform walk and `totalSize()`	`instanceof` everywhere
`SearchCriteria.Builder`	**Builder**	Named optional settings, validated once, immutable	Telescoping constructors
Recursive-descent parser	Interpreter-style	One method per precedence level	String splitting
Explicit stack, `Consumer` output	Streaming	No stack overflow; constant memory	Recursion; always a full list
Short-circuit AND/OR	—	Cheap tests first	Evaluate every test
`ConcurrentSkipListMap` children	Concurrent collection	Sorted, safe to iterate during writes	`TreeMap` (CME)

Java implementation

```java
// File: lld/ch21/filesearch/FileFilter.java
public interface FileFilter {
    boolean matches(Node n);
    String describe();
    default FileFilter and(FileFilter other) { return AndFilter.of(this, other); }
    default FileFilter or(FileFilter other) { return OrFilter.of(this, other); }
    default FileFilter negate() { return new NotFilter(this); }
}
```

```java
// File: lld/ch21/filesearch/Filters.java  (SizeFilter.matches)
            if (node.type() != FileType.FILE) return false;
            long units = (node.size() + unitBytes() - 1) / unitBytes();      // round up
            return switch (cmp) { case '+' -> units > n; case '-' -> units < n; default -> units == n; };
```

```java
// File: lld/ch21/filesearch/FileSearcher.java
    public int search(SearchCriteria c, Consumer<Node> out) {
        Deque<Frame> stack = new ArrayDeque<>();
        stack.push(new Frame(fs.resolve(c.startPath()), 0));
        int found = 0;
        while (!stack.isEmpty() && found < c.limit()) {
            Frame f = stack.pop();
            Node n = f.node();
            if (f.depth() >= c.minDepth() && c.filter().matches(n)) { out.accept(n); found++; }
            if (n instanceof DirectoryNode d && f.depth() < c.maxDepth() && !c.pruneNames().contains(d.name())) {
                List<Node> kids = new ArrayList<>(d.children());
                Collections.reverse(kids);                    // push in reverse so we pop in name order
                for (Node k : kids) stack.push(new Frame(k, f.depth() + 1));
            }
        }
        return found;
    }
```

```java
// File: lld/ch21/filesearch/FindCommandParser.java
    private FileFilter parseOr() {
        FileFilter left = parseAnd();
        while (peekIs("-o", "-or")) { pos++; left = left.or(parseAnd()); }
        return left;
    }
    // parseAnd: unary ( [-a] unary )* ; parseUnary: ! unary | ( expr ) | -name G | -size N | ...
```

Full runnable code: code/lld/ch21/filesearch/ (see the Full Edition).

How it works: globs compile once to a regex on the last path component; `-size` accepts `[+-]N[cbkMG]` (no suffix = 512-byte blocks); `-mtime` counts whole days against an injected `now`; each Builder `where` is ANDed; the parser skips `-maxdepth`/`-mindepth` at every level; `searchParallel` forks one task per sub-directory and joins in name order — identical results.

Output (excerpt):

```
$ find /home ( -name '*.jpg' -o -name '*.png' ) ! -size +1M
  parsed: from /home where ((name='*.jpg' OR name='*.png') AND NOT size+1M)
   /home/asha/photos/c.png
   /home/asha/photos/d.jpg
$ find /home -size -1M
  parsed: from /home where size-1M
   /home/ravi/todo.md
$ from /home/asha/projects where (name='*.java' OR name='*.js') prune=[node_modules]
   /home/asha/projects/app/Main.java
   /home/asha/projects/app/Util.java
   find / ( -name a                  -> Missing ')'
   find / -maxdepth 1 -mindepth 3    -> mindepth 3 > maxdepth 1
   sequential=1600 parallel=1600 same order=true first=/data/d000/sub0/f060.csv last=/data/d199/sub4/f099.csv
```

`-size -1M` found only the empty file (the rounding gotcha); pruning skipped `node_modules`; 20,000 generated files gave the same 1,600 matches sequentially and in parallel.

Concurrency & edge cases

- Weakly consistent `ConcurrentSkipListMap` iterators: no exception when files are added mid-search.
- Parallel walk only reads; each task builds its own list — no locks.
- Real disks: don't follow symlinks by default (GNU `find` needs `-L`), or track visited inodes.
- Handled: missing start path, `mindepth > maxdepth`, unknown predicate, missing argument, bad `-size`/`-mtime`/`-type`, unbalanced `(`, unclosed `[` in a glob, limit mid-walk.

Extensions & follow-up questions

- **Real disk** → adapter over `java.nio` / `Files.walkFileTree` (`SKIP_SUBTREE` for prune).
- **Actions** (`-delete`, `-exec`) → a `Consumer` per action; delete in post-order.
- **Content search** → `ContainsTextFilter`, placed last in the AND.
- **Repeated searches** → an index like `locate`, possibly stale.
- **More tests** → one record + one parser `case`.

Real-world example

GNU findutils' `find` uses this precedence, short-circuits, rounds `-size` up and does not follow symlinks by default; Java's `Files.find(start, maxDepth, predicate)` splits walking from testing the same way; IDE "Find in files" builds the same filter tree from checkboxes.

In the interview

First 10 minutes: `Node` model + `FileFilter` with two leaves → demo. Then And/Or/Not, the Builder with depths, the iterative walk; the parser last. Say it: "a new filter is one class; the searcher never changes."

Self-check

- **Q:** `-name a -o -name b -type f`? → **A:** `a OR (b AND file)`.
- **Q:** Why does `-size -1M` match only empty files? → **A:** Rounded up, any non-empty file is ≥ 1 MiB unit.
- **Q:** Prune vs filtering a directory out? → **A:** Prune never enters it; a filter only hides the entry.
- **Q:** Why records instead of lambdas for And/Or/Not? → **A:** Printable, inspectable, what the parser builds.

TL;DR

Two Composites (files, filters); Specification leaves combined by AND/OR/NOT; a Builder for criteria; an iterative streaming DFS; a recursive-descent parser with `!` > AND > OR and GNU's rounded-up `-size`.

Chapter summary

Problem	Core entities	Key pattern(s)	Concurrency point	Trap to avoid
Task Management	Task, TaskStatus, TaskService	Specification filters, enum state machine, Observer	per-task `synchronized`	TODO → DONE jumps; filters as many parameters
Digital Wallet	Wallet, Transaction, Offer	Strategy offers, idempotency, funded cashback	lock both wallets in id order	`double` money; check-then-act on keys; cashback from nothing
Event Calendar	TimeSlot, Event, EventStore	Strategy for representatives, plan-then-save	atomic check-and-book	closed intervals; trusting the sample blindly
Coding Blox	Contest, Level, User	Command map, Strategy (solver, scoring)	atomic rating update	unseeded randomness; running a contest twice
PropertyHunt	Property, SearchCriteria	Builder + predicate list, Strategy sort	synchronized mark-sold	comparing "90L" strings; stale shortlist copies
Ledger Co	Loan, LumpSum, Balance	Strategy interest, derived state	synchronized lump sums	storing a running balance; floor instead of ceil
Inventory	StockMovement, AllocationStrategy	Strategy allocation, Observer alerts	per-SKU lock	partial shipments; alert spam
Unix find	Node (Composite), FileFilter, SearchCriteria	Specification + Composite filters, Builder, recursive-descent parser	weakly consistent children map; read-only parallel walk	parameter-list search methods; `-o` vs AND precedence

The pattern behind all eight: entities guard their own rules, services orchestrate, strategies hold the "X or Y" rules, repositories hide storage, the driver parses and prints, and every error is a clear message — not a crash.

Chapter 22 — Revision: Cheat Sheets & Glossary

> **What this chapter is.** The whole handbook on a few pages: the answer framework (Ch 1); OOP, relationships and UML (Ch 2–3); SOLID, other principles and clean code (Ch 4–6); all 24 smells (Ch 7); all 23 GoF patterns + extras and a "which pattern?" table (Ch 8–11); the concurrency toolbox (Ch 12–13); a map of every case study (Ch 14–21); machine-coding and edge-case checklists; and a glossary. Nothing here is new and there is no code — each row tells you where the full explanation lives.

How to read the references. "Ch 12 · Deadlock" means the *Deadlock* topic in Chapter 12; "Ch 16 · Car Rental" means the Car Rental case study in Chapter 16. When a row or heading already names the topic, only the chapter is given ("Ch 2"). Numbers such as "22.7" refer to sections of this chapter.

22.1 How to revise

In 15 minutes (the morning of the interview)

Minutes	Read	Why
0–3	22.2 (the framework)	You will use it in the first minute of the interview
3–6	22.4 (SOLID one-pager) and the relationship table in 22.3	The two things asked about most often by name
6–10	"Which pattern?" table in 22.7	Patterns should come from a signal, not from memory
10–13	Concurrency cheat sheet and deadlock checklist in 22.8	Every booking/inventory problem has a race
13–15	The row(s) of 22.9 for the problems you expect	One sentence per problem: entities, pattern, the atomic step

The night before (about 2 hours)

1. **Read 22.2–22.8 slowly** (45 min), saying *one handbook example* per row; if you can't, open the referenced topic.
2. **Three unpractised case studies** from 22.9 (45 min): requirements, classes, class diagram and the one hard method, without looking; then compare with the chapter's TL;DR.
3. **Scan the glossary** (22.11); read up only the terms you can't explain in one sentence (20 min).
4. **Stop.** Sleep matters more than one more pattern.

Common mistakes when revising

- Re-reading chapters instead of **recalling** first and then checking. Recall is what the interview tests.
- Memorising all 23 patterns equally. Six come up in almost every round (22.7): Strategy, Factory, Observer, State, Singleton, Builder.
- Skipping the "when *not* to use it" column. Interviewers ask "would you always do that?" (Ch 4 · SOLID in interviews).

22.2 The LLD interview answer framework (one page)

Source: Chapter 1 (how the interview runs, the answer framework, requirements, classes, SOLID check, patterns, interview code, communication tips, common mistakes).

TL;DR

Clarify → requirements → use cases → entities → relationships → class diagram → key flows → patterns/SOLID → code → edge cases/concurrency → extensions. Say each step out loud as you enter it.

The eleven steps

#	Step	Ask yourself	Output
1	Clarify	Who uses it? Single site or many?	Shared understanding
2	Requirements	Functional? Non-functional (thread-safe, extensible, in-memory)? **Out of scope?**	8–12 numbered lines + out-of-scope list
3	Use cases	What can each actor do?	Actor → use-case list
4	Entities	Which nouns hold state or behaviour? Which are just values?	Classes, one sentence each
5	Relationships	Uses / knows / has / owns / is-a? How many?	Typed lines with multiplicity
6	Class diagram	Fields, key methods, interfaces, enums	UML class diagram
7	Key flows	Steps of the 2–3 main use cases; lifecycle of the core object	Sequence / activity / state diagram
8	Patterns & SOLID	A rule that varies? A lifecycle? Many listeners? Two reasons to change?	1–2 named patterns + one rejected
9	Code	Interfaces, enums, models, services, driver	Running code for the main flows
10	Edge cases & concurrency	Bad input? Full? Two users, one seat?	Validation, exceptions, locks
11	Extensions	"What if X is added?"	Short answers, maybe one class

Time budgets

45-minute OOD round		**90-minute machine-coding round**	
Clarify requirements	5–8 min	Read, requirements, assumptions	5–10 min
Actors and use cases	3–5 min	Model: entities, enums, service interfaces	10–15 min
Find the classes	8–10 min	Core flow end to end	30–40 min
Connect the classes	8–10 min	Remaining features, validation, edge cases	15–20 min
Write the code	12–15 min	Polish, one extension point, demo	5–10 min
		Buffer	5 min (never zero)

Rule of thumb: roughly **a quarter of the hour goes before you name a class**, and in machine coding you **get something running by the halfway mark** (Ch 1 · The interview).

Finding classes (Ch 1 · Finding the classes)

- **Nouns → candidate classes; verbs → methods.** A verb with no class to live on means a class is missing.
- Cut candidates with five tests: is it a **value**? the **same thing** under another name? does it have **changing state**? does anything **call** it? does it differ in **behaviour or only data**?
- **Different data → field or enum. Different behaviour → class.**
- One sentence per class, **no "and"**. Put each method where its data is (Information Expert).
- Beginners most often forget the **record-keeping class** (ticket, loan, booking) and the **rule class** (rate, strategy).

What the interviewer judges (Ch 1 · The interview)

1. Did you **bound** the problem? 2. Does each class have **one job** you can say in a sentence? 3. Are the **relationships** right? 4. Does the **code match the diagram**? 5. Can you explain a **trade-off** (the choice and the rejected option)?

Machine coding adds: does it **run**, is it **modular**, **readable**, does it **handle bad input**, and (senior roles) is shared state **thread-safe**.

Code habits interviewers notice (Ch 1 · Interview code)

Habit	Instead of
Pass time in (`long nowMinute`, `Instant now`, injected `Clock`)	Reading the clock inside logic
Money as `BigDecimal` from strings, or whole cents in a `long`	`double` (0.1 + 0.2 ≠ 0.3)
`IllegalArgumentException` / `IllegalStateException` / a custom exception	Silent failure or error codes
`Optional` when "not found" is normal	Returning `null`
`private final` fields	Public mutable fields
Domain names (`ParkingTicket`)	`Manager`, `Helper`, `Data2`

Skeleton: **Models** → **Repositories** → **Strategies** → **Services** → **Driver** (Ch 1 · Interview code).

In the interview

30-second answer (from Ch 1 · Common mistakes): "I clarify and write 8 to 12 requirements with an out-of-scope list, list actors and use cases, pick classes from the nouns and give each one responsibility, connect them and draw a class diagram, sketch the key flow, run a quick SOLID check and pick one or two patterns where the signal is clear, then code the two or three core flows, handle edge cases and concurrency, and finish with extensions and trade-offs."

Common mistakes (Ch 1)

Mistake	Fix
Coding in minute two	8–13 minutes on requirements and actors first
No out-of-scope list	Write it down and agree it
One subclass per value (`CompactSpot`, `LargeSpot`)	Different data → field/enum
God class (`ParkingLotManager`)	One sentence per class; split on "and"
Inheritance for reuse ("Car extends Engine")	Composition
Pattern stuffing	One or two patterns, each with its signal
All twelve classes at equal depth	Two or three flows fully
Ignoring two users on one resource	Lock or atomic operation on the shared state
Non-running code in machine coding	Thin end-to-end path running by halfway

---

22.3 OOP & relationships cheat sheet

Source: Chapter 2 (OOP) and Chapter 3 (relationships and UML).

The four pillars (Ch 2)

Pillar	One line	Java tools	Smell when missing
Encapsulation (Ch 2)	Data is private; only the owning class's methods change it, and they enforce the rules	`private` fields, no setters, validation in constructors	Public fields; setters that break invariants
Abstraction (Ch 2)	Expose only what callers need; hide how	Interfaces, abstract classes, small public API	Callers reach into internals
Inheritance (Ch 2)	Child reuses and specialises parent — **only when "child is a parent" is truly correct**	`extends`, `super`, `@Override`	Subclass refuses or throws on parent methods
Polymorphism (Ch 2)	One call, different behaviour per object; removes type switches	Overriding + dynamic dispatch	`switch` / `instanceof` chains on type

Encapsulation vs abstraction: hiding *data* behind rule-enforcing methods vs hiding *complexity* behind a minimal set of operations (Ch 2 self-check).

Other OOP facts worth one line each

Topic	Remember	Where
Overloading vs overriding	Compile time, by declared parameter types vs run time, by the actual object. Fields and `static` methods are not polymorphic (statics are hidden, not overridden)	Ch 2 · Compile-time vs runtime polymorphism
`this`, `static`, `final`	Current object; one per class; cannot be reassigned/overridden/extended	Ch 2 · Classes & keywords
Interfaces	Contracts; can have `default`, `static`, `private` methods; a functional interface has one abstract method (lambdas)	Ch 2
Enums	Fixed sets with fields and behaviour ("which spot types fit a truck" lives on `VehicleType`)	Ch 2 · Enums with fields and behaviour
Composition over inheritance	Avoids fragile base classes; allows run-time swapping	Ch 2, Ch 5 · Composition over inheritance
`equals` / `hashCode`	Equal objects must have equal hash codes; override both or hash collections break	Ch 2 · Object identity vs equality
Immutability & records	Final class, private final fields, no setters, defensive copies. Records are only **shallowly** immutable — copy collections in the compact constructor	Ch 2 · Immutability & records
Access modifiers	Default to `private`; package-private = same package only	Ch 2 · Access modifiers and packages
Generics	Type-safe reusable infrastructure; **PECS**: producer `extends`, consumer `super`	Ch 2 · Generics basics for LLD
Coupling & cohesion	Low coupling (between classes), high cohesion (inside a class)	Ch 2 · Coupling and cohesion
OOA → OOD → OOP	What the problem is → how, with objects → code. Connections that carry data become objects	Ch 2 · Object-oriented analysis and design

Abstract class vs interface (Ch 2 · Abstract classes vs interfaces)

	Abstract class	Interface
Instantiable?	No	No
Instance fields (state)	Yes	No (only `public static final` constants)
Constructors	Yes	No
Method bodies	Any	`default`, `static`, `private` only
How many per class	**One** (`extends`)	**Many** (`implements`)
Relationship expressed	"is a" (shared identity and state)	"can do" (capability / role)
Typical use	Shared state + partial implementation for a close family (Template Method)	Contracts, roles, strategies, test seams

Rule of thumb: start with an interface; add an abstract class underneath when implementations share real state or a fixed algorithm (JDK: `List` + `AbstractList` + `ArrayList`).

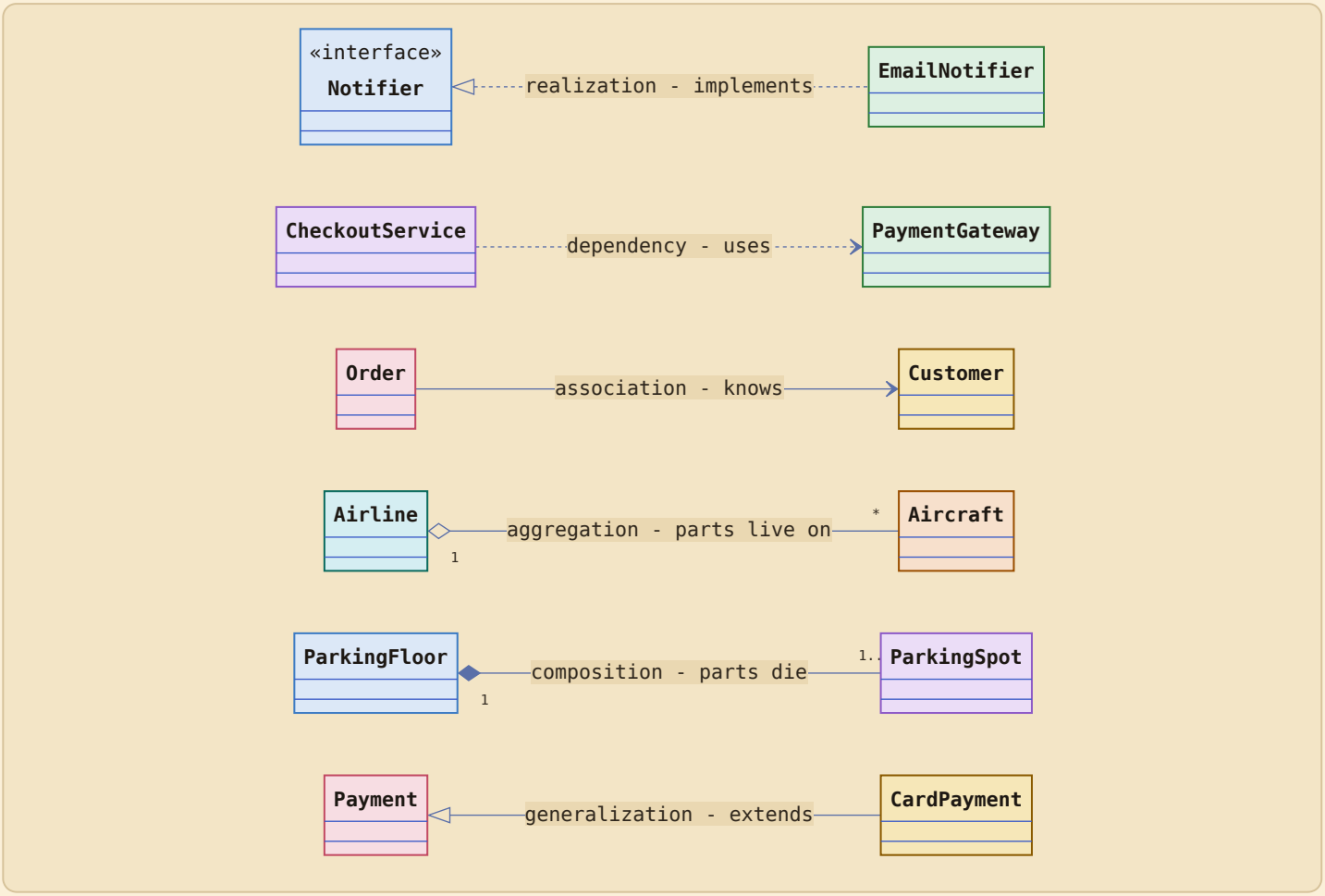
The six relationships (Ch 3)

Weakest to strongest coupling: **dependency, association, aggregation, composition, realization, generalization** (Ch 3 · self-check).

Relationship	Phrase	UML line	Java shape	Lifetime	Example
Dependency (Ch 3)	A *uses* B	dashed, open arrow → B	B as parameter / local / return / static call	none	`CheckoutService` uses `PaymentGateway`
Association (Ch 3)	A *knows* B	solid line (arrow for one-way)	field of type B, created elsewhere	independent	`Order` → `Customer`
Aggregation (Ch 3)	A *has* B, parts live on	solid, hollow ◇ at A (the whole)	field/collection, parts passed in	B outlives A	`Airline` ◇— `Aircraft`
Composition (Ch 3)	A *owns* B	solid, filled ◆ at A (the whole)	A creates B, hides it, B not shared	B dies with A	`ParkingFloor` ◆— `ParkingSpot`
Realization (Ch 3)	A *implements* B	dashed, hollow △ at B	`implements`	—	`EmailNotifier` ⇢ `Notifier`
Generalization (Ch 3)	A *is a* B	solid, hollow △ at B	`extends`	—	`CardPayment` → `Payment`

Choosing, in order (Ch 3 · Choosing the relationship): interface A promises to fulfil → realization. Substitutable with no refused methods → generalization. Not kept in a field → dependency. Part that dies with the whole → composition. Part that outlives it → aggregation. Otherwise → association.

UML arrow legend (Mermaid)



Mermaid syntax: `A ..> B` dependency · `A --> B` association (navigable A→B; `A -- B` unspecified) · `A o-- B` aggregation and `A *-- B` composition (A is the whole) · `B <|.. A` realization · `B <|-- A` generalization · `A "1" *-- "1..*" B` adds multiplicities.

Multiplicity → code (Ch 3 · TL;DR): `1` required field; `0..1` `Optional`/nullable; `1..*` non-empty collection checked in the constructor; exact numbers checked in constructors. Visibility: `+` public, `-` private, `#` protected, `~` package.

UML diagrams in the interview (Ch 3 · UML diagrams)

Diagram	Use it for	Budget in 45 min
Use case (Ch 3 · Use-case diagrams)	Scope: actors + what each does. «include» = always; «extend» = sometimes	Usually spoken, 3–5 min
Class (Ch 3 · Class diagrams)	**The answer.** 10–15 boxes, every line typed, with multiplicity	Always drawn, 10–15 min
Sequence (Ch 3 · Sequence diagrams)	Call order between objects (3–4 objects, 6–8 messages)	One behaviour diagram only
Activity (Ch 3 · Activity diagrams)	A flow that branches; fork/join for parallel steps	"
State machine (Ch 3 · State machine diagrams)	One object's lifecycle; transition label `event [guard] / action`	"

UML has 14 diagram types: 7 structural, 7 behavioural (Ch 3 · What UML is).

Common mistakes

- Diamond at the wrong end (it goes at the **whole**); triangle at the wrong end (it goes at the **parent/interface**) (Ch 3 · Choosing the relationship).

- Lines without type or multiplicity (Ch 3 · Choosing the relationship).
- Inheritance used to share data; subclasses that only pass a constant to `super` → one class with an enum field (Ch 3 · self-check).

22.4 SOLID one-pager

Source: Chapter 4 (the five principles, SOLID and patterns, SOLID in the interview, over-engineering); smells from Chapter 7 (smell lookup table).

TL;DR

SOLID = five names for the ways code becomes painful to change. **Use them to diagnose, not as a checklist**, apply a fix when a real change asks for it, and always say what the fix costs.

Principle	One-line rule	Smell that signals a violation	Fix	Patterns that help
S — Single Responsibility (Ch 4 · Single Responsibility Principle)	A class should have one reason to change (one actor asks for its changes)	The responsibility sentence needs an "and"; *Divergent Change*, *Large Class*	Split by actor (Extract Class); each new class needs a name that stands alone	Command, Proxy, Decorator (each concern in its own class)
O — Open/Closed (Ch 4 · Open/Closed Principle)	Add new behaviour by adding code, not by editing working code	A `switch`/`if` chain over a type; *Repeated Switches*, *Shotgun Surgery*	Put the part that *has* varied behind an interface; add variants as classes (Replace Conditional with Polymorphism)	Strategy, Template Method, Decorator, Factory Method, Observer, Command, Visitor
L — Liskov Substitution (Ch 4 · Liskov Substitution Principle)	A subtype must be usable anywhere its parent is expected	Subclass throws, does nothing, or refuses a parent method; *Refused Bequest*	Better parent, a narrower interface, or composition (Replace Subclass with Delegate)	Composite, Null Object
I — Interface Segregation (Ch 4 · Interface Segregation Principle)	No client should depend on methods it does not use	Empty method bodies or `UnsupportedOperationException` in implementers; fat interfaces	Split into **role interfaces**, one per kind of caller	Adapter
D — Dependency Inversion (Ch 4 · Dependency Inversion Principle)	Depend on abstractions, not concrete details; the policy owns the abstraction	A high-level class does `new` of (or names) a low-level one; cannot test without the real DB/provider	Inject an interface owned by the high-level class; `main` wires the details	Strategy, Factory Method / Abstract Factory, Adapter, Observer

LSP in detail (Ch 4 · Liskov Substitution Principle, TL;DR): a subclass must accept at least as much, promise at least as much, keep invariants and history, and throw nothing new. Square/Rectangle is worse than `Penguin.fly()` throwing because it fails **silently** (wrong area, no error).

Five-minute pass over your diagram (Ch 4 · SOLID in interviews, Ch 1 · SOLID check)

Run S → O → L → I → D over the diagram you just drew. **Two findings is a good pass; five means rework; zero means you did not look.** S and O earn the most credit: O is what interviewers probe with a late requirement ("what if trucks are charged differently?" → "add a class").

Related ideas (Ch 4 · Cohesion, Coupling, SoC, SOLID Q&A)

Idea	Question it asks
SRP	How many actors can ask this class to change?
Cohesion	Do the parts of this class belong together?
Coupling	How much does A depend on B?
Separation of Concerns	Is each area of work in its own module?
SRP vs ISP	SRP splits **classes** (guards the worker); ISP splits **interfaces** (guards the caller)
DIP vs DI vs IoC	DIP = principle (depend on abstractions); DI = technique (pass dependencies in); IoC = the broader idea that a framework calls your code; IoC containers perform DI

Common mistakes: over-engineering (Ch 4)

Warning sign	Principle misapplied	Do instead
`XHelper`, `XUtils`, `XManager2` after a split	SRP	Split by actor; names must stand alone
Six small classes that always change in the same commit	SRP over-split	Merge them back
Interface + factory + registry for one implementation	OCP (*Speculative Generality*)	Write the `if`; refactor at the second or third case (rule of three)
A subclass that "works" by silently doing nothing	LSP	Better parent or composition
One interface per method	ISP over-split	Group by caller into role interfaces
An interface in front of stable `String`-like classes	DIP	Invert only volatile dependencies
A DI container for a 200-line program	DIP	Wire by hand in `main`

YAGNI is the counterweight: SOLID says *what* a good seam looks like; YAGNI says *when* to cut it.

In the interview

Say **observation → cost → fix → price of the fix**, and describe before you label: "`Reporting` implements nine methods but calls two — an interface segregation problem" (Ch 4 · SOLID in interviews).

22.5 Other design principles and Clean Code rules

Other principles (Chapter 5)

Principle	One-line idea	Smell it fixes	Key Java tool	Where
DRY	Each piece of knowledge has one authoritative home	Duplicated rules drifting apart	Value objects, constants, methods	Ch 5 · DRY
KISS	Choose the simplest design that works	Over-engineering	Plain code, `switch` expressions	Ch 5 · KISS
YAGNI	Don't build features before they are needed	Speculative Generality	Delete unused code	Ch 5 · YAGNI
Law of Demeter	Only talk to your immediate friends	Train wrecks `a.getB().getC().do()`	Intent-revealing delegate methods	Ch 5 · Law of Demeter
Tell, Don't Ask	Tell objects what to do; don't pull their data out and decide for them	Anemic model, Feature Envy	Command methods on the owner	Ch 5 · Tell, Don't Ask
Composition over inheritance	Prefer "has-a" to "is-a" for reuse	Fragile base class	Forwarding / decorators	Ch 5 · Composition over inheritance
Program to an interface	Depend on supertypes, choose the concrete class once at the edge	Hard-wired concrete dependencies	Interfaces + DI	Ch 5 · Program to an interface
Encapsulate what varies	Isolate the part that changes behind an interface	Growing type switches	Strategy, lambdas	Ch 5 · Encapsulate what varies
Loose coupling	Interacting objects know little about each other	God orchestrators	Observer, events, DI	Ch 5 · Loose coupling
Hollywood Principle / IoC	"Don't call us, we'll call you" — the framework owns the flow	Duplicated skeletons	Template Method, callbacks, DI	Ch 5 · Hollywood Principle
Separation of Concerns	Each part handles one concern	Spaghetti code	Layers, MVC	Ch 5 · Separation of Concerns, Ch 4
Least Astonishment	Do what the name promises and nothing more	Hidden side effects	Honest names, `Optional`	Ch 5 · Least Astonishment
Fail fast	Detect problems early and loudly	Distant `NullPointerException`s	Guard clauses, `requireNonNull`	Ch 5 · Fail fast
Immutability	Objects never change after creation	Aliasing, races	Records, `List.copyOf`	Ch 5 · Immutability
Design by Contract	Preconditions (caller), postconditions (method), invariants (always)	Magic error values	Checks at method entry/exit	Ch 5 · Design by Contract
GRASP	Nine guidelines for "which class should do this?"	God classes	Expert, Creator, Controller…	Ch 5 · GRASP
Domain-Driven Design	Model the business: bounded contexts, entities vs value objects, small aggregates, domain events, repositories per root	Anemic model, big ball of mud, primitive obsession	Records as value objects, aggregate roots, events + outbox	Ch 5 · DDD

GRASP in one line each (Ch 5 · GRASP):

Guideline	Rule
Information Expert	The class with the data does the work
Creator	The container (or the one with the initial data) creates
Controller	A thin non-UI object receives each system event and delegates
Low Coupling	Choose the design with fewer, more stable dependencies
High Cohesion	Each class does one related set of things
Polymorphism	Let each type implement the varying behaviour; don't switch on types
Pure Fabrication	Invent a helper class (e.g. a repository) when no domain class is a clean home
Indirection	Put a middleman between things that shouldn't know each other
Protected Variations	Wrap what's unstable behind a stable interface

Tensions to mention (Ch 5 summary): DRY vs loose coupling (sharing code couples its users); YAGNI vs encapsulate-what-varies (abstract only real or clearly predicted variation); Law of Demeter vs cohesion (too many delegates → Middle Man); Information Expert vs SoC (domain objects compute, fabrications persist).

Clean Code rules checklist (Chapter 6)

Area	Rule of thumb	Where
Names	Reveal intent; pronounceable; searchable; one word per concept; code reads like the requirement	Ch 6 · Meaningful names
Functions	Small; do one thing; one level of abstraction; top-down (stepdown); ≤ 2–3 arguments; **no flag arguments**; no hidden side effects; commands *or* queries (CQS); exceptions, not error codes	Ch 6 · Functions
Comments	Say it in code; comment only the *why*, warnings, legal text, public contracts; delete commented-out code	Ch 6 · Comments
Formatting	Newspaper order (headline first); related things close; one automated team style	Ch 6 · Formatting
Objects vs data structures	Objects hide data and expose behaviour; data structures expose data; no hybrids; no train wrecks	Ch 6 · Objects and data structures
Error handling	Unchecked exceptions with context; wrap vendor exceptions; empty collections / `Optional` / special case instead of `null`; never pass `null`; always clean up	Ch 6 · Error handling
Boundaries	Keep third-party code behind your own interfaces; learn it with tests	Ch 6 · Boundaries
Unit tests	Three laws of TDD; one concept per test; Given–When–Then; **F.I.R.S.T.** (Fast, Independent, Repeatable, Self-validating, Timely)	Ch 6 · Unit tests
Classes	Small by responsibility; cohesive; open for extension; depend on abstractions	Ch 6 · Classes
Systems	Build objects in one place (composition root) and inject them; cross-cutting concerns in proxies/aspects	Ch 6 · Systems
Simple design	Beck: passes the tests → reveals intention → no duplication → fewest elements (*Clean Code* swaps the middle two)	Ch 6 · Simple design
Concurrency	Separate concurrency code; share as little as possible; use the library; keep locks small; plan shutdown	Ch 6 · Concurrency
Smells	The C/E/F/G/J/N/T heuristics catalogue as a review vocabulary	Ch 6 · Smells and heuristics
Code review	Correctness → design → readability → tests → security → performance → operability; automate style; small PRs	Ch 6 · Code review checklist
Error strategy	Classify, translate at boundaries, retry only transient + idempotent with backoff, one global handler that logs once	Ch 6 · Error-handling strategy
Testing strategy	Many unit, some integration, few E2E; right test double; verify state over interactions; don't mock what you don't own	Ch 6 · Testing strategies

Boy Scout Rule: leave the code a little cleaner than you found it (Ch 6 intro).

22.6 Code smells → refactorings (all 24)

Source: Chapter 7 (code smells, refactoring catalogue, smell lookup table). ***Bold*** *= the usual first move.*

#	Smell	Signal (one line)	Group	Refactorings that cure it
1	Mysterious Name	Name doesn't say what it does/holds (`data`, `proc()`, `Manager`)	2nd-ed extra	**Change Function Declaration**, Rename Variable, Rename Field
2	Duplicated Code	Same structure in more than one place	Dispensables	**Extract Function**, Slide Statements, Pull Up Method
3	Long Function	You want a comment to explain a block	Bloaters	**Extract Function**, Replace Temp with Query, Introduce Parameter Object, Preserve Whole Object, Replace Function with Command, Decompose Conditional, Replace Conditional with Polymorphism, Split Loop
4	Long Parameter List	Many parameters, same types, boolean flags	Bloaters	Replace Parameter with Query, **Preserve Whole Object**, **Introduce Parameter Object**, Remove Flag Argument, Combine Functions into Class
5	Global Data	Readable and writable from anywhere (incl. mutable singletons)	2nd-ed extra	**Encapsulate Variable** (then shrink its scope)
6	Mutable Data	Shared data changed after creation	2nd-ed extra	**Encapsulate Variable**, Split Variable, Slide Statements, Extract Function, Separate Query from Modifier, Remove Setting Method, Replace Derived Variable with Query, Combine Functions into Class/Transform, Change Reference to Value
7	Divergent Change	One class changed for many different reasons (SRP)	Change Preventers	**Split Phase**, Move Function, Extract Function, **Extract Class**
8	Shotgun Surgery	One change forces many small edits in many classes	Change Preventers	**Move Function**, **Move Field**, Combine Functions into Class/Transform, Split Phase, Inline Function, Inline Class
9	Feature Envy	Method uses another class's data more than its own	Couplers	**Move Function**, Extract Function
10	Data Clumps	Same group of values always travel together (`startDate, endDate`)	Bloaters	**Extract Class**, **Introduce Parameter Object**, Preserve Whole Object
11	Primitive Obsession	Money as `double`, status as `String`	Bloaters	**Replace Primitive with Object**, Replace Type Code with Subclasses, Replace Conditional with Polymorphism, Extract Class, Introduce Parameter Object
12	Repeated Switches	The *same* switch on a type code in several places	OO Abusers	**Replace Conditional with Polymorphism**
13	Loops	Classic loop that filters/maps/aggregates	2nd-ed extra	**Replace Loop with Pipeline**
14	Lazy Element	A class/function that doesn't pull its weight	Dispensables	**Inline Function**, **Inline Class**, Collapse Hierarchy
15	Speculative Generality	Hooks "we might need someday"; abstract class with one subclass	Dispensables	Collapse Hierarchy, Inline Function, Inline Class, Change Function Declaration, Remove Dead Code
16	Temporary Field	Field set only in some situations, `null` otherwise	OO Abusers	**Extract Class**, Move Function, Introduce Special Case

#	Smell	Signal (one line)	Group	Refactorings that cure it
17	Message Chains	`order.getCustomer().getAddress().getCity()`	Couplers	**Hide Delegate**, Extract Function, Move Function
18	Middle Man	Most methods just forward to another class	Couplers	**Remove Middle Man**, Inline Function, Replace Superclass/Subclass with Delegate
19	Insider Trading	Two modules poke at each other's internals	Couplers	**Move Function**, **Move Field**, Hide Delegate, Replace Subclass/Superclass with Delegate
20	Large Class	Too many fields/methods — a God Class	Bloaters	**Extract Class**, Extract Superclass, Replace Type Code with Subclasses
21	Alternative Classes with Different Interfaces	Same job, different method names (`sendMail` vs `dispatch`)	OO Abusers	**Change Function Declaration**, Move Function, Extract Superclass
22	Data Class	Only fields and getters/setters; others manipulate it	Dispensables	**Encapsulate Record**, Remove Setting Method, Move Function, Extract Function, Split Phase
23	Refused Bequest	Subclass doesn't want inherited methods, or throws on them (LSP)	OO Abusers	Push Down Method/Field, **Replace Subclass with Delegate**, **Replace Superclass with Delegate**
24	Comments	Comment used as deodorant for unclear code	Dispensables	**Extract Function**, Change Function Declaration, Introduce Assertion

Smell family → principle violated (Ch 7 · Smell lookup table)

Smell family	Principle
Divergent Change, Large Class	SRP
Repeated Switches, Shotgun Surgery	OCP
Refused Bequest	LSP
Long Parameter List with flags, fat interfaces	ISP / clean function design
Global Data, Mutable Data	Encapsulation
Hard-wired `new` of concrete dependencies	DIP
Duplicated Code	DRY
Speculative Generality	YAGNI
Message Chains, Insider Trading	Law of Demeter
Feature Envy, Data Class	Tell, Don't Ask

Refactoring toward patterns (Ch 7)

Smell → pattern, in short (the full symptom table is in 22.7): algorithm-choosing switch → **Strategy**; methods checking and changing a status field → **State**; `if (x == null)` everywhere → **Null Object / Special Case**; sibling subclasses with the same skeleton → **Template Method**; many confusing constructors → **Factory Method** / creation methods; object built by many ordered calls → **Builder**; undo/queueing, or a huge function with shared locals → **Command**; logging/caching mixed into a core class → **Decorator**; polling or hard-coded notification → **Observer**; two classes, same job, different interfaces → **Adapter**; subclass explosion from feature combinations → **Strategy / Decorator / Bridge**; a pattern that no longer earns its keep → **remove it** (Inline Class, Collapse Hierarchy).

Refactoring = changing structure without changing behaviour, in small steps, with tests as the safety net. Wear one of **two hats** at a time: adding function or refactoring (Ch 7 · Refactoring fundamentals).

22.7 Design patterns: all 23 GoF + extras

Source: Chapters 8–11 (catalogue in Ch 8 · The 23 GoF patterns; comparison tables at the end of Chapters 8, 9, 10 and 11).

Count: 5 creational + 7 structural + 11 behavioral = **23**. **Class-scoped** (inheritance) patterns: Factory Method, class Adapter, Template Method, Interpreter; almost everything else is object-scoped (composition) (Ch 8 · The 23 GoF patterns).

Creational (Chapter 8)

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Singleton	Exactly one instance with a global access point	There really must be exactly one (config, pool, registry)	`Runtime.getRuntime()`; Spring singleton-scoped beans	A global variable; a DI singleton scope (preferred for testing)	Ch 8
Factory Method	Subclasses decide which concrete class to create via an overridable method	Shared procedure, varying product; others will add types	`Collection.iterator()`	Simple Factory (one method with a switch)	Ch 8
Abstract Factory	Create *families* of related objects without naming concrete classes	Products must match; swap a whole set at once	`DocumentBuilderFactory`; JDBC `Connection`	Factory Method (one product by inheritance vs a family by composition)	Ch 8
Builder	Build a complex object step by step; one validating `build()`	Many optional fields, cross-field validation, immutable result	`StringBuilder`, `HttpClient.newBuilder()`	A plain constructor (fine for a handful of required fields); telescoping constructors	Ch 8
Prototype	Create new objects by copying a configured instance	Setup is expensive, or only the interface is known	Copy constructors, `ArrayList.clone()`	Memento (a snapshot to restore, not a copy to use)	Ch 8

Singleton notes (Ch 8 · TL;DR): eager, holder-class or **`enum`** (most robust: blocks reflection and serialization duplicates); double-checked locking needs a **`volatile`** field. It is global state that hurts testing — prefer injecting one shared instance. Many case studies reject it (Ch 14 self-check).

Structural (Chapter 9)

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Adapter	Convert one interface into another that clients expect	"The vendor API doesn't match ours" (after the fact)	`Arrays.asList`, `InputStreamReader`	Bridge (same shape, but Bridge is designed up front); Facade	Ch 9 · Adapter Pattern
Bridge	Split abstraction from implementation so both vary independently	Class names with two nouns; two dimensions multiplying classes	JDBC `Connection` + drivers	Adapter; Strategy (same shape)	Ch 9 · Bridge Pattern
Composite	Treat single objects and trees uniformly	Part-whole trees (files, menus, UI)	`java.awt.Container` is a `Component` holding `Component`s	Decorator (also wraps a component, but adds behaviour to one)	Ch 9 · Composite Pattern
Decorator	Add responsibilities dynamically by wrapping; same interface, stackable	"Sometimes logging, sometimes caching, sometimes both"	`BufferedInputStream`, `Collections.synchronizedList`	Proxy (same shape; Proxy *controls access*); Chain of Responsibility (in CoR someone can stop the flow)	Ch 9 · Decorator Pattern
Facade	One simple entry point to a complex subsystem	Every caller repeats the same multi-step sequence	SLF4J `LoggerFactory`, service layers	Adapter (converts one object's interface); Mediator	Ch 9 · Facade Pattern
Flyweight	Share intrinsic (immutable) state across huge numbers of objects	Millions of near-identical objects	`Integer.valueOf`, `String.intern()`	Object Pool (reuses objects one borrower at a time; flyweights are shared at once)	Ch 9 · Flyweight Pattern
Proxy	A stand-in that controls access (lazy, remote, protective, caching)	It's expensive, remote or restricted	`java.lang.reflect.Proxy`, Spring `@Transactional`, Hibernate lazy load	Decorator	Ch 9 · Proxy Pattern

Memory hooks (Ch 9 · Structural comparison): Adapter *make it fit* · Decorator *add to it* · Proxy *guard it* · Facade *simplify it* · Bridge *split it* · Composite *nest it* · Flyweight *share it*.

Behavioral (Chapters 10–11)

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
Chain of Responsibility	Pass a request along handlers until one handles (or stops) it	Several checks, any may reject; handler not known in advance	Servlet `Filter` chain; `java.util.logging` handlers	Decorator (every layer always runs)	Ch 10
Command	Turn a request into an object: queue, log, retry, undo	Undo/redo, job queues, scheduling	`Runnable` / `Callable`; Swing `UndoManager`	Strategy (Strategy = *how*; Command = *what*)	Ch 10
Iterator	Traverse a collection without exposing how it is stored	Walk a collection/tree/paged API	`Iterator`, `Spliterator`, JDBC `ResultSet`	Visitor (what to do at each node)	Ch 10
Mediator	Centralise communication between peers in one coordinator	Many objects interact in tangled ways; order of reactions matters	Chat servers, UI controllers; `Executor` / `Timer` are a loose fit (Ch 10)	Observer (broadcast, no coordination); Facade	Ch 10
Memento	Capture and restore state without breaking encapsulation	Undo, checkpoints, rollback	A `record` snapshot; DB `ROLLBACK`, Git commits (real-world)	Command undo (store actions, not states); Prototype	Ch 10
Interpreter	One class per grammar rule; evaluate sentences of a small language	User-written rules/expressions	`java.util.regex.Pattern`, SpEL	Composite (same tree); Visitor (many operations over the AST)	Ch 10
Observer	Notify many dependents automatically when one object changes	An open-ended set of independent listeners	`PropertyChangeListener`, `java.util.concurrent.Flow`, Spring `@EventListener`	Mediator; pub-sub (Observer through a broker, Ch 20 · Pub-sub)	Ch 11
State	Behaviour changes with internal state, as if the class changed	Many states/transitions; methods repeat a `switch` on status	TCP connection (GoF's example), order lifecycles. `Thread.State` is only an enum of states, not the pattern	Strategy (same diagram; states know each other and advance themselves)	Ch 11
Strategy	Make a family of algorithms interchangeable behind one interface	Several ways to do one job, chosen at run time	`Comparator`; `RejectedExecutionHandler` policies	State; Template Method; Decorator (Strategy *replaces* a rule, Decorator *wraps* one)	Ch 11
Template Method	Fixed algorithm skeleton in a base class; subclasses fill in steps	Long fixed flow, a few varying steps	`AbstractList`, `InputStream.read(byte[],…)`, `HttpServlet.service()`	Strategy (Template = inheritance, one step varies;	Ch 11

Pattern	Intent (one line)	Use when	Java / JDK example	Commonly confused with	Where
				Strategy = composition, whole algorithm varies)	
Visitor	Add operations to a stable set of classes without changing them (double dispatch)	Many new operations over a fixed type set (ASTs)	`javax.lang.model.element.ElementVisitor`; `FileVisitor` is named after it but is really a tree-walk callback	Iterator; in Java 21, **sealed types + pattern-matching** `switch` usually replace it	Ch 11

Beyond GoF: extras you will meet

Pattern	Intent (one line)	Use when	Java / real example	Commonly confused with	Where
Simple Factory (idiom)	One `create(type)` method maps a type value to a class	Small, stable set of types chosen by config	`Calendar.getInstance()`, `EnumSet.noneOf`	Factory Method	Ch 8
Object Pool	Borrow and return a bounded set of expensive objects	DB connections, limited resources	HikariCP, `ThreadPoolExecutor`	Flyweight	Ch 8
Null Object	A do-nothing implementation instead of `null`	`null` checks everywhere for an optional collaborator	`Collections.emptyList()`, `OutputStream.nullOutputStream()`	`Optional` (for lookups)	Ch 11 · Beyond GoF
Dependency Injection	Receive collaborators (as interfaces) through the constructor	Classes create their own collaborators; hard to test	Spring constructor injection; a hand-wired `main`	Service Locator (anti-pattern: the class asks for its dependencies)	Ch 11 · Beyond GoF
Repository	Collection-like interface over storage	Business logic mixed with storage code	Spring Data `CrudRepository`	DAO (table-oriented)	Ch 11 · Beyond GoF
Specification	A named, combinable business rule (`isSatisfiedBy`, and/or/not)	Same rule in queries and validation; finder-method explosion	`Predicate`; Spring Data JPA `Specification`	—	Ch 11 · Beyond GoF
MVC	Separate model, view and controller (Observer + Strategy + Composite)	UI, input handling and rules tangled	Spring MVC; Swing `JTable` + `TableModel`	MVP, MVVM (relatives)	Ch 11 · Beyond GoF
Event Sourcing (lite)	Store the events; derive current state by replaying them	Full history, audit, time travel	Append-only logs	Plain audit log	Ch 11 · Beyond GoF

"Which pattern?" by problem symptom

Combined from Ch 8 · How to pick a pattern, Creational patterns compared, Ch 9 · Structural comparison, Ch 11 · Choosing a behavioral pattern and Ch 7 · Refactoring toward patterns.

If you see / need…	Use	Not to be confused with
One operation, several interchangeable rules chosen at run time (pricing, matching, routing)	**Strategy**	State, Template Method
The class to create depends on a type value; caller shouldn't know the concrete class	**Factory** (Simple Factory / Factory Method)	Abstract Factory
Products must always match as a family	**Abstract Factory**	Factory Method
Constructor has ~6+ arguments, many optional	**Builder**	—
Another one of *this*, cheaply or without knowing its class	**Prototype**	Memento
Exactly one instance must exist	**Singleton** (or DI singleton scope)	A convenient global handle
Expensive objects created and destroyed repeatedly	**Object Pool**	Flyweight
Existing class has the wrong interface	**Adapter**	Facade, Bridge
Two dimensions multiplying classes (`WindowsButton`, `MacButton`, …)	**Bridge**	Adapter
Tree of part-whole objects treated the same	**Composite**	Decorator
Add behaviour in combinations without editing the class	**Decorator**	Proxy
Every caller repeats the same five steps into a subsystem	**Facade**	Mediator
Millions of near-identical objects	**Flyweight**	Object Pool
Lazy load, access control, remote, caching (same interface)	**Proxy**	Decorator
A request must pass several checks; any may reject it	**Chain of Responsibility**	Decorator (all layers run)
Undo/redo, queue, schedule, retry or log *requested actions*	**Command**	Strategy
Evaluate user-written rules in a small language	**Interpreter** (+ Specification)	Visitor
Walk a collection/tree/paged API without exposing structure	**Iterator**	Visitor
Many objects depend on each other and the **order** of reactions matters	**Mediator**	Observer
Roll back to an earlier state without exposing internals	**Memento**	Command undo
One change must reach an open-ended set of listeners	**Observer** (or a broker)	Mediator
Three or more methods repeat one `switch` over a status; legal actions depend on it	**State** (or enum + guarded method for 3–4 states)	Strategy
Several classes share a long fixed flow, differ in a few steps	**Template Method**	Strategy
Many new operations over a stable set of types	**Visitor**, or sealed types + pattern matching	Iterator
`null` checks everywhere	**Null Object** / `Optional`	—
Classes create their own collaborators	**Dependency Injection**	Service Locator
Business logic mixed with storage	**Repository**	DAO
Same rule duplicated in queries and validation	**Specification**	—

Three questions for the famous look-alikes (Ch 11 · Choosing a behavioral pattern):

1. Do the alternatives reference each other? Yes → **State**. No → **Strategy** (or Bridge if chosen once at wiring).
2. Can one participant stop the flow? Yes → **Chain of Responsibility**. Everyone always runs → **Decorator**. Everyone is told, nobody stops anyone → **Observer**.
3. Does a central object decide what happens next? Yes → **Mediator**. Each listener decides for itself → **Observer**.

The six that come up in almost every round (Ch 1 · Patterns in this round, Ch 8 · How to pick a pattern)

Pattern	Signal	Do not use when
Strategy	One operation, several swappable rules	Only one rule exists and nothing hints at a second
Factory	Creation depends on a type value	Only one concrete class exists
Observer	Several listeners the source should not know	A single listener the source already references
State	Behaviour depends on status; many transitions	3–4 states handled by one guarded method
Singleton	Genuinely exactly one	You just want a global handle
Builder	Many optional fields; long constructor	A handful of required fields

Common mistakes (Ch 8 · Anti-patterns)

Anti-pattern	Fix
God Object (one class does everything)	Split by responsibility; push behaviour to the data owners
Golden Hammer (favourite pattern everywhere)	Start from the problem's signal
Pattern overuse ("patternitis") — factories for one class, single-implementation interfaces	YAGNI: add the pattern when the second variant arrives
Singletonitis (singletons as globals)	Dependency injection
Cargo-cult patterns (structure without intent)	Name the property the pattern gives you, or drop it

In the interview

Use **one or two** patterns in a 45-minute design, placed where their signal appears, and **name one you rejected** and why (Ch 8 · How to pick a pattern, self-check).

22.8 Concurrency cheat sheet

Source: Chapter 12 (concepts) and Chapter 13 (classic problems).

TL;DR (Ch 12)

Races come from **shared mutable state**. Make the critical section atomic, or remove sharing/mutability. Visibility needs **happens-before** edges. Prevent deadlock with **lock ordering** or **tryLock + back-off**. In LLD: name the contended resource, keep **check + claim atomic**, lock fine-grained and in order, and say what changes with many servers.

Primitive | use for | gotcha

Primitive	Use for	Gotcha	Where
`synchronized`	Making a multi-step update atomic (mutex + visibility)	Don't lock on a changing reference, a boxed value or a string literal; no slow I/O while holding it; readers must use the same lock	Ch 12 · Mutex and synchronized
`ReentrantLock`	Same as `synchronized` plus try/timed/fair locking and `Condition`s	Call `lock()` **just before** `try`, `unlock()` in `finally`; fairness is noticeably slower	Ch 12 · ReentrantLock and reentrancy
`tryLock(timeout)`	Give up if the lock is busy; deadlock prevention with back-off	Never `unlock()` when `tryLock` returned `false`; don't retry in a tight loop (livelock)	Ch 12 · Try-lock & timed locking
`ReentrantReadWriteLock`	Many readers, few writers	Cannot upgrade read → write (self-deadlock); don't modify under the read lock; writer starvation in non-fair mode	Ch 12 · Read-write locks
`StampedLock`	Read-mostly data with optimistic reads	**Not reentrant**; `validate` the stamp before using values read optimistically	Ch 12 · Read-write locks
`volatile`	Visibility of a flag or a published immutable snapshot	Gives visibility, **not** atomicity: `count++` is still a race	Ch 12 · Java Memory Model
`AtomicInteger` / `AtomicLong` / CAS	Lock-free counters and single-variable claims	Separate `get()` then `set()` is still a race — use `compareAndSet` / `updateAndGet`; no side effects in the lambda (may run several times); ABA → `AtomicStampedReference`	Ch 12 · CAS & atomics
`LongAdder`	Very hot counters (metrics)	Use instead of `AtomicLong` when contention is high	Ch 12 · CAS & atomics
`Semaphore(N)`	Limit concurrent access to N; signalling between threads	Release in `finally`; don't release when `acquire` never succeeded	Ch 12 · Semaphores
`wait/notifyAll`, `Condition`	Waiting for a condition on shared state	Always `while`, never `if` (spurious wakeups); hold the monitor; `notify()` can wake the wrong thread; state change without notify = lost wakeup	Ch 12 · Condition variables
`CountDownLatch`	Wait for N events once	One-shot; `countDown()` in `finally`, `await(timeout)`	Ch 12 · CountDownLatch and CyclicBarrier
`CyclicBarrier` / `Phaser`	N threads meet at each phase (reusable)	Fewer threads than parties → everyone waits forever	Ch 12 · CountDownLatch and CyclicBarrier
`ConcurrentHashMap`	Shared map	`containsKey` then `put` is a race — use `putIfAbsent` / `computeIfAbsent` / `merge`; no slow work or recursive updates inside `compute`; no `null` keys/values	Ch 12 · Concurrent collections
`CopyOnWriteArrayList`	Read-mostly lists (listener lists)	Every write copies the array	Ch 12 · Concurrent

Primitive	Use for	Gotcha	Where
			collections
`BlockingQueue` (bounded)	Handing work between threads (producer–consumer)	Unbounded queue → no back-pressure → memory blow-up; one poison pill per consumer	Ch 12 · Producer-consumer
`ExecutorService` / `ThreadPoolExecutor`	Running tasks on a bounded pool	Call `shutdown()`; `newFixedThreadPool` has an **unbounded** queue; exceptions in `submit`ted tasks are lost unless you call `get()`; tasks waiting on tasks in the same pool → thread-starvation deadlock	Ch 12 · Creating threads, Thread pool pattern
Virtual threads	Many blocking I/O tasks	Pinning (Java 21–23: blocking inside `synchronized`); heavy `ThreadLocal`s waste memory	Ch 12 · Creating threads, Immutability & confinement
`ForkJoinPool` / parallel streams	Splitting CPU work	Use a threshold: tiny tasks cost more to schedule than to run; subtasks must touch disjoint data	Ch 12 · Choosing the tool, Ch 13 · Parallel merge sort
`CompletableFuture`	Combining async results (`thenApply`, `thenCombine`, `allOf`)	Handle errors with `exceptionally` rather than calling `get()` at every step	Ch 12 · Creating threads
`ThreadLocal`	Per-thread state (thread confinement)	Clear it (`remove()`) in pooled threads, or values leak into the next task	Ch 12 · Immutability & confinement
Immutability / confinement	Avoiding synchronization entirely	`final List` is not immutable if the list is mutable and exposed	Ch 12 · Immutability & confinement

Which `java.util.concurrent` class for which job (Ch 12 · Choosing the tool)

Counter → `AtomicInteger`/`AtomicLong` (`LongAdder` if very hot) · stop a loop from another thread → `volatile boolean` or `interrupt()` · atomic multi-step update → `synchronized`/`ReentrantLock` · give up if busy → `tryLock(timeout)` · many readers, few writers → `ReentrantReadWriteLock`, `StampedLock` or an immutable snapshot in a `volatile` field · limit access to N → `Semaphore(N)` · wait for a condition → `Lock` + `Condition` (or `wait/notifyAll`) in a `while` loop · wait for N events once → `CountDownLatch` · N threads meet each phase → `CyclicBarrier`/`Phaser` · hand work between threads → bounded `BlockingQueue` · shared map → `ConcurrentHashMap` with `putIfAbsent`/`compute`/`merge` · read-mostly list → `CopyOnWriteArrayList` · run tasks → bounded `ExecutorService`, virtual threads for blocking I/O · split CPU work → `ForkJoinPool`/parallel streams · combine async results → `CompletableFuture` · avoid synchronization → immutability, thread confinement.

Deadlock checklist (Ch 12 · Deadlock, Livelock, Starvation)

A deadlock needs **all four Coffman conditions**; break any one and it cannot happen.

#	Condition	How to break it
1	Mutual exclusion	Lock-free/immutable data, shareable resources (read locks)
2	Hold and wait	Acquire everything at once, or `tryLock` and release everything on failure
3	No preemption	Timeouts (`tryLock(timeout)`), leases that expire
4	Circular wait	**Global lock ordering** (always lock the lower id first)

- [ ] Do I ever hold two locks? If yes, is there **one global order** (account id, seat id), with a tie-breaker for equal keys?

- [ ] Do I call a **listener, callback or another object's synchronized method while holding my lock**? Move it outside the lock.
- [ ] Do tasks in a bounded pool **wait for other tasks in the same pool**? (thread-starvation deadlock)
- [ ] Are retries randomised (**jitter**) so two threads don't back off in lockstep? (livelock, Ch 12)
- [ ] Could one thread wait forever while others progress? Consider fairness (Ch 12 · Starvation).
- [ ] Detection: a thread dump (`jstack` / `jcmd`) prints "Found one Java-level deadlock"; `ThreadMXBean.findDeadlockedThreads()`.

Deadlock vs livelock vs starvation: nobody progresses and all wait / everybody runs but nobody progresses / some progress while one never does (Ch 12 · self-check).

The LLD concurrency recipe (Ch 12 · Concurrency in LLD interviews)

1. **Name the shared mutable state** (free spots, seat → owner, stock).
2. **State the invariant** ("a seat has at most one booking", "stock ≥ 0").
3. **Find the critical operation** — usually a check-then-act (*find free → assign*).
4. **Pick the smallest tool that makes it atomic:** one item → `queue.poll()`, `putIfAbsent`, `compareAndSet`; several items all-or-nothing → per-item locks **in sorted order**; across servers → DB constraints/transactions, optimistic versioning, or a distributed lock.
5. **Choose granularity:** per floor / show / SKU, not one global lock.
6. **Handle time:** holds with expiry, timeouts, idempotent retries.

Classic problems (Ch 13 · summary)

Problem	Key primitive	Invariant to state
FooBar (Ch 13 · Print FooBar Alternately)	2 semaphores	Exactly one permit exists across both
Zero Even Odd (Ch 13 · Print Zero Even Odd)	3 semaphores	Zero dispatches; one permit circulates
FizzBuzz (Ch 13 · Fizz Buzz Multithreaded)	monitor + `notifyAll`	Predicates partition 1..n
H2O (Ch 13 · Building H2O)	2 semaphores + barrier	≤ 2 H and ≤ 1 O per round
Dining Philosophers (Ch 13)	ordered locks / n−1 semaphore	No circular wait
TTL cache (Ch 13 · Thread-safe cache with TTL)	`ConcurrentHashMap` + `compute` + conditional remove	Never return or keep expired data
Striped map (Ch 13 · Striped concurrent map)	lock array	Each bucket belongs to exactly one stripe
Blocking queue (Ch 13 · Thread-safe bounded Blocking Queue)	lock + 2 conditions	0 ≤ count ≤ capacity; wait in loops
Bloom filter (Ch 13 · Concurrent Bloom Filter)	atomic OR	Bits only go 0 → 1; no false negatives
Merge sort (Ch 13 · Parallel merge sort)	Fork/Join	Subtasks touch disjoint ranges
Token-bucket rate limiter (Ch 13)	lock or CAS on immutable state	Refill + take is one atomic step

Common mistakes (Chapter 12)

- A global `synchronized` on `ParkingLot.park()` — correct, but serialises every gate.
- Checking availability with one call and booking with another (check-then-act).
- Locking seats in the order the user clicked them (deadlock between two multi-seat bookings).
- Forgetting that in-memory locks don't work across multiple app servers.
- Swallowing `InterruptedException` — rethrow, or restore the flag with `Thread.currentThread().interrupt()` (Ch 12 · Thread states).

- Calling `run()` instead of `start()` (Ch 12 · Thread states).

22.9 Case-study map

Source: Chapters 14–21. "Concurrency concern" is the race each chapter guards against, and "atomic step" is how.

Everyday objects (Chapter 14)

Problem	Key entities	Patterns used	Concurrency concern	Where
Parking lot	`ParkingLot`, `ParkingFloor`, `ParkingSpot`, `Vehicle`, `Ticket`, `DisplayBoard`, `SpotType`/`VehicleType` enums	Strategy (pricing, spot-finding), Factory (electric spots), Observer (boards); Singleton rejected	Two gates take one spot → atomic `pollFirst` + CAS; per-ticket lock for payment; `putIfAbsent` for plates	Ch 14
Vending machine	`VendingMachine`, `Slot`, `Inventory`, `CashBox`, `Denomination`, `VendingState` + 5 states	**State** (the whole design)	`synchronized` machine; change-making by bounded coin-change DP, not greedy	Ch 14
Coffee machine	`CoffeeMachine`, `Recipe`, `RecipeBook`, `IngredientStore`, `OrderResult` (sealed)	Observer (low stock), recipes as data	Two orders take the last milk → one `ReentrantLock` around check + deduct; notify outside the lock	Ch 14 · Coffee Vending Machine
ATM	`Atm`, `AtmState` + 4 states, `Transaction` (sealed), `NoteHandler`, `CashDispenser`, `BankService`, `Account`	State (session), Command (transactions), Chain of Responsibility (note cassettes)	Overdraw from two ATMs → per-account lock; transfers lock in id order	Ch 14
Elevator	`Elevator`, `StopList` (two sorted sets), `Door`, `Dispatcher`, `DispatchStrategy`, `Building`	Strategy (dispatch) + modes; LOOK scheduling	Per-car lock, synchronized dispatcher; tick-driven `step()`	Ch 14 · Elevator System
Traffic signal	`Intersection`, `TrafficLight`, `Phase`, `SignalState` + 5 states, `TimingPolicy`, `SafetyMonitor`	State (timed), Strategy (timing), Observer (lights)	Synchronized tick; safe with a timer thread plus emergency threads	Ch 14 · Traffic Signal Control System
Shipping locker	`LockerStation`, `Compartment`, `Booking`, `CompartmentSelector`, `PickupResult` (sealed), `LockerListener`, `LockerNetwork`	Strategy (door choice), Observer (SMS), enum state machines, sealed results	Two carriers, one door → station lock around choose-and-mark; lazy expiry of holds and pickup windows; hashed single-use codes, keypad lockout	Ch 14 · Shipping Locker System

Booking & inventory (Chapters 15–16)

Problem	Key entities	Patterns used	Concurrency concern	Where
Library	`Book` (title) vs `BookItem` (copy), `Member`, `BookLending`, `LendingService`, `Catalog`	Strategy (fines), notifier interface; FIFO reservation queue per title	Last copy lent twice → lock member then copy (fixed order)	Ch 15 · Library
Movie tickets (BookMyShow)	`Show`, `ShowSeat`, `Booking`, `SeatLockProvider`, `BookingService`, `CancellationPolicy`	Strategy (refund rule), gateway behind an interface	Book `ShowSeat`s; all-or-nothing hold with TTL → pay → atomic commit; refund if hold lost	Ch 15 · BookMyShow
Concert tickets	`SeatedInventory`, `GaPool`, `SeatState` (sealed), `Hold`, `VirtualWaitingRoom`, `PurchaseLimiter`	Sealed seat states, waitlist	Extreme contention → CAS per seat, CAS counter for GA, FIFO waiting room, per-user limits, TTL holds with a reaper	Ch 15 · Concert tickets
Hotel	`Hotel`, `Room`, `RoomBooking`, `Invoice`, `RoomCharge`, `Payment` subclasses, `FrontDesk`	Template Method (`RoomCharge`)	Room × night reserved atomically in `synchronized Room.tryReserve`	Ch 15 · Hotel
Car rental	`Vehicle`, `DateRange`, `Reservation`, `Bill` / `BillItem`, `ReservationAddOn`, `PricingStrategy`	Template Method (add-ons), Strategy + Decorator (pricing), state machine	Availability is about **dates** → `Vehicle.tryBook(range)` synchronized	Ch 16 · Car Rental
Restaurant	`Menu` / `MenuSection` / `MenuItem`, `Table`, `Reservation`, `Order`, `Meal` / `MealItem`, `Kitchen`, `Check`	Observer (order ready), Strategy (chef assignment, payment), state machine	Two receptionists, one table → `Table.tryHold(time)` synchronized; listeners after the lock	Ch 16 · Restaurant
Airline	`Flight` vs `FlightInstance`, `Seat` vs `FlightSeat`, `FlightReservation`, `Itinerary`, `FareStrategy`	Strategy + Decorator (fares), Facade	Selling one seat twice → `FlightInstance.holdSeats` all-or-nothing under the instance lock	Ch 16 · Airline
Course registration	`Course` vs `Section`, `Student`, `RegistrationRule`s, `Registrar`	Chain of rules / Specification	Overselling a section → section lock inside a student lock (fixed order)	Ch 16 · Course Registration

Games (Chapter 17)

Problem	Key entities	Patterns used	Concurrency concern	Where
Blackjack + deck	`Card`, `Deck<T>`, `Shoe`, `Hand`, `BlackjackScoring`, `PlayerStrategy`, `DealerPolicy`	Strategy (player, dealer); injected randomness	One table = one thread; shared balances need an atomic wallet service	Ch 17 · Blackjack
Chess	`Piece` (sealed) + subclasses, `SlidingPiece`, `Board`, `Move`, `Rules`, `Game`	Polymorphic pieces, Factory Method, Command + Memento (undo)	`synchronized` game so a move and a resignation don't interleave	Ch 17 · Chess
Tic-tac-toe	`Board` (row/col/diagonal counters), `Game`, `Player`, `MoveStrategy`	Strategy (players)	Many games in a `ConcurrentHashMap`; per-game lock	Ch 17 · Tic-Tac-Toe
Snake & ladder	`Board`, `Jump`, `Dice`, `Rules`, `OverflowRule`, `WinStrategy`, `Game`	Strategy (dice, overflow, win)	Each game its own `Random` and its own thread of turns	Ch 17 · Snake and Ladder
2048	`LineMerger`, `Board`, `TileSpawner`, `Game`, `Direction`	Strategy (spawner); one pure merge function	Single-threaded; `synchronized move` if UI and input threads both call it	Ch 17 · 2048

Platforms (Chapters 18–19)

Problem	Key entities	Patterns used	Concurrency concern	Where
Stack Overflow	`Post` (sealed) → `Question` / `Answer` / `Comment`, `Member`, `Tag`, `Bounty`, `QuestionIndex`	Strategy (reputation), Observer (badges), Facade; inverted index	Two votes at once → `synchronized` post; `AtomicInteger` reputation	Ch 18
Cricinfo	`Match` → `Innings` → `Over` → `Ball`, `Team`, `Playing11`, `StatQuery`	Template Method (stat queries), Observer (live pages)	Two scorers → `synchronized` match; observers in `CopyOnWriteArrayList`	Ch 18
LinkedIn	`Member`, `Profile`, `SocialContent`, `ConnectionInvitation`, `ConnectionService`, `SearchIndex`, `FeedService`	State (invitation), Strategy (feed ranking, notifications), Facade	Symmetric connections → all graph changes synchronized on the service	Ch 18
Facebook	`Member`, `Audience` (sealed), `PrivacyList`, `FriendService`, `VisibilityService`, `NewsFeedService`	Strategy (feed ranker), Observer (notifications)	Accept can't leave a one-way friendship → synchronized service, concurrent adjacency sets	Ch 18
Amazon	`Product`, `ShoppingCart`, `Order`, `OrderLog`, `Payment`, `Shipment`, `Customer` → `Guest` / `Member`	Strategy (payment), Observer (notifications), order state machine, Facade	Last unit, two buyers → `synchronized Product.reserve`; all-or-nothing multi-line reservation	Ch 19 · Amazon
Stock brokerage + exchange	`Order` → Market/Limit/StopLoss/StopLimit, `OrderPart`, `StockLot`, `OrderBook`, `MatchingEngine`	Observer (fills), Template Method (statements), trade-price policy	Over-spending → reserved funds; one lock per symbol; price-then-time matching	Ch 19 · Stock Brokerage
Auction	`AuctionListing`, `Bid`, `BidResult`, `AuctionService`, `AuctionListener`	Observer, Specification (search), status state machine	Concurrent bids → all bid rules in one per-listing critical section; notify after unlocking	Ch 19 · Auction
Music streaming (Spotify)	`Song` / `Album` / `Artist`, `MusicLibrary`, `Playlist`, `MusicPlayer`, `PlaybackOrder`, `Recommender`	Strategy (play order, recommendations), Observer (plays), plan rules in an enum	Two devices press "next" → `synchronized` player methods	Ch 19 · Music Streaming
Food delivery (Swiggy) + FoodKart	`Restaurant`, `Cart`, `Order`, `DeliveryAgent`, `DeliveryAssignmentStrategy`, `RestaurantSelectionStrategy`	Strategy (assignment, selection, payment), Observer, order state machine	Kitchen capacity reserved atomically; riders claimed with CAS	Ch 19 · Food Delivery
Grocery store (POS)	`Register`, `Sale`, `LineItem`, `PricingEngine`, `ItemPromotion` / `BasketPromotion`, `TaxPolicy`, `Tender` (sealed), `Receipt`, `Inventory`	Strategy (offers, tax, payment gateway), Facade (register), Observer (stock), sale state machine	Registers share stock → atomic `merge`, may go negative, edge-triggered alerts; returns claimed with `compute`	Ch 19 · Grocery Store

Pattern-driven components (Chapter 20)

Problem	Key entities	Patterns used	Concurrency concern	Where
Splitwise	`Expense`, `SplitStrategy` (Equal/Exact/Percent), `Share`, `Ledger`, `DebtSimplifier`	Strategy (splits)	One signed number per pair, one mutator; `BigDecimal`, no lost cents	Ch 20
Notification service	`NotificationChannel` (abstract), channels, `SendRule`, `ChannelFactory`, `DeliveryListener`	Template Method (channels), Strategy (send rules), Observer, Simple Factory, Adapter	Rate-limit check-then-act gap; idempotency	Ch 20
In-memory cache / LRU	`Cache<K,V>`, `Node`, `DoublyLinkedList`, `EvictionPolicy`, `Cache.Builder`, `ShardedCache`	Strategy (eviction), Observer (removals), Builder	One lock (a `get` is a write in LRU); listeners outside the lock; shard to scale	Ch 20
Ride sharing (Uber)	`Rider`, `Driver`, `Trip`, `MatchingStrategy`, `PricingStrategy`, `SurgePricing`	Strategy ×3, Decorator (surge), trip state machine	Double-booking a driver → atomic claim (`AtomicBoolean`)	Ch 20 · Ride sharing
Logging framework	`Logger`, `LogManager`, `LogRecord`, `Appender`, `AsyncAppender`, `Formatter`	Chain of Responsibility (hierarchy), Singleton (`LogManager`), Strategy (formatters), Template Method (appender), Decorator (async)	Thread-safe sinks; never block on a full queue; never crash the app	Ch 20
Pub-sub	`Broker`, `Topic`, `Subscription`, `Subscriber`, `Message`	Observer through a broker	One dispatcher per subscription; ack after processing (at-least-once) → idempotent subscribers	Ch 20 · Pub-sub
Rate limiter	`Limiter` (TokenBucket, FixedWindow, SlidingWindowLog/Counter), `CompositeLimiter`, `LimiterFactory`	Strategy, Simple Factory, Composite, Facade	Atomic refill-check-take; one limiter per key via `computeIfAbsent`; Redis + Lua across servers	Ch 20

Machine-coding problems (Chapter 21)

Problem	Key entities	Patterns used	Concurrency concern	Where
Task management	`Task`, `TaskStatus`, `TaskService`, `TaskRepository`, `TaskFilter`	Specification (filters), Strategy (sorts), Observer, enum state machine	Per-task `synchronized`	Ch 21 · Task management
Digital wallet	`Wallet`, `Transaction`, `Money`, `Offer`, `WalletService`	Strategy (offers), idempotency key	Lock both wallets in a global (id) order; debit before credit; total conserved	Ch 21 · Digital wallet
Event calendar (Flipkart)	`TimeSlot` (half-open), `Event`, `EventStore`, `RepresentativeSelector`, `CalendarService`	Strategy (representative), plan-then-save	Atomic check-and-book	Ch 21
Coding Blox (Flipkart)	`Contest`, `Level`, `User`, `Question`, `SolveSimulator`, `CommandProcessor`	Command map, Strategy (solver, scoring)	Atomic rating update; don't run a contest twice	Ch 21
PropertyHunt (ClearTrip)	`Property`, `SearchCriteria` (+ Builder), `PropertyService`	Builder + predicate list, Strategy (sort)	Synchronized mark-sold	Ch 21
Ledger Co (Navi)	`Loan`, `LumpSum`, `Balance`, `InterestPolicy`, `LedgerService`	Strategy (interest), derived state, command loop	Synchronized lump sums	Ch 21
Inventory management	`Product`, `Warehouse`, `StockMovement`, `AllocationStrategy`, `ReorderListener`	Strategy (allocation), Observer (edge-triggered alerts)	Overselling → lock per SKU	Ch 21 · Inventory
Unix file search (`find`)	`Node` → `FileNode` / `DirectoryNode`, `FileFilter`, `AndFilter` / `OrFilter` / `NotFilter`, `SearchCriteria` (+ Builder), `FileSearcher`, `FindCommandParser`	Specification + Composite (filters and file tree), Builder, recursive-descent parser	Weakly consistent children map; read-only fork/join walk	Ch 21 · Unix File Search

> **TL;DR — what every case study teaches**
>
> - **Name the real unit of inventory** (copy, show seat, room-night, vehicle-day) (Ch 15 summary).
> - **Every "if free then take" is one atomic step** — a monitor, a CAS, or a conditional `UPDATE` (Ch 15 summary).
> - **Put rules on the object that owns the data**; the service only orders the steps (Ch 16, Ch 20 summaries).
> - **Strategy** for the rule most likely to change; **Observer** for "tell someone when it happens"; **State** (or an enum state machine) for lifecycles.
> - **Inject** time and randomness; money in `BigDecimal` or integer cents; call listeners **outside** locks.

22.10 Machine-coding and edge-case checklists

Source: Ch 21 · Machine-coding round (and Ch 1 · Interview code).

Machine-coding checklist

Before coding (0–20 min)

- [ ] Read the statement twice; mark **mandatory vs bonus**; ask clarifying questions.
- [ ] Put the statement's sample input into the driver as a `List<String>` of commands.

- [ ] Ask: "Where does it need to be thread-safe?" (wallets and inventory: always yes).
- [ ] Nouns → entities, verbs → service methods. Pick the 1–2 patterns the statement begs for ("can be sorted by X or Y" → Strategy).
- [ ] Create the six packages: **model, repository, service, strategy, exception, driver**.

While coding (20–75 min)

- [ ] Implement mandatory requirements **in the statement's order**; run the driver after each one (at least every 15 minutes).
- [ ] Entities guard their own invariants (e.g. `Task.changeStatus` rejects `TODO → DONE`).
- [ ] Services return data and throw domain exceptions; **no `System.out` in services**.
- [ ] Repository = interface + `ConcurrentHashMap` implementation; IDs from an `AtomicInteger`.
- [ ] Driver = `Map<String, Consumer<String[]>>` command processor; catch domain exceptions per command, **print and continue**.

Before you stop (75–90 min)

- [ ] Edge cases and validation done (see below); custom exceptions with readable messages.
- [ ] Thread-safety where money or stock is shared; say where the lock is and why.
- [ ] Bonus only after every mandatory feature runs.
- [ ] Clean names, delete dead code, run everything once more, prepare a 2-minute walkthrough.

Rubric the interviewer uses (Ch 21 · Machine-coding round)

(1) **Working, demo-able code** — thin end-to-end path in the first hour, never finish non-compiling; (2) **functional completeness** — mandatory first, in order; (3) **clean models** — enums for fixed sets, records for values, classes for changing things; (4) **separation of concerns** — Driver → Service → Repository; (5) **extensibility** — new rule = new class; (6) **edge cases** — custom exceptions, the driver keeps going; (7) **readability** — names from the statement's nouns and verbs; (8) **concurrency (when asked)** — say where the lock is, demo with a pool; (9) **tests (bonus)** — an `assert`-style self-check in `main` is enough.

Edge-case checklist (Ch 1 · Interview code, Ch 21 · Machine-coding round, case studies)

Category	Questions to ask your code
Input	`null`, empty, negative, zero, duplicate IDs, unknown IDs, invalid numbers
Ownership	Wrong user acting on someone else's object; a player moving out of turn
Capacity	Full lot, zero seats left, empty playlist, max loans reached, party larger than any table
State	Paying twice, exiting without entering, cancelling a delivered order, moving after game over, running a contest twice
Time	Exit before entry, holds/bookings expiring, part hours (round up?), an event exactly at the end instant (half-open intervals), late returns
Money	`BigDecimal` scale and rounding; no lost cents in splits; compare with `compareTo`, not `equals` (Ch 4 money note)
Concurrency	Two threads booking the same seat/spot/rider; multi-item bookings locking in sorted order; retries that must be idempotent
Failure	Payment declined after a hold; hold lost before payment → compensating refund; listener throws
Randomness	Seeded `Random` or a scripted strategy so the demo is repeatable

Common mistakes (Ch 21 · Machine-coding round)

Besides the Chapter 1 mistakes in 22.2: 20 minutes on a perfect diagram (enums/records first); one `Manager` with 15 maps (a service per area + repositories); `System.out` in services (the driver prints); `double` money (`BigDecimal` or whole units in a `long`); crashing on a bad command (catch per command); bonus before mandatory; unseeded `Random` (seed it or use a Strategy); `LocalDate.now()` in logic (inject a `Clock`).

---|---| | 20 minutes on a perfect class diagram | Enums/records first; the diagram lives in names | | One `Manager` with 15 maps | One service per area + repositories | | `System.out.println` inside a service | Services return data; the driver prints | | `double` for money | `BigDecimal` with a fixed scale, or whole units in a `long` | | Crashing on the first bad command | Catch domain exceptions in the driver loop | | Bonus before mandatory | Mandatory in order; bonus last | | `Random` without a seed | Seed it, or put randomness behind a Strategy | | `LocalDate.now()` in logic | Inject a `java.time.Clock` |

22.11 Glossary

Alphabetical. "Where" gives the chapter and topic where the term is explained in full. **Not repeated here** (see the tables above): the 23 GoF patterns and the extras (Simple Factory, Object Pool, Null Object, Dependency Injection, Repository, Specification, MVC, Event Sourcing) → 22.7; the five SOLID principles → 22.4; DRY, KISS, YAGNI, Law of Demeter, Tell Don't Ask, Program to an interface, Encapsulate what varies, Loose coupling, Hollywood Principle, Separation of Concerns, Least Astonishment, Fail fast, Design by Contract and the nine GRASP guidelines → 22.5; the 24 code smells → 22.6; the four pillars, overloading/overriding, interfaces vs abstract classes, the six relationships and the other one-line OOP facts → 22.3; the `java.util.concurrent` primitives, Coffman conditions, deadlock, livelock and starvation → 22.8.

Term	Plain meaning	Where
ABA problem	A value changes A → B → A, so a compare-and-swap cannot notice the change; fix with a version stamp (`AtomicStampedReference`)	Ch 12 · CAS & atomics
Activity diagram	UML flowchart of a process, with decisions, swimlanes and fork/join	Ch 3 · Activity diagrams
Actor	Someone or something that uses the system (customer, admin, the system itself)	Ch 1 · Requirements
Aggregate (DDD)	A cluster of objects changed as one unit; only its root is reachable from outside, and the root guards the invariants	Ch 5 · DDD
Anemic domain model	Classes with data but no behaviour; the logic lives in services that pull the data out	Ch 5 · Tell, Don't Ask
Anti-corruption layer	Translation layer at a bounded context's border that keeps a foreign or legacy model out of yours	Ch 5 · DDD
Anti-pattern	A common "solution" that reliably causes more problems than it solves	Ch 8 · Anti-patterns
Append-only log	Messages are only ever added at the end; readers track their own offset	Ch 20 · Pub-sub
Association class	A class that holds data about a link itself (e.g. `Enrollment{grade}`)	Ch 3 · Class diagrams
At-least-once delivery	A message may be delivered more than once, never lost; consumers must be idempotent	Ch 20 · Pub-sub
Atomic variable	`AtomicInteger` / `AtomicLong` / `AtomicReference`: lock-free single-variable updates using CAS	Ch 12 · CAS & atomics
Back-pressure	Slowing producers when consumers can't keep up, e.g. with a bounded queue	Ch 12 · Producer-consumer
`BigDecimal`	Exact decimal type used for money; build from strings, compare with `compareTo`	Ch 1 · Interview code, Ch 4 (money note)
Bloom filter	Probabilistic set: may say "maybe present" wrongly, never says "absent" wrongly	Ch 13 · Concurrent Bloom Filter
Bounded context	Boundary inside which one domain model and one vocabulary apply consistently (often one team or service)	Ch 5 · DDD
Boy Scout Rule	Leave the code a little cleaner than you found it	Ch 6 (intro)
Caretaker / Originator	Memento roles: the originator makes and restores snapshots; the caretaker stores them without looking inside	Ch 10 · Memento
CAS (compare-and-swap)	Atomic "set to new value only if it still equals the expected value"	Ch 12 · CAS & atomics
Characterisation test	A test that records what existing code does now, before you refactor it	Ch 7 · Full source code and output
Check-then-act	Read a condition, then act on it, as two steps: a race unless made atomic	Ch 12 · Race conditions, Concurrency in LLD interviews
Circular wait	T1 waits for T2's lock while T2 waits for T1's; one of the four deadlock conditions	Ch 12 · Deadlock
Class	A blueprint for objects: fields plus methods	Ch 2 · Classes & keywords
Class diagram	UML diagram of classes, fields, methods and relationships — "the answer" in an LLD round	Ch 3 · Class diagrams

Term	Plain meaning	Where
Class explosion	Subclasses multiplying across two dimensions (`WindowsButton`, `MacButton`, …)	Ch 9 · Bridge Pattern
Clean code	Code whose intent is obvious to the next reader and which is easy to change safely	Ch 6 (intro)
Code review	A peer reads a change before merge: correctness first, then design, readability, tests, security, performance	Ch 6 · Code review checklist
Code smell	A surface symptom that usually signals a deeper design problem; not a bug by itself	Ch 7 · Code smells
Command–Query Separation (CQS)	A method either changes state or returns a value, not both	Ch 6 · Functions
Compensation	Undoing a completed step with a new action (e.g. a refund) because money can't be rolled back	Ch 15 · BookMyShow
Composition root	The one place (usually `main`) where objects are created and wired together	Ch 6 · Systems
Concurrency vs parallelism	Many tasks in progress (structure) vs many executing at the same instant (hardware)	Ch 12 · Concurrency vs parallelism
Condition variable	Lets a thread wait until shared state meets a condition (`wait/notify`, `Condition`)	Ch 12 · Condition variables
Constructor chaining	One constructor calling another with `this(...)`	Ch 2 · Classes & keywords
Critical section	Code that touches shared state and must run in one thread at a time	Ch 12 · Race conditions
Dead-letter queue	Where messages go after retries are exhausted, for later inspection	Ch 20 · Pub-sub
Deep vs shallow copy	Copying nested objects too vs sharing them with the original	Ch 8 · Prototype
Derive, don't store	Compute counts and totals from the underlying records so they can't drift	Ch 18 · summary
Design pattern	A named, reusable solution to a recurring design problem	Ch 8 · What is a design pattern?
Director	Optional Builder role that runs a fixed sequence of building steps	Ch 8 · Builder
Domain event	Past-tense record of a business fact (`OrderPlaced`) that other parts of the system react to	Ch 5 · DDD
Domain-Driven Design (DDD)	Modelling software on the business: bounded contexts, entities, value objects, aggregates, domain events, repositories	Ch 5 · DDD
Double dispatch	Choosing a method by both the element's and the visitor's type (Visitor's `accept`)	Ch 11 · Visitor
Double-checked locking	Lazy singleton that checks, locks, checks again; the field must be `volatile`	Ch 8 · Singleton
Dynamic dispatch	The JVM picks an overridden method from the object's actual class at run time	Ch 2 · Polymorphism, Compile-time vs runtime polymorphism
Entity (DDD)	Object defined by an identity that survives changes; two entities are equal when their ids are equal	Ch 5 · DDD
Expression problem	Visitor makes adding operations cheap but adding types expensive (and the reverse for plain polymorphism)	Ch 11 · Visitor

Term	Plain meaning	Where
Fairness	Serving waiting threads roughly in order so none starves; costs throughput	Ch 12 · Starvation
F.I.R.S.T.	Clean tests are Fast, Independent, Repeatable, Self-validating, Timely	Ch 6 · Unit tests
Flag argument	A boolean parameter that makes a function do two things; split it instead	Ch 6 · Functions
Fork/Join	Split a task recursively into subtasks run in parallel, then combine results	Ch 12 · Choosing the tool, Ch 13 · Parallel merge sort
Fragile base class	A change in a parent class silently breaks subclasses	Ch 5 · Composition over inheritance
Functional interface	An interface with one abstract method, usable with lambdas	Ch 2 · Interfaces
Functional requirement	A behaviour the system must have ("a customer takes a ticket")	Ch 1 · Requirements
Glob	Filename wildcard pattern: `*` any characters, `?` one character, `[0-9]` one character from a set	Ch 21 · Unix File Search
God class / God Object	One class that knows and does almost everything	Ch 8 · Anti-patterns, Ch 7 · Code smells
Golden Hammer	Applying a favourite tool or pattern to every problem	Ch 8 · Anti-patterns
GRASP	Larman's nine guidelines for assigning responsibilities to classes	Ch 5 · GRASP
Guard clause	An early check-and-return/throw at the top of a method instead of nested `if`s	Ch 7 · Refactoring catalogue, Ch 5 · Fail fast
Happens-before	Java Memory Model rule guaranteeing one action's effects are visible to another	Ch 12 · Java Memory Model
HLD vs LLD	Many services and machines vs classes inside one application	Ch 1 · The interview
Hold with TTL	A temporary claim (e.g. seats during payment) that expires automatically	Ch 15 · BookMyShow
Idempotency	Running an operation twice has the same effect as once; an idempotency key enforces it	Ch 21 · Digital wallet, Ch 20 · Pub-sub
Immutability	Objects that never change after creation; safe to share between threads	Ch 2 · Immutability & records, Ch 5 · Immutability
Intrinsic vs extrinsic state	Flyweight's shared, immutable part vs the per-use part passed in by the caller	Ch 9 · Flyweight Pattern
Invariant	A fact about an object that must always be true ("balance is never negative")	Ch 5 · Design by Contract
Inversion of Control (IoC)	The framework controls the flow and calls your code; DI is one way to achieve it	Ch 5 · Hollywood Principle, Ch 4 · SOLID Q&A
Inverted index	A map from each word/tag to the list of items containing it, for search	Ch 18 · Stack Overflow
Invoker / Receiver	Command roles: the invoker triggers commands; the receiver does the real work	Ch 10 · Command
Java Memory Model (JMM)	Rules for when one thread's writes become visible to another	Ch 12 · Java Memory Model

Term	Plain meaning	Where
Lock ordering	Always acquire locks in one global order to prevent circular wait	Ch 12 · Deadlock
Lock striping	Guarding a map with an array of locks, each covering some buckets	Ch 13 · Striped concurrent map
LOOK scheduling	Elevator keeps moving in one direction while stops remain ahead, then reverses	Ch 14 · Elevator System
Lost wakeup	A signal sent before the waiter started waiting, so the waiter sleeps forever	Ch 12 · Condition variables
LRU cache	Evicts the least recently used entry; hash map + doubly linked list gives O(1)	Ch 20 · Cache / LRU
Machine-coding round	LLD round where working, runnable in-memory code is the answer (90–120 min)	Ch 1 · The interview, Ch 21 · Machine-coding round
Matching engine	Per-symbol order book matching buy and sell orders by best price, then oldest first	Ch 19 · Stock Brokerage
Multiplicity	How many objects take part at each end of a relationship (`1`, `0..1`, `*`, `1..*`)	Ch 3 · Association
Mutex	A lock that lets only one thread into a critical section	Ch 12 · Mutex and synchronized
Non-functional requirement	A quality: thread safety, extensibility, in-memory storage, latency	Ch 1 · Requirements
Object	One instance of a class with real values	Ch 2 · Classes & keywords
Optimistic locking	Read with a version; the write succeeds only if the version is unchanged	Ch 12 · Read-write locks, Concurrency in LLD interviews
`Optional`	A container that may be empty; return it when "not found" is normal	Ch 1 · Interview code
Out-of-scope list	Written list of what you will not build; shrinks the problem	Ch 1 · Requirements
Pinning	A virtual thread stuck on its carrier thread while it blocks	Ch 12 · Creating threads
PLU code	Four-digit price look-up code for loose produce, keyed at the till together with the scale weight	Ch 19 · Grocery Store
Poison pill	A special message telling a consumer to stop; one per consumer	Ch 12 · Producer-consumer
Producer–consumer	Threads hand work through a (bounded) queue	Ch 12 · Producer-consumer
Pub-sub	Observer through a broker: publishers and subscribers don't know each other	Ch 20 · Pub-sub
Qualified association	An association looked up by a key (`Library [isbn] → Book`, i.e. a `Map`)	Ch 3 · Class diagrams
Race condition	The result depends on the timing of threads touching shared mutable state	Ch 12 · Race conditions
Rate limiter	Caps how many requests a key may make per time window (token bucket, windows)	Ch 13 · Rate limiter, Ch 20 · Rate Limiter
Record	Compact Java class for immutable data; only shallowly immutable	Ch 2 · Immutability & records
Reentrancy	A thread may re-acquire a lock it already holds	Ch 12 · ReentrantLock and reentrancy

Term	Plain meaning	Where
Refactoring	Changing internal structure without changing observable behaviour	Ch 7 · Refactoring fundamentals
Requirement	One thing the system must do, written so you can check it	Ch 1 · Requirements
Role interface	A small interface serving one kind of caller (the ISP fix)	Ch 4 · Interface Segregation Principle
Rule of three	Refactor to an abstraction at the second or third case, not the first	Ch 4 · Over-engineering
Sealed type	A type whose permitted subtypes are fixed, enabling exhaustive `switch`	Ch 11 · Visitor
Sequence diagram	UML diagram of messages between objects over time	Ch 3 · Sequence diagrams
Service Locator	Anti-pattern where a class asks a registry for its dependencies, hiding them	Ch 11 · Beyond GoF
Sharding	Splitting data across independent parts (each with its own lock) to scale	Ch 20 · Cache / LRU
Signaling	One thread tells another that something has happened (semaphore, latch)	Ch 12 · Signaling pattern
Sliding window	Rate-limit algorithm counting requests in a moving time window (log or counter)	Ch 20 · Rate Limiter
SOLID	SRP, OCP, LSP, ISP, DIP: five guidelines that keep one change from spreading	Ch 4 · What SOLID is
Split tender	Paying one bill with several payment methods (e.g. part UPI, part cash)	Ch 19 · Grocery Store
Spurious wakeup	A waiting thread wakes with no signal; why `wait()` sits in a `while` loop	Ch 12 · Condition variables
State machine diagram	UML diagram of one object's states and transitions (`event [guard] / action`)	Ch 3 · State machine diagrams
`static`	Belongs to the class, one per class, not to each object	Ch 2 · Classes & keywords
Test double	Anything standing in for a real dependency in a test: dummy, stub, spy, mock, fake	Ch 6 · Testing strategies
Test-Driven Development (TDD)	Failing test → make it pass → refactor, in very short cycles	Ch 6 · Unit tests
Thread	An independent path of execution inside a process, sharing its memory	Ch 12 · Processes vs threads
Thread confinement	Keeping data inside one thread so it never needs locking	Ch 12 · Immutability & confinement
Thread pool	A fixed set of reusable threads fed from a task queue	Ch 12 · Thread pool pattern
Thread states	NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED	Ch 12 · Thread states
Thread-starvation deadlock	Tasks in a bounded pool wait for tasks queued behind them in the same pool	Ch 12 · Deadlock, Thread pool pattern
Token bucket	Rate-limit algorithm: tokens refill at a fixed rate; each request takes one; allows bursts	Ch 13 · Rate limiter
Two hats	While coding you are either adding function or refactoring, never both at once	Ch 7 · Refactoring fundamentals

Term	Plain meaning	Where
Ubiquitous language	One vocabulary shared by domain experts and the code (`place()`, not `setStatus(2)`)	Ch 5 · DDD
UML	Unified Modeling Language: 14 diagram types, 7 structural and 7 behavioural	Ch 3 · What UML is
Use case	One thing an actor can do with the system	Ch 1 · Requirements, Ch 3 · Use-case diagrams
Use-case diagram	UML diagram of actors, use cases and the system boundary («include», «extend»)	Ch 3 · Use-case diagrams
Value object	A small immutable object defined by its values (money, date range), usually a record	Ch 2 · Immutability & records, Ch 5 · DRY, Ch 5 · DDD
Virtual waiting room	FIFO queue that admits buyers gradually during a ticket rush	Ch 15 · Concert tickets

Self-check

- **Q:** Name the eleven framework steps in order. **A:** Clarify, requirements, use cases, entities, relationships, class diagram, key flows, patterns & SOLID, code, edge cases & concurrency, extensions.
- **Q:** Where does the diamond go, and what does filled mean? **A:** At the whole; filled = composition (the part dies with the whole).
- **Q:** A `switch (vehicleType)` computes fees. Which principle, smell and pattern? **A:** OCP; Repeated Switches (if it repeats); Strategy.
- **Q:** Decorator vs Proxy vs Chain of Responsibility? **A:** Decorator adds behaviour and every layer runs; Proxy controls access; in CoR one handler can stop the flow.
- **Q:** State vs Strategy? **A:** States know each other and advance themselves; strategies are chosen from outside.
- **Q:** Four Coffman conditions, and the usual fix? **A:** Mutual exclusion, hold and wait, no preemption, circular wait; global lock ordering.
- **Q:** Why is `volatile` not enough for `count++`? **A:** Visibility, not atomicity; use `AtomicInteger` or a lock.
- **Q:** The six machine-coding packages? **A:** model, repository, service, strategy, exception, driver.
- **Q:** What makes every booking problem safe? **A:** Naming the real unit of inventory and making "if free then take" one atomic step (plus holds that expire).

TL;DR

- **Framework:** clarify → requirements → use cases → entities → relationships → diagram → flows → patterns/SOLID → code → edge cases/concurrency → extensions.
- **Classes:** different data → field; different behaviour → class; one sentence per class; composition over inheritance.
- **SOLID** diagnoses change pain; always state the cost of the fix, and know when not to apply it.
- **Patterns** come from signals: Strategy (rules vary), Factory (type value → class), Observer (many listeners), State (lifecycle), Singleton (exactly one), Builder (many optional fields). Use one or two, name a rejected one.
- **Concurrency:** find the shared state, state the invariant, make check-then-act atomic with the smallest tool, lock in a fixed order, notify outside locks.
- **Machine coding:** run early and often, mandatory first, services return data, the driver prints and keeps going.

Chapter 23 — Interview Question Bank

How to use this bank

This chapter collects the low-level design (LLD) and machine-coding questions that come up again and again in interviews, worded the way they usually appear on LeetCode Discuss interview-experience posts and in the popular public question lists (see the sources at the end). Use it as a checklist so that nothing surprises you.

- ⋆ marks a question that is asked very often. Do those first.
- **Part A** is the "Design X" prompts (parking lot, BookMyShow, Splitwise …). When this book has a full case study, the entry is a short reminder of what to say plus a link to the full design. When it doesn't, the entry is a complete outline you can build from.
- **Part B** is the concept questions (OOP, UML, SOLID, patterns, concurrency, Java specifics). Each has a short answer you can say out loud and a link to the full explanation.
- **Part C** is a minute-by-minute plan for a 60–90 minute machine-coding round.

How to drill: read the question, cover the answer, say your answer out loud (or sketch the classes), then compare. Mark the ones you stumbled on and come back to them the next day.

Part A — "Design X" (LLD and machine coding) questions

These are the "Design X" prompts that show up again and again in object-oriented design (OOD) rounds and in machine-coding rounds. An OOD round is a whiteboard-style talk about classes; a machine-coding round is 90–150 minutes of writing working code. ⋆ marks prompts asked very often, and every machine-coding favourite also gets a ⋆. When this book has a full case study, the entry here is a short reminder of what to say, followed by a link. When the book has no case study, the entry is a complete outline you can build from.

Everyday objects

Q1. ⋆ Design a Parking Lot

Asked at: Flipkart (SDE2 LLD round)

- Entities: `ParkingLot` → `Level` → `ParkingSpot` (sized: small, compact, large), `Vehicle`, `Ticket`, `EntryGate` / `ExitGate`, `FeeCalculator`.
- Patterns: Strategy for spot allocation and for pricing; Singleton (or one injected instance) for the lot; Factory for vehicles.
- What interviewers probe: two cars racing for the last free spot (claim the spot atomically), hourly vs flat pricing, and adding EV charging spots without editing old code. → *Full design:* Ch 14 — 14.1 Design a Parking Lot

Q2. ⋆ Design an Elevator / Lift System

- Entities: `ElevatorController`, `Elevator` (current floor, direction, door state), `Request` (hall call or car call), `Dispatcher`.
- Patterns: State (moving up, moving down, idle, doors open), Strategy for dispatching (nearest car, SCAN / LOOK).
- What interviewers probe: the scheduling algorithm (why plain first-come-first-served makes cars zig-zag), keeping requests consistent when many buttons are pressed at once, and overload or maintenance modes. → *Full design:* Ch 14 — 14.5 Design an Elevator System

Q3. ⋆ Design a Vending Machine

- Entities: `VendingMachine`, `Inventory`, `Product`, `Coin` / `Note`, `ChangeDispenser`.
- Pattern: State (idle, has money, dispensing, sold out) is the whole point of this question.
- What interviewers probe: making change with a limited coin stock, refunding on cancel, and what happens when a product sells out while money is inserted. → *Full design:* Ch 14 — 14.2 Design a Vending Machine

Q4. Design a Coffee Vending Machine

- Entities: `CoffeeMachine`, `Recipe` (ingredient → quantity), `IngredientStore`, `Beverage`, add-ons.
- Patterns: Decorator for add-ons (extra shot, milk), Factory for drinks, State for the machine.
- What interviewers probe: checking and deducting every ingredient as one atomic step, and refill alerts (Observer). → *Full design:* Ch 14 — 14.3 Design a Coffee Vending Machine

Q5. ⋆ Design an ATM

- Entities: `ATM`, `Card`, `Account`, `BankService`, `CashDispenser`, `Transaction` (withdraw, deposit, balance enquiry).
- Patterns: State (no card, card inserted, PIN verified), Chain of Responsibility for dispensing notes by denomination.
- What interviewers probe: what to do if cash is debited but not dispensed (reversal), PIN retry limits, and two ATMs withdrawing from one account at the same time. → *Full design:* Ch 14 — 14.4 Design an ATM

Q6. Design a Traffic Signal Control System

- Entities: `Intersection`, `TrafficLight`, `SignalPhase`, `SignalController`, `Timer`.
- Patterns: State for the light cycle, Strategy for timing plans (fixed, rush hour, sensor-driven), Observer for emergency vehicles.
- What interviewers probe: never letting conflicting directions be green together, and handling emergency override safely. → *Full design:* Ch 14 — 14.6 Design a Traffic Signal Control System

Q7. Design an Amazon Locker / shipping locker

- Entities: `LockerLocation`, `Locker` (by size), `Package`, `PickupCode`, `LockerService`.
- Patterns: Strategy for choosing a locker size, State for the locker (free, reserved, occupied).
- What interviewers probe: code expiry and returning the package to the sender, and two deliveries claiming the same locker. → *Full design:* Ch 14 — 14.7 Design a Shipping Locker System

Q8. Design a toll booth / parking-fee calculator (pricing strategies)

- This is the pricing part of the parking lot pulled out on its own: a `FeeCalculator` interface with hourly, flat, slab and weekend implementations, chosen by vehicle type and time.
- Patterns: Strategy (one class per pricing rule), optionally Decorator for surcharges and discounts layered on top.
- What interviewers probe: time zones and midnight boundaries, rounding (charge per started hour?), and keeping money as integer paise or `BigDecimal`, never `double`. → *Full design:* Ch 14 — 14.1 Design a Parking Lot (pricing section)

Q9. Design a home automation system

Requirements. A user owns devices (lights, fans, thermostats, door locks, sensors) made by different vendors. The user can switch a device on or off, set values (brightness, temperature), group actions into a "scene" ("Good night" turns off lights and locks doors), undo the last action, and create rules ("if motion after 23:00, turn on the hall light"). Schedules ("at 07:00") are nice to have.

Main classes.

- `Device` (interface): `id()`, `status()`. Subtypes `Light`, `Thermostat`, `Lock`, `MotionSensor`.
- `VendorAdapter`: wraps each vendor's protocol (Zigbee, Wi-Fi API) behind the `Device` interface.
- `DeviceCommand`: one action with `execute()` and `undo()`, for example `SetBrightness(light, 40)` remembers the old brightness.
- `Scene`: a list of commands run as one; it is itself a command.
- `Hub`: registry of devices, runs commands, keeps an undo stack.
- `Rule`: a trigger (sensor event, time) plus a condition plus a command.
- `EventBus`: sensors publish events; rules subscribe.

Patterns. Adapter (vendor protocols), Command (actions with undo), Composite (a scene is a command made of commands), Observer (sensor events reach rules), Strategy for the condition check.

Concurrency and edge cases. Send commands to one device in order: give each device its own single-threaded executor (or a lock), while different devices run in parallel on virtual threads. A device may be offline: queue the command with a timeout and report failure instead of hanging. Avoid rule loops (rule A turns on a light, which fires rule B, which turns it off) by tracking which rule caused an event and limiting the depth.

Extension question. "Support user roles: a guest can unlock the front door only between 10:00 and 18:00." Add an `AccessPolicy` checked by the `Hub` before running any command (a Proxy around devices works well).

Q10. Design a Jackpot / slot machine

Requirements. A player inserts credit, places a bet, and pulls the lever. Three (or five) reels spin and stop on symbols. A pay table decides the win. The machine tracks credit, pays out, and refuses to play when it is out of service or cannot cover the payout. Optional: a progressive jackpot shared by many machines.

Main classes.

- `SlotMachine`: holds the credit balance and the current state, and exposes `insert`, `bet`, `spin` and `cashOut`.
- `MachineState` (sealed interface): `Idle`, `HasCredit`, `Spinning`, `OutOfService`. Each state allows only its own actions.
- `Reel`: a list of symbols with weights, so a cherry can be more common than a seven.
- `RandomSource`: an interface wrapping `SecureRandom`. Tests swap in a fixed sequence, so results are repeatable.
- `PayTable`: maps a symbol combination to a multiplier. A Strategy, so different machines can pay differently.
- `JackpotPool`: one shared progressive pool. Each bet adds a small cut.

Patterns. State (the machine lifecycle), Strategy (pay rules, random source), Singleton-like shared `JackpotPool` (inject one instance rather than a static).

Concurrency and edge cases. Two machines can hit the jackpot at the same moment. Take the pool with one atomic step, `AtomicLong.getAndSet(seed)`, so only one machine wins the full amount. Use whole numbers for money. Reject a bet bigger than the credit. Go out of service if the machine's cash cannot cover the largest possible win. Log every spin (seed, result) for audits.

Extension question. "Add a bonus round after three scatter symbols." Add a `BonusRound` state that the machine moves into after the normal payout. No existing state class needs to change.

Q11. Design a music playlist

- Entities: `Song`, `Playlist` (ordered list), `Player` (current position, play mode), `User`.
- Patterns: Iterator (sequential, shuffle, repeat as different iterators) or Strategy for play order.
- What interviewers probe: shuffle without repeats until all songs played, and editing the playlist while it is playing. → *Full design:* Ch 1 — 1.3 A complete answer in five minutes

Booking and reservations

Q12. ⋆ Design BookMyShow / a movie ticket booking system

Asked at: Flipkart

- Entities: `Cinema` → `Screen` → `Seat`, `Movie`, `Show`, `ShowSeat` (seat state per show), `Booking`, `Payment`.
- Patterns: State for a seat (available, held, booked), Strategy for pricing, Observer for confirmations.
- What interviewers probe: two users picking the same seat (hold with expiry, then confirm), releasing holds when payment fails or times out. → *Full design:* Ch 15 — 15.2 Design a Movie Ticket Booking System

Q13. ⋆ Design a Library Management System

- Entities: `Book` (title info) vs `BookItem` (physical copy), `Member`, `Loan`, `Reservation`, `Fine`, `Catalog` (search).
- Patterns: Strategy for fine rules, Observer for "your reserved book is back".
- What interviewers probe: the book vs copy split, borrowing limits, and the reservation queue. → *Full design:* Ch 15 — 15.1 Design a Library Management System

Q14. ⋆ Design a Hotel Management System

- Entities: `Hotel`, `Room` (type, rate), `Guest`, `Reservation` (date range), `Invoice`, `HousekeepingTask`.
- Patterns: State for reservation and room status, Strategy for room pricing.
- What interviewers probe: date-range overlap checks, overbooking rules, and late check-out charges. → *Full design:* Ch 15 — 15.4 Design a Hotel Management System

Q15. Design a concert ticket booking system

- Like BookMyShow, but one big event with a sudden rush: seat sections, general admission, and a waiting room.
- Patterns: State for seats, plus a queue in front of booking.
- What interviewers probe: thousands of buyers at the same second, hold timeouts, and per-user ticket limits. → *Full design:* Ch 15 — 15.3 Design a Concert Ticket Booking System

Q16. ⋆ Design a car / vehicle rental system (Zoomcar)

Asked at: Flipkart (SDE2, 2024)

- Entities: `Branch`, `Vehicle`, `Reservation` (pickup and drop time and branch), `Customer`, `PricingPolicy`, `Invoice`.
- Patterns: Strategy for pricing and for vehicle selection, State for reservation lifecycle.
- What interviewers probe: availability across overlapping time windows, one-way rentals, and damage or late-return charges. → *Full design:* Ch 16 — Design a Car Rental System

Q17. Design a restaurant management system (table booking / waiting queue)

- Entities: `Table`, `Reservation`, `WaitlistEntry`, `Order`, `MenuItem`, `Bill`, `Kitchen`.
- Patterns: Command for orders sent to the kitchen, Observer for "your table is ready", State for orders.
- What interviewers probe: fitting party sizes to tables, walk-ins vs reservations, and splitting the bill. → *Full design:* Ch 16 — Design a Restaurant Management System

Q18. Design an airline management / flight booking system

- Entities: `Flight` (route) vs `FlightInstance` (a dated departure), `Aircraft`, `Seat`, `Itinerary`, `Passenger`, `Booking`.
- Patterns: State for booking, Strategy for fare rules.
- What interviewers probe: the route vs instance split, multi-leg trips booked all-or-nothing, and seat holds. → *Full design:* Ch 16 — Design an Airline Management System

Q19. Design a course registration system

- Entities: `Course`, `Section` (time slot, capacity), `Student`, `Enrollment`, `Prerequisite`, `Waitlist`.
- Patterns: Specification or Chain of Responsibility for eligibility checks (prerequisites, clashes, credit limit).
- What interviewers probe: the last seat taken by two students at once, and automatic waitlist promotion. → *Full design:* Ch 16 — Design a Course Registration System

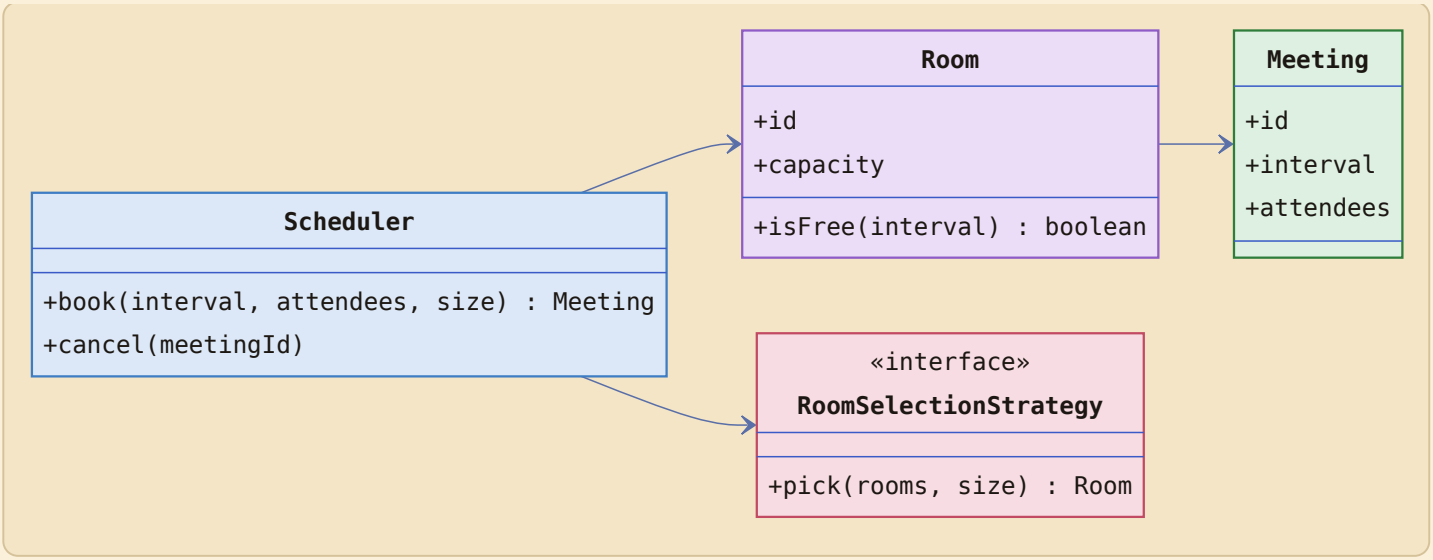
Q20. Design a learning management system / online exam portal

- Start from course registration and add `Exam`, `Question` (single choice, multiple choice, text), `Attempt`, `Grader`.
- Patterns: Strategy for grading each question type, State for an attempt (started, submitted, graded), and a timer that auto-submits.
- What interviewers probe: saving answers often so a dropped connection loses nothing, and randomising question order per student. → *Related design:* Ch 16 — Design a Course Registration System

Q21. ⋆ Design a Meeting Scheduler / meeting room booking

Requirements. An office has rooms, each with a capacity. A user books a room for a time interval with a list of attendees. The system picks a free room large enough, rejects clashes, supports cancellation, shows a room's or a person's calendar, and notifies attendees. A common follow-up asks for the first free slot that suits every attendee.

Main classes.



- `Interval`: a record `(start, end)` with `overlaps(other)`. Treat it as half-open, so 10:00–11:00 and 11:00–12:00 do not clash.
- `Room`: keeps its bookings in a `TreeMap<start, Meeting>`. To check a clash, look only at the neighbours (`floorEntry`, `ceilingEntry`), which takes O(log n).
- `RoomSelectionStrategy`: picks the smallest room that fits, or the nearest room.
- `NotificationService`: tells attendees about invites, changes and cancellations (Observer).

Concurrency and edge cases. Two people can book the same room for overlapping times. Lock per room, then check and insert under that lock. If you try rooms one by one, release each lock before taking the next, so the code cannot deadlock. Reject `end <= start`. A recurring meeting expands into instances, and the booking fails if any instance clashes (or the system offers to skip it).

Extension question. "Find the earliest 30-minute slot when all attendees are free." Collect every attendee's busy intervals, sort them, merge the overlapping ones, then scan the gaps. This is the classic merge-intervals technique.

Q22. ⋆ Design a doctor appointment booking system

Asked at: Flipkart (SDE2, 2021)

Requirements. Doctors register with a speciality (cardiologist, dermatologist). Each doctor declares available one-hour slots for the day. Patients search by speciality and see free slots, sorted by start time by default. A patient books, cancels, and views their bookings. A patient may not hold two appointments at the same time. When a slot is full, the patient can join its waitlist, and a cancellation promotes the first person waiting.

Main classes.

- `Doctor` (id, name, speciality, slots) and `Patient` (id, name, appointments).
- `Slot`: doctor, start time, status (`FREE`, `BOOKED`), booked patient, and a waitlist (a `Deque<Patient>`).
- `Appointment`: a record (id, patient, slot).
- `BookingService`: `book`, `cancel` and `joinWaitlist`. It enforces "no double booking for the patient".
- `SearchService` with a `SlotRankingStrategy`. Order by start time by default, or by doctor rating later.
- Repositories: in-memory maps, as the machine-coding round expects.

Patterns. Strategy (ranking), Observer (tell a waitlisted patient they were promoted), a simple State on `Slot`.

Concurrency and edge cases. Two patients may book the last slot together. Guard the slot status with a per-slot lock, or a compare-and-set on an `AtomicReference<SlotStatus>`. Check the patient's own clash under the patient's lock. Lock patient then slot, always in that order, so no deadlock can form. Validate that slots are whole hours inside working hours. Cancelling an already-cancelled booking is a no-op.

Extension question. "Show the trending doctor: the one with the most bookings today." Keep a counter per doctor that `book` updates, and read it through a max-heap or a sorted view.

Games

Q23. ⋆ Design Snake and Ladder

- Entities: `Board` (size, snakes and ladders as jumps), `Player`, `Dice`, `Game` (turn order queue).
- Patterns: Strategy for dice (one die, two dice, a loaded die in tests), a simple game loop.
- What interviewers probe: input-driven setup in machine coding, exact-roll-to-win rules, and chained jumps. → *Full design:* Ch 17 — Design Snake and Ladder

Q24. ⋆ Design Chess

- Entities: `Board`, `Square`, `Piece` subclasses, `Move`, `Player`, `Game` (status, history).
- Patterns: Strategy (or polymorphism) for each piece's moves, Command for moves with undo, State for game status.
- What interviewers probe: check / checkmate detection (simulate the move, then test the king), castling and en passant, and why moves should be objects. → *Full design:* Ch 17 — Design Chess

Q25. ⋆ Design a 2048 game

Asked at: Flipkart (2021)

- Entities: `Board` (grid), `Tile`, `Game` (score, state), `MoveDirection`, `TileSpawner`.
- Pattern: write one "slide and merge a row left" routine and rotate the grid for the other three directions.
- What interviewers probe: each tile merging only once per move, spawning only after a real change, and detecting game over. → *Full design:* Ch 17 — Design 2048

Q26. ⋆ Design Tic-Tac-Toe

- Entities: `Board`, `Player`, `Game`, `WinChecker`.
- Patterns: Strategy for players (human, AI), and per-row / column / diagonal counters so a win check is O(1).
- What interviewers probe: N×N boards, the O(1) win check, and keeping input handling out of the game rules. → *Full design:* Ch 17 — Design Tic-Tac-Toe

Q27. Design a deck of cards / Blackjack

- Entities: `Card` (rank, suit enums), `Deck`, `Shoe`, `Hand`, `Player`, `Dealer`, `Game`.
- Patterns: Template Method or Strategy for the dealer's rule, and a generic deck reused by other card games.
- What interviewers probe: ace counted as 1 or 11, and making a deck that works for any card game. → *Full design:* Ch 17 — Design Blackjack and a Deck of Cards

Q28. Design Minesweeper

Requirements. A rows × cols grid hides a number of mines. The player reveals a cell or toggles a flag. Revealing a mine loses the game. Revealing a number shows how many of the eight neighbours are mines. Revealing a zero opens all connected empty cells and their numbered borders. The player wins when every non-mine cell is revealed. The first click must never be a mine.

Main classes.

- `Cell`: `mine` (boolean), `adjacentMines`, and `state` (`HIDDEN`, `REVEALED`, `FLAGGED`).
- `Board`: the grid, `neighbours(r, c)`, `revealedCount`, `mineCount`.
- `MinePlacer` (interface): random placement with a seed. Tests use a fixed layout. Mines are placed *after* the first click, avoiding that cell (and its neighbours, if you like).
- `Game`: `reveal(r, c)`, `toggleFlag(r, c)`, and `status` (`PLAYING`, `WON`, `LOST`).
- `Renderer`: prints the board. Keep it separate from the rules.

Patterns. Strategy (mine placement), State for game status (no moves after a win or loss), and Observer if a UI must redraw.

Algorithm. For reveal, use a breadth-first search with a queue, not recursion. A recursive flood fill on a 1000×1000 empty board can overflow the stack. Mark each cell revealed when you add it to the queue, so no cell is processed twice. Flagged cells are not opened by the flood. Win check: `revealedCount == rows * cols - mineCount`, an O(1) test.

Edge cases. Ignore clicks outside the board, on revealed cells, or after the game ended. Reject more mines than cells minus one.

Extension question. "Add chording: clicking a number whose flags are all placed opens its other neighbours." Write it as one more method that reuses `reveal`, so no existing rule changes.

Q29. Design a Bowling Alley scoring system

- Entities: `Game`, `Frame` (rolls), `TenthFrame` (up to three rolls), `Player`, `Lane`.
- Key logic: a strike adds the next two rolls, a spare adds the next one. Score from the stored list of rolls instead of keeping running totals, so bonuses resolve naturally.
- What interviewers probe: the tenth frame, partial scores while bonuses are pending, and validating pin counts per frame.
- The scoring core for a finished game (validate rolls first: 0–10 each, at most 10 per frame outside the tenth):

```
static int score(List<Integer> rolls) {
    int total = 0, i = 0;
    for (int frame = 1; frame <= 10; frame++) {
        if (rolls.get(i) == 10) {                                  // strike
            total += 10 + rolls.get(i + 1) + rolls.get(i + 2); i += 1;
        } else if (rolls.get(i) + rolls.get(i + 1) == 10) {        // spare
            total += 10 + rolls.get(i + 2); i += 2;
        } else { total += rolls.get(i) + rolls.get(i + 1); i += 2; }
    }
    return total;   // 12 strikes -> 300; 21 rolls of 5 -> 150
}
```

This book has no Bowling Alley case study, so this outline is the whole answer here.

Q30. Design an online game engine

Requirements. Provide the core that any 2D game can build on: a loop that updates and draws the world at a steady rate, game objects with position and behaviour, keyboard and mouse input, collision detection, events between objects, loading images and sounds once, and screens (menu, playing, paused).

Main classes.

- `GameLoop`: runs a fixed-timestep update (say 60 times a second) and renders as often as possible. It catches up with several updates if a frame was slow.
- `Entity`: an id that holds `Component`s (`Position`, `Velocity`, `Sprite`, `Collider`). This is the Entity-Component-System (ECS) idea: behaviour comes from parts, not from a deep inheritance tree.
- `System`: `MovementSystem`, `CollisionSystem` and `RenderSystem`. Each one walks the entities that have the components it needs.
- `InputHandler`: maps keys to `Command` objects (`Jump`, `MoveLeft`), so key bindings can change.
- `EventBus`: lets objects publish "player died" or "coin collected" without knowing each other.
- `AssetManager`: loads each texture once and shares it.
- `SceneManager`: holds a stack of `Scene`s. Each scene is a state, such as `Menu`, `Level` or `Pause`.

Patterns. Component (composition over inheritance), Command (input), Observer (events), Flyweight (shared assets), State (scenes), Game Loop.

Concurrency and edge cases. Keep the world single-threaded. Input arrives from other threads into a `ConcurrentLinkedQueue`, and the loop drains it at the start of each tick. Asset loading can run on virtual threads, publishing "loaded" events. Use a fixed time step, so physics gives the same result on fast and slow machines.

Extension question. "Make it multiplayer." The server owns the true state and runs ticks. Clients send commands, predict movement locally, and correct themselves when the server state arrives.

Platforms and marketplaces

Q31. ⋆ Design a cab booking / ride-sharing service (Uber; Flipkart "offer and select a ride")

Asked at: Flipkart

- Entities: `Rider`, `Driver`, `Vehicle`, `Location`, `RideRequest`, `Trip`, `FareCalculator`, `MatchingService`.

- Patterns: Strategy for matching (nearest, best rated) and for fares (surge), State for the trip, Observer for live updates.
- What interviewers probe: two riders matched to one driver, cancellation rules, and the carpool variant (offer a ride with seats, select the best ride). → *Full design:* Ch 20 — 20.4 Design a Ride Sharing Service (Uber)

Q32. ⋆ Design a food delivery system (Swiggy / Zomato; Flipkart "FoodKart")

Asked at: Flipkart

- Entities: `Restaurant`, `Menu`, `Item`, `Cart`, `Order`, `DeliveryAgent`, `RestaurantSelectionStrategy`.
- Patterns: State for the order, Strategy for choosing the restaurant (lowest cost, highest rating) and assigning agents, Observer for tracking.
- What interviewers probe: restaurant capacity (max orders at once) freed when an order completes, and one-restaurant-per-cart rules. → *Full design:* Ch 19 — Design an Online Food Delivery Service (Swiggy) + FoodKart

Q33. ⋆ Design CricInfo / a cricket scoreboard (Udaan "Cricket Match Dashboard")

Asked at: Udaan

- Entities: `Match`, `Team`, `Player`, `Innings`, `Over`, `Ball` (the event that everything derives from), `Scorecard`.
- Patterns: Template Method for match formats, Observer for live updates, "derive stats from balls, don't store totals".
- What interviewers probe: extras (wide, no-ball), strike rotation at the end of an over, and the machine-coding version that prints the scorecard after every over. → *Full design:* Ch 18 — 18.2 Design Cricinfo

Q34. ⋆ Design a stock exchange / order matching engine (LLD)

- Entities: `Order` (buy or sell, limit or market, quantity, price, time), `OrderBook` per symbol, `Trade`, `MatchingEngine`.
- Data structure: for each side, a `TreeMap<price, Deque<Order>>`. The best bid is the highest key and the best ask is the lowest. Match by price first, then by time.
- What interviewers probe: partial fills, cancelling an order sitting in the book, and a single writer thread per symbol so no locks are needed. → *Related design:* Ch 19 — Design an Online Stock Brokerage System

Q35. Design an online stock brokerage system

- Entities: `Account`, `Portfolio`, `Position`, `Order` (market, limit, stop), `Exchange` adapter, `Watchlist`.
- Patterns: Strategy/Factory for order types, State for order lifecycle, Observer for price alerts, Adapter for the exchange.
- What interviewers probe: blocking funds when an order is placed, partial fills, and order status kept in step with the exchange. → *Full design:* Ch 19 — Design an Online Stock Brokerage System

Q36. Design a cryptocurrency exchange

- Same core as Q34 (order book and matching) plus wallets per currency, deposits that wait for blockchain confirmations, and withdrawals that need approval.
- Patterns: State for deposits and withdrawals, Strategy for fees.
- What interviewers probe: exact decimal amounts (`BigDecimal` with each coin's scale), and never letting a balance go negative under concurrent trades. → *Related design:* Ch 19 — Design an Online Stock Brokerage System

Q37. Design Stack Overflow

- Entities: `User` (reputation, badges), `Question`, `Answer`, `Comment`, `Vote`, `Tag`, `Bounty`.
- Patterns: Observer for reputation changes and notifications, Strategy for search and ranking.
- What interviewers probe: one vote per user per post, accepting an answer, and permissions that depend on reputation. → *Full design:* Ch 18 — 18.1 Design Stack Overflow

Q38. Design LinkedIn

- Entities: `Member`, `Profile`, `Connection`, `Invitation`, `Post`, `Job`, `Message`.
- Patterns: State for invitations, BFS for 1st/2nd/3rd degree connections, Strategy for feed ranking.

- What interviewers probe: two-sided operations (accepting adds both sides), and degree computation limits. → *Full design:* Ch 18 — 18.3 Design LinkedIn

Q39. Design Facebook / a social network

- Entities: `User`, `Friendship`, `Follow`, `Post`, `Audience` (privacy), `Page`, `Group`, `NewsFeed`.
- Patterns: one visibility policy used by feed and search, Strategy for feed ranking, pull-model feed.
- What interviewers probe: friends vs followers, privacy checks on every read, and friend suggestions. → *Full design:* Ch 18 — 18.4 Design Facebook

Q40. ⋆ Design Twitter (post / follow / feed; LeetCode 355 "Design Twitter")

- The LeetCode version: `postTweet`, `getNewsFeed` (the 10 newest tweets from self and followees), `follow`, and `unfollow`.
- Data: a map from user to followees, and per user a list of tweets with a global increasing timestamp. Build the feed by merging the users' newest-first lists with a max-heap, k-way merge style, and stop after 10.
- What interviewers probe: why you pull (merge on read) rather than push here, self-follow, and unfollowing yourself. → *Related design:* Ch 18 — 18.4 Design Facebook (feed and follow)

Q41. ⋆ Design Amazon (online shopping system / cart / checkout)

- Entities: `Product`, `Catalog`, `Cart`, `Order`, `OrderItem`, `Payment`, `Shipment`, `Inventory`.
- Patterns: State for orders, Strategy for payment and shipping, Observer for status emails.
- What interviewers probe: reserving stock at checkout vs at payment, price changing while the item sits in the cart, and partial cancellations. → *Full design:* Ch 19 — Design Amazon — Online Shopping System

Q42. Design an online auction system (eBay; live IPL auction)

- Entities: `Auction` (item, reserve price, end time), `Bid`, `Bidder`, `Seller`, `AuctionHouse`.
- Patterns: State (scheduled, live, closed), Observer for outbid alerts, Strategy for auction type (English, sealed bid).
- What interviewers probe: two bids arriving at the same moment (serialise per auction), last-second sniping and extending the end time. → *Full design:* Ch 19 — Design an Online Auction System

Q43. Design Spotify / a music streaming service (LLD)

- Entities: `Track`, `Album`, `Artist`, `Playlist`, `User`, `Subscription`, `Player`, `PlayQueue`.
- Patterns: Strategy for recommendations and shuffle, State for the player, Iterator for queues.
- What interviewers probe: free vs premium limits, collaborative playlists, and the play-next queue vs the playlist order. → *Full design:* Ch 19 — Design a Music Streaming Service (Spotify)

Q44. Design a grocery store / POS / checkout (BigBasket)

- Entities: `Product` (priced by unit or by weight), `Inventory`, `Basket`, `Receipt`, `Discount`, `Payment`.
- Patterns: Strategy and Composite for discounts (buy-2-get-1, percent off a category), Decorator for tax.
- What interviewers probe: the order in which discounts apply, and returns. → *Full design:* Ch 19 — Design a Grocery Store (POS & Checkout)

Q45. Design a subscription-based sports score website

- Reuse the Cricinfo model for matches and add `Subscription` (plan, expiry), `Entitlement` checks and live score pushes.
- Patterns: Observer (subscribers to a match), Proxy for checking entitlements before content is served.
- What interviewers probe: free vs paid content per request, and fan-out of score updates to many subscribers. → *Related design:* Ch 18 — 18.2 Design Cricinfo

Q46. Design a logistics / shipment tracking system

- Entities: `Shipment`, `Parcel`, `Hub`, `Route`, `TrackingEvent`, `Courier`.
- Patterns: State for shipment status, Observer for customer updates, Strategy for route choice.
- What interviewers probe: events that arrive out of order (sort by event time, not arrival), and returns. → *Related design:* Ch 14 — 14.7 Design a Shipping Locker System

Q47. Design a newsletter service

- Entities: `Publication`, `Issue`, `Subscriber`, `Subscription` (topics, frequency), `Delivery`.

- Patterns: Observer / Pub-Sub (subscribers per topic), Strategy for channel (email, in-app), Template Method for rendering.
- What interviewers probe: unsubscribe taking effect at once, retries without double delivery, and daily digests. → *Related design:* Ch 20 — 20.6 Design a Pub-Sub System

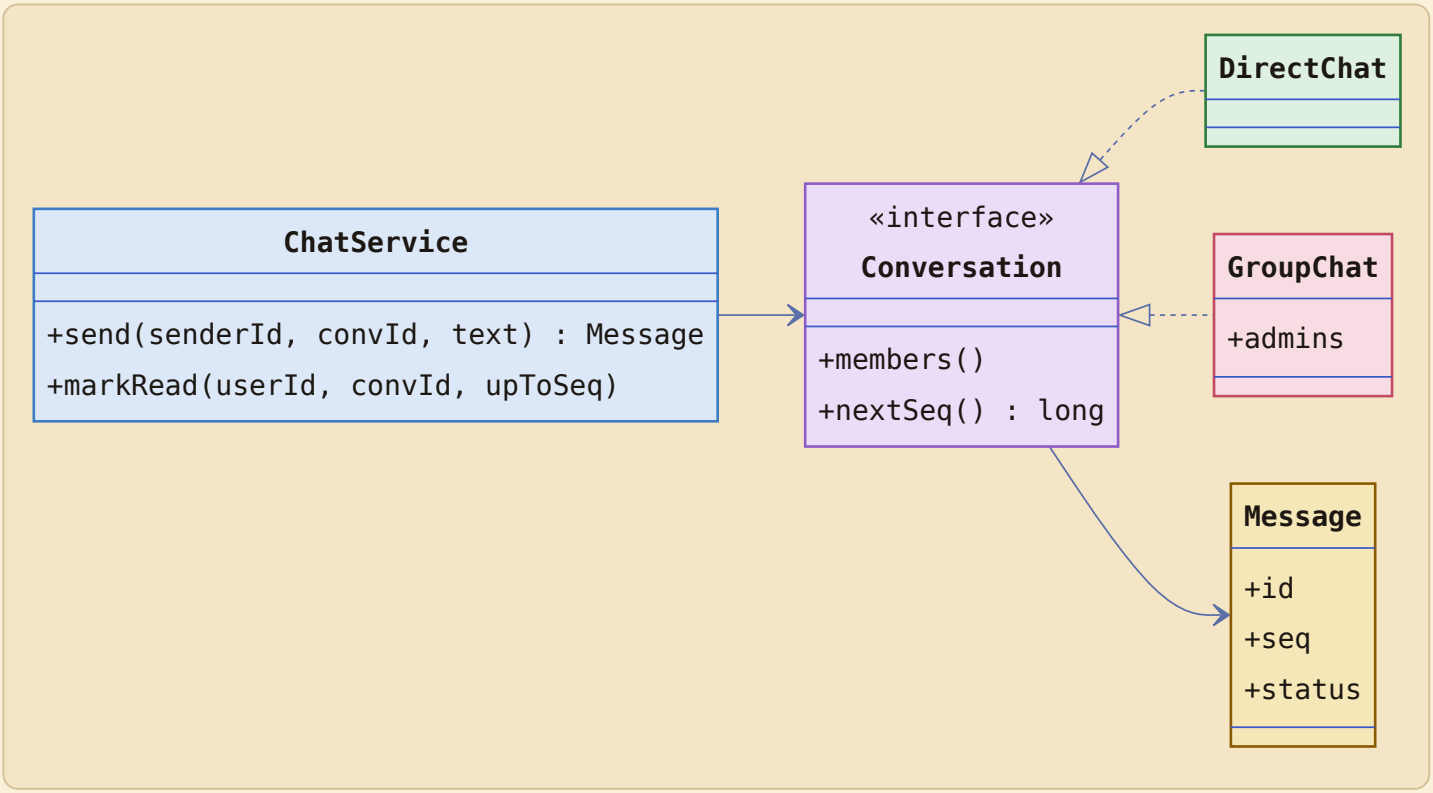
Q48. Design a maps navigator client (walk / car / bus / bike)

- Entities: `Navigator`, `Route`, `RouteStrategy` (one per travel mode), `Location`.
- Pattern: Strategy is the whole question. The navigator swaps strategies at runtime without `if` chains.
- What interviewers probe: adding a new travel mode without touching old code, and combining modes (walk + bus). → *Related design:* Ch 11 — 11.3 Strategy

Q49. Design a chat server / WhatsApp (LLD)

Requirements. Users send text messages one-to-one and in groups. Messages show sent, delivered and read ticks. Offline users receive their messages when they come back. Each conversation keeps its order. Groups have admins who add and remove members. Online and last-seen status is a nice extra.

Main classes.



- `Conversation` (sealed interface): `DirectChat` (two members) and `GroupChat` (members, admins).
- `Message`: a record (id, conversationId, senderId, seq, text, sentAt). Its delivery status is kept per recipient, because in a group each person reads at a different time.
- `Inbox`: per user, a queue of messages not yet delivered.
- `PresenceService`: online or offline, and last seen.
- `ConnectionRegistry`: the user's live sessions. Delivery listeners hang off it (Observer).

Patterns. Observer (deliver to connected sessions), Strategy (delivery channel: socket or push notification), and Mediator (`ChatService` coordinates instead of users talking to each other).

Concurrency and edge cases. Give each conversation an `AtomicLong` sequence. Sort messages by sequence, not by clock time, so senders on two phones cannot reorder history. Make delivery idempotent: a client acknowledges the highest sequence it has seen, and re-sent messages are ignored. A removed group member must not receive later messages, so check membership when the message is sent.

Extension question. "Add message editing and deletion." Store edits as new events that point at the original id, and let clients apply them, so the order stays clean.

Q50. Design Truecaller

Requirements. Users sign up with a phone number and may upload their contacts. Anyone can look up a number to see the most likely name and a spam score. Users report numbers as spam and block numbers. Users can

search by name. A user can mark their own number as private.

Main classes.

- `PhoneNumber`: a value object. Its constructor normalises the input to E.164 format (for example `+919876543210`), so the same number can never be stored two ways.
- `User`: an id, an own number, contacts and a block list.
- `Contact`: a record (number, savedName).
- `GlobalDirectory`: maps each number to a name-count table (name → how many users saved it under that name). A lookup returns the most frequent name, and a registered user's own name always wins.
- `SpamService`: counts reports per number and gives a score. A number is "likely spam" past a threshold, weighted by how trustworthy each reporter is.
- `CallerIdService`: answers `lookup(number)` with `CallerInfo(name, spamScore, blocked)`.
- `NameSearchIndex`: a trie (prefix tree) from name prefixes to numbers.

Patterns. Value object, Strategy (spam scoring, name ranking), Observer (a new contact upload updates the directory and the index), Facade (`CallerIdService`).

Concurrency and edge cases. Uploads and reports arrive at the same time from many users. Use `ConcurrentHashMap<PhoneNumber, ConcurrentHashMap<String, LongAdder>>` for name counts and `LongAdder` for spam counts, so updates never block reads. Count one report per reporter per number, so repeat reports don't inflate the score. A private user's name should not be served from the crowd data.

Extension question. "Mark verified business numbers." Add a `VerifiedBusiness` record checked before the crowd data, so a verified name always beats crowd guesses.

Q51. Design a mentorship / mock-interview platform (Pramp)

Requirements. Users sign up with the topics they practise (data structures, system design) and their available time slots. The platform pairs two users who share a topic and a free slot. Each person interviews the other for half the session. Afterwards both leave feedback. Users can cancel, and they are reminded before a session.

Main classes.

- `User`: id, topics, a calendar of `AvailabilitySlot`s, and a rating.
- `AvailabilitySlot`: a record (start, end).
- `MatchRequest`: user, topic, preferred slots.
- `MatchingService`: keeps the open requests per topic and slot, and pairs requests through a `MatchingStrategy` (first come first served, similar rating).
- `Session`: two participants, topic, slot, question set, and a state (`SCHEDULED`, `IN_PROGRESS`, `COMPLETED`, `CANCELLED`, `NO_SHOW`).
- `Feedback`: a record (from, to, scores, notes).
- `ReminderScheduler`: uses a `ScheduledExecutorService` to fire reminders.

Patterns. Strategy (matching), State (session), Observer (send notifications on state change), Builder for sessions.

Concurrency and edge cases. One user must never be matched twice for the same slot. Run the matching in one step: lock both users' calendars in a fixed order (lowest id first) to avoid deadlock, check both are still free, then book both. A cancellation within an hour counts against the user's reliability, and the partner goes back into the pool. A no-show is detected when a session never moves to `IN_PROGRESS`.

Extension question. "Add paid sessions with expert mentors." Add a `Mentor` role with a price. Payment must succeed before the session moves to `SCHEDULED`. The rest of the flow stays the same.

Q52. Design an Application Tracking System (Greenhouse)

Requirements. A company posts jobs. Each job has a pipeline of stages (Applied → Phone screen → Onsite → Offer → Hired). Candidates apply, and recruiters move applications forward or reject them. Interviewers submit scorecards. Hiring managers see a dashboard of each job's pipeline. Every action is audited.

Main classes.

- `Job`: id, title, department, status (`OPEN`, `ON_HOLD`, `CLOSED`), and its own `Pipeline` (an ordered list of `Stage`s).
- `Candidate`: a person, who may have several `Application`s.

- `Application`: candidate + job, the current stage, a status (`ACTIVE`, `REJECTED`, `WITHDRAWN`, `HIRED`), and a history of stage changes.
- `Interview`: application, interviewers and time. `Scorecard`: interviewer, ratings and a recommendation.
- `ApplicationService`: `advance`, `reject` and `hire`. It checks the rules (no skipping to offer without scorecards, unless overridden).
- `PermissionPolicy`: decides what a recruiter, an interviewer or a hiring manager may see or do.
- `AuditLog` and `NotificationService`.

Patterns. State (application status), Template Method or configurable stage lists (each job has its own pipeline), Chain of Responsibility (the checks before advancing), Observer (emails and audit entries on each change).

Concurrency and edge cases. Two recruiters may move the same application at the same time. Use optimistic locking: a version number on `Application`, where an update fails if the version changed. Duplicate applications (same candidate, same job) are merged or rejected. Closing a job should not delete applications, only stop new ones.

Extension question. "Auto-reject candidates who fail a knock-out question." Add a rule to the advance chain that runs on `Applied`.

Q53. Design a call center

Requirements. The center has three levels of employee: respondents, managers and directors. An incoming call goes to a free respondent. If the respondent can't handle it, or none is free, it escalates to a manager, then to a director. If nobody is free, the call waits in a queue. When an employee finishes a call, they take the next waiting call at their level.

Main classes.

- `Employee` (abstract or sealed): id, `Rank`, and a current call. Subtypes `Respondent`, `Manager`, `Director`. Each can `canHandle(call)`.
- `Call`: caller, required minimum rank (it rises when escalated), state (`WAITING`, `IN_PROGRESS`, `ESCALATED`, `ENDED`).
- `CallDispatcher`: holds a queue of free employees per rank (`BlockingQueue<Employee>`) and a queue of waiting calls per rank. `dispatch(call)` tries the call's rank and above.
- `EscalationHandler`: a chain from respondent to manager to director.

Patterns. Chain of Responsibility (escalation), Strategy (routing: least busy, skills-based), State (call lifecycle).

Concurrency and edge cases. Calls arrive and end on many threads. Use `poll()` on the free-employee queues, so taking an employee is atomic. When an employee becomes free, first check the waiting-call queue for their rank (and lower ranks), and only then return to the free pool. Do these two steps under one lock, or a call could be missed. If a call is dropped mid-escalation, end it cleanly and free the employee. A call that waits too long can be escalated or offered a callback.

Extension question. "Add priority customers." Change the waiting queue to a `PriorityBlockingQueue` ordered by priority, then by arrival time.

Q54. Design a Survey / Google Forms system

Requirements. An owner builds a form with a title and questions of several types: short text, paragraph, single choice, multiple choice, rating, and date. Questions can be required and can have validation (email format, number range). The form can show or skip a section based on an earlier answer. Respondents submit responses. The owner sees a summary per question and can close the form.

Main classes.

- `Form`: id, owner, settings (one response per user, deadline), an ordered list of `Section`s, a version, and a status.
- `Question` (sealed interface): `TextQuestion`, `ChoiceQuestion` (options, multiSelect), `RatingQuestion` (scale), `DateQuestion`. Each has `validate(Answer)`.
- `ValidationRule`: composable rules (required, regex, range).
- `BranchRule`: "if the answer to question 3 is 'No', go to section 5".
- `Response`: form id, form version, respondent, and `Map<questionId, Answer>`.
- `FormBuilder`: builds a form step by step and checks it is valid.
- `Summary`: per question type, a different aggregation (counts, average, word list).

Patterns. Builder (forms), Composite (form → sections → questions), Strategy/Specification (validation), Visitor or a pattern-matching `switch` over the sealed `Question` types for summaries.

Edge cases. Editing a form after responses exist: create a new version, so old responses still point at the questions they answered. Enforce one response per user with a set of respondent ids, updated atomically with `ConcurrentHashMap.newKeySet().add(...)`. Validate every answer on the server, never trusting the client.

Extension question. "Add a quiz mode with scores." Add a correct answer and points to each question, plus a `Grader` that walks the response. Forms that are not quizzes are unaffected.

Q55. Design an employee management / payroll system

Requirements. Store employees with department, role and manager. Show the org chart and everyone who reports (directly or indirectly) to a manager. Track leave balances and requests. Run monthly payroll for salaried, hourly and commission employees, with deductions (tax, provident fund) and payslips.

Main classes.

- `Employee`: id, name, department, role, managerId, join date, and a `PayPolicy`.
- `OrgChart`: manager id → list of direct reports. `allReports(id)` walks the tree breadth-first.
- `PayPolicy` (interface): `SalariedPolicy` (monthly amount, prorated for part months), `HourlyPolicy` (rate × timesheet hours, overtime), `CommissionPolicy` (base + percent of sales).
- `Deduction` (interface): `IncomeTax`, `ProvidentFund`, `LeaveWithoutPay`, applied in order.
- `PayrollRun`: a month and the list of `Payslip`s it produced. Its status is `DRAFT`, `APPROVED` or `PAID`.
- `LeaveService`: balances per leave type, and requests that a manager approves.

Patterns. Strategy (pay policies), Chain of Responsibility or a list of deductions, Composite (org tree), State (payroll run and leave request).

Edge cases. Hold money as `long` paise or `BigDecimal`, and round only once, at the end. Mid-month joiners and leavers get prorated pay. A manager cycle (A reports to B, B to A) must be rejected when a manager is assigned. Once a payroll run is `PAID`, it is never edited. Corrections go into next month as adjustments.

Extension question. "Change an employee's salary from the middle of the month." Store pay policies with an effective date, and prorate between the two policies.

Components and infrastructure

Q56. ⋆ Design a Rate Limiter (LLD)

Asked at: Atlassian (P4 LLD round)

- Entities: `RateLimiter` interface, `TokenBucket`, `SlidingWindowLog`, `FixedWindowCounter`, a `RateLimiterRegistry` (one limiter per client key), and a config.
- Patterns: Strategy for the algorithm, Factory to build from config.
- What interviewers probe: thread safety (per-key lock or CAS), lazy refill based on elapsed time instead of a timer thread, and the burst behaviour of each algorithm. → *Full design:* Ch 20 — 20.7 Design a Rate Limiter; also Ch 13 — 13.11 Rate limiter — token bucket

Q57. ⋆ Design an LRU Cache

- Data structure: a `HashMap<key, Node>` plus a doubly linked list. Every `get` or `put` moves the node to the front, and eviction removes the tail. Both operations are O(1).
- In Java you can also show `LinkedHashMap` with `accessOrder = true` and an overridden `removeEldestEntry`.
- What interviewers probe: writing the linked list by hand with dummy head and tail nodes, and making it thread-safe. → *Full design:* Ch 20 — 20.3 Design an In-Memory Cache (and an LRU Cache from scratch)

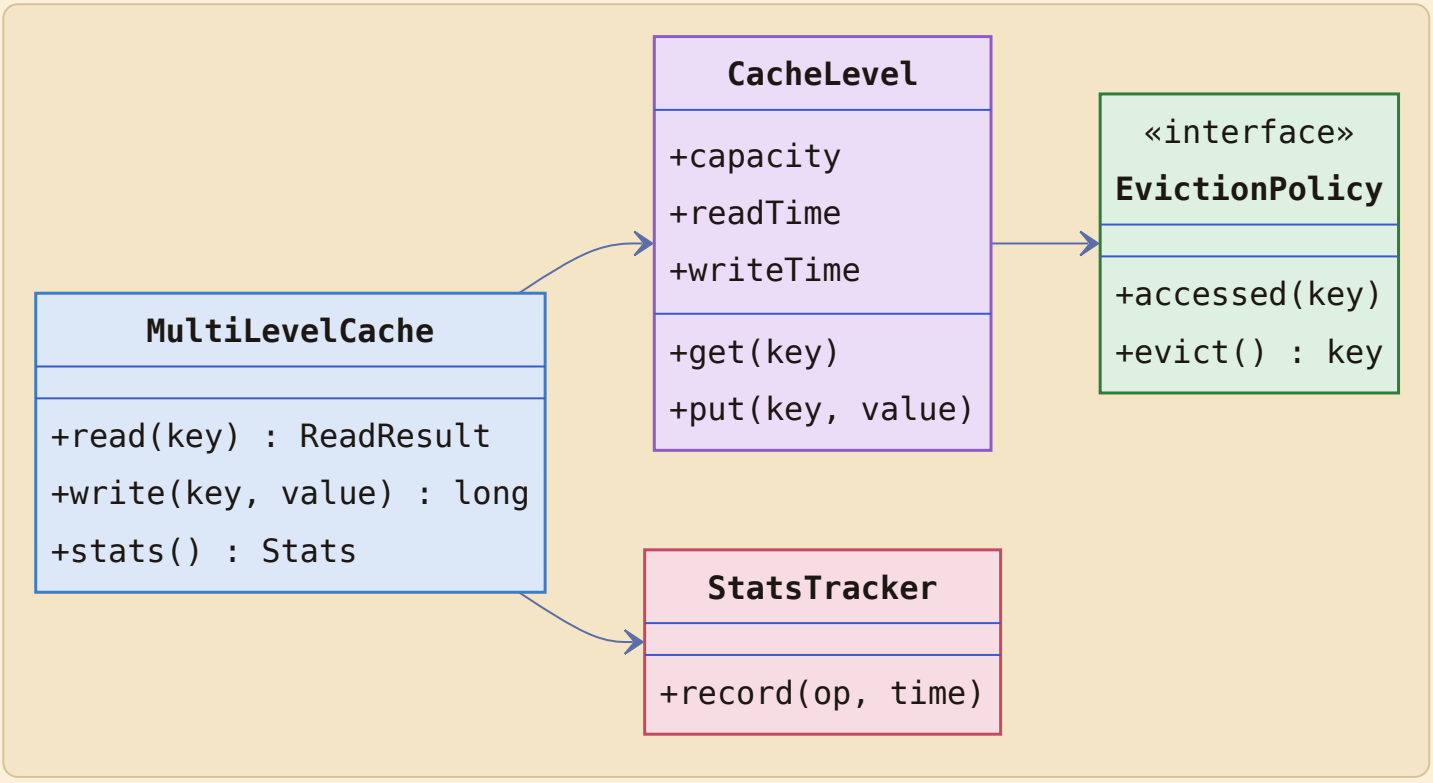
Q58. ⋆ Design a cache with pluggable eviction policies (LRU / LFU)

- Entities: `Cache<K, V>`, `Storage`, `EvictionPolicy` (`keyAccessed`, `evictKey`) with LRU, LFU and FIFO implementations.
- Pattern: Strategy for eviction. LFU in O(1) uses a frequency → linked set map plus a `minFreq` pointer (LeetCode 460).
- What interviewers probe: tie-breaking in LFU, and keeping the policy and the storage in step. → *Full design:* Ch 20 — 20.3 Design an In-Memory Cache

Q59. ⋆ Design a multi-level cache

Requirements. A cache has N levels (L1, L2, … Ln). Each level has a capacity, a read time and a write time. Reading a key checks L1 first, then each lower level, until the key is found. When found at level i, write the value into every level above i. Writing a key writes it to every level, skipping a level that already holds the same value. Report the time taken by every read and write, plus the average read and write time over the last N operations. Show usage per level.

Main classes.



- `CacheLevel`: one level, with its own storage and `EvictionPolicy` (LRU by default).
- `MultiLevelCache`: holds the ordered list of levels and runs the read and write paths. Time is the sum of each level's read time visited, plus the write times added.
- `StatsTracker`: a ring buffer of the last N timings for reads and for writes, with running sums so averages are O(1).

Patterns. Chain of Responsibility (a level passes a miss to the next one), Strategy (eviction), Builder (configure levels).

Concurrency and edge cases. In a machine-coding round, guard the whole cache with one lock (`synchronized` or a `ReentrantLock`). A read-write lock does not help here, because a read is not read-only: it updates each level's LRU order and copies the value into the levels above, so every read would need the write lock anyway. For more throughput, lock levels one at a time, always top-down, so no deadlock can form. An eviction at L1 does not remove the key from L2. Reject capacity 0 or an empty list of levels.

Extension question. "Make it write-back instead of write-through." Writes go to L1 only and are marked dirty. They reach lower levels when evicted, and a flush is forced on shutdown.

Q60. ⋆ Design an in-memory key-value store (with transactions / TTL / versioning)

- Base: `ConcurrentHashMap<String, Entry>` where `Entry` holds value and expiry; expired entries are removed lazily on read plus by a periodic sweeper.
- Transactions (`begin`, `set`, `get`, `commit`, `rollback`, nestable): keep a stack of change maps. Reads check the top map, then the maps below it, then the base store. `commit` merges the top map down, and `rollback` pops it.
- Versioning: store a list of `(version, value)` per key and answer `get(key, version)` by binary search.
- What interviewers probe: isolation between clients (one transaction stack per session), and deletes inside a transaction (store a tombstone marker). → *Related design:* Ch 20 — 20.3 Design an In-Memory Cache; Ch 13 — 13.6 Thread-safe cache with TTL

Q61. ⋆ Design a Logging Framework / Logger library

- Entities: `Logger`, `LogLevel`, `LogMessage`, `Appender` (console, file, database), `Formatter`, `LoggerConfig`.

- Patterns: Singleton or factory for loggers, Strategy for appenders and formatters, Chain of Responsibility for level filtering.
- What interviewers probe: async logging through a queue without losing messages on shutdown, and per-package log levels. → *Full design:* Ch 20 — 20.5 Design a Logging Framework

Q62. ⋆ Design a Pub-Sub system / message queue (LLD)

- Entities: `Broker`, `Topic`, `Message`, `Publisher`, `Subscriber`, `Subscription` (with offset).
- Patterns: Observer, and a per-subscriber offset so slow consumers don't block fast ones.
- What interviewers probe: at-least-once delivery with acks and retries, ordering within a topic, and replay from an offset. → *Full design:* Ch 20 — 20.6 Design a Pub-Sub System

Q63. ⋆ Design a Notification System (LLD)

Asked at: Flipkart

- Entities: `Notification`, `Channel` (email, SMS, push), `Template`, `UserPreference`, `NotificationService`, `RetryPolicy`.
- Patterns: Strategy per channel, Template Method or Builder for messages, Observer for events, Decorator for retries.
- What interviewers probe: user opt-outs, retries with backoff, falling back to another channel, and not sending duplicates. → *Full design:* Ch 20 — 20.2 Design a Notification Service

Q64. Design a HashMap

- Array of buckets; each bucket is a linked list (Java turns long chains into balanced trees); index = `hash(key) & (capacity - 1)`; resize and rehash when size exceeds capacity × load factor (0.75).
- For the concurrent version: lock striping (a lock per group of buckets) or CAS on empty buckets.
- What interviewers probe: `equals`/`hashCode` contract, why capacity is a power of two, and resizing under concurrency. → *Full design:* Ch 13 — 13.7 Design a (simplified) Concurrent HashMap

Q65. ⋆ Design a thread-safe blocking queue / producer-consumer

- A bounded buffer: a lock with two conditions, `notFull` and `notEmpty`. `put` waits while the queue is full, and `take` waits while it is empty. Always wait inside a `while` loop, never an `if`.
- Alternative: two `Semaphore`s (free slots, filled slots) plus a mutex.
- What interviewers probe: spurious wakeups, `signal` vs `signalAll`, and shutdown with a poison pill. → *Full design:* Ch 13 — 13.8 Thread-safe bounded Blocking Queue; Ch 12 — 12.22 Producer-consumer pattern

Q66. Design a thread-safe cache with TTL

- `ConcurrentHashMap` of entries with expiry time; lazy expiry on `get`; a `ScheduledExecutorService` or a `DelayQueue` for active cleanup.
- What interviewers probe: get-or-compute without two threads loading the same key (`computeIfAbsent`), and clock choice (`System.nanoTime` for durations). → *Full design:* Ch 13 — 13.6 Thread-safe cache with TTL

Q67. Design a concurrent Bloom filter

- A bit array with k hash functions. "Might contain" can give false positives but never false negatives.
- Concurrent version: an `AtomicLongArray` with CAS on each word, so adds are lock-free.
- What interviewers probe: choosing the size and k for a target false-positive rate, and why you cannot delete (use a counting Bloom filter for that). → *Full design:* Ch 13 — 13.9 Concurrent Bloom Filter

Q68. Design a thread pool / executor

- Entities: a work queue (`BlockingQueue<Runnable>`), worker threads looping on `take()`, and `submit` returning a `Future`. Shutdown has a graceful mode and an immediate mode.
- What interviewers probe: what happens when the queue is full (rejection policy), worker death on exceptions, and when virtual threads make a pool unnecessary. → *Full design:* Ch 12 — 12.21 Thread pool pattern

Q69. Dining philosophers / print FooBar alternately / building H2O (LeetCode concurrency)

- Short puzzles about making threads take turns or meet at a point. The tools are semaphores, `CyclicBarrier`, lock ordering, and conditions.
- Dining philosophers: break the deadlock by always taking the lower-numbered fork first, or by letting at most four philosophers sit at once.

- What interviewers probe: proving there is no deadlock and no starvation. → *Full design:* Ch 13 — 13.1–13.5

Q70. Design a connection pool / object pool

- Entities: `Pool` with idle and in-use sets, `acquire(timeout)` and `release`, max size, validation on borrow, and idle eviction.
- Pattern: Object Pool, with a `Semaphore` capping how many objects can be borrowed at once.
- What interviewers probe: leaked connections (borrow timeouts, try-with-resources wrapper), and broken connections returned to the pool. → *Full design:* Ch 8 — 8.14 Bonus — Object Pool

Q71. Design a payment gateway (Razorpay) — LLD

- Entities: `PaymentRequest`, `Payment` (state: created, authorised, captured, failed, refunded), `PaymentMethod` (card, UPI, net banking), `ProcessorAdapter` per bank, `Refund`, `WebhookNotifier`.
- Patterns: State (payment lifecycle), Strategy/Adapter (methods and processors), Observer (webhooks to merchants).
- What interviewers probe: an idempotency key so retries never double-charge, timeouts with an unknown result (check status before retrying), and refunds that are partial. → *Related design:* Ch 21 — 21.2 Design a Digital Wallet Service

Q72. Design a text editor with undo / redo (Notion / Google Docs LLD)

Asked at: Notion

- Entities: `Document` (a buffer, such as a list of lines or a gap buffer), `Cursor`, `EditCommand` (insert, delete), and `History` (an undo stack and a redo stack).
- Patterns: Command (each edit knows how to undo itself) or Memento (save snapshots). Any new edit clears the redo stack.
- What interviewers probe: Command vs Memento memory cost, grouping keystrokes into one undo step, and collaborative editing (operational transforms) as a stretch. → *Full design:* Ch 10 — 10.2 Command and 10.5 Memento

Q73. Design a business rule engine

- Entities: `Rule` (condition + action, priority), `Condition` (composable with and / or / not), `Fact` (input data), `RuleEngine` (evaluate in priority order, first-match or all-match).
- Patterns: Specification (composable conditions), Chain of Responsibility (rule order), Interpreter if rules come from a text language.
- What interviewers probe: loading rules from config without redeploying, and conflicts when two rules match. → *Related design:* Ch 11 — 11.7 Beyond GoF (Specification); Ch 10 — 10.1 Chain of Responsibility

Q74. Design a circuit breaker (LLD)

- States: `Closed` (count failures), `Open` (fail fast until a cool-down ends), `HalfOpen` (let a few trial calls through; success closes, failure re-opens).
- Patterns: State, plus Proxy/Decorator wrapping the protected call.
- What interviewers probe: a sliding window of failure rate vs a plain count, and thread-safe state changes (`AtomicReference` with CAS). → *Related design:* Ch 20 — 20.2 Design a Notification Service (provider circuit breaker)

Q75. Design a Hit Counter (LeetCode 362)

Requirements. `hit(timestamp)` records a hit at a time given in seconds. `getHits(timestamp)` returns how many hits happened in the past 300 seconds, including the current second. Timestamps arrive in non-decreasing order. Follow-ups: what if there are millions of hits per second, and what if it must be thread-safe?

Main classes.

- `HitCounter` (interface) with `hit` and `getHits`.
- `QueueHitCounter`: a deque of timestamps. `getHits` drops entries older than `t - 299` and returns the size. It is simple, but memory grows with the number of hits.
- `BucketHitCounter`: two arrays of size 300, `times[]` and `counts[]`. The index is `t % 300`. If `times[i] != t`, the bucket is stale, so reset it to `(t, 1)`; otherwise increment. `getHits` sums the buckets whose `times[i] > t - 300`. Memory is fixed, and `getHits` costs O(300).
- `HitCounterRegistry`: one counter per key (per URL or per user), in a `ConcurrentHashMap`.

Patterns. Strategy (swap implementations), with the sliding window as the core idea. This is the same window as a rate limiter, and interviewers often link the two.

Concurrency and edge cases. For thread safety, guard each bucket's reset-or-increment as one step. Lock per bucket (stripes), or store `(time, count)` packed into one `AtomicLong` updated with CAS. For very high hit rates, use a `LongAdder` per bucket. Hits from the future should not happen, but clamp them anyway. If time jumps forward by more than 300 seconds, the stale check already clears old buckets.

Extension question. "Make the window configurable and also report per minute and per hour." Keep several bucket arrays at different resolutions (seconds, minutes), like a time-series database's roll-ups.

Q76. Design a URL shortener (LLD / class design)

Requirements. `shorten(longUrl)` returns a short code such as `aZ3k9Q`. `resolve(code)` returns the long URL. Users may request a custom alias. Links can expire. The service counts clicks. The same long URL may get the same code or a new one; ask which.

Main classes.

- `ShortUrl`: a record (code, longUrl, ownerId, createdAt, expiresAt).
- `CodeGenerator` (interface): `CounterBase62Generator` (an `AtomicLong` counter encoded in base 62, short with no collisions, but guessable), `HashGenerator` (the first 7 characters of a hash of the URL, retrying with a salt on collision), and `RandomGenerator`.
- `UrlRepository`: `saveIfAbsent(ShortUrl)`, `find(code)`. The in-memory version is a `ConcurrentHashMap`.
- `UrlShortenerService`: validates the URL, applies the alias or generates a code, and handles expiry.
- `ClickTracker`: a `LongAdder` per code, with events sent on asynchronously for analytics.

Patterns. Strategy (code generation), Repository, Facade (the service), Observer (click events).

Concurrency and edge cases. Two users may ask for the same custom alias. `putIfAbsent` decides the winner atomically, and the loser gets "alias taken". The counter generator never collides, while hash and random generators must retry. Reject non-HTTP(S) URLs and links to the shortener itself (loops). Expired codes return "not found". Remove them lazily, plus a sweeper. Keep codes case-sensitive and avoid look-alike characters if humans type them.

Extension question. "Hide the order of codes, so people cannot enumerate links." Scramble the counter with a reversible mixing function before encoding. It stays collision-free but no longer looks sequential.

Q77. Design a JSON / CSV parser

Requirements. Parse JSON text into an in-memory tree: objects, arrays, strings (with escapes and `\uXXXX`), numbers (sign, fraction, exponent), `true`, `false` and `null`. Give clear errors with line and column. Serialise the tree back to text. For CSV: split rows into fields, where quoted fields may contain commas, newlines and doubled quotes (`""`).

Main classes.

- `JsonValue` (sealed interface) with records `JsonObject(Map)`, `JsonArray(List)`, `JsonString`, `JsonNumber(BigDecimal)`, `JsonBool`, `JsonNull`.
- `Lexer`: turns characters into `Token`s (`LBRACE`, `STRING`, `NUMBER`, …) and tracks position.
- `Parser`: recursive descent. `parseValue` looks at the next token and calls `parseObject`, `parseArray` and so on. One method per grammar rule.
- `JsonWriter`: walks the tree with a pattern-matching `switch` over the sealed types. The compiler (Java 21 and later) checks that every case is covered.
- `ParseException`: message, line, column.
- `CsvReader`: a small state machine with the states `FIELD_START`, `IN_UNQUOTED`, `IN_QUOTED`, `QUOTE_IN_QUOTED`.

Patterns. Interpreter/Composite (the value tree), Visitor-style traversal (here a sealed `switch`), State (the CSV reader), Builder for writing.

Edge cases. Duplicate keys (last wins, or an error). Very deep nesting, so add a depth limit against stack overflow. Leading zeros (`01` is invalid in JSON). Trailing commas (invalid). Big numbers (use `BigDecimal`). Text left over after the root value (an error). For huge inputs, offer a streaming, event-based API (`startObject`, `key`, `value`) instead of building the whole tree.

Extension question. "Map JSON onto Java records." Use reflection on record components (`getRecordComponents()`) to match keys to constructor parameters.

Q78. Design a configuration management system (LLD)

Requirements. Services read typed config values (`int`, `boolean`, `duration`) by key. Values can be set at several layers: defaults, then environment (dev, staging, prod), then service, then instance. The most specific layer wins. Every change creates a new version and records who changed what. Users can roll back to an earlier version. Running services are told about changes without restarting.

Main classes.

- `ConfigKey`: a record (namespace, name), plus a `ConfigSchema` that holds the type, default, and validation per key.
- `ConfigSnapshot`: an immutable map for one version: version number, author, time, values.
- `ConfigStore`: keeps the list of snapshots per scope, and an `AtomicReference<ConfigSnapshot>` for the current one. `set` builds a new snapshot (copy-on-write) and swaps it in with CAS.
- `ConfigResolver`: looks a key up through the layers in order (instance → service → environment → default).
- `ConfigWatcher` (Observer): a client registers a callback on keys or prefixes.
- `AuditLog`: who, what, when, old value and new value.

Patterns. Chain of Responsibility (layered lookup), Observer (change listeners), Memento/immutable snapshots (versions and rollback), Strategy for value parsers.

Concurrency and edge cases. Readers never lock: they read the current snapshot reference. Writers race through CAS and retry on conflict, or use compare-by-version so a stale edit is rejected. Validate against the schema before publishing, so a bad value never goes live. A listener that throws must not stop the other listeners.

Extension question. "Add feature flags rolled out to 10% of users." Add a `FlagRule` with a percentage. Hash the user id into 0–99 and compare it, so each user gets a stable result.

Q79. Design AWS Lambda (serverless) — LLD

Requirements. Users register a function: name, runtime (Java, Python), memory, timeout, code location. `invoke(name, payload)` runs it synchronously and returns the result, or runs it asynchronously with retries. Warm environments are reused, and new ones are started (cold start) when none is free. A per-function limit caps how many run at once. Idle environments shut down after a while.

Main classes.

- `FunctionConfig`: a record (name, runtime, memoryMb, timeout, maxConcurrency, codeRef).
- `FunctionRegistry`: create, update and delete functions, with versions.
- `ExecutionEnvironment`: one sandbox. Its state goes `INITIALISING` → `READY` → `BUSY` → `READY` …, and finally `TERMINATED`.
- `RuntimeFactory`: builds an environment for each runtime type.
- `EnvironmentPool`: per function, a deque of warm environments (newest first, so old ones age out).
- `Invoker`: takes a concurrency permit, borrows or creates an environment, runs with a timeout, and returns the environment to the pool.
- `AsyncQueue` + `RetryPolicy` + `DeadLetterQueue` for asynchronous calls.
- `IdleReaper`: a scheduled task that terminates environments idle longer than a limit.

Patterns. Object Pool (warm environments), Factory (runtimes), State (environment lifecycle), Proxy (the `Invoker` hides pooling and limits), Strategy (retry backoff).

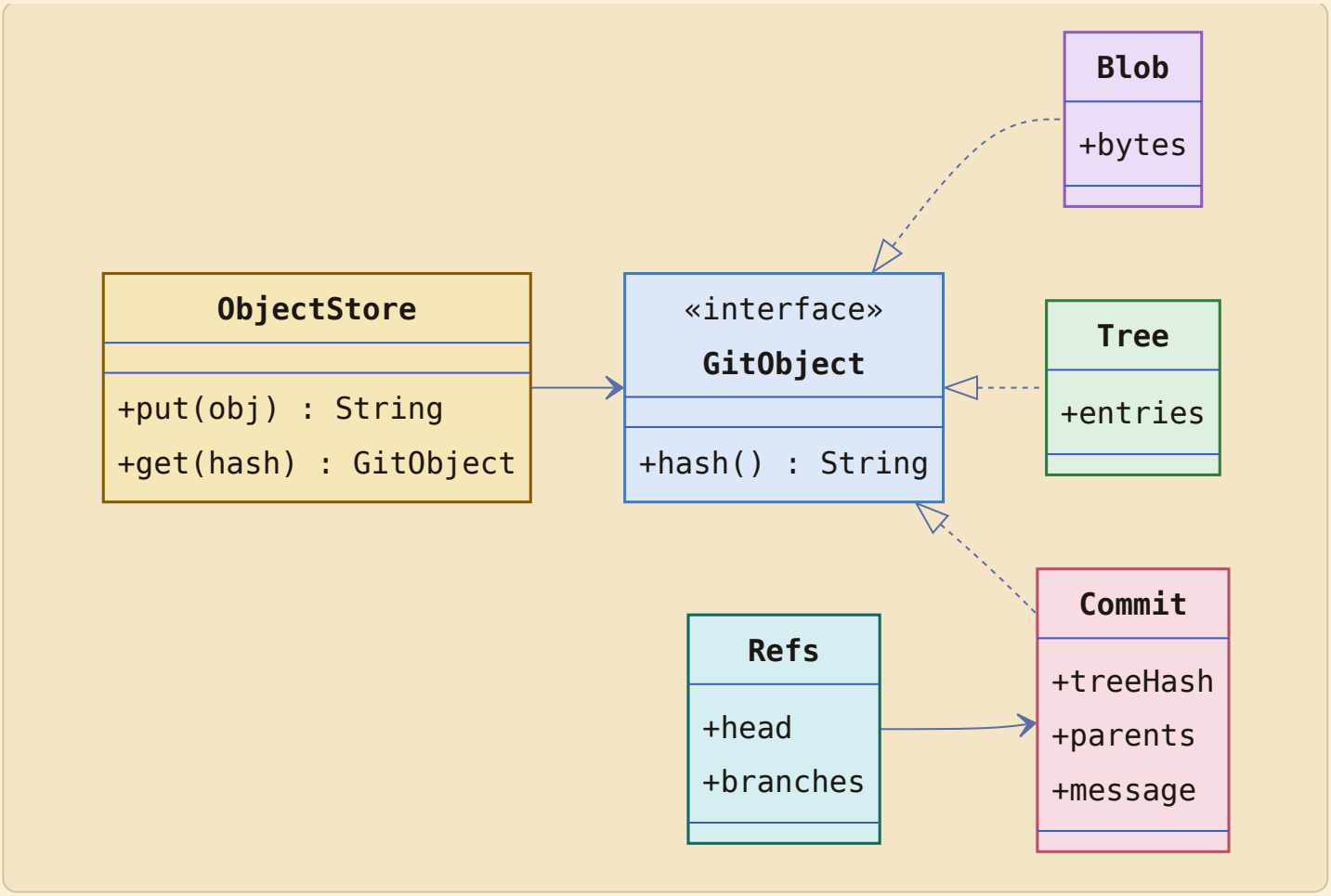
Concurrency and edge cases. A per-function `Semaphore` enforces `maxConcurrency`. Throttle by failing fast, or queue async calls. Run the handler on a virtual thread and wait with `Future.get(timeout)`. On timeout, kill and discard that environment, never reuse it. A crashed environment is removed, not returned. An update to the code marks old warm environments as stale.

Extension question. "Add provisioned concurrency." Pre-create N environments at deploy time, and keep the reaper from removing them.

Q80. Design a version-control / Git-like system

Requirements. Support `init`, `add` (stage files), `commit`, `log`, `branch`, `checkout`, `diff`, and `merge` for a single local repository. Identical content is stored once. History is never changed by normal commands.

Main classes.



- `Blob`: file content. `Tree`: a directory with name → (blob or tree) hash entries. `Commit`: tree hash, parent hashes, author, time, message. All of them are immutable records.
- `ObjectStore`: content-addressed storage, a map from hash (SHA-256 of the content) to object. The same content always gets the same hash, so duplicates cost nothing.
- `Index` (staging area): path → blob hash for the next commit.
- `Refs`: branch name → commit hash, and `HEAD` → a branch name (or a commit, the "detached" state).
- `Repository`: a facade exposing the commands, with each command as a `Command` object for the CLI.

Patterns. Composite (trees of trees), Command (CLI operations), immutable value objects, Facade.

Algorithms and edge cases. `log` follows parent links. `merge` first finds the merge base, the lowest common ancestor of the two commits (BFS from both sides). If one side is the base, just move the branch pointer (a fast-forward). Otherwise do a three-way merge per file and mark conflicts. `checkout` with uncommitted changes must refuse, or warn. Moving a branch is one atomic reference update.

Extension question. "Add `revert`." Create a new commit that undoes the changes of an earlier one. Never rewrite history.

Machine-coding round favourites

Q81. ⋆ Design Splitwise (expense sharing)

Asked at: Swiggy (SDE1; SDE2, twice)

- Entities: `User`, `Group`, `Expense` with a `Split` type (equal, exact, percent), `BalanceSheet` (who owes whom), `SettlementService`.
- Patterns: Strategy/Factory for split types, plus validation that the splits add up to the total.
- What interviewers probe: rounding (give the leftover paise to one person), simplifying debts (greedy matching of the biggest creditor with the biggest debtor), and the command-line input format. → *Full design:* Ch 20 — 20.1 Design Splitwise

Q82. ⋆ Design a task scheduler / distributed task scheduler (LLD)

Asked at: Flipkart (SDE2, Jan 2026)

- Entities: `Task` (id, payload, run-at time, recurrence, retries), `Scheduler` (`schedule`, `cancel`), a `DelayQueue` or `PriorityQueue` ordered by run time, a `Worker` pool, and an `ExecutionRecord`.
- Design: one dispatcher thread waits for the next due task (a `Condition` with a timed wait, woken early when a sooner task arrives) and hands it to a worker pool. A recurring task schedules its next run after finishing.
- What interviewers probe: cancelling a queued task, retries with backoff, and the distributed version (lease a task in storage with a timeout, so a crashed worker's task is picked up again). → *Related design:* Ch 12 — 12.21 Thread pool pattern

Q83. ⋆ Design an in-memory file system (mkdir / ls / find)

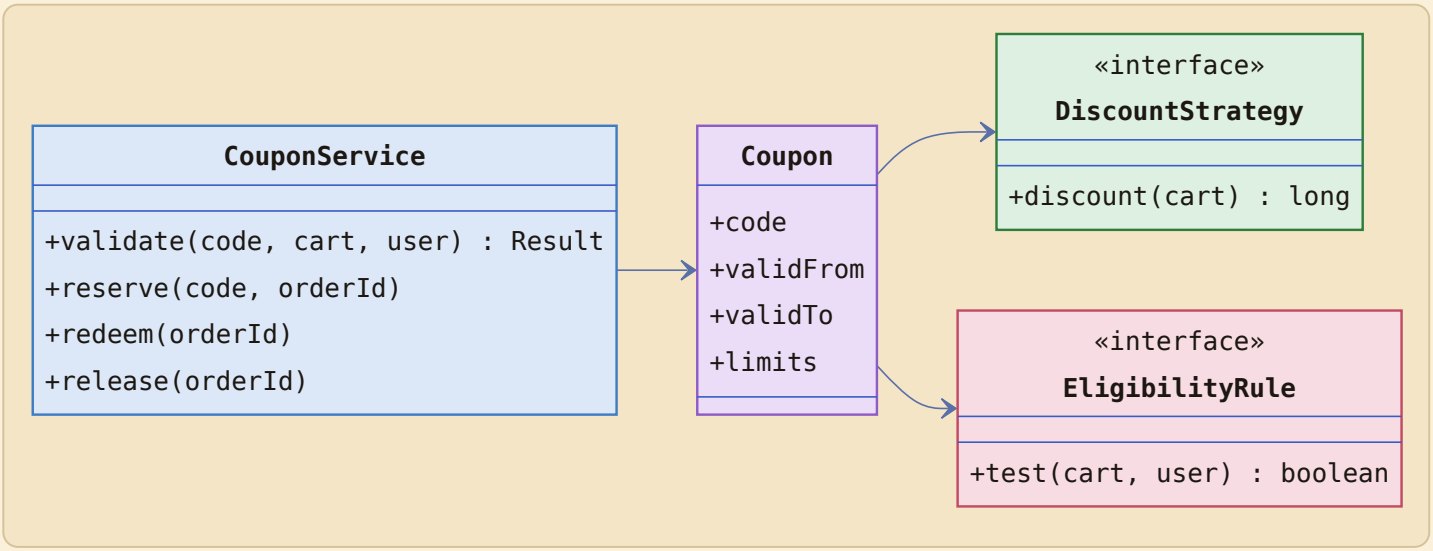
Asked at: ClearTrip and others (Unix `find`)

- Entities: `FileSystemNode` (sealed: `File`, `Directory`), `Path` parsing, and `FileSystem` (`mkdir -p`, `ls`, `addContent`, `readContent`).
- Patterns: Composite for the tree, Specification/Strategy for `find` filters (name, extension, size) combined with and / or.
- What interviewers probe: adding a new filter without changing existing ones, and walking a deep tree without recursion limits. → *Full design:* Ch 21 — 21.8 Design a Unix File Search (`find`); Ch 9 — 9.3 Composite Pattern

Q84. ⋆ Design a coupon / promo-code system

Requirements. Admins create coupons: a code, a discount type (flat ₹100 off, 20% off up to ₹150, buy-one-get-one), a validity window, a total usage limit, and a per-user limit. Eligibility rules include a minimum cart value, certain categories, first order only, and certain user segments. At checkout the user applies a code. The system validates it, computes the discount, and redeems it when payment succeeds. Coupons can or cannot be combined (stacking).

Main classes.



- `DiscountStrategy`: `FlatDiscount`, `PercentDiscount(cap)`, `BuyXGetY`. The result never exceeds the eligible amount.
- `EligibilityRule`: small rules joined with `and` / `or` (Specification). Each failure gives a reason the user can read.
- `Redemption`: a record (code, userId, orderId, status: `RESERVED`, `REDEEMED`, `RELEASED`).
- `UsageCounter`: total and per-user counts.

Patterns. Strategy (discounts), Specification / Chain of Responsibility (eligibility), Factory (build a coupon from an admin request).

Concurrency and edge cases. A "first 1,000 users" coupon must not be used 1,001 times. At checkout, *reserve* a use: increment the counter atomically only while it is below the limit (a CAS loop on an `AtomicInteger`). Move it to `REDEEMED` when payment succeeds, and *release* it on failure or timeout. Make redeeming idempotent per order id. Codes are case-insensitive. Check validity windows with a time zone in mind.

Extension question. "Auto-apply the best coupon." Evaluate every eligible coupon's discount against the cart and pick the biggest, and allow stacking only within the stacking rules.

Q85. ⋆ Design an Event Calendar (Flipkart; Google Calendar)

Asked at: Flipkart

- Entities: `User`, `Event` (time range, organiser, participants), `Team`, `Invite` (accept / reject), `Calendar` view.
- Patterns: Observer for invites, and an interval-overlap check for conflicts.
- What interviewers probe: finding a common free slot for a team, recurring events, and the per-user agenda for a day. → *Full design:* Ch 21 — 21.3 Design an Event Calendar (Flipkart)

Q86. ⋆ Design a LeetCode-like coding platform ("Coding Blox", Flipkart)

Asked at: Flipkart

- Entities: `User` (score), `Contest`, `Question` (difficulty level, score), `Submission`, `Leaderboard`.
- Patterns: Strategy for scoring and question selection, Observer for leaderboard updates.
- What interviewers probe: contest states (created, running, ended), only the creator can start a contest, and ordering the leaderboard. → *Full design:* Ch 21 — 21.4 Design a LeetCode-like Coding Platform

Q87. ⋆ Design a property listing site ("PropertyHunt", ClearTrip)

Asked at: ClearTrip

- Entities: `User`, `Property` (location, price, size, type, status), `Shortlist`, `SearchQuery`.
- Patterns: Specification/Builder for search filters, Strategy for sorting.
- What interviewers probe: combining many optional filters cleanly, and marking a property sold so it leaves search results. → *Full design:* Ch 21 — 21.5 Design a Property Listing Site

Q88. ⋆ Design a loan ledger ("Ledger Co", Navi)

Asked at: Navi

- Entities: `Loan` (principal, rate, years, EMI), `Payment` (lump sum after EMI n), `Bank`, `LedgerService`, and commands `LOAN`, `PAYMENT` and `BALANCE`.
- Patterns: Command for each input line, and pure functions for the EMI and balance maths.
- What interviewers probe: rounding EMIs up, how a lump sum shortens the remaining EMIs, and parsing the input file robustly. → *Full design:* Ch 21 — 21.6 Design a Loan Ledger

Q89. ⋆ Design a Task Management System / Jira / task planner

- Entities: `Task` (title, assignee, status, due date, priority), `Story`, `Bug`, `Sprint`, `User`, `TaskService`.
- Patterns: State for status transitions, Factory for task types, Observer for assignment notifications.
- What interviewers probe: valid transitions per task type, subtasks, and filtering or sorting by many fields. → *Full design:* Ch 21 — 21.1 Design a Task Management System

Q90. ⋆ Design a To-Do list app

Asked at: PhonePe

- A smaller task manager: `TodoItem` (text, done, due date, tags), `TodoList`, add / edit / complete / delete, filters, and an activity log.
- Patterns: Command for actions (gives undo and the activity log for free), Strategy for sorting.
- What interviewers probe: soft delete, undo, and recurring tasks. → *Related design:* Ch 21 — 21.1 Design a Task Management System

Q91. ⋆ Design a Digital Wallet (Paytm / PhonePe)

- Entities: `User`, `Wallet` (balance), `Transaction` (credit, debit, transfer), `Offer`, `Statement`.
- Patterns: Strategy for offers and cashback, Command or a ledger of immutable transactions.
- What interviewers probe: transfers between two wallets without deadlock (lock in id order), money as whole paise, and idempotent transfers. → *Full design:* Ch 21 — 21.2 Design a Digital Wallet Service

Q92. ⋆ Design an Inventory / Warehouse Management System

- Entities: `Product` / `SKU`, `Warehouse`, `StockLevel`, `StockMovement` (in, out, transfer), `Reservation`, `ReorderRule`.

- Patterns: Observer for low-stock alerts, Strategy for picking a warehouse, and reserve-then-commit for orders.
- What interviewers probe: preventing overselling under concurrent orders, and keeping a movement log so stock can be audited. → *Full design:* Ch 21 — 21.7 Design an Inventory Management System

Part B — Concept questions

These are the "explain X" and "X vs Y" questions that come up in a low-level design (LLD) round, usually in the first 10 minutes or while you are coding. Answer in two to four sentences, then give a tiny example from your design.

OOP basics

Q93. ⋆ What are the four pillars of OOP? Explain each with an example. Encapsulation keeps an object's data private and changes it only through methods that check the rules (a `BankAccount` whose `withdraw` refuses overdrafts). **Abstraction** shows *what* something does and hides *how* (a `PaymentGateway` interface with `pay()`). **Inheritance** lets a subclass reuse and specialise a parent (`SavingsAccount extends Account`). **Polymorphism** means one call runs different code depending on the real object type (`account.monthlyFee()`). → *More:* Ch 02 — 2.3–2.6 The four pillars

Q94. ⋆ Encapsulation vs abstraction: what is the difference? Encapsulation is about *protecting state*: fields are private and every change goes through a method that keeps the object valid. Abstraction is about *hiding complexity*: callers depend on a simple contract (an interface or a small public API) and never see the details. Encapsulation is done inside one class, while abstraction is a design decision about what to expose at all. → *More:* Ch 02 — 2.3 Encapsulation, 2.4 Abstraction

Q95. ⋆ Compile-time vs runtime polymorphism; overloading vs overriding. Overloading means several methods share one name but have different parameter lists. The compiler picks one from the declared argument types, so this is compile-time (static) polymorphism. **Overriding** means a subclass redefines an inherited instance method with the same signature. The JVM picks the version from the object's actual class while the program runs (dynamic dispatch), so this is runtime polymorphism. Return types may be covariant (a subtype) when overriding, and the override cannot reduce visibility or throw broader checked exceptions. → *More:* Ch 02 — 2.7 Overloading vs overriding

Q96. Can you override a static or a private method? No. A `static` method with the same signature in a subclass *hides* the parent's method, and the call is chosen by the declared type, not the object. A `private` method is invisible to subclasses, so a method with the same name there is a brand-new method. `final` methods cannot be overridden either. → *More:* Ch 02 — 2.7 Overloading vs overriding

Q97. ⋆ Interface vs abstract class: when do you use which? Use an **interface** for a capability that unrelated classes can share (`Comparable`, `PaymentMethod`). A class can implement many interfaces, and an interface has no instance state. Use an **abstract class** when closely related classes share state or a fixed algorithm skeleton (constructor, protected fields, Template Method). A class can extend only one class. In LLD rounds, default to interfaces and add an abstract base class only when real shared code appears. → *More:* Ch 02 — 2.9 Abstract classes vs interfaces

Q98. What are default, static and private methods in interfaces? A **default** method (Java 8) has a body that implementing classes inherit, so you can add methods to an interface without breaking existing implementers. A **static** interface method is a helper called as `Shape.unit()` and is not inherited. **Private** interface methods (Java 9) share code between default methods. Interfaces still cannot hold instance fields, only constants. → *More:* Ch 02 — 2.8 Interfaces

Q99. Can an abstract class have a constructor? Can it be instantiated? It can have constructors, and they run when a subclass is created (via `super(...)`) to initialise the shared fields. You cannot write `new` on it directly, although an anonymous subclass works. Since Java 25 (flexible constructor bodies, JEP 513), a constructor may run statements *before* calling `super(...)`: it can validate or prepare arguments and assign its own fields, but it cannot read fields, call instance methods or otherwise use `this`. → *More:* Ch 02 — 2.2 Classes, objects, constructors

Q100. ⋆ Composition vs inheritance: why "favor composition"? Inheritance ties a subclass to its parent's internals at compile time, so a parent change can break every child (the "fragile base class"), and Java allows only one parent. Composition means an object *holds* collaborators behind interfaces and delegates to them. You can swap them at runtime and combine behaviours without an explosion of subclasses. Use

inheritance only for a true, stable IS-A relationship where the subclass honours the parent's contract. → *More:* Ch 02 — 2.11 Composition over inheritance

Q101. IS-A vs HAS-A. IS-A is inheritance or interface implementation: a `Car` *is a* `Vehicle`, so it can be used wherever a `Vehicle` is expected. HAS-A is a field reference (association, aggregation or composition): a `Car` *has an* `Engine`. If you can't say "every X is a Y and can stand in for it" truthfully, use HAS-A. → *More:* Ch 03 — 3.8 Decision guide

Q102. ⋆ What is the diamond problem? How does Java handle multiple inheritance? The diamond problem appears when class D inherits the same method from two parents B and C that both override A's version, so it is unclear which one D gets. Java avoids it for state by allowing only one superclass. With interfaces, if two default methods clash, the class *must* override the method and may pick one with `B.super.method()`. A class method always beats an interface default. → *More:* Ch 02 — 2.8 Interfaces

Q103. ⋆ Explain the `equals()` and `hashCode()` contract. Two objects that are `equals` must return the same `hashCode`. The reverse is not required, because collisions are allowed. Hash collections find the bucket with `hashCode` and then compare with `equals`. If you override only `equals`, a `HashSet` will hold "duplicates" and `map.get` will miss keys. Build both from the same fields, never from mutable fields of a key while it sits in a map, and use records for value objects. → *More:* Ch 02 — 2.12 equals/hashCode contract

Q104. ⋆ How do you make a class immutable? What do records give you? Make the class `final`, all fields `private final`, set them once in the constructor, provide no setters, and make defensive copies of mutable inputs and outputs (`List.copyOf`). A **record** gives final fields, a canonical constructor, accessors, `equals`, `hashCode` and `toString` in one line. You still copy mutable components in a compact constructor.

```
record Order(String id, List<String> items) {
    Order { items = List.copyOf(items); }   // defensive copy, also rejects nulls
}
```

→ *More:* Ch 02 — 2.13 Immutability and records

Q105. Explain the access modifiers, and `static` vs `final`. `private` means this class only. *No modifier* (package-private) means the same package. `protected` means the package plus subclasses. `public` means everyone. `static` means "belongs to the class, not to each object" (one shared counter, a factory method). `final` means "cannot change": a final variable is assigned once, a final method cannot be overridden, a final class cannot be extended. They are independent, so `static final` is a class-level constant. → *More:* Ch 02 — 2.2 and 2.14

Q106. ⋆ Coupling vs cohesion. Cohesion measures how closely the parts of one class belong together. Aim high: one clear job. **Coupling** measures how much one class depends on others' details. Aim low: depend on small interfaces. High cohesion plus low coupling means a change stays in one place and is easy to test. These two yardsticks sit behind almost every principle and pattern. → *More:* Ch 02 — 2.16 Coupling and cohesion

Relationships and UML

Q107. ⋆ Association vs aggregation vs composition. Association means two classes know about each other (`Doctor`–`Patient`). **Aggregation** is a weak whole–part link where the part can live on its own and be shared (`Team` has `Player`s). **Composition** is a strong whole–part link where the part is created and destroyed with the whole and belongs to only one owner (`Order` owns its `OrderLine`s). In UML, aggregation is a hollow diamond and composition is a filled diamond on the whole's side. → *More:* Ch 03 — 3.2–3.4

Q108. Dependency vs association. A **dependency** is a short-lived use: the class receives another as a method parameter, local variable or return value (a dashed arrow in UML). An **association** is a lasting structural link stored in a field (a solid line). If deleting the field removes the link, it was an association. → *More:* Ch 03 — 3.5 Dependency

Q109. What is UML? Class vs sequence vs activity vs state diagrams. UML (Unified Modeling Language) is a standard notation for drawing software designs. **Class diagrams** show static structure: classes, fields, methods, relationships. **Sequence diagrams** show which objects call which, in time order, for one scenario. **Activity diagrams** are flowcharts of a process. **State machine diagrams** show an object's states and the events that move it between them, as for a vending machine or an order. → *More:* Ch 03 — 3.9–3.14

Q110. Draw a class diagram (or use-case diagram) for X. Start with a **use-case diagram** only if actors matter: actors as stick figures, use cases as ovals ("Park vehicle", "Pay fee"). For the class diagram, list the nouns from the requirements as classes, put two or three key fields and methods in each, and draw only the relationships that carry meaning, with multiplicities (`1`, `0..*`). Mark interfaces and enums. Keep it to what you will actually code. → *More:* Ch 03 — 3.10 Use-case, 3.11 Class diagrams

Q111. ⋆ How do you approach an LLD interview, and how much UML should you draw? First clarify the requirements and agree on the scope (core use cases, what is out of scope). Then find the entities, then draw a lean class diagram with the key interfaces. Walk through one main flow, and only then write code with the patterns that fit. Add concurrency and extensibility last. Draw just enough UML to agree on the shape, usually one class diagram and maybe one sequence or state sketch. Interviewers judge the reasoning and the code, not notation perfection. → *More:* Ch 01 — 1.4 The step-by-step answer framework

Q112. ⋆ LLD vs HLD: what is the difference? HLD (high-level design) chooses the services, databases, caches, queues and how they scale across machines, with boxes and arrows. LLD (low-level design) works inside one service: classes, interfaces, relationships, patterns, method signatures and thread safety, often ending in working code. HLD asks "which components and how do they scale?", LLD asks "how is this component built cleanly?" → *More:* Ch 01 — 1.1 What this round is

Q113. ⋆ Machine-coding round: how do you structure code in 90 minutes, and what do evaluators look for? They want working, runnable code that shows the core flows end to end with a demo `main`. They also look for clean separation into models, services and strategies, sensible names, input validation with clear errors, and an easy path to the extension they will ask for. Keep the code in memory (maps for storage), get a vertical slice working early, then add features one at a time. Part C below gives a minute-by-minute plan. → *More:* Ch 21 — 21.0 How the machine-coding round works

SOLID

Q114. ⋆ Explain SOLID with examples. **S**ingle Responsibility: one reason to change. **O**pen/Closed: add behaviour with new classes, not by editing tested ones. **L**iskov Substitution: subtypes must work wherever the parent is expected. **I**nterface Segregation: small role-based interfaces. **D**ependency Inversion: high-level code depends on abstractions, not on concrete details. Together they keep changes local and code testable. Apply them where change is likely, not everywhere. → *More:* Ch 04 — SOLID principles

Q115. ⋆ Single Responsibility Principle with an example. A class should have one reason to change, meaning one actor or concern drives its edits. An `Invoice` that calculates totals, prints PDFs and saves to the database changes for finance, design and DBA reasons. Split it into `Invoice` (rules), `InvoicePrinter` and `InvoiceRepository`. The test: describe the class without the word "and". → *More:* Ch 04 — 4.2 SRP

Q116. ⋆ Open/Closed Principle with an example. Code should be open for extension but closed for modification. A `FareCalculator` with a growing `switch (vehicleType)` must be edited for every new type. Instead, define a `PricingStrategy` interface and add `EvPricing` as a new class while the calculator stays untouched. Strategy, Decorator and Factory registries are the usual tools. → *More:* Ch 04 — 4.7 OCP

Q117. ⋆ Liskov Substitution: explain the rectangle/square problem. LSP says a subtype must keep the parent's promises: no stronger preconditions, no weaker postconditions, no surprise exceptions. If `Square extends Rectangle`, then `setWidth(5)` silently changes the height too, so code that expects area = width × height breaks. The fix is not to make Square a subtype of a mutable Rectangle. Use a common `Shape` interface or immutable shapes instead. → *More:* Ch 04 — 4.8 LSP

Q118. Interface Segregation with an example. Clients should not depend on methods they don't use. A fat `Machine` interface with `print`, `scan` and `fax` forces a basic printer to throw `UnsupportedOperationException`. Split it into `Printer`, `Scanner` and `Fax`, and let a multifunction device implement all three. → *More:* Ch 04 — 4.9 ISP

Q119. ⋆ Dependency Inversion vs Dependency Injection vs Inversion of Control. **DIP** is a principle: high-level policy and low-level details both depend on an abstraction that the high-level side owns (`OrderService` → `PaymentGateway` interface ← `StripeGateway`). **Dependency Injection** is a technique: dependencies are handed in from outside, usually through the constructor, instead of being created with `new`. **IoC** is the broader idea that a framework or container calls your code and wires objects ("don't call us, we'll call you"). DI is one way to do IoC, and both help you apply DIP. → *More:* Ch 04 — 4.10 DIP

Q120. Can SOLID be overdone? How do you use it in an interview? Yes. Interfaces with a single implementation that will never vary, factories for trivial objects and ten-class hierarchies for a two-case `if` are over-engineering. In an interview, apply SOLID at the points the interviewer is likely to

extend (pricing, notification channels, strategies) and say why. Leave stable parts simple. → *More:* Ch 04 — 4.15 Over-engineering

Design principles and clean code

Q121. ⋆ What are DRY, KISS and YAGNI? DRY (Don't Repeat Yourself): each piece of knowledge, such as a rule or formula, lives in one place. **KISS** (Keep It Simple): choose the simplest design that meets the need. **YAGNI** (You Aren't Gonna Need It): don't build features or hooks for imagined future needs. They balance each other, because too much DRY or flexibility hurts simplicity. → *More:* Ch 05 — §1–3

Q122. Law of Demeter and Tell, Don't Ask. The **Law of Demeter** says a method should talk only to its direct friends (its fields, its parameters, objects it creates), not to strangers through chains like `order.getCustomer().getWallet().debit(x)`. **Tell, Don't Ask** says tell an object to do the work (`order.pay(x)`) instead of pulling its data out and deciding for it. Both keep logic next to its data and cut coupling. → *More:* Ch 05 — §4–5

Q123. What does "program to an interface, not an implementation" mean? Declare fields, parameters and return types with the abstraction (`List`, `NotificationSender`), and create the concrete class in one place (a factory or the composition root). Callers then work with any implementation, including test fakes, without changes. → *More:* Ch 05 — §7

Q124. What does "encapsulate what varies" mean? Find the parts of the design that are likely to change, such as pricing rules, payment providers or eviction policies. Hide each one behind an interface so that the stable code around it doesn't change when they do. Most behavioural patterns (Strategy, State, Observer) are this idea applied to one kind of change. → *More:* Ch 05 — §8

Q125. What are the GRASP principles? GRASP (General Responsibility Assignment Software Patterns) are nine guidelines for deciding which class gets which job. **Information Expert**: give the job to the class that has the data. **Creator**: the class that holds or aggregates B creates B. **Controller**: one object receives system events. **Low Coupling** and **High Cohesion**. **Polymorphism**. **Pure Fabrication**: invent a service class when no domain class fits. **Indirection**. **Protected Variations**. → *More:* Ch 05 — §16 GRASP

Q126. DDD basics: entity vs value object, and what is an aggregate? An **entity** has an identity that lasts while its attributes change (a `Customer` with an ID). A **value object** is defined only by its values, is immutable and is compared by value (`Money`, `Address`). An **aggregate** is a cluster of objects changed as one unit through a single **root** (`Order` with its lines), which protects the cluster's rules. Outsiders hold references only to the root. → *More:* Ch 05 — §17 DDD essentials

Q127. What does "fail fast" mean? Detect invalid input or state as early as possible and stop with a clear exception, instead of carrying bad data forward until it fails somewhere confusing. Examples are `Objects.requireNonNull` in constructors, validating in a record's compact constructor and rejecting an invalid state transition right away. → *More:* Ch 05 — §13 Fail Fast

Q128. ⋆ What are anti-patterns? What is a God class? What is over-engineering? An **anti-pattern** is a common "solution" that causes more problems than it solves. A **God class** knows and does too much (a `ParkingLotManager` with 40 methods), so every change touches it and it can't be tested in isolation. **Over-engineering** adds layers, patterns and configurability nobody asked for. Other anti-patterns are Singleton abuse, anemic models, magic numbers and copy-paste code. → *More:* Ch 08 — 8.6 Anti-patterns

Q129. ⋆ What are code smells? Name a few and their refactorings. A **code smell** is a surface sign of a deeper design problem. **Long Method** → Extract Method. **Large Class** → Extract Class. **Long Parameter List** → Introduce Parameter Object. **Switch on type** → Replace Conditional with Polymorphism. **Feature Envy** (a method that uses another class's data more than its own) → Move Method. **Duplicate Code** → Extract and reuse. **Refactoring** means changing structure in small, behaviour-preserving steps, with tests as the safety net. → *More:* Ch 07 — 7.2 Code smells, 7.4 Lookup table

Q130. What makes code "clean" (naming, functions, error handling)? Names reveal intent (`activeBookings`, not `list2`). Functions are small, do one thing at one level of abstraction and take few arguments. Errors are reported with exceptions that carry context, not with error codes or `null`. Comments explain *why*, not *what*. Clean code reads like the problem description. → *More:* Ch 06 — Clean code

Q131. Checked vs unchecked exceptions: which do you use? Checked exceptions extend `Exception` (but not `RuntimeException`). The compiler forces callers to catch or declare them, which suits recoverable conditions from outside the program, such as `IOException`. **Unchecked** exceptions extend `RuntimeException` and usually signal programming errors or broken rules (`IllegalArgumentException`, a custom `SeatAlreadyBookedException`).

Modern Java design prefers unchecked domain exceptions with clear names, plus checked ones only where the caller can really recover. → *More:* Ch 06 — §6 Error Handling

Q132. Unit testing: mocks vs stubs, and how do you make a design testable? A **stub** returns canned answers so the code under test can run. A **mock** also *verifies* interactions (for example, "`send` was called once"). A **fake** is a working lightweight version, such as an in-memory repository. Designs are testable when dependencies are injected through interfaces, time and randomness are injected (`Clock`), and logic sits apart from I/O. → *More:* Ch 06 — §17 Testing strategies

Creational patterns

Q133. ⋆ What is a design pattern? What are the three GoF categories? A design pattern is a named, reusable solution to a recurring design problem. It is a template you adapt, not a library you import. The "Gang of Four" (GoF) book lists 23 patterns in three groups. **Creational** patterns deal with how objects are made (Singleton, Factory Method, Abstract Factory, Builder, Prototype). **Structural** patterns deal with how objects are composed (Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy). **Behavioral** patterns deal with how objects communicate and share work (Strategy, Observer, State, Command and others). → *More:* Ch 08 — 8.1–8.3

Q134. ⋆ Singleton: how do you make it thread-safe? Use an **enum** (simplest, and safe against reflection and serialization), or the **holder idiom**: a static nested class that the JVM loads, thread-safely, on first use. **Double-checked locking** also works but needs a `volatile` field. Without `volatile`, another thread can see a half-constructed object.

```
final class Config {
    private Config() {}
    private static class Holder { static final Config INSTANCE = new Config(); }
    static Config get() { return Holder.INSTANCE; }
}
```

→ *More:* Ch 08 — 8.8 Singleton

Q135. How can a Singleton be broken? Reflection can call the private constructor. Deserialization creates a new instance unless you add `readResolve`. Cloning breaks it if the class is `Cloneable`. Each class loader can also hold its own copy. An enum singleton blocks the first three. → *More:* Ch 08 — 8.8 Singleton

Q136. ⋆ Why is Singleton often called an anti-pattern? It is global mutable state in disguise. It hides dependencies (callers reach out with `getInstance()` instead of receiving them), it makes unit tests hard because you can't swap or reset it, and it creates concurrency hot spots. Prefer creating one instance at the composition root (or a DI container's singleton scope) and injecting it. → *More:* Ch 08 — 8.8 Singleton

Q137. ⋆ Simple Factory vs Factory Method vs Abstract Factory. A **Simple Factory** is one method with a `switch` that returns the right subtype (`VehicleFactory.create(type)`). It is an idiom, not a GoF pattern. **Factory Method** lets subclasses decide which product to create by overriding a creation method. **Abstract Factory** creates *families* of related products that must match (a `WindowsUiFactory` makes `WinButton` + `WinCheckbox`). In interviews, a simple factory or a `Map<Type, Supplier<T>>` registry is usually enough. → *More:* Ch 08 — 8.9–8.11, 8.15

Q138. ⋆ When would you use Builder? Use it when an object has many parameters, several of them optional (the "telescoping constructor" problem), or when you want to validate everything before building an immutable object. The builder collects values with readable chained calls and `build()` checks them once. Records with a few fields don't need one.

```
var pizza = Pizza.builder().size(LARGE).cheese(true).topping("olive").build();
```

→ *More:* Ch 08 — 8.12 Builder

Q139. Prototype pattern; shallow vs deep copy. Prototype creates new objects by copying a configured existing one, which helps when construction is expensive or the configuration is complex. A **shallow copy** copies fields as they are, so the copy shares nested mutable objects with the original. A **deep copy** also copies those nested objects, so changes don't leak between them. In Java, prefer copy constructors or factory methods over `clone()`. → *More:* Ch 08 — 8.13 Prototype

Q140. What is an object pool, and when is it worth it? A pool keeps a fixed set of expensive, reusable objects (database connections, threads). It lends one out and takes it back, so you avoid repeated

creation and cap the total. It needs a thread-safe borrow/return (often a `BlockingQueue` or a `Semaphore`) and must reset each object's state on return. For cheap objects it only adds bugs. → *More:* Ch 08 — 8.14 Object Pool

Structural patterns

Q141. ⋆ Adapter: when would you use it? Adapter vs Facade vs Bridge. An **Adapter** converts one interface into another that the client expects. A typical use is wrapping a third-party `LegacyPaymentApi` so it implements your `PaymentGateway`. A **Facade** gives a *simpler* front to a whole subsystem (`HomeTheater.watchMovie()`) without changing the interfaces. A **Bridge** splits an abstraction from its implementation up front, so the two can vary independently (`Shape` × `Renderer`). Adapter fixes a mismatch after the fact, Facade simplifies, and Bridge is planned in from the start. → *More:* Ch 09 — 9.1, 9.2, 9.5, 9.8

Q142. ⋆ Decorator vs Proxy (and vs inheritance). Both wrap an object behind the same interface. A **Decorator** *adds* behaviour, and decorators can be stacked at runtime (`new Logging(new Compressing(stream))`), which avoids a subclass for every combination. A **Proxy** *controls access* to the same behaviour: lazy loading, caching, security checks, remote calls. Usually the proxy, not the client, decides whether to create the real object. → *More:* Ch 09 — 9.4 Decorator, 9.7 Proxy

Q143. ⋆ Composite pattern (file system / org chart). Composite treats single objects and groups of objects through one interface, so the client doesn't care which it has. A `File` and a `Directory` both implement `FileSystemNode.size()`, and the directory sums its children recursively. Use it for any tree: menus, org charts, expression trees. → *More:* Ch 09 — 9.3 Composite

Q144. What is the Flyweight pattern? Flyweight saves memory when there are huge numbers of similar objects. It shares the *intrinsic* (unchanging, shareable) state and passes the *extrinsic* (per-use) state in from outside. Examples: one shared `Glyph` per character in a text editor, with the position stored separately, or tree types shared across a million trees in a game. `Integer.valueOf` caching small values is a JDK example. → *More:* Ch 09 — 9.6 Flyweight

Q145. How does Java support Proxy at runtime (dynamic proxies, Spring AOP)? `java.lang.reflect.Proxy` creates, at runtime, a class that implements given interfaces and sends every call to an `InvocationHandler`. That is how logging, transactions and retries get added around calls. Spring AOP builds either a JDK proxy (interfaces only) or a generated subclass (CGLIB). Plain Spring uses a JDK proxy when the bean has an interface, while Spring Boot defaults to CGLIB subclasses for everything. Either way the advice lives in the proxy, so a `@Transactional` method called from inside the same class (through `this`) bypasses it. → *More:* Ch 09 — 9.7 Proxy

Behavioral patterns

Q146. ⋆ Strategy pattern: where did you use it? Strategy puts each interchangeable algorithm behind an interface and lets the context pick one at runtime. Examples are a parking fee per vehicle type, Splitwise's equal/exact/percent splits, cache eviction (LRU/LFU) and payment methods. It removes `switch` statements and follows Open/Closed. In modern Java a strategy is often just a lambda.

```
interface SplitStrategy { Map<User, Money> split(Money total, List<User> users); }
```

→ *More:* Ch 11 — 11.3 Strategy

Q147. ⋆ Strategy vs State. They have the same class shape but a different intent. With **Strategy**, the *client* picks the algorithm, and strategies don't know about each other. With **State**, the *object itself* changes behaviour as its internal state changes, and the states usually trigger the transitions (`IdleState` → `HasCoinState`). Ask "who chooses?": the caller means Strategy, the object's own lifecycle means State. → *More:* Ch 11 — 11.8 Choose the right behavioral pattern

Q148. Strategy vs Template Method. **Template Method** fixes the steps of an algorithm in a base class and lets subclasses fill in some of them (inheritance, chosen at compile time). **Strategy** swaps the whole algorithm through composition, at runtime. Prefer Strategy when variants change independently or need to be combined. Template Method suits a stable skeleton with small hooks. → *More:* Ch 11 — 11.4 Template Method

Q149. ⋆ Observer pattern; Observer vs Pub-Sub. In **Observer**, a subject keeps a list of observers and calls them directly when its state changes (a stock price notifying displays). The subject knows its observers' interface, and delivery is usually synchronous and in-process. **Pub-Sub** puts a broker or event bus between them: publishers send to a topic without knowing the subscribers, and delivery can be asynchronous and cross-process. Pub-Sub decouples more, at the cost of extra infrastructure. → *More:* Ch 11 — 11.1 Observer

Q150. ⋆ Command pattern: how does it give you undo/redo? Command turns a request into an object with `execute()` and `undo()`, so it can be queued, logged, retried or reversed. For undo/redo, push each executed command on an undo stack. Undo pops it, calls `undo()` and pushes it on the redo stack. Any new command clears the redo stack. Text editors, remote controls and job queues use it. → *More:* Ch 10 — 10.2 Command

Q151. Chain of Responsibility (middleware, approval flows). A request passes along a chain of handlers. Each one either handles it or passes it to the next, so the sender doesn't know who will deal with it. Examples are expense approval (manager → director → CFO by amount), ATM cash dispensing by note value, and servlet filters or HTTP middleware (auth → rate limit → logging). Handlers can be added or reordered without changing the client. → *More:* Ch 10 — 10.1 Chain of Responsibility

Q152. Iterator, Mediator, Memento, Interpreter, Visitor: when would you use each? Iterator: walk a collection without exposing how it is stored. **Mediator**: objects talk through a hub instead of to each other (an air-traffic control tower, a chat room). **Memento**: save and restore an object's state without breaking encapsulation (snapshots for undo). **Interpreter**: evaluate sentences of a small grammar (rule expressions). **Visitor**: add new operations to a stable class hierarchy without editing it (export or tax calculation over node types). In modern Java, sealed types with pattern-matching `switch` often replace Visitor. → *More:* Ch 10 — 10.3–10.6, Ch 11 — 11.5 Visitor

Q153. ⋆ State pattern (vending machine / ATM / elevator). Each state becomes a class that implements the same interface (`insertCoin`, `selectItem`, `dispense`). The machine delegates to its current state object, and states switch the machine to the next state. This replaces large `if (state == ...)` blocks, makes illegal actions explicit per state, and lets you add a state without touching the others. A small enum-based state machine is fine when the states have little behaviour. → *More:* Ch 11 — 11.2 State

Q154. What are Null Object, Repository, Specification and MVC? Null Object: a do-nothing implementation (`NoDiscount`) used instead of `null`, which removes null checks. **Repository**: a collection-like interface for storing and finding aggregates (`BookingRepository.findById`) that hides the storage. **Specification**: a business rule as a combinable object (`isAvailable.and(inCity)`). **MVC** (Model–View–Controller): the model holds data and rules, the view displays it, and the controller turns input into model updates. → *More:* Ch 11 — 11.7 Beyond GoF

Q155. ⋆ Which patterns would you use for a parking lot, an elevator or a vending machine? Parking lot: Strategy for pricing and spot allocation, Factory for vehicles or spots, Singleton or injected for the lot, Observer for display boards. **Elevator**: State for each car (moving/idle/doors open), Strategy for the dispatch algorithm, Command for requests. **Vending machine**: State for idle/has-money/dispensing, and Chain of Responsibility for giving change. Name a pattern only when you can say what change it absorbs. → *More:* Ch 22 — 22.9 Case-study map

Q156. How have lambdas and modern Java changed classic patterns? Patterns with a single method, such as Strategy, Command, Observer callbacks and Template Method hooks, are often just functional interfaces with lambdas (`Comparator`, `Runnable`, `Function`). Records cover simple value objects, and sealed interfaces with pattern-matching `switch` give exhaustive case handling in place of many Visitor uses. The *ideas* stay the same, with less boilerplate. → *More:* Ch 11 — 11.6 Quick-reference summary

Concurrency

Q157. ⋆ Process vs thread. A **process** is a running program with its own memory space. Processes are isolated and costly to create, and they communicate through IPC (inter-process communication: pipes, sockets). A **thread** is a path of execution *inside* a process. Threads share the heap with their siblings but each has its own stack, so they are cheap to switch and communicate through shared memory, which is why they need synchronisation. → *More:* Ch 12 — 12.2

Q158. ⋆ Concurrency vs parallelism. Concurrency means dealing with many tasks over the same period by interleaving them. It is about structure, and it works on one core. **Parallelism** means literally running tasks at the same instant on several cores. It is about speed. A web server handling 10,000 waiting connections is concurrent, while a multi-core merge sort is parallel. → *More:* Ch 12 — 12.1

Q159. What are the states in a thread's lifecycle? `Thread.State` has six values. **NEW** means created but not started. **RUNNABLE** means running or ready to run. **BLOCKED** means waiting to enter a `synchronized` monitor. **WAITING** means waiting without a time limit (`wait()`, `join()`, `LockSupport.park`). **TIMED_WAITING** means waiting with a timeout (`sleep(ms)`, `wait(ms)`). **TERMINATED** means finished. → *More:* Ch 12 — 12.3

Q160. Why call `start()` and not `run()`? `start()` asks the JVM to create a new thread, which then calls `run()`. Calling `run()` directly just executes the method on the *current* thread, so nothing runs concurrently. Calling `start()` twice on the same thread throws `IllegalThreadStateException`. → *More:* Ch 12 — 12.4

Q161. ⋆ Runnable vs Callable; ExecutorService; CompletableFuture. A `Runnable` returns nothing and can't throw checked exceptions. A `Callable<V>` returns a value and may throw. An `ExecutorService` runs tasks on a managed pool and hands back a `Future` for the result. `CompletableFuture` adds non-blocking chaining and combining (`thenApply`, `thenCombine`, `allOf`) and error handling. Always shut executors down. Since Java 19, `ExecutorService` is `AutoCloseable`, so try-with-resources works: `close()` shuts the executor down and waits for submitted tasks to finish. → *More:* Ch 12 — 12.4

Q162. ⋆ What are virtual threads, and when should you not use them? Virtual threads (final in Java 21) are cheap JVM-managed threads. When one blocks on I/O it unmounts from its OS carrier thread, so you can run one per request or task, even millions. They help blocking, I/O-heavy work, not CPU-bound work. Don't pool them, and limit access to scarce resources with a `Semaphore`. Since JDK 24 (JEP 491), blocking inside `synchronized` or in `Object.wait()` no longer pins the carrier. Pinning remains in rare cases: blocking while a native frame is on the stack, and blocking in class loading or a class initializer. → *More:* Ch 12 — 12.4

Q163. ⋆ What is a race condition? What is a critical section? A **race condition** happens when the result depends on the timing of threads touching shared mutable data. A classic case is `count++`, which is read–modify–write: two threads read 5 and both write 6. Another is "check-then-act", where two users both see seat A1 free. A **critical section** is the code that touches the shared state and must run by one thread at a time (or be made atomic). → *More:* Ch 12 — 12.5

Q164. ⋆ What does `volatile` do? What is happens-before (the Java Memory Model)? The Java Memory Model (JMM) defines when one thread's writes become visible to another. **Happens-before** is its guarantee: if A happens-before B, then B sees A's effects. Releasing a lock happens-before the next acquire of it, `Thread.start` and `join` create the edge too, and so do volatile writes and reads. `volatile` gives visibility and ordering for one variable but **not** atomicity, so `volatile int count; count++` is still a race. It is good for flags like `running = false`. → *More:* Ch 12 — 12.6

Q165. ⋆ `synchronized` vs `ReentrantLock`. Both give mutual exclusion and are **reentrant**, meaning the thread holding the lock can enter it again. `synchronized` is simpler and releases automatically. `ReentrantLock` adds `tryLock` with a timeout, interruptible locking, optional fairness and several `Condition`s, but you must `unlock()` in `finally`. Start with `synchronized` and switch when you need those features.

```
lock.lock();
try { /* critical section */ } finally { lock.unlock(); }
```

→ *More:* Ch 12 — 12.7, 12.8

Q166. ReadWriteLock and StampedLock: when do you use them? A `ReentrantReadWriteLock` lets many readers in at once but a writer gets exclusive access, which helps when reads far outnumber writes (a config cache). `StampedLock` adds **optimistic reads**: read without locking, then `validate(stamp)`, and retry under a real read lock if a write happened. It is faster for read-heavy data but not reentrant and harder to use correctly. → *More:* Ch 12 — 12.10

Q167. ⋆ Semaphore vs mutex. A **mutex** (lock) lets exactly one thread in and has an owner: only the locker should unlock. A **semaphore** holds N permits, so up to N threads proceed at once (limiting a pool to 10 connections). It has no owner, so one thread can release a permit another acquired, which is useful for signalling. A binary semaphore looks like a mutex but lacks ownership and reentrancy. → *More:* Ch 12 — 12.12 Semaphores

Q168. `wait()`/`notify()` vs `Condition`; `notify()` vs `notifyAll()`. `wait()` releases the object's monitor and sleeps until notified. It must be called while holding the monitor (`synchronized`), and always inside a `while` loop that re-checks the condition, because of spurious wakeups. `notify()` wakes one arbitrary waiter, `notifyAll()` wakes all. `notifyAll` is safer when waiters wait for different conditions. A `Condition` from a `Lock` does the same job, but one lock can have several conditions (`notFull`, `notEmpty`), so you wake only the right group. → *More:* Ch 12 — 12.13 Condition variables

Q169. ⋆ **`wait()` vs `sleep()`.** `Object.wait()` **releases** the monitor, must be called inside `synchronized` (otherwise it throws `IllegalMonitorStateException`), and returns on `notify`/`notifyAll`, a timeout, an interrupt or a spurious wakeup. Use it to wait for a condition. `Thread.sleep()` is a static method that pauses the current thread for a time and **keeps** any locks it holds. Use it for delays, not coordination. Both throw `InterruptedException`. → *More:* Ch 12 — 12.3, 12.13

Q170. ⋆ **`CountDownLatch` vs `CyclicBarrier`.** A `CountDownLatch` lets threads wait until a count reaches zero. It is one-shot, and the threads counting down are usually different from the waiters ("start when all 5 services are ready"). A `CyclicBarrier` makes a fixed group of threads wait for *each other* at a point and can be reused round after round, with an optional barrier action ("each simulation step"). `Phaser` is the flexible, dynamic version. → *More:* Ch 12 — 12.14

Q171. ⋆ **What is CAS? How do atomics like `AtomicInteger` work?** CAS (compare-and-swap) is a CPU instruction: "set the value to new **only if** it still equals expected", which is atomic and lock-free. Atomic updates are a CAS loop: read, compute, CAS, and retry if another thread changed the value (`updateAndGet` works exactly like this, and on x86 the JIT turns `incrementAndGet` into a single atomic add instruction). That beats locks under low contention, and `LongAdder` spreads hot counters across cells under high contention. Watch for the **ABA problem** (a value changes A→B→A unnoticed). `AtomicStampedReference` solves it with a version stamp. → *More:* Ch 12 — 12.15

Q172. ⋆ **How does `ConcurrentHashMap` work? How is it different from `Hashtable` and `synchronizedMap`?** `Hashtable` and `Collections.synchronizedMap` lock the whole map for every call. Since Java 8, `ConcurrentHashMap` uses CAS to place the first node in an empty bucket and otherwise locks only that bucket's head node. Reads take no lock at all, and writes to different buckets run in parallel. It forbids `null` keys and values. Its iterators are weakly consistent and never throw `ConcurrentModificationException`. Use `compute`/`merge`/`putIfAbsent` for atomic read-modify-write. → *More:* Ch 12 — 12.16, Ch 13 — 13.7

Q173. ⋆ **What is a deadlock? What are its conditions, and how do you prevent and detect it?** A deadlock happens when threads wait on each other forever, for example T1 holds lock A and wants B while T2 holds B and wants A. The four Coffman conditions must all hold: mutual exclusion, hold-and-wait, no preemption and circular wait. To prevent it, break one condition. The most common way is to always acquire locks in a global order (for example by account ID). Others are `tryLock` with a timeout and backoff, or holding fewer locks. To detect it, take a thread dump (`jstack`), use `ThreadMXBean.findDeadlockedThreads()`, or use a wait-for graph. → *More:* Ch 12 — 12.17 Deadlock

Q174. Livelock vs starvation. In a **livelock**, threads are active but make no progress because they keep reacting to each other, like two people stepping aside the same way in a corridor. The fix is random backoff. In **starvation**, a thread never gets the CPU or the lock because others keep winning. The fix is fair locks (`new ReentrantLock(true)`), bounded queues and avoiding priority tricks. → *More:* Ch 12 — 12.18, 12.19

Q175. ⋆ **Explain the producer-consumer problem and how to solve it.** Producers put items into a bounded buffer and consumers take them out. Producers must wait when the buffer is full, and consumers must wait when it is empty, without busy-spinning or racing. The easy solution is a `BlockingQueue` (`ArrayBlockingQueue`), whose `put` and `take` block. Hand-rolled versions use a lock with `notFull`/`notEmpty` conditions, or `wait`/`notifyAll` in `while` loops. → *More:* Ch 12 — 12.22, Ch 13 — 13.8

Q176. The reader-writer problem. Many readers may read shared data at once, but a writer needs exclusive access. A naive solution can starve writers while readers keep arriving. In Java, use `ReentrantReadWriteLock` (with the fair option if writer starvation matters), `StampedLock`, or copy-on-write structures when writes are rare. → *More:* Ch 12 — 12.23

Q177. ⋆ **How do you prevent double booking of a seat or parking spot?** The danger is check-then-act: two requests both see the seat free and both book it. In memory, make the check and the claim one atomic step, for example `seatHolds.putIfAbsent(seatId, userId)` on a `ConcurrentHashMap`, or lock per show or seat (fine-grained), acquiring several seats in sorted order. With a database, use **optimistic locking** (a version column, and the update fails if the version changed, so you retry or report "taken"), **pessimistic locking** (`SELECT ... FOR UPDATE`) or a unique constraint. Add temporary holds with a timeout during payment. → *More:* Ch 12 — 12.25

Q178. Optimistic vs pessimistic locking. Pessimistic locking takes the lock *before* working, assuming conflicts are likely. It is safe but blocks others. **Optimistic** locking works without a lock and checks at commit that nobody changed the data (a version number or CAS), retrying if they did. It is fast when conflicts are rare. Use pessimistic locking for hot, contended rows (the last seats of a flash sale) and optimistic locking for most other updates. → *More:* Ch 12 — 12.25

Q179. Coarse-grained vs fine-grained locking. **Coarse-grained** means one lock guards a lot (the whole parking lot). It is simple and deadlock-free but serialises everything. **Fine-grained** means many locks guard small parts (per floor, per spot, per bucket). Throughput goes up, but so do complexity and deadlock risk. Start coarse, measure, and split only the hot path. Lock striping is the middle ground. → *More:* Ch 12 — 12.11

Q180. Name the ways to make a class thread-safe. In rough order of preference: make it **immutable** (records, final fields). **Confine** the state to one thread (local variables, `ThreadLocal`, or a single-threaded owner reached via a queue). Use **thread-safe building blocks** (`ConcurrentHashMap`, atomics, `BlockingQueue`). Last, **lock** carefully around every invariant that spans several fields. Document which one you chose. → *More:* Ch 12 — 12.24 Immutability & thread confinement

Java specifics for LLD rounds

Q181. ⋆ Why is `String` immutable in Java? Strings are shared everywhere, so immutability makes that safe. Literals live in the **string pool** and are reused, which would be dangerous if one holder could change them. Immutable strings are automatically **thread-safe**. Their `hashCode` can be **cached**, which makes them fast, stable `HashMap` keys. They are also **secure** for values like file paths, URLs and class names, which cannot be changed after they are checked. "Modifying" methods such as `concat` and `replace` return new strings. → *More:* Ch 02 — 2.13 Immutability

Q182. ⋆ Is Java pass-by-value or pass-by-reference? Always **pass-by-value**. For objects, the value copied is the *reference*. So a method can change the object's fields through its copy (`list.add(x)` is visible to the caller), but reassigning the parameter (`list = new ArrayList<>()`) does not affect the caller's variable. That is why a `swap(a, b)` method can't swap the caller's objects.

```
void reset(StringBuilder sb) { sb.setLength(0); sb = new StringBuilder("x"); }
// the caller's sb is now empty, not "x"
```

→ *More:* Ch 02 — 2.2 Classes and objects

Q183. ⋆ `==` vs `equals()` for strings (and the string pool). `==` compares references (is this the same object?), while `equals` compares contents. `"ab" == "ab"` is true because string literals (and compile-time constants such as `"a" + "b"`) are interned into the pool. `new String("ab") == "ab"` is false, and so is a string built at runtime unless you call `intern()`. Always compare strings with `equals` (or `Objects.equals` for null-safety). → *More:* Ch 02 — 2.12 Identity vs equality

Q184. `String` vs `StringBuilder` vs `StringBuffer`. `String` is immutable, so concatenating in a loop creates many temporary objects. `StringBuilder` is mutable and fast but not thread-safe, which makes it the default for building text. `StringBuffer` is the old synchronised version and is rarely needed, since builders are almost always used inside one thread. → *More:* Ch 02 — 2.13 Immutability

Q185. `final` vs `finally` vs `finalize`. `final` is a modifier for a constant variable, a non-overridable method or a non-extendable class. `finally` is the block after `try` that always runs, used for cleanup, though try-with-resources is better. `finalize()` was a GC cleanup hook. It has been deprecated for removal since JDK 18 (JEP 421) because it is unpredictable and slow. Use `AutoCloseable` or `java.lang.ref.Cleaner` instead. → *More:* Ch 02 — 2.2 static and final

Q186. ⋆ How does `HashMap` work internally? `HashMap` keeps an array of buckets. It spreads the key's `hashCode` bits and masks the result to a bucket index, then compares with `equals` inside the bucket. Collisions form a linked list. When a bucket already holding 8 entries (`TREEIFY_THRESHOLD`) gets another, the list becomes a red-black tree, but only if the table has at least 64 slots (`MIN_TREEIFY_CAPACITY`); a smaller table is resized instead. A tree shrinks back to a list at 6 entries or fewer. When the size exceeds capacity × load factor (0.75 by default), the table doubles and entries are redistributed. It allows one `null` key and is not thread-safe. → *More:* Ch 13 — 13.7 Concurrent HashMap

Q187. `Comparable` vs `Comparator`. `Comparable` gives a class its single natural order through `compareTo`, for example `Money` by amount. A `Comparator` is an external, swappable ordering, which makes it a Strategy (`Comparator.comparing(Ride::fare).thenComparing(Ride::eta)`). Keep `compareTo` consistent with `equals`, or sorted sets and maps will behave oddly. → *More:* Ch 11 — 11.3 Strategy

Q188. Fail-fast vs fail-safe iterators. **Fail-fast** iterators (`ArrayList`, `HashMap`) throw `ConcurrentModificationException` if the collection is structurally changed during iteration, other than through the iterator's own `remove`. **Fail-safe** (weakly consistent or snapshot) iterators (`ConcurrentHashMap`,

`CopyOnWriteArrayList`) never throw, but may not show the latest changes. To remove while looping, use `iterator.remove()` or `removeIf`. → *More:* Ch 12 — 12.16 Concurrent collections

Q189. ⋆ Sealed classes, records and pattern matching: how do they help in LLD? A `sealed` interface lists its only allowed implementations (Java 17). Combined with records and pattern-matching `switch` (Java 21), the compiler checks that every case is handled, with no `default` needed. This is a clean way to model closed sets like payment results or vehicle types.

```
sealed interface Payment permits Card, Upi {}
record Card(String last4) implements Payment {}
record Upi(String vpa) implements Payment {}
String describe(Payment p) { return switch (p) {
    case Card c -> "Card " + c.last4();
    case Upi u  -> "UPI " + u.vpa(); }; }
```

→ *More:* Ch 02 — 2.13 Immutability and records

Q190. Generics: what are wildcards and PECS? `List<? extends Shape>` accepts lists of any Shape subtype and lets you *read* Shapes, but you can't add to it. `List<? super Circle>` lets you *add* Circles. **PECS** ("Producer Extends, Consumer Super") is the rule: use `extends` for sources you read from and `super` for sinks you write to, as in `Collections.copy(List<? super T> dst, List<? extends T> src)`. Generics are erased at runtime, so `new T()` is impossible. → *More:* Ch 02 — 2.15 Generics basics

Q191. When should you use `Optional`, and how do you avoid `null` in a design? Return `Optional<T>` from lookups that may find nothing (`findSpot()`), so callers must handle absence. Don't use it for fields, parameters or collections; return an empty collection instead. Elsewhere, avoid `null` with validated constructors, the Null Object pattern and `Objects.requireNonNull`. → *More:* Ch 11 — 11.7.1 Null Object

Part C — How to run a 60–90 minute machine-coding round

A machine-coding round asks you to build a small working system, such as Splitwise, a parking lot or a ride-sharing service, in one sitting, usually in an IDE. It is judged on whether the code runs, how it is structured, and how easily it absorbs a change. The plan below assumes 90 minutes. For a 60-minute round, roughly halve each step.

Minutes 0–10: clarify and cut scope. Read the problem twice. Write down the must-have features (usually 3–5 commands), the input/output format, and what you will leave out (persistence, auth, UI). Ask about the tricky rules, such as rounding, ties, invalid input and concurrent users. Confirm that in-memory storage and a console demo are fine.

Minutes 10–20: model. List the entities (nouns) and the services (verbs). Decide the few interfaces where change is likely, such as a pricing, split or allocation strategy. Sketch the packages: `model`, `service`, `strategy`, `repository` (in-memory maps), `exception`, and `Main` for the demo. Don't draw full UML. A short list of classes is enough.

Minutes 20–60: build a vertical slice first. Get the main flow working end to end (create → act → show) before adding anything else. Use records and enums for the models, constructor injection for the services, and clear custom exceptions for invalid operations. Commit or save each working state. Write code that compiles at every step, and don't leave a half-built feature that breaks the demo.

Minutes 60–75: complete the features and handle edge cases. Add the remaining commands one by one. Validate input, handle "not found" and duplicates, and make any shared state thread-safe if concurrency was asked (`ConcurrentHashMap`, per-entity locks).

Minutes 75–85: demo and test. Run `Main` on the sample input plus two or three edge cases. If time allows, add a few unit tests for the core rule, such as the split math or the fare calculation.

Minutes 85–90: walk through and extend. Explain the structure in one minute, then show how the likely extension fits: "a new split type is one new class". Be honest about anything you skipped.

Common ways to lose marks: one giant class, no working demo, over-engineering before the basics run, ignoring input validation, and staying silent. Talk while you code.

→ *More:* Ch 01 — The LLD interview approach, Ch 21 — 21.0 How the machine-coding round works

Sources of the question lists

Question wording and frequency only; every answer above is written for this book.

- LeetCode Discuss interview-experience posts (Flipkart SDE2, Swiggy SDE1/SDE2, Atlassian P4, PhonePe, Meta E5) and the "How do you prepare for machine coding rounds" thread
- prasadgujar/low-level-design-primer — `questions.md` (LeetCode-Discuss-sourced list)
- ashishps1/awesome-low-level-design — "LLD Interview Problems" and concurrency problems
- kumaransg/LLD — machine-coding questions tagged Flipkart / Udaan / Navi / ClearTrip
- donnemartin/system-design-primer — "Object-oriented design interview questions"
- workat.tech machine-coding practice problems
- Course tables of contents (titles only): Grokking the Object-Oriented Design Interview